

50 1190 0101

Утвержден

РУСБ.10015-01-УД

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ОПЕРАЦИОННАЯ СИСТЕМА СПЕЦИАЛЬНОГО НАЗНАЧЕНИЯ
«ASTRA LINUX SPECIAL EDITION»

Руководство по КСЗ. Часть 1

РУСБ.10015-01 97 01-1

Листов 491

2026

Литера О₁

АННОТАЦИЯ

Настоящий документ является первой частью руководства по комплексу средств защиты информации (КСЗ) программного изделия РУСБ.10015-01 «Операционная система специального назначения «Astra Linux Special Edition» (далее по тексту — ОС).

Документ предназначен для администраторов безопасности.

Руководство по КСЗ состоит из трех частей:

- РУСБ.10015-01 97 01-1 «Операционная система специального назначения «Astra Linux Special Edition». Руководство по КСЗ. Часть 1»;
- РУСБ.10015-01 97 01-2 «Операционная система специального назначения «Astra Linux Special Edition». Руководство по КСЗ. Часть 2»;
- РУСБ.10015-01 97 01-3 «Операционная система специального назначения «Astra Linux Special Edition». Руководство по КСЗ. Часть 3. Защищенная СУБД».

В первой части руководства приведены общие сведения о КСЗ, рассмотрены идентификация и аутентификация, дискреционное и мандатное управление доступом, защита памяти, изоляция процессов, защита среды виртуализации, маркировка документов, контроль подключения съемных машинных носителей информации, сопоставление пользователя с устройством, регистрация событий безопасности, надежное функционирование, фильтрация сетевого потока, контроль целостности, генерация КСЗ, режим ограничения действий пользователя Киоск-2, а также порядок запуска и применения ОС.

Во второй части руководства приведено описание тестов КСЗ.

В третьей части руководства приведено описание защищенной СУБД и порядок работы с ней.

Дополнительная информация о настройке компонентов и управлении программными пакетами, а также варианты реализации отдельных решений с использованием ОС приведены на официальном сайте wiki.astralinux.ru.

СОДЕРЖАНИЕ

1. Общие сведения	15
1.1. Приемка ОС	15
1.2. Состав КСЗ	16
1.3. Контролируемые функции	16
1.4. Безопасная установка ОС	19
1.5. Средства организации ЕПП	19
2. Идентификация и аутентификация	21
3. Дискреционное управление доступом	23
3.1. Общие сведения	23
3.1.1. Права доступа	23
3.1.2. Списки контроля доступа	26
3.1.3. Идентификаторы принадлежности процессов	30
3.2. Linux-привилегии	31
3.2.1. Linux-привилегии пользователей и процессов	31
3.2.2. Инструменты управления Linux-привилегиями файлов	33
3.2.2.1. setcap	33
3.2.2.2. getcap	34
3.2.3. Наследование и формирование наборов Linux-привилегий	35
3.2.4. Изменение наборов Linux-привилегий ходе выполнения процесса	39
3.3. Средства управления дискреционными ПРД	40
3.3.1. chown	40
3.3.2. chgrp	42
3.3.3. chmod	43
3.3.4. umask	47
3.3.5. getfacl	47
3.3.6. setfacl	49
3.4. Особенности управления дискреционными ПРД в NFSv4.x	52
3.4.1. nfs4_getfacl	53
3.4.2. nfs4_setfacl	55
4. Мандатное управление доступом и мандатный контроль целостности	58
4.1. Общие сведения	58
4.2. Мандатное управление доступом	58
4.2.1. Уровень конфиденциальности	58

4.2.2. Категория конфиденциальности	59
4.2.3. Дополнительные атрибуты сущностей для мандатного управления доступом . . .	59
4.3. Мандатный контроль целостности	60
4.3.1. Метка целостности	60
4.3.2. Дополнительные атрибуты сущностей для мандатного контроля целостности .	63
4.4. Метка безопасности и мандатный контекст безопасности	64
4.5. Расширенный режим мандатного контроля целостности	65
4.6. Применение правил мандатного управления доступом и мандатного контроля целостности	67
4.7. PARSEC-привилегии	70
4.8. Включение и выключение мандатного управления доступом	73
4.8.1. Включение мандатного управления доступом	73
4.8.2. Выключение мандатного управления доступом	74
4.9. Включение и выключение мандатного контроля целостности	75
4.10. Мандатный контроль целостности на файловой системе	75
4.11. Администрирование ОС при включенном МКЦ	78
4.12. Разделяемые каталоги при включенном мандатном управлении доступом	78
4.13. Запуск службы systemd с заданной меткой безопасности и привилегиями	80
4.14. Сетевое взаимодействие. Механизм privsock	82
4.15. Шина межпроцессного взаимодействия D-Bus	84
4.15.1. Виды сообщений	84
4.15.2. Процесс взаимодействия с шиной	85
4.15.3. Процесс соединения с системной шиной	86
4.15.4. Объекты и субъекты системы	87
4.15.5. Алгоритм проверки меток для различных сообщений и режимов работы	87
4.15.6. Привилегии процесса dbus-daemon	89
4.15.7. Расширенное управление политиками	90
4.15.7.1. Конфигурационный файл	90
4.15.7.2. Формирование клиентских политик из политик шины	95
4.16. Средства управления мандатными ПРД	96
4.16.1. pdpl-file	97
4.16.2. pdp-id	99
4.16.3. pdp-init-fs	100
4.16.4. pdp-ls	100

4.16.5. pdpl-ps	101
4.16.6. pdpl-user	102
4.16.7. pdpl-group	103
4.16.8. pdp-exec	104
4.16.9. sumac	105
4.16.10. sumic	107
4.16.11. userlev	107
4.16.12. usercat	108
4.17. Средства управления привилегиями пользователей и процессов	110
4.17.1. usercaps	110
4.17.2. execaps	113
4.17.3. pscaps	114
4.18. Мандатное управление доступом в комплексах программ гипертекстовой обработки данных электронной почты	115
4.19. Настройка параметров ядра в загрузчике GRUB 2	115
5. Защита среды виртуализации	120
5.1. Дискреционное управление доступом в среде виртуализации	120
5.2. Мандатное управление доступом к виртуальной машине	121
5.2.1. Применение правил мандатного управления доступом	122
5.2.2. Порядок запуска ВМ и входа в гостевую операционную систему ВМ	124
5.3. Ролевое управление доступом в среде виртуализации	126
5.3.1. Настройка ролей	126
5.3.2. Настройка ролевого управления доступом с использованием драйвера доступа polkit	129
5.3.3. Настройка ролевого управления доступом с использованием драйвера доступа parsec	130
5.4. Режим «только чтение»	131
5.5. Идентификация и аутентификация пользователей в среде виртуализации	132
5.6. Контроль целостности в среде виртуализации	134
5.6.1. Применение регламентного контроля целостности AFICK	134
5.6.2. Применение динамического контроля целостности в режиме ЗПС	135
5.6.2.1. Режим контроля неизменности и подлинности загружаемых исполняемых файлов	135
5.6.2.2. Режим контроля целостности файлов при их открытии	135
5.7. Доверенная загрузка виртуальных машин	136
5.7.1. Применение режима контроля целостности файлов при их открытии	136

5.7.1.1. Контроль файлов конфигурации виртуального оборудования виртуальных машин	136
5.7.1.2. Контроль файлов виртуальной базовой системы ввода-вывода (первичного загрузчика VM)	138
5.7.2. Применение механизма контроля целостности с использованием алгоритма работы с контрольными суммами («отпечаток конфигурации»)	141
5.7.3. Применение механизма контроля целостности файлов гостевой операционной системы	144
5.7.4. Контроль целостности образов VM	150
5.8. Регистрация событий безопасности в среде виртуализации	151
5.8.1. Настройка регистрации событий безопасности, связанных с функционированием средства виртуализации	151
5.8.2. Механизм централизованного сбора журналов с удаленных хостов виртуализации	151
5.9. Резервное копирование в среде виртуализации	153
5.9.1. Резервное копирование образов виртуальных машин	153
5.9.2. Резервное копирование конфигурации виртуального оборудования виртуальных машин	155
5.9.3. Резервное копирование параметров настройки средства виртуализации	156
5.9.4. Создание снимков текущего состояния машины	156
5.9.5. Резервное копирование виртуальных машин	157
5.9.5.1. Создание резервной копии	158
5.9.5.2. Восстановление из резервной копии	165
5.9.6. Резервное копирование и полная очистка журналов	169
5.9.7. Экспорт и импорт виртуальных машин	169
5.10. Управление потоками информации в среде виртуализации	170
5.10.1. Управление сетевыми фильтрами <code>nwfilter</code>	171
5.10.2. Реализация собственных правил	175
5.10.3. Управление сетевым трафиком в условиях мандатного управления доступом .	178
5.11. Защита памяти в среде виртуализации	179
5.12. Применение механизма контроля целостности областей памяти по запросу из гостевой операционной системы	182
5.13. Ограничение программной среды в среде виртуализации	187
5.14. Централизованное управление	187
5.14.1. Пример организации распределенного хранилища	188
5.14.1.1. Создание и подключение сетевого хранилища	188
5.14.1.2. Подключение на серверах виртуализации сетевого блочного устройства . . .	192

5.14.1.3. Настройка кластера	194
5.14.1.4. Автоматическое монтирование	200
5.14.1.5. Централизованное управление в среде виртуализации	201
5.14.2. Пример миграции виртуальных машин	202
6. Регистрация событий безопасности	206
6.1. Правила регистрации событий	206
6.2. Регистрация событий на основе меток безопасности	209
6.3. Журнал аудита	209
6.4. Средства управления аудитом	210
6.4.1. Графические утилиты	210
6.4.2. getfaud	210
6.4.3. setfaud	211
6.4.4. useraud	215
6.4.5. psaud	219
6.4.6. ausearch	221
6.4.7. Отключение регистрации событий	223
6.5. Подсистема регистрации событий	224
6.5.1. Регистрация событий и уведомление о событиях	224
6.5.2. Конфигурационный файл службы syslog-ng	227
6.5.3. Журнал событий	227
6.5.4. Самодиагностика подсистемы регистрации событий	230
6.6. Средства централизованного аудита и протоколирования	231
7. Изоляция процессов	232
8. Создание и защита изолированных программных сред (контейнеров)	233
8.1. Изоляция контейнеров и пространств	233
8.2. Выявление уязвимостей в образах контейнеров	235
8.2.1. Общие сведения	235
8.2.2. Базовое использование	237
8.2.3. Настройка сканирования и работа с oval-описаниями	237
8.2.3.1. Инструмент oval-db	237
8.2.3.2. Действия с файлами oval-описаний	238
8.2.3.3. Обновление oval-описаний	240
8.2.3.4. Вывод описаний уязвимостей	241
8.2.3.5. Загрузка файла игнорируемых уязвимостей	241

8.2.3.6. Конфигуратор системы сканирования уязвимостей	242
8.2.3.7. Конфигурационные файлы системы сканирования уязвимостей	244
8.2.3.8. Конфигурационный файл игнорируемых уязвимостей	249
8.2.4. Обновление oval-описаний в закрытом контуре	253
8.2.4.1. Настройка сервера обновления oval-описаний	253
8.2.4.2. Настройка клиента для обновления oval-описаний	256
8.3. Обеспечение корректности конфигурации контейнеров	256
8.3.1. Ограничение прав на использование ресурсов хостовой машины для Docker . .	257
8.3.2. Ограничение прав на использование ресурсов хостовой машины для Podman .	259
8.4. Контроль целостности образов и параметров настройки средства контейнеризации	259
8.5. Регистрация событий безопасности, связанных с контейнерами	261
8.6. Идентификация и аутентификация пользователей	262
8.7. Работа с Docker в непривилегированном режиме с ненулевыми метками безопас- ности	263
8.7.1. Принцип функционирования	263
8.7.2. Управление запуском контейнера с ненулевой меткой безопасности	266
8.7.3. Копирование образа в репозиторий пользователя	268
8.7.3.1. Копирование образа из системного репозитория	268
8.7.3.2. Копирование образа из пользовательского репозитория привилегированным пользователем	270
8.7.3.3. Копирование образа из пользовательского репозитория непривилегированным пользователем	271
8.7.4. Выполнение команд и запуск контейнеров в непривилегированном режиме от имени пользователя	273
8.8. Работа с Podman в непривилегированном режиме с ненулевыми метками безопас- ности	274
8.8.1. Создание пользовательской среды выполнения команд Podman	275
8.8.2. Работа с Podman от имени другого пользователя	277
8.8.3. Работа с Podman в сессии с ненулевой меткой безопасности	280
8.9. Защита приложений, установленных из репозитория Flatpak	281
8.9.1. Изоляция приложений, установленных из репозитория Flatpak	281
8.9.2. Выявление уязвимостей в приложениях из репозитория Flatpak	283
8.9.2.1. Общие сведения	283
8.9.2.2. Сканирования приложений на наличие уязвимостей	283
8.9.2.3. Настройка системы поиска уязвимостей	284

8.9.2.4. Настройка расписания поиска уязвимостей	286
8.9.3. Обеспечение корректности конфигурации приложений	287
8.9.4. Контроль целостности приложений и параметров настройки Flatpak	287
8.9.4.1. Контроль целостности приложений	287
8.9.4.2. Контроль целостности параметров настройки Flatpak	292
8.9.5. Регистрация событий безопасности, связанных с Flatpak	293
8.9.6. Идентификация и аутентификация пользователей	293
8.9.7. Запуск приложения с указанной меткой целостности	294
8.9.7.1. Формирование метки целостности запущенного приложения	294
8.9.7.2. Назначение метки целостности приложению	295
8.9.7.3. Просмотр метки целостности, назначенной приложению	296
8.9.7.4. Переопределение метки целостности при запуске приложения	297
8.9.7.5. Сохранение файлов из приложений с отрицательным значением линейного уровня целостности	298
9. Защита памяти	300
9.1. Очистка памяти	300
9.2. Средства ограничения прав доступа к страницам памяти	303
10. Контроль целостности	305
10.1. Подсчет контрольных сумм файлов и оптических дисков	305
10.2. Подсчет контрольных сумм файлов в deb-пакетах	306
10.3. Контроль соответствия дистрибутиву	307
10.4. Регламентный контроль целостности	309
10.5. Сервис электронной подписи	314
10.5.1. Принцип функционирования	315
10.5.2. Условия применения	316
10.5.3. Установка	317
10.5.4. Просмотр электронного документа	320
10.5.5. Проверка уровня конфиденциальности сессии	321
11. Надежное функционирование	322
11.1. Восстановление файловой системы	322
11.2. Режим восстановления ОС	322
11.2.1. Режим восстановления с использованием установочного носителя	323
11.2.2. Режим восстановления с использованием загрузчика GRUB 2	324
11.2.3. Доступ к корневой файловой системе	325

11.3. Средство восстановления	326
11.3.1. Установка средства восстановления	327
11.3.2. Использование средства восстановления	328
11.3.2.1. Использование стандартного набора инструментов восстановления	328
11.3.2.2. Использование расширенного набора инструментов восстановления	330
11.4. Инструмент командной строки <code>tar</code>	333
11.5. Комплекс программ <code>Vacula</code>	335
11.6. Снимки состояния ОС	336
11.6.1. Общие сведения	336
11.6.2. Управление снимками	336
11.6.3. Автоматическое управление снимками	338
11.7. Восстановление в режиме «Мобильный»	343
12. Фильтрация сетевого потока	344
12.1. Включение фильтрации сетевого потока	344
12.2. Фильтр сетевых пакетов <code>iptables</code>	344
12.3. Формирование правил	345
12.4. Порядок прохождения таблиц и цепочек	346
12.5. Механизм трассировки соединений	350
12.6. Критерии выделения пакетов	356
12.7. Действия и переходы	356
12.8. Поддержка фильтрации на основе классификационных меток	366
12.8.1. Модули <code>iptables</code> для работы с классификационными метками	366
12.8.2. Использование <code>ufw</code> для работы с классификационными метками	368
12.8.3. Использование <code>Open vSwitch</code> для работы с классификационными метками	369
12.8.4. Использование <code>nftables</code> для работы с классификационными метками	372
13. Маркировка документов	374
13.1. Общие сведения	374
13.2. Настройка печати в условиях мандатного управления доступом	375
13.3. Настройка маркировки	378
13.3.1. Шаблон маркера	379
13.3.2. Переменные маркировки	382
13.3.3. Файлы описания маркера	385
13.3.4. Изменение шрифта маркировки	386
14. Контроль подключения съемных машинных носителей информации	389

15. Сопоставление пользователя с устройством	391
16. Ограничение программной среды и функции безопасности	392
16.1. Замкнутая программная среда	392
16.1.1. Режимы функционирования	392
16.1.2. Типы подписей и наборы ключей	393
16.1.3. Создание дополнительных ключей	395
16.1.4. Отзыв открытых ключей	398
16.1.5. Модуль <code>digsig_verif</code>	400
16.1.5.1. Настройка модуля <code>digsig_verif</code>	400
16.1.5.2. Проверка подписи в исполняемых файлах и разделяемых библиотеках . . .	401
16.1.5.3. Проверка подписи в неисполняемых файлах	402
16.1.5.4. Системные интерфейсы модуля <code>digsig_verif</code>	403
16.1.6. Подписание файлов	406
16.1.7. Защита памяти при использовании ЗПС	408
16.1.7.1. Запрет исполнения кода из области памяти	408
16.1.7.2. Режимы запрета исполнения кода	409
16.1.7.3. Формирование хешей содержимого областей памяти	411
16.1.7.4. Подписание файлов хешей	412
16.1.7.5. Добавление программ в исключения	412
16.2. Режим Киоск-2	414
16.2.1. Профили пакета <code>parsec-kiosk2</code>	414
16.2.2. Синтаксис профилей <code>parsec-kiosk2</code>	414
16.2.3. Синтаксис, совместимый с <code>parsec-kiosk</code>	416
16.2.4. Работа с Киоск-2 через консоль	417
16.2.4.1. Включение и выключение Киоск-2	417
16.2.4.2. Протоколирование процессов	418
16.2.4.3. Создание профиля	418
16.2.5. Графическая утилита управления профилями	419
16.3. Графический киоск	419
16.4. Графический киоск в режиме «Мобильный»	420
16.5. Изоляция приложений	421
16.6. Функции безопасности системы	422
16.6.1. Общие параметры вызова переключателей	423
16.6.2. Монитор безопасности	423

16.6.3. Установка квот на использование системных ресурсов	425
16.6.4. Запрет установки бита исполнения	425
16.6.5. Блокировка консоли для пользователей	426
16.6.6. Блокировка консоли в режиме «Мобильный»	427
16.6.7. Блокировка интерпретаторов	428
16.6.8. Блокировка макросов	429
16.6.9. Блокировка трассировки ptrace	430
16.6.10. Блокировка клавиши <SysRq>	431
16.6.11. Управление автоматическим входом	432
16.6.12. Блокировка запуска программ пользователями	432
16.6.13. Запуск контейнеров Docker на пониженном уровне МКЦ	432
16.6.14. Управление сетевыми службами	433
16.6.15. Управление загрузкой модуля ядра lkrg	433
16.6.16. Блокировка автоматического конфигурирования сетевых подключений	433
16.6.17. Отключение отображения меню загрузчика	434
16.6.18. Ограничение на форматирование съемных носителей	434
16.6.19. Запрет монтирования съемных носителей	434
16.6.20. Включение на файловой системе режима работы «только чтение»	435
16.6.21. Запрет выключения компьютера пользователями	435
16.6.22. Управление вводом пароля для sudo	436
16.6.23. Блокировка использования инструмента sumac	436
16.6.24. Управление межсетевым экраном ufw	437
16.6.25. Блокировка доступа по протоколу SSH для root	437
16.6.26. Повышение уровня защищенности	438
16.6.27. Управление мандатным контролем целостности	440
16.6.28. Управление замкнутой программной средой	440
16.6.29. Управление безопасным удалением файлов	441
16.6.30. Управление очисткой разделов подкачки	442
16.6.31. Включение и выключение мандатного управления доступом	442
16.6.32. Управление AstraMode и MacEnable	443
16.6.33. Выключение и включение аудита файлов и процессов	443
16.6.34. Выключение и включение сетевого аудита	444
16.6.35. Выключение и включение мандатного управления доступом в кластерах СУБД	445
16.6.36. Блокировка загрузки и выгрузки модулей ядра	446

16.6.37. Запрет входа с несколькими категориями конфиденциальности	446
16.6.38. Запрет создания файла устройства /dev/mem	447
16.6.39. Запрет загрузки в режиме восстановления	447
16.7. Модуль безопасности Yama	448
16.8. Модуль безопасности lockdown	449
17. Генерация КСЗ	452
17.1. Предварительная настройка КСЗ	452
17.2. Безопасная настройка ОС	453
17.3. Профили настройки КСЗ	454
17.4. Управление настройками КСЗ	456
17.4.1. Структура настроек КСЗ	457
17.4.2. Просмотр и изменение настроек КСЗ	458
17.4.3. Экспорт настроек КСЗ	459
17.4.4. Добавление настроек КСЗ в существующий архив	461
17.4.5. Импорт настроек КСЗ	462
17.4.6. Удаление настроек КСЗ	464
17.5. Возможности по защите от угроз безопасности информации средствами защиты ОС	465
17.5.1. Угрозы, связанные с внедрением вредоносного кода и повышением привилегий	465
17.5.2. Угрозы виртуальной инфраструктуры	466
17.5.3. Угрозы несанкционированного изменения конфигурации системы и средств защиты информации	468
17.5.4. Угрозы несанкционированного доступа к информации	469
17.5.5. Угрозы несанкционированного использования ресурсов системы	470
17.5.6. Угрозы процедур идентификации/аутентификации	471
17.5.7. Угрозы сетевой инфраструктуры и веб-технологий	472
17.5.8. Угрозы раскрытия информации	473
17.5.9. Угрозы целостности данных	474
17.5.10. Угрозы некорректного использования функционала системы и ПО	475
17.5.11. Угрозы несанкционированного восстановления информации	476
17.5.12. Угрозы несанкционированного доступа к низкоуровневым данным	476
17.5.13. Угрозы «отказ в обслуживании»	477
18. Условия эксплуатации ОС	479
18.1. Обеспечение безопасности среды функционирования	479

18.2. Указания по эксплуатации ОС	480
18.3. Условия применения ПО	483
18.4. Условия исключения скрытых каналов	486
Перечень терминов	487
Перечень сокращений	488
РУСБ.10015-01 97 01-2 «Операционная система специального назначения «Astra Linux Special Edition». Руководство по КСЗ. Часть 2»	
РУСБ.10015-01 97 01-3 «Операционная система специального назначения «Astra Linux Special Edition». Руководство по КСЗ. Часть 3. Защищенная СУБД»	

1. ОБЩИЕ СВЕДЕНИЯ

КСЗ предназначен для реализации функций ОС по защите информации от НСД и предоставления администратору безопасности информации средств управления функционированием КСЗ.

ВНИМАНИЕ! После установки ОС интерактивный вход в систему суперпользователя `root` по умолчанию заблокирован. Создаваемый при установке операционной системы пользователь включается в группу `astra-admin`. Пользователям, входящим в названную группу, через механизм `sudo` предоставляются права для выполнения действий по настройке ОС, требующих привилегий суперпользователя `root`. Далее по тексту такой пользователь именуется администратором.

1.1. Приемка ОС

При приемке экземпляра ОС необходимо провести следующие проверки:

- 1) при способе поставки BOX:
 - а) проверка целостности упаковки;
 - б) проверка комплектности в соответствии с формуляром из комплекта поставки;
 - в) проверка маркировки — наличия в документе РУСБ.10015-01 30 02 «Операционная система специального назначения «Astra Linux Special Edition». Формуляр» уникального идентификатора ФСТЭК России¹⁾ или наличия знака СЗИ на упаковке и нерабочей поверхности установочного диска²⁾;
 - г) проверка контрольной суммы установочного диска и диска с документацией;
- 2) при способе поставки OEM:
 - а) проверка целостности упаковки;
 - б) проверка комплектности в соответствии с формуляром из комплекта поставки;
 - в) проверка маркировки — наличия в документе РУСБ.10015-01 30 02 уникального идентификатора ФСТЭК России¹⁾ или наличия знака СЗИ на упаковке и нерабочей поверхности установочного диска²⁾;
 - г) проверка контрольной суммы установочного диска;
- 3) при способе поставки по сетям связи¹⁾:
 - а) проверка электронной подписи изготовителя образа установочного диска и документа РУСБ.10015-01 30 02 в формате PDF;
 - б) проверка комплектности в соответствии с документом РУСБ.10015-01 30 02;
 - в) проверка маркировки — наличия в документе РУСБ.10015-01 30 02 уникального идентификатора ФСТЭК России;

¹⁾ При поставке для применения в автоматизированных системах, находящихся в компетенции ФСТЭК России.

²⁾ При поставке для применения в автоматизированных системах в защищенном исполнении, находящихся в компетенции Министерства обороны Российской Федерации.

г) проверка контрольной суммы образа установочного диска.

Подлинность и неизменность программного обеспечения изделия после установки на средство вычислительной техники подтверждаются средствами контроля целостности (`fly-admin-int-check`, `astra-int-check`) в соответствии с 10.3 путем вычисления и сравнения контрольных сумм исполняемых файлов и библиотек с эталонными значениями, хранящимися в файле `gostsums.txt`, размещенном на установочном диске и в обновлениях изделия.

Порядок контроля целостности при применении оперативных обновлений приведен в документе РУСБ.10015-01 31 01 «Операционная система специального назначения «Astra Linux Special Edition». Описание применения».

1.2. Состав КСЗ

В состав КСЗ входят следующие основные подсистемы:

- модули подсистемы безопасности PARSEC, входящие в состав ядра ОС;
- библиотеки;
- утилиты безопасности;
- подсистема протоколирования (регистрации);
- модули аутентификации;
- графическая подсистема;
- консольный вход в систему;
- средства восстановления;
- средства разграничения доступа к виртуальным машинам¹⁾;
- средства контроля подключаемых устройств;
- средства контроля целостности.

1.3. Контролируемые функции

КСЗ обеспечивает реализацию функций защиты информации от НСД.

Средства защиты информации из состава ОС обеспечивают в соответствии с требованиями безопасности информации к операционным системам²⁾ :

- идентификацию и аутентификацию;
- управление доступом;
- регистрацию событий безопасности;

¹⁾ Для процессоров, поддерживающих технологию виртуализации. Недоступно в режиме «Мобильный».

²⁾ Требования безопасности информации к операционным системам утверждены приказом ФСТЭК России от 19 августа 2016 г. № 119.

- ограничение программной среды;
- изоляцию процессов;
- защиту памяти;
- контроль целостности;
- надежное функционирование;
- маркирование документов;
- фильтрацию сетевого потока.

ЗСУБД из состава ОС обеспечивает в соответствии с требованиями по безопасности информации к системам управления базами данных¹⁾ :

- управление доступом;
- идентификацию и аутентификацию пользователей;
- контроль целостности в системе управления базами данных;
- регистрацию событий безопасности;
- резервное копирование и восстановление;
- обеспечение доступности;
- очистку памяти;
- производительность системы управления базами данных;
- ограничение программной среды.

Средства виртуальной инфраструктуры из состава ОС обеспечивают в соответствии с требованиями по безопасности информации к средствам виртуализации²⁾ :

- идентификацию и аутентификацию пользователей;
- доверенную загрузку виртуальных машин;
- контроль целостности;
- регистрацию событий безопасности;
- управление доступом;
- резервное копирование;
- управление потоками информации;
- защиту памяти;
- централизованное управление образами виртуальных машин и виртуальными машинами;
- ограничение программной среды.

¹⁾ Требования по безопасности информации к системам управления базами данных утверждены приказом ФСТЭК России от 14 апреля 2023 г. № 64.

²⁾ Требования по безопасности информации к средствам виртуализации утверждены приказом ФСТЭК России от 27 октября 2022 г. № 187.

Средства контейнеризации из состава ОС обеспечивают в соответствии с требованиями по безопасности информации к средствам контейнеризации¹⁾ :

- изоляцию контейнеров;
- выявление уязвимостей в образах контейнеров;
- проверку корректности конфигурации контейнеров;
- контроль целостности контейнеров и их образов;
- регистрацию событий безопасности;
- идентификацию и аутентификацию пользователей.

Реализация функций безопасности ОС основана на следующих основных положениях:

1) при моделировании и описании управления доступом в ОС на основе ГОСТ Р 59453.1-2021 используются следующие термины:

а) субъект доступа — активный компонент ОС (например, процесс, запущенный от имени учетной записи пользователя), доступы которого регламентируются политиками управления доступом;

б) сущность (объект доступа) — пассивный компонент ОС, доступ к которому регламентируется политиками управления доступом, при этом используются следующие виды сущностей (объектов доступа):

- сущность-объект (объект) — пассивный компонент ОС (например, файл, сетевое соединение (файл-сокет), устройство (файл-устройство), файл-образ виртуальной машины или контейнера, как средства изоляции программной среды), доступ к которому регламентируется политиками управления доступом, к частям которого по отдельности управление доступом не осуществляется;
- сущность-контейнер (контейнер) — пассивный составной компонент ОС (например, каталог, том), доступ к которому регламентируется политиками управления доступом, состоящий из сущностей-объектов или сущностей-контейнеров, к которым по отдельности возможно осуществление управления доступом;

2) с каждым пользователем системы связан уникальный численный идентификатор — идентификатор пользователя (UID), который однозначно соотносится с записью в БД пользователей, содержащей информацию о пользователях, включая их реальные и системные имена. БД пользователей поддерживается и управляется системным администратором. UID является ярлыком субъекта (номинальный субъект), которым система пользуется для определения прав доступа. БД пользователей в ОС может быть как локальной для системы, так и являться частью домена, функционирующего на основе протокола LDAP;

¹⁾ Требования по безопасности информации к средствам контейнеризации утверждены приказом ФСТЭК России от 4 июля 2022 г. № 118.

- 3) каждый пользователь входит в одну или более групп. Группа — это список пользователей системы, имеющий собственный идентификатор (GID). Поскольку группа объединяет несколько пользователей системы, в терминах политики безопасности она соответствует понятию «множественный субъект». GID является ярлыком множественного субъекта, которых у номинального субъекта может быть более одного. Таким образом, одному UID соответствует список GID;
- 4) роль действительного (работающего с сущностями) субъекта играет процесс. Каждому процессу присваивается единственный UID, являющийся идентификатором запустившего процесс номинального субъекта, т. е. пользователя. Процесс, порожденный некоторым процессом пользователя, наследует UID родительского процесса. Таким образом, все процессы, запускаемые пользователем, имеют его идентификатор. Все процессы, принадлежащие пользователю, образуют сеанс пользователя. Первый процесс сеанса пользователя порождается после прохождения процедур идентификации и аутентификации. При обращении процесса к сущности доступ предоставляется по результатам процедуры авторизации, т. е. обработки запроса на основе мандатных и дискреционных ПРД;
- 5) механизм ПРД реализован в ядре ОС, что обеспечивает его правильное функционирование при использовании любых компонентов, предоставляемых ОС. Реализация мандатного управления доступом затрагивает все подсистемы ядра, в которых реализовано дискреционное управление доступом. При этом оба вида управления доступом функционируют параллельно, не влияя на принятие решений друг друга (непротиворечивость). Доступ разрешается в том случае, если он возможен с учетом дискреционных и мандатных ПРД. Запрещается в случае, если доступ запрещен хотя бы одним из видов ПРД (дискреционным или мандатным).

1.4. Безопасная установка ОС

Безопасная установка ОС осуществляется в соответствии с РУСБ.10015-01 95 01-2 «Операционная система специального назначения «Astra Linux Special Edition». Руководство администратора. Часть 2. Установка и обновление».

1.5. Средства организации ЕПП

Организация ЕПП обеспечивает:

- сквозную аутентификацию в сети;
- централизацию хранения информации об окружении пользователей;
- централизацию хранения настроек системы защиты информации на сервере.

Сетевая аутентификация и централизация хранения информации об окружении пользователя подразумевает использование двух основных механизмов: поддержки кросс-платформенных серверных приложений для обеспечения безопасности (NSS) и подгружае-

мых аутентификационных модулей (PAM). Сквозная аутентификация в сети реализуется на основе протокола Kerberos с использованием службы каталогов LDAP в качестве источника данных для базовых системных служб на базе механизмов NSS и PAM. Подобный подход обеспечивает централизацию хранения информации об окружении пользователей (в том числе предназначенную для обеспечения мандатного управления доступом):

- существующие в системе уровни и категории конфиденциальности;
- минимальные и максимальные уровни конфиденциальности, доступные пользователям при входе в систему;
- минимальные и максимальные наборы категорий конфиденциальности, доступные пользователям при входе в систему;
- члены привилегированных групп, которые могут получать из БД службы каталогов LDAP определенную информацию о пользователях.

Кроме того, с использованием СЗФС CIFS обеспечено централизованное хранение домашних каталогов пользователей.

Для снижения нагрузки на сеть и повышения производительности в ЕПП может применяться кэширование редко изменяемой информации в локальном кэше.

ВНИМАНИЕ! Измененная на сервере информация может попасть в локальный кэш с задержкой.

Более подробное описание ЕПП приведено в РУСБ.10015-01 95 01-1 «Операционная система специального назначения «Astra Linux Special Edition». Руководство администратора. Часть 1».

2. ИДЕНТИФИКАЦИЯ И АУТЕНТИФИКАЦИЯ

Идентификация и аутентификация пользователей в ОС выполняется с учетом требований ГОСТ Р 58833-2020 «Защита информации. Идентификация и аутентификация. Общие положения».

В общем случае идентификация и аутентификация охватывают:

- 1) первичную идентификацию, включающую подготовку, формирование и регистрацию информации о субъекте (объекте) доступа, а также присвоение субъекту (объекту) доступа идентификатора доступа и его регистрацию в перечне присвоенных идентификаторов;
- 2) хранение и поддержание актуального состояния (обновление) идентификационной и аутентификационной информации субъекта (объекта) доступа в соответствии с установленными правилами;
- 3) вторичную идентификацию, которая обеспечивает опознавание субъекта доступа, запросившего доступ к объекту доступа, по предъявленному идентификатору;
- 4) аутентификацию, включающую проверку подлинности субъекта (объекта) доступа и принадлежности ему предъявленных идентификатора и аутентификационной информации.

Функция идентификации и аутентификации пользователей в ОС основывается на использовании механизма PAM.

PAM представляют собой набор разделяемых библиотек (т. н. модулей), с помощью которых системный администратор может организовать процедуру аутентификации (подтверждение подлинности) пользователей прикладными программами. Каждый модуль реализует собственный механизм аутентификации. Изменяя набор и порядок следования модулей, можно построить сценарий аутентификации.

Подобный подход позволяет изменять процедуру аутентификации без изменения исходного кода и повторного компилирования PAM.

Сценарии аутентификации (т. е. работа этих функций) описываются в конфигурационном файле `/etc/pam.conf` и в конфигурационных файлах, расположенных в каталоге `/etc/pam.d/`. Сама аутентификация выполняется с помощью PAM. Модули располагаются в каталоге `/lib/security` в виде динамически загружаемых объектных файлов.

Если не используется единое пространство пользователей (ЕПП), то аутентификация осуществляется с помощью локальной БД пользователей, расположенной в файле `/etc/passwd`, и локальной БД пользовательских паролей, расположенной в файле `/etc/shadow`. Подробную информацию о структуре этих файлов можно получить с помощью команд `man 5 passwd` и `man 5 shadow` соответственно. В ОС реализована воз-

возможность хранения аутентификационной информации пользователей, полученной с использованием хеш-функций по ГОСТ Р 34.11-2012 (ГОСТ Р 34.11-94).

При использовании ЕПП аутентификация пользователей осуществляется централизованно по протоколу Kerberos. Для защиты аутентификационной информации по умолчанию используются отечественные алгоритмы по ГОСТ 28147-89 и ГОСТ Р 34.11-2012.

В ЕПП в качестве источника данных для идентификации и аутентификации пользователей применяются службы каталогов LDAP. В результате вся служебная информация пользователей сети может располагаться на выделенном сервере в распределенной гетерогенной сетевой среде. Добавление новых сетевых пользователей в этом случае производится централизованно на сервере службы каталогов. Сетевые службы, поддерживающие возможность аутентификации пользователей (web, FTP, почта), могут вместо локальных учетных записей использовать тот же каталог LDAP проверки аутентификационной информации. Администратор сети может централизованно управлять конфигурацией сети, в т. ч. разграничивать доступ к сетевым службам.

Благодаря предоставлению информации LDAP в иерархической древовидной форме разграничение доступа в рамках службы каталогов LDAP может быть основано на введении доменов. В качестве домена в данном случае будет выступать поддереву службы каталогов LDAP. Сервисы LDAP позволяют разграничивать доступ пользователей к разным поддеревьям каталога, хотя по умолчанию в ОС реализуется схема одного домена.

По умолчанию в конфигурационном файле `/etc/pam.d/common-auth`, содержащем общие для всех служб настройки и предоставляющем службу для входа в систему, используется PAM-модуль `pam_faillock.so`. В данном PAM-модуле определено каким образом будет применяться блокировка учетной записи при исчерпании числа неудачных попыток аутентификации. Описание политик блокировки при неудачном входе и порядок их настройки приведены в документе РУСБ.10015-01 95 01-1 «Операционная система специального назначения «Astra Linux Special Edition». Руководство администратора. Часть 1».

3. ДИСКРЕЦИОННОЕ УПРАВЛЕНИЕ ДОСТУПОМ

3.1. Общие сведения

Дискреционное управление доступом применяется к каждому субъекту и сущности. Механизм дискреционного управления доступом именованных субъектов (пользователей) к именованным сущностям (файлам, каталогам) обеспечивает создание для каждой пары субъект-сущность явного и недвусмысленного перечисления разрешенных типов доступа, формирующих правила разграничения доступа (ПРД). На защищаемые именованные сущности при их создании автоматически устанавливаются базовые дискреционные ПРД в виде:

- идентификаторов пользователя-владельца (UID) и группы-владельца (GID), которые вправе распоряжаться доступом к данной сущности;
- прав доступа данных субъектов к созданной сущности.

Сущностями доступа являются:

- файлы;
- каталоги;
- соединения (сокеты);
- сетевые пакеты;
- механизмы IPC (разделяемая память, очереди сообщений и др.).

3.1.1. Права доступа

Доступ к сущности определяется тремя видами операций:

- чтение (`read`, символьный код `r`, восьмеричный код 4, битовая маска 100);
- запись (`write`, символьный код `w`, восьмеричный код 2, битовая маска 010);
- исполнение (`execution`, буквенный код `x`, восьмеричный код 1, битовая маска 001).

В записях прав доступа на первом месте всегда операция чтения, на втором — операция записи, а на третьем — операция исполнения.

Права доступа к сущности определяют разрешенные с ней операции для каждого из трех классов субъектов:

- пользователь-владелец сущности (`user`, `u`);
- группа-владелец сущности (`group`, `g`);
- все остальные (`other`, `o`).

При отображении/указании прав доступа к сущности на первом месте записываются права доступа пользователя-владельца, на втором — группы-владельца, на третьем — прочих субъектов доступа.

Указываться права доступа могут символьной строкой, восьмеричными числами и битовыми масками:

- 1) при обозначении символьной строкой каждый тип операции обозначается соответствующим ему символьным кодом, а отсутствие права на операцию — символом «—» в соответствующей позиции, например:
 - а) `rwX` — разрешены все операции;
 - б) `r-x` — разрешены операции чтения и исполнения (`r` и `x`), а операция записи запрещена;
- 2) при обозначении восьмеричными числами используются суммы восьмеричных чисел разрешенных операций, например:
 - а) символьной строке `rwX` соответствует восьмеричная цифра 7 ($4+2+1$);
 - б) символьной строке `r-x` соответствует восьмеричная цифра 5 ($4+0+1$);
- 3) при обозначении битовыми масками суммируются битовые маски разрешенных операций:
 - а) символьной строке `rwX` соответствуют восьмеричная цифра 7 и битовая маска 111;
 - б) символьной строке `r-x` соответствуют восьмеричная цифра 5 и битовая маска 101.

Набор прав доступа к сущности для всех классов субъектов может быть представлен:

- символьной строкой из 9 символов, по три символа для каждого класса субъектов доступа: `rwXr-xr-x`;
- строкой из трех восьмеричных цифр, по одной цифре для каждого класса субъектов доступа: 755;
- битовой маской, по три бита для каждого класса субъектов доступа: 111 101 101.

При обращении субъекта к сущности (с запросом операции определенного вида, т. е. на чтение, запись или исполнение) система проверяет совпадение идентификаторов UID и GID процесса с идентификаторами UID и GID файла в определенном порядке, и, в зависимости от результата, применяет ту или иную группу прав.

Права доступа сущности могут быть изменены, если это разрешено (санкционировано) текущими правилами разграничения доступа.

Дополнительно для сущностей могут быть установлены следующие специальные биты:

- SUID (Set User ID) — бит, позволяющий запускать исполняемый файл с правами пользователя-владельца;
- SGID (Set Group ID) — бит, позволяющий запускать исполняемый файл с правами группы-владельца;
- Sticky — бит, ограничивающий удаление чужих файлов в каталоге.

Когда пользователь или процесс запускают исполняемый файл с установленными битами SUID и/или SGID, то файл запускается с правами своего владельца и/или группы (в зависимости от того, какой бит задан). Таким образом, пользователь может запускать файлы даже от имени `root`, не обладая его привилегиями. При установке на каталог бита SGID все вновь создаваемые в нем файлы будут наследовать группу каталога, а вновь создаваемые подкаталоги — и группу каталога, и SGID-бит (установка SUID-бита на каталог ни на что не влияет).

Sticky-бит используется для ограничения удаления объектов в каталоге. При его установке на каталог удалять или переименовывать сущности в этом каталоге сможет только их владелец или суперпользователь. Это полезно при совместной работе нескольких пользователей, так как без Sticky-бита любой пользователь с правами записи в каталог сможет удалять в нем любые файлы, включая созданные другими пользователями. Установить или снять Sticky-бит с каталога может владелец каталога или `root`. Установка Sticky-бита на файл ни на что не влияет.

Для просмотра прав, установленных на сущности в каталоге, выполнить из данного каталога команду:

```
ls -l
```

Вывод команды будет иметь следующий вид:

```
-rw-r--r-- 1 user user 485 авг 28 10:17 readme.txt
-rwSr-Sr-- 1 user user 20 авг 28 10:17 script.sh
drwxr-xr-x 2 user user 4096 авг 22 13:12 Видео
drwxr-xr-x 2 user user 4096 авг 22 13:12 Документы
drwxr-xr-t 2 user user 4096 авг 22 13:12 Музыка
```

В первом столбце вывода перечисляются установленные дискреционные ПРД сущностей. В таблице 1 приведена их расшифровка.

Таблица 1

Тип сущности	Права пользователя- владельца			Права группы- владельца			Права всех остальных			...	Имя сущности
-	r	w	S	r	-	S	r	-	-		script.sh
d	r	w	x	r	-	x	r	-	t		Музыка

Специальные обозначения:

- тип сущности: символ «-» — файл, d — каталог;
- символ S в правах пользователя-владельца и/или группы-владельца означает установленные SUID и/или SGID биты соответственно;
- символ t на последней позиции означает установленный Sticky-бит.

Изменение прав доступа к сущностям осуществляется с помощью инструмента `chmod` (см. 3.3.3). Смена пользователя-владельца и группы-владельца сущности возможна с помощью инструментов `chown` (см. 3.3.1) и `chgrp` (см. 3.3.2).

Также изменить права доступа к сущностям и сменить их пользователя-владельца и группу-владельца можно в программе «Менеджер файлов» (`fly-fm`). Операции выполняются во вкладке «Дискреционные атрибуты» окна «Свойства», описание операций приведено в электронной справке.

3.1.2. Списки контроля доступа

В дополнение к правам доступа к сущностям в ОС поддерживаются списки контроля доступа ACL (Access Control List), реализованные на основе расширенных атрибутов файловых систем. ACL состоит из набора записей, каждая из которых описывает права доступа. Использование ACL позволяет задавать права доступа индивидуально для любых пользователей и/или групп пользователей, дополняя и расширяя базовый механизм «пользователь-владелец — группа-владелец — все остальные». С помощью ACL доступ может быть предоставлен пользователю или группе независимо от доступа других пользователей, участия в группах и предоставления доступа к иным сущностям.

Дополнительно ACL предоставляет:

- механизм масок, позволяющий ограничивать права доступа;
- механизм наследования ACL (ACL по умолчанию), позволяющий определять порядок присвоения прав доступа вновь создаваемым сущностям.

Права доступа к сущности для пользователя-владельца, группы-владельца и всех остальных имеют соответствующее представление в ACL в виде отдельных записей. Таким образом,

каждой сущности всегда сопоставляется минимальный ACL, включающий три записи: для пользователя-владельца, группы-владельца и всех остальных.

При использовании минимального ACL:

- стандартные права доступа для класса пользователь-владелец (например, `rwX`) отображаются в идентичные права доступа записи ACL типа пользователь-владелец (например, `user::rwX`);
- стандартные права доступа для класса группа-владелец (например, `rw-`) отображаются в идентичные права доступа записи ACL типа группа-владелец (например, `group::rw-`);
- стандартные права доступа для класса все остальные (например, `r--`) отображаются в идентичные права доступа записи ACL типа все остальные (например, `other::r--`).

В дополнение к минимальному ACL могут быть указаны записи для прав доступа других именованных субъектов, а также запись для маски доступа. ACL, содержащий более трех записей, называется расширенным ACL.

В общем случае расширенный ACL включает записи следующих типов:

- пользователь-владелец (текстовое представление `user::rwX`);
- именованный пользователь (текстовое представление `user:<пользователь>:rwX`);
- группа-владелец (текстовое представление `group::rwX`);
- именованная группа (текстовое представление `user:<группа>:rwX`);
- маска доступа (текстовое представление `mask::rwX`);
- все остальные (текстовое представление `other::rwX`).

Запись для маски доступа появляется добавлении прав для именованных пользователей и/или именованных групп в списке прав доступа. Если маска не была указана в явном виде, то в записи будет отображаться эффективная маска, представляющая собой сумму прав доступа группы-владельца, именованных пользователей и именованных групп.

Маска доступа, указанная в явном виде, может использоваться для ограничения прав доступа группы-владельца, именованных пользователей и именованных групп. На права пользователя-владельца и остальных пользователей маска не влияет. В маске указываются максимальные эффективные права, которые получают группа-владелец, именованные пользователи и именованные группы, независимо от того, какие именно права были для них заданы. Например, если в маске указано `r-x`, то группа-владелец, именованные пользователи и именованные группы будут иметь доступ на чтение и исполнение, но не получают доступ на запись, даже если он будет явно указан в их записях ACL.

Реализованная в механизме дискреционных ПРД проверка прав доступа субъекта к сущности (файлу/каталогу) выполняется в два этапа.

На первом этапе выбирается запись ACL, соответствующая субъекту. Дискреционные ПРД для сущностей просматриваются в следующем порядке, при этом решение о доступе принимается только на основе одной выбранной записи ACL:

- права пользователя-владельца;
- права именованных пользователей;
- права группы-владельца;
- права именованных групп;
- права для всех остальных.

На втором этапе проверяется, что запись ACL содержит права на вид доступа, который запрашивает субъект.

Алгоритм проверки прав доступа имеет следующий вид:

- 1) проверяется, является ли субъект доступа пользователем-владельцем сущности. Если это так, то проверяется, разрешен ли в соответствии с выбранной записью ACL запрошенный субъектом вид доступа. По результатам проверки доступ либо разрешается, либо запрещается. Если субъект доступа не является пользователем-владельцем сущности, проверка продолжается;
- 2) проверяется, является ли субъект доступа одним из именованных пользователей, указанных в записях ACL. Если это так, то проверяется, разрешен ли в соответствии с выбранной записью ACL и записью маски доступа ACL запрошенный субъектом вид доступа. По результатам проверки доступ либо разрешается, либо запрещается. Если субъект доступа не является именованным пользователем, указанным в записях ACL, проверка продолжается;
- 3) проверяется, является ли одна из групп субъекта доступа группой-владельцем сущности. Если это так, то проверяется, разрешен ли в соответствии с выбранной записью ACL запрошенный субъектом вид доступа. По результатам проверки доступ либо разрешается, либо запрещается. Если ни одна из групп субъекта доступа не является группой-владельцем сущности, проверка продолжается;
- 4) проверяется, является ли одна из групп субъекта доступа одной из именованных групп, указанных в записях ACL. Если это так, то проверяется, разрешен ли в соответствии с выбранной записью ACL и записью маски доступа ACL запрошенный субъектом вид доступа. По результатам проверки доступ либо разрешается, либо запрещается. Если ни одна из групп субъекта доступа не является именованной группой, указанной в записях ACL, проверка продолжается;

- 5) проверяется, разрешен ли в соответствии с записью ACL для всех остальных запрошенный субъектом вид доступа. По результатам проверки доступ либо разрешается, либо запрещается;
- 6) если доступ не был разрешен при проведении предыдущих проверок, то он запрещается.

Дискреционные ПРД сущностей, как стандартные для трех классов (пользователя-владельца, группы-владельца и всех остальных), так и указанные в записях ACL, могут изменяться следующими субъектами:

- суперпользователем `root` (пользователь с `uid=0`, по умолчанию заблокирован);
- администратором с использованием механизма `sudo`;
- пользователем-владельцем сущности.

Изменение стандартных прав доступа пользователя-владельца, группы-владельца и всех остальных автоматически приводит к изменению соответствующих прав в записи ACL. Аналогичным образом, смена прав в записях минимального ACL автоматически изменяет стандартные права.

При использовании для сущности расширенного ACL изменение дискреционных ПРД обеспечивается следующим образом:

- 1) при изменении стандартных прав доступа они идентично изменяются в соответствующих записях ACL;
- 2) при изменении в ACL прав доступа пользователя-владельца, группы-владельца и всех остальных они идентично изменяются в стандартных правах;
- 3) при задании в ACL прав доступа именованных пользователей и/или именованных групп в стандартных правах для группы-владельца будет отображаться эффективная маска ACL;
- 4) при явном задании маски ACL она будет отображаться в стандартных правах группы-владельца.

Если для сущности назначен расширенный ACL, то в выводе команды:

```
ls -l
```

в конце записи стандартных прав доступа присутствует символ «+».

Пример

```
-rw-r--r-x+ 1 user user 20 авг 28 11:44 1.txt
```

Каталогам дополнительно возможно добавить записи ACL, которые будут являться ACL по умолчанию. ACL по умолчанию применяется для назначения создаваемыми внутри каталога сущностям заданный ACL. ACL по умолчанию не применяется к самому каталогу и к уже существующим сущностям. Вновь создаваемые каталоги будут наследовать ACL по умолчанию.

ACL по умолчанию включает в себя тот же набор записей, что и обычный ACL:

- пользователь-владелец (текстовое представление `default:user::rwx`);
- именованный пользователь (текстовое представление `default:user:<пользователь>:rwx`);
- группа-владелец (текстовое представление `default:<группа>::rwx`);
- именованная группа (текстовое представление `default:group:<группа>:rwx`);
- маска доступа (текстовое представление `default:mask::rwx`);
- все остальные (текстовое представление `default:other::rwx`).

Маска доступа добавляется автоматически при наличии в ACL по умолчанию записей для именованных пользователей и/или групп.

Просмотреть записи ACL, назначенные сущности, можно с помощью инструмента `getfacl` (см. 3.3.5). Изменение записей выполняется инструментом `setfacl` (см. 3.3.6).

Таким образом, реализованный в ОС механизм, регулирующий принцип дискреционного управления доступом, предусматривает санкционированное изменение дискреционных ПРД, включая санкционированное изменение списка субъектов и списка защищаемых сущностей.

3.1.3. Идентификаторы принадлежности процессов

В ОС каждый процесс имеет следующие виды идентификатора принадлежности, которые определяют владельца процесса и его привилегии:

- RUID (Real User ID) — реальный идентификатор, равен значению UID пользователя, запустившего процесс. Как правило, значение RUID остается постоянным на протяжении всего жизненного цикла процесса, запущенного конкретным пользователем;
- EUID (Effective User ID) — действующий идентификатор. В большинстве случаев EUID совпадает с RUID. Этот идентификатор используется для проверки разрешений доступа к системным ресурсам, таким как файлы, каталоги и устройства. Значение EUID изменяется при запуске исполняемых файлов с установленным SUID-битом (Set User ID). В этом случае значение EUID становится равным значению UID пользователя-владельца файла (см. 3.1.1);
- SUID (Saved User ID) — сохраненный идентификатор. При запуске процесса этому идентификатору присваивается значение EUID. Как правило, значение SUID остается

постоянным на протяжении всего жизненного цикла процесса. Используется для восстановления значения EUID после его изменения при выполнении процесса.

Идентификаторам RUID, EUID, SUID первого процесса сессии пользователя присваивается значение UID этого пользователя. Дочерний процесс наследует значения всех идентификаторов от родительского процесса без изменений.

Процесс, запущенный с привилегиями `root`, то есть значение EUID этого процесса равно 0, может изменить значения своих идентификаторов RUID, EUID и SUID, если такая функция заложена в исполняемом коде процесса. Процесс, запущенный от имени непривилегированного пользователя, то есть значение EUID этого процесса не равно 0, может изменить только значение своего идентификатора EUID, если такая функция заложена в исполняемом коде процесса.

При изменении значений идентификаторов RUID, EUID и SUID будут изменены привилегии этого процесса (см. 3.2.4).

Пример

Исполняемый файл `/etc/passwd` принадлежит пользователю `root` и имеет установленный SUID-бит. Если непривилегированный пользователь выполнит команду `passwd`, то запущенному процессу будут присвоены следующие значения идентификаторов:

- значению RUID — значение UID пользователя, запустившего процесс, например 1000;
- значению EUID — значение UID пользователя-владельца файла, то есть 0, UID пользователя `root`;
- значению SUID — значение EUID, присвоенное процессу при запуске, то есть 0, UID пользователя `root`.

Процесс будет выполняться с привилегиями `root`, что позволит ему изменять файл системных паролей `/etc/shadow`.

3.2. Linux-привилегии

3.2.1. Linux-привилегии пользователей и процессов

Linux-привилегии используются для предоставления отдельным пользователям прав выполнения определенных административных действий и являются стандартными для системы Linux. Каждая Linux-привилегия имеет номер, шестнадцатеричное значение и название. Описание представлено на справочной странице `man capabilities`.

Для управления Linux-привилегиями пользователей можно использовать модуль «Пользователи» в разделе «Пользователи и группы» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку) или инструмент `usercaps` (см. 4.17.1).

В качестве субъекта, выполняющего административные действия, выступает процесс, принадлежащий пользователю. Linux-привилегии процесса могут быть включены в следующие наборы:

- `bounding` (ограничивающий) — максимальный набор Linux-привилегий, доступных для процесса и его потомков. В ОС этот набор не используется для ограничения Linux-привилегий, которые могут быть назначены процессу. Поэтому в набор `bounding` добавлены все Linux-привилегии;
- `permitted` (разрешенный) — набор Linux-привилегий, которые могут быть добавлены в набор `effective`;
- `effective` (действующий) — текущие активные привилегии процесса. Ядро ОС проверяет этот набор при выполнении привилегированных операций. Процесс может динамически изменять его на основе набора `permitted`, если такая функция заложена в исполняемом коде процесса. При этом процесс не может добавить Linux-привилегию в набор `effective`, если ее нет в наборе `permitted`;
- `inheritable` (наследуемый) — набор Linux-привилегий, которые могут быть унаследованы дочерним процессом;
- `ambient` (набор окружения) — набор Linux-привилегий, которые могут быть унаследованы дочерним процессом. Применяется при запуске дочернего процесса из файла. Процесс может динамически изменять набор `ambient` на основе набора `permitted`, если такая функция заложена в исполняемом коде процесса. При этом процесс не может включить Linux-привилегию в набор `ambient`, если ее нет в наборе `permitted`.

Linux-привилегии, включенные в набор, формируют битовую маску этого набора.

Первому процессу сессии пользователя присваиваются Linux-привилегии этого пользователя. При этом Linux-привилегии пользователя включаются в набор `inheritable` Linux-привилегий процесса. Процесс, порожденный некоторым процессом пользователя, наследует Linux-привилегии родительского процесса.

Если непривилегированный пользователь запускает процесс из исполняемого файла, то для выполнения административных действий недостаточно Linux-привилегий, назначенных пользователю. Требуется такие же Linux-привилегии назначить исполняемому файлу, из которого запускается процесс.

Для управления Linux-привилегиями исполняемых файлов применяется инструмент `setcap` (см. 3.2.2.1). Для просмотра Linux-привилегий, назначенных исполняемому файлу, используется инструмент `getcap` (см. 3.2.2.2). Особенности назначения Linux-привилегий процессу,

который запускается из исполняемого файла, описан в 3.2.3.

Процессы, запущенные от имени суперпользователя, независимо от наличия у них привилегий, обладают возможностями выполнения всех административных действий.

Для администрирования ОС с включенным мандатным управлением доступом множество привилегий Linux расширено специальными PARSEC-привилегиями, описание которых приведено в 4.7. Для настройки КСЗ могут использоваться как Linux-, так и PARSEC-привилегии. Порядок управления PARSEC-привилегиями описан в 4.17.

3.2.2. Инструменты управления Linux-привилегиями файлов

3.2.2.1. setcap

Инструмент командной строки `setcap` применяется администратором для управления Linux-привилегиями исполняемых файлов. Наборы Linux-привилегий исполняемого файла используются для формирования наборов Linux-привилегий нового процесса, который запускает непривилегированный пользователь из этого файла (см. 3.2.3).

Для назначения Linux-привилегий исполняемому файлу выполнить команду:

```
sudo setcap <названия_Linux-привилегий><действие><наборы> <файл>
```

Названия Linux-привилегий перечисляются через запятую. Названия всех Linux-привилегий можно получить с помощью инструмента `usercaps` (см. 4.17.1).

Действие определяет порядок формирования наборов Linux-привилегий исполняемого файла. В команде используются следующие обозначения действий:

- «=» — сначала очистить все наборы, а затем добавить указанные Linux-привилегии в заданные наборы;
- «+» — добавить указанные Linux-привилегии в заданные наборы;
- «-» — исключить указанные Linux-привилегии из заданных наборов.

Наборы Linux-привилегий исполняемого файла в команде обозначаются следующими буквами латинского алфавита:

- p — набор `permitted`;
- i — набор `inheritable`;
- e — набор `effective`.

Следует учитывать, что нельзя добавить Linux-привилегию в набор `effective`, если ее нет в наборе `permitted`. Допускается одновременно добавлять Linux-привилегию в эти наборы, то есть использовать комбинацию флагов `pe`.

Примечание. Наборы *p*, *i*, *e* можно указывать в любом порядке. Например, запись *pei* эквивалентна *epi*.

Пример

Для исполняемого файла *my_executable* добавить Linux-привилегии *cap_chown* и *cap_sys_module* в наборы *permitted* и *effective*:

```
sudo setcap cap_sys_module,cap_chown=ep my_executable
```

Для просмотра Linux-привилегий, назначенных исполняемому файлу, используется инструмент *getcap* (см. 3.2.2.2).

Для добавления всех возможных Linux-привилегий в наборы исполняемого файла выполнить команду:

```
sudo setcap all=<наборы> <файл>
```

или

```
sudo setcap =<наборы> <файл>
```

Для сброса всех Linux-привилегий, ранее назначенных исполняемому файлу, выполнить команду:

```
sudo setcap -r <файл>
```

Подробное описание *setcap* приведено на справочной странице `man setcap`.

3.2.2.2. *getcap*

Для просмотра Linux-привилегий, назначенных исполняемому файлу, используется инструмент командной строки *getcap*.

Синтаксис команды:

```
sudo getcap <файл>
```

Пример

Вывести все Linux-привилегии, назначенные исполняемому файлу *my_executable*:

```
sudo getcap my_executable
```

Вывод после выполнения команды:

```
my_executable cap_chown,cap_sys_module=ep
```

В выводе отображено, что для исполняемого файла `my_executable` в наборы `permitted` и `effective` включены Linux-привилегии `cap_chown` и `cap_sys_module`.

Подробное описание `getcap` приведено на справочной странице `man getcap`.

3.2.3. Наследование и формирование наборов Linux-привилегий

Первому процессу сессии пользователя присваиваются Linux-привилегии этого пользователя, которые включаются в набор `inheritable` процесса.

Если новый процесс создается как дочерний от процесса пользователя, когда такая функция заложена в его исполняемом коде, то дочерний процесс получает наборы Linux-привилегий родительского процесса без изменений.

Если новый процесс создается из исполняемого файла от имени пользователя `root`, то есть значение EUID нового процесса равно 0, то в наборы Linux-привилегий нового процесса будут включены все возможные Linux-привилегии. Порядок присвоения значения идентификатору EUID процесса описан в 3.1.3.

Если новый процесс создается из исполняемого файла от имени непривилегированного пользователя, то есть значение EUID нового процесса отлично от нуля, то по умолчанию все наборы нового процесса, за исключением `inheritable`, будут пустыми. Чтобы в набор `effective` этого процесса были включены Linux-привилегии пользователя, необходимо предварительно включить их в наборы `permitted` и `effective` исполняемого файла (см. 3.2.2.1).

ВНИМАНИЕ! Назначать Linux-привилегии необходимо непосредственно исполняемому файлу. Не допускается назначать Linux-привилегии символической ссылке, которая используется для запуска исполняемого файла.

При запуске процесса от имени непривилегированного пользователя из исполняемого файла формирование наборов Linux-привилегий нового процесса выполняется по следующим правилам:

- 1) состав набора `ambient` нового процесса зависит от наличия Linux-привилегий для исполняемого файла, из которого запускается процесс:
 - если исполняемому файлу назначены Linux-привилегии, то набор `ambient` нового процесса очищается, то есть из набора `ambient` удаляются все Linux-привилегии;

- если исполняемому файлу не назначены Linux-привилегии, то набор `ambient` нового процесса будет соответствовать набору `ambient` родительского процесса;
- 2) состав набора `permitted` нового процесса формируется как объединение, то есть побитовое ИЛИ битовых масок следующих наборов:
- набор Linux-привилегий, которые присутствуют одновременно как в наборе `inheritable` родительского процесса, так и в наборе `inheritable` исполняемого файла, из которого запускается процесс;
 - набор `permitted` исполняемого файла;
 - набор `ambient` нового процесса;
- 3) состав набора `effective` нового процесса зависит от состава набора `effective` исполняемого файла, из которого запускается процесс:
- если набор `effective` исполняемого файла не пустой, то набор `effective` нового процесса будет соответствовать набору `permitted` нового процесса;
 - если набор `effective` исполняемого файла пустой, то набор `effective` нового процесса будет соответствовать набору `ambient` нового процесса;
- 4) набор `inheritable` нового процесса наследуется от родительского процесса без изменений.

Примечание. В качестве родительского выступает процесс, с помощью которого пользователь взаимодействует с исполняемым файлом.

Пример

Непривилегированному пользователю назначена Linux-привилегия `CAP_SYS_MODULE`, которая позволяет загружать и выгружать модули ядра. Linux-привилегии пользователя включаются в набор `inheritable` Linux-привилегий процессов, запущенных этим пользователем. По умолчанию все остальные наборы Linux-привилегий процессов пользователя пустые.

Для вывода перечня Linux-привилегий, назначенных первому процессу сессии пользователя, то есть процессу с идентификатором 0, выполнить команду:

```
/sbin/getpcaps 0
```

Вывод после выполнения команды:

```
0: cap_sys_module=i
```

В выводе отображено, что процессу с идентификатором 0 назначена Linux-привилегия `CAP_SYS_MODULE`, которая включена в набор `inheritable`.

Команда на загрузку модуля ядра будет выполнена в эмуляторе терминала, который будет выступать в качестве родительского процесса. Для вывода Linux-привилегий, назначенных этому процессу, необходимо:

- 1) определить идентификатор процесса, соответствующего эмулятору терминала:

```
pgrep terminal
```

Вывод после выполнения команды:

```
3215
```

- 2) вывести перечень Linux-привилегий, назначенных процессу:

```
/sbin/getpcaps 3215
```

Вывод после выполнения команды:

```
3215: cap_sys_module=i
```

В выводе отображено, что процессу, соответствующему эмулятору терминала, также назначена Linux-привилегия `CAP_SYS_MODULE`, которая включена в набор `inheritable`.

В ОС управление модулями ядра выполняют процессы, запускаемые из исполняемого файла `/bin/kmod`. Для управления загрузкой модулей ядра с учетом зависимостей используется файл `/sbin/modprobe`, который является символической ссылкой, предназначенной для запуска процесса из файла `/bin/kmod`.

Если исполняемому файлу `/bin/kmod` не назначены Linux-привилегии, то процесс, который будет запущен из этого файла, получит следующие наборы Linux-привилегий:

- 1) в наборе `ambient` нового процесса будут отсутствовать Linux-привилегии, так как в наборе `ambient` родительского процесса Linux-привилегии отсутствуют;
- 2) в наборе `permitted` нового процесса будут отсутствовать Linux-привилегии, так как:
 - нет Linux-привилегий, которые одновременно присутствуют как в наборе `inheritable` родительского процесса, так и в наборе `inheritable` исполняемого файла;
 - в наборе `permitted` исполняемого файла Linux-привилегии отсутствуют;
 - в наборе `ambient` этого процесса Linux-привилегии отсутствуют;
- 3) в наборе `effective` нового процесса будут отсутствовать Linux-привилегии, так как набор `effective` исполняемого файла пустой и тогда

набор `effective` нового процесса должен соответствовать набору `ambient` этого процесса;

4) в наборе `inheritable` нового процесса будет присутствовать Linux-привилегия `CAP_SYS_MODULE`, так как набор `inheritable` наследуется от родительского процесса без изменений.

Таким образом процессу, запущенному из исполняемого файла `/bin/kmod`, будет запрещено загружать и выгружать модули ядра, так как в наборе `effective` этого процесса отсутствует Linux-привилегия, разрешающая такие действия.

При попытке загрузить модуль ядра `evbug` командой:

```
/sbin/modprobe evbug
```

Будет выведено сообщение об ошибке выполнения команды следующего содержания:

```
modprobe: ERROR: could not insert 'evbug': Operation not permitted
```

Чтобы непривилегированному пользователю предоставить возможность загружать и выгружать модули ядра, необходимо для исполняемого файла `/bin/kmod` добавить Linux-привилегию `CAP_SYS_MODULE` в наборы `permitted` и `effective`. Для этого необходимо от имени администратора выполнить команду:

```
sudo setcap cap_sys_module=ep /bin/kmod
```

Для вывода всех Linux-привилегий, назначенных исполняемому файлу `/bin/kmod`, в сессии непривилегированного пользователя выполнить команду:

```
/sbin/getcap /bin/kmod
```

Вывод после выполнения команды:

```
/bin/kmod cap_sys_module=ep
```

Процесс, который будет запущен из файла `/bin/kmod` непривилегированным пользователем, получит следующие наборы Linux-привилегий:

- 1) в наборе `ambient` нового процесса будут отсутствовать Linux-привилегии, так как этот набор должен быть очищен;
- 2) в составе набора `permitted` нового процесса будет присутствовать только Linux-привилегия `CAP_SYS_MODULE`, так как:
 - нет Linux-привилегий, которые одновременно присутствуют как в наборе `inheritable` родительского процесса, так и в наборе `inheritable` исполняемого файла;

- в наборе `permitted` исполняемого файла присутствует Linux-привилегия `CAP_SYS_MODULE`;
 - в наборе `ambient` этого процесса Linux-привилегии отсутствуют;
- 3) в наборе `effective` нового процесса будет присутствовать Linux-привилегия `CAP_SYS_MODULE`, так как набор `effective` исполняемого файла не пустой и тогда набор `effective` нового процесса должен соответствовать набору `permitted` нового процесса;
- 4) в наборе `inheritable` нового процесса будет присутствовать Linux-привилегия `CAP_SYS_MODULE`, так как набор `inheritable` наследуется от родительского процесса без изменений.

Таким образом процессу, запущенному из исполняемого файла `/bin/kmod`, будет разрешено загружать и выгружать модули ядра, так как в наборе `effective` этого процесса присутствует Linux-привилегия `CAP_SYS_MODULE`, разрешающая такие действия. При попытке загрузить модуль ядра `evbug` командой:

```
/sbin/modprobe evbug
```

Команда будет выполнена без вывода сообщения об ошибке.

Для проверки успешной загрузки модуля ядра `evbug` выполнить команду:

```
/bin/kmod list | grep evbug
```

Вывод после выполнения команды

```
evbug 16384 0
```

3.2.4. Изменение наборов Linux-привилегий ходе выполнения процесса

В ходе выполнения процесса могут быть изменены составы некоторых наборов Linux-привилегий этого процесса, если такая функция заложена в исполняемом коде:

- процесс может удалить Linux-привилегии из своих наборов `permitted` и `inheritable`, но не может добавить новые;
- процесс может исключить Linux-привилегию из набора `effective`, а также добавить Linux-привилегию в этот набор, но только в том случае, если эта Linux-привилегия имеется в наборе `permitted`.

В ходе выполнения процесса также могут быть изменены значения идентификаторов RUID, EUID или SUID этого процесса, если такая функция заложена в исполняемом коде. В этом случае будут проведены следующие изменения наборов Linux-привилегий процесса:

- если значения RUID, EUID или SUID до изменения были равны значению UID пользователя `root`, то есть 0, а после все они отличны от нуля, то для этого

процесса все наборы Linux-привилегий очищаются, то есть из наборов удаляются все Linux-привилегии;

- если значение EUID до изменения было равно значению UID пользователя `root`, то есть 0, а после отлично от нуля, то набор `effective` очищается;
- если значение EUID было отлично от нуля до изменения, а после равно значению UID пользователя `root`, то есть 0, то набор `effective` будет равен набору `permitted`.

Примечание. В исполняемом коде процесса может быть включен запрет очистки наборов Linux-привилегий при изменении значений RUID, EUID или SUID этого процесса.

3.3. Средства управления дискреционными ПРД

Для управления дискреционными ПРД используется графическая утилита `fly-fm` («Менеджер файлов», см. электронную справку).

Для управления Linux-привилегиями пользователей системы используется модуль «Пользователи» в разделе «Пользователи и группы» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_users
```

Описание инструментов управления дискреционными ПРД в режиме командной строки приведено в 3.3.1–3.3.6

3.3.1. `chown`

Инструмент `chown` позволяет изменять пользователя и/или группу, владеющих сущностями.

Синтаксис команды:

```
chown [параметр]... <ВЛАДЕЛЕЦ>[:[ГРУППА]] <ФАЙЛ>...  
chown [параметр]... <:ГРУППА> <ФАЙЛ>...  
chown [параметр]... --reference=RFILE <ФАЙЛ>...
```

Смена пользователя-владельца и/или группы-владельца производится согласно заданным параметрам, интерпретируемым в порядке указания следующим образом:

- 1) если задано только имя пользователя (или его числовой идентификатор), то данный пользователь становится владельцем каждого из указанных файлов, а группа-владелец этих файлов не изменяется;

- 2) если за именем пользователя через двоеточие следует имя группы (или числовой идентификатор группы) без пробелов между ними, то изменяется также и группа-владелец файлов;
- 3) если двоеточие или точка следует за именем пользователя, но группа не задана, то данный пользователь становится владельцем указанных файлов, а группа-владелец указанных файлов изменяется на основную группу пользователя;
- 4) если не указано имя пользователя, а двоеточие или точка вместе с группой заданы, то будет изменена только группа-владелец указанных файлов. В этом случае `chown` выполняет ту же функцию, что и `chgrp` (см. 3.3.2).

Описание основных параметров команды приведено в таблице 2.

Таблица 2

Параметр	Описание
<code>-c, --changes</code>	Подробно описывать только файлы, чей владелец изменяется. Аналогичен параметру <code>--verbose</code>
<code>--dereference</code>	Изменить владельца файла, на который указывает символьная ссылка, вместо самой символьной ссылки
<code>-h, --no-dereference</code>	Работать с самими символьными ссылками, а не с файлами, на которые они указывают. Данный параметр доступен, только если имеется системный вызов <code>lchown</code>
<code>--from=CURRENT_OWNER:CURRENT_GROUP</code>	Изменить владельца и/или группу каждого файла, только если текущий владелец и/или группа совпадает с <code>CURRENT_OWNER:CURRENT_GROUP</code> . Как группа, так и владелец могут не указываться, в этом случае совпадение для данного атрибута не обязательно
<code>-f, --silent, --quiet</code>	Не выводить сообщения об ошибках на файлы, чей владелец не может быть изменен
<code>--reference=RFILE</code>	Установить такого же владельца и группу, как и у файла-образца <code>RFILE</code> . Если <code>RFILE</code> является символьической ссылкой, будут использованы данные файла, на который она ссылается
<code>-R, --recursive</code>	Рекурсивно изменять владельца каталогов и всего их содержимого
<code>-v, --verbose</code>	Подробно описывать только файлы, чей владелец изменяется
<code>--help</code>	Вывести справку и выйти
<code>--version</code>	Вывести информацию о версии и выйти

Примеры:

1. Назначить пользователя `user` и группу `admins` пользователем-владельцем и группой-владельцем всех сущностей в каталоге `/var/www`:

```
sudo chown -R user:admins /var/www
```

2. Назначить группу `group1` группой-владельцем файла `example.txt`, не изменяя пользователя-владельца:

```
sudo chown :group1 example.txt
```

3. Назначить пользователя `user` и его основную группу пользователем-владельцем и группой-владельцем каталога `/usr/bin`:

```
sudo chown user: /usr/bin
```

3.3.2. chgrp

Инструмент `chgrp` изменяет группу, владеющую каждым из указанных файлов `FILE`, на указанную группу. Группа может быть задана именем или ее числовым идентификатором.

Синтаксис команды:

```
chgrp [параметр]... <ГРУППА> <ФАЙЛ>...
chgrp [параметр]... --reference=RFILE <ФАЙЛ>...
```

Описание параметров инструмента `chgrp` приведено в таблице 3.

Таблица 3

Параметр	Описание
<code>-c, --changes</code>	То же, что и <code>--verbose</code> , но выводить сообщение только тогда, когда действительно была изменена группа файла
<code>--dereference</code>	Изменить владельца файла, на который указывает символьная ссылка, вместо самой символьной ссылки
<code>-h, --no-dereference</code>	Изменить владельца символьной ссылки, а не владельца файла, на который указывает эта ссылка (доступна только на системах, имеющих системный вызов <code>lchown</code>)
<code>-f, --silent, --quiet</code>	Не выводить сообщения об ошибках
<code>--reference=RFILE</code>	Установить такую же группу, как и у файла-образца <code>RFILE</code> . Если <code>RFILE</code> является символьической ссылкой, будут использованы данные файла, на который она ссылается
<code>-R, --recursive</code>	Рекурсивно изменять владельца каталогов и их содержимого
<code>-v, --verbose</code>	Подробно описывать изменение владельца для каждого файла
<code>--help</code>	Вывести справку и выйти
<code>--version</code>	Вывести информацию о версии и выйти

Пример

Изменить группу-владельца файла `example.txt` на группу `admins`:

```
sudo chgrp admins example.txt
```

3.3.3. chmod

Инструмент `chmod` применяется для изменения прав доступа к сущностям.

Синтаксис команды:

```
chmod [параметр]... <РЕЖИМ>[,РЕЖИМ]... <ФАЙЛ>...
chmod [параметр]... <ОКТЕТ-РЕЖИМ>... <ФАЙЛ>...
chmod [параметр]... --reference=RFILE <ФАЙЛ>...
```

Описание параметров инструмента `chmod` приведено в таблице 4.

Таблица 4

Параметр	Описание
<code>-c, --changes</code>	То же, что и <code>--verbose</code> , но выводить сообщение только тогда, когда были произведены изменения
<code>-f, --silent, --quiet</code>	Не выдавать сообщения об ошибках на те файлы, чьи права не могут быть изменены
<code>-v, --verbose</code>	Подробно описывать измененные права доступа
<code>--reference=RFILE</code>	Изменить права доступа к файлу на те права, что имеет <code>RFILE</code>
<code>-R, --recursive</code>	Рекурсивное изменение прав доступа для каталогов и их содержимого
<code>--help</code>	Вывести справку и выйти
<code>--version</code>	Вывести информацию о версии и выйти

Инструмент `chmod` изменяет права доступа к сущности `ФАЙЛ` в соответствии с новыми правами доступа, указанными в параметре `РЕЖИМ` или в параметре `ОКТЕТ-РЕЖИМ`. В параметре `РЕЖИМ` права доступа указываются в символьном виде, в параметре `ОКТЕТ-РЕЖИМ` — в восьмеричном виде.

В таблице 5 представлены возможные варианты записи прав пользователя в разных видах.

Таблица 5

Восьмеричная запись	Биты	Символьная запись	Права на файл	Права на каталог
0	000	---	Нет	Нет
1	001	--x	Исполнение	Чтение свойств файлов

Окончание таблицы 5

Восьмеричная запись	Биты	Символьная запись	Права на файл	Права на каталог
2	010	-w-	Запись	Нет
3	011	-wx	Запись и исполнение	Все права, кроме получения имен файлов
4	100	r--	Чтение	Чтение имен файлов
5	101	r-x	Чтение и исполнение	Чтение файлов и их свойств
6	110	rw-	Чтение и запись	Чтение имен файлов
7	111	rwX	Все права	Все права

Для установки с помощью параметра ОКТЕТ-РЕЖИМ комбинированного доступа, например на чтение и запись, необходимо сложить соответствующие цифры, например $4+2=6$. Сумма четырех слагаемых является уникальным числом, позволяющим однозначно определить набор прав доступа.

Права доступа в параметре ОКТЕТ-РЕЖИМ задаются четырьмя восьмеричными цифрами от 0 до 7.

Порядок подстановки цифр прав доступа в команде:

1) первая цифра в параметре ОКТЕТ-РЕЖИМ определяет наличие специальных атрибутов:

- а) бит пользователя SUID задается цифрой 4;
- б) бит группы SGID задается цифрой 2;
- в) Sticky-бит задается цифрой 1.

При комбинировании этих атрибутов значения складываются. Например, для установки Sticky-бита и SGID необходимо использовать цифру 3 (1+2). При их отсутствии указывается цифра 0 или не указывается ничего;

2) вторая цифра определяет права доступа для пользователя, владеющего данной сущностью. Цифра от 0 до 7 определяет точный набор предоставляемых прав;

3) третья цифра определяет права доступа для группы, владеющей сущностью. Права доступа задаются тем же образом, что и для пользователя-владельца сущности;

4) четвертая цифра задает права доступа для всех остальных пользователей, которые не входят в группу-владельца сущности. Права доступа задаются тем же образом, что и для пользователя-владельца сущности.

Пример

Команда:

```
sudo chmod 0755 <имя_файла>
```

идентична команде:

```
sudo chmod 755 <имя_файла>
```

Данные команды задают права:

- специальный бит не установлен;
- пользователю-владельцу (u) назначены права на чтение, запись и исполнение rwx;
- группе-владельцу (g) назначены права на чтение и исполнение r-x;
- остальным (o) назначены права на чтение и исполнение r-x.

В восьмеричном виде файлу или каталогу в явном виде задаются права сразу для всех трех категорий субъектов доступа. В символьном виде можно установить права для отдельных категорий, не затрагивая остальные, а также добавить или отнять права относительно уже имеющихся.

Формат символьного вида параметра РЕЖИМ:

```
[ugoa][[+|=][rwxXstugo]...][, ...]
```

Каждый аргумент — это список символьных команд изменения прав доступа, разделенных запятыми. Каждая такая команда может начинаться с комбинации букв u, g, o или a, которые указывают, чьи права доступа к файлу будут изменены:

- пользователя, владеющего файлом (u);
- группы, владеющей файлом (g);
- остальных пользователей, не входящих в данную группу (o);
- всех пользователей (a).

Буква a эквивалентна одновременному указанию u, g и o. Если не задана ни одна буква, то автоматически будет использоваться буква a, но биты, установленные в umask, не будут затронуты.

Оператор «+» добавляет указанные права доступа к уже имеющимся у каждого файла, оператор «-» удаляет эти права, а оператор «=» заменяет имеющиеся права доступа на указанные.

Описание параметров символьного вида РЕЖИМ приведено в таблице 6.

Таблица 6

Параметр	Описание
u	Пользователь (владелец файла) — от user (пользователь)
g	Группа — от group (группа)
o	Остальные пользователи — от other (остальные)
a	Все пользователи — от all (все)
+	Добавить разрешения к текущим правам доступа
–	Удалить разрешения из текущих прав доступа
=	Установить разрешения вне зависимости от текущих прав доступа

Буквы r, w, x, X, s, t, u, g и/или o, идущие после оператора «+», «-» или «=», устанавливают новые права доступа для субъекта доступа, заданного одной из букв u, g, o или a. Описание параметров символьной записи приведено в таблице 7.

Таблица 7

Параметр	Описание
r	Чтение начинается с нуля или более букв
w	Запись
x	Исполнение
X	Исполнение, только если файл является каталогом или уже имеет право на исполнение для какого-нибудь пользователя
s	SUID или SGID-биты
t	Sticky-бит
u	Установка таких же прав доступа, как и у пользователя-владельца
g	Установка таких же прав доступа, как и у группы-владельца
o	Установка таких же прав доступа, которые имеют остальные пользователи (не входящие в группу-владельца файла)

Примеры:

1. Снять бит SGID:

```
sudo chmod g-s file
```

2. Установить биты SUID и SGID:

```
sudo chmod ug+s file
```

3. Команда не выполнит никаких действий (противоречивый набор параметров):

```
sudo chmod o+s file
```

Инструмент `chmod` не изменяет права на символьные ссылки, т. к. этого не делает системный вызов `chmod` (права символьных ссылок в системе не используются). Если в параметрах командной строки указаны символьные ссылки, то `chmod` изменит права сущностей, на которые они ссылаются. В то же время символьные ссылки, встречающиеся при рекурсивной обработке каталогов, игнорируются.

3.3.4. `umask`

Инструмент `umask` представляет собой маски режима создания файлов.

Синтаксис команды:

```
sudo umask [-p] [-S] [маска]
```

Пользовательская маска создания файла устанавливается равной значению `маска`. Если маска начинается с цифры, она интерпретируется как восьмеричное число, иначе — как маска в символьном формате, аналогичном используемому в команде `chmod` (см. 3.3.3).

Если маска не указана, то по умолчанию выводится текущее значение маски в восьмеричном формате. Если задан параметр `-S`, то выводится текущее значение маски в символьном формате.

Если указан параметр `-p`, а маска не задана, то результат выдается в виде, который можно использовать во входной команде.

Если маска была успешно изменена или не указана, команда завершается со статусом 0, в противном случае — со статусом 1.

Команды инструмента `umask` распознаются и выполняются оболочкой `shell`.

Команду инструмента `umask` целесообразно включить в пользовательский файл профиля `~/.profile`. Тогда она будет автоматически вызываться при входе в систему и устанавливать нужный режим доступа к создаваемым файлам и каталогам.

3.3.5. `getfacl`

Инструмент `getfacl` служит для отображения списков контроля доступа сущностей. Для каждого файла `getfacl` выводит имя файла, пользователя-владельца, группу-владельца и ACL. Если каталог имеет ACL по умолчанию, то `getfacl` выводит также ACL по умолчанию. Файлы не могут иметь ACL по умолчанию.

Синтаксис команды:

```
getfacl [-dRLP] <файл> ...
```

Формат вывода:

```
1: # file: somedir/
2: # owner: lisa
3: # group: staff
4: user::rwx
5: user:joe:rwx           #effective:r-x
6: group::rwx            #effective:r-x
7: group:cool:r-x
8: mask:r-x
9: other:r-x
10: default:user::rwx
11: default:user:joe:rwx   #effective:r-x
12: default:group::r-x
13: default:mask:r-x
14: default:other:---
```

Строки 4, 6 и 9 относятся к битам стандартных прав доступа к файлу, соответственно, прав пользователя-владельца, группы-владельца и всех остальных. Они представляют собой записи базового ACL. Строки 5 и 7 являются записями ACL для именованных пользователей и группы. Строка 8 — маска эффективных прав. Если маска задана в явном виде, она ограничивает права, предоставляемые всем группам и именованным пользователям (итоговые эффективные права отображаются с отметкой `#effective`). Если права для именованных пользователей и/или групп заданы без явного указания маски, она отображается равной сумме их прав доступа и ни на что не влияет. В таком случае отметка `#effective` напротив прав именованных пользователей и групп отсутствует. Маска не влияет на права для пользователя-владельца файла и всех других. Строки 10–14 показывают ACL по умолчанию, ассоциированный с данным каталогом.

Для двух и более файлов `getfacl` выводит ACL, разделенные для удобства пустыми строками. Вывод инструмента `getfacl` может использоваться как входные данные для инструмента `setfacl` (см. 3.3.6).

Описание параметров инструмента приведено в таблице 8.

Таблица 8

Параметр	Описание
<code>--access</code>	Вывести только ACL файла
<code>-d, --default</code>	Вывести только ACL по умолчанию
<code>--omit-header</code>	Не показывать заголовков (имя файла)

Окончание таблицы 8

Параметр	Описание
--all-effective	Показать все эффективные права
--no-effective	Не показывать эффективные права
--skip-base	Пропускать файлы, имеющие только основные записи
-R, --recursive	Для подкаталогов рекурсивно
-L, --logical	Следовать по символическим ссылкам, по умолчанию символические ссылки, не указанные в командной строке, игнорируются
-P, --physical	Не следовать по символическим ссылкам, даже если они указаны в командной строке
--tabular	Использовать табулированный формат вывода
--numeric	Показывать числовые значения пользователя/группы
--absolute-names	Не удалять ведущие «/» из пути файла
--help	Вывести справку и выйти
--version	Вывести информацию о версии и выйти

3.3.6. setfacl

Инструмент `setfacl` вносит изменения в ACL файлов и каталогов.

Синтаксис команды:

```
setfacl [-bkndRLP] [-mMxX ... ] <файл> ...
```

В команде после параметров можно указать несколько файлов и/или каталогов, после которых также возможно указать новые параметры и относящиеся к ним файлы и/или каталоги. Подобных блоков, состоящих из параметров и файлов/каталогов, может быть указано любое количество.

Описание параметров инструмента приведено в таблице 9.

Таблица 9

Параметр	Описание
-m, --modify=acl	Изменить текущий ACL для файла
-M, --modify-file=file	Прочитать записи ACL для модификации из файла
-x, --remove=acl	Удалить записи из ACL файла
-X, --remove-file=file	Прочитать записи ACL для удаления из файла
-b, --remove-all	Удалить все расширенные записи ACL
-k, --remove-default	Удалить ACL по умолчанию
--set=acl	Установить ACL для файла, заменив текущий ACL

Окончание таблицы 9

Параметр	Описание
<code>--set-file=file</code>	Прочитать записи ACL для установления из файла
<code>--mask</code>	Пересчитать маску эффективных прав
<code>-n, --no-mask</code>	Не пересчитывать маску эффективных прав. Обычно <code>setfacl</code> пересчитывает маску (если она не была задана явно) и устанавливает ее равной сумме всех прав группы-владельца, именованных пользователей и групп (т.е. всех субъектов, на права которых она влияет)
<code>-d, --default</code>	Применить ACL по умолчанию
<code>-R, --recursive</code>	Для подкаталогов рекурсивно
<code>-L, --logical</code>	Следовать по символическим ссылкам, по умолчанию ссылки, не указанные в командной строке, игнорируются
<code>-P, --physical</code>	Не следовать по символическим ссылкам, даже если они указаны в командной строке
<code>--restore=file</code>	Восстановить резервную копию прав доступа, созданную командой: <code>getfacl -R</code> или подобной. Все права доступа дерева каталогов восстанавливаются, используя этот механизм. Если вводимые данные содержат записи для пользователя-владельца или группы-владельца и инструмент <code>setfacl</code> запускается с правами <code>root</code> , то пользователь-владелец и группа-владелец всех файлов также восстанавливаются. Этот параметр не может использоваться совместно с другими параметрами, за исключением параметра <code>--test</code>
<code>--test</code>	Режим тестирования (ACL не изменяются)
<code>--help</code>	Вывести справку и выйти
<code>--version</code>	Вывести информацию о версии и выйти

При использовании параметров `--set`, `-m` и `-x` должны быть перечислены записи ACL в командной строке. Записи ACL разделяются запятыми.

При чтении ACL из файла при помощи параметров `--set-file`, `-M` и `-X` инструмент `setfacl` принимает множество записей в формате вывода инструмента `getfacl`. В строке обычно содержится только одна запись ACL.

Инструмент `setfacl` использует следующие форматы записей ACL:

1) права доступа отдельного пользователя:

```
[d:] [u:]uid [:perms]
```

Если не задан `uid`, то устанавливаются права доступа пользователя-владельца файла;

2) права доступа отдельной группы:

```
[d:] g:gid [:perms]
```

Если не задан `gid`, то устанавливаются права доступа группы-владельца;

3) маска эффективных прав:

```
[d:] m[:] [:perms]
```

4) права доступа всех остальных:

```
[d:] o[:] [:perms]
```

Значения `uid` и `gid` задаются именем или числом. Параметр `perms` может быть представлен в символьном (комбинация `r`, `w`, `x`, `-`) или восьмеричном (цифры от 0 до 7) виде.

ВНИМАНИЕ! При перечислении записей ACL маска всегда указывается последней.

При создании прав доступа инструментом `setfacl` выполняются следующие правила:

- 1) если права доступа для именованного пользователя и/или группы добавлены в ACL, а маски прав не существует, то создается маска с правами доступа группы-владельца;
- 2) если создана запись ACL по умолчанию, а трех базовых записей не было, тогда делается их копия и они добавляются в ACL по умолчанию;
- 3) если ACL по умолчанию содержит какие-либо права доступа для именованного пользователя и/или группы и не содержит маску прав доступа по умолчанию, то при создании эта маска будет иметь те же права, что и группа по умолчанию.

Примеры:

1. Задать в ACL файла `1.txt` права `rwX` для именованного пользователя `steeve`, права `-w-` для именованной группы `admins` и права `r-x` для группы-владельца файла:

```
sudo setfacl -m u:steeve:rwX,g:admins:w,g::r-x 1.txt
```

Результат выполнения команды:

```
getfacl 1.txt:
```

```
# file: 1.txt
# owner: user
# group: user
user::rw-
user:steeve:rwX
group::r-x
group:admins:-w-
mask::rwX
```

```
other::r-x
```

2. Задать в ACL каталога `reports` и всех его подкаталогов права `r-x` для пользователя-владельца и маску `rw` для ограничения максимальных прав группы-владельца и именованных пользователей/групп, заданных ранее (они, в итоге, не получают доступ на исполнение, так как маска разрешает только чтение и запись):

```
sudo setfacl -R -m u::r-x,m:rw reports
```

Результат выполнения команды:

```
getfacl reports:

# file: reports
# owner: user
# group: user
user::r-x
user:steeve:rw #effective:rw-
group::r-x #effective:r--
mask::rw-
other::r-x
```

3.4. Особенности управления дискреционными ПРД в NFSv4.x

В протоколе NFSv4.x реализована поддержка списков контроля доступа (ACL), которые похожи на списки контроля доступа, используемые в ОС. При этом ACL в NFSv4.x могут содержать отрицательные права (отказ в доступе), имеют больше типов разрешений и менее строгую структуру наследования прав.

Для управления ACL, реализованных в NFSv4.x, в ОС должен быть установлен пакет `nfs4-acl-tools`.

ACL в NFSv4.x представляет собой список ПРД, каждое из которых расположено на отдельной строке и имеет следующий вид:

```
<тип>:<флаги>:<пользователь/группа>:<разрешения>
```

где <тип> — тип ПРД. Возможные значения:

- A — предоставить пользователю/группе права доступа, определенные в параметре <разрешения>;
- D — запретить пользователю/группе выполнять операции, указанные в параметре <разрешения>;

- U — сохранять в системные журналы сведения о каждой попытке пользователя или группы выполнить операции, указанные в параметре <разрешения>;
- L — формировать системное сообщение о каждой попытке пользователя/группы выполнить операции, указанные в параметре <разрешения>;

<флаги> — последовательность буквенных символов, определяющих дополнительный контекст для ПРД. Существуют следующие типы флагов: группирование, наследование и администрирование. Дополнительные сведения о флагах представлены на справочной странице `man nfs4_acl`;

<пользователь/группа> — параметр, определяющий пользователя или группу, в отношении которых будут применены ПРД. В качестве значения параметра может быть указано имя пользователя домена или наименование группы пользователей домена в формате <имя>@<наименование_домена>. Кроме того, параметр может принимать одно из следующих значений:

- OWNER@ — пользователь-владелец;
- GROUP@ — группа-владелец;
- EVERYONE@ — все пользователи.

Примечание. Если ПРД применяется для группы, то в параметре <флаги> следует дополнительно указать символ `g`.

<разрешения> — последовательность буквенных символов, определяющих права доступа. Каждый вид операций определяется одной буквой латинского алфавита (например, `r` — чтение, `d` — удаление и т. д.). Дополнительные сведения о формате записи ПРД в NFSv4.x представлены на справочной странице `man nfs4_acl`.

Просмотреть ПРД, назначенные файлам и каталогам, можно с помощью инструмента `nfs4_getfacl` (см. 3.4.1). Изменение ПРД выполняется инструментом `nfs4_setfacl` (см. 3.4.2).

3.4.1. `nfs4_getfacl`

Инструмент `nfs4_getfacl` служит для отображения ПРД, назначенных файлам и каталогам, общий доступ к которым предоставлен с использованием протокола NFSv4.x. Для каждой сущности (файла или каталога) инструмент `nfs4_getfacl` выводит наименование этой сущности и назначенные ей ПРД.

Синтаксис команды:

```
nfs4_getfacl [параметры] <файл>
```

Пример

Вывести ПРД, назначенные каталогу `/mnt/share/`:

```
nfs4_getfacl /mnt/share/
```

Вывод команды:

```
# file: /mnt/share/
A::OWNER@:rwaDxtTnNcCy
A:g:GROUP@:rwaDxtTnNcy
A::EVERYONE@:tcy
```

В представленном примере:

- пользователь-владелец имеет полный доступ;
- группа-владелец имеет полный доступ;
- у всех остальных нет доступа.

В команде после параметров можно указать несколько файлов и/или каталогов, после которых также возможно указать новые параметры и относящиеся к ним файлы и/или каталоги. Подобных блоков, состоящих из параметров и файлов/каталогов, может быть указано любое количество.

Описание параметров инструмента `nfs4_getfacl` приведено в таблице 10.

Таблица 10

Параметр	Описание
<code>-R, --recursive</code>	Выполнить команду рекурсивно (включая подкаталоги и файлы)
<code>-H, --more-help</code>	Вывести сведения о формате записи ПРД и выйти
<code>-h, --help</code>	Вывести справку и выйти
<code>-c, --omit-header</code>	Не показывать заголовок (имя файла)
<code>--sACL</code>	Вывести системный список управления доступом (SACL — System Access Control List). Доступно только при использовании протокола NFSv4.1 и выше
<code>--dACL</code>	Вывести список управления доступом на уровне пользователей (DACL — Discretionary Access Control List). Доступно только при использовании протокола NFSv4.1 и выше

Более подробное описание инструмента `nfs4_getfacl` приведено на справочной странице `man nfs4_getfacl`.

3.4.2. nfs4_setfacl

Инструмент `nfs4_setfacl` вносит изменения в ПРД, назначенные файлам и каталогам, общий доступ к которым предоставлен с использованием протокола NFSv4.x.

Синтаксис команды:

```
nfs4_setfacl [--test] [-RLP] <параметры> <файл>
```

В команде можно указать несколько ПРД, разделенных запятыми. После параметров можно указать несколько файлов и/или каталогов, для которых будут применены указанные параметры.

Описание параметров инструмента `nfs4_setfacl` приведено в таблице 11.

Таблица 11

Параметр	Описание
<code>--test</code>	Режим тестирования (ACL не изменяется)
<code>-R, --recursive</code>	Выполнить команду рекурсивно (включая подкаталоги и файлы)
<code>-L, --logical</code>	Следовать по символическим ссылкам. По умолчанию ссылки, не указанные в командной строке, игнорируются
<code>-P, --physical</code>	Не следовать по символическим ссылкам, даже если они указаны в командной строке
<code>-a <ПРД></code>	В ACL добавить указанное ПРД. По умолчанию ПРД добавляется в первую строку списка контроля доступа с последующим сдвигом имеющихся строк. Для указания конкретного номера строки добавляемого ПРД используется параметр <code>-i</code> (с последующим сдвигом имеющихся строк). Примечание. Можно указать несколько ПРД, разделенных запятыми
<code>-A <файл_ПРД></code>	В ACL добавить ПРД, приведенное в указанном файле. По умолчанию ПРД добавляется в первую строку списка контроля доступа с последующим сдвигом имеющихся строк. Для указания конкретного номера строки добавляемого ПРД используется параметр <code>-i</code> (с последующим сдвигом имеющихся строк)
<code>-i <номер_строки></code>	Задать номер строки. Используется только при добавлении или удалении ПРД. При добавлении ПРД запись размещается на указанной строке (с последующим сдвигом имеющихся строк). При удалении — исключается указанная строка (с последующим сдвигом имеющихся строк)
<code>-x <ПРД></code>	Из ACL удалить указанное ПРД (с последующим сдвигом имеющихся строк). Примечания: 1. Можно указать несколько ПРД, разделенных запятыми. ПРД могут быть указаны в команде в произвольном порядке. Поиск ПРД для удаления ведется независимо от очередности размещения. 2. Вместо ПРД можно указать непосредственно номер строки (используется параметр <code>-i</code>)

Окончание таблицы 11

Параметр	Описание
-X <файл_ПРД>	Из ACL удалить ПРД, приведенное в указанном файле (с последующим сдвигом имеющихся строк)
-s <ACL>	Установить указанный ACL (заменяв текущий ACL)
-S <файл_ACL>	Установить ACL, приведенный в указанном файле (заменяв текущий ACL)
-m <имеющееся_ПРД> <новое_ПРД>	Заменить имеющееся ПРД на новое
-e, --edit	Открыть текстовый редактор для изменения ACL. Если в команде указано несколько файлов и/или каталогов, то для каждого из них будет последовательно открыт текстовый редактор
-, -h, --help	Вывести справку и выйти
--version	Вывести информацию о версии и выйти
--sACL	Вносить изменения только в системный список управления доступом (SACL — System Access Control List). Доступно только при использовании протокола NFSv4.1 и выше
--dACL	Вносить изменения только в список управления доступом на уровне пользователей (DACL — Discretionary Access Control List). Доступно только при использовании протокола NFSv4.1 и выше

Примечания:

1. При использовании параметров -A, -X и -S может использоваться стандартный ввод (stdin). В этом случае в параметре вместо имени файла следует указать символ «-».
2. При указании прав доступа в ПРД могут использоваться псевдонимы (alias):
 - символ R соответствует праву доступа вида rntcy (право доступа на чтение, установленное в ОС);
 - символ W соответствует праву доступа вида watTNcCy для файла и watTNcCyD для каталога (право доступа на запись, установленное в ОС);
 - символ X соответствует праву доступа вида xtcy (право доступа на исполнение, установленное в ОС).

Более подробное описание инструмента `nfs4_getfacl` приведено на справочной странице `man nfs4_getfacl`.

Примеры:

1. Задать в ACL файла `1.txt` права доступа на чтение и исполнение для именованного пользователя `astra-user@local.ru`, право доступа на запись для именованной группы `astra-group@local.ru`:

```
nfs4_setfacl -a A::astra-user@local.ru:rxtncy, \
```



```
A:g:astra-group@local.ru:watTNcCy 1.txt
```

2. Задать права доступа, перечисленные в примере выше, но с использованием псевдонимов:

```
nfs4_setfacl -a A::astra-user@local.ru:RX, \  
A:g:astra-group@local.ru:W 1.txt
```

3. Для файлов 2.txt и 3.txt установить такие же ACL, как и для файла 1.txt:

```
nfs4_getfacl 1.txt | nfs4_setfacl -S - 2.txt 3.txt
```

4. МАНДАТНОЕ УПРАВЛЕНИЕ ДОСТУПОМ И МАНДАТНЫЙ КОНТРОЛЬ ЦЕЛОСТНОСТИ

4.1. Общие сведения

Мандатное управление доступом и мандатный контроль целостности (МКЦ) реализованы в ядре ОС и затрагивают следующие подсистемы:

- механизмы IPC;
- стек TCP/IP (IPv4, IPv6);
- ФС ext2/ext3/ext4/XFS;
- сетевые ФС CIFS, OCFS2, Ceph;
- ФС proc, tmpfs.

4.2. Мандатное управление доступом

При реализации политики мандатного управления доступом субъектам и сущностям присваиваются классификационные метки (уровни конфиденциальности и категории конфиденциальности, описание которых приведено в 4.2.1 и 4.2.2 соответственно).

Также могут быть присвоены дополнительные атрибуты сущностей для мандатного управления доступом, описание которых приведено в 4.2.3.

Для администрирования подсистем с мандатным управлением доступом множество привилегий Linux расширено специальными PARSEC-привилегиями, полное описание которых приведено в 4.7.

Уровни и категории конфиденциальности, используемые в ОС по умолчанию, приведены в 17.1.

4.2.1. Уровень конфиденциальности

Иерархический уровень конфиденциальности (уровень конфиденциальности) определяет степень секретности сущности и соответствующий уровень доступа к ней, назначенный субъекту.

Уровень конфиденциальности представляет собой числовое значение от 0 до 255 (включительно). Каждой классификационной метке в каждый момент времени может быть назначен только один уровень конфиденциальности.

Числовые значения уровня конфиденциальности сравнимы между собой и технически реализованы как 8-битная беззнаковая величина (`uint8_t`). В пользовательских интерфейсах представляется десятичным значением или наименованием.

Описание применения правил мандатного управления доступом на основе результатов сравнения уровней конфиденциальности субъекта и сущности приведено в 4.6.

4.2.2. Категория конфиденциальности

Множество неиерархических категорий конфиденциальности (категории конфиденциальности) определяет набор категорий, доступных субъекту или назначенных сущности.

Категории конфиденциальности представляют собой битовую маску, состоящую из набора категорий конфиденциальности, каждая из которых представляется в виде отдельного разряда битовой маски. Значение разряда битовой маски, равное 1, означает наличие соответствующей категории конфиденциальности у сущности (субъекта), а равное 0 — отсутствие данной категории.

В ОС реализовано использование до 64 категорий конфиденциальности. Каждой классификационной метке в каждый момент времени может быть присвоен единственный набор категорий конфиденциальности. Набор категорий конфиденциальности может принимать значения от 0 до 0xFFFF FFFF FFFF FFFF (включительно), технически реализован как 64-битная маска, беззнаковая величина (`uint64_t`).

Категории конфиденциальности могут быть несравнимы между собой. В пользовательских интерфейсах представляются шестнадцатеричным значением или списком наименований категорий. Числовое значение конкретной категории конфиденциальности является маской одного бита и может быть представлено степенью числа 2, например 2, 4, 8.

С помощью инструмента командной строки `astra-singlecat-control` администратор может запретить пользователям входить в систему с несколькими категориями конфиденциальности. Описание инструмента приведено в 16.6.37.

Описание применения правил мандатного управления доступом на основе результатов сравнения категорий конфиденциальности субъекта и сущности приведено в 4.6.

4.2.3. Дополнительные атрибуты сущностей для мандатного управления доступом

Дополнительные атрибуты сущностей для мандатного управления доступом позволяют уточнять или изменять правила мандатного управления доступом для тех или иных сущностей:

- `ccnr` — присваивается сущностям-контейнерам. Определяет, что сущность-контейнер может содержать сущности с различными классификационными метками, но не большими, чем ее собственная классификационная метка. Чтение содержимого такой сущности-контейнера разрешается субъекту вне зависимости от значения его классификационной метки, при этом субъекту доступна информация только про находящиеся в ней другие сущности с классификационной меткой не большей, чем его собственная классификационная метка, либо про сущности-контейнеры, также

имеющие атрибут `ccnr`;

- `ehole` — присваивается объектам (файлам), имеющим минимальную классификационную метку и нулевую метку целостности. Приводит к игнорированию мандатных правил управления доступом при получении доступа на запись к данным объекта. Атрибут предназначен для объектов, из которых субъект не может прочитать данные, записанные в них субъектами с более высокой классификационной меткой, чем его собственная классификационная метка (например, `/dev/null`);
- `whole` — присваивается объектам (файлам), имеющим максимальную классификационную метку. Разрешает запись в них субъектам, имеющим более низкую классификационную метку (в обычном случае записывать «снизу вверх» запрещено).

Примечание. Атрибут `ccnri` более не используется в ОС для управления доступом, при этом штатное функционирование ОС соответствует функционированию с установленным атрибутом `ccnri`. Возможность установки и получения данного атрибута сохранена в ОС для обеспечения совместимости с системами и ПО, которые его используют. Атрибут `ccnra` в ОС приравнивается к атрибуту `ccnr` и также сохранен для обеспечения совместимости и не рекомендован к использованию.

Дополнительные атрибуты сущностей для мандатного управления доступом могут быть установлены (сняты) только субъектом с наличием привилегии `PARSEC_CAP_CNMAC` (см. 4.7) или субъектом с привилегией `root` и с максимальной меткой целостности (такой субъект обладает привилегией `PARSEC_CAP_CNMAC` по умолчанию).

4.3. Мандатный контроль целостности

При реализации политики мандатного контроля целостности субъектам и сущностям задаются метки целостности: неиерархический уровень (категория) целостности и иерархический (линейный) уровень целостности. Описание метки целостности приведено в 4.3.1.

Также сущностям могут быть присвоены дополнительные атрибуты для мандатного контроля целостности, описание которых приведено в 4.3.2.

Для администрирования подсистем с мандатным контролем целостности множество привилегий Linux расширено специальными PARSEC-привилегиями, полное описание которых приведено в 4.7.

4.3.1. Метка целостности

Субъектам и сущностям задаются метки целостности, каждая из которых представляет собой пару: неиерархический уровень целостности (категория целостности) и иерархический уровень целостности (линейный уровень целостности).

Метка целостности сущности отражает степень критичности для программной среды ОС факта изменения этой сущности и, соответственно, степень защищенности сущности от изменений субъектами. Чем больше метка целостности сущности, тем более защищена сущность от изменений субъектами.

Метка целостности субъекта соответствует его полномочиям по доступу к сущности. Чем больше метка целостности у субъекта (пользователя или процесса), тем большими возможностями по изменению сущностей он обладает.

Категория целостности технически реализована как 32-битная маска, беззнаковая величина (`uint32_t`). В пользовательских интерфейсах представляется десятичным или шестнадцатеричным числом или наименованием.

В ОС по умолчанию выделены нулевая категория целостности, четыре ненулевых и несравнимых между собой (далее — изолированных) категорий целостности, а также максимальная категория целостности, которая не меньше всех остальных в системе.

При установке ОС по умолчанию предлагается максимальная категория целостности `max_ilev`, равная 63 (битовая маска 00111111), и минимальная категория целостности, равная нулю.

Линейный уровень целостности реализован в виде числа со знаком длиной 1 байт и диапазоном значений от -128 до 127. Значение по умолчанию равно 0. Отрицательные значения введены для того, чтобы субъекты с линейным уровнем целостности 0 могли запускать недоверенные процессы с линейным уровнем целостности ниже собственного (отрицательным). В отличие от категорий целостности, которые задаются битовой маской и могут быть несравнимы между собой, любые два линейных уровня целостности сравнимы между собой, так как задаются целым числом.

В графическом интерфейсе системы и в консоли категория целостности обозначается термином «Уровень целостности» с двумя зарезервированными значениями: Высокий (High) для максимальной категории целостности и Низкий (Low) для минимальной категории целостности.

Метка целостности может быть назначена пользователю или группе пользователей. Метка целостности пользователя указывается в `/etc/parsec/micdb` (для локальной учетной записи) или в базе FreeIPA (для доменной учетной записи). Метка целостности для группы пользователей (локальной или доменной) указывается в `/etc/parsec/micgrdb`.

ВНИМАНИЕ! Если пользователю назначена метка целостности, то она имеет приоритет над меткой целостности группы, в которую входит данный пользователь (т. е. метка целостности группы в таком случае не применяется).

Если пользователю не назначена метка целостности, то при входе в сессию ему присваивается метка целостности группы, в которую он входит.

На одну группу может быть назначена только одна метка целостности, при этом пользователь может состоять в нескольких группах. В таком случае при входе пользователя в сессию вычисляется его эффективная метка целостности на основе меток целостности всех групп, в которых он состоит. Эффективная категория целостности представляет собой битовую маску, в которой каждый бит устанавливается равным 1, если он равен 1 в категории целостности хотя бы одной из групп (побитовое «ИЛИ»). Если в категориях целостности всех групп бит равен нулю, то в эффективной категории целостности он также устанавливается равным нулю. Линейный уровень целостности всегда принимается равным нулю.

Непривилегированным пользователям, которым явно не назначена метка целостности и у которых отсутствует метка целостности группы, неявно присваивается нулевая метка целостности.

Администратору, создаваемому при установке ОС, присваивается максимальная категория целостности 63.

За системными службами, перечень и описание которых приведены в таблице 12, зарезервированы четыре изолированные категории целостности.

Т а б л и ц а 12

Категория	Значение	Битовая маска	Описание
1	001	0000 0001	Категория задействована для сетевых служб
2	002	0000 0010	Категория задействована для виртуализации
3	004	0000 0100	Категория задействована для специального ПО
4	008	0000 1000	Категория задействована для графического сервера

После установки ОС максимальная категория целостности в системе может быть повышена. Максимальными категориями целостности в системе могут быть числа, у которых битовая маска включает битовые маски всех остальных используемых категорий целостности в системе, например, 63 (0x3F, битовая маска 00111111), 127 (0x7F, битовая маска 01111111), 191 (0xBF, битовая маска 10111111) и т. д.

П р и м е ч а н и е. В текущей реализации, с учетом 32-битной маски, количество изолированных категорий целостности может быть увеличено до 32 при повышении максимальной категории целостности до 0xFFFF FFFF.

ВНИМАНИЕ! Перед повышением максимальной категории целостности выше значения 63, заданного при установке ОС, необходимо назначить новую максимальную категорию целостности администратору ОС. Это можно сделать, указав в файле /etc/parsec/micdb/<UID_администратора> новое значение максимальной категории целостности. Иначе администрирование ОС будет недоступно.

Порядок повышения категории целостности приведен в 17.1.

Описание применения правил мандатного контроля целостности на основе результатов сравнения меток целостности субъекта и сущности приведено в 4.6.

4.3.2. Дополнительные атрибуты сущностей для мандатного контроля целостности

Дополнительные атрибуты сущностей для МКЦ позволяют уточнять или изменять правила МКЦ для тех или иных сущностей:

- `silev` (числовое значение 0x10) — присваивается исполняемым бинарным файлам. Данный атрибут позволяет пользователю с низкой меткой целостности, который не является суперпользователем (`root, uid=0`), запускать процесс с высокой меткой целостности из файла с высокой меткой целостности. При запуске процесса из файла с атрибутом `silev` процессу присваивается метка целостности, равная наибольшему значению, которое одновременно меньше или равно значению метки целостности файла и значению максимальной метки целостности в системе (параметр командной строки ядра `parsec.max_ilev` и нулевой линейный уровень целостности). Например, если метка целостности файла меньше или равна максимальной метке целостности в системе, то процессу назначается метка целостности, равная метке целостности файла. В частности, атрибут `silev` нужен, чтобы пользователь с низкой меткой целостности мог запустить из файла `/usr/bin/passwd`, имеющего высокую метку целостности, процесс смены пароля с высокой меткой целостности.

ВНИМАНИЕ! Использовать атрибут рекомендуется в исключительных случаях в соответствии с принятой политикой безопасности;

- `irelax` (числовое значение 0x20) — присваивается каталогам. Данный атрибут определяет, что в каталог может осуществлять запись процесс с любой меткой целостности. Создаваемым в данном каталоге сущностям назначается метка целостности, равная наибольшему значению, которое одновременно меньше или равно значениям метки целостности создающего их процесса и метки целостности данного каталога (результатирующая метка целостности вычисляется как совпадающие биты категорий целостности и минимальное значение из линейных уровней целостности каталога и процесса). Атрибут применяется при включенном расширенном режиме МКЦ (см. 4.5);
- `inh` (числовое значение 0x80) — присваивается каталогам. Данный атрибут позволяет создаваемым в каталоге сущностям наследовать метку целостности родительского каталога (создаваемым в нем сущностям присваивается метка целостности каталога), при этом создаваемые в данном каталоге дочерние каталоги также наследуют атрибут `inh`. Метка целостности создающего процесса должна быть не меньше метки целостности каталога. Атрибут применяется при включенном расширенном режиме МКЦ (см. 4.5).

ВНИМАНИЕ! Если каталогу одновременно присвоены атрибуты `inh` и `irelax`, то создаваемым в нем сущностям присваивается метка целостности по правилам работы атрибута `irelax`, т.к. `irelax` обладает приоритетом над `inh`;

- `ssi` (числовое значение 0x40) — присваивается файлам и каталогам для ограничения доступа на чтение и выполнение. При наличии данного атрибута у сущности доступ к ней на чтение и выполнение разрешается только субъектам (процессам) с метками целостности не ниже, чем у данной сущности. При этом если исполняемому бинарному файлу также присвоен атрибут `silev`, то ограничение доступа на выполнения к данному файлу не применяется.

Дополнительный атрибут сущностей для МКЦ `silev` может быть установлен или снят только субъектом с правами `root` и с максимальной меткой целостности.

Дополнительные атрибуты сущностей для МКЦ `irelax`, `ssi` и `inh` могут быть установлены или сняты субъектом с меткой целостности не ниже, чем у соответствующих сущностей.

4.4. Метка безопасности и мандатный контекст безопасности

Метка безопасности состоит из классификационной метки и метки целостности:

- 1) классификационная метка, определяется:
 - а) иерархическим уровнем конфиденциальности;
 - б) неиерархическими категориями конфиденциальности;
- 2) метка целостности, определяется:
 - а) иерархическим (линейным) уровнем целостности;
 - б) неиерархическим уровнем (категорией) целостности.

Мандатный контекст безопасности субъекта определяется его меткой безопасности совместно с назначенными ему привилегиями.

Мандатный контекст безопасности сущности определяется меткой безопасности сущности совместно с присвоенными ей дополнительными атрибутами сущностей для мандатного управления доступом и для МКЦ.

Субъекты или сущности, которым явно не задан мандатный контекст безопасности, считаются имеющими нулевой мандатный контекст безопасности, т.е. имеют равный нулю уровень конфиденциальности, отсутствуют категории конфиденциальности, имеют равный нулю линейный уровень целостности и равную нулю категорию целостности, отсутствуют привилегии и дополнительные атрибуты.

Порядок присвоения мандатного контекста безопасности и правила принятия решения о предоставлении доступа на его основе описаны в 4.6.

4.5. Расширенный режим мандатного контроля целостности

При включенном МКЦ (без расширенного режима МКЦ) процесс при его непосредственном запуске наследует метку целостности процесса-родителя (процесс наследует метку целостности запустившего его пользователя).

В расширенном режиме МКЦ (strict mode) непосредственный запуск процесса запрещен в том случае, если исполняемый файл, из которого запускается процесс, имеет метку целостности меньше или несравнимую с меткой целостности процесса-родителя. В этом случае процесс возможно запустить только с использованием инструмента `sumic`, описание которого приведено в 4.16.10.

При создании сущности ей назначается нулевая категория целостности и линейный уровень целостности, равный минимальному значению из линейного уровня целостности родительского каталога и нуля.

При наличии у родительского каталога атрибутов `irelax` и/или `iinh` для назначения метки целостности создаваемой сущности используются правила, описанные в 4.3.2.

Если процесс, создающий сущность, обладает привилегией `PARSEC_CAP_INHERIT_INTEGRITY` (см. 4.7), то создаваемой сущности назначается метка целостности, равная метке целостности родительского каталога. При этом запрещено создавать сущность с меткой целостности выше или несравнимой с меткой целостности процесса, создающего данную сущность.

ВНИМАНИЕ! Если родительскому каталогу присвоен атрибут `irelax`, то создаваемым в нем сущностям присваивается метка целостности по правилам работы атрибута `irelax` (см. 4.3.2), т.к. `irelax` обладает приоритетом над привилегией `PARSEC_CAP_INHERIT_INTEGRITY`.

ВНИМАНИЕ! Расширенный режим МКЦ предназначен для усиления защиты ОС. Включение расширенного режима МКЦ необратимо, после включения он не может быть выключен. Вместе с тем его включение может привести к сбоям в работе некоторого прикладного ПО (особенно уже установленного). Поэтому перед включением расширенного режима МКЦ в ОС администратору рекомендуется провести тестирование используемого прикладного ПО с целью проверки его работоспособности при включенном расширенном режиме МКЦ и необходимости его дополнительной настройки.

Включение расширенного режима МКЦ осуществляется запуском от имени администратора команды:

```
astra-strictmode-control enable
```

Выключение расширенного режима МКЦ не предусмотрено.

После включения расширенного режима МКЦ будут выполнены необходимые настройки метки целостности сущностей, для параметра командной строки ядра `parsec.strict_mode` будет установлено значение 1. Для активации расширенного режима МКЦ необходимо перезагрузить ОС.

Для проверки текущего состояния расширенного режима МКЦ (активен/неактивен) можно воспользоваться командой:

```
astra-strictmode-control status
```

Если после выполнения команды включения расширенного режима МКЦ не выполнялась перезагрузка ОС, то результат команды проверки состояния режима будет НЕАКТИВНО. Для получения информации о состоянии расширенного режима МКЦ, которое будет после перезагрузки ОС, выполнить команду:

```
astra-strictmode-control is-enabled
```

При включении расширенного режима МКЦ домашним каталогам (и их содержимому) пользователей, для которых задана ненулевая метка целостности, присваивается метка целостности, соответствующая максимально доступной метке целостности для данного пользователя. В дальнейшем при назначении пользователю ненулевой метки целостности также следует его домашнему каталогу назначить метку целостности, соответствующую максимально доступной метке целостности для данного пользователя. Если домашнему каталогу пользователя не будет назначена метка целостности, то после перезагрузки ОС она будет назначена автоматически.

При включенном расширенном режиме МКЦ применяются следующие изменения в работе ОС:

- 1) запрещен запуск процесса, если исполняемый файл, из которого запускается процесс, имеет метку целостности меньше или несравнимую с меткой целостности процесса-родителя (в данном случае для запуска необходимо использовать `sumic`, см. 4.16.10);
- 2) запрещен запуск заданий `crontab`, если метка целостности сущности, указанной в задании, больше или несравнима с максимально доступной меткой целостности пользователя, которому принадлежит данное задание. Процессу, запускаемому при выполнении задания, присваивается метка целостности, равная метке целостности сущности, указанной в задании;
- 3) не применяется параметр командной строки ядра `parsec.ccnr_relax` (см. 4.19);
- 4) возможно применение привилегии `PARSEC_CAP_CCNRELAX` (см. 4.7);

5) возможно применение атрибутов `irelax` и `inh` (см. 4.3.2).

4.6. Применение правил мандатного управления доступом и мандатного контроля целостности

Использование уровня и категорий конфиденциальности обеспечивает защиту от несанкционированного доступа к информации.

Использование метки целостности обеспечивает целостность информации путем запрета модификации сущностей с высокой меткой целостности недоверенными субъектами с более низкой меткой целостности.

При создании локальных пользователей или пользователей ЕПП необходимо устанавливать для них диапазоны допустимых уровней и категорий конфиденциальности: минимальный и максимальный уровни конфиденциальности, минимальный и максимальный наборы категорий конфиденциальности. Отсутствие установленных допустимых диапазонов уровней и категорий конфиденциальности приводит к запрету доступа к защищенному комплексу программ гипертекстовой обработки данных, к защищенному комплексу программ электронной почты, к защищенному комплексу программ печати и маркировки, к СУБД.

Порядок задания мандатных атрибутов системы и присвоения мандатного контекста безопасности пользователям приведен в разделе 17.

Значения мандатных атрибутов, указанные при входе в систему, формируют классификационную метку и метку целостности сессии пользователя.

Значения мандатных атрибутов сессии пользователя ОС можно просмотреть с помощью инструмента `pdp-id` (см. 4.16.2).

Управление контекстом безопасности сущностей осуществляется с помощью инструмента `pdpl-file` в соответствии с 4.16.1.

Классификационная метка субъекта/сущности наследуется от создающего его/ее субъекта, т. е. процесс наследует классификационную метку процесса-родителя, а файл/каталог/сокет наследует классификационную метку создающего процесса.

Классификационная метка сущности не может превышать значение классификационной метки сущности-контейнера, в которой она расположена.

Метка целостности субъекта наследуется от создающего его субъекта, т. е. процесс при его непосредственном запуске наследует метку целостности процесса-родителя. При этом при включенном расширенном режиме МКЦ субъект не может запустить процесс из файла, метка целостности которого меньше или несравнима с его собственной (см. 4.5).

При запуске процесса от имени другого пользователя с помощью механизма `sudo` метка целостности пользователя, выполняющего запуск, должна находиться в разрешенных пределах для пользователя, от имени которого запускается процесс. Если метка находится вне разрешенных пределов (например, запуск процесса администратором с категорией целостности 63 от имени пользователя с нулевой меткой целостности), то запуск запрещается. Запуск процесса в заданном окружении, в том числе с произвольной меткой целостности от имени другого пользователя, возможен с помощью инструмента `pdp-exec` (см. 4.16.8).

Метка целостности сущности не наследуется от создающего ее субъекта. При создании сущности, не являющейся сокетом, ей назначается нулевая категория целостности и линейный уровень целостности, являющийся минимальным значением из линейного уровня целостности каталога, в котором создается сущность, и нуля. При этом метка целостности субъекта, создающего сущность, должна быть не ниже метки целостности каталога, в котором создается сущность. При создании сокета ему назначается нулевая категория целостности и линейный уровень целостности, равный -128 , независимо от линейного уровня целостности каталога.

ВНИМАНИЕ! Процесс может изменять линейный уровень целостности созданного им сокета в диапазоне между -128 и своим линейным уровнем целостности. При наличии привилегии `PARSEC_CAP_MAC_SOCKET` процесс может изменять метку безопасности сокета (см. 4.7).

Порядок наследования меток целостности при запуске процессов или создании сущностей может быть изменен путем применения PARSEC-привилегий в соответствии с 4.7 и/или назначения дополнительных атрибутов сущностей для МКЦ в соответствии с 4.3.2.

ВНИМАНИЕ! Изменять классификационную метку сущности (т. е. изменять уровень конфиденциальности и/или категории конфиденциальности) может только субъект, имеющий метку целостности не ниже метки целостности этой сущности и обладающий привилегией `PARSEC_CAP_CHMAC` (см. 4.7), или субъект с правами `root` и с максимальной меткой целостности.

ВНИМАНИЕ! Понижать метку целостности сущности может только субъект с наличием привилегии `PARSEC_CAP_CHMAC` (см. 4.7) и имеющий метку целостности не ниже метки целостности этой сущности. Произвольно изменять метку целостности сущности может только субъект с правами `root` и с максимальной меткой целостности.

ВНИМАНИЕ! Субъект может изменять только собственную классификационную метку (т. е. изменять свой уровень конфиденциальности и/или категории конфиденциальности) при наличии у него привилегии `PARSEC_CAP_SETMAC` (см. 4.7).

ВНИМАНИЕ! Для процесса возможно только понизить метку целостности. Для выполнения этого безопасным способом, т. е. с закрытием наследуемых от родительского процесса файловых дескрипторов, рекомендуется использовать утилиту `sumic` (см. 4.16.10) и библиотечную функцию `pdp_set_pid_safe`.

Для создания в сущности-контейнере с атрибутом `ccnr` вложенной сущности с уровнем и категориями конфиденциальности меньшими, чем у сущности-контейнера, необходимо обладать привилегиями `PARSEC_CAP_IGNMACCAT` и `PARSEC_CAP_IGNMACLVL` (см. таблицу 13).

Мандатный контекст безопасности корневой файловой системы определяет максимальный мандатный контекст безопасности сущностей. Устанавливаемый по умолчанию мандатный контекст безопасности корневой файловой системы, а также мандатный контекст отдельных сущностей определены в сценарии `/usr/sbin/pdp-init-fs` (см. 4.16.3).

Принятие решения о запрете или разрешении доступа субъекта к сущности принимается на основе типа операции (чтение/запись/исполнение), мандатного контекста безопасности субъекта и мандатного контекста безопасности сущности.

Операции чтения разрешены, если:

- 1) уровень конфиденциальности субъекта не ниже уровня конфиденциальности сущности;
- 2) биты набора категорий конфиденциальности субъекта включают биты набора категорий конфиденциальности сущности;
- 3) метка целостности субъекта может быть любой.

Операции исполнения разрешены, если:

- 1) уровень конфиденциальности субъекта не ниже уровня конфиденциальности сущности;
- 2) биты набора категорий конфиденциальности субъекта включают биты набора категорий конфиденциальности сущности;
- 3) метка целостности субъекта:
 - а) при отключенном расширенном режиме МКЦ — может быть любой;
 - б) при включенном расширенном режиме МКЦ — не выше метки целостности сущности.

Операция записи разрешена, если:

- 1) уровни и категории конфиденциальности субъекта и сущности совпадают;
- 2) метка целостности субъекта не ниже метки целостности сущности.

ВНИМАНИЕ! При переименовании или редактировании файла в текстовом формате с ненулевой меткой безопасности для сохранения значения метки на файле (предотвращения сброса в 0) необходимо использовать параметр ядра `parsec.rename_mask` (см. 4.19).

При сравнении меток целостности между собой используются следующие правила:

- 1) метка А равна метке В при выполнении обоих условий:
 - а) в категории целостности метки А установлены все те же биты, что и в категории целостности метки В;
 - б) линейный уровень целостности метки А равен линейному уровню целостности метки В;
- 2) метка А больше метки В при выполнении одного из условий:
 - а) в категории целостности метки А установлены все те же биты, что и в категории целостности метки В, а также в категории целостности метки А установлены и другие биты, при этом линейный уровень целостности метки А равен или больше линейного уровня целостности метки В;
 - б) в категории целостности метки А установлены все те же биты, что и в категории целостности метки В, при этом линейный уровень целостности метки А больше линейного уровня целостности метки В;
- 3) две метки несравнимы при выполнении одного из условий:
 - а) в категории целостности каждой метки есть биты, отсутствующие у другой метки;
 - б) в категории целостности метки А установлены все те же биты, что и в категории целостности метки В, а также в категории целостности метки А установлены и другие биты, при этом линейный уровень целостности метки А меньше линейного уровня целостности метки В;
 - в) в категории целостности метки В установлены все те же биты, что и в категории целостности метки А, а также в категории целостности метки В установлены и другие биты, при этом линейный уровень целостности метки В меньше линейного уровня целостности метки А.

Субъект не может получить доступ к управлению другим субъектом или сущностью, если его метка целостности ниже или несравнима с меткой целостности субъекта или сущности.

Порядок предоставления доступа на чтение, исполнения и запись может быть изменен путем применения PARSEC-привилегий в соответствии с 4.7, назначения дополнительных атрибутов сущностей для мандатного управления доступом в соответствии с 4.2.3 и/или дополнительных атрибутов сущностей для МКЦ в соответствии с 4.3.2.

4.7. PARSEC-привилегии

PARSEC-привилегии назначаются субъектам и позволяют изменить порядок назначения меток безопасности и предоставления доступа на чтение, исполнение и запись. PARSEC-привилегии, как и Linux-привилегии, приведенные в 3.2, наследуются процессами от процессов-родителей.

ВНИМАНИЕ! Назначение PARSEC-привилегий недоверенным субъектам может привести к утечке информации.

Процессы, запущенные от имени суперпользователя, имеющего максимальную (Высокий) категорию целостности по умолчанию, независимо от явного назначения им привилегий, имеют возможность осуществлять большинство привилегированных действий.

PARSEC-привилегии и их описание приведены в таблице 13.

Таблица 13

Привилегия Битовая маска	Описание
PARSEC_CAP_FILE_CAP 0x000001	Не используется. Для изменения меток безопасности файловых объектов использовать привилегию PARSEC_CAP_CHMAC. Для изменения меток безопасности процессов использовать PARSEC_CAP_SETMAC. Для изменения привилегий процессов использовать PARSEC_CAP_CAP
PARSEC_CAP_AUDIT 0x000002	Позволяет управлять политикой аудита
PARSEC_CAP_SETMAC 0x000004	Позволяет процессу изменять собственную классификационную метку (уровень и категории конфиденциальности)
PARSEC_CAP_CHMAC 0x000008	Позволяет субъекту изменять метку безопасности сущности, если метка целостности сущности меньше или равна метки целостности субъекта. Привилегия позволяет изменять атрибуты сущностей для мандатного управления доступом (см. 4.2.3). Привилегия позволяет только понижать метку целостности сущности. Привилегия не позволяет изменять атрибуты сущностей для МКЦ (см. 4.3.2)
PARSEC_CAP_IGNMACLVL 0x000010	Позволяет игнорировать уровень конфиденциальности при применении правил мандатного управления доступом
PARSEC_CAP_IGNMACCAT 0x000020	Позволяет игнорировать категории конфиденциальности при применении правил мандатного управления доступом
PARSEC_CAP_SIG 0x000040	Позволяет посылать сигналы процессам, игнорируя мандатные права
PARSEC_CAP_UPDATE_ETIME 0x000080	Не используется. Позволяет изменять время доступа к файловым объектам

Продолжение таблицы 13

Привилегия Битовая маска	Описание
PARSEC_CAP_PRIV_SOCK 0x000100	Сокеты, которые создаются процессом с этой привилегией, получают дополнительный атрибут <code>ehole</code>. Привилегированный сокет позволяет осуществлять сетевое взаимодействие, игнорируя мандатную политику. Привилегию рекомендуется использовать для сетевых сокетов. Для UNIX-сокетов рекомендуется использовать привилегию <code>PARSEC_CAP_MAC_SOCK</code>. ВНИМАНИЕ! Привилегия предназначена для назначения только службам, назначение привилегии пользователям недопустимо
PARSEC_CAP_READSEARCH 0x000200	Позволяет игнорировать мандатную политику при чтении и поиске файловых объектов (но не при записи)
PARSEC_CAP_CAP 0x000400	Позволяет процессу назначать себе любой непротиворечивый набор привилегий и читать привилегии, присвоенные процессам
PARSEC_CAP_MAC_SOCK 0x000800	Позволяет изменять метки безопасности точек соединения — сокетов. Процесс с данной привилегией может изменять классификационную метку сокета без ограничений и метку целостности сокета, у которого метка целостности не выше метки целостности процесса. При этом повышать метку целостности сокета возможно не выше метки целостности процесса. Процесс без привилегии <code>PARSEC_CAP_MAC_SOCK</code> может изменять только линейный уровень целостности созданного им сокета в диапазоне между -128 и своим линейным уровнем целостности. Привилегию рекомендуется использовать для UNIX-сокетов. Для сетевых сокетов рекомендуется использовать привилегию <code>PARSEC_CAP_PRIV_SOCK</code>. ВНИМАНИЕ! Привилегия предназначена для назначения только службам, назначение привилегии пользователям недопустимо
PARSEC_CAP_UNSAFE_SETXATTR 0x001000	Позволяет устанавливать мандатные атрибуты объектов файловой системы без учета мандатных атрибутов родительского каталога. Также позволяет создавать жесткие ссылки без учета мандатных атрибутов родительского каталога. Привилегия используется для восстановления объектов файловой системы из резервных копий и только после установки значения 1 для параметра <code>/parsecfs/unsecure_setxattr</code>
PARSEC_CAP_IGNMACINT 0x002000	Позволяет игнорировать политику МКЦ. Устанавливать данную привилегию может только субъект с правами <code>root</code> и с максимальной меткой целостности в системе. ВНИМАНИЕ! Запрещено использование данной привилегии при штатной работе МКЦ и при разработке. Данная привилегия предназначена только для временного использования при устранении сбоев и отладке работы ОС или прикладного ПО
PARSEC_CAP_SUMAC 0x004000	Позволяет запускать процессы с другой классификационной меткой (уровнем и категорией конфиденциальности)

Окончание таблицы 13

Привилегия Битовая маска	Описание
PARSEC_CAP_BYPASS_KIOSK 0x008000	Не используется. Позволяет игнорировать ограничения киоска (parsec-kiosk)
PARSEC_CAP_IPC_OWNER 0x010000	Отменяет мандатные ограничения при работе с сущностями IPC, такими как shared memory, message queue и т.д. (PARSEC-привилегия, аналогичная Linux-привилегии CAP_IPC_OWNER)
PARSEC_CAP_INHERIT_INTEGRITY 0x020000	При создании сущностей им назначается метка целостности, равная метке целостности родительского каталога. При этом запрещено создавать сущности с меткой целостности выше или несравнимой с меткой целостности процесса, создающего данную сущность. ВНИМАНИЕ! Если родительскому каталогу присвоен атрибут irelax, то создаваемым в нем сущностям присваивается метка целостности по правилам работы атрибута irelax (см. 4.3.2), т.к. irelax обладает приоритетом над данной привилегией. Используется для установщика пакетов (dpkg)
PARSEC_CAP_BYPASS_XATTR 0x040000	Отключает проверку подписи в xattr файлов
PARSEC_CAP_PROCFS 0x080000	Включает игнорирование уровней конфиденциальности и метки целостности при работе с /proc
PARSEC_CAP_CCNR_RELAX 0x100000	Позволяет осуществлять в каталоге с установленным атрибутом ccnr действия (создание, удаление и др.) над вложенными файлами и каталогами с уровнями конфиденциальности не выше уровня конфиденциальности данного каталога. Привилегия применяется при включенном расширенном режиме МКЦ (см. 4.5)

Список доступных PARSEC-привилегий возможно просмотреть с помощью инструмента usercaps (см. 4.17.1).

Для настройки KC3 могут использоваться как PARSEC-, так и Linux-привилегии. Порядок управления привилегиями описан в 4.17.

4.8. Включение и выключение мандатного управления доступом

4.8.1. Включение мандатного управления доступом

Мандатное управление доступом может быть включено в процессе установки ОС путем выбора соответствующего пункта программы установки ОС.

Включение мандатного управления доступом после установки ОС выполняется с помощью инструмента astra-mac-control, описанного в 16.6.31. Также включить можно в модуле «Мандатное управление доступом» (графическая утилита astra-systemsettings «Пара-

метры системы», раздел «Управление доступом», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_mac
```

При включении мандатного управления доступом для параметра `parsec.mac` командной строки ядра в загрузчике ОС устанавливается значение 1. При этом все доработанные для функционирования в условиях мандатного управления доступом сетевые службы, системы IPC и системы инициализации будут работать в режиме мандатного управления доступом. Также программам будет доступна возможность работать с файловыми объектами, имеющими ненулевую классификационную метку.

4.8.2. Выключение мандатного управления доступом

Выключение мандатного управления доступом целесообразно применять только для отладочных целей. Выключать мандатное управление доступом в системе, введенной в эксплуатацию, не рекомендуется, поскольку выключение может привести к несогласованному состоянию объектов файловой системы с разными уровнями конфиденциальности.

Выключение мандатного управления доступом выполняется с помощью инструмента `astra-mac-control`, описанного в 16.6.31. Также выключить можно в модуле «Мандатное управление доступом» (графическая утилита `astra-systemsettings` «Параметры системы», раздел «Управление доступом», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_mac
```

При выключении мандатного управления доступом для параметра `parsec.mac` командной строки ядра устанавливается значение 0.

ВНИМАНИЕ! При выключенном мандатном управлении доступом службы, которые должны запускаться на ненулевом уровне конфиденциальности или в процессе работы повышают свой уровень конфиденциальности, будут запущены на нулевом уровне конфиденциальности или не будут запущены с выводом информации об ошибке.

После выключения мандатного управления доступом для программ будет отключена возможность работать с файловыми объектами, имеющими ненулевую классификационную метку.

ВНИМАНИЕ! Отключение возможности работы программ с файловыми объектами, имеющими ненулевую классификационную метку, не равносильно удалению таких объектов. Файловые объекты, имеющие ненулевую классификационную метку, остаются в системе после отключения возможности работы с ними. Доступ к таким объектам не может быть

получен штатными средствами ОС, но может быть получен при наличии неконтролируемого физического доступа к компьютеру и возможности использовать на нем нештатные средства.

4.9. Включение и выключение мандатного контроля целостности

Включение МКЦ может быть выполнено в процессе установки ОС путем выбора пункта «Мандатный контроль целостности» в программе установки ОС.

Включение и выключение МКЦ после установки ОС выполняется с помощью инструмента `astra-mic-control`, описанного в 16.6.27. Также включить или выключить можно в модуле «Мандатный контроль целостности» (графическая утилита `astra-systemsettings` «Параметры системы», раздел «Управление доступом», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_mic
```

При включении/выключении МКЦ автоматически включается/выключается МКЦ на файловой системе (см. 4.10).

При включении МКЦ для параметра командной строки ядра `parsec.max_ilev` в загрузчике ОС устанавливается значение максимальной категории целостности в системе. По умолчанию значение максимальной категории целостности в системе 63 (если при включении МКЦ не было указано другое значение, см. 16.6.27).

ВНИМАНИЕ! Графический сервер Xorg по умолчанию работает от имени служебной учетной записи `fly-dm` на выделенном уровне целостности 8. При отсутствии поддержки драйверами видеокарты KMS (Kernel Mode-Setting) запуск Xorg производится от имени учетной записи `root`.

При выключении МКЦ для параметра командной строки ядра `parsec.max_ilev` устанавливается значение 0.

4.10. Мандатный контроль целостности на файловой системе

При включении МКЦ согласно 4.9 объектам файловой системы автоматически присваиваются метки целостности.

Присвоение меток целостности объектам файловой системы осуществляется в соответствии с конфигурационным файлом `/etc/parsec/fs-ilev.conf`. Объектам, не указанным в файле, метка целостности не назначается.

В конфигурационном файле перечислены объекты файловой системы и их метка целостности в формате:

<категория_целостности>[:<атрибут>:<линейный_уровень_целостности>] <путь>

где <категория_целостности> — категория целостности для объекта файловой системы;

<атрибут> — дополнительный атрибут сущности для МКЦ, может быть указано несколько атрибутов;

<линейный_уровень_целостности> — линейный уровень целостности для объекта файловой системы;

<путь> — объект файловой системы или путь к нему.

Если в файле указаны несуществующие и неабсолютные пути, то они игнорируются. Корневому каталогу («/») метка целостности не назначается.

Значения, указываемые в конфигурационном файле в качестве категории целостности <категория_целостности>, приведены в таблице 14.

Таблица 14

Значение	Описание
<число>	Определенная категория целостности, заданная числовым значением. Может быть десятичным, восьмеричным, шестнадцатеричным или двоичным числом
high	Текущее значение max_ilev — максимальная категория целостности в ОС, заданная в параметре командной строки ядра parsec.max_ilev
max	Текущее значение max_ilev — максимальная категория целостности в ОС, заданная в параметре командной строки ядра parsec.max_ilev
low	То же, что и нулевая категория целостности
min	То же, что и нулевая категория целостности
exc	Игнорировать файл при проверке целостности. В качестве символа подстановки в конце пути можно использовать символ «*»

Пример

Конфигурационный файл /etc/parsec/fs-ilev.conf

```
exc    /etc/xdg/autostart/vboxclient.desktop
exc    /etc/X11/Xsession.d/98vboxadd-xclient
exc    /etc/ld.so.*
exc    /etc/resolv.conf
exc    /root/.config/*
exc    /root/.gnupg/gpg-agent.conf
max    /etc
```

```
max      /lib
max      /lib64
max      /lib32
max      /bin
max      /sbin
max      /boot
max      /root
max      /opt
max      /srv
max      /usr
max      /var/lib/dpkg/info
min      /var/lib/docker
max      /var/lib/polkit-1/localauthority
max      /etc/polkit-1
max      /usr/share/polkit-1
```

Для управления метками целостности на файловой системе используется инструмент командной строки `set-fs-ilev`.

После установки новых пакетов, а также в процессе работы ОС могут создаваться новые файлы в каталоге `/etc/`, которым метки целостности МКЦ автоматически не присваиваются. Чтобы привести объекты файловой системы в соответствие конфигурационному файлу `/etc/parsec/fs-ilev.conf`, необходимо выполнить команду:

```
sudo set-fs-ilev enable
```

Подробное описание инструмента `set-fs-ilev` приведено на справочной странице `man set-fs-ilev`.

Также для управления МКЦ на файловой системе может использоваться модуль «Мандатный контроль целостности» в разделе «Управление доступом» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_mic
```

Выключение МКЦ на файловой системе осуществляется автоматически при выключении МКЦ согласно 4.9.

Для обеспечения совместимости в ОС сохранен устаревший инструмент выключения МКЦ на файловой системе `unset-fs-ilev`.

4.11. Администрирование ОС при включенном МКЦ

Привилегированному пользователю (администратору) следует входить в систему с высокой категорией целостности (соответствует максимальной категории целостности ОС) и только для выполнения настроек ОС.

Для обычного (штатного) режима работы рекомендуется осуществлять вход в систему от имени непривилегированного пользователя с низкой категорией целостности (соответствует минимальной категории целостности).

Применяются ограничения на сочетание классификационной метки и категории целостности при входе в сессию.

Если выбрана ненулевая классификационная метка (выбран ненулевой уровень конфиденциальности и/или категории конфиденциальности), то вход может быть выполнен только с нулевой категорией целостности (при этом в расширенном режиме МКЦ выбирается автоматически при выполнении входа).

При включенном расширенном режиме МКЦ, если выбрана нулевая классификационная метка (или у пользователя отсутствуют уровень и категории конфиденциальности), то вход автоматически выполняется с максимальной категорией целостности, доступной пользователю.

ВНИМАНИЕ! При отключенном расширенном режиме МКЦ не рекомендуется выполнять вход в систему администратором с нулевой классификационной меткой и нулевой категорией целостности.

При консольном входе в систему администратор должен вручную выставлять категорию целостности (для высокой — 63, для низкой — 0 или пропустить данный шаг).

При входе в сессию с высокой категорией целостности графический рабочий стол имеет красную тему оформления, которая не может быть изменена.

4.12. Разделяемые каталоги при включенном мандатном управлении доступом

В ОС реализована собственная структура разделяемых каталогов, которая используется при включенном мандатном управлении доступом для ограничения доступа к разделяемым каталогам. В данной структуре созданным при установке ОС основным разделяемым каталогам сопоставляются дублирующие разделяемые каталоги.

Для автоматического создания структуры используется файл `/etc/parsec/mlinks`. По умолчанию в файле указаны разделяемые каталоги, создаваемые при установке ОС:

```
/tmp /var/private/tmp 1777
```

```

/var/tmp /var/private/vartmp 1777
# /var/cache /var/private/varcache 0755
/dev/shm /run/private/shm 1777
/run/mount /run/private/mount 0755

```

В каждой строке файла `/etc/parsec/mlinks` содержатся три информационных поля, разделенных пробелами:

- 1) путь основного разделяемого каталога, для которого должен быть создан дублирующий разделяемый каталог;
- 2) путь дублирующего разделяемого каталога, в который будут перенаправлены обращения к основному разделяемому каталогу;
- 3) дискреционные права доступа к дублирующему разделяемому каталогу:
 - а) 1777 — полный доступ для всех пользователей с применением sticky-бита;
 - б) 0755 — разрешение чтения и поиска для всех пользователей.

Дублирующие разделяемые каталоги создаются при установке (загрузке) ОС.

При необходимости в файле можно указать основные и дублирующие разделяемые каталоги для ПО, устанавливаемого в ОС.

При первом входе в сессию с ненулевой классификационной меткой в дублирующих разделяемых каталогах создаются каталоги данной сессии. Название каталогов сессии имеет вид:

```

l<уровень_конфиденциальности>i<неиерархический_уровень_целостности>
c0x<категории_конфиденциальности>t0x0

```

где `<уровень_конфиденциальности>` — иерархический уровень конфиденциальности пользовательской сессии, десятичное число;
`<уровень_целостности>` — неиерархический уровень целостности пользовательской сессии, десятичное число;
`<категории_конфиденциальности>` — неиерархические категории конфиденциальности, шестнадцатеричное число;
`t0x0` — метка типа, не изменяется.

При втором и последующих входах в сессию с ненулевой классификационной меткой будут использоваться ранее созданные каталоги данной сессии. При этом будут доступны для просмотра каталоги сессии, имеющие уровень конфиденциальности меньше.

В сессии с ненулевой классификационной меткой все обращения к основным разделяемым каталогам автоматически перенаправляются в соответствующие каталоги сессии, расположенные в дублирующих разделяемых каталогах. При этом содержимое основных

разделяемых каталогов отображается в каталогах сессии с нулевой классификационной меткой (10i0c0x0t0x0).

Все дублирующие разделяемые каталоги автоматически очищаются при перезагрузке ОС. Параметры очистки определены в разделе `clear_private` сценария `/usr/lib/parsec/systemd/parsec_systemd_init`.

Если требуется сохранять после перезагрузки содержимое отдельных каталогов из состава дублирующих разделяемых каталогов, можно добавить в сценарий строку, позволяющую пропускать очистку конкретного каталога.

Пример

```
#
# Clear private dirs.
#
clear_private() {
  for d in `find /var/private/*/* 2>/dev/null?`; do
    # Следующая строка предотвращает очистку каталога /var/private/test
    # и его подкаталогов
    if [[ "$d" =~ ^/var/private/test/* ]]; then continue; fi
    # Следующая строка очищает каталог
    rm -rf $d
  done
  return $?;
}
```

ВНИМАНИЕ! При внесении изменений в файл `/etc/parsec/mlinks` изменится его контрольная сумма, указанная в `gostsums.txt` на установочном диске. Если файл `/etc/parsec/mlinks` поставлен на контроль целостности, то после его изменения следует обновить настройки контроля целостности, чтобы избежать невозможности загрузки ОС из-за нарушения целостности контролируемого файла.

4.13. Запуск службы `systemd` с заданной меткой безопасности и привилегиями

По умолчанию службы `systemd` запускаются с нулевой классификационной меткой и с высокой категорией целостности (значение параметра ядра `parsec.max_ilev`, после установки ОС равно 63).

Для запуска службы `systemd` с определенной меткой безопасности (классификационной меткой и категорией целостности) необходимо в конфигурационном файле соответствующей службы (юните, имеет имя вида `<имя_юнита>.service`) в разделе `[Service]` добавить параметр `PDPLabel` и указать для него значение метки безопасности:

```
[Service]
```



```
PDPLabel=<Уровень_конфиденциальности>:<Категория_целостности>:
<Категории_конфиденциальности>
```

ВНИМАНИЕ! Задавать определенную метку безопасности в юните возможно в том случае, если программное обеспечение, запускаемое этим юнитом, доработано для работы в условиях мандатного управления доступом и МКЦ.

Формат метки безопасности в параметре PDPLabel аналогичен принятому в системе PARSEC за исключением поля дополнительных атрибутов — метка безопасности службы не может иметь дополнительный атрибут сущности для мандатного управления доступом (наличие `ehole/whole` недопустимо). Более подробная информация о формате метки безопасности доступна в выводе команды:

```
pdpl-file --help
```

В метке безопасности рекомендуется использовать числовые обозначения, так как при разрешении имен могут оказаться задействованы сетевые ресурсы, например LDAP-каталоги, что может привести к ошибкам конфигурации, которые сложно диагностировать.

Метка безопасности может быть задана только в пределах определенных в ОС значений (значение параметра ядра `parsec.max_ilev` и значения в сценарии инициализации `pdp-init-fs`, см. 4.16.3).

Для управления службой `systemd` и изменения юнита пользователь должен иметь категорию целостности, включающую в себя категорию целостности данной службы. Попытки пользователя управлять службой, имеющей категорию целостности выше, чем его, регистрируются в журнале `/var/log/syslog`.

Также службе `systemd` могут быть назначены PARSEC-привилегии, приведенные в 4.7. Для этого необходимо в соответствующем юните в разделе `[Service]` добавить параметр `CapabilitiesParsec` и перечислить назначаемые привилегии:

```
[Service]
CapabilitiesParsec=PARSEC_CAP_PRIV_SOCK PARSEC_CAP_CHMAC
```

При назначении нескольких привилегий они указываются через пробел.

После редактирования юнита необходимо перезапустить `systemd` и юнит или соответствующую юниту службу, выполнив команды:

```
systemctl daemon-reload
systemctl restart <имя_юнита>.service
```

Для просмотра метки безопасности службы выполнить команду:

```
pdpl-ps <PID>
```

где <PID> — идентификатор службы.

Для просмотра PARSEC-привилегий, назначенных службе, выполнить команду:

```
pscaps <PID>
```

Для определения PID используется команда:

```
systemctl status <имя_юнита>.service
```

4.14. Сетевое взаимодействие. Механизм privsock

В качестве основного протокола доступа к сетевой ФС используется протокол SMB, который поддерживает передачу расширенных атрибутов, содержащих информацию о мандатном контексте (метке безопасности и мандатных атрибутах управления доступом). Данный протокол широко распространен и работает в гетерогенных сетях (поддерживается различными операционными системами), а также поддерживает собственную аутентификацию и аутентификацию с использованием Kerberos.

Взаимодействие при помощи сетевого протокола IPv4 (IPv6) осуществляется через программный интерфейс объектов доступа, являющихся элементами межпроцессного и сетевого взаимодействия, например сетевых сокетов, которые обеспечивают обмен данными между процессами в рамках одной или нескольких ОС, объединенных в локальную вычислительную сеть.

Для поддержки мандатного управления доступом в сетевые пакеты протокола IPv4 (IPv6) внедряются классификационные метки. Порядок присвоения классификационных меток и их формат соответствует национальному стандарту ГОСТ Р 58256-2018. Прием сетевых пакетов подчиняется мандатным ПРД. Сетевой сервис может создать сокет с дополнительным атрибутом, позволяющим принимать соединения с любыми уровнями секретности (см. 4.7).

При необходимости для обеспечения целостности заголовка IP-пакетов, содержащего классификационную метку, допускается применение программного средства OpenVPN. Описание использования OpenVPN приведено в документе РУСБ.10015-01 95 01-1.

Отсутствие у сущности метки безопасности эквивалентно нулевой метке безопасности. Таким образом, ядро ОС, в которой все сущности и субъекты доступа имеют уровень секретности «несекретно», функционирует аналогично стандартному ядру операционной

системы семейства Linux.

Для ряда сетевых служб (сервера LDAP, DNS, Kerberos и т. д.) необходимо обеспечить возможность их работы с клиентами, имеющими разный мандатный контекст безопасности, без внесения изменений в исходные тексты службы. Для предоставления данной возможности в подсистеме безопасности PARSEC реализован механизм запуска сетевых служб с использованием привилегированного сокета для ожидания входящих соединений — механизм `privsock`.

Механизм `privsock` предназначен для обеспечения функционирования системных сетевых служб, не осуществляющих обработку информации с использованием мандатного контекста, но взаимодействующих с процессами, работающими в мандатном контексте субъекта доступа.

Для его использования при функционировании сетевой службы необходимо отредактировать файл `/etc/parsec/privsock.conf`, добавив в него строку, содержащую полный путь к исполняемому файлу службы. Далее приведен пример строки из файла `/etc/parsec/privsock.conf` для запуска DNS-сервера с использованием механизма `privsock`.

Пример

```
/usr/sbin/named
```

Для использования механизма `privsock` необходимо, чтобы переменная `PATH`, используемая при запуске службы, содержала следующий путь:

```
/usr/lib/parsec/bin
```

Далее приведен пример задания требуемого пути в переменной окружения для запуска DNS-сервера.

Пример

Установка значения переменной окружения `PATH` может быть выполнена добавлением в файл `/etc/default/bind9` следующей строки:

```
PATH=/usr/lib/parsec/bin:$PATH
```

Подробное описание механизма `privsock` приведено на справочной странице `man privsock`.

4.15. Шина межпроцессного взаимодействия D-Bus

D-Bus позволяет организовать взаимодействие процессов с использованием сообщений и общих шин. Для передачи сообщения между процессом и шиной используется механизм сокетов. Выделяют два типа шин: сессионные шины (session bus) и системная шина (system bus).

Сессионная шина создается для каждой пользовательской сессии при ее запуске. Она работает с меткой безопасности пользователя системы и все взаимодействующие через нее процессы также имеют данную метку безопасности. Взаимодействие процессов с иной меткой безопасности не происходит. Дополнительные сессионные шины могут быть созданы с использованием утилиты `dbus-launch`, которая позволяет запустить процесс одновременно с созданием новой сессионной шины. При этом имеется возможность указать, используя переменную `DBUS_SESSION_BUS_ADDRESS`, какую из сессионных шин данный процесс будет воспринимать как сессионную шину.

Пример

```
user@astra:~$ echo $DBUS_SESSION_BUS_ADDRESS
unix:abstract=/tmp/dbus-FMSYkkteW0,guid=2f874cb94fd70c984eff1d8857d24781
```

Аналогично функционирует системная шина. Данная шина создается только в одном экземпляре при старте службы D-Bus и через нее взаимодействуют процессы различных уровней.

Модуль `dbus-daemon` реализует и сессионные, и системную шины, поэтому механизм мандатного управления доступом используется во всех шинах.

4.15.1. Виды сообщений

D-Bus поддерживает следующие виды сообщений:

- `method_call` — вызов метода. Процесс требует вызова метода, реализованного другим процессом. Данное сообщение имеет конкретного адресата и содержит информацию об источнике сообщения;
- `method_return` — возврат. Как правило, после вызова метода данное сообщение используется для возврата значения. Адресатом сообщения является источник исходного сообщения;
- `error` — ошибка. Если при вызове произошла ошибка, вместо возврата может быть отправлено сообщение об ошибке. Адресатом сообщения является источник исходного сообщения;
- `signal` — сигнал. Данное сообщение имеет источник, но, как правило, не имеет конкретного адресата и рассылается шиной всем, кто подписан на данное сообщение

(с использованием match-фильтра). Если указано имя службы, то сообщение направляется только определенному соединению (т. е. первичному владельцу данной службы).

Все сообщения являются однонаправленными, могут иметь назначение, а также могут подразумевать (но не требовать) ответ, например, после `method_call` можно не посылать `method_return`. Функционал отправителя и получателя должен быть согласован.

4.15.2. Процесс взаимодействия с шиной

Для идентификации соединения на шине используются параметры, приведенные в таблице 15.

Таблица 15

Параметр	Описание
<code>unique connection name (UCN)</code>	Уникальное имя соединения — идентификатор соединения
<code>org.share.server</code>	Общеизвестное имя (служба) соединения. Данное имя используется для приема сообщения от других клиентов (т. е. по этому имени клиенты могут найти данный процесс)
<code>/org/share/server/object</code>	Данный путь является именем объекта. Каждая служба может создавать несколько объектов
<code>com.share.interface</code>	Имя интерфейса. Соответствует некоторому набору методов (в том числе стандартным методам, общим для объектов), которые должны корректно обрабатываться объектом
<code>GetMyName</code>	Имя метода объекта или интерфейса, который можно вызвать

Для вызова метода необходимо:

- указать общеизвестное имя соединения, имя объекта, имя интерфейса и имя метода;
- сформировать набор аргументов;
- выполнить вызов.

Для обработки результата необходимо принять ответное сообщение (которое может быть как сообщением типа возврат, так и ошибкой). Также существуют широковещательные сообщения-сигналы, которые отправляются процессом на шину, где переадресовываются всем соединениям, которые на него подписаны.

Каждый процесс для работы через D-Bus подключается к заданной шине (при этом создается уникальное соединение с именем вида `:1.8`) с использованием функции `dbus_bus_get()`. При этом при подключении осуществляется механизм аутентификации клиента с использованием данных сокета. После этого клиент может работать с шиной с использованием сообщений.

4.15.3. Процесс соединения с системной шиной

После аутентификации клиент отправляет сообщение типа `method_call Hello` серверу D-Bus. Далее проверяется возможность доставки данного сообщения с использованием `rbp` и `selinux` функций. Для того, чтобы D-Bus мог обработать данный вызов с ненулевыми уровнями, необходимо для него установить соответствующие привилегии. В частности, это реализовано за счет установки привилегий `0x30 (Ignore Lev & Cat)` на службу `org.freedesktop.Dbus`. Данные привилегии позволяют процессам (соединениям) с более высоким уровнем выполнять функции процессов с низким уровнем (`dbus`), что не предусматривает мандатное управление доступом, но разрешается привилегиями `0x30`, которые позволяют субъекту (`dbus`) игнорировать метку безопасности соединения, пытающегося выполнить его метод.

После этого формируется ответное сообщение типа `method_return`, которое возвращает ответный статус об успешном создании соединения с шиной. Одновременно происходит установка владельца службы. Под службой понимается имя на шине, которое соответствует некоторому соединению. При этом владельцами уникальных имен (вида `:1.6`) могут быть только сами соединения. Если используется пользовательское имя, например, `org.share.linux`, то его владельцем может быть соединение с некоторым уникальным именем. Другие процессы (соответственно с другими уникальными именами соединений) также могут запросить доступ владения данным именем. В зависимости от настроек может произойти как замена владельца, так и установка его в очередь на владение. При этом, когда текущий владелец освободит данное имя (отключится от шины), владельцем станет следующее по списку соединение (если такие есть). Важно, что именно владелец, т.е. соединение и соответствующий ему процесс, будет обрабатывать адресованное данному имени (службе) сообщение. В дополнение к ответу источнику сообщения `method_call Hello` происходит формирование сигнала `signal`. D-Bus формирует сигнал `NameAcquired` (т.е. источником является D-Bus), который сообщает, что имя (в данном случае UCN) получено.

Далее для получения владения некоторым именем (функция `dbus_bus_request_name`) формируется сообщение `method_call RequestName` серверу D-Bus. При этом могут использоваться следующие флаги:

- разрешить замену владельца. Если другое соединение потребует заменить владельца, то этот флаг разрешит выполнить смену с учетом метки безопасности;
- заменить владельца. Если разрешена замена, то он будет заменен у ранее созданной службы;
- не ставить в очередь, если владельца нельзя сейчас заменить. Иначе данное соединение встанет в конец очереди, и когда будут освобождены все владельцы — оно станет владельцем.

Если замена возможна, то вызывается драйвер-функция `RequestName`. После смены владельца службы формируется сигнал `NameOwnerChanged`. После освобождения имени формируется сигнал `NameLost`.

4.15.4. Объекты и субъекты системы

В рамках D-Bus объектами и субъектами являются сущности, сопоставленные с именами соединений и служб (т. е. уникальными именами и общеизвестными именами). Соответственно, каждое соединение имеет свою метку безопасности. Кроме этого, при создании службы (получения имени через `dbus_request_name`), а также создании уникального имени, происходит назначение им метки безопасности. Сведения о метках безопасности содержатся в хеш-таблице, где каждому имени соответствует метка безопасности. При освобождении имени, когда последний владелец уходит из очереди или клиент отсоединяется от шины, выполняется удаление соответствующей записи таблицы (при использовании `LOCKED`-настройки метка безопасности остается).

Проверка `pdf` доступа осуществляется при следующих событиях:

- принятие решения о доставке сообщения в зависимости от его типа;
- получение владения службой;
- выполнения методов службы D-Bus.

Каждое соединение при своем создании (получении уникального имени) получает соответствующую метку безопасности исходя из метки безопасности сокета.

4.15.5. Алгоритм проверки меток для различных сообщений и режимов работы

Взаимодействие по шине D-Bus можно разделить на два типа:

- сообщения между службами и соединениями;
- сообщения, адресованные службе D-Bus. Здесь представлены служебные сообщения, которые позволяют получать информацию о других службах и соединениях на шине. Т. е. в параметрах сообщения может быть указана заданная служба, информацию о которой необходимо получить.

Для сообщений между службами и соединениями используется функция `bus_pdflinux_allows_send()`. При широковещательной рассылке сигналов также происходит проверка каждого сообщения.

Для сообщений, адресованных службе D-Bus, используется сначала функция `bus_pdflinux_allows_send()`. Она определяет возможность отправки сообщения на D-Bus. Затем вызывается функция `bus_pdflinux_service_allows_connection_to_service` с заданными параметрами, которая определяет возможность выполнения тех или иных методов.

Для методов `service_exists`, `get_service_owner`, `list_queued_owners`, `get_connection_unix_user`, `get_connection_unix_process_id`, `get_adt_audit_session_data`, `get_connection_selinux_security_context`, `get_connection_pdplinux_security_context_helper`, `introspect`, `bus_driver_handle_get_id` и др. в зависимости от настроек проверяется возможность доступа `READ (TOS_READ)` к заданной службе.

Для метода `list_queued_owners` выводятся только те владельцы, метка безопасности соединения которых меньше, чем у соединения, создавшего данное сообщение (т. е. вызвавшего метод). Наличие привилегий может разрешать данный вызов в случае невыполнения последнего условия.

Для метода `list_services` выводится информация только по тем службам, метка безопасности которых меньше, чем у соединения, создавшего данное сообщение (т. е. вызвавшее метод). Наличие привилегий может разрешать данный вызов в случае невыполнения последнего условия.

Для получения владения службой в методах `acquire_service`, `add_owner`, `swap_owner` функция `bus_pdplinux_service_allows_connection_to_service` вызывается с указанием псевдодоступа `ETOS_ADDOWNER` (для `acquire_service` используется `ETOS_ADDOWNER_ACQUIRE_SERVICE`), которая в зависимости от настроек параметра конфигурации `<pdplinux_allow_different_owners>` (`BUS_CONTEXT_PDPLINUX_GET_pdplinux_allow_different_owners`) проверяет наличие потенциальных владельцев в очереди с меткой безопасности, отличной от метки безопасности службы. Если параметр `pdplinux_allow_different_owners` установлен, то возможно получение владения службой.

Для непосредственной проверки возможности замены владельца службы модифицирована функция `bus_service_get_allow_replacement()`, которая выполняет проверку метки целостности и не позволяет проводить соответствующую замену в случае, если метка целостности службы выше метки целостности нового владельца.

При пересылке сообщений наряду с получателем сообщения может присутствовать соединение `eavesdropping`. При этом сообщение адресуется не только истинному получателю, но и перенаправляется на дополнительное соединение, которое может быть создано, например, программой `dbus-monitor`.

Для каждого типа сообщения формируется соответствующий вид доступа. Для `method_call` применяется доступ `exec`. Для этого сравниваются метки безопасности источника сообщения и соответствующей службы, совпадающей с ее текущим владельцем. Если метки безопасности равны, то доступ разрешается. Иначе вызывается функция `bus_pdplinux_allows_execution()`, которая разрешает низкоуровневым объектам запуск функций объекта более высокого уровня. Если запуск запрещен, то проверяются привилегии. Например, если служба, метод которой запускается, имеет

привилегии 0x30, то запуск разрешается. Также существует возможность запуска метода, если соединение-получатель имеет дополнительный атрибут `ehole` в функции `bus_pdp_linux_allows_execution_helper()`.

Для `method_return`, `error`, `signal` получатель сообщения считывает данные из источника. Таким образом они меняются местами (получатель является субъектом, источник — объектом) и доступ назначается равным `read`. Проверка `pdp1_permission()`.

4.15.6. Привилегии процесса `dbus-daemon`

Установка привилегий процесса `dbus-daemon` зависит от установки параметра сборки `--enable-audit/--disable-audit`. Это влечет за собой `define: HAVE_LIBAUDIT` (`--enable-libaudit`).

В случае, если `HAVE_LIBAUDIT` определен, то вследствие доработки функции `dbus_change_to_daemon_user` в файле `audit.c` в точку считывания привилегии `CAP_AUDIT_WRITE` добавлен код считывания `PERMITTED` (`CAPNG_PERMITTED`) привилегий процесса в переменные флагов `have_cap_override`, `have_cap_ptrace`, `have_cap_admin` (для соответствующих привилегий). Если значение данных переменных `true`, то производится установка `CAP_DAC_OVERRIDE`, `CAP_SYS_PTRACE`, `CAP_SYS_ADMIN` привилегий для процесса `dbus-daemon`. Выполняется для обеспечения возможности доступа `dbus-daemon` к именам процессов.

В случае, если `HAVE_LIBAUDIT` не определен (`--disable-audit`), то производится обработка функции `dbus_change_to_daemon_user` в файле `dbus-sysdeps-util-unix.c`. При этом добавлен функционал, позволяющий скопировать все привилегии процесса из `CAP_INHERITABLE` в `CAP_EFFECTIVE` (при условии, что `dbus` запущен от имени пользователя `root`). Предполагается получение привилегий `CAP_SYS_ADMIN` `CAP_DAC_OVERRIDE` `CAP_SYS_PTRACE` для считывания имени процесса. Данные привилегии предварительно устанавливаются с использованием `systemd` в конфигурационном файле `dbus.service`:

```
# AmbientCapabilities=CAP_SYS_ADMIN CAP_DAC_OVERRIDE CAP_SYS_PTRACE
# SecureBits=keep-caps
```

В настоящий момент параметры `AmbientCapabilities` и `SecureBits` в `dbus.service` закомментированы, и используется `HAVE_LIBAUDIT`. Также при использовании нового `systemd` возможно использование `PARSEC`-привилегий для процесса `dbus.socket`:

```
CapabilitiesParsec=PARSEC_CAP_PRIV SOCK PARSEC_CAP_IGNORE
PARSEC_CAP_IGNORE
```

4.15.7. Расширенное управление политиками

4.15.7.1. Конфигурационный файл

Основными секциями конфигурационного файла D-Bus `system.conf` являются `<busconfig>` (корневая секция), `<type>` и `<policy>`.

Пример

Конфигурационный файл шины `accessibility`:

```
<!DOCTYPE busconfig PUBLIC "-//freedesktop//DTD D-Bus Bus Configuration
  1.0//EN" "http://www.freedesktop.org/standards/dbus/1.0/
  busconfig.dtd">
<busconfig>
  <type>accessibility</type>
<servicedir>/usr/share/dbus-1/accessibility-services</servicedir>
  <auth>EXTERNAL</auth>
  <listen>unix:tmpdir=/tmp</listen>

  <policy context="default">
    <!-- Allow root to connect -->
    <allow user="root"/>
    <!-- Allow everything to be sent -->
    <allow send_destination="*" eavesdrop="true"/>
    <!-- Allow everything to be received -->
    <allow eavesdrop="true"/>
    <!-- Allow anyone to own anything -->
    <allow own="*" />
    <deny send_interface="org.ally.atspi.Text" send_member=
      "GetStringAtOffset"/>
    <deny send_interface="org.ally.atspi.Text" send_member="GetText"/>
    <deny send_interface="org.ally.atspi.Text" send_member=
      "GetTextBeforeOffset"/>
    <deny send_interface="org.ally.atspi.Text" send_member=
      "GetTextAtOffset"/>
    <deny send_interface="org.ally.atspi.Text" send_member=
      "GetTextAfterOffset"/>
    <deny send_interface="org.ally.atspi.Text" send_member=
      "GetCharacterAtOffset"/>
    <deny send_interface="org.ally.atspi.EditableText" send_member=
      "SetTextContents"/>
    <deny send_interface="org.ally.atspi.EditableText" send_member=
      "InsertText"/>
    <deny send_interface="org.ally.atspi.DeviceEventListener"
      send_member="NotifyEvent"/>
    <deny send_interface="org.ally.atspi.DeviceEventController"
      send_member="RegisterKeystrokeListener"/>
    <deny send_interface="org.ally.atspi.DeviceEventController"
      send_member="RegisterDeviceEventListener"/>
```

```
<deny send_interface="org.ally.atspi.DeviceEventController"
    send_member="GenerateKeyboardEvent"/>
<deny send_interface="org.ally.atspi.DeviceEventController"
    send_member="GenerateMouseEvent"/>
<deny send_interface="org.ally.atspi.Document" send_member=
    "GetAttributeValue"/>
</policy>
```

Секция `<policy>` определяет политику безопасности, которая должна применяться к конкретному набору подключений к шине. Политика состоит из правил `<allow>` (разрешение) и `<deny>` (запрет). Политика, как правило, задается для системной шины и используется для разрешения либо запрета передачи сообщений.

Системная шина имеет политику по умолчанию, которая запрещает отправку сообщений типа `method_call` и получение владения службами на шине (`own`). Остальные действия, в частности, ответы на сообщения (`reply`), получение ошибок (`error`) и сигналов (`signal`) по умолчанию в политике разрешены.

Таким образом, предпочтительнее реализовывать системные службы в виде небольших, целевых программ, которые работают в одном процессе и требуют одного общеизвестного имени (имени службы) на шине. В результате для определения политики достаточно будет создать `<allow>`-правила для разрешений типа `own`, чтобы позволить процессам создавать собственные службы, и добавить параметр `send_destination`, чтобы разрешить трафик от некоторых или всех пользователей к данным службам.

Секции `<policy>` назначается один из четырех параметров:

- `context` — возможные значения: `default`, `mandatory`;
- `at_console` — возможные значения: `true`, `false`;
- `user` — имя пользователя или его идентификатор;
- `group` — имя группы или ее идентификатор.

Политики применяются к соединениям в следующем порядке:

- 1) `all context="default";`
- 2) `all group="connection's user's group";`
- 3) `all user="connection's auth user";`
- 4) `all at_console="true";`
- 5) `all at_console="false";`
- 6) `all context="mandatory".`

Порядок применения политик изменяется в том случае, когда политики пересекаются по условиям применения. Несколько политик с тем же пользователем/группой/контекстом

применяются в порядке их появления в конфигурационном файле.

Возможные параметры правил политик приведены в таблице 16. Параметры правил <deny> определяют необходимость запрета, указанного в параметрах сообщения.

Таблица 16

Параметр	Описание
send_interface	Имя интерфейса отправителя сообщения
send_member	Имя метода или сигнала отправителя сообщения
send_error	Имя ошибки
send_destination	В качестве значения указывается имя. Для правила <deny> параметр означает, что сообщения не могут быть отправлены владельцем данного имени, а не данной службой. То есть, если соединение владеет службами А, В, С, и отправка к А запрещена, то отправка к В или С также будет запрещена
send_type	Возможные значения: method_call, method_return, signal и error. Задаёт тип исходящего сообщения
send_path	Путь вида /path/name отправителя сообщения
receive_interface	Имя интерфейса получателя сообщения
receive_member	Имя метода или сигнала получателя сообщения
receive_error	Имя ошибки
receive_sender	В качестве значения указывается имя. Для правила <deny> параметр означает, что сообщения не могут быть получены владельцем данного имени, а не данной службой
receive_type	Возможные значения: method_call, method_return, signal и error. Задаёт тип получаемого сообщения
receive_path	Путь вида /path/name получателя сообщения

Продолжение таблицы 16

Параметр	Описание
send_requested_reply	Возможные значения: true, false. Параметр работает аналогично параметру eavesdrop: разрешает или запрещает (<allow> или <deny>) сообщение, полученное в качестве ожидаемого ответа (requested_reply), соответственно, предшествует этому сообщение вызова метода (method_call). Данный параметр актуален только для ответных сообщений (error и method_return) и игнорируется для других типов сообщений. Для правила <allow> значение параметра send_requested_reply="true" является значением по умолчанию и указывает на то, что только запрошенные ответы разрешены правилом. Значение параметра send_requested_reply="false" означает, что правило позволяет любой ответ, даже если он не ожидался. Для правила <deny> значение параметра send_requested_reply="false" является значением по умолчанию и указывает на то, что правило работает только когда ответ не был запрошен. Значение параметра send_requested_reply="true" указывает на то, что правило работает всегда, независимо от состояния ожидания ответа
receive_requested_reply	Аналогично параметру send_requested_reply
eavesdrop	Возможные значения: true, false. Прослушивание (Eavesdropping) возникает, когда приложение получает сообщение, адресованное службе, которой не владеет приложение, или ответ на такое сообщение. Для правил <allow> значение параметра eavesdrop="true" указывает на то, что правило будет применяться даже когда осуществляется прослушивание. Значение параметра eavesdrop="false" задано по умолчанию и означает, что правило разрешает те сообщения, которые доставляются только указанному получателю. Для правил <deny> значение параметра eavesdrop="true" указывает на то, что правило будет применяться только тогда, когда выполняется прослушивание. Значение параметра eavesdrop="false" задано по умолчанию и означает, что правило применяется всегда, даже когда не осуществляется прослушивание. Параметр eavesdrop можно комбинировать только с правилами типа приема и передачи (send_* и receive_*)
own	Имя

Окончание таблицы 16

Параметр	Описание
own_prefix	В качестве значения указывается имя. Правило <code><allow own_prefix="a.b"/></code> позволяет владеть именем a.b или любым именем, начинающимся с a.b, например это выполняется для a.b.c или a.b.c.d, но не выполняется для a.bc или a.c. Актуально, когда службы, например Telepathy и ReserveDevice, определяют значения служб для поддеревьев общеизвестных имен, например org.freedesktop.Telepathy.ConnectionManager.<произвольное_продолжение> и org.freedesktop.ReserveDevice1.<произвольное_продолжение>
user	В качестве значения указывается имя пользователя. Правила <code><deny></code> означают, что данный пользователь не может подключиться к шине сообщений
group	В качестве значения указывается имя группы. Действие параметра аналогично параметру user

Запрет пользователя и/или группы необходимо осуществлять в политике `context="default"` или `context="mandatory"`, а не внутри секции `<policy>`.

Отдельное правило `<deny>` может содержать комбинации параметров. В таком случае запрет выполняется только тогда, когда все параметры соответствуют сообщению.

Пример

Правило запрета сообщений от пользователя john:

```
<deny user="john"/>
```

Нельзя включать одновременно параметры `send_*` и `receive_*` в одно правило.

Необходимо с осторожностью использовать параметры `send_interface` и `receive_interface`, т.к. поле интерфейса в сообщениях является необязательным и явное указание запрета интерфейса может привести к блокировке сообщений без интерфейса для всех служб. Всегда следует использовать `send_interface` и `receive_interface` совместно с другими параметрами.

Примеры:

```
1. <deny send_interface="org.foo.Bar"
send_destination="org.foo.Service"/>
```

```
2. <deny send_destination="org.freedesktop.Service"
send_interface="org.freedesktop.System" send_member="Reboot"/>
```

3. `<deny send_destination="org.freedesktop.System"/>`
4. `<deny receive_sender="org.freedesktop.System"/>`
5. `<deny group="enemies"/>`

4.15.7.2. Формирование клиентских политик из политик шины

Обработка файла конфигурации и считывание файлов настроек, в том числе политик в parent. Если в обработчике возникает ошибка, то функция возвращает NULL и, соответственно, ничего не считывается.

Пример

```
get_correct_parser ->
bus_context_new ->
bus_context_reload_config ->
include_file ->

bus_config_load (const DBusString      *file,
                 dbus_bool_t          is_toplevel,
                 const BusConfigParser *parent,
                 DBusError             *error)
```

Инициализация обработчика в функции XML_SetElementHandler:

```
expat_StartElementHandler -> bus_config_parser_start_element ->
start_policy_child -> append_rule_from_element
```

При анализе вызовов формирование политик происходит следующим образом:

```
bus_driver_handle_hello ? bus_connection_complete
process_config_every_time ? bus_connections_reload_policy
bus_connections_reload_policy или bus_connection_complete ->
bus_context_create_client_policy -> bus_policy_create_client_policy
```

Функция `bus_policy_create_client_policy` разбирает политики шины `BusPolicy *` и создает `BusClientPolicy*` исходя из соединения `DBusConnection *`. Таким образом `BusPolicy *` на основе `DBusConnection *` соответствует `BusClientPolicy*`.

4.16. Средства управления мандатными ПРД

Для управления мандатными ПРД в локальной системе используются следующие графические утилиты:

- 1) `fly-fm` («Менеджер файлов») — управление мандатными атрибутами файлов;
- 2) `astra-systemsettings` («Параметры системы») — управление привилегиями и мандатными атрибутами пользователей, работа с пользователями и группами.

Более подробное описание утилит см. в электронной справке.

Для управления мандатными ПРД в локальной системе из командной строки используются следующие инструменты:

- 1) `pdpl-file` — управление мандатными атрибутами файлов, описание приведено в 4.16.1;
- 2) `pdpl-id` — отображение мандатных атрибутов сессии пользователя ОС, описание приведено в 4.16.2;
- 3) `pdpl-init-fs` — сценарий инициализации мандатных атрибутов ФС, описание приведено в 4.16.3;
- 4) `pdpl-ls` — вывод, аналогично стандартной команде `ls`, информации о файлах с отображением мандатных атрибутов, описание приведено в 4.16.4;
- 5) `pdpl-ps` — управление мандатными атрибутами процессов, описание приведено в 4.16.5;
- 6) `pdpl-user` — управление допустимыми уровнями и категориями конфиденциальности пользователей ОС, описание приведено в 4.16.6;
- 7) `pdpl-group` — управление меткой целостности группы пользователей ОС, описание приведено в 4.16.7;
- 8) `pdpl-exec` — запуск процессов в заданном окружении, описание приведено в 4.16.8;
- 9) `sumac` — запуск процесса с заданными уровнем и категорией конфиденциальности в отдельной графической сессии, описание приведено в 4.16.9;
- 10) `sumic` — запуск процесса с пониженной (заданной) меткой целостности, описание приведено в 4.16.10;
- 11) `userlev` — изменение БД уровней конфиденциальности, описание приведено в 4.16.11;
- 12) `usercat` — изменение БД категорий конфиденциальности, описание приведено в 4.16.12.

Для управления мандатными ПРД в режиме ЕПП с развернутым доменом FreeIPA возможно использовать веб-интерфейс FreeIPA либо инструмент командной строки `ipa user-mod`

(подробная информация по использованию инструмента командной строки доступна на справочной странице `ipa help user-mod`).

4.16.1. pdpl-file

Инструмент `pdpl-file` предназначен для управления мандатными атрибутами (меткой безопасности, дополнительными мандатными атрибутами управления доступом и дополнительными атрибутами для МКЦ) сущностей ОС.

Синтаксис команды:

```
sudo pdpl-file [параметр[...]] [уровень_конфиденциальности][:категория_целостности
[:категория_конфиденциальности[:дополнительный_атрибут1,...]
[:линейный_уровень_целостности]]] <сущность>
```

Уровень конфиденциальности и категория целостности могут быть заданы именами или десятичными значениями. Линейный уровень целостности задается десятичным значением. Категория конфиденциальности может быть задана именем или шестнадцатеричным значением (см. 4.2.1, 4.2.2 и 4.3.1).

Пример

Рекурсивно для всех файлов каталога `/tmp` изменить уровень конфиденциальности на Секретно и категорию конфиденциальности на Категория_A (уровень и категория должны быть определены в системе):

```
sudo pdpl-file -Rv Секретно:0:Категория_A /tmp
```

Для присвоения сущности одновременно всех категорий, которые определены в системе, можно использовать значение `-1` для `<категория_конфиденциальности>`.

Пример

```
sudo pdpl-file 1:0:-1 /tmp
```

Дополнительные атрибуты для мандатного управления доступом `ccnr`, `ehole`, `whole` и дополнительные атрибуты для МКЦ `silev`, `irelax`, `ssi` и `iinh` могут быть заданы числовыми значениями или именами через запятую.

Пример

```
sudo pdpl-file 2:0:0:ccnr,irelax /tmp
```

Для присвоения сущности линейного уровня целостности необходимо указать его значение после дополнительных атрибутов.

Примеры:

1. Установка отрицательного линейного уровня целостности на файл:

```
sudo pdpl-file :::-128 file.txt
```

2. Установка нулевого линейного уровня целостности на каталог вместе с атрибутом iinh:

```
sudo pdpl-file :::iinh:0 /tmp
```

Описание параметров инструмента `pdpl-file` приведено в таблице 17.

Таблица 17

Параметр	Описание
<code>-f, --silent, --quiet</code>	Не выводить сообщений об ошибках
<code>-v, --verbose</code>	Выводить диагностические сообщения для каждого файла
<code>-c, --changes</code>	То же, что и <code>--verbose</code> , но сообщать только об изменениях
<code>-u, --unite</code>	Объединить текущую метку безопасности сущности с указанной в качестве аргумента
<code>-s, --subtract</code>	Вычесть из текущей метки безопасности сущности метку безопасности, указанную в качестве аргумента. При этом для итоговой метки безопасности значения задаются по следующим правилам: • уровень конфиденциальности — минимальное значение из текущей метки безопасности и указанной в качестве аргумента; • метка целостности — минимальное значение из текущей метки безопасности и указанной в качестве аргумента (должно быть указано в десятичном виде); • категории конфиденциальности — из текущей метки безопасности вычитаются категории, указанные в качестве аргумента; • дополнительные атрибуты — из текущей метки безопасности вычитаются дополнительные атрибуты, указанные в качестве аргумента
<code>-R, --recursive</code>	Применить рекурсивно

Окончание таблицы 17

Параметр	Описание
-r, --reverse	Сначала файлы в каталоге, потом каталог
-h, --help	Вывести справку и выйти
--version	Вывести информацию о версии и выйти

4.16.2. pdp-id

Инструмент `pdp-id` выводит мандатные атрибуты сессии пользователя ОС.

Синтаксис команды:

`pdp-id [параметры]`

Параметры и их описание приведены в таблице 18.

Таблица 18

Параметр	Описание
-a	Игнорируется, добавлен для совместимости с программой <code>id</code>
-l, --level	Вывести только уровень конфиденциальности
-i, --ilevel	Вывести только категорию целостности
-d, --ilinear	Вывести только линейный уровень целостности
-c, --category	Вывести только категории конфиденциальности
-n, --name	Для параметров <code>-l</code> , <code>-i</code> и <code>-c</code> выводить имена вместо числовых значений
-h, --help	Вывести справку и выйти
--version	Вывести информацию о версии и выйти

При отсутствии параметров инструмент выводит строку текущих мандатных атрибутов сессии.

Пример

Выполнить команду:

`pdp-id`

Вывод команды:

Уровень конф.=2 (Секретно), Уровень целостности=0 (Низкий),
Линейная целостность=(), Категории=0x1 (Категория_A)

Вывод команды показывает, что текущая сессия пользователя имеет уровень конфиденциальности Секретно, минимальную категорию целостности, нулевой линейный уровень целостности и категорию Категория_А.

4.16.3. pdp-init-fs

Сценарий инициализации мандатных атрибутов ФС `pdp-init-fs` вызывается при инициализации и перезапуске системы для установки корректных мандатных атрибутов на системные файловые объекты (файлы и каталоги), начиная с корня ФС.

Для запуска сценария следует выполнить в консоли:

```
sudo pdp-init-fs
```

Сценарий располагается в каталоге `/usr/sbin/` и доступен для правки только администратору.

ВНИМАНИЕ! Настоятельно не рекомендуется вносить изменения в указанный сценарий без необходимости. Изменения требуются только в случае изменения максимального уровня конфиденциальности или максимальной категории целостности в системе.

4.16.4. pdp-ls

Инструмент `pdp-ls` выводит, аналогично стандартной команде `ls`, информацию о файлах и каталогах (по умолчанию о текущем каталоге).

Синтаксис команды:

```
pdp-ls [параметры] [имя_файла]
```

Данный инструмент поддерживает все параметры инструмента `ls` и используется аналогичным образом.

Дополнительно поддерживает параметр `-M` для вывода информации об атрибутах мандатного управления доступом и МКЦ:

```
pdp-ls -M [имя_файла]
```

Также данный инструмент имеет следующие дополнительные особенности:

- если на файле установлены ACL, то к ним добавляется символ «+»;

- если на файле установлена ненулевая метка безопасности, то к ACL добавляется m;
- если на файле установлены списки регистрации событий, то к ACL добавляется a.

Пример

```
pdpl-ls -M
```

Вывод команды:

```
итого 4
drwxr-xr-x--- 2 user user Уровень_0:Низкий:Нет:0x0 common
-rw-r--r--m-- 1 user user Секретно:Низкий:Категория_А, Категория_Б:0x0
file.txt
```

4.16.5. pdpl-ps

Инструмент `pdpl-ps` позволяет считать мандатный контекст безопасности процесса, идентификатор которого задан в качестве значения параметра.

Синтаксис команды:

```
pdpl-ps [-nzhv] <PID>
```

Только администратор может считывать мандатный контекст безопасности произвольного процесса, обычный пользователь может считывать контекст только с собственного процесса, для этого параметр должен иметь нулевое значение.

Параметры инструмента приведены в таблице 19.

Таблица 19

Параметр	Описание
-n, --numeric	Вывести информацию о контексте безопасности в численном виде
-z, --iszero	Если метка безопасности нулевая, то завершиться с кодом 0, иначе с кодом 1. Если не удалось получить метку безопасности процесса, то завершиться с кодом 74
-h, --help	Вывести справку и выйти
--version	Вывести информацию о версии и выйти

4.16.6. pdpl-user

Инструмент `pdpl-user` отображает и изменяет метки безопасности пользователей ОС.

Синтаксис команды:

```
sudo pdpl-user [-dzhv [-l минимальный:максимальный уровень конфиденциальности]
                [-i максимальная метка целостности] [-с минимальная категория:
                максимальная категория]] <пользователь>
```

Параметры инструмента приведены в таблице 20.

Т а б л и ц а 20

Параметр	Описание
<code>-d, --delete</code>	Удалить строку пользователя из файла
<code>-z, --zero</code>	Обнулить значения уровней и категорий
<code>-l, --level</code>	Установить допустимые уровни конфиденциальности
<code>-i, --ilevel</code>	Установить максимальную метку целостности
<code>-m, --maclabel</code>	То же, что и <code>-l</code> (используется для совместимости)
<code>-c, --category</code>	Установить допустимые категории конфиденциальности
<code>-h, --help</code>	Вывести справку и выйти
<code>-v, --version</code>	Вывести информацию о версии и выйти

ВНИМАНИЕ! При назначении уровней и категорий конфиденциальности следует убедиться, что указанные уровни или категории присутствуют в БД уровней или БД категорий конфиденциальности соответственно (см. 4.16.11 и 4.16.12). Если пользователь при входе в систему выберет уровень или категорию конфиденциальности, которые входят в разрешенный диапазон, но отсутствуют в БД уровней или БД категорий конфиденциальности, то вход в систему с выбранными значениями не будет выполнен.

Если для параметров `-m` или `-c` указано одно значение или одно значение с предшествующим двоеточием, то это значение интерпретируется как максимальное значение, если одно значение с последующим двоеточием — как минимальное.

Инструмент `pdpl-user` при успешном выполнении всегда выводит значения установленных допустимых меток безопасности.

Чтобы просмотреть текущие допустимые метки безопасности, выполнить команду без ключей:

```
sudo pdpl-user <пользователь>
```

ВНИМАНИЕ! Значения Низкий и Высокий в выводе команды обозначают метку целостности, т.е. совокупность категории целостности и линейного уровня целостности.

Пример

```
sudo pdpl-user -l Уровень_0:Уровень_3 -i 0 -c 0:Категория_2 user1
```

Команда установит для пользователя `user1` следующие мандатные атрибуты:

- минимальный уровень конфиденциальности — `Уровень_0(0)`;
- максимальный уровень конфиденциальности — `Уровень_3(3)`;
- максимальную метку целостности — `Низкий(0)`;
- минимальную категорию конфиденциальности — `0x0(0)` (без категорий);
- максимальную категорию конфиденциальности — `0x2(2)`.

При этом уровни конфиденциальности `Уровень_0`, `Уровень_3`, категория конфиденциальности `Категория_2` должны быть определены в системе. Значения уровней и категорий могут быть заданы в числовой форме.

4.16.7. pdpl-group

Инструмент `pdpl-group` позволяет устанавливать и просматривать метку целостности группы пользователей ОС (см. 4.3.1).

Синтаксис команды:

```
sudo pdpl-group [параметр] <группа>
```

Имя группы задается в текстовом формате. Описание параметров приведено в таблице 21.

Таблица 21

Параметр	Описание
<code>-d, --delete <группа></code>	Удалить строку группы из файла
<code>-z, --zero <группа></code>	Обнулить значение уровней и категорий целостности группы
<code>-i, --ilevel <метка_целостности> <группа></code>	Установить максимальную метку целостности
<code>-h, --help</code>	Вывести справку по использованию инструмента
<code>-v, --version</code>	Вывести информацию о версии инструмента

Пример

Присвоить группе `group1` категорию целостности, равную 63:

```
sudo pdpl-group -i 63 group1
```

Результат выполнения команды:

```
minimal pdpl: Уровень_0:Низкий:Нет:0x0
0:0:0x0:0x0
maximal pdpl: Уровень_0:Высокий:Нет:0x0
0:63:0x0:0x0
```

4.16.8. pdp-exec

Инструмент `pdp-exec` позволяет администратору запускать процессы в заданном окружении:

- имя пользователя, от имени которого запускается процесс;
- метка безопасности процесса;
- PARSEC-привилегии.

При использовании инструмента `pdp-exec` следует учитывать, что возможен запуск процесса без применения мандатного контекста безопасности, поэтому использование `pdp-exec` должно быть регламентировано и ограничено.

Синтаксис команды:

```
sudo pdp-exec [параметр[параметр...]] [--] <команда_запуска_процесса>
[параметры_запуска_процесса]
```

В случае если с командой заданы параметры ее запуска, то указание символов «--» перед командой обязательно.

Описание основных параметров инструмента `pdp-exec` приведено в таблице 22, описание всех параметров приведено на справочной странице `man pdp-exec`.

Таблица 22

Параметр	Описание
<code>-s <привилегии></code> , <code>--capability=<привилегии></code>	Установить процессу указанные привилегии в качестве эффективных (текущих), наследуемых и разрешенных
<code>-u <имя_пользователя></code> , <code>--user=<имя_пользователя></code>	Запустить процесс от имени указанного пользователя

Окончание таблицы 22

Параметр	Описание
<code>-l <метка_безопасности>, --label=<метка_безопасности></code>	Установить процессу указанную метку безопасности
<code>-v, --version</code>	Вывести информацию о версии и выйти
<code>-h, --help</code>	Вывести справку и выйти

Привилегии задаются в виде битовой маски (как правило, в шестнадцатеричном виде). Соответствие отдельных битов полномочиям приведено на справочной странице `man parsec_capset`, а также в таблице 13.

Метка безопасности задается в виде:

`[уровень_конфиденциальности] [:метка_целостности[:категория_конфиденциальности]]`

Примеры:

1. Перезапустить службу `dbus` с PARSEC-привилегией `PARSEC_CAP_PRIV_SOCKET`:

```
sudo pdp-exec -c 0x100 -- /etc/init.d/dbus restart
```

2. Запустить оболочку `bash` от имени пользователя `secretuser` с PARSEC-привилегией `PARSEC_CAP_SIG` и с меткой безопасности `1:1`:

```
sudo pdp-exec -c 0x40 -u secretuser -l 1:1 -- bash
```

Более подробное описание `pdp-exec` приведено на справочной странице `man pdp-exec`.

4.16.9. sumac

Инструмент `sumac` используется для запуска процесса с заданными уровнем и категорией конфиденциальности в отдельной графической сессии с использованием виртуального графического сервера Xephyr. Пользователь может запускать процесс только в пределах разрешенных ему уровней и категорий.

Для применения инструмента пользователю должна быть назначена привилегия `PARSEC_CAP_SUMAC` (см. 4.7).

Синтаксис команды:

```
sumac [параметры] <команда_запуска_процесса>
```

Параметры инструмента `sumac` приведены в таблице 23.

Таблица 23

Параметр	Описание
<code>-l , --level=<значение></code>	Запустить процесс с указанным уровнем конфиденциальности
<code>-c , --category=<значение></code>	Запустить процесс с указанной категорией конфиденциальности
<code>-i , --stdin=<файл></code>	Перенаправить <code>stdin</code> запущенного процесса в указанный файл. Не используется
<code>-o , --stdout=<файл></code>	Перенаправить <code>stdout</code> запущенного процесса в указанный файл. Не используется
<code>-e , --stderr=<файл></code>	Перенаправить <code>stderr</code> запущенного процесса в указанный файл. Не используется
<code>-x, --xauth</code>	Попытаться создать запись в <code>.Xauthority</code> . В случае неудачи прервать выполнение процесса. Не используется
<code>-h, --help</code>	Вывести справку и выйти
<code>-v, --version</code>	Вывести информацию о версии и выйти

ВНИМАНИЕ! Для использования `sumac` при включенном МКЦ также должно быть включено МКЦ на файловой системе.

Если указанный уровень конфиденциальности выше текущего, т. е. происходит увеличение уровня, то переменные окружения наследуются от текущего процесса. Текущие переменные окружения сбрасываются, чтобы избежать утечки информации. Также при порождении нового процесса закрываются все файловые дескрипторы, метка безопасности которых не совпадает с указанной в командной строке. В том числе закрываются `stdin`, `stdout`, `stderr`. Вследствие этого сообщения об ошибках, которые могут произойти при запуске процесса, не будут отображены. Параметры `-x`, `-i`, `-o` и `-e` ранее использовались для перенаправления `stdin`, `stdout`, `stderr` в файлы. В текущей версии они не выполняют полезных действий, но сохранены для совместимости с предыдущими версиями. Если в качестве значений параметров указаны имена файлов, то эти файлы создаются с нулевым размером.

ВНИМАНИЕ! Запуск процесса с понижением уровня конфиденциальности или с сокращением набора категорий конфиденциальности запрещен для предотвращения утечки информации на более низкие уровни.

Окно графического приложения, запущенного в отдельной сессии на другом уровне, будет иметь цветную рамку, цвет которой соответствует уровню конфиденциальности приложения. Описание цветовой индикации приведено в документе РУСБ.10015-01 93 01 «Операционная система специального назначения «Astra Linux Special Edition». Руководство пользователя».

Пример

Запуск графического приложения `xterm` с уровнем конфиденциальности 2 и категорией конфиденциальности `0xffff`:

```
sumac -l 2 -c 0xffff xterm
```

Запустить процесс с другими уровнем и/или категориями конфиденциальности также возможно с помощью графической утилиты `fly-run` («Запуск приложения», см. электронную справку).

Блокировка использования инструмента `sumac` описана в 16.6.23.

4.16.10. sumic

Инструмент `sumic` позволяет запускать процессы с меткой целостности ниже, чем метка целостности процесса-родителя. При этом запускаемый с использованием `sumic` процесс будет иметь метку целостности не выше метки целостности исполняемого файла, из которого он запущен. При запуске процесса с помощью `sumic` запрещается наследование открытых в процессе-родителе ресурсов, имеющих метку целостности, превышающую метку целостности создаваемого процесса. Запуск графической утилиты с использованием `sumic` выполняется в изолированном X-сервере.

Синтаксис команды:

```
sumic [параметры] [--] <команда_запуска_процесса>
      [параметры_запуска_процесса]
```

Описание параметров инструмента `sumic` приведено в таблице 24.

Таблица 24

Параметр	Описание
<code>-i <категория_целостности></code> <code>[:линейный_уровень_целостности]</code>	Метка целостности, с которой будет запущен указанный процесс. В случае отсутствия параметра процесс будет запущен с нулевой меткой целостности
<code>-v, --version</code>	Вывести информацию о версии и выйти
<code>-h, --help</code>	Вывести справку и выйти

4.16.11. userlev

Инструмент `userlev` предназначен для просмотра и изменения БД уровней конфиденциальности. Вносить изменения в БД уровней конфиденциальности может только администратор. Описание уровня конфиденциальности и его представление в ОС приведено в 4.2.1.

ВНИМАНИЕ! Если пользователь при входе в систему выберет уровень конфиденциальности, который входит в диапазон назначенных ему уровней, но не имеет названия в БД уровней конфиденциальности, то вход в систему с данным уровнем не будет выполнен.

Синтаксис команды:

```
sudo userlev [параметры]
```

Для просмотра всех уровней конфиденциальности выполнить команду без параметров.

Параметры инструмента приведены в таблице 25.

Т а б л и ц а 25

Параметр	Описание
-d, --delete <значение_или_название>	Удалить уровень конфиденциальности из БД. С параметром указать название или значение уровня конфиденциальности
-a, --add <значение> <название>	Добавить новый уровень конфиденциальности в БД. В параметре задать числовое значение и название нового уровня конфиденциальности. Числовое значение задавать в десятичном формате
-r, --rename <новое_название> <текущее_название_или_значение>	Переименовать существующий уровень конфиденциальности. В параметре указать новое название уровня конфиденциальности и значение или название уровня конфиденциальности, который необходимо переименовать
-m, --modify <новое_значение> <текущее_название_или_значение>	Изменить значение уровня конфиденциальности. В параметре указать новое значение уровня конфиденциальности и значение или название уровня конфиденциальности, значение которого необходимо изменить. Числовое значение задавать в десятичном формате
-h, --help	Вывести справку и выйти
--version	Вывести информацию о версии и выйти

Пример

Добавить уровень конфиденциальности с числовым значением 80 и названием <<Секретно>>:

```
sudo userlev -a 80 Секретно
```

4.16.12. usercat

Инструмент `usercat` предназначен для просмотра и изменения БД категорий конфиденциальности. Вносить изменения в БД категорий конфиденциальности может только адми-

нистратор. Описание категорий конфиденциальности и их представление в ОС приведено в 4.2.2.

ВНИМАНИЕ! Если пользователь при входе в систему выберет категорию конфиденциальности, которая входит в диапазон назначенных ему категорий, но не имеет названия в БД категорий конфиденциальности, то вход в систему с данной категорией не будет выполнен.

Синтаксис команды:

```
sudo usercat [параметры]
```

Для просмотра всех категорий конфиденциальности выполнить команду без параметров.

Параметры инструмента приведены в таблице 26.

Таблица 26

Параметр	Описание
-d, --delete <значение_или_название>	Удалить категорию конфиденциальности из БД. С параметром указать название или значение категории конфиденциальности
-a, --add <значение> <название>	Добавить новую категорию конфиденциальности в БД. В параметре задать числовое значение и название новой категории конфиденциальности. Числовое значение задавать в шестнадцатеричном формате без префикса «0x»
-r, --rename <новое_название> <текущее_название_или_значение>	Переименовать существующую категорию конфиденциальности. В параметре указать новое название категории конфиденциальности и значение или название категории конфиденциальности, которую необходимо переименовать
-m, --modify <новое_значение> <текущее_название_или_значение>	Изменить значение категории конфиденциальности. В параметре указать новое значение категории конфиденциальности и значение или название категории конфиденциальности, значение которой необходимо изменить. Числовое значение задавать в шестнадцатеричном формате без префикса «0x»
-h, --help	Вывести справку и выйти
--version	Вывести информацию о версии и выйти

Пример

Добавить категорию конфиденциальности с числовым значением 4 и названием <<Категория_4>>:

```
sudo usercat -a 4 Категория_4
```

4.17. Средства управления привилегиями пользователей и процессов

Для управления привилегиями пользователей и процессов используется графическая утилита `astra-systemsettings` («Параметры системы», см. электронную справку).

Для управления привилегиями пользователей и процессов используются инструменты командной строки `usercaps`, `execaps` и `pscaps`, описание которых приведено, соответственно, в 4.17.1, 4.17.2 и 4.17.3.

ВНИМАНИЕ! Управление привилегиями пользователей выполняется только администратором с максимальной категорией целостности, установленной в ОС.

4.17.1. `usercaps`

Инструмент командной строки `usercaps` позволяет просматривать и устанавливать привилегии пользователей и предназначен для использования администратором через механизм `sudo`.

Каждая привилегия имеет номер, шестнадцатеричное значение и название. Информация о привилегиях отображается в выводе команд:

1) для Linux-привилегий:

```
sudo usercaps -L
```

2) для PARSEC-привилегий:

```
sudo usercaps -M
```

Синтаксис команды:

```
sudo usercaps [параметр] [пользователь]  
sudo usercaps [-l linux-привилегии] [-m PARSEC-привилегии] <пользователь>
```

Указать <linux-привилегии> и <PARSEC-привилегии> в команде можно тремя способами:

- 1) в виде названий привилегий в верхнем или нижнем регистре, разделенных запятой «,» или двоеточием «:»;
- 2) в виде номеров привилегий со знаками «+» или «-», разделенных запятой «,» или двоеточием «:»;
- 3) в виде шестнадцатеричной битовой маски, соответствующей набору привилегий.

Верхний регистр для названий и знак «+» для номеров добавляют пользователю соответствующие привилегии. Нижний регистр и знак «-» снимают соответствующие привилегии с пользователя.

При использовании битовой маски существующий набор привилегий пользователя заменяется на набор, соответствующий указанной битовой маске. Маска может вводиться в упрощенном виде: 150 будет интерпретировано как 0x000150.

Описание параметров `usercaps` приведено в таблице 27.

Таблица 27

Параметр	Описание
<code>-d, --delete</code>	Удалить файл привилегий пользователя из каталога <code>/etc/parsec/capdb</code>
<code>-z, --zero</code>	Сбросить все привилегии пользователя
<code>-f, --full</code>	Присвоить все возможные привилегии пользователю
<code>-l <linux-привилегии>, --linux <linux-привилегии></code>	Изменить Linux-привилегии пользователя
<code>-m <PARSEC-привилегии>, --parsec <PARSEC-привилегии></code>	Изменить PARSEC-привилегии пользователя
<code>-L, --Linux</code>	Вывести список возможных Linux-привилегий
<code>-M, --PARSEC</code>	Вывести список возможных PARSEC-привилегий
<code>-n, --numeric</code>	Вывести привилегии пользователя в шестнадцатеричном формате
<code>-x <маска>, --hex <маска></code>	Вывести PARSEC-привилегии, соответствующие указанной битовой маске. Маска может быть задана в форматах <code>dec</code> , <code>hex</code> или <code>oct</code> следующим образом: 1) 70 — <code>dec</code> ; 2) 0x46 — <code>hex</code> ; 3) 0106 — <code>oct</code>
<code>-h, --help</code>	Вывести справку и выйти
<code>-v, --version</code>	Вывести информацию о версии и выйти

Параметры `-f` и `-z` могут использоваться вместе с дополнительными параметрами `-L` или `-M` для присвоения или сброса у пользователя сразу всех Linux- или PARSEC-привилегий соответственно.

При вызове инструмента без параметров выводится список пользователей с указанием битовых масок их привилегий:

```
sudo usercaps
```

Результат выполнения команды:

```
root -l 0x0 -m 0x0
user -l 0x0 -m 0x123
```

Примеры:

1. Вывести список возможных Linux-привилегий с номерами, шестнадцатеричными значениями и названиями:

```
sudo usercaps -L
```

Вывод команды:

```
...  
23  0x00800000  cap_sys_nice  
...
```

2. Вывести список возможных PARSEC-привилегий с номерами, шестнадцатеричными значениями и названиями:

```
sudo usercaps -M
```

Вывод команды:

```
...  
17  0x020000  parsec_cap_inherit_integrity  
...
```

3. Вывести все привилегии пользователя user1:

```
sudo usercaps user1
```

4. Сбросить все PARSEC-привилегии пользователя user1:

```
sudo usercaps -z -M user1
```

5. Добавить пользователю user1 привилегию parsec_cap_audit (номер привилегии — 1, битовая маска — 0x000002) и снять привилегию parsec_cap_chmac (номер привилегии — 3, битовая маска — 0x000008):

```
sudo usercaps -m PARSEC_CAP_AUDIT,parsec_cap_chmac user1
```

или

```
sudo usercaps -m +1,-3 user1
```


6. Установить пользователю `user1` набор привилегий, соответствующий битовой маске `0x000130`:

```
sudo usercaps -m 130 user1
```

Результат выполнения команды:

```
PARSEC-привилегии пользователя 'user1':
4  0x000010  parsec_cap_ignmaclvl
5  0x000020  parsec_cap_ignmaccat
8  0x000100  parsec_cap_priv_sock
```

Описание `usercaps` также приведено на справочной странице `man usercaps`.

4.17.2. `execaps`

Инструмент командной строки `execaps` может быть использован администратором для запуска процесса с одновременной установкой выбранных PARSEC-привилегий.

Синтаксис команды:

```
sudo execaps [параметр] [-с привилегии] [--] [команда]
```

Привилегии задаются в виде битовой маски (как правило, в шестнадцатеричном виде). Соответствие отдельных битов полномочиям приведено на справочной странице `man parsec_capset`, а также в таблице 13.

В качестве команды может быть задана программа с параметрами ее запуска. Если команда задается с параметрами запуска, то указание символов «--» перед командой обязательно.

Пример

Выполнить команду `pscaps` с привилегией `0x000008`:

```
sudo execaps -c 0x000008 pscaps 0
```

Процесс `pscaps` запустится с заданными привилегиями и отобразит их в выводе:

```
00000008 00000008 00000008
```

Описание основных параметров `execaps` приведено в таблице 28.

Таблица 28

Параметр	Описание
-с <привилегии>, --capability=<привилегии>	Установить процессу указанные привилегии в качестве эффективных (текущих), наследуемых и разрешенных
-f, --force	Вызвать процесс, даже если не удалось установить привилегии
-h, --help	Вывести справку и выйти
-v, --version	Вывести информацию о версии и выйти

Более подробное описание `exescaps` приведено на справочной странице `man exescaps`.

4.17.3. pscaps

Инструмент командной строки `pscaps` применяется для просмотра и установки PARSEC-привилегий процесса.

Синтаксис команды:

```
pscaps <параметр>
pscaps <PID> [текущие_привилегии [разрешенные_привилегии
               [наследуемые_привилегии]]]
```

где <PID> — идентификатор процесса.

Если в качестве параметра указан только идентификатор процесса <PID>, то `pscaps` показывает набор PARSEC-привилегий (указанных в виде битовых масок привилегий) процесса. При указании в качестве <PID> значение 0 будет показан набор привилегий процесса `pscaps`.

При указании в команде битовых масок привилегий (в десятичном или шестнадцатеричном виде) будут изменены привилегии процесса `pscaps`. В этом случае в качестве значения <PID> должно быть указано «0» или идентификатор процесса `pscaps`.

Описание параметров `pscaps` приведено в таблице 29.

Таблица 29

Параметр	Описание
-h, --help	Вывести справку и выйти
-v, --version	Вывести информацию о версии и выйти

Описание `pscaps` также приведено на справочной странице `man pscaps`.

4.18. Мандатное управление доступом в комплексах программ гипертекстовой обработки данных и электронной почты

Обеспечение мандатного управления доступом в комплексе программ гипертекстовой обработки данных и в комплексе программ электронной почты реализовано на основе программного интерфейса библиотек подсистемы безопасности PARSEC.

На сервере комплекса программ гипертекстовой обработки данных и комплекса программ электронной почты при обработке запросов на соединение выполняется получение мандатного контекста соединения, унаследованного от субъекта (процесса). Сокет сервера, ожидающий входящих запросов на соединение, работает в контексте процесса, имеющего привилегию для приема соединений с любыми уровнями секретности.

После установки соединения и успешного прохождения процедуры идентификации и аутентификации пользователя процесс сервера, обрабатывающий запросы пользователя, переключается в контекст безопасности пользователя, сбрасывает привилегии, обрабатывает запросы пользователя и завершается.

В комплексе программ гипертекстовой обработки данных пользователь получает доступ к ресурсам, являющимся объектами ФС. Комплекс программ электронной почты использует технологию maildir, обеспечивающую хранение почтовых сообщений в виде отдельных объектов ФС. Создаваемые файлы почтовых сообщений маркируются метками безопасности, унаследованными от процесса-создателя. Таким образом, в комплексе программ гипертекстовой обработки данных и в комплексе программ электронной почты ресурсы, к которым осуществляется доступ от имени серверных процессов, обрабатывающих запросы пользователей, являются объектами ФС. Следовательно, доступ к защищаемым ресурсам при приеме и обработке запросов пользователей в процессе функционирования комплекса программ гипертекстовой обработки данных и комплекса программ электронной почты подчиняется мандатным ПРД.

4.19. Настройка параметров ядра в загрузчике GRUB 2

В загрузчике GRUB 2 возможно задать параметры командной строки ядра PARSEC, приведенные в таблице 30.

Таблица 30

Параметр	Описание
<code>parsec.max_ilev</code>	Задаёт максимальную категорию целостности (воспринимаемую модулями) в ОС на момент запуска. Допустимые значения от 0 до 255. При установке ОС задается по умолчанию значение 63. ВНИМАНИЕ! Если в системе присутствуют файлы с категорией целостности выше задаваемого в данный момент значения, это может вызвать отказ доступа к данным файлам (сокетам, каталогам и т. д.)
<code>parsec.ccnr_relax</code>	При заданном значении 1 позволяет непривилегированному пользователю выполнять запись файлов с разным уровнем конфиденциальности в контейнер (каталог) с установленным дополнительным мандатным атрибутом <code>ccnr</code>. Значение по умолчанию 0. Параметр не применяется при включенном расширенном режиме МКЦ (см. 4.5)
<code>parsec.noload_files</code>	Запрет загрузки модулей ядра, встроенного программного обеспечения, образов <code>kexes</code>, ключей и сертификатов открытого ключа X.509 привилегированным пользователем с низкой категорией целостности (<code>root, uid=0</code>). Допустимые значения: 1 или 0 (y или n). Значение по умолчанию 1 (загрузка запрещена)
<code>parsec.enable_exec_on_fuse</code>	Если включен запрет установки бита исполнения (см. 16.6.4), то при заданном значении 1 параметр позволяет запускать исполняемые файлы и сценарии из файловых систем, смонтированных с помощью файловой системы FUSE (например, подключенные устройства с файловой системой NTFS, архивы). Значение по умолчанию 0 (запуск запрещен). Если запрет установки бита исполнения выключен, то параметр не работает

Продолжение таблицы 30

Параметр	Описание
<code>parsec.rename_mask</code>	Позволяет управлять механизмом сложения меток безопасности, правил PARSEC-аудита и дополнительного атрибута <code>ssi</code> при замене и перезаписи файлов. Данный механизм, в частности, предотвращает сброс меток безопасности, правил PARSEC-аудита и атрибута <code>ssi</code> у файлов при следующих операциях: 1) редактирование или переименование файлов текстовыми редакторами, использующими временные файлы (изменения записываются во временный файл, исходный файл заменяется временным при сохранении); 2) переименование файла командой: <pre>mv <старое_имя> <новое_имя></pre> Если уже существует файл с указанным новым именем, то он будет перезаписан переименоваемым файлом. Значение параметра представляется в виде битовой маски, в которой каждый бит отвечает за установку соответствующего флага. В системе битовая маска задается суммой соответствующих десятичных значений: 1) первый бит <code>RENAME_MASK_LEV</code> (двоичное значение 00001, десятичное значение 1) — выбор максимального из уровней конфиденциальности; 2) второй бит <code>RENAME_MASK_CAT</code> (двоичное значение 00010, десятичное значение 2) — объединение категорий конфиденциальности (побитовое ИЛИ); 3) третий бит <code>RENAME_MASK_ILEV</code> (двоичное значение 00100, десятичное значение 4) — объединение категорий целостности и выбор максимального линейного уровня целостности (побитовое ИЛИ); 4) четвертый бит <code>RENAME_MASK_FLAGS</code> (двоичное значение 01000, десятичное значение 8) — сохранение атрибута <code>ssi</code> при его наличии у исходного или перезаписываемого файлов; 5) пятый бит (двоичное значение 10000, десятичное значение 16) — сохранение правил PARSEC-аудита при их наличии у исходного или перезаписываемого файла. Если правила установлены для обоих файлов, то сохраняются правила исходного файла. Значение по умолчанию 31 (все биты включены, метки безопасности и дополнительные атрибуты сущностей объединяются)

Окончание таблицы 30

Параметр	Описание
<code>parsec.execstack</code>	Запрет исполнения областей памяти в стеке. Делает невозможным выполнение данных из разделяемых библиотек на страницах памяти, помеченных «только для данных», включая страницы, содержащие стек. Это не предотвращает размещение эксплойта, но запрещает установить разрешение исполнения его кода. Допустимые значения: 1 или 0. Значение по умолчанию 0 (исполнение запрещено). ВНИМАНИЕ! Включение механизма защиты может привести к отказу прикладного ПО, использующего в своей работе исполняемый стек
<code>parsec.mac=0</code>	Выключение мандатного управления доступом. Если параметр отсутствует, то мандатное управление доступом включено. ВНИМАНИЕ! Запрещается непосредственно изменять значение параметра для выключения мандатного управления доступом. Мандатное управление доступом может быть включено или выключено в процессе установки ОС (см. РУСБ.10015-01 95 01-2). Выключение и включение мандатного управления доступом после установки ОС выполняется с помощью инструмента <code>astra-mac-control</code> (см. 16.6.31), а также с помощью модуля «Мандатное управление доступом» в разделе «Управление доступом» графической утилиты <code>astra-systemsettings</code> («Параметры системы»). Описание модуля «Мандатное управление доступом» приведено в электронной справке
<code>parsec.mount_ilev</code>	Включение механизма защиты точек монтирования. Механизм запрещает монтирование, если категория целостности точки монтирования выше категории целостности монтирующего процесса. Допустимые значения: 1 или 0. Значение по умолчанию 0 — механизм защиты точек монтирования выключен

После установки ОС изменение настроек загрузчика GRUB 2 осуществляется в конфигурационном файле `/etc/default/grub`.

Также параметры загрузчика GRUB 2 могут быть изменены с помощью модуля «Загрузчик GRUB2» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings kcm_grub2
```

ВНИМАНИЕ! После внесения изменений в настройки загрузчика GRUB 2 необходимо в ОС от имени администратора выполнить команду:

```
sudo update-grub
```

5. ЗАЩИТА СРЕДЫ ВИРТУАЛИЗАЦИИ

ОС разработана с учетом применения встроенных средств защиты в виртуальной инфраструктуре и включает в свой состав ядро с поддержкой технологии виртуализации¹⁾ Kernel-based Virtual Machine (KVM, гипервизор II-типа). KVM использует технологию аппаратной виртуализации Intel-VT и AMD-V, поддерживаемую процессорами от Intel и AMD соответственно.

В архитектуре KVM виртуальная машина выполняется как обычный процесс ОС, на который распространяется действие функций безопасности ОС. ОС предоставляет возможность создания и защиты среды виртуализации с обеспечением выполнения следующих функций безопасности:

- идентификацию и аутентификацию пользователей;
- доверенную загрузку виртуальных машин;
- контроль целостности;
- регистрацию событий безопасности;
- управление доступом;
- резервное копирование;
- управление потоками информации;
- защиту памяти;
- централизованное управление образами виртуальных машин и виртуальными машинами;
- ограничение программной среды.

Основными средствами, необходимыми для создания среды виртуализации, являются:

- сервер виртуализации `libvirt`;
- программа эмуляции аппаратного обеспечения QEMU.

Управление средой виртуализации обеспечивается инструментом командной строки `virsh` с использованием программного интерфейса `libvirt`, который предоставляет средства создания и управления ВМ: настройка конфигурации и запуск, управление файлами-образами дисковых носителей, управление виртуальными сетевыми адаптерами и сетями, формирование контекста функционирования ВМ в виде процесса ОС.

5.1. Дискреционное управление доступом в среде виртуализации

Работа в среде виртуализации `libvirt` подчиняется правилам дискреционного управления доступом и возможна только после прохождения обязательной процедуры аутентификации.

¹⁾ Недоступно в режиме «Мобильный».

Дискреционное управление доступом при работе с сервером виртуализации `libvirt` осуществляется совместным использованием модуля дискреционного управления доступом `das` и специально разработанного модуля мандатного управления доступом `parsec`, взаимодействующего с подсистемой безопасности `PARSEC` ОС. Модуль поддержки дискреционного управления доступом `das` использует дискреционные атрибуты, состоящие из уникальных идентификаторов. Основанием для принятия решения о предоставлении доступа является сравнение дискреционных атрибутов ВМ и дискреционных атрибутов пользователя с учетом выполняемой операции, а также с учетом роли пользователя.

Дискреционные атрибуты ВМ могут быть динамическими и статическими:

- динамические генерируются в момент запуска ВМ на основе атрибутов запускающего пользователя (его уникального идентификатора);
- статические задаются администратором в конфигурации ВМ и определяют атрибуты ее запуска.

Все операции с виртуальной машиной разделяются на непривилегированные и привилегированные, например операции получения списка виртуальных машин или информации о конфигурации виртуальной машины являются операциями непривилегированными, а операции создания, удаления или изменения конфигурации виртуальных машин — привилегированными.

Привилегированные операции по изменению состава или конфигурации виртуальных машин требуют вхождения пользователя в локальную административную группу, имя которой задается в конфигурационном файле `/etc/libvirt/libvirtd.conf` в качестве значения параметра `admin_group`, значение по умолчанию `libvirt-admin`:

```
admin_group = "libvirt-admin"
```

Непривилегированные действия доступны пользователям группы `libvirt`.

5.2. Мандатное управление доступом к виртуальной машине

При включенном в ОС мандатном управлении доступом (см. раздел 4) работа в среде виртуализации подчиняется правилам мандатного управления доступом и возможна только после прохождения обязательной процедуры аутентификации. Мандатное управление доступом при работе с сервером виртуализации `libvirt` выполняется модулем поддержки мандатного управления доступом `parsec`, разработанным с использованием прикладного программного интерфейса драйверов доступа `libvirt`. Модуль `parsec` использует метку безопасности подсистемы безопасности `PARSEC`.

Формат метки безопасности в среде виртуализации аналогичен принятому в подсистеме PARSEC (см. 4.4), но без линейного уровня целостности:

[уровень_конфиденциальности] : [категория_целостности] :
[категория_конфиденциальности]

Пример

1:16:0

Предоставление доступа к ВМ с учетом выполняемой операции осуществляется по результатам сравнения уровня и категорий конфиденциальности ВМ с уровнем и категориями конфиденциальности сессии пользователя в хостовой ОС (см. 5.2.1). Операции с ВМ разделяются на операции чтения и записи.

К операциям чтения относятся:

- отображение ВМ в списке доступных ВМ в графической утилите `virt-manager`;
- просмотр информации о ВМ и конфигурационного XML-файла ВМ.

К операциям записи относятся:

- создание, удаление или изменение ВМ;
- запуск ВМ;
- доступ к гостевой операционной системе ВМ.

Управление ВМ и предоставление доступа к ВМ осуществляется с помощью средств управления виртуализации. При этом должно быть реализовано ролевое управление доступом в среде виртуализации в соответствии с 5.3.

5.2.1. Применение правил мандатного управления доступом

Мандатное управление доступом к ВМ осуществляется по следующим правилам:

- 1) операция записи разрешена, если совпадают классификационные метки ВМ и сессии пользователя в хостовой ОС;
- 2) операция чтения разрешена, если уровень конфиденциальности ВМ не выше уровня конфиденциальности сессии пользователя в хостовой ОС и набор категорий конфиденциальности ВМ включен в набор категорий конфиденциальности сессии пользователя в хостовой ОС;
- 3) метка безопасности ВМ может быть динамической или статической. По умолчанию ВМ имеет динамическую метку безопасности. До включения ВМ можно изменить тип

ее метки безопасности, например с помощью графической утилиты `virt-manager` («Менеджер виртуальных машин», см. электронную справку). Варианты использования типов метки безопасности приведены в 5.2.2. Значение метки безопасности зависит от ее типа:

а) значение динамической метки безопасности генерируется в момент включения ВМ, при этом уровень и категории конфиденциальности наследуются от сессии пользователя в хостовой ОС, а категория целостности задается на основе значения параметра `ilev_vm` в конфигурационном файле `/etc/libvirt/libvirtd.conf` средства виртуализации;

б) значение статической метки безопасности следует задать в конфигурационном XML-файле до включения ВМ, при этом уровни и категории конфиденциальности ВМ и сессии пользователя в хостовой ОС должны совпадать, категорию целостности возможно указать любую.

ВНИМАНИЕ! Если ВМ была запущена со статической меткой безопасности, в которой установлен ненулевой уровень конфиденциальности, то в дальнейшем невозможно будет указать для этой ВМ динамическую метку безопасности или понизить уровень конфиденциальности в статической метке. Таким образом, в статической метке безопасности ВМ возможно только повысить уровень конфиденциальности — для этого необходимо войти в сессию хостовой ОС с указанием более высокого уровня конфиденциальности и изменить значение уровня конфиденциальности в статической метке ВМ;

4) в сессии пользователя хостовой ОС с нулевой классификационной меткой возможно создать ВМ (конфигурационный XML-файл ВМ) с динамической меткой безопасности или со статической меткой безопасности, задав в ней нулевую классификационную метку;

5) в сессии пользователя хостовой ОС с ненулевой классификационной меткой возможно создать ВМ (конфигурационный XML-файл ВМ) только со статической меткой безопасности, при этом уровень и категории конфиденциальности в статической метке безопасности должны совпадать с уровнем и категориями конфиденциальности, с которыми пользователь выполнил вход в сессию хостовой ОС;

6) уровень и категории конфиденциальности ВМ должны совпадать с уровнем и категориями конфиденциальности информации, обрабатываемой в гостевой операционной системе ВМ;

7) при создании образа ВМ ему присваивается нулевая метка безопасности;

8) при включении ВМ ее метка безопасности присваивается процессу ВМ в хостовой ОС, образу ВМ и устройствам, принадлежащим ВМ;

9) после выключения ВМ образу ВМ присваивается метка безопасности в зависимости от значения параметра `keep_security_label_on_disks` в конфигурационном файле `/etc/libvirt/libvirtd.conf`:

а) если для параметра задано значение 0, то образу ВМ присваивается нулевая метка безопасности;

- б) если для параметра задано значение 1, то для образа ВМ сохраняется метка безопасности, с которой ВМ была запущена;
- 10) при сетевом взаимодействии хостовой ОС и гостевой операционной системы ВМ необходимо настроить механизм переопределения классификационных меток в сетевых пакетах:
 - а) если гостевая операционная система ВМ не реализует мандатное управление доступом самостоятельно и/или не поддерживает классификационные метки по ГОСТ Р 58256-2018, то для ВМ включить принудительное переопределение классификационных меток сетевых пакетов (см. 5.10.3);
 - б) если гостевая операционная система ВМ реализует мандатное управление доступом и поддерживает классификационные метки по ГОСТ Р 58256-2018, то для ВМ настроить сохранение классификационных меток сетевых пакетов (см. 5.10.3).

5.2.2. Порядок запуска ВМ и входа в гостевую операционную систему ВМ

Выбор способа запуска ВМ и типа метки безопасности зависит от цели запуска ВМ. Запускать ВМ следует с учетом организационно-технических мер и в соответствии с политикой разграничения доступа на объекте информатизации одним из разрешенных способов:

- 1) ВМ с динамической меткой безопасности в режиме запуска «только чтение» (см. 5.4) в сессии хостовой ОС с любой классификационной меткой — для запуска в ней общего прикладного программного обеспечения и обработки данных, в том числе с ограниченным доступом, при этом хранение данных в образе ВМ недопустимо;
- 2) ВМ со статической меткой безопасности только в сессии хостовой ОС, классификационная метка которой равна классификационной метке ВМ:
 - а) с нулевой классификационной меткой без режима запуска «только чтение» — для запуска общего прикладного программного обеспечения, обработки и хранения общедоступных данных. Дополнительно для исключения возможности установки в ВМ вредоносного программного обеспечения рекомендуется запускать в режиме «только чтение»;
 - б) с ненулевой классификационной меткой в режиме запуска «только чтение» (см. 5.4) — для запуска специального программного обеспечения, доступ к которому ограничен, и обработки данных с ограниченным доступом. При запуске ВМ исключена возможность хранения данных на образе ВМ и установки в ВМ вредоносного программного обеспечения;
 - в) с ненулевой классификационной меткой с настроенным сохранением значения метки безопасности образа ВМ после ее выключения — для запуска специального программного обеспечения, доступ к которому ограничен, обработки и хранения в образе ВМ данных с ограниченным доступом. При хранении в образе ВМ данных с ограниченным доступом необходимо включить сохранение значения метки безопасности ВМ, для этого указать в конфигурационном

файле средства виртуализации `/etc/libvirt/libvirtd.conf` для параметра `keep_security_label_on_disks` значение 1:

```
keep_security_label_on_disks=1
```

Затем перезапустить службу `libvirtd` командой:

```
sudo systemctl restart libvirtd
```

Если гостевая операционная система ВМ реализует мандатное управление доступом и поддерживает классификационные метки по ГОСТ Р 58256-2018, то входить в сессию гостевой операционной системы ВМ следует с уровнем конфиденциальности, равным уровню конфиденциальности ВМ. Если ВМ запущена с ненулевым уровнем конфиденциальности, то допускается входить в сессию гостевой операционной системы ВМ с нулевым уровнем конфиденциальности только для администрирования гостевой операционной системы ВМ.

Кроме того, если гостевая операционная система ВМ реализует мандатное управление доступом, то включить механизм запрета входить в сессию гостевой операционной системы ВМ с уровнем конфиденциальности выше, чем уровень конфиденциальности ВМ. Для включения механизма раскомментировать в конфигурационном файле средства виртуализации `/etc/libvirt/libvirtd.conf` параметр `set_levels_in_vm` и указать для него значение 1:

```
set_levels_in_vm=1
```

Затем перезапустить службу `libvirtd` командой:

```
sudo systemctl restart libvirtd
```

После включения механизма запрета входить в сессию гостевой операционной системы ВМ с уровнем конфиденциальности выше, чем уровень конфиденциальности ВМ:

- 1) при входе в сессию гостевой операционной системы ВМ доступны для выбора:
 - а) уровень конфиденциальности, который меньше или равен уровню конфиденциальности ВМ;
 - б) категории конфиденциальности, которые включены в набор категорий конфиденциальности ВМ;
- 2) при входе в сессию гостевой операционной системы ВМ недоступны для выбора назначенные учетной записи пользователя гостевой операционной системы:
 - а) уровни конфиденциальности выше, чем у ВМ;
 - б) категории конфиденциальности, которые не включены в набор категорий конфиденциальности ВМ;

- 3) если у учетной записи пользователя гостевой операционной системы ВМ отсутствуют уровни конфиденциальности, которые ниже или равны уровню конфиденциальности ВМ, или отсутствуют категории конфиденциальности, которые включены в набор категорий конфиденциальности ВМ, то с данной учетной записью пользователя невозможно войти в сессию гостевой операционной системы;
- 4) в сессии гостевой операционной системы ВМ возможно настраивать мандатное управление доступом только в пределах классификационной метки ВМ (например, невозможно добавить в гостевой операционной системе или назначить учетной записи пользователя гостевой операционной системы уровни конфиденциальности, которые выше уровня конфиденциальности данной ВМ).

Особенности настройки конфигурации ВМ и среды виртуализации, а также порядок применения перечисленных способов запуска ВМ приводятся в эксплуатационной документации на автоматизированную систему и/или отдельной инструкции по защите информации, подлежащей согласованию с подразделением, ответственным за защиту информации.

5.3. Ролевое управление доступом в среде виртуализации

В ОС ролевое управление доступом при работе с сервером виртуализации `libvirt` реализуется как с использованием драйвера доступа `polkit`, так и с использованием драйвера доступа `parsec`. Ролевое управление доступом обеспечивает разграничение возможностей выполнения привилегированных операций со средствами виртуализации.

Драйвер управления доступом `polkit` применим только в случае, если соединения с `libvirt` ограничены сокетом домена UNIX. Если соединения выполняются с сокетом TCP, идентифицирующая информация субъекта будет недоступна и доступ субъектов доступа к операциям с сервером виртуализации будет запрещен. Если субъект подключается по SSH, то будет идентифицирован локальный пользователь SSH.

Драйвер управления доступом `parsec` не зависит от сокета и позволяет осуществлять удаленные соединения. Проверки разрешений на использование полномочий ролей осуществляются на хосте сервера виртуализации.

Для реализации ролевой политики должен быть установлен пакет `astra-kvm-secure`.

5.3.1. Настройка ролей

Реализация ролевого метода управления доступом подразумевает разграничение доступа по ролям:

- администратор средства виртуализации;
- администратор безопасности средства виртуализации;
- разработчик виртуальной машины;

- администратор виртуальной машины.

Назначение ролей и полномочий осуществляется администратором системы.

Для работы в средстве виртуализации для ролей зарезервированы системные группы `libvirt-dev`, `libvirt-adm`, `libvirt-admin`, `astra-audit` и `libvirt-admvm`.

Роль администратора средства виртуализации позволяет создавать и управлять учетными записями пользователей средств виртуализации, назначать права доступа пользователям средства виртуализации к виртуальным машинам, создавать и удалять виртуальное оборудование, изменять конфигурации виртуального оборудования, управлять доступом виртуальных машин к физическому и виртуальному оборудованию, управлять квотами доступа виртуальных машин к физическому и виртуальному оборудованию, управлять перемещением виртуальных машин, удалять виртуальные машины, запускать и останавливать виртуальные машины, создавать снимки состояния виртуальных машин (включающих файлы конфигурации виртуальной машины, образа виртуальной машины и образа памяти виртуальной машины). Для пользователя с данной ролью необходимо:

1) при использовании драйвера `polkit`:

а) включить его в группы `libvirt-adm`, `libvirt-admin`, `libvirt`, `libvirt-qemu`, `kvm` и `astra-admin`;

б) добавить в `/etc/sudoers` для получения полномочий на применение механизма `sudo`:

- открыть файл:

```
sudo visudo /etc/sudoers
```

- добавить строку:

```
%libvirt-adm ALL=(ALL:ALL) ALL
```

2) при использовании драйвера `parsec`:

а) включить его в группы `libvirt-admin`, `libvirt`, `libvirt-qemu`, `kvm` и `astra-admin`;

б) добавить в `/etc/sudoers` для получения полномочий на применение механизма `sudo`:

- открыть файл:

```
sudo visudo /etc/sudoers
```

- добавить строку:

```
%libvirt-admin ALL=(ALL:ALL) ALL
```

Дополнительно, при включенном МКЦ, пользователю с ролью администратора средства виртуализации необходимо предоставить возможность работы с высокой меткой целостности.

Роль администратора безопасности средства виртуализации позволяет иметь доступ на чтение к журналу событий безопасности средств виртуализации, формировать отчеты с

учетом заданных критериев отбора, осуществлять выгрузку (экспорт) данных из журнала событий безопасности средства виртуализации. Пользователя с данной ролью необходимо включить в группу `astra-audit` и исключить из групп `libvirt`, `libvirt-qemu`, `kvm`.

Роль разработчика виртуальной машины позволяет создавать виртуальные машины и изменять конфигурации виртуальных машин. Пользователя с данной ролью необходимо включить в группы:

- при использовании драйвера `polkit` — в группы `libvirt-dev`, `libvirt-admin`, `libvirt`, `libvirt-qemu` и `kvm`;
- при использовании драйвера `parsec` — в группы `libvirt-dev`, `libvirt`, `libvirt-qemu` и `kvm`.

Роль администратора виртуальной машины позволяет осуществлять доступ пользователя средства виртуализации к виртуальной машине из интерфейса средства виртуализации. Пользователя с данной ролью необходимо включить в группу `libvirt-admvm`. Дополнительно для осуществления доступа к ВМ пользователю из группы `libvirt-admvm` должны быть назначены права доступа к данной ВМ путем настройки правил дискреционной политики доступа (ACL). Для этого необходимо в графической утилите `virt-manager`:

- 1) перейти в свойства ВМ;
- 2) выбрать раздел «Безопасность»;
- 3) во вкладке «Подробности» в таблице «Дискреционный контроль доступа» для пользователя установить разрешение «Использование» в соответствии с рис. 1.

Подробности XML

Модель: DAC
 Тип: ☒ Динамический *i*
☐ Статический *i*
 Метка:

Модель: PARSEC
 Тип: ☒ Динамический *i*
☐ Статический *i*
 Метка:

Дискреционный контроль доступа

Тип	Субъект	Права доступа
Пользователь	test4	Использование

Тип: ☒ Пользователи и группы
☐ Пользователи
☐ Группы
☐ Показывать системные объекты

Режим только чтение
 Тип: ☒ Отключен
☐ Временный снимок
☐ Снимок
☐ Полная копия

Рис. 1

Принятие решения о доступе пользователя к ВМ осуществляется на основе проверки вхождения пользователя в группу `libvirt-admvm` и наличия для пользователя соответствующего права доступа в ACL данной ВМ.

5.3.2. Настройка ролевого управления доступом с использованием драйвера доступа `polkit`

Для включения ролевого управления доступом на основе драйвера доступа `polkit` необходимо в конфигурационном файле `/etc/libvirt/libvirtd.conf` для параметра `access_drivers` задать значение `[ "polkit", "parsec" ]`:

```
access_drivers = [ "polkit", "parsec" ]
```

Затем перезагрузить службу `libvirtd`, выполнив команды:

```
sudo systemctl reload libvirtd
sudo systemctl restart libvirtd
```

При установке средства виртуализации `libvirt` в каталоге `/usr/share/polkit-1/actions/` создаются по умолчанию файлы с описанием возможных привилегированных операций:

- 1) файл `org.libvirt.unix.policy` — описывает два глобальных разрешения по доступу к виртуальной инфраструктуре:
 - `org.libvirt.unix.monitor` — позволяет выполнять мониторинг виртуальной инфраструктуры;
 - `org.libvirt.unix.manage` — позволяет выполнять управление виртуальной инфраструктурой;
- 2) файл `org.libvirt.api.policy` — содержит список операций (например, создание ВМ, удаление сети и т.д.), которые становятся доступны пользователям, если проверка по правилу `org.libvirt.unix.policy` пройдена.

После установки пакета `astra-kvm-secure` в каталоге `/var/lib/polkit-1/localauthority/` создаются правила, реализующие разрешения для групп администраторов в соответствии с их ролями.

В каталоге `/var/lib/polkit-1/localauthority/10-vendor.d/` создается правило `60-libvirt.pkla`, которое позволяет пользователям групп `libvirt-dev`, `libvirt-adm` и `libvirt` осуществлять доступ к операциям по управлению виртуальной инфраструктурой, со следующим содержимым:

```
[Allow group libvirt-dev, libvirt-adm, libvirt managment permission]
Identity=unix-group:unix-group:libvirt;libvirt-dev;unix-group:libvirt-adm;
Action=org.libvirt.unix.manage
ResultAny=yes
ResultInactive=yes
ResultActive=yes
```

В каталоге `/var/lib/polkit-1/localauthority/15-libvirtd.d/` создаются правила вида `<.org.libvirt.api.*.pkla>`, которые определяют разрешения для ролей.

Для редактирования правил используется графическая утилита `fly-admin-policykit-1` (описание утилиты приведено в электронной справке).

5.3.3. Настройка ролевого управления доступом с использованием драйвера доступа `parsec`

Для включения ролевого управления доступом на основе драйвера доступа `parsec`:

- 1) в конфигурационном файле `/etc/libvirt/libvirtd.conf` для параметра `access_drivers` должно быть задано значение `[ "parsec" ]`:

```
access_drivers = [ "parsec" ]
```

2) в конфигурационном файле `/etc/libvirt/parsec_roledb.conf` для параметра `enable` должно быть задано значение 1:

```
enable = 1
```

После выполненных действий необходимо перезагрузить службу `libvirtd`:

```
sudo systemctl reload libvirtd  
sudo systemctl restart libvirtd
```

Конфигурационный файл `/etc/libvirt/parsec_roledb.conf` содержит перечень всех привилегированных операций при работе со средством виртуализации `libvirt`, к которым применяется политика доступа, и разрешения на выполнение данных операций для групп администраторов в соответствии с их ролями, определенные ролевой политикой.

Дополнительно перераспределение полномочий пользователей средства виртуализации в пределах назначенных им ролей осуществляется в конфигурационном файле `/etc/libvirt/parsec_roledb.conf`. Для редактирования конфигурационного файла возможно использовать инструмент `virsh`, для этого выполнить команду:

```
virsh -c qemu:///system config --edit-config /etc/libvirt/parsec_roledb.conf
```

Пример

Запретить пользователю `u_adm1`, реализующему роль администратора средства виртуализации, удалять ВМ. Для этого необходимо отредактировать разрешения для операции `domain_delete` следующим образом:

```
domain_delete = ["g:libvirt-admin", "-u:u_adm1"]
```

При внесении изменений с помощью `virsh` изменения применяются после сохранения конфигурационного файла и перезапуск службы `libvirtd` не требуется.

5.4. Режим «только чтение»

Для исключения возможности установки вредоносного программного обеспечения внутри ВМ и хранения защищаемых данных в виртуальном диске используется режим «только чтение» (режим запрета модификации образа ВМ).

Выбор режима функционирования ВМ «только чтение» осуществляется при запуске ВМ. В зависимости от выбранного типа режима «только чтение» будет создана копия образа ВМ или будут применяться различные варианты создания снимка образа ВМ. После завершения работы ВМ созданная копия/снимок образа ВМ будет удален, в образ ВМ изменения записаны не будут.

Типы режима «только чтение»:

- 1) временный снимок — при запуске ВМ создается временный файл в каталоге размещения образа ВМ, во временном файле фиксируются все результаты работы пользователя в ВМ. После выключения ВМ временный файл удаляется;
- 2) снимок — при запуске ВМ создается снимок ВМ в заданном каталоге, в снимке ВМ фиксируются все результаты работы пользователя в ВМ. После выключения ВМ снимок удаляется;
- 3) полная копия — при запуске ВМ создается копия образа ВМ в заданном каталоге, в копии образа ВМ фиксируются все результаты работы пользователя в ВМ. После выключения ВМ копия образа удаляется.

Каталог размещения копии образа ВМ или снимка ВМ задается в конфигурационном файле `/etc/libvirt/qemu.conf` в параметре `run_images_dir`:

```
run_images_dir = "/var/lib/libvirt/runimages"
```

Создание полной копии образа ВМ для запуска ВМ в режиме «только чтение» может замедлять процесс запуска ВМ по причине копирования образа большого размера, но данный вариант позволяет использовать возможности по сохранению состояния ВМ (снимка ВМ) для последующего его восстановления.

При использовании снимков для запуска ВМ в режиме «только чтение» после любого завершения работы ВМ (выключение ВМ, сбой сервера виртуализации и др.) все результаты работы пользователя в ВМ будут удалены.

5.5. Идентификация и аутентификация пользователей в среде виртуализации

При доступе к виртуальным машинам различается доступ к средству виртуализации `libvirt` для управления виртуальными машинами и доступ непосредственно к рабочему столу виртуальной машины.

Пользователь может выполнить запуск ВМ или получить доступ к ранее запущенной ВМ после прохождения процедуры идентификации и аутентификации в ОС. Доступ предоставляется в соответствии с установленными правилами разграничения доступа. Запущенная виртуальная машина представляет собой процесс ОС, который функционирует от имени учетной записи пользователя с его мандатными атрибутами безопасности.

Вход в систему осуществляется по идентификатору и паролю. При входе в систему осуществляется идентификация и проверка подлинности субъектов доступа процедурой аутентификации. По умолчанию в системе отключен режим автоматического входа по сохраненным учетным данным или без пароля. Настройка входа в систему осуществляется с использованием утилиты `fly-admin-dm`.

Установка пароля пользователя осуществляется администратором с помощью модуля «Пользователи» в разделе «Пользователи и группы» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_users
```

Ограничение количества неуспешных попыток входа и блокирования учетной записи и сеанса доступа пользователя при превышении числа неуспешных попыток аутентификации в системе устанавливается администратором с помощью инструментов управления политикой безопасности.

В ОС обеспечивается возможность смены установленного администратором ОС пароля пользователя после его первичной аутентификации.

Каждому пользователю в ОС по умолчанию присваивается уникальное имя и идентификатор. При попытке ввода неправильного значения идентификатора или пароля пользователя графической подсистемой ОС выводится сообщение о том, что вход в систему не выполнен, и пользователю повторно предлагается ввести правильный идентификатор и пароль.

Блокировка учетных записей пользователей выполняется автоматически по достижении заданного количества следующих подряд неудачных попыток аутентификации пользователя. Разблокировка учетной записи пользователя выполняется автоматически в соответствии с установленным администратором в политике учетных записей временным интервалом.

Защита пароля пользователя при его вводе обеспечивается за счет отображения вводимых символов условными знаками.

Хранение аутентификационной информации пользователей осуществляется с использованием хеш-функции по ГОСТ Р 34.11-94 и по ГОСТ Р 34.11-2012, что обеспечивает защиту аутентификационной информации.

Описание перечисленных выше процедур приведено в разделе 2.

Для аутентификации в условиях ЕПП и при удаленном доступе к серверу виртуализации используются методы SSH-, SASL- или SSL/TLS-аутентификации в соответствии с РУСБ.10015-01 95 01-1.

5.6. Контроль целостности в среде виртуализации

Для осуществления контроля целостности в среде виртуализации используются следующие механизмы:

- 1) механизм регламентного контроля целостности Another File Integrity Checker (AFICK) в соответствии с 5.6.1;
- 2) механизм контроля запуска исполняемых файлов в условиях ЗПС в соответствии с 5.6.2;
- 3) механизм контроля целостности «отпечаток конфигурации» в соответствии с 5.7.2;
- 4) механизм контроля целостности исполняемых файлов гостевой операционной системы в соответствии с 5.7.3;
- 5) механизм контроля целостности образов ВМ в соответствии с 5.7.4;
- 6) механизм контроля целостности областей памяти ВМ (по запросу из гостевой операционной системы) в соответствии с 5.12.

Контроль целостности осуществляется для следующих типов объектов: конфигурации виртуального оборудования виртуальных машин, конфигураций объектов виртуальной инфраструктуры, исполняемых файлов и параметров настройки средств виртуализации, файлов виртуальной базовой системы ввода-вывода (первичного загрузчика виртуальной машины), областей памяти виртуальной машины, исполняемых файлов гостевой операционной системы виртуальной машины.

Регистрация событий безопасности, связанных с целостностью средств виртуализации и виртуальных машин, осуществляется подсистемой регистрации событий в соответствии с 6.5.

5.6.1. Применение регламентного контроля целостности AFICK

Организация регламентного контроля целостности объектов контроля обеспечивается программным средством AFICK путем вычисления контрольных сумм файлов и соответствующих им атрибутов расширенной подсистемы безопасности PARSEC (мандатных атрибутов и атрибутов расширенной подсистемы протоколирования) с последующим сравнением вычисленных значений с эталонными в соответствии с 10.4.

Возможность проведения периодического контроля осуществляется с использованием системного планировщика заданий `cron`. Системный планировщик заданий `cron` также позволяет выполнять проверку целостности объектов контроля в процессе загрузки ОС.

Для контроля целостности необходимо в конфигурационном файле `/etc/afick.conf` задать список файлов и каталогов, которые будет контролировать AFICK.

Контролю подлежат следующие файлы:

- 1) конфигурационные файлы средства виртуализации `libvirt` с расширением `.conf`, расположенные в каталоге `/etc/libvirt/`;
- 2) конфигурационные XML-файлы виртуальных машин, расположенные в каталоге `/etc/libvirt/qemu/`;
- 3) файлы виртуальной базовой системы ввода-вывода с расширением `.fd`, расположенные в каталоге `/usr/share/OVMF/`.

5.6.2. Применение динамического контроля целостности в режиме ЗПС

Инструменты ЗПС предоставляют возможность внедрения цифровой подписи в исполняемые файлы и в расширенные атрибуты ФС, обеспечивая таким образом контроль целостности как в процессе загрузки средства виртуализации, так и динамически в процессе функционирования.

5.6.2.1. Режим контроля неизменности и подлинности загружаемых исполняемых файлов

Для обеспечения контроля целостности исполняемых файлов средства виртуализации должен быть включен и настроен режим запрета запуска исполняемых файлов и разделяемых библиотек с неверными цифровыми подписями или без них в соответствии с 16.1.

Исполняемые файлы средства виртуализации подписаны по умолчанию для контроля запуска в ЗПС.

ВНИМАНИЕ! Для контроля целостности исполняемых файлов гостевой операционной системы рекомендуется в качестве гостевой операционной системы использовать ОС с включенным режимом запрета загрузки на исполнение файлов и разделяемых библиотек с неверными цифровыми подписями, а также без цифровых подписей.

5.6.2.2. Режим контроля целостности файлов при их открытии

Для обеспечения динамического контроля целостности применяется режим запрета открытия поставленных на контроль файлов с неверной цифровой подписью или без цифровой подписи в расширенных атрибутах в соответствии с 5.7.1.

Контролю подлежат следующие файлы:

- 1) конфигурационные файлы средства виртуализации `libvirt` с расширением `.conf`, расположенные в каталоге `/etc/libvirt/`;
- 2) конфигурационные XML-файлы виртуальных машин, расположенные в каталоге `/etc/libvirt/qemu/`;

3) файлы виртуальной базовой системы ввода-вывода с расширением `.fd`, расположенные в каталоге `/usr/share/OVMF/`.

5.7. Доверенная загрузка виртуальных машин

Доверенная загрузка виртуальных машин обеспечивается с использованием средств динамического контроля целостности (создание замкнутой программной среды) в режиме контроля целостности файлов при их открытии на основе цифровой подписи в расширенных атрибутах ФС. При выявлении нарушения целостности конфигурации виртуального оборудования или целостности файлов виртуальной базовой системы ввода-вывода средствами ОС осуществляется блокировка запуска виртуальных машин. Описание применения замкнутой программной среды (ЗПС) приведено в 16.1.

Также в целях блокировки запуска виртуальной машины при выявлении нарушения целостности конфигурации виртуального оборудования данной виртуальной машины или нарушения целостности файлов виртуальной базовой системы ввода-вывода может применяться механизм контроля целостности «отпечаток конфигурации».

5.7.1. Применение режима контроля целостности файлов при их открытии

Блокировка запуска виртуальной машины при выявлении нарушения целостности конфигурации виртуального оборудования или файлов виртуальной базовой системы ввода-вывода осуществляется с использованием средств динамического контроля целостности в режиме запрета открытия поставленных на контроль файлов с неверными цифровыми подписями в расширенных атрибутах ФС или без них в соответствии с 16.1.

Для возможности подписания файлов необходимо создать ключевую пару с помощью утилиты `gpg`, описание настройки модуля `digests_verify`, создания ключевой пары и подписания файлов приведено в 16.1.

5.7.1.1. Контроль файлов конфигурации виртуального оборудования виртуальных машин

На контроль целостности файлов при их открытии на основе цифровой подписи устанавливаются конфигурационные XML-файлы виртуальных машин, расположенные в каталоге `/etc/libvirt/qemu/`.

Для внедрения цифровой подписи в расширенные атрибуты необходимо подписать на созданном ключе конфигурационные XML-файлы ВМ с использованием утилиты `bsign`.

Пример

Для внедрения цифровой подписи в конфигурационный XML-файл VM `alse` необходимо выполнить команду:

```
sudo bsign --sign --xattr /etc/libvirt/qemu/alse.xml
```

Проверить наличие цифровой подписи можно командой:

```
sudo getfattr -dm- /etc/libvirt/qemu/alse.xml
```

или командой:

```
sudo bsign -w /etc/libvirt/qemu/alse.xml
```

Далее выполнить настройку записей имен конфигурационных XML-файлов VM, на которые распространяется проверка цифровой подписи в расширенных атрибутах ФС. Для этого необходимо в файле `/etc/digsig/xattr_control` задать их список.

Пример

Для добавления в список конфигурационного XML-файла `/etc/libvirt/qemu/alse.xml` выполнить:

```
echo '/etc/libvirt/qemu/alse.xml' | sudo tee -a /etc/digsig/xattr_control
```

ВНИМАНИЕ! После внесения изменений в конфигурационный файл `/etc/digsig/xattr_control` необходимо выполнить команду:

```
sudo update-initramfs -u -k all
```

Также возможно внести имя конфигурационного XML-файла VM в `/etc/digsig/xattr_control` с использованием модуля «Замкнутая программная среда» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_digsig
```

Необходимо добавить имя конфигурационного XML-файла в поле «Шаблоны имен файлов в расширенных атрибутах» (рис. 2).

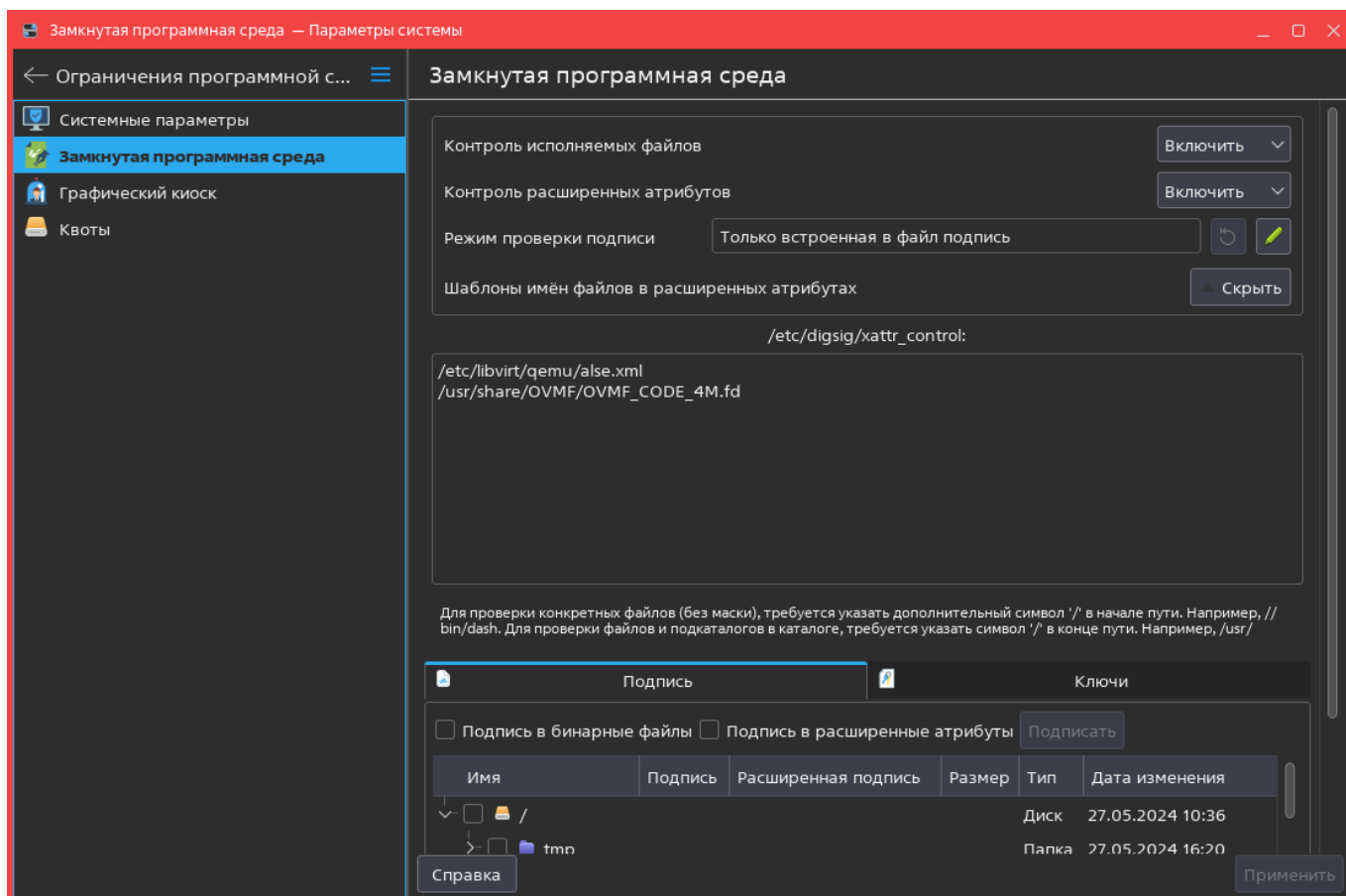


Рис. 2

5.7.1.2. Контроль файлов виртуальной базовой системы ввода-вывода (первичного загрузчика VM)

Загрузчик BIOS встроен в QEMU, исполняемый файл `qemu-system-x86_64` которого по умолчанию подписан для контроля запуска в ЗПС. При этом должен быть включен режим запрета запуска исполняемых файлов и разделяемых библиотек с неверными цифровыми подписями, а также без них (см. 16.1).

В ОС входит пакет `ovmf` (должен быть установлен пакет `astra-kvm-secure`), который предоставляет поддержку загрузчика UEFI и позволяет использовать в VM загрузчик UEFI вместо BIOS. Загрузчик UEFI для VM необходимо выбрать на этапе создания VM. Затем для контроля запуска файлов в режиме ЗПС требуется подписать файлы загрузчика UEFI в расширенных атрибутах ФС.

Для использования в VM загрузчика UEFI необходимо в процессе создания VM с использованием `virt-manager` установить флаг «Проверить конфигурацию перед установкой» и нажать **[Готово]** в соответствии с рис. 3.

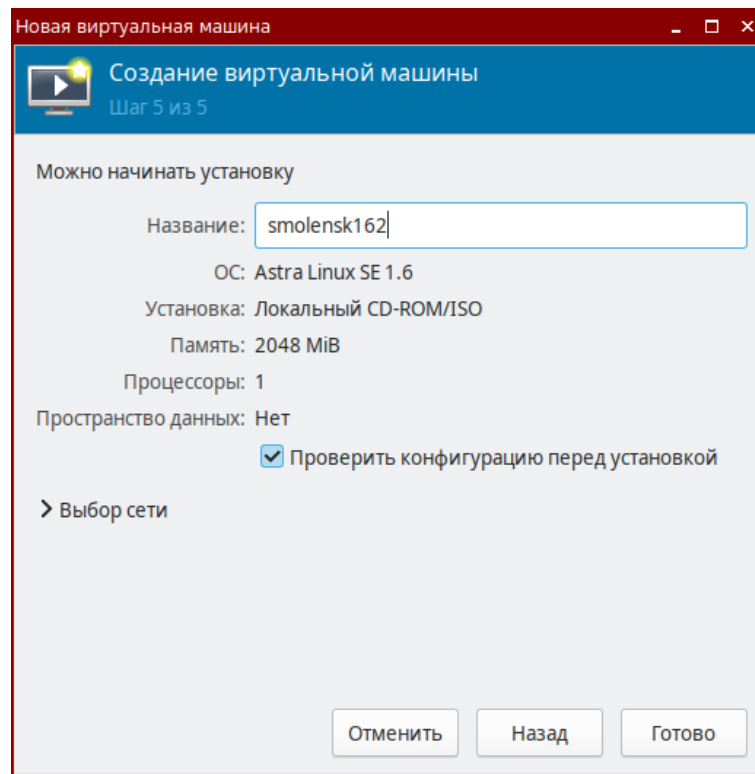


Рис. 3

В открывшемся окне конфигурации ВМ в разделе «Обзор» в секции «Свойства гипервизора» в раскрывающемся списке «Микропрограмма» выбрать необходимую версию загрузчика UEFI, например «UEFI x86-64: /usr/share/OVMF/OVMF_CODE_4M.fd». Затем нажать **[Применить]** и **[Начать установку]**.

Проверить свойства гипервизора возможно в свойствах созданной ВМ в соответствии с рис. 4.

The screenshot shows a window with two tabs: 'Подробности' (Details) and 'XML'. The 'Подробности' tab is active. It contains two sections: 'Основные параметры' (Basic parameters) and 'Свойства гипервизора' (Hypervisor properties).

Основные параметры

- Название: smolensk162
- UUID: a76fdb9b-6469-4c74-93dc-05c7dbfc83e7
- Состояние: Работает (Загружена)
- Заголовок: (empty field)
- Описание: (empty field)

Свойства гипервизора

- Гипервизор: QEMU TCG
- Архитектура: x86_64
- Эмулятор: /usr/bin/qemu-system-x86_64
- Набор микросхем: Q35
- Firmware: UEFI x86_64: /usr/share/OVMF/OVMF_CODE_4M.fd

Рис. 4

Расположенные в каталоге `/usr/share/OVMF` файлы с расширением `.fd` должны быть установлены администратором на контроль целостности, для этого внедрить цифровую подпись в расширенные атрибуты ФС.

Подписывать файлы необходимо на созданном вторичном ключе с использованием утилиты `bsign`.

Пример

Для внедрения цифровой подписи в файл `/usr/share/OVMF/OVMF_CODE_4M.fd` необходимо выполнить команду:

```
sudo bsign --sign --xattr /usr/share/OVMF/OVMF_CODE_4M.fd
```

Проверить наличие цифровой подписи командой:

```
sudo getfattr -dm- /usr/share/OVMF/OVMF_CODE_4M.fd
```

Далее выполнить настройку записей имен файлов, на которые распространяется проверка цифровой подписи в расширенных атрибутах ФС. Для этого можно использовать модуль «Замкнутая программная среда» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Необходимо добавить в поле «Шаблоны имен файлов в расширенных атрибутах» путь к контролируемому файлу загрузчика в соответствии с рис. 2.

5.7.2. Применение механизма контроля целостности с использованием алгоритма работы с контрольными суммами («отпечаток конфигурации»)

Применение механизма контроля целостности «отпечаток конфигурации» обеспечивает динамическое создание хеша для каждого легитимно загружаемого/изменяемого файла конфигурации виртуальной инфраструктуры и сохранение файла с эталонным значением хеша в каталоге с высокой меткой целостности `/var/lib/libvirt/hash/`. Создание хеша осуществляется с применением алгоритма подсчета контрольных сумм по стандарту ГОСТ Р 34.11-2012.

ВНИМАНИЕ! Для обеспечения защиты каталога с эталонными значениями хеша рекомендуется включить мандатный контроль целостности согласно 4.9.

Для использования механизма «отпечаток конфигурации» должен быть установлен пакет `astra-kvm-secure`.

Механизм «отпечаток конфигурации» контролирует следующие файлы конфигураций виртуальной инфраструктуры:

- 1) конфигурационные XML-файлы виртуальных машин, расположенные в каталоге `/etc/libvirt/qemu/`;
- 2) файлы конфигурации других объектов виртуальной инфраструктуры в формате XML (`nwfilter`, `storage`, `cpu_map` и др.);
- 3) конфигурационные файлы средства виртуализации `libvirt` с расширением `.conf`, расположенные в каталоге `/etc/libvirt/` и содержащие параметры настройки средства виртуализации;
- 4) файлы виртуальной базовой системы ввода-вывода (первичного загрузчика VM) с расширением `.fd`, расположенные в каталоге `/usr/share/OVMF/`.

Легитимное управление конфигурациями виртуальной инфраструктуры при включенном механизме «отпечаток конфигурации» возможно только в контексте доверенных процессов средства виртуализации `libvirt` и выполняется с помощью инструментов, представленных в таблице 31.

Таблица 31

Параметр	Описание
<code>virt-manager</code>	Управление средой виртуализации и виртуальными машинами в графическом интерфейсе. Подробная информация об использовании инструмента приведена в электронной справке «Менеджер виртуальных машин»
<code>virsh</code>	Управление средой виртуализации и виртуальными машинами в интерфейсе командной строки. Подробная информация об использовании инструмента приведена на справочной странице <code>man virsh</code>

Окончание таблицы 31

Параметр	Описание
virt-clone	Клонирование ВМ. Подробная информация об использовании инструмента приведена на справочной странице <code>man virt-clone</code>
virt-export	Экспорт ВМ. Подробная информация об использовании инструмента приведена на справочной странице <code>man virt-export</code> , а также в 5.9.7
virt-import	Импорт ВМ из файла архива, полученного при экспорте ВМ. Подробная информация об использовании инструмента приведена на справочной странице <code>man virt-import</code> , а также в 5.9.7
virt-install	Создание ВМ. Подробная информация об использовании инструмента приведена на справочной странице <code>man virt-install</code>
virt-xml	Изменение конфигурации ВМ. Подробная информация об использовании инструмента приведена на справочной странице <code>man virt-xml</code>
virtnbdbackup	Создание резервной копии ВМ. Подробная информация об использовании инструмента приведена на справочной странице <code>man virtnbdbackup</code> , а также в 5.9.5.1
virtnbdrestore	Восстановление ВМ из резервной копии. Подробная информация об использовании инструмента приведена на справочной странице <code>man virtnbdrestore</code> , а также в 5.9.5.2

Для ограничения возможностей изменения конфигураций виртуальной инфраструктуры рекомендуется применять ролевое управление доступом в соответствии с 5.3, при котором изменение конфигураций разрешено только пользователям, обладающим полномочиями роли администратора средства виртуализации (управление конфигурациями средства виртуализации) или роли разработчика виртуальных машин (управление конфигурациями виртуальных машин).

После включения механизма «отпечаток конфигурации» при каждой загрузке службы `libvirt` будет выполняться сравнение хеша загружаемых в память файлов конфигураций с эталонными значениями. Если было осуществлено нелегитимное изменение файла конфигурации, то вычисленное значение хеша не совпадет с эталонным. При нарушении целостности конфигурации ВМ будет недоступна данная ВМ, при нарушении целостности средства виртуализации будет заблокирован доступ к средству виртуализации.

События нарушения целостности конфигурации ВМ и конфигурации средства виртуализации регистрируются подсистемой регистрации событий и записываются в журнал событий (см. 5.8).

Для включения механизма «отпечаток конфигурации» необходимо в конфигурационном файле `/etc/libvirt/libvirtd.conf` для параметра `integrity_control` задать значение 1:

```
integrity_control = 1
```

ВНИМАНИЕ! Включать динамический контроль целостности рекомендуется только после всех настроек системы виртуализации.

Для применения настроек следует перезагрузить службу `libvirtd`:

```
sudo systemctl restart libvirtd
```

После включения механизма «отпечаток конфигурации» легитимное изменение конфигурационных файлов средства виртуализации с расширением `.conf`, расположенных в каталоге `/etc/libvirt/`, возможно с использованием инструмента `virsh` путем выполнения команды:

```
virsh -c qemu:///system config --edit-config <конфигурационный_файл>
```

Пример

Легитимное изменение конфигурационного файла `/etc/libvirt/libvirtd.conf`:

```
virsh -c qemu:///system config --edit-config /etc/libvirt/libvirtd.conf
```

Применение `virsh` для редактирования файлов конфигураций обеспечивает автоматический пересчет хеша после сохранения изменений.

Пересчет контрольной суммы конфигурационных файлов средства виртуализации `libvirt` с расширением `.conf` также возможен с использованием команды:

```
virsh -c qemu:///system config --update-hash <конфигурационный_файл>
```

Пример

Пересчет контрольной суммы конфигурационного файла `/etc/libvirt/libvirtd.conf`:

```
virsh -c qemu:///system config --update-hash /etc/libvirt/libvirtd.conf
```

Легитимное изменение конфигураций ВМ и объектов виртуальной инфраструктуры (nwfilter, storage и др.) возможно путем корректировки XML-файла как с использованием инструмента `virsh`, так и с помощью графической утилиты `virt-manager`.

Для изменения с помощью `virsh` следует выполнить команду:

```
virsh -c qemu:///system edit <имя_ВМ>
```

Для изменения с помощью утилиты `virt-manager` необходимо запустить `virt-manager`, открыть свойства ВМ (см. электронную справку).

Примечание. Для редактирования XML в `virt-manager` необходимо включить в настройках утилиты соответствующую возможность (см. электронную справку).

При использовании `virsh` и `virt-manager` для редактирования файла конфигураций автоматически пересчитывается хеш файла конфигурации после сохранения изменений, а изменения вступают в силу после перезагрузки ВМ.

Создание и удаление объектов виртуальной инфраструктуры при использовании механизма «отпечаток конфигурации» необходимо выполнять с помощью `virsh` или `virt-manager`, обеспечивающих легитимное создание и удаление эталонных значений хеша.

После выключения механизма «отпечаток конфигурации» (задания значения 0 для параметра `integrity_control` в конфигурационном файле `/etc/libvirt/libvirtd.conf`) необходимо, для обеспечения работоспособности средства виртуализации, удалить файлы с рассчитанными значениями хеша из каталога `/var/lib/libvirt/hash/`. Удаление необходимо выполнять администратору.

5.7.3. Применение механизма контроля целостности файлов гостевой операционной системы

Применение механизма контроля целостности обеспечивает выборочную постановку на контроль целостности файлов из состава гостевой операционной системы ВМ. Контроль осуществляется средством виртуализации путем вычисления контрольных сумм файлов в соответствии с ГОСТ Р 34.11-2012 (с длиной хеш-кода 256 или 512 бит) и последующим сравнением вычисленных значений с эталонными.

Для использования механизма в системе должен быть установлен пакет `astra-kvm-secure`.

Контроль файлов гостевой операционной системы может осуществляться как в процессе загрузки ВМ, так и периодически в процессе ее функционирования. Эталонные значения контрольных сумм хранятся в конфигурационных XML-файлах контролируемых ВМ, что позволяет осуществлять контроль и при миграции ВМ.

Использование механизма периодического контроля целостности (в процессе функционирования ВМ) возможно только при условии использования ОС в качестве гостевой операционной системы и установки в ней гостевого агента `qemu-guest-agent` из состава ОС, позволяющего считывать значения контрольных сумм объектов контроля. Установка агента выполняется командой:

```
sudo apt install qemu-guest-agent
```

В случае отсутствия в гостевой операционной системе ВМ установленного агента `qemu-guest-agent` периодический контроль целостности в процессе функционирования ВМ выполняться не будет, а контроль целостности файлов может осуществляться только в процессе загрузки ВМ.

При выявлении нарушения целостности файлов гостевой операционной системы ВМ осуществляется блокировка запуска ВМ и регистрация фактов нарушения целостности объектов контроля (см. 5.8).

Настройка механизма контроля целостности файлов гостевой операционной системы ВМ осуществляется в конфигурационном файле `/etc/libvirt/libvirtd.conf`, при этом эти настройки применяются ко всем ВМ. Настройку рекомендуется выполнять с использованием инструмента `virsh`. Для редактирования конфигурационного файла выполнить на хосте гипервизора команду:

```
virsh -c qemu:///system config --edit-config /etc/libvirt/libvirtd.conf
```

В конфигурационном файле `/etc/libvirt/libvirtd.conf` доступны следующие настройки:

- 1) для включения механизма контроля целостности файлов гостевой операционной системы ВМ в процессе ее загрузки необходимо для параметра `file_integrity_on_startup_VM` задать значение 1:

```
file_integrity_on_startup_VM = 1
```

При этом будет осуществляться регистрация фактов нарушения целостности объектов контроля и блокировка запуска ВМ, если хотя бы у одного установленного на контроль файла гостевой операционной системы ВМ значение контрольной суммы не совпадет с эталонным значением, хранящимся в конфигурационном XML-файле ВМ;

- 2) для включения механизма контроля целостности файлов гостевой операционной системы ВМ в процессе ее функционирования необходимо для параметра `file_integrity_check_period_s` задать значение периода проверки (в секундах):

```
file_integrity_check_period_s = 30
```

При этом будет обеспечиваться только регистрация фактов нарушения целостности объектов контроля без блокировки работы ВМ;

3) для принудительного выключения функционирующей ВМ в случае нарушения целостности установленных на контроль файлов необходимо для параметра `file_integrity_check_shutdown_domain` задать значение 1:

```
file_integrity_check_shutdown_domain = 1
```

При этом будет осуществляться регистрация фактов нарушения целостности объектов контроля с последующим принудительным выключением ВМ, если хотя бы у одного установленного на контроль файла гостевой операционной системы ВМ значение контрольной суммы не совпадет с эталонным значением, хранящимся в конфигурационном XML-файле ВМ.

Для применения настроек следует перезапустить службу `libvirtd`:

```
sudo systemctl restart libvirtd
```

Перед постановкой на контроль целостности файлов гостевой операционной системы ВМ необходимо для каждого файла определить:

- 1) полный путь к контролируемому файлу в гостевой операционной системе ВМ;
- 2) наименование или идентификатор (UUID) раздела диска, на котором хранится контролируемый файл гостевой операционной системы ВМ;
- 3) точку монтирования раздела диска ВМ.

Для определения наименования и идентификатора раздела диска, а также его точки монтирования в гостевой операционной системе ВМ возможно использовать команду, которая отобразит таблицу разделов:

```
lsblk -f
```

Пример

Выполнить команду:

```
lsblk -f
```

Вывод команды:

NAME	FSTYPE	FSVER	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINTS
vda							
-vda1							

```
| -vda2 ext4 1.0 system 44c1be57-1593-4e6d-82a4 25,8G 5% /
```

В выводе команды информация о том, что все файлы размещены на дисковом разделе vda2 с идентификатором 44c1be57-1593-4e6d-82a4, раздел vda2 смонтирован в корневой каталог «/».

Постановку на контроль целостности файлов гостевой операционной системы ВМ должен выполнять администратор средства виртуализации в сессии хостовой ОС с нулевой классификационной меткой при выключенной ВМ.

Для постановки на контроль целостности файлов гостевой операционной системы ВМ необходимо в хостовой ОС для каждого файла выполнить команду:

```
virsh -c qemu:///system file-integrity <имя_ВМ> --path-add <путь_к_файлу> --mount-\
point <точка_монтирования> --disk-name <имя_раздела>
```

или

```
virsh -c qemu:///system file-integrity <имя_ВМ> --path-add <путь_к_файлу> --mount-\
point <точка_монтирования> --disk-uuid <UUID_раздела>
```

где <имя_ВМ> — наименование ВМ;

<путь_к_файлу> — полный путь к файлу, подлежащему контролю, в гостевой ОС;

<точка_монтирования> — точка монтирования раздела, на котором хранится файл в гостевой ОС;

<имя_раздела> — наименование раздела, на котором хранится файл в гостевой ОС;

<UUID_раздела> — идентификатор раздела, на котором хранится файл в гостевой ОС.

Примеры:

1. Постановить файл на контроль с указанием наименования раздела, на котором хранится файл:

```
virsh -c qemu:///system file-integrity else --path-add /bin/afick --mount-\
point / --disk-name vda2
```

2. Постановить файл на контроль с указанием идентификатора раздела, на котором хранится файл:

```
virsh -c qemu:///system file-integrity else --path-add /bin/astra-version --\
mount-point / --disk-uuid 44c1be57-1593-4e6d-82a4
```

После постановки на контроль файлов они будут указаны в виде записей блока `fileintegrity` в конфигурационном XML-файле ВМ.

Пример

Блок `fileintegrity` в конфигурационном XML-файле ВМ с добавленным для контроля файлом в гостевой операционной системе ВМ, контрольная сумма файла не рассчитана:

```
<fileintegrity check="-1">  
<item mount_point="/" disk_name="vda1" alg="gost512" path="/bin/afick">  
</fileintegrity>
```

Параметр `check` в блоке `fileintegrity` отображает состояние механизмов контроля целостности файлов операционной системы ВМ и может принимать следующие значения:

1) значение `-1`:

- а) первичное состояние периодического контроля целостности, когда в операционной системе ВМ еще не определено наличие агента `qemu-guest-agent`. После загрузки ВМ и успешно выполненной проверки наличия агента значение `-1` автоматически изменится на `1`;
- б) агент `qemu-guest-agent` не найден, периодический контроль в процессе функционирования ВМ не будет выполняться;
- в) отключен периодический контроль целостности в процессе функционирования (`file_integrity_check_period_s = 0`);

2) значение `1` — агент найден и периодический контроль целостности файлов операционной системы для данной ВМ выполняется;

3) значение `0` — может быть установлено только администратором для принудительного отключения периодического контроля целостности и контроля целостности при загрузке для конкретной ВМ.

ВНИМАНИЕ! Значения параметра `check=-1` или `check=1` отображают только состояние периодического контроля целостности файлов в процессе функционирования ВМ и не влияют на контроль целостности при загрузке ВМ. Значение параметра `check=0` отображает состояние периодического контроля целостности и контроля целостности при загрузке ВМ.

После настройки контроля целостности и постановки на контроль файлов гостевой операционной системы ВМ перед первой загрузкой ВМ будет выполнена проверка наличия в гостевой операционной системе ВМ файлов, поставленных на контроль. Если проверка пройдена успешно, то рассчитываются и указываются в конфигурационном XML-файле ВМ эталонные значения контрольных сумм контролируемых файлов.

ВНИМАНИЕ! Если после настройки контроля целостности и постановки файлов на контроль первый запуск ВМ будет осуществляться с ненулевым уровнем конфиденциальности,

то расчет эталонных значений контрольных сумм контролируемых файлов должен быть выполнен до первого запуска ВМ с помощью команды:

```
virsh -c qemu:///system file-integrity <имя_ВМ> --update-hash <путь_к_файлу>
```

Пример

Блок `fileintegrity` в конфигурационном XML-файле ВМ с рассчитанной контрольной суммой файла в операционной системе:

```
<fileintegrity check="1">
<item mount_point="/" disk_name="vda1" alg="gost512" path="/bin/afick"
hash="67f6...">
</fileintegrity>
```

Примечание. Для `item` возможно автоматическое добавление параметров `disk_num` (порядковый номер диска в конфигурации ВМ), `part_num` (порядковый номер раздела на диске) и `alg` (длина ключа алгоритма подсчета контрольных сумм). Значения для данных параметров также формируются автоматически и необходимы для корректной работы механизма.

При необходимости установить на контроль целостности новые файлы гостевой операционной системы ВМ администратору средства виртуализации следует при выключенной ВМ и с нулевой классификационной меткой повторить действия по постановке на контроль файлов с помощью команды:

```
virsh -c qemu:///system file-integrity <имя_ВМ> --path-add <путь_к_файлу> \
--mount-point <точка_монтирования> --disk-name <имя_раздела>
```

Перерасчет контрольных сумм в случае изменения объектов контроля выполняется с использованием команды:

```
virsh -c qemu:///system file-integrity <имя_ВМ> --update-hash <путь_к_файлу>
```

Перерасчет контрольных сумм рекомендуется выполнять при выключенной ВМ.

Выключение контроля целостности файлов гостевой операционной системы ВМ выполняется командой:

```
virsh -c qemu:///system file-integrity <имя_ВМ> --stop
```

Более подробная информация об использовании команды `virsh file-integrity` для контроля целостности файлов гостевой операционной системы ВМ приведена на странице помощи:

```
virsh help file-integrity
```

Перечень поставленных на контроль файлов гостевой операционной системы ВМ также можно просмотреть в графической утилите `virt-manager` («Менеджер виртуальных машин», см. электронную справку). В утилите также выводится результат контроля файлов.

5.7.4. Контроль целостности образов ВМ

Механизм контроля целостности образов ВМ позволяет блокировать запуск ВМ из образов, расположенных в файловых хранилищах системы виртуализации, целостность которых была нарушена. Применение механизма возможно только при включенном механизме контроля целостности с использованием алгоритма работы с контрольными суммами (см. 5.7.2).

Примечание. Механизм эффективно работает с небольшими дисками в формате QCOW2, при работе с дисками в формате RAW возможно увеличение затрат времени и ресурсов.

Для включения контроля целостности образов ВМ необходимо в конфигурационном файле `/etc/libvirt/libvirtd.conf` для параметра `integrity_image_control` задать значение 1:

```
integrity_image_control = 1
```

Для применения настроек требуется перезапустить службу `libvirtd`:

```
sudo systemctl restart libvirtd
```

После включения контроля целостности образов ВМ при первом запуске ВМ рассчитывается хеш образа и сохраняется в каталоге `/var/lib/libvirt/hash`. При выключении ВМ рассчитывается хеш измененного образа ВМ и перезаписывается. При каждом последующем включении ВМ выполняется проверка хеша образа. Если вычисленный хеш не соответствует сохраненному в `/var/lib/libvirt/hash`, то запуск ВМ блокируется. Также выполняется перерасчет хеша при создании снимков, сохранении и перезагрузке ВМ.

После включения механизма контроля целостности при создании новой ВМ хеш образа вычисляется сразу при создании образа.

При включенном контроле целостности образов запуск ВМ выполняется дольше, поэтому для более быстрого вычисления рекомендуется использовать алгоритм xxhash128 вместо gost512. Для выбора алгоритма необходимо в `/etc/libvirt/libvirtd.conf` для параметра `hash_type` указать соответствующее значение:

```
hash_type = "xxhash128"
```

5.8. Регистрация событий безопасности в среде виртуализации

5.8.1. Настройка регистрации событий безопасности, связанных с функционированием средства виртуализации

В среде виртуализации обеспечивается регистрация событий безопасности, связанных с функционированием средств виртуализации, и оповещение администратора безопасности средства виртуализации о событиях безопасности. Состав регистрируемой информации соответствует ГОСТ Р 59548-2022.

Для обеспечения регистрации указанных событий безопасности в системе должен быть установлен пакет `astra-kvm-secure`.

Регистрация событий безопасности, настройка реагирования системы на события и информирование администратора осуществляется подсистемой регистрации событий из состава ОС. Описание подсистемы регистрации событий и журнала событий приведено в 6.5.

Также регистрацию событий осуществляет служба `libvirtd`. Запись событий осуществляется в журнал подсистемы регистрации событий и в системный журнал `/var/log/syslog`.

Для каждой записи в журнале регистрируются номер (уникальный идентификатор), дата, время и тип события безопасности.

Просмотр системного журнала осуществляется администратором с использованием графической утилиты `kssystemlog` и инструментов командной строки (`ausearch`, `aureport`, `aulast` и `auvirt`).

Инструмент `auvirt` используется для поиска в журнале аудита записей, созданных `libvirt`, чтобы вывести список сеансов ВМ. Также он выполняет поиск событий, связанных с гостевыми системами (остановка хостовой системы, отказ в доступе), и событий, связанных с QEMU-процессами.

5.8.2. Механизм централизованного сбора журналов с удаленных хостов виртуализации

Для решения задач централизованного сбора журналов с удаленных хостов виртуализации возможно применение встроенных механизмов подсистемы регистрации событий (см. 6.5).

Служба `syslog-ng` из состава подсистемы регистрации событий предоставляет возможность централизованного сбора событий безопасности, связанных с функционированием средства виртуализации, из журналов событий безопасности.

При установке пакета `astra-kvm-secure` создаются конфигурационные файлы для удаленного сбора данных о событиях безопасности.

Для настройки отправки событий с удаленного хоста необходимо в конфигурационном файле `/etc/syslog-ng/conf.d/mod-astra-libvirt-d-remote.conf` в соответствующей строке установить значение IP-адреса сервера сбора журналов (вместо указанного по умолчанию `"127.0.0.1"`) и раскомментировать параметр `destination(libvirt-d-remote-dst)`:

```
destination libvirt-d-remote-dst {
network(
"127.0.0.1" #ip for remote host
port(2222) # for example
transport(tcp) # or udp
);
};

log {
source(s_src);
filter(libvirt-d-syslog-filter);
destination(libvirt-d-remote-dst);
};
```

Для осуществления возможности приема пакетов с удаленных хостов необходимо в конфигурационном файле сервера сбора журналов `/etc/syslog-ng/conf.d/mod-astra-libvirt-d-server.conf` раскомментировать параметр `destination(astra-json-dst)`:

```
source libvirt-d-remote-src {
network(
port(2222) # for example
transport(tcp) # or udp
);
};

log {
source(libvirt-d-remote-src);
filter(libvirt-d-syslog-filter);
parser(libvirt-d-syslog-parser);
parser(astra-libvirt-d-parser);
destination(astra-json-dst);
};
```


Для применения настроек необходимо на удаленных хостах виртуализации и сервере сбора журналов перезагрузить службу `syslog-ng`:

```
sudo systemctl restart syslog-ng
```

События, принятые с удаленных хостов виртуализации, будут регистрироваться в журнал событий (см. 6.5.3).

5.9. Резервное копирование в среде виртуализации

В средства виртуализации из состава ОС встроены механизмы резервного копирования, которые обеспечивают:

- 1) создание резервной копий образа ВМ (см. 5.9.1);
- 2) создание резервной копий конфигурации виртуального оборудования ВМ (см. 5.9.2);
- 3) создание снимков состояния ВМ и восстановление из них (см. 5.9.4);
- 4) создание резервных копий ВМ и восстановление из них (см. 5.9.5)

Резервное копирование параметров настройки средства виртуализации и сведений о событиях безопасности реализуется с использованием средств резервного копирования из состава ОС (см. 5.9.3).

ПО для резервного копирования и восстановления из состава ОС включает инструменты командной строки и распределенные системы управления хранилищами данных:

- 1) комплекс программ Bacula;
- 2) инструмент копирования `rsync`;
- 3) инструменты архивирования и копирования `tar`, `cpio`, `gzip`, `cp`.

Описание указанных инструментов приведено в РУСБ.10015-01 95 01-1.

Также возможны ротации и архивирования журнала событий безопасности средствами ОС, импорт и экспорт ВМ (см. 5.9.6 и 5.9.7 соответственно).

5.9.1. Резервное копирование образов виртуальных машин

Полное резервное копирование текущего состояния образа виртуальной машины выполняется с помощью инструмента `virsh backup-begin` от имени администратора средства виртуализации.

Для резервного копирования используется следующая команда, при этом ВМ должна быть включена:

```
virsh -c qemu:///system backup-begin <имя_ВМ>
```

где <имя_ВМ> — наименование ВМ. Допускается указывать идентификатор ВМ, присвоенный средой виртуализации (ID), или всемирно уникальный идентификатор ВМ (UUID).

После выполнения резервного копирования копии образов сохраняются в каталог /var/lib/libvirt/images/.

Пример

Для выполнения резервного копирования образа ВМ `alse` необходимо от имени администратора средства виртуализации:

1) вывести перечень имеющихся ВМ:

```
virsh -c qemu:///system list --all
```

Вывод команды:

```
ID Имя Состояние
-----
- alse выключен
```

2) запустить ВМ `alse`:

```
virsh -c qemu:///system start alse
```

3) запустить резервное копирование образа ВМ `alse`:

```
virsh -c qemu:///system backup-begin alse
```

4) для проверки статуса задания, запущенного в отношении ВМ `alse`, выполнить команду:

```
virsh -c qemu:///system domjobinfo alse
```

Для отображения информации о завершенных заданиях, запущенных в отношении ВМ `alse`, выполнить команду:

```
virsh -c qemu:///system domjobinfo alse --completed
```

Вывод команды:

```
Тип задания: Завершено
Операция: Резервное копирование
Прошло: 63740 ms
```

Обработано файлов: 19,531 GiB
Осталось файлов: 0,000 B
Всего файлов: 19,531 GiB

5) проверить наличие созданной резервной копии можно командой:

```
sudo ls -lash /var/lib/libvirt/images/
```

Более подробная информация об использовании команды резервного копирования образов VM приведена на странице помощи:

```
virsh help backup-begin
```

5.9.2. Резервное копирование конфигурации виртуального оборудования виртуальных машин

Выполнение резервного копирования конфигурации виртуального оборудования созданных виртуальных машин (существующего XML-файла VM) выполняется с помощью инструмента `virsh dumpxml` от имени администратора средства виртуализации. Для резервного копирования используется команда:

```
virsh -c qemu:///system dumpxml <имя_VM> > <имя_копии>
```

где `<имя_VM>` — наименование VM, допускается указывать идентификатор VM, присвоенный средой виртуализации (ID), или всемирно уникальный идентификатор VM (UUID); `<имя_копии>` — имя файла или полный путь к файлу, в который необходимо сохранить копию конфигурации виртуального оборудования. Если не указан полный путь к файлу, то он будет сохранен в текущем каталоге (в котором выполнялась команда).

Пример

Для выполнения резервного копирования конфигурации VM `alse` необходимо от имени администратора средства виртуализации выполнить команду:

```
virsh -c qemu:///system dumpxml alse > alse_backup.xml
```

Более подробная информация об использовании команды резервного копирования конфигурации VM приведена на странице помощи:

```
virsh help dumpxml
```

5.9.3. Резервное копирование параметров настройки средства виртуализации

Выполнение резервного копирования параметров настройки средства виртуализации осуществляется с использованием инструментов архивирования и копирования из состава ОС: tar, cpio, gzip, cp.

Пример

Для выполнения резервного копирования конфигурационных файлов в каталоге /etc/libvirt/ выполнить команды:

```
cd /etc/libvirt
tar --xattrs --acls -czv conf_backup.tar.gz /etc/libvirt/libvirtd.conf \
    /etc/libvirt/libvirt.conf /etc/libvirt/libvirt-admin.conf
```

5.9.4. Создание снимков текущего состояния машины

Создание снимков текущего состояния ВМ выполняется администратором средства виртуализации при запущенной ВМ с помощью инструмента `virsh snapshot-create`. Снимок состояния ВМ содержит снимки состояния образа ВМ и оперативной памяти ВМ. Восстановление ВМ из снимка выполняется с помощью инструмента `virsh snapshot-revert`.

Для создания снимка ВМ выполнить команду:

```
virsh -c qemu:///system snapshot-create <имя_ВМ> [--disk-only]
```

где <имя_ВМ> — наименование ВМ, допускается указывать идентификатор ВМ, присвоенный средой виртуализации (ID), или всемирно уникальный идентификатор ВМ (UUID); `[-disk-only]` — необязательный параметр, указывает на необходимость сохранения в снимке состояния ВМ только снимка состояния диска, без снимка оперативной памяти ВМ.

Более подробная информация об использовании команды создания снимков приведена на странице помощи:

```
virsh help snapshot-create
```

Созданные снимки ВМ сохраняются в каталог /var/lib/libvirt/qemu/snapshot/<имя_ВМ>/.

Для просмотра существующих снимков ВМ выполнить команду:

```
virsh -c qemu:///system snapshot-list <имя_ВМ>
```

Наименование снимка ВМ задается меткой времени создания снимка в формате Unix.

Если необходимо задать собственное наименование снимка ВМ, то для создания снимка можно воспользоваться командой:

```
virsh -c qemu:///system snapshot-create-as <имя_ВМ> [наименование_снимка] \
[описание_снимка] [--disk-only]
```

Более подробная информация об использовании команды приведена на странице помощи:

```
virsh help snapshot-create-as
```

Для восстановления ВМ из снимка выполнить команду:

```
virsh -c qemu:///system snapshot-revert <имя_ВМ> [наименование_снимка]
```

Более подробная информация об использовании команды восстановления из снимков приведена на странице помощи:

```
virsh help snapshot-revert
```

Для управления снимками ВМ в утилите `virt-manager` необходимо:

- 1) в главном окне утилиты выбрать ВМ;
- 2) нажать **[Открыть]**;
- 3) в открывшемся окне нажать **[Управление снимками]**.

5.9.5. Резервное копирование виртуальных машин

В ОС реализовано резервное копирование ВМ. Резервное копирование ВМ позволяет сохранять конфигурацию и состояние жестких дисков ВМ, при этом ВМ может быть включена или выключена.

Для создания резервных копий ВМ используется инструмент `virtnbdbackup` (см. 5.9.5.1). Для восстановления из резервных копий ВМ используется инструмент `virtnbdrestore` (см. 5.9.5.2).

Создание резервной копии ВМ и восстановление из нее должно выполняться пользователем с назначенной ролью разработчика виртуальной машины или администратора средства виртуализации (см. 5.3.1).

5.9.5.1. Создание резервной копии

Инструмент `virtnbdbackup` позволяет создавать полные и инкрементальные копии ВМ. Синтаксис команды:

```
virtnbdbackup [-h] -U <URI_гипервизора> [параметры_подключения_к_гипервизору]
               d <имя_ВМ> [параметры_копирования] -o <каталог_сохранения_резервной_копии>
               [параметры_резервной_копии]
```

Описание параметров инструмента `virtnbdbackup` и порядок их применения приведены в таблице 32.

Таблица 32

Параметр	Описание
<code>-h, --help</code>	Вывести справку и выйти
<code>--version</code>	Вывести информацию о версии и выйти
Параметры подключения к гипервизору	
<code>-U, --uri <URI_гипервизора></code>	Указать унифицированный идентификатор гипервизора. Значение по умолчанию <code>qemu:///session</code> (локальный гипервизор, запущенный от имени непривилегированного пользователя). В ОС для подключения к локальному гипервизору, запущенному от имени суперпользователя <code>root</code> , необходимо указать значение <code>qemu:///system</code>
<code>--user <пользователь></code>	Указать имя пользователя для подключения к гипервизору. По умолчанию аутентификация не используется. При подключении к локальному гипервизору используется локальная <code>peer-cred</code> аутентификация — имя пользователя предоставляется ядром ОС. Пользователь считается доверенным, так как уже прошел аутентификацию при входе в систему
<code>--ssh-user <удаленный_пользователь></code>	Указать имя пользователя для подключения к гипервизору при использовании удаленной SSH-аутентификации. Значение по умолчанию <code>abi</code>
<code>--ssh-port <SSH_порт></code>	Указать порт для подключения к гипервизору по протоколу SSH. По умолчанию используется порт 22
<code>--password <пароль></code>	Указать пароль пользователя для подключения к гипервизору. По умолчанию аутентификация не используется. При подключении к локальному гипервизору используется локальная <code>peer-cred</code> аутентификация — имя пользователя предоставляется ядром ОС. Пользователь считается доверенным, так как уже прошел аутентификацию при входе в систему
<code>-P, --nbd-port <NBD_порт></code>	Указать порт для подключения к удаленной службе NBD-сервера. По умолчанию используется порт 10809

Продолжение таблицы 32

Параметр	Описание
<code>-I, --nbd-ip <NBD_IP></code>	Указать адрес для подключения к удаленной службе NBD-сервера. По умолчанию используется сетевое имя хостовой машины
<code>--tls</code>	Использовать TLS для подключения к гипервизору. При этом дополнительно необходимо указать путь к сертификатам TLS (см. <code>--tls-cert</code>)
<code>--tls-cert <путь></code>	Указать путь к сертификатам TLS (если при подключении к гипервизору используется TLS, см. <code>--tls</code>). Значение по умолчанию <code>/etc/pki/qemu/</code>
Параметры копирования	
<code>-d, --domain <имя_ВМ></code>	Задать ВМ, для которой требуется выполнить резервное копирование

Продолжение таблицы 32

Параметр	Описание
-l, --level <режим_копирования>	Задать режим копирования: 1) copy — полное резервное копирование выключенной ВМ. В этом режиме контрольная точка не создается и последующее инкрементальное резервное копирование невозможно; 2) full — полное резервное копирование включенной ВМ. В этом режиме будет создана первая контрольная точка для последующих инкрементальных резервных копий этой ВМ; 3) inc — инкрементальное резервное копирование с созданием новой контрольной точки в цепочке инкрементальных резервных копий ВМ; 4) diff — инкрементальное резервное копирование без создания новой контрольной точки; 5) auto — автоматический выбор режима резервного копирования в зависимости от состояния ВМ и наличия полной резервной копии ВМ в заданном каталоге: а) если ВМ работает и в каталоге отсутствует полная резервная копия включенной ВМ, то выполняется полное резервное копирование включенной ВМ; б) если ВМ работает и в каталоге имеется полная резервная копия включенной ВМ, то выполняется инкрементальное резервное копирование с созданием новой контрольной точки; в) если ВМ выключена и в каталоге отсутствует полная резервная копия включенной или выключенной ВМ, то выполняется полное резервное копирование выключенной ВМ. ВНИМАНИЕ! Не допускается выполнять повторное полное резервное копирование включенной или выключенной ВМ в каталог, в котором уже хранится резервная копия. При необходимости повторного полного резервного копирования ВМ предварительно следует очистить каталог сохранения резервной копии, либо создать новый каталог. Кроме того, не допускается сохранение резервных копий разных ВМ в один и тот же каталог. По умолчанию используется режим copy
-S, --start-domain	Проверить состояние ВМ перед выполнением резервного копирования, если эта ВМ выключена, то включить ее. Примечание. После выполнения резервного копирования эта ВМ будет приведена в исходное состояние
p, --printonly	Вывести предполагаемый размер резервной копии и выйти (не выполнять резервное копирование)
s, --startonly	Запустить задачу резервного копирования с помощью инструмента virsh backup-begin

Продолжение таблицы 32

Параметр	Описание
k, --killonly	Принудительно завершить задачу резервного копирования, выполняемую с помощью инструмента <code>virsh backup-begin</code>
-scratchdir <каталог>	Указать временный каталог для выгрузки данных виртуального жесткого диска. По умолчанию используется каталог <code>/var/tmp</code> ВНИМАНИЕ! В ОС доступ к каталогу временных файлов <code>/var/tmp</code> разрешен всем пользователям. В соответствии с правилами дискреционного управления доступом, установленными в ОС, запрещено использовать каталог <code>/var/tmp</code> для выгрузки данных виртуального жесткого диска. При создании резервных копий VM рекомендуется использовать каталог <code>/run/libvirt</code>
f, --socketfile <IPC-сокет>	Указать сокет межпроцессного взаимодействия для службы NBD-сервера. Значение по умолчанию <code>/var/tmp/virtnbdbackup.174352</code>
n, --noprogess	Не использовать шкалу прогресса
w, --worker <количество>	Задать количество обработчиков, используемых для резервного копирования виртуальных жестких дисков. По умолчанию количество обработчиков равно количеству резервируемых виртуальных жестких дисков
e, --strict	Использовать строгий режим резервного копирования — если на каком-либо из этапов возникнет ошибка, то незамедлительно завершить процесс резервного копирования
T, --threshold <порог>	Задать порог необходимости инкрементального резервного копирования (в байтах) — если объем отличающихся данных меньше заданного значения, то резервное копирование не выполняется
L, --syslog	Дополнительно сохранять информацию о событиях, возникающих в ходе резервного копирования, в журнал <code>/var/log/syslog</code>
-quiet	Не выводить в терминал информацию о событиях, возникающих в ходе резервного копирования
v, --verbose	Выводить в терминал отладочную информацию
-nocolor	Не использовать цветное форматирование текста при выводе в терминал
q, --qemu	Использовать инструмент командной строки <code>nbdinfo</code> для определения блоков данных, для которых требуется резервное копирование
Параметры копирования	
-o, --output <каталог_резервной_копии>	Задать каталог сохранения резервной копии

Продолжение таблицы 32

Параметр	Описание
-t, --type <формат_хранения>	Задать формат хранения резервной копии виртуального жесткого диска: • <code>stream</code> — оптимизированный формат, при котором в файле резервной копии хранятся блоки, заполненные данными, и метаданные о смещениях и длинах нулевых блоков. ВНИМАНИЕ! Формат <code>stream</code> поддерживается только для файлов образов дисков формата QCOW2; • <code>raw</code> — формат, при котором в файле резервной копии хранятся необработанные данные, которые являются фактическим представлением виртуального жесткого диска. ВНИМАНИЕ! Для формата <code>raw</code> не поддерживается инкрементальное резервное копирование. Значение по умолчанию <code>stream</code>
-r, --raw	Включить в резервную копию виртуальные жесткие диски, представленные файлами в формате RAW. Примечание. Для резервных копий таких виртуальных жестких дисков будет использован формат хранения <code>raw</code> (см. описание параметра <code>-t</code>)
-i, --include <диск>	Включить в резервную копию только указанный виртуальный жесткий диск
-x, --exclude <диск>	Исключить из резервной копии указанный виртуальный жесткий диск
-F, --freeze-mountpoint <точка_монтирования>	Указать каталог, в который смонтирована файловая система виртуального жесткого диска, доступ к которой необходимо прекратить («заморозить») при создании резервной копии. По умолчанию осуществляется заморозка всех файловых систем, размещенных на виртуальном жестком диске
-z, --compress [уровень_сжатия]	Сжимать резервируемые данные с использованием алгоритма LZ4, при этом можно дополнительно указать уровень сжатия
-C, --checkpointdir <каталог>	Указать каталог размещения контрольных точек. По умолчанию используется каталог <code>checkpoints</code>, размещенный в заданном каталоге сохранения резервной копии

Окончание таблицы 32

Параметр	Описание
<pre>--encryption-key-fingerprint <идентификатор_ключа></pre>	Выполнить защитное преобразование данных, хранящихся на виртуальном жестком диске. При защитном преобразовании использовать указанный открытый ключ RSA. Для защитного преобразования применяется сжатие, в связи с этим необходимо дополнительно указать значение параметра <code>-z</code>. Одновременное использование параметров <code>--encryption-key-fingerprint</code> и <code>--encryption-key-path</code> не допускается. Примечание. Для создания пары ключей RSA и экспорта открытого ключа можно использовать программу «Управление ключами — KGpg». Описание программы приведено в электронной справке
<pre>--encryption-key-path <путь_к_файлу></pre>	Выполнить защитное преобразование данных, хранящихся на виртуальном жестком диске. При защитном преобразовании использовать один из вариантов: • защитное преобразование на основе открытого ключа RSA, который хранится в текстовом формате в указанном файле. Примечание. Для создания пары ключей RSA и экспорта открытого ключа можно использовать программу «Управление ключами — KGpg». Описание программы приведено в электронной справке; • защитное преобразование на основе сертификата открытого ключа, который хранится в указанном файле. Примечание. Для создания и экспорта сертификата можно использовать программу создания и управления центром аутентификации XCA из состава ОС. Описание использования XCA приведено в документе РУСБ.10015-01 95 01-1. Для защитного преобразования применяется сжатие, в связи с этим необходимо дополнительно указать значение параметра <code>-z</code>. Одновременное использование параметров <code>--encryption-key-fingerprint</code> и <code>--encryption-key-path</code> не допускается
<pre>--hash <алгоритм_хеширования></pre>	Задать алгоритм хеширования для генерации контрольных сумм. Возможные значения: • <code>adler32</code> — алгоритм хеширования Adler-32; • <code>xxh64</code> — алгоритм хеширования XXH64. По умолчанию используется алгоритм хеширования Adler-32

Примеры:

1. Для выполнения полного резервного копирования включенной VM `alse` (включая диски, представленные файлами в формате RAW) необходимо выполнить команду:

```
virtnbdbackup -U qemu:///system -d alse -l full -o /home/astra/alse \
--scratchdir /run/libvirt --raw
```

При этом полная резервная копия будет размещена в каталоге `/home/astra/alse`.

2. Для выполнения полного резервного копирования только диска `vda` включенной VM `alse` необходимо выполнить команду:

```
virtnbdbackup -U qemu:///system -d alse -l full -i vda \
-o /home/astra/alse --scratchdir /run/libvirt
```

3. Для предварительной оценки размера инкрементальной резервной копии VM `alse` необходимо выполнить команду:

```
virtnbdbackup -U qemu:///system -d alse -l inc -o /home/astra/alse \
-p --scratchdir /run/libvirt
```

Вывод команды:

```
...
INFO root virtnbdbackup - main [main]: Estimated checkpoint backup
size: [4325376] Bytes
```

Подробная информация об использовании инструмента резервного копирования VM приведена на справочной странице `man virtnbdbackup`.

Можно настроить расписание резервного копирования VM с использованием системного планировщика заданий `cron`. Предварительно пользователю, от имени которого будет осуществляться резервное копирование, следует разрешить формировать список заданий для `cron`. Для этого от имени администратора системы необходимо:

1) добавить пользователя в группу `crontab`:

```
sudo usermod -aG crontab <имя_пользователя>
```

2) добавить имя пользователя в файле `/etc/cron.allow` (при этом файл должен оканчиваться пустой строкой).

Формирование заданий для системного планировщика заданий `cron` выполняется с помощью текстового редактора, который запускается от имени пользователя командой:

```
crontab -e
```

Каждое задание пользователя представляет собой строку, которая содержит шаблон времени и команду:

<минуты> <часы> <день_месяца> <месяц> <день_недели> <команда>

Можно создать любое количество заданий. Задание выполняется тогда, когда текущее время соответствует приведенному шаблону.

В таблице 33 приведены допустимые значения полей шаблона.

Таблица 33

Поле	Диапазон
<минуты>	00—59
<часы>	00—23 (полночь — 00)
<день_месяца>	01—31
<месяц>	01—12
<день_недели>	01—07 (понедельник — 01, воскресенье — 07)

Кроме того, в поле шаблона может быть указан символ «*», который означает соответствие любому корректному значению.

Пример

Задание на резервное копирование VM `alse` ежедневно в 01:30:

```
30 01 * * * virtnbdbackup -S -n -U qemu:///system -d alse -l auto \
-o /home/virt-admin/alse
```

Подробное описание системного планировщика заданий `cron` и команд `crontab` приведено в `man cron`, `man crontab` и `man 5 crontab`.

Резервное копирование VM также возможно выполнить с помощью графической утилиты `virt-manager` («Менеджер виртуальных машин», см. электронную справку).

5.9.5.2. Восстановление из резервной копии

Для восстановления из резервных копий VM используется инструмент `virtnbdrestore`. Синтаксис команды:

```
virtnbdrestore [-h] -U <URI_гипервизора> [параметры_подключения_к_гипервизору]
-i <каталог_хранения_резервной_копии> -o <каталог_восстановленной_VM>
[параметры_восстановления]
```

Описание параметров подключения к гипервизору приведено в таблице 32, описание остальных параметров инструмента `virtnbdrestore` и порядок их применения приведены в таблице 34.

Таблица 34

Параметр	Описание
<code>-h, --help</code>	Вывести справку и выйти
<code>--version</code>	Вывести информацию о версии и выйти
Параметры восстановления	
<code>-a, --action</code> <режим_восстановления>	Задать режим восстановления: • <code>dump</code> — отобразить информацию о всех созданных резервных копиях; • <code>restore</code> — восстановить VM из резервной копий. По умолчанию восстановление производится из последней резервной копии этой VM; • <code>verify</code> — проверить контрольные суммы всех созданных резервных копий VM. По умолчанию используется режим <code>restore</code>
<code>-i, --input</code> <каталог_резервной_копии>	Указать каталог, в котором хранится резервная копия
<code>-o, --output</code> <каталог_восстановленной_VM>	Указать каталог, в который требуется восстановить диски и конфигурационный XML-файл VM
<code>-u, --until</code> <контрольная_точка>	Указать контрольную точку для восстановления VM
<code>-s, --sequence</code> <последовательность>	Указать последовательность резервных копий, которую необходимо использовать для восстановления VM. Например: <pre>-s vda.full.data,vda.inc.virtnbdbackup.1.data, \ vda.diff.1725893111.data</pre> ВНИМАНИЕ! Требуется строго соблюдать очередность резервных копий, в противном случае использование восстановленного образа диска будет невозможно
<code>-d, --disk <диск></code>	Восстановить из резервной копии только указанный виртуальный жесткий диск (например, <code>-d vda</code>)
<code>A, --preallocate</code>	При восстановлении создать предварительно выделяемый (<code>preallocated</code>) файл образа диска
<code>r, --raw</code>	При восстановлении скопировать в каталог восстановленной VM резервные копии виртуальных жестких дисков, представленные файлами в формате RAW
<code>f, --socketfile <IPC-сокет></code>	Указать сокет межпроцессного взаимодействия для службы NBD-сервера. Значение по умолчанию <code>/var/tmp/virtnbdbackup.9553</code>
<code>n, --noprogess</code>	Не использовать шкалу прогресса

Окончание таблицы 34

Параметр	Описание
c, --adjust-config	Выполнить автоматическую корректировку конфигурации VM (изменить каталог размещения файлов образов дисков и изменить наименование VM — добавить префикс restore_)
N, --name <наименование>	Задать новое наименование восстановленной VM
D, --define	Зарегистрировать восстановленную VM на гипервизоре
C, --config-file <файл>	Указать конфигурационный XML-файл VM, который необходимо использовать при восстановлении из резервной копии. По умолчанию используется файл <code>vmconfig.<последняя_контрольная_точка>.xml</code> , размещенный в каталоге резервной копии
B, --buffsize <размер>	Задать размер буфера (в байтах), который используется при проверке восстановленных данных. Значение по умолчанию 8192
L, --logfile <файл>	Указать файл журнала, в который сохраняется информация о событиях, возникающих в ходе восстановления. По умолчанию информация о событиях сохраняется в файл <code>virtnbdrestore.log</code> , размещенный в домашнем каталоге пользователя
v, --verbose	Выводить в терминал отладочную информацию
-nocolor	Не использовать цветное форматирование текста при выводе в терминал

Пример

Для восстановления VM `alse` из резервной копии необходимо выполнить команду:

```
virtnbdrestore -U qemu:///system -i /home/astra/alse/ \
-o /home/astra/ -cD
```

При этом на гипервизоре будет зарегистрирована VM `restore_alse`, файлы образов дисков которой размещены в каталоге `/home/astra/`.

ВНИМАНИЕ! В соответствии с правилами дискреционного управления доступом, установленными в ОС, файлы образов дисков запускаемых VM должны размещаться в зарегистрированных пулах хранения.

Возможный сценарий действий, необходимых для запуска восстановленной VM (требуется выполнить для всех дисков, подключенных к VM):

1) вывести информацию об образах дисков, подключенных к восстановленной VM:

```
virsh -c qemu:///system domblklist <имя_VM>
```

2) отключить диск:

```
virsh -c qemu:///system detach-disk <имя_VM> <наименование_диска> --config
```

3) скопировать (перенести) файл образа диска в каталог пула хранения:

```
sudo cp <файл_образа_диска> <каталог_пула_хранения>
```

4) подключить диск, файл образа которого размещен в каталоге пула хранения:

```
virsh -c qemu:///system attach-disk <имя_ВМ> <файл_образа_диска> \
    <наименование_диска> --config
```

Пример

Корректировка конфигурации ВМ `restore_alse`, файл образа диска которой размещен в каталоге `/home/astra/`:

1) вывести информацию об образах дисков, подключенных к ВМ `restore_alse`:

```
virsh -c qemu:///system domblklist restore_alse
```

Вывод команды:

Назначение	Источник
vda	/home/astra/alse.qcow2

2) из конфигурации ВМ `restore_alse` исключить диск с наименованием `vda`:

```
virsh -c qemu:///system detach-disk restore_alse vda --config
```

3) скопировать файл образа диска `/home/astra/alse.qcow2` в каталог пула хранения `default` (копии файла образа диска будет задано наименование `restore_alse.qcow2`):

```
sudo cp /home/astra/alse.qcow2 \
    /var/lib/libvirt/images/restore_alse.qcow2
```

4) подключить диск, файл образа которого размещен в каталоге пула хранения, к ВМ `restore_alse` (в конфигурации ВМ диску будет задано наименование `vda`):

```
virsh -c qemu:///system attach-disk restore_alse \
    /var/lib/libvirt/images/restore_alse.qcow2 vda --config
```

5) вывести информацию об образах дисков, подключенных к ВМ `restore_alse`:

```
virsh -c qemu:///system domblklist restore_alse
```

Вывод команды:

Назначение	Источник
vda	/var/lib/libvirt/images/restore_alse.qcow2

Подробная информация об использовании инструмента восстановления из резервной копии ВМ приведена на справочной странице `man virtbdrestore`.

Восстановление ВМ из резервной копии также возможно выполнить с помощью графической утилиты `virt-manager` («Менеджер виртуальных машин», см. электронную справку).

5.9.6. Резервное копирование и полная очистка журналов

Резервное копирование сведений о событиях безопасности реализуется с использованием встроенных в ОС средств ротации и архивирования журналов `logrotate`. Настройка ротации возможна с использованием утилиты `fly-admin-events` («Настройка регистрации системных событий»), описание утилиты приведено в электронной справке.

Запустить ротацию журнала возможно вручную, без ожидания установленного в `cron` правила, для этого выполнить команду:

```
sudo logrotate /etc/logrotate.d/syslog-ng-mod-astra
```

5.9.7. Экспорт и импорт виртуальных машин

В ОС реализован экспорт имеющегося экземпляра ВМ с использованием библиотек сервера виртуализации `libvirt`. В результате экспорта будет сформирован архив, содержащий копии образов дисков, снимков состояний и файла конфигурации виртуального оборудования заданной ВМ. Полученный архив может быть использован для формирования новой (импорта) ВМ с идентичной конфигурацией виртуального оборудования, имя импортируемой ВМ будет соответствовать имени архива. При этом значения параметров, которые должны быть уникальными, будут автоматически переопределены для исключения коллизий.

Экспортируемая ВМ должна быть выключена.

Экспорт ВМ выполняется от имени администратора средства виртуализации с использованием инструмента командной строки `virt-export`:

```
virt-export --connect qemu:///system --original <имя_ВМ> --name <имя_архива> \
    --path <каталог>
```

где `<имя_ВМ>` — наименование экспортируемой ВМ;

`<имя_архива>` — наименование копии экспортируемой ВМ;

`<каталог>` — каталог сохранения архива с копией экспортируемой ВМ.

Более подробная информация об использовании инструмента командной строки `virt-export` приведена на справочной странице:

```
man virt-export
```

Для экспорта имеющегося экземпляра ВМ с помощью графической утилиты `virt-manager` необходимо в главном окне утилиты открыть контекстное меню ВМ и выбрать «Экспортировать». В открывшемся окне в поле «Укажите путь к пространству хранения» задать каталог сохранения архива с копией экспортируемой ВМ и нажать **[Экспортировать]**.

Для импорта экземпляра ВМ можно воспользоваться инструментом командной строки `virt-import`:

```
virt-import --connect qemu:///system --original <имя_архива> --pool <пул>
```

где `<имя_архива>` — файл архива копии ВМ, которую необходимо импортировать. Если файл архива не размещен в текущем каталоге, то необходимо указать путь к нему; `<пул>` — наименование пула, в котором необходимо разместить файлы образов дисков новой ВМ.

Более подробная информация об использовании инструмента командной строки `virt-import` приведена на справочной странице страницы:

```
man virt-import
```

Для импорта экземпляра ВМ с помощью графической утилиты `virt-manager` необходимо в главном окне утилиты открыть контекстное меню подключения «QEMU/KVM» и выбрать «Импортировать». В открывшемся окне в поле «Укажите путь к пространству хранения» указать путь к архиву копии ВМ, которую необходимо импортировать, и нажать **[Импортировать]**.

5.10. Управление потоками информации в среде виртуализации

Управление потоками информации между виртуальными машинами и информационными (автоматизированными) системами на канальном и сетевом уровнях реализуется штатными средствами ОС, реализующими технологию виртуальных локальных сетей (VLAN) стандарта IEEE 802.1q.

Для управления потоками информации в виртуальной инфраструктуре применяются следующие механизмы, обеспечивающие сетевую фильтрацию:

1) драйвер виртуальных сетей `libvirt` — предоставляет изолированное мостовое устройство (без подключения физических сетевых карт), к которому подключены гостевые TAP-устройства. Для ВМ разрешается обмен сетевым трафиком между собой и с хостом. Для виртуальной сети возможны три конфигурации:

- а) «Изолированный» (`isolated`) — весь внешний трафик полностью блокируется;
- б) NAT — исходящий трафик во внешнюю сеть разрешен, но замаскирован;
- в) «Маршрутизация» (`forward`) — исходящий трафик в локальную сеть разрешен;

2) драйвер сетевых фильтров `libvirt` — обеспечивает полностью настраиваемую сетевую фильтрацию трафика на гостевых сетевых картах с использованием сетевых фильтров `nwfilter`. Наборы правил для управления трафиком определяются на уровне хоста. Затем наборы правил связываются с определенными гостевыми сетевыми картами;

3) изоляция сетей с помощью VLAN — программный многоуровневый коммутатор Open vSwitch (OVS) для виртуальных сетей обеспечивает изоляцию сети с помощью VLAN путем тегирования портов, а также фильтрацию сетевого трафика. Описание настройки и работы OVS приведено в РУСБ.10015-01 95 01-1.

Также средствами ОС осуществляется управление сетевыми потоками между гипервизором и ВМ в условиях мандатного управления доступом (см. 5.10.3).

5.10.1. Управление сетевыми фильтрами `nwfilter`

Контроль взаимодействия виртуальных машин между собой реализуется с использованием сетевых фильтров `nwfilter`. Сетевые фильтры предоставляют возможность настраивать и применять правила фильтрации сетевого трафика на виртуальных машинах, а также управлять параметрами входящего и исходящего сетевого трафика. Поскольку правила фильтрации нельзя обойти внутри виртуальной машины, это делает их обязательными при использовании виртуальной машины.

Управление фильтрами выполняется как автономными объектами верхнего уровня и осуществляется администратором средства виртуализации с использованием интерфейсов управления средствами виртуализации `libvirt`. В сетевых фильтрах могут быть настроены правила фильтрации сетевого трафика ВМ отдельно для каждого сетевого интерфейса через конфигурационный XML-файл ВМ. Правила применяются на хосте при запуске виртуальной машины и могут быть изменены во время работы виртуальной машины (путем изменения XML-описания сетевого фильтра).

Несколько виртуальных машин могут использовать один и тот же сетевой фильтр. При изменении такого фильтра обновляются правила фильтрации сетевого трафика всех запущенных виртуальных машин, в которых применяется данный сетевой фильтр.

Также возможно настраивать правила фильтрации сетевого трафика на отдельных сетевых интерфейсах, настроенных для определенных типов сетевых конфигураций виртуальных машин (`network`, `ethernet` в режиме моста, `bridge`).

Сетевые фильтры задаются в формате XML и располагаются в каталоге `/etc/libvirt/nwfilter/`.

Пример

```
<filter name='no-spamming' chain='XXXX'>
```

```

<uuid>d217f2d7-5a04-0e01-8b98-ec2743436b74</uuid>
<rule ...>
....
</rule>
<filterref filter='XXXX'/>
</filter>

```

Каждый фильтр имеет имя и UUID, которые служат уникальными идентификаторами. Фильтр может содержать `<rule>`, которые используются для фактического определения сетевых элементов управления. Элемент `<rule>` имеет параметры `action` (действие), `direction` (направление) и необязательный `priority` (приоритет).

Пример

```
<rule action='drop' direction='out' priority='500'>
```

В правилах используются элементы сопоставления с конкретным протоколом. Поддерживаемые протоколы: `mac`, `arp`, `rarp`, `ip`, `ipv6`, `tcp/ip`, `icmp/ip`, `igmp/ip`, `udp/ip`, `udplite/ip`, `esp/ip`, `ah/ip`, `sctp/ip`, `tcp/ipv6`, `icmp/ipv6`, `igmp/ipv6`, `udp/ipv6`, `udplite/ipv6`, `esp/ipv6`, `ah/ipv6`, `sctp/ipv6`. Каждый протокол определяет, что допустимо указывать внутри элемента `<rule>`. Запись имеет вид:

```
<protocol match='yes|no' attribute1='value1' attribute2='value2'/>
```

Пример

Запись для протокола TCP с портами 0-1023:

```
<tcp match='yes' srcportstart='0' srcportend='1023'/>
```

Набор правил по умолчанию можно посмотреть, выполнив команду:

```
virsh -c qemu:///system nwfilter-list
```

Перечень фильтров, автоматически устанавливаемых вместе с `libvirt`, приведен в таблице 35.

Таблица 35

Фильтр	Описание
no-arp-spoofing	Запретить виртуальной машине подделку ARP-трафика. Фильтр разрешает только сообщения запросов и ответов ARP и обеспечивает, чтобы эти пакеты содержали MAC-адреса и IP-адреса виртуальной машины
allow-arp	Разрешить трафик ARP в обоих направлениях
allow-ipv4	Разрешить трафик IPv4 в обоих направлениях
allow-ipv6	Разрешить трафик IPv6 в обоих направлениях
allow-incoming-ipv4	Разрешить входящий трафик IPv4
allow-incoming-ipv6	Разрешить входящий трафик IPv6
allow-dhcp	Разрешить виртуальной машине запрашивать IP-адрес через DHCP (с любого DHCP-сервера)
allow-dhcpv6	Аналогично allow-dhcp, но для DHCPv6
allow-dhcp-server	Разрешить виртуальной машине запрашивать IP-адрес с указанного DHCP-сервера. Десятичный IP-адрес DHCP-сервера с точками должен быть указан в ссылке на этот фильтр. Имя переменной должно быть DHCPSEVER
allow-dhcpv6-server	Аналогично allow-dhcp-server, но для DHCPv6
no-ip-spoofing	Запретить виртуальной машине отправлять пакеты IPv4 с исходным IP-адресом, отличным от адреса в пакете
no-ipv6-spoofing	Аналогичен no-ip-spoofing, но для IPv6
no-ip-multicast	Запретить виртуальной машине отправлять многоадресные IP-пакеты
no-ipv6-multicast	Аналогичен no-ip-multicast, но для IPv6
clean-traffic	Запретить виртуальной машине подделку MAC, IP и ARP. Данный фильтр ссылается на остальные фильтры
clean-traffic-gateway	Запретить трафик между VM на основе их MAC-адресов. Данный фильтр ссылается на другие фильтры
no-arp-ip-spoofing	Запретить VM подмену ARP-трафика. Фильтр разрешает отправку сообщений запросов и ответов ARP только в том случае, если пакеты содержат IP-адрес VM. Данный фильтр используется фильтром no-arp-spoofing
no-arp-mac-spoofing	Запретить VM подмену ARP-трафика. Фильтр разрешает отправку сообщений запросов и ответов ARP только в том случае, если пакеты содержат MAC-адрес VM. Данный фильтр используется фильтром no-arp-spoofing
no-mac-broadcast	Запретить VM отправлять широковещательные кадры (MAC-адрес назначения ff:ff:ff:ff:ff:ff)
no-mac-spoofing	Запретить подмену MAC-адреса VM
no-other-l2-traffic	Запретить весь трафик канального уровня (L2), кроме трафика, для которого указаны правила (т.е. запретить весь трафик, кроме ARP и IPv4/v6)

Окончание таблицы 35

Фильтр	Описание
no-other-rarp-traffic	Позволяют VM определить собственный IP-адрес с помощью протокола RARP, но блокируют весь остальной трафик RARP
qemu-announce-self-rarp	
qemu-announce-self	

Большинство фильтров по умолчанию являются составными (используют другие фильтры по умолчанию). При этом фильтр `clean-traffic` объединяет все фильтры в один фильтр, который затем можно связать с сетевой картой VM. Такое применение позволяет предотвратить самые распространенные атаки: подделка MAC, IP и ARP.

Посмотреть конфигурацию фильтра можно, выполнив команду:

```
virsh -c qemu:///system nwfilter-dumpxml <filter_name>|<UUID>
```

Для обновления фильтров рекомендуется использовать инструмент `virsh nwfilter-define`, что гарантирует корректное применение правил `iptables/ebtables` для VM.

Для управления сетевыми фильтрами используется набор инструментов `virsh`:

- 1) `virsh nwfilter-define` — позволяет определить или обновить сетевой фильтр из XML-файла;
- 2) `virsh nwfilter-undefine` — позволяет отменить определение сетевого фильтра;
- 3) `virsh nwfilter-dumpxml` — позволяет вывести информацию о сетевом фильтре в формате XML;
- 4) `virsh nwfilter-list` — выводит список сетевых фильтров;
- 5) `virsh nwfilter-edit` — позволяет изменить конфигурацию XML для сетевого фильтра.

Для применения правила фильтрации требуется в XML-файл VM добавить ссылку на фильтр.

Пример

Связать фильтр `clean-traffic` с VM. Для этого необходимо отредактировать XML-файл VM, включив в `<interface>` ссылку на фильтр `<filterref>`, а также указав IP-адрес, который разрешено использовать VM:

```
<interface type='bridge'>
<mac address='52:54:00:56:44:32'/>
<source bridge='br1'/>
<ip address='10.33.8.131'/>
```

```
<target dev='vnet0'/>
<model type='virtio'/>
<filterref filter='clean-traffic'/>
</interface>
```

Если IP-адрес не указан, драйвер сетевого фильтра активирует learning mode (режим обучения). Для этого используется модуль libpcap, который отслеживает сетевой трафик, отправляемый VM, и пытается определить первый используемый им IP-адрес. В результате будет заблокирован трафик на этот адрес.

Драйвер сетевого фильтра использует комбинацию ebtables, iptables и ip6tables в зависимости от того, на какие протоколы ссылается фильтр. Правила фильтрации clean-traffic требуют использования только ebtables. При указании протоколов tcp, udp и др. (например, если добавить новый фильтр, блокирующий порт 25, для предотвращения спама по электронной почте) будет также использоваться iptables.

Пример

Фильтр сетевого трафика, предотвращающий подмену IP-адреса со стороны VM:

```
<filter name='no-ip-spoofing' chain='ipv4-ip' priority='-710'>
<uuid>64f897b5-4ca1-46ee-b459-68fe2df325c1</uuid>
<rule action='return' direction='out' priority='100'>
<ip srcipaddr='0.0.0.0' protocol='udp'/>
</rule>
<rule action='return' direction='out' priority='500'>
<ip srcipaddr='$IP'/>
</rule>
<rule action='drop' direction='out' priority='1000'>
</filter>
```

Данный фильтр реализует отбрасывание трафика, если IP-адрес (указанный через значение переменной IP) в исходящем IP-пакете не соответствует ожидаемому. Исключением являются исходящие UDP-пакеты с IP-адресом 0.0.0.0 (пакет с начальным запросом к DHCP-серверу). Пакеты, соответствующие правилам, будут возвращены в исходный фильтр.

5.10.2. Реализация собственных правил

На хосте вся фильтрация трафика, осуществляемая подсистемой сетевых фильтров libvirt, сначала проходит через поддержку фильтрации, реализованную ebtables, и только затем через фильтры iptables или ip6tables. Если в дереве фильтров есть правила с протоколами mac, stp, vlan arp, rarp, ipv4 или ipv6, то правила ebtables будут созданы автоматически.

Роль атрибута цепочки в XML-файле сетевого фильтра заключается в том, что внутри создается определяемая пользователем таблица `ebtables`, которая затем, например если указана цепочка `ARP`, получает весь трафик `ARP`, исходящий от `VM` или направляющийся к ней. Далее в корневой цепочке интерфейса генерируется правило, которое направляет весь `IPv4`-трафик в заданную пользователем цепочку. Следовательно, все правила трафика `ARP` затем должны быть помещены в фильтры, определяющие эту цепочку. Этот тип ветвления в пользовательские таблицы поддерживается только при фильтрации на уровне `ebtables`.

Можно создавать несколько цепочек для одного и того же протокола. Для этого имя цепочки должно иметь префикс одного из ранее перечисленных протоколов. Чтобы создать дополнительную цепочку для обработки `ARP`-трафика, можно указать цепочку с именем `arp-test`.

Например, можно фильтровать `UDP`-трафик по исходным и конечным портам, используя фильтр `IP`-протокола и указывая атрибуты для протокола, исходного и конечного `IP`-адресов и портов `UDP`-пакетов, которые должны быть приняты. Это позволяет фильтровать `UDP`-трафик с помощью `ebtables`. Однако после того, как пакет `IP` или `IPv6` (например, `UDP`) прошел уровень `ebtables` и если в дереве фильтров есть хотя бы одно правило, которое создает экземпляры правил `iptables` или `ip6tables`, необходимо наличие правила, разрешающего прохождение пакета `UDP`. Этого можно добиться с помощью правила, содержащего соответствующий узел фильтрации трафика `udp` или `udp-ipv6`.

Пример

Для `VM test` с интерфейсом `eth0` создать фильтр `test-eth0`, отвечающий следующим требованиям:

- 1) предотвращает подделку `MAC`, `IP` и `ARP`;
- 2) открывает на `VM` только порты 22 и 80 протокола `TCP`;
- 3) позволяет `VM` отправлять `ping` с интерфейса, но не позволяет получать `ping` на интерфейс;
- 4) позволяет `VM` выполнять поиск `DNS` (`UDP` к порту 53).

Требование по предотвращению подделки выполняется существующим фильтром `clean-traffic`, поэтому возможно сослаться на фильтр `libvirt` из создаваемого пользовательского фильтра.

Чтобы включить трафик для `TCP`-портов 22 и 80, необходимо добавить два правила для включения этого типа трафика. Чтобы разрешить виртуальной машине отправлять `ping`, необходимо добавить правило для `ICMP`-трафика. Возможно разрешить инициировать общий `ICMP`-трафик с `VM`, а не только эхо-запросы `ICMP` и ответные сообщения. Чтобы в дальнейшем запретить остальной исходящий и входящий трафик для `VM`, потребуется добавить правило, которое отбрасывает весь другой трафик.

Для создания фильтра необходимо:

1) создать файл фильтра:

```
touch test-eth0.xml
```

Файл будет создан в каталоге выполнения команды;

2) добавить в созданный файл XML-код:

```
<filter name='test-eth0'>
<!-- reference the clean traffic filter to prevent
MAC, IP and ARP spoofing. By not providing
and IP address parameter, libvirt will detect the
IP address the VM is using. -->
<filterref filter='clean-traffic'/>
<!-- enable TCP ports 22 (ssh) and 80 (http) to be reachable -->
<rule action='accept' direction='in'>
<tcp dstportstart='22'/>
</rule>
<rule action='accept' direction='in'>
<tcp dstportstart='80'/>
</rule>
<!-- enable general ICMP traffic to be initiated by the VM;
this includes ping traffic -->
<rule action='accept' direction='out'>
<icmp/>
</rule>
<!-- enable outgoing DNS lookups using UDP -->
<rule action='accept' direction='out'>
<udp dstportstart='53'/>
</rule>
<!-- drop all other traffic -->
<rule action='drop' direction='inout'>
<all/>
</rule>
</filter>
```

Правила в добавляемом XML-коде не содержат IP-адрес ВМ в качестве исходного или конечного адреса, однако фильтрация трафика будет работать корректно, т.к. внутренняя интерпретация правил выполняется для каждого интерфейса, и правила интерпретируются на основе информации о том, какой интерфейс (TAP) отправил или получит пакет, а не на том, какой IP-адрес источника или получателя может быть;

3) загрузить (применить) фильтр:

```
virsh -c qemu:///system nwfilter-define test-eth0.xml
```

Файл с фильтром будет скопирован в каталог /etc/libvirt/nwfilter/;

4) в конфигурации ВМ в секции описания сетевого интерфейса в качестве значения параметр `filterref filter` указать фильтр:

```
<interface type='bridge'>
<source bridge='mybridge'/>
<filterref filter='test-eth0'/>
</interface>
```

5) включить ВМ и проконтролировать, что правило применилось, для этого выполнить команду:

```
virsh -c qemu:///system nwfilter-binding-list
```

Результат выполнения команды:

Устройство порта	Фильтр

vnet195	test-eth0

Чтобы более строго контролировать трафик ICMP и гарантировать, что с ВМ могут отправляться только эхо-запросы ICMP, а ВМ может получать только эхо-ответы ICMP, приведенное в пункте перечисления 2) правило ICMP можно заменить следующими двумя правилами:

```
<!-- enable outgoing ICMP echo requests-->
<rule action='accept' direction='out'>
<icmp type='8'/>
</rule>
<!-- enable incoming ICMP echo replies-->
<rule action='accept' direction='in'>
<icmp type='0'/>
</rule>
```

5.10.3. Управление сетевым трафиком в условиях мандатного управления доступом

При сетевом взаимодействии (в том числе в среде виртуализации) в условиях мандатного управления доступом ОС внедряет в исходящие сетевые пакеты классификационную метку, во входящих сетевых пакетах ОС проверяет классификационную метку и принимает сетевые пакеты в соответствии с правилами мандатного управления доступом (см. 4.14).

Если гостевая операционная система ВМ не поддерживает классификационные метки, то для сетевого взаимодействия хостовой ОС и гостевой операционной системы должно осуществляться переопределение классификационных меток сетевых пакетов.

Для переопределения классификационных меток сетевых пакетов должно быть задано в конфигурационном XML-файле ВМ для тега `keep-IP-security-option` значение `no`

(задано по умолчанию):

```
<keep-IP-security-option>no</keep-IP-security-option>
```

При этом для ВМ с нулевой классификационной меткой не выполняется переопределение классификационных меток в сетевых пакетах.

Принудительное переопределении классификационных меток сетевых пакетов выполняется для ВМ с ненулевой классификационной меткой по правилам:

- исходящим из ВМ сетевым пакетам с нулевой классификационной меткой присваивается классификационная метка ВМ (процесса ВМ в хостовой ОС);
- исходящие из ВМ сетевые пакеты с ненулевой классификационной меткой блокируются;
- входящие в ВМ сетевые пакеты с нулевой классификационной меткой блокируются;
- из входящих в ВМ сетевых пакетов с ненулевой классификационной меткой принудительно удаляется классификационная метка.

Если гостевая операционная система ВМ поддерживает классификационные метки, то для сетевого взаимодействия хостовой ОС и гостевой операционной системы требуется настроить сохранение классификационных меток сетевых пакетов.

Для сохранения классификационных меток сетевых пакетов должно быть задано в конфигурационном XML-файле ВМ для тега `keep-IP-security-option` значение `yes`:

```
<keep-IP-security-option>yes</keep-IP-security-option>
```

Настройку переопределения или сохранения классификационных меток также возможно выполнить в графической утилите `virt-manager` («Менеджер виртуальных машин», см. электронную справку).

5.11. Защита памяти в среде виртуализации

Для очистки остаточной информации в памяти средства вычислительной техники используются механизмы, которые обеспечивают очищение неиспользуемых блоков ФС непосредственно при их освобождении (распределении) и очищение активных разделов страничного обмена. Описание механизмов приведено в 9.1.

Управление механизмом очистки неиспользуемых блоков ФС непосредственно при их освобождении осуществляется с использованием инструмента `astra-secdel-control`. Управление механизмом очистки активных разделов страничного обмена осуществляется с

использованием инструмента `astra-swapwiper-control`.

Защита задач ядра и процессов пользователей при доступе к страницам оперативной памяти обеспечивается архитектурой и параметрами ядра ОС (см. 9.2).

Для предупреждения несанкционированных изменений модулей ядра в составе ОС применяется модуль `lkrp` (см. 16.6.15), который обеспечивает мониторинг угроз и блокирование несанкционированных изменений в ядре ОС. Таким образом, целостность всей области памяти ВМ при использовании `lkrp` обеспечивается по умолчанию. Для обработки критически важных данных рекомендуется на хосте использовать ОС с включенным модулем `lkrp` и ЗПС (см. 16.1), а в гостевых операционных системах применять ОС на усиленном или максимальном уровне защищенности и включенным ЗПС.

Для изоляции и управления виртуальными гостевыми машинами используется технология KVM, которая включает специальный модуль ядра KVM и средство создания виртуального аппаратного окружения QEMU для изоляции и управления виртуальными гостевыми машинами. KVM, используя загруженный в память модуль ядра, с помощью драйвера пользовательского режима (который представляет собой модифицированный драйвер от QEMU) эмулирует слой аппаратного обеспечения, в среде которого могут создаваться и запускаться виртуальные машины.

В архитектуре KVM виртуальная машина выполняется как обычный процесс ОС, на который распространяется действие мер по изоляции процессов. Каждая гостевая операционная система в рамках своей виртуальной машины на уровне хостовой операционной системы является обычным процессом, выполняющимся с копией процесса `qemu`. Для каждого виртуального процессора создается поток. Эти процессы и потоки не отличаются от аналогичных в хостовой операционной системе. В части изоляции и разграничения доступа процессы виртуальных машин будут рассматриваться как обычные процессы операционной системы и для них будут применяться стандартные правила разграничения доступа и контроля ресурсов системы. Такие правила гарантируют, что программное обеспечение, запущенное в виртуальной машине, не сможет снизить производительность или нарушить работу других виртуальных машин или гипервизора.

Решение задачи изоляции адресных пространств процессов основано на архитектуре ядра ОС (см. раздел 7). Изоляция областей памяти виртуальных машин обеспечивается путем применения механизма управления памятью аппаратной платформы IOMMU (input/output memory management unit), который отвечает за трансляцию виртуальных адресов, видимых аппаратным устройством, в физические адреса и позволяет задавать ограничения операций ввода-вывода для изоляции при использовании виртуализации.

Блок управления памятью для операций ввода-вывода (IOMMU) — это тип блока управления памятью (MMU), который подключает шину расширения с поддержкой прямого доступа к памяти (DMA) к основной памяти. Он расширяет системную архитектуру, добавляя поддержку виртуализации адресов памяти, используемых периферийными устройствами. Кроме

того, он обеспечивает изоляцию и защиту памяти, позволяя системному программному обеспечению контролировать, к каким областям физической памяти может обращаться устройство ввода-вывода. Это также помогает фильтровать и переназначать прерывания от периферийных устройств. Устройство может получить доступ только к областям памяти, которые сопоставлены для него. Следовательно, неисправные и/или вредоносные устройства не могут повредить память. Изоляция памяти позволяет безопасно назначать устройства виртуальной машине без ущерба для хоста и других гостевых ОС.

Для применения механизма управления памятью аппаратной платформы необходимо в настройках BIOS/UEFI включить аппаратную поддержку виртуализации, а также включить поддержку технологии виртуализации ввода-вывода:

- 1) убедиться, что в настройках BIOS/UEFI аппаратной платформы включена поддержка виртуализации, технология AMD-Vi/Intel VT-d поддерживается и включена в настройках BIOS/UEFI;
- 2) на хосте гипервизора включить IOMMU. Для этого в конфигурационном файле GRUB 2 /etc/default/grub в строку параметров загрузки ядра добавить параметр `intel_iommu=on` или `amd_iommu=on` в зависимости от производителя процессора.

Пример

```
GRUB_CMDLINE_LINUX_DEFAULT="...parsec.max_ilev=63 intel_iommu=on"
```

Затем обновить конфигурацию загрузчика:

```
sudo update-grub
```

- 3) перезагрузить систему и выполнить проверку наличия модуля IOMMU в системе, выполнив команду:

```
sudo virt-host-validate
```

Результат выполнения команды при успешной проверке:

```
QEMU: Checking if IOMMU is enabled by kernel : OK
```

- 4) в конфигурационный XML-файл ВМ в секцию `<features>` добавить строку `<ioapic driver="qemu"/>`:

```
<features>
<acpi/>
<apic/>
<vmport state="off"/>
<ioapic driver="qemu"/>
</features>
```

- 5) в конфигурационный XML-файл ВМ в секцию `<devices>` добавить следующие строки (указав для параметра `iommu model` в качестве значения `intel` или `amd` в

зависимости от архитектуры):

```
<iommu model="intel">  
<driver intremap="on" caching_mode="on"/>  
</iommu>
```

6) загрузить ВМ и в гостевой операционной системе включить IOMMU. Для этого в конфигурационном файле GRUB 2 /etc/default/grub в строку параметров загрузки ядра добавить параметр `intel_iommu=on` или `amd_iommu=on` в зависимости от производителя процессора.

Пример

```
GRUB_CMDLINE_LINUX_DEFAULT="...parsec.max_ilev=63 intel_iommu=on"
```

Затем обновить конфигурацию загрузчика:

```
sudo update-grub
```

7) перезагрузить гостевую операционную систему;

8) в гостевой операционной системе выполнить команду:

```
sudo dmesg | grep -e DMAR -e IOMMU
```

В выводе проверить наличие информации об активации IOMMU:

```
...  
[ 0.021407] DMAR: IOMMU enabled  
...
```

5.12. Применение механизма контроля целостности областей памяти по запросу из гостевой операционной системы

Механизм контроля целостности областей памяти ВМ базируется на создании канала приема-передачи данных между ВМ и средством виртуализации с использованием механизма передачи запросов из гостевой операционной системы посредством служебного сокета типа `vsock`.

Для приема запросов на осуществление контроля целостности областей памяти операционной системы ВМ используется служба `vsockd` на хосте гипервизора, которая открывает слушающий сокет `vsock`. Виртуальная машина передает в сокет сообщение с параметрами `start_addr` (физический адрес начала блока памяти гостевой операционной системы) и `end_addr` (физический адрес конца блока памяти гостевой операционной системы). Служба `vsockd` на хосте принимает сообщение, получает адрес CID сокета `vsock` и отправляет запрос в средство виртуализации `libvirt` для последующего вычисления контрольной суммы. Вычисление контрольных сумм осуществляется с применением алгоритма по ГОСТ Р 34.11-2012.

Контроль целостности областей памяти VM настраивается в запущенной VM и осуществляется до выключения или перезагрузки VM.

При постановке на контроль целостности областей памяти VM средство виртуализации вычисляет эталонные значения контрольных сумм заданных областей памяти и сохраняет их в памяти средства виртуализации. После выключения или перезагрузки VM эталонные значения контрольных сумм удаляются из памяти средства виртуализации.

Контроль целостности областей памяти VM осуществляется с заданной периодичностью в процессе работы операционной системы VM путем вычисления контрольных сумм заданных областей памяти и их сравнения с эталонными значениями. При миграции запущенной VM данные об эталонных значениях установленных на контроль областей памяти будут также передаваться вместе с другой служебной информацией о VM для возможного последующего контроля целостности областей памяти на другом хосте. После выключения или перезагрузки VM области памяти для контроля удаляются.

Для использования механизма контроля целостности областей памяти VM в хостовой ОС должен быть установлен пакет `astra-kvm-secure`, а в качестве операционной системы VM необходимо использовать ОС с установленным в ней гостевым агентом `qemu-guest-agent` из состава ОС.

При выявлении нарушения целостности областей памяти VM осуществляется регистрация фактов нарушения целостности объектов контроля (см. 5.8), а также возможна блокировка работы VM.

Настройка механизма контроля целостности областей памяти VM осуществляется установкой соответствующих параметров в конфигурационном файле гипервизора `/etc/libvirt/libvirtd.conf`. Настройку рекомендуется выполнять с использованием `virsh` путем выполнения команды:

```
virsh -c qemu:///system config --edit-config /etc/libvirt/libvirtd.conf
```

Для включения механизма контроля целостности областей памяти VM необходимо в файле `/etc/libvirt/libvirtd.conf` для параметра `memory_integrity_check_period_s` задать значение периода проверки (в секундах):

```
memory_integrity_check_period_s = 30
```

При этом будет обеспечиваться только регистрация фактов нарушения целостности объектов контроля без блокировки работы VM.

Для принудительного выключения функционирующей VM в случае нарушения целостности установленных на контроль областей памяти

необходимо в файле `/etc/libvirt/libvirtd.conf` для параметра `memory_integrity_check_shutdown_domain` задать значение 1:

```
memory_integrity_check_shutdown_domain = 1
```

При этом будет обеспечиваться регистрация фактов нарушения целостности областей памяти с последующим принудительным выключением ВМ, если значение контрольной суммы хотя бы одной установленной на контроль области памяти не совпадет с эталонным значением.

Для применения настроек следует перезагрузить службу `libvirtd`:

```
sudo systemctl restart libvirtd
```

Для осуществления контроля областей памяти ВМ к ней необходимо подключить сокет VirtIO VSOCK, через который будет осуществляться связь гостевой операционной системы ВМ с хостом. Для этого необходимо в `virt-manager`:

- 1) открыть свойства ВМ и нажать **[Добавить оборудование]**;
- 2) в окне «Добавление виртуального оборудования» выбрать «VirtIO VSOCK»;
- 3) во вкладке «Подробности» для параметра «CID гостевой системы» установить флаг «Авто»;
- 4) нажать **[Готово]**.

В результате в конфигурационном XML-файле ВМ будет сформирована запись вида:

```
<vsock model="virtio">  
<cid auto="yes" address="3"/>  
<address type="pci" domain="0x0000" bus="0x07" slot="0x00" function="0x0"/>  
</vsock>
```

Каждому сокету автоматически присваивается метка CID гостевой операционной системы ВМ, которая обозначает сквозной идентификатор каждой ВМ.

После добавления сокета необходимо выполнить запуск гостевой операционной системы ВМ.

Постановку на контроль областей памяти обеспечивает инструмент `/sbin/memcontrol_VM`, принимающий в качестве параметров идентификатор контролируемого процесса (PID) и диапазон виртуальных адресов блока памяти в шестнадцатеричном формате.

Синтаксис команды:

1) в случае использования виртуальных адресов процесса с PID:

```
memcontrol_VM --virt <pid> <address_start> <address_end>
```

Начало и конец адреса должны указываться в шестнадцатеричном формате вида 0x5803734e1000;

2) в случае использования физических адресов памяти операционной системы ВМ для постановки на контроль:

```
memcontrol_VM --phys <address_start> <address_end>
```

Для определения диапазона виртуальных адресов блока памяти в гостевой операционной системе ВМ используется инструмент `ps`, выводящий идентификатор процесса по его названию, и файл `/proc/<pid>/maps`, содержащий адреса областей памяти, которые используются процессом в данный момент, с указанием прав доступа к ним.

Пример

Для постановки на контроль областей памяти, занимаемых процессом `syslog-ng`, необходимо в операционной системе ВМ:

1) определить идентификатор процесса командой:

```
ps aux | grep syslog-ng
```

Вывод команды:

```
root 394 0.5 1.8 624928 38180 ? Ssl 16:20 0:27
/usr/sbin/syslog-ng -F --no-caps
u 1438 0.0 0.0 6228 872 pts/2 S+ 17:44 0:00 grep
syslog-ng
```

В выводе команды PID процесса равен 394;

2) выполнить поиск адресов областей памяти, которые используются процессом с PID, равным 394, в данный момент:

```
sudo cat /proc/394/maps
```

Вывод команды:

```
5e1996453000-5e1996455000 r--p 00000000 fc:02 270678 /usr/sbin/syslog-ng
5e1996455000-5e1996456000 r-xp 00002000 fc:02 270678 /usr/sbin/syslog-ng
5e1996456000-5e1996457000 r--p 00003000 fc:02 270678 /usr/sbin/syslog-ng
5e1996457000-5e1996458000 r--p 00003000 fc:02 270678 /usr/sbin/syslog-ng
5e1996458000-5e1996459000 rw-p 00004000 fc:02 270678 /usr/sbin/syslog-ng
5e199702b000-5e199753d000 rw-p 00000000 00:00 0 [heap]
7ab170000000-7ab170021000 rw-p 00000000 00:00 0
7ab170021000-7ab174000000 ---p 00000000 00:00 0
7ab174000000-7ab17403c000 rw-p 00000000 00:00 0
7ab17403c000-7ab178000000 ---p 00000000 00:00 0
7ab178000000-7ab178021000 rw-p 00000000 00:00 0
```

```

7ab178021000-7ab17c000000 ---p 00000000 00:00 0
7ab17c000000-7ab17c021000 rw-p 00000000 00:00 0
7ab17c021000-7ab180000000 ---p 00000000 00:00 0
7ab180000000-7ab180021000 rw-p 00000000 00:00 0
7ab180021000-7ab184000000 ---p 00000000 00:00 0
7ab184000000-7ab184021000 rw-p 00000000 00:00 0
7ab184021000-7ab188000000 ---p 00000000 00:00 0
7ab188ffa000-7ab188ffb000 ---p 00000000 00:00 0
7ab188ffb000-7ab1897fb000 rw-p 00000000 00:00 0

```

...

Сегмент с правами доступа r-хр указывает на хранение исполняемого кода syslog-ng. Соответствующая ему область памяти имеет диапазон адресов 5e1996455000–5e1996456000 (или в требуемом виде 0x5e1996455000–0x5e1996456000);

3) выполнить запрос постановки на контроль целостности выбранной области памяти для процесса с PID, равным 394, при этом рекомендуется из адреса конца блока памяти вычесть 1, т.к. адрес конца блока области памяти совпадает с адресом начала следующего блока области памяти:

```
sudo memcontrol_VM --virt 394 0x5e1996455000 0x5e1996455fff
```

В выводе команды будут указаны физические адреса начала и конца блока поставленной на контроль памяти гостевой операционной системы и рассчитанное для этой области значение эталонной контрольной суммы.

В результате контрольная сумма области памяти будет записана в памяти средства виртуализации с целью дальнейшего периодического контроля целостности.

Для управления из хостовой ОС режимом контроля целостности областей памяти для отдельной VM используется инструмент `virsh memory-integrity`. Для снятия с контроля областей памяти или возобновления/приостановки контроля выполнить команду:

```
virsh -c qemu:///system memory-integrity <имя_VM> [--clear] [--start] [--stop]
```

где <имя_VM> — наименование VM, допускается указывать идентификатор VM, присвоенный средой виртуализации (ID), или всемирно уникальный идентификатор VM (UUID).

Более подробная информация об использовании инструмента `virsh memory-integrity` приведена на странице помощи:

```
virsh help memory-integrity
```

Перечень поставленных на контроль областей памяти ВМ также можно просмотреть в графической утилите `virt-manager` («Менеджер виртуальных машин», см. электронную справку). В утилите также выводится результат контроля.

5.13. Ограничение программной среды в среде виртуализации

Контроль за запуском компонентов программного обеспечения, обеспечивающий выявление и блокировку запуска компонентов программного обеспечения, не включенных в перечень (список) компонентов, разрешенных для запуска, осуществляется штатными средствами ОС.

Механизм динамического контроля целостности (режим ЗПС) обеспечивает выявление и блокировку запуска компонентов программного обеспечения, целостность которого нарушена. В исполняемые файлы и разделяемые библиотеки, входящие в состав ПО, внедряется цифровая подпись. При включенном режиме ЗПС выполняться будут только файлы с верными цифровыми подписями. Запуск файлов с неверными цифровыми подписями или без них будет блокироваться. Описание ЗПС приведено в 16.1.

Применение режима Киоск-2 ограничивает права пользователей на запуск программ в ОС. Степень этих ограничений задается маской киоска, которая накладывается на права доступа к исполняемым файлам при любой попытке пользователя получить доступ. Описание Киоск-2 приведено в 16.2.

5.14. Централизованное управление

В среде виртуализации реализовано создание, модификация, хранение, получение и удаление (в т. ч. централизованное) образов виртуальных машин в информационной (автоматизированной) системе.

Для централизованного хранения образов ВМ используется хранилище данных — пул (pool). Хранилище определяет физическое расположение файлов-образов и может быть различных типов. Хранилище должно быть доступно для подключения на серверах виртуализации.

Для распределенного хранилища следует использовать кластерную файловую систему OCFS2 или блочное устройство `ceph/rbd`. Образы виртуальных машин помещаются в данное хранилище и становятся доступными для всех узлов виртуализации.

ОС поддерживает возможность миграции виртуальных машин между серверами виртуализации. Миграция позволяет перенести работу виртуальной машины с одного физического хоста (сервера виртуализации) на другой без остановки ее работы. При этом образ диска виртуальной машины доступен на обоих серверах виртуализации. Для этого используется распределенное хранилище данных: на обоих серверах виртуализации хранилище должно монтироваться в одно и то же место и образ виртуальной машины должен быть расположен

в данном хранилище.

Управление перемещением виртуальных машин реализуется с использованием интерфейсов управления средствами виртуализации `libvirt` в соответствии с установленными правилами сетевого взаимодействия.

Для решения задач централизованного протоколирования и анализа журналов аудита используется `Zabbix`.

Для решения задач централизованного сбора журналов с удаленных хостов виртуализации возможно применение встроенных механизмов сбора информации о событиях `syslog-ng`. Вместе с установкой пакета `astra-kvm-secure` обеспечивается создание конфигурационных файлов для удаленного сбора данных о событиях безопасности, связанных с функционированием средства виртуализации, из журналов событий безопасности.

5.14.1. Пример организации распределенного хранилища

Организация распределенного хранилища для серверов виртуализации на примере двух серверов виртуализации и сервера хранения данных (СХД):

- 1) СХД — имя компьютера `astra-storage`, IP-адрес `172.16.1.20`;
- 2) первый сервер виртуализации — имя компьютера `astra1`, IP-адрес `172.16.1.21`;
- 3) второй сервер виртуализации — имя компьютера `astra2`, IP-адрес `172.16.1.22`.

Компьютеры должны быть объединены в локальную сеть и доступны по имени компьютера (в `/etc/hosts` на каждом компьютере должны быть выполнены соответствующие настройки).

Для создания сетевого хранилища данных на СХД применяется технология SAN, позволяющая монтировать на компьютере сетевое блочное устройство как локальное блочное устройство. Доступ к сетевому блочному устройству настраивается по протоколу iSCSI.

5.14.1.1. Создание и подключение сетевого хранилища

Для создания SAN-хранилища (сетевого блочного устройства) необходимо выполнить следующие действия на `astra-storage`:

- 1) вывести перечень имеющихся блочных устройств командой:

```
lsblk
```

Вывод команды:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	465,8G	0	disk	
sdb	8:16	0	298,1G	0	disk	
__sdb1	8:17	0	512M	0	part	/boot/efi
__sdb2	8:18	0	296,7G	0	part	/

```

|__sdb3  8:19    0   977M  0 part [SWAP]
sdc       8:32    0 465,8G  0 disk
|__sdc1   8:33    0 465,8G  0 part
sr0       11:0    1  1024M  0 rom

```

Из вывода команды определить блочное устройство, на основе которого будет создано сетевое блочное устройство (например, sda);

2) установить консоль управления Linux-IO Target командой:

```
sudo apt install targetcli-fb
```

3) запустить консоль управления Linux-IO Target командой:

```
sudo targetcli
```

4) в консоли управления Linux-IO Target вывести текущую конфигурацию командой:

```
ls
```

Вывод команды:

```

o- / ..... [....]
o- backstores ..... [....]
| o- block ..... [Storage Objects: 0]
| o- fileio ..... [Storage Objects: 0]
| o- pscsi ..... [Storage Objects: 0]
| o- ramdisk ..... [Storage Objects: 0]
o- iscsi ..... [Targets: 0]
o- loopback ..... [Targets: 0]
o- vhost ..... [Targets: 0]
o- xen-pvscsi ..... [Targets: 0]

```

5) создать (зарегистрировать) сетевое блочное устройство в разделе /backstores/block командой:

```
/backstores/block create <имя_устройства> /dev/<блочное_устройство>
```

где <имя_устройства> — наименование, которое будет присвоено создаваемому сетевому блочному устройству;

<блочное_устройство> — выбранное блочное устройство для создания сетевого блочного устройства.

Команда для текущей конфигурации:

```
/backstores/block create san_storage /dev/sda
```

Результат выполнения команды:

```
Created block storage object san_storage using /dev/sda
```

6) проверить создание сетевого блочного устройства san_storage командой:

```
ls
```

Результат выполнения команды:

```

o- / ..... [....]
o- backstores ..... [....]

```

```

| o- block ..... [Storage Objects: 1]
| | o- san_storage ..... [/dev/sda (465.8GiB) write-thru deactivated]
| |   o- alua ..... [ALUA Groups: 1]
| |     o- default_tg_pt_gp ..... [ALUA state: Active/optimized]
| o- fileio ..... [Storage Objects: 0]
| o- pscsi ..... [Storage Objects: 0]
| o- ramdisk ..... [Storage Objects: 0]
o- iscsi ..... [Targets: 0]
o- loopback ..... [Targets: 0]
o- vhost ..... [Targets: 0]
o- xen-pvscsi ..... [Targets: 0]

```

7) создать цель для сетевого блочного устройства (iSCSI-target) в разделе /iscsi командой:

```
/iscsi create
```

Результат выполнения команды:

```

Created target iqn.2003-01.org.linux-iscsi.astra-storage.x8664:
sn.727a2d3719d4.
Created TPG 1.
Global pref auto_add_default_portal=true
Created default portal listening on all IPs (0.0.0.0), port 3260

```

Для сетевого блочного устройства была создана цель:

```
iqn.2003-01.org.linux-iscsi.astra-storage.x8664:sn.727a2d3719d4
```

8) проверить создание цели командой:

```
ls
```

Результат выполнения команды:

```

o- / ..... [...]
o- backstores ..... [...]
| o- block ..... [Storage Objects: 1]
| | o- san_storage ..... [/dev/sda (465.8GiB) write-thru deactivated]
| | o- alua ..... [ALUA Groups: 1]
| | o- default_tg_pt_gp ..... [ALUA state: Active/optimized]
| o- fileio ..... [Storage Objects: 0]
| o- pscsi ..... [Storage Objects: 0]
| o- ramdisk ..... [Storage Objects: 0]
o- iscsi ..... [Targets: 1]
| o- iqn.2003-01.org.linux-iscsi.astra-storage.x8664:sn.727a2d3719d4
[TPGs: 1]
| o- tpg1 ..... [no-gen-acls, no-auth]
| o- acls ..... [ACLs: 0]
| o- luns ..... [LUNs: 0]
| o- portals ..... [Portals: 1]
| o- 0.0.0.0:3260 ..... [OK]
o- loopback ..... [Targets: 0]
o- vhost ..... [Targets: 0]

```

```
o- xen-pvscsi ..... [Targets: 0]
```

9) создать LUN (Logical Unit Number — номер логического устройства) на основе сетевого блочного устройства (зарегистрированного в разделе /backstores/block) командой:

```
/iscsi/<цель>/tpg1/luns/ create /backstores/block/<имя_устройства>
```

где <цель> — цель для сетевого блочного устройства, созданная согласно пункту перечисления 7);

<имя_устройства> — наименование сетевого блочного устройства, заданное при его создании согласно пункту перечисления 5).

Команда для текущей конфигурации:

```
/iscsi/iqn.2003-01.org.linux-iscsi.astra-storage.x8664:sn.727a2d3719d4/\
tpg1/luns/ create /backstores/block/san_storage
```

Результат выполнения команды:

```
Created LUN 0
```

10) проверить результат создания LUN командой:

```
ls
```

Результат выполнения команды:

```
o- / ..... [...]
o- backstores ..... [...]
| o- block ..... [Storage Objects: 1]
| | o- san_storage ..... [/dev/sda (465.8GiB) write-thru activated]
| | o- alua ..... [ALUA Groups: 1]
| | o- default_tg_pt_gp ..... [ALUA state: Active/optimized]
| o- fileio ..... [Storage Objects: 0]
| o- pscsi ..... [Storage Objects: 0]
| o- ramdisk ..... [Storage Objects: 0]
o- iscsi ..... [Targets: 1]
| o- iqn.2003-01.org.linux-iscsi.astra-storage.x8664:sn.727a2d3719d4
TPGs: 1]
| o- tpg1 ..... [no-gen-acls, no-auth]
| o- acls ..... [ACLs: 0]
| o- luns ..... [LUNs: 1]
| | o- lun0 .... [block/san_storage (/dev/sda) (default_tg_pt_gp)]
| o- portals ..... [Portals: 1]
| o- 0.0.0.0:3260 ..... [OK]
o- loopback ..... [Targets: 0]
o- vhost ..... [Targets: 0]
o- xen-pvscsi ..... [Targets: 0]
```

11) в данном примере не используется авторизация и аутентификация, поэтому для отключения контроля доступа к цели необходимо последовательно выполнить команды:

```
cd /iscsi/<цель>/tpg1
```

```
set attribute generate_node_acls=1
set attribute demo_mode_write_protect=0
```

Команды для текущей конфигурации:

а) перейти на уровень tpg1 цели:

```
cd /iscsi/iqn.2003-01.org.linux-iscsi.astra-storage.x8664:\
sn.727a2d3719d4/tpg1
```

б) включить динамически генерируемые списки контроля доступа для узла инициатора во время входа в систему:

```
set attribute generate_node_acls=1
```

Результат выполнения команды:

```
Parameter generate_node_acls is now '1'
```

в) отключить защиту от записи для экспортируемых логических дисков:

```
set attribute demo_mode_write_protect=0
```

Результат выполнения команды:

```
Parameter demo_mode_write_protect is now '0'
```

12) сохранить конфигурацию сетевого блочного устройства командой:

```
/ saveconfig
```

Результат выполнения команды:

```
Configuration saved to /etc/rtslib-fb-target/saveconfig.json
```

Конфигурация сетевого блочного устройства сохранена в файл /etc/rtslib-fb-target/saveconfig.json;

13) выйти из консоли управления Linux-IO Target командой:

```
exit
```

Результат выполнения команды:

```
Global pref auto_save_on_exit=true
Last 10 configs saved in /etc/rtslib-fb-target/backup/.
Configuration saved to /etc/rtslib-fb-target/saveconfig.json
```

5.14.1.2. Подключение на серверах виртуализации сетевого блочного устройства

Созданное на astra-storage сетевое блочное устройство (iSCSI-target) необходимо подключить в качестве локального блочного устройства на серверах виртуализации (iSCSI-initiator).

Данные настройки должны быть выполнены на всех компьютерах, на которых настраивается доступ к сетевому блочному устройству. В данном примере настройки выполняются на серверах виртуализации astra1 и astra2. Для подключения сетевого блочного устройства

к серверу виртуализации необходимо:

1) установить пакет `open-iscsi` командой:

```
sudo apt install open-iscsi
```

2) настроить автоматическое подключение LUN при перезагрузке сервера виртуализации. Для этого в конфигурационном файле `/etc/iscsi/iscsid.conf` для параметра `node.startup` установить значение `automatic`:

```
node.startup = automatic
```

3) запустить службу `iscsi` командой:

```
sudo systemctl start iscsi
```

После первого запуска службы `iscsi` будет сгенерирован уникальный идентификатор инициатора (iSCSI-initiator), который можно просмотреть в файле `/etc/iscsi/initiatorname.iscsi`, выполнив команду:

```
sudo cat /etc/iscsi/initiatorname.iscsi
```

Результат выполнения команды на `astral`:

```
## DO NOT EDIT OR REMOVE THIS FILE!
## If you remove this file, the iSCSI daemon will not start.
## If you change the InitiatorName, existing access control lists
## may reject this initiator. The InitiatorName must be unique
## for each iSCSI initiator. Do NOT duplicate iSCSI InitiatorNames.
InitiatorName=iqn.1993-08.org.debian:01:694dac4a9eba
```

Результат выполнения команды на `astra2`:

```
## DO NOT EDIT OR REMOVE THIS FILE!
## If you remove this file, the iSCSI daemon will not start.
## If you change the InitiatorName, existing access control lists
## may reject this initiator. The InitiatorName must be unique
## for each iSCSI initiator. Do NOT duplicate iSCSI InitiatorNames.
InitiatorName=iqn.1993-08.org.debian:01:7c60f380d294
```

4) выполнить поиск доступных целей (сетевых блочных устройств, iSCSI-target):

```
sudo iscsiadm -m discovery -t st -p <IP-адрес>
```

где `<IP-адрес>` — IP-адрес компьютера, на котором зарегистрировано сетевое блочное устройство.

Команда для текущей конфигурации:

```
sudo iscsiadm -m discovery -t st -p 172.16.1.20
```

Результат выполнения команды:

```
172.16.1.20:3260,1 iqn.2003-01.org.linux-iscsi.astra-storage.x8664:
sn.727a2d3719d4
```

5) автоматически подключить все найденные цели:

```
sudo iscsiadm -m node -l
```

Результат выполнения команды:

```
Logging in to [iface: default, target:
iqn.2003-01.org.linux-iscsi.astra-storage.x8664:sn.727a2d3719d4,
portal: 172.16.1.20,3260]
Login to [iface: default, target:
iqn.2003-01.org.linux-iscsi.astra-storage.x8664:sn.727a2d3719d4,
portal: 172.16.1.20,3260] successful.
```

6) проверить, что на сервере виртуализации было добавлено новое блочное устройство:

```
lsblk
```

Результат выполнения команды на astra1:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	931,5G	0	disk	
_sda1	8:1	0	512M	0	part	/boot/efi
_sda2	8:2	0	28G	0	part	/
_sda3	8:3	0	18,6G	0	part	[SWAP]
_sda4	8:4	0	884,5G	0	part	/home
sdb	8:16	0	465,8G	0	disk	
sr0	11:0	1	1024M	0	rom	

На astra1 добавлено новое блочное устройство /dev/sdb.

Результат выполнения команды на astra2:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	465,8G	0	disk	
nvme0n1	259:0	0	238,5G	0	disk	
_nvme0n1p1	259:1	0	512M	0	part	/boot/efi
_nvme0n1p2	259:2	0	28G	0	part	/
_nvme0n1p3	259:3	0	7,5G	0	part	[SWAP]
_nvme0n1p4	259:4	0	202,6G	0	part	/home

На astra2 добавлено новое блочное устройство /dev/sda.

5.14.1.3. Настройка кластера

Для использования сетевого хранилища серверами виртуализации требуется создать кластер и разметить на нем кластерную файловую систему OCFS2.

Для обеспечения взаимодействия узлов кластера на каждом сервере виртуализации предварительно необходимо настроить межсетевой экран firewallld:

1) разрешить доступ к TCP-порту 7777:

```
sudo firewall-cmd --permanent --add-port=7777/tcp
```

2) перезагрузить конфигурацию межсетевого экрана:

```
sudo firewall-cmd --reload
```

Создание кластера

Для создания кластера необходимо на одном из узлов кластера (например, `astral`) выполнить следующие действия:

1) установить пакет `ocfs2-tools` командой:

```
sudo apt install ocfs2-tools
```

2) создать кластер OCFS2 командой:

```
sudo o2cb add-cluster ocfs2cluster
```

где `ocfs2cluster` — имя кластера.

В результате выполнения команды будет создан файл конфигурации кластера `/etc/ocfs2/cluster.conf`;

3) добавить описание узлов кластера, выполнив команду для каждого узла кластера:

```
sudo o2cb add-node <имя_кластера> <сетевое_имя_узла> --ip <IP-адрес_узла>
```

где `<имя_кластера>` — имя созданного кластера;

`<сетевое_имя_узла>` — имя компьютера, добавляемого в качестве узла кластера;

`<IP-адрес_узла>` — IP-адрес компьютера, добавляемого в качестве узла кластера.

Имя узла кластера должно соответствовать имени компьютера, указанному в `/etc/hostname`, также имена компьютеров и их IP-адреса должны быть указаны в `/etc/hosts` на других узлах кластера.

Команды для текущей конфигурации:

```
sudo o2cb add-node ocfs2cluster astral --ip 172.16.1.21
```

```
sudo o2cb add-node ocfs2cluster astra2 --ip 172.16.1.22
```

Данные о добавленных узлах кластера записываются в `/etc/ocfs2/cluster.conf`.

Проверить содержимое файла конфигурации кластера возможно с помощью команды:

```
cat /etc/ocfs2/cluster.conf
```

Результат выполнения команды:

```
cluster:
```

```
heartbeat_mode = local
```

```
node_count = 2
```

```
name = ocfs2cluster
```

```
node:
```

```
number = 0
```

```
cluster = ocfs2cluster
```

```
ip_port = 7777
```

```
ip_address = 172.16.1.21
name = astral
```

```
node:
number = 1
cluster = ocfs2cluster
ip_port = 7777
ip_address = 172.16.1.22
name = astra2
```

На всех остальных узлах кластера (в данном примере на узле `astra2`) требуется выполнить следующие действия для настройки кластера:

1) установить пакет `ocfs2-tools` командой:

```
sudo apt install ocfs2-tools
```

2) скопировать в локальный каталог `/etc/ocfs2/` файл конфигурации кластера:

```
sudo scp \
    <локальный_администратор>@<IP-адрес_узла>:/etc/ocfs2/cluster.conf \
    /etc/ocfs2/
```

где `<IP-адрес_узла>` — IP-адрес узла кластера, на котором была выполнена настройка кластера и с которого копируется файл конфигурации;

`<локальный_администратор>` — имя локального администратора узла кластера, на котором была выполнена настройка кластера и с которого копируется файл конфигурации.

В ходе выполнения команды требуется:

- а) на запрос пароля для команды `sudo` — ввести пароль локального администратора узла кластера, на который копируется файл;
- б) на запрос установки соединения ответить «yes» («Да»);
- в) ввести пароль локального администратора узла кластера, с которого копируется файл конфигурации.

Команда для текущей конфигурации:

```
sudo scp admin1@172.16.1.21:/etc/ocfs2/cluster.conf /etc/ocfs2
```

Результат выполнения команды:

```
[sudo] пароль для admin2:
The authenticity of host '172.16.1.21 (172.16.1.21)' can't be established.
ECDSA key fingerprint is SHA256:mONkRKlGRAc1ZEyWS/sYRVdZINN2PAHJIKRwLFzMGSM.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.16.1.21' (ECDSA) to the list of known hosts.
admin1@172.16.1.21's password:
cluster.conf 100% 371 197.3KB/s 00:00
```

На узле `astra2` проверить полученный файл конфигурации:

```
cat /etc/ocfs2/cluster.conf
```

Файл конфигурации кластера `/etc/ocfs2/cluster.conf` на узле `astra2` должен быть идентичен файлу конфигурации кластера на узле `astra1`.

На каждом узле настроить кластер OCFS2:

1) запустить мастер настройки кластера OCFS2 командой:

```
sudo dpkg-reconfigure ocfs2-tools
```

2) в мастере настройки кластера OCFS2:

а) разрешить запускать кластер OCFS2 (O2CB) во время загрузки, нажав кнопку **[Да]**;

б) ввести имя кластера (`ocfs2cluster`) и нажать кнопку **[Ok]**;

в) для остальных параметров выбрать значения, установленные по умолчанию;

3) перезапустить службу `o2cb` командой:

```
sudo systemctl restart o2cb
```

4) проверить успешный запуск службы `o2cb`. Для этого в журнале регистрации просмотреть события, относящиеся к службе:

```
sudo journalctl -u o2cb
```

Вывод команды на узле `astra1`:

```
-- Logs begin at Thu 2023-06-01 13:49:19 MSK, end at Thu 2023-06-01
14:12:52 MSK. --
июн 01 14:10:38 astral systemd[1]: Starting Load o2cb Modules...
июн 01 14:10:38 astral o2cb[3293]: checking debugfs...
июн 01 14:10:39 astral o2cb[3293]: Loading stack plugin "o2cb": OK
июн 01 14:10:39 astral o2cb[3293]: Loading filesystem "ocfs2_dlmfs": OK
июн 01 14:10:39 astral o2cb[3293]: Mounting ocfs2_dlmfs filesystem at /dlm: OK
июн 01 14:10:39 astral o2cb[3293]: Setting cluster stack "o2cb": OK
июн 01 14:10:39 astral o2cb[3293]: Registering O2CB cluster "ocfs2cluster": OK
июн 01 14:10:39 astral o2cb[3293]: Setting O2CB cluster timeouts : OK
июн 01 14:10:39 astral o2hbmonitor[3342]: Starting
июн 01 14:10:39 astral systemd[1]: Started Load o2cb Modules.
июн 01 14:12:34 astral systemd[1]: Stopping Load o2cb Modules...
июн 01 14:12:34 astral o2cb[3418]: Clean userdlm domains: OK
июн 01 14:12:34 astral o2cb[3418]: Stopping O2CB cluster ocfs2cluster:
Unregistering O2
июн 01 14:12:34 astral o2cb[3418]: Unmounting ocfs2_dlmfs filesystem: OK
июн 01 14:12:34 astral o2cb[3418]: Unloading module "ocfs2_dlmfs": OK
июн 01 14:12:34 astral o2cb[3418]: Unloading module "ocfs2_stack_o2cb": OK
июн 01 14:12:34 astral systemd[1]: o2cb.service: Succeeded.
июн 01 14:12:34 astral systemd[1]: Stopped Load o2cb Modules.
июн 01 14:12:34 astral systemd[1]: o2cb.service: Consumed 185ms CPU time.
```

```
июн 01 14:12:34 astral systemd[1]: Starting Load o2cb Modules...
июн 01 14:12:34 astral o2cb[3485]: checking debugfs...
июн 01 14:12:34 astral o2cb[3485]: Loading stack plugin "o2cb": OK
июн 01 14:12:34 astral o2cb[3485]: Loading filesystem "ocfs2_dlmfs": OK
июн 01 14:12:34 astral o2cb[3485]: Mounting ocfs2_dlmfs filesystem at /dlm: OK
июн 01 14:12:34 astral o2cb[3485]: Setting cluster stack "o2cb": OK
июн 01 14:12:34 astral o2cb[3485]: Registering O2CB cluster "ocfs2cluster": OK
июн 01 14:12:34 astral o2cb[3485]: Setting O2CB cluster timeouts : OK
июн 01 14:12:34 astral o2hbmonitor[3531]: Starting
июн 01 14:12:34 astral systemd[1]: Started Load o2cb Modules.
```

В журнале должны отсутствовать сообщения об ошибках запуска службы. Для выхода из просмотра журнала нажать клавишу **<q>**.

Разметка кластерной файловой системы

Разметка подключенного сетевого блочного устройства для использования кластерной файловой системы OCFS2 выполняется на одном из узлов кластера.

Для разметки необходимо выполнить команду:

```
sudo mkfs.ocfs2 -T vmstore <блочное_устройство>
```

где **-T** — запустить автоматическую тонкую настройку параметров файловой системы;
vmstore — производить тонкую настройку для обеспечения максимальной производительности при размещении на блочном устройстве файлов образов виртуальных машин;
<блочное_устройство> — сетевое блочное устройство, подключенное согласно 5.14.1.2.

Для текущей конфигурации выполнить разметку сетевого блочного устройства на узле **astral** командой:

```
sudo mkfs.ocfs2 -T vmstore /dev/sdb
```

Вывод команды:

```
[sudo] пароль для admin1:
mkfs.ocfs2 1.8.5
Cluster stack: classic o2cb
Filesystem Type of vmstore
Label:
Features: sparse extended-slotmap backup-super unwritten inline-data
strict-journal-super xattr indexed-dirs refcount discontig-bg append-dio
Block size: 4096 (12 bits)
Cluster size: 1048576 (20 bits)
Volume size: 500107837440 (476940 clusters) (122096640 blocks)
```

```

Cluster groups: 15 (tail covers 25356 clusters, rest cover 32256 clusters)
Extent allocator size: 188743680 (45 groups)
Journal size: 134217728
Node slots: 8
Creating bitmaps: done
Initializing superblock: done
Writing system files: done
Writing superblock: done
Writing backup superblock: 5 block(s)
Formatting Journals: done
Growing extent allocator: done
Formatting slot map: done
Formatting quota files: done
Writing lost+found: done
mkfs.ocfs2 successful

```

В результате на блочном устройстве /dev/sdb (сетевое блочное устройство san_storage) была создана файловая система OCFS2.

Для просмотра доступных сетевых блочных устройств с кластерной файловой системой OCFS2 выполнить команду на любом из узлов кластера:

```
sudo mounted.ocfs2 -d
```

Результат выполнения команды на узле astral текущей конфигурации:

Device	Stack	Cluster	F	UUID	Label
/dev/sdb	o2cb			E42202DFA47B42DB9B7DE6778555B4A4	

Размеченное сетевое блочное устройство необходимо примонтировать на каждом узле кластера, для этого требуется:

1) создать точку монтирования:

```
sudo mkdir -p <точка_монтирования>
```

2) примонтировать сетевое блочное устройство:

```
sudo mount <сетевое_блочное_устройство> <точка_монтирования>
```

Команды на узле astral для текущей конфигурации:

1) создать точку монтирования:

```
sudo mkdir -p /mnt/ocfs2-storage
```

2) примонтировать сетевое блочное устройство /dev/sdb:

```
sudo mount /dev/sdb /mnt/ocfs2-storage/
```

3) для проверки монтирования сетевого блочного устройства выполнить команду:

```
lsblk
```

Результат выполнения команды:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	931,5G	0	disk	
_sda1	8:1	0	512M	0	part	/boot/efi
_sda2	8:2	0	28G	0	part	/
_sda3	8:3	0	18,6G	0	part	[SWAP]
_sda4	8:4	0	884,5G	0	part	/home
sdb	8:16	0	465,8G	0	disk	/mnt/ocfs2-storage
sr0	11:0	1	1024M	0	rom	

Для просмотра примонтированных сетевых блочных устройств с кластерной файловой системой OCFS2 необходимо выполнить команду на любом из узлов кластера:

```
sudo mounted.ocfs2 -f
```

Результат выполнения команды на узле `astral` текущей конфигурации:

Device	Stack	Cluster	F	Nodes
/dev/sdb	o2cb			astral, astra2

В выводе команды отображается сетевое блочное устройство и список узлов кластера, на которых оно примонтировано.

5.14.1.4. Автоматическое монтирование

Для обеспечения автоматического монтирования сетевого блочного устройства при загрузке узла необходимо:

1) определить UUID блочного устройства, выполнив команду на любом из узлов кластера:

```
sudo blkid
```

Результат выполнения команды на узле `astral` текущей конфигурации:

```
/dev/sda1: UUID="503E-AC24" TYPE="vfat"
PARTUUID="8dabfe8a-031e-4da6-b5be-33a8cba97322"
/dev/sda2: UUID="68de36df-e90e-4fb9-8bba-47dd0b5f4dca"
TYPE="ext4" PARTUUID="e2aa5324-2f15-4962-98f2-744ca4e8f844"
/dev/sda3: UUID="b2f367e4-3e3f-4c58-a159-0e222ebdc422"
TYPE="swap" PARTUUID="f2e75fdf-db3e-40ee-8a9f-ab634363c6fe"
/dev/sda4: UUID="abc173bd-a889-499e-9424-96ce8f8c2b9b"
TYPE="ext4" PARTUUID="f5de7620-7636-4bb0-afe2-1c4192a5c1aa"
/dev/sdb: UUID="e42202df-a47b-42db-9b7d-e6778555b4a4" TYPE="ocfs2"
```


2) на каждом узле кластера отредактировать файл `/etc/fstab`, добавив строку вида:

```
UUID=e42202df-a47b-42db-9b7d-e6778555b4a4    /mnt/ocfs2-storage
ocfs2    _netdev,x-systemd.requires=o2cb.service    0    0
```

где `e42202df-a47b-42db-9b7d-e6778555b4a4` — UUID сетевого блочного устройства;
`/mnt/ocfs2-storage` — точка монтирования;
`ocfs2` — тип файловой системы OCFS2;
`_netdev,x-systemd.requires=o2cb.service` — параметры монтирования, задающие монтирование после загрузки сетевых служб и при запущенной службе `o2cb`.

5.14.1.5. Централизованное управление в среде виртуализации

Для создания централизованного хранилища образов (`pool`, пул) необходимо на каждом сервере виртуализации, объединенном в кластер (`astral`, `astra2`), выполнить от имени администратора средства виртуализации следующие действия:

1) настроить права доступа к точке монтирования сетевого блочного устройства (для текущей конфигурации — каталог `/mnt/ocfs2-storage`), для этого последовательно выполнить команды:

```
sudo chgrp libvirt-admin /mnt/ocfs2-storage/
sudo chmod g+w /mnt/ocfs2-storage/
sudo pdpl-file 0:63 /mnt/ocfs2-storage/
```

2) зарегистрировать каталог `/mnt/ocfs2-storage` в качестве пула с наименованием, например `ocfs2-pool`:

```
virsh -c qemu:///system pool-define-as ocfs2-pool --type dir \
--target /mnt/ocfs2-storage
```

3) включить автоматический запуск пула при загрузке сервера виртуализации:

```
virsh -c qemu:///system pool-autostart ocfs2-pool
```

4) запустить пул:

```
virsh -c qemu:///system pool-start ocfs2-pool
```

5) проверить наличие созданного пула:

```
virsh -c qemu:///system pool-list --all
```

Результат выполнения команды:

Имя	Состояние	Автозапуск
default	активен	yes
ocfs2-pool	активен	yes

Для проверки совместного использования пула необходимо загрузить в созданный пул образ диска и удостовериться в доступности этого образа на каждом сервере виртуализации.

Возможная последовательность действий, необходимых для проверки совместного использования пула:

- 1) на компьютере, с которого выполняется подключение к серверам виртуализации `astral` и `astra2`, скопировать на сервер виртуализации `astral` в каталог `/var/tmp/` образ установочного диска ОС командой:

```
scp <установочный_диск> admin1@172.16.1.21:/var/tmp/install-alse.iso
```

В результате выполнения команды загруженному файлу будет присвоено наименование `install-alse.iso`;

- 2) на сервере виртуализации `astral` перенести файл образа диска в каталог пула хранения:

```
sudo mv /var/tmp/install-alse.iso /mnt/ocfs2-storage/
```

- 3) на каждом из узлов кластера от имени администратора средства виртуализации выполнить следующие действия:

- а) обновить отображение файлов в пуле `ocfs2-pool`:

```
virsh -c qemu:///system pool-refresh ocfs2-pool
```

- б) проверить наличие в пуле `ocfs2-pool` файла образа установочного диска:

```
virsh -c qemu:///system vol-list ocfs2-pool
```

Результат выполнения команды:

Имя	Путь
install-alse.iso	/mnt/ocfs2-storage/install-alse.iso
...	

5.14.2. Пример миграции виртуальных машин

Миграция запущенной ВМ с одного сервера виртуализации на другой сервер виртуализации возможна при использовании серверами виртуализации распределенного хранилища.

В качестве примера будут описаны действия, необходимые для миграции ВМ с сервера виртуализации `astral` на сервер виртуализации `astra2`, для которых было организовано распределенное хранилище (см. 5.14.1). Локальным администраторам `admin1` (на сервере виртуализации `astral`) и `admin2` (на сервере виртуализации `astra2`) была назначена роль администратора средства виртуализации в соответствии с 5.3.1.

По умолчанию при миграции используется открытое TCP-соединение, при этом используется диапазон портов от 49152 до 49215. Кроме того, в описываемом примере для подключения к консоли ВМ через систему VNC будет использоваться TCP-порт 5901. Поэтому на каждом сервере виртуализации предварительно необходимо настроить межсетевой экран

firewalld:

- 1) разрешить доступ к указанному диапазону TCP-портов:

```
sudo firewall-cmd --permanent --add-port=49152-49215/tcp
```

- 2) разрешить доступ к TCP-порту 5901:

```
sudo firewall-cmd --permanent --add-port=5901/tcp
```

- 3) перезагрузить конфигурацию межсетевого экрана:

```
sudo firewall-cmd --reload
```

Для удаленного подключения к серверам виртуализации предварительно требуется настроить SSH-аутентификацию по ключам. Для этого на компьютере, с которого будет выполняться подключение к этим серверам виртуализации, необходимо от имени пользователя, который будет подключаться удаленно, выполнить следующие действия:

- 1) создать ключ командой:

```
ssh-keygen -t rsa
```

- 2) скопировать созданный ключ на каждый сервер виртуализации:

```
ssh-copy-id <локальный_администратор>@<IP-адрес_узла>
```

где <локальный_администратор> — имя локального администратора средства виртуализации;

<IP-адрес_узла> — IP-адрес сервера виртуализации.

При соответствующем запросе ввести пароль локального администратора средства виртуализации.

Пример команд для текущей конфигурации, выполняется на компьютере, с которого будет выполняться подключение к серверам виртуализации *astral* и *astra2*:

```
ssh-copy-id admin1@172.16.1.21
```

```
ssh-copy-id admin2@172.16.1.22
```

Кроме того, необходимо настроить двустороннюю SSH-аутентификацию по ключам между серверами виртуализации *astral* и *astra2* (выполняется от имени локального администратора средства виртуализации):

- 1) на каждом сервере виртуализации создать ключ командой:

```
ssh-keygen -t rsa
```

- 2) на сервере виртуализации *astral* скопировать созданный ключ на сервер виртуализации *astra2* (при соответствующем запросе ввести пароль локального администратора средства виртуализации *admin2*):

```
ssh-copy-id admin2@172.16.1.22
```

3) на сервере виртуализации `astra2` скопировать созданный ключ на сервер виртуализации `astra1` (при соответствующем запросе ввести пароль локального администратора средства виртуализации `admin1`):

```
ssh-copy-id admin1@172.16.1.21
```

Для создания ВМ на сервере виртуализации `astra1` необходимо от имени администратора средства виртуализации выполнить команду:

```
virt-install --connect qemu:///system --name else --os-type=else18 \
--cdrom /mnt/ocfs2-storage/install-else.iso \
--graphics vnc,port=5901,listen=0.0.0.0 \
--disk pool=ocfs2-pool,size=12,bus=virtio,format=qcow2 \
--ram 2048 --vcpus=2 --hvm --virt-type=kvm --network none
```

где `--name` — задает наименование ВМ (`else`);

`--os-type` — задает тип ОС (`else18`);

`--cdrom` — задает путь к файлу образа установочного диска (`/mnt/ocfs2-storage/install-else.iso`);

`--graphics vnc,port=5901,listen=0.0.0.0` — задает параметры графического сервера (будет использоваться система VNC, для подключения к графическому серверу VNC будет использоваться TCP-порт 5901, при этом для подключения может использоваться любой сетевой интерфейс);

`--disk pool=ocfs2-pool,size=12,bus=virtio,format=qcow2` — задает параметры образа жесткого диска ВМ (файл образа жесткого диска будет размещен в пуле `ocfs2-pool`, размер образа жесткого диска будет равен 12 ГиБ, для подключения образа жесткого диска будет использован драйвер VirtIO, образ жесткого диска будет создан в формате QCOW2);

`--ram` — задает объем оперативной памяти (2048 МиБ);

`--vcpus` — задает количество процессоров (2);

`--hvm` — указывает на то, что ВМ должна быть полностью виртуализированной;

`--virt-type=kvm` — указывает на необходимость использовать модуль ядра KVM, который задействует аппаратные возможности виртуализации процессора;

`--network none` — указывает на то, что ВМ не будет подключена к виртуальной сети.

Для подключения к консоли созданной ВМ необходимо на компьютере, с которого выполняется подключение к серверам виртуализации, запустить графическую утилиту `virt-viewer`:

```
virt-viewer -c <URI_сервера_виртуализации> -d <имя_ВМ>
```

Пример команды для текущей конфигурации:

```
virt-viewer -c qemu+ssh://admin1@172.16.1.21/system -d else
```

При соответствующем запросе ввести пароль администратора средства виртуализации `admin1`.

В консоли ВМ следует выполнить предварительную настройку и запустить процесс установки ОС.

Для запуска процесса миграции ВМ используется следующая команда (при соответствующем запросе ввести пароль локального администратора сервера виртуализации, на котором размещена ВМ):

```
virsh -c <URI_исходного_сервера_виртуализации> migrate <наименование_ВМ> \  
    <URI_конечного_сервера_виртуализации>
```

По умолчанию конфигурационный XML-файл ВМ не перемещается на другой сервер виртуализации, при выключении ВМ она вновь появится на исходном сервере виртуализации. Для выполнения полной миграции, при котором конфигурационный XML-файл ВМ будет перемещена на другой сервер виртуализации, необходимо использовать параметры `--persistent` и `--undefinesource`.

Пример команды для текущей конфигурации (выполняется на компьютере, с которого осуществляется подключение к серверам виртуализации):

```
virsh -c qemu+ssh://admin1@172.16.1.21/system migrate else \  
    --persistent --undefinesource qemu+ssh://admin2@172.16.1.22/system
```

После запуска процесса миграции соединение с консолью созданной ВМ будет разорвано, т.к. на сервере виртуализации `astra1` ВМ будет удалена.

Для проверки успешного завершения процесса миграции следует подключиться к консоли перемещенной ВМ, указав URI сервера виртуализации `astra2`.

Пример команды для текущей конфигурации (выполняется на компьютере, с которого осуществляется подключение к серверам виртуализации):

```
virt-viewer -c qemu+ssh://admin2@172.16.1.22/system -d else
```

При соответствующем запросе ввести пароль администратора средства виртуализации `admin2`.

В консоли ВМ будет отображен продолжающийся процесс установки ОС. Это свидетельствует о том, что миграция была выполнена без прекращения работы ВМ.

6. РЕГИСТРАЦИЯ СОБЫТИЙ БЕЗОПАСНОСТИ

В ОС регистрация событий безопасности выполняется с учетом требований ГОСТ Р 59548-2022 «Защита информации. Регистрация событий безопасности. Требования к регистрируемой информации».

Регистрация событий безопасности реализуется использованием службы `auditd` и подсистемы регистрации событий из состава ОС, описание которой приведено в 6.5.

Служба `auditd` выполняет регистрацию событий объектов файловой системы (аудит файлов) и пользователей (аудит процессов) согласно заданным правилам. Работа с правилами аудита описана в 6.1 и 6.2. Регистрация событий осуществляется в журнал аудита, описание которого приведено в 6.3. Описание настройки параметров аудита приведено в 6.4.

Применение настроенных параметров аудита процессов осуществляется РАМ-модулем `pam_parsec_aud`. По умолчанию регистрация событий аудита процессов включена в РАМ-сценарии: `fly-dm`, `fly-dm-np`, `login`, `su`, `sshd`, `sumac.xauth`. Для регистрации событий аудита процессов пользователя, проходящего аутентификацию через другие РАМ-сценарии, необходимо включить в соответствующие сценарии строку следующего вида:

```
session required pam_parsec_aud.so
```

В библиотеках подсистемы безопасности PARSEC реализован программный интерфейс для регистрации событий с использованием службы регистрации событий безопасности ОС. Данный программный интерфейс применяется для регистрации событий в СУБД (описание приведено в РУСБ.10015-01 97 01-3) и в комплексе программ электронной почты.

6.1. Правила регистрации событий

Регистрация событий осуществляется в соответствии с правилами аудита, правила делятся на два типа:

- 1) временные — действуют до перезагрузки системы. Данные правила относятся к подсистеме ядерного аудита: они задаются посредством инструмента `auditctl` и начинают выполняться при запуске службы `auditd`;
- 2) постоянные — действуют всегда, даже после перезагрузки системы. Такие правила задаются в файлах формата `.rules`, располагающихся в каталоге `/etc/audit/rules.d/`.

Подробное описание правил аудита, а также синтаксис использования инструмента `auditctl` приведены на справочной странице `man auditctl`.

Примеры:

1. Регистрировать все системные вызовы от процесса с идентификатором (pid) 1005:

```
auditctl -a exit,always -S all -F pid=1005
```

2. Регистрировать все файлы, открытые пользователем аудита с идентификатором (auid) 510:

```
auditctl -a exit,always -S open -F auid=510
```

При добавлении постоянных правил аудита в файлах используется синтаксис инструмента `auditctl` без указания имени инструмента.

Пример

Регистрировать все системные вызовы от процесса с идентификатором (PID) 1005:

```
-a exit,always -S all -F pid=1005
```

Подсистема безопасности PARSEC предоставляет дополнительный способ выбора событий для регистрации — PARSEC-аудит. Он использует списки правил регистрации событий, назначаемые на процессы и файлы с помощью инструментов командной строки `setfaud` (см. 6.4.3), `useraud` (см. 6.4.4), `psaud` (см. 6.4.5).

Для работы PARSEC-аудита в файлах `/etc/audit/rules.d/10-parsec.rules` (аудит процессов и файлов) и `/etc/audit/rules.d/10-parsec-nw.rules` (сетевой аудит) заданы следующие постоянные правила:

1) правила аудита процессов (необходимы для работы `useraud` и `psaud`):

```
-a always,exit -F subj_type=psaud -F arch=<архитектура> -S \
    <системные_вызовы> -k parsec-p
```

где `-a always,exit` — регистрировать события в журнале, добавить правило в список `exit` (события, происходящие при выходе из системного вызова);
`-F subj_type=psaud` — обрабатывать правила, заданные с помощью `useraud` и `psaud`;
`-F arch=<архитектура>` — аппаратная платформа;
`-S <системные_вызовы>` — перехватывать события при вызовах, указанных в `<системные_вызовы>` (список вызовов, разделенных запятой);
`-k parsec-p` — присвоить метку фильтрации `parsec-p` событиям по данному правилу;

2) правила аудита файлов (необходимы для работы `getfaud` и `setfaud`):

```
-a always,exit -F obj_type=faud -F arch=<архитектура> -S \
    <системные_вызовы> -k parsec-f
```

где `-a always,exit` — регистрировать события в журнале, добавить правило в список `exit` (события, происходящие при выходе из системного вызова);
`-F obj_type=faud` — обрабатывать правила, заданные с помощью `setfaud`;
`-F arch=<архитектура>` — аппаратная платформа;
`-S <системные_вызовы>` — перехватывать события при вызовах, указанных в `<системные_вызовы>` (список вызовов, разделенных запятой);
`-k parsec-f` — присвоить метку фильтрации `parsec-f` событиям по данному правилу;

3) правила сетевого аудита (анализируют работу системных вызовов, связанных со взаимодействием по сети):

```
-a always,exit -F subj_type=psaud -F arch=<архитектура> -S \
    <сетевые_системные_вызовы> -k parsec-p
```

где `-a always,exit` — регистрировать события в журнале, добавить правило в список `exit` (события, происходящие при выходе из системного вызова);
`-F subj_type=psaud` — обрабатывать правила, заданные с помощью `useraud` и `psaud`;
`-F arch=<архитектура>` — аппаратная платформа;
`-S <сетевые_системные_вызовы>` — перехватывать события при вызовах, указанных в `<сетевые_системные_вызовы>` (список вызовов, разделенных запятой);
`-k parsec-p` — присвоить метку фильтрации `parsec-p` событиям по данному правилу.

Включение и выключение правил PARSEC-аудита процессов и файлов выполняется с помощью инструмента командной строки `astra-audit-control` (описание приведено в 16.6.33). Включение и выключение правил сетевого PARSEC-аудита выполняется с помощью инструмента командной строки `astra-audit-network-control` (описание приведено в 16.6.34).

В дополнение к постоянным правилам PARSEC-аудита (файлов, процессов и сетевого) можно задавать собственные постоянные правила аудита.

Постоянные правила рекомендуется добавлять в файл `/etc/audit/rules.d/audit.rules`. При необходимости возможно создать в каталоге `/etc/audit/rules.d/` новый файл с произвольным именем и расширением `*.rules` и задать в нем необходимые правила. Файлы правил можно редактировать вручную. Файл `/etc/audit/rules.d/audit.rules` также можно редактировать с

помощью графической утилиты `system-config-audit` («Конфигурация аудита», описание утилиты приведено в электронной справке).

6.2. Регистрация событий на основе меток безопасности

В ОС поддерживается регистрация событий на основе меток безопасности субъектов и объектов. Для этого используются параметры `subj_type` и `obj_type` и операции «=» (равно) и «!=» (не равно) с ними.

Метка имеет следующий вид (при этом любое из значений может быть пустым):

```
<Уровень_конфиденциальности>:<Категория_целостности>:  
<Категории_конфиденциальности>:<Дополнительные_мандатные_атрибуты>
```

Уровень конфиденциальности и категория целостности задаются в виде десятичных чисел, а категории конфиденциальности и дополнительные мандатные атрибуты — в виде шестнадцатеричных чисел без префикса 0x. Числовые значения дополнительных мандатных атрибутов приведены в 4.3.2.

Примеры:

1. Регистрация событий доступа к объектам каталога субъектами нулевого уровня конфиденциальности:

```
auditctl -a always,exit -S all -F dir=/var/level1_dir -F perm=rwxa -F \  
subj_type=0:::
```

2. Регистрация событий доступа к сущностям с категорией целостности 63 и дополнительным мандатным атрибутом `ssi`:

```
auditctl -a exit,always -S all -F obj_type=0:63:0:40
```

6.3. Журнал аудита

Служба `auditd` регистрирует события безопасности в журнале аудита `/var/log/audit`.

Также в журнале аудита `/var/log/audit` регистрируются действия с журналом событий подсистемы регистрации событий (см. 6.5) — удаление, переименование, перемещение, ротация файла журнала событий.

Для просмотра журнала аудита используется графическая утилита `kssystemlog`, описание утилиты приведено в электронной справке.

Действия с журналом аудита `/var/log/audit` (удаление, переименование, перемещение файла журнала аудита) регистрируются подсистемой регистрации событий и указываются в журнале событий (см. 6.5.3).

6.4. Средства управления аудитом

6.4.1. Графические утилиты

Для управления аудитом могут использоваться следующие графические утилиты (описание утилит доступно в электронной справке):

- `astra-systemsettings` («Параметры системы») — управление аудитом, привилегиями и мандатными атрибутами пользователей, работа с пользователями и группами;
- `system-config-audit` («Конфигурация аудита») — включение и выключение регистрации событий, настройка службы `auditd`, настройка журнала аудита, а также добавление, удаление и редактирование правил аудита;
- `fly-admin-events` («Настройка регистрации системных событий») — утилита из состава подсистемы регистрации событий (см. 6.5), с помощью которой возможно настроить регистрацию событий запуска и остановки службы `auditd`, регистрацию событий добавления и удаления правил `auditd`, регистрацию действий с журналом аудита. Дополнительно утилита позволяет добавлять правила аудита.

6.4.2. `getfaud`

Инструмент командной строки `getfaud` используется для получения списков правил регистрации событий файлов и каталогов. Если файл является символической ссылкой, то будет получен список правил регистрации событий файла, на который она ссылается.

Синтаксис команды:

```
sudo getfaud [параметры] <файлы_и/или_каталоги>
```

Описание параметров `getfaud` приведено в таблице 36.

Таблица 36

Параметр	Описание
<code>-d, --default</code>	Вывести список регистрируемых событий по умолчанию для каталога. Данный список устанавливается на создаваемые в каталоге файлы и подкаталоги
<code>-R, --recursive</code>	Рекурсивно вывести список регистрируемых событий для подкаталогов

Окончание таблицы 36

Параметр	Описание
-L, --logical	Следовать по символическим ссылкам
-P, --physical	Не следовать по символическим ссылкам
-n, --numeric	Вывести флаги событий в числовом формате
-l, --long	Вывести полные имена флагов событий
-p, --absolute-names	Абсолютные имена
-c, --omit-header	Не показывать заголовков (имя файла)
-s, --skip-empty	Пропускать файлы с пустыми атрибутами
-h, --help	Вывести справку и выйти
-v, --version	Вывести информацию о версии и выйти

Следующие события доступны для регистрации:

- 1) o, open — открытие файла;
- 2) c, create — создание файла;
- 3) x, exec — исполнение файла;
- 4) u, delete — удаление файла (в каталоге);
- 5) r, acl — смена ACL;
- 6) n, chown — смена владельца;
- 7) m, mac — изменение метки;
- 8) y, modify — изменение файла;
- 9) a, audit — изменение списка регистрируемых событий файла;
- 10) d, chmod — изменение прав доступа к файлу.

Информация о списках посылается на стандартный вывод и может являться входными данными для инструмента `setfaud` (см. 6.4.3).

Описание инструмента приведено на справочной странице `man getfaud`.

6.4.3. setfaud

Инструмент командной строки `setfaud` позволяет устанавливать на файлы и каталоги списки правил регистрации событий. Если файл является символической ссылкой, то список правил регистрации событий будет установлен на файл, на который она ссылается.

Установить правила регистрации возможно:

- непосредственно с помощью параметров `-s`, `-m`;
- считать из файла с помощью параметров `-S`, `-M`, `-V`. Файлы могут быть сформированы перенаправлением вывода `getfaud` (см. 6.4.2).

Регистрируемые события приведены в 6.4.2. Описание инструмента и регистрируемые события также приведены на справочной странице `man setfaud`.

Синтаксис команды:

```
sudo setfaud [параметры] [файлы_и/или_каталоги]
```

Описание параметров `setfaud` приведено в таблице 37.

Таблица 37

Параметр	Описание
<code>-s <правила>, --set <правила></code>	Установить правила регистрации событий. Если уже установлены списки правил регистрации событий, то они будут заменены
<code>-b, --remove</code>	Удалить все элементы списка регистрации событий
<code>-m <правила>, --modify <правила></code>	Изменить существующие правила регистрации событий на указанные или добавить новые
<code>-d <правила>, --default <правила></code>	Для каталога изменить список регистрации событий по умолчанию. Данный список будет устанавливаться на создаваемые в каталоге объекты. Параметр всегда должен быть первым. Несовместим с параметром <code>-R</code>
<code>-S <файл>, --set-file <файл></code>	Установить правила регистрации событий из файла. Если до этого были установлены списки регистрации событий, то они будут заменены
<code>-X, --remove-all</code>	Удалить все списки регистрируемых событий
<code>-M <файл>, --modify-file <файл></code>	Изменить существующие правила регистрации событий на указанные в файле или добавить новое
<code>-B <файл>, --restore <файл></code>	Восстановить атрибуты из файла
<code>-R, --recursive</code>	Используется для каталогов. Назначить список регистрации событий рекурсивно
<code>-L, --logical</code>	Следовать по символическим ссылкам
<code>-P, --physical</code>	Не следовать по символическим ссылкам
<code>-h, --help</code>	Вывести справку и выйти
<code>-v, --version</code>	Вывести информацию о версии и выйти

Правила регистрации задаются в виде:

```
[u:<пользователь>:<флаги_регистрации>] [,g:<группа>:<флаги_регистрации>]
[,o:<флаги_регистрации>]
```

где <пользователь> и <группа> — символические или численные идентификаторы пользователя и группы;

u: — правила для пользователя;

g: — правила для группы;

o: — правила для всех остальных.

Флаги регистрации задаются в виде:

<флаги_успешных_событий>[:<флаги_неуспешных_событий>]

ВНИМАНИЕ! Если в команде установки правил регистрации указаны только флаги успешных событий, то они также будут применены и к неуспешным событиям.

Можно указать один или несколько флагов событий без разделителей и пробелов между ними (как успешных, так и неуспешных), при этом они могут иметь один из следующих видов:

1) флаг добавления или отключения регистрации события:

<+|-><имя_события>

Пример

Установка списка правил регистрации успешного события выполнения файла, регистрации неуспешного удаления файла и удаление правила регистрации неуспешного открытия файла пользователем user:

```
sudo setfaud -m u:user:+exec:+delete-open start.sh
```

2) флаг только добавления регистрации события:

<сокращенное_имя_события>

Примеры:

1. Регистрация успешных и неуспешных событий открытия и удаления файла (и никаких других) пользователями группы group:

```
sudo setfaud -s g:group:ou file.pdf
```

2. Регистрация всех успешных и неуспешных событий (кроме изменения) с файлом, выполняемых остальными пользователями:

```
sudo setfaud -m o:oxudnarmc:oxudnarmc file.pdf
```

Для каталогов также возможно установить список правил регистрации событий по умолчанию. Если на каталог установлен список правил регистрации по умолчанию, то на все файлы и каталоги, которые будут созданы в данном каталоге, будут установлены списки регистрации событий из списка по умолчанию. Если файл данного каталога является символической ссылкой, то список правил регистрации событий по умолчанию будет установлен на саму ссылку, а не на файл, на который она ссылается.

Для внесения изменений в список по умолчанию используется параметр `-d`. Данный параметр взаимоисключаем с параметром `-R`. При использовании параметра `-R` изменяются рекурсивно списки правил регистрации событий для каталога и всех уже существующих файлов и дочерних каталогов в нем. При использовании нескольких параметров параметр `-d` должен быть первым (`-ds` — правильно, `-sd` — не будет работать).

Примеры:

1. Регистрация успешных и неуспешных событий удаления файлов и дочерних каталогов в каталоге `/opt/test`, выполняемых пользователем `username`. Правила регистрации будут назначаться всем объектам, создаваемым в `/opt/test`. Правила для уже существующих объектов изменены не будут.

```
sudo setfaud -ds u:username:+delete:+delete /opt/test
```

2. Регистрация успешных и неуспешных событий запуска файлов на исполнение в каталоге `/opt/test`, выполняемых пользователем `username`. Правила регистрации будут назначены данному каталогу и уже существующим объектам в данном каталоге. Создаваемые в каталоге `/opt/test` объекты не будут иметь назначенных правил регистрации.

```
sudo setfaud -Rs u:username:+exec:+exec /opt/test
```

Для отключения регистрации всех событий в качестве флага регистрации событий используется `0`.

Пример

Отключение регистрации всех успешных и неуспешных событий действий пользователя `user` с файлом:

```
sudo setfaud -m u:user:0:0 file.pdf
```

При необходимости добавления или удаления правил регистрации событий одного типа, не затрагивая существующие правила для событий другого типа, следует для неизменяемых правил в качестве флага регистрации событий использовать `+0`.

Примеры:

1. Отключение регистрации неуспешного события удаления файла пользователем `user`, не меняя правила регистрации успешных событий:

```
sudo setfaud -m u:user:+0:-delete file.pdf
```

2. Отключение регистрации успешного события удаления файла пользователем `user`, не меняя правила регистрации неуспешных событий:

```
sudo setfaud -m u:user:-delete:+0 file.pdf
```

Для сброса назначенных на файлы/каталоги правил регистрации событий используется команда:

```
sudo setfaud -b <файлы_и/или_каталоги>
```

6.4.4. useraud

Инструмент командной строки `useraud` позволяет назначать, просматривать и изменять правила регистрации событий для пользователей и групп пользователей.

При назначении правил регистрации событий создаются в каталоге `/etc/parsec/audb/` файлы с именами вида `user:<UID>`, `group:<GID>` и `other:0` для правил пользователей, групп и всех остальных, соответственно. Если пользователю не назначены индивидуальные правила регистрации событий (отсутствует файл вида `/etc/parsec/audb/user:<UID>`), то для него применяются правила регистрации для группы. Если они также отсутствуют, то применяются правила регистрации событий для остальных пользователей (при их наличии).

Список регистрируемых событий приведен в 6.4.2. Описание инструмента и список регистрируемых событий также приведены на справочной странице `man useraud`.

Синтаксис команды:

```
useraud [параметр] [имя_пользователя(группы)] [--] [флаги_регистрации_событий]
```

Описание параметров инструмента `useraud` приведено в таблице 38.

Таблица 38

Параметр	Описание
<code>-d, --delete</code>	Сбросить правила регистрации событий
<code>-n, --numeric</code>	Вывести флаги событий в числовом формате

Окончание таблицы 38

Параметр	Описание
-l, --long	Вывести полные имена флагов событий
-g, --group	Правила регистрации событий для группы
-o, --other	Правила регистрации событий для остальных (любой пользователь)
-m, --modify	Назначить или изменить существующее правило регистрации событий
-h, --help	Вывести справку и выйти
-v, --version	Вывести информацию о версии и выйти

Для просмотра индивидуальных правил регистрации событий, назначенных локальному пользователю, используется команда:

```
useraud <имя_пользователя>
```

Для просмотра правил регистрации событий, назначенных одной или несколькими группам, используется команда:

```
useraud -g [имя_группы]
```

Если имя группы не задано, выводятся все группы с назначенными правилами.

Если правила регистрации событий для пользователя или группы не назначены (отсутствует соответствующий файл настроек в каталоге /etc/parsec/audddb), то при попытке их просмотра выводится сообщение:

```
useraud: невозможно прочесть файл настроек аудита: Нет такого файла или каталога
```

Для просмотра правил регистрации событий, назначенных всем остальным, используется команда:

```
useraud -o
```

Флаги регистрации событий задаются в виде:

```
<флаги_успешных_операций> [ : <флаги_неуспешных_операций> ]
```

ВНИМАНИЕ! Если в команде указаны только флаги успешных событий, то они также будут применены и к неуспешным событиям.

Пример

При добавлении флага регистрации успешного события выполнения файла:

```
useraud -m user1 +exec
```

также будет добавлен флаг регистрации неуспешного события выполнения файла.

Можно указать один или несколько флагов событий без разделителей и пробелов между ними (как успешных, так и неуспешных), при этом они могут иметь один из следующих видов:

1) флаг добавления или отключения регистрации события:

<+|-><имя_события>

Если первый флаг события в строке отключает регистрацию, то перед флагами необходимо использовать символы «--».

Примеры:

1. Регистрация успешного события выполнения файла и отключение регистрации неуспешного события открытия:

```
useraud -m user1 +exec:-open
```

2. Отключение регистрации успешного и неуспешного событий открытия файла, включение регистрации успешного события выполнения файла:

```
useraud -m user2 -- -open+exec:-open
```

2) флаг только добавления регистрации события:

<сокращенное_имя_события>

Пример

Регистрация успешных событий удаления и создания файла и неуспешных событий выполнения файла:

```
useraud -m user3 uc:x
```

Для отключения регистрации всех событий для пользователя/группы в качестве флага регистрации событий используется 0.

Пример

Отключение регистрации успешных и неуспешных событий для группы:

```
useraud -m -g group1 0:0
```

При необходимости добавления или удаления правил регистрации событий одного типа, не затрагивая существующие правила для событий другого типа, следует для неизменяемых правил в качестве флага регистрации событий использовать +0.

Примеры:

1. Отключение регистрации неуспешного события удаления файлов и/или каталогов, не меняя правил регистрации успешных событий:

```
useraud -m user1 +0:-delete
```

2. Отключение регистрации всех успешных событий для пользователя, не меняя правила регистрации неуспешных событий:

```
useraud -m user2 0:+0
```

Для сброса назначенных пользователю правил регистрации событий используется команда:

```
useraud -d <имя_пользователя>
```

В результате выполнения данной команды удаляется файл настроек регистрации /etc/parsec/auddb/user:<UID>, и к пользователю начинают применяться настройки регистрации событий для групп или для всех остальных (при их наличии). Удаление файла настроек вручную аналогично выполнению данной команды.

ВНИМАНИЕ! Отключение регистрации событий для пользователя командой:

```
useraud -m <имя_пользователя> 0:0
```

не приводит к удалению файла /etc/parsec/auddb/user:<UID>, поэтому к пользователю не будут применяться имеющиеся настройки для групп или всех остальных.

Изменения правил регистрации событий применяются к пользователю при его новом входе в систему.

6.4.5. psaud

Правила регистрации событий наследуются порожденными процессами. Инструмент командной строки `psaud` позволяет изменить или считать правила регистрации событий указанного процесса.

Список регистрируемых событий приведен в 6.4.2. Описание инструмента и список регистрируемых событий также приведены на справочной странице `man psaud`.

Синтаксис команды:

```
psaud [параметры] <идентификатор_процесса> [--] [флаги_регистрации]
```

Описание параметров `psaud` приведено в таблице 39.

Т а б л и ц а 39

Параметр	Описание
<code>-d, --delete</code>	Сбросить правила регистрации событий
<code>-n, --numeric</code>	Выводить информацию о правилах регистрации событий в числовом формате
<code>-l, --long</code>	Выводить информацию о правилах регистрации событий в полной форме
<code>-h, --help</code>	Вывести справку и выйти
<code>--version</code>	Вывести информацию о версии и выйти

Если в команде указаны флаги регистрации, то инструмент устанавливает указанные флаги регистрации событий на процесс. Если флаги регистрации событий не указаны, то инструмент считывает их с процесса, заданного параметром (идентификатор процесса).

Флаги регистрации задаются в виде:

```
<флаги_успешных_событий>[:<флаги_неуспешных_событий>]
```

ВНИМАНИЕ! Если в команде указаны только флаги успешных событий, то они также будут применены и к неуспешным событиям.

Можно указать один или несколько флагов событий без разделителей и пробелов между ними (как успешных, так и неуспешных), при этом они могут иметь один из следующих видов:

1) флаг добавления или отключения регистрации события:

```
<+|-><имя_события>
```

Если первый флаг события в строке отключает регистрацию, то перед флагами необходимо использовать символы «--».

Примеры:**1. Регистрация успешного события выполнения файла:**

```
psaud 10234 +exec
```

2. Отключение регистрации успешного события открытия файла и включение регистрации успешного события выполнения файла:

```
psaud 10234 -- -open+exec
```

2) флаг только добавления регистрации события:

<сокращенное_имя_события>

Пример

Регистрация успешных событий открытия и удаления файла:

```
psaud 10234 ou
```

Для отключения регистрации всех событий для процесса в качестве флага регистрации событий используется 0.

Пример

Отключение регистрации успешных и неуспешных событий для процесса:

```
psaud 10234 0:0
```

При необходимости добавления или удаления правил регистрации событий одного типа, не затрагивая существующие правила для событий другого типа, следует для неизменяемых правил в качестве флага регистрации событий использовать +0.

Примеры:**1. Отключение регистрации неуспешного события удаления для процесса, не меняя правил регистрации успешных событий:**

```
psaud 10234 +0:-delete
```

2. Отключение регистрации успешного события удаления для процесса, не меняя правила регистрации неуспешных событий:

```
psaud 10234 -- -delete:+0
```

Для сброса назначенных процессу правил регистрации событий используется команда:

```
psaud -d <идентификатор_процесса>
```

6.4.6. ausearch

Инструмент командной строки `ausearch` предназначен для просмотра файлов журнала регистрации событий ядра, а также событий пользователя.

Синтаксис команды:

```
ausearch [параметры]
```

Описание параметров `ausearch` приведено в таблице 40.

Таблица 40

Параметр	Описание
<code>-a, --event</code>	Поиск событий с заданным идентификатором события. Сообщения обычно начинаются с записи вида <code>msg=audit(1116360555.329:2401771)</code> . Идентификатор события — число после «:». Все события аудита, связанные с одним системным вызовом, имеют одинаковый идентификатор
<code>-c, --comm</code>	Искать события с заданным именем исполняемого файла задачи
<code>--debug</code>	Вывести информацию о некорректных событиях, не попавших в стандартный вывод ошибок
<code>-e, --exit</code>	Искать события по заданному коду завершения системного вызова или коду последней ошибки
<code>-f, --file</code>	Искать события с заданным именем файла
<code>-ga, --gid-all</code>	Искать события с заданным эффективным или обычным идентификатором группы
<code>-ge, --gid-effective</code>	Искать события с заданным эффективным идентификатором группы или именем группы
<code>-gi, --gid</code>	Искать события с заданным идентификатором группы или именем группы
<code>-h, --help</code>	Вывести справку и выйти
<code>-hn, --host</code>	Искать события с заданным именем узла. Имя узла может быть именем узла, полным доменным именем или сетевым адресом
<code>-i, --interpret</code>	Транслировать числовые значения в текстовые. Например, идентификатор пользователя будет транслирован в имя пользователя. Трансляция выполняется с использованием данных с той машины, где запущен <code>ausearch</code>

Продолжение таблицы 40

Параметр	Описание
-if, --input	Использовать указанный файл или каталог вместо журнала аудита. Может быть полезно при анализе журнала аудита с другой машины или при анализе частично сохраненного журнала
--input-logs	Использовать расположение файла журнала, указанное в auditd.conf, в качестве входных данных для поиска. Это необходимо при использовании ausearch в задаче cron
--just-one	Искать до первого совпадения
-k, --key	Искать события с заданным ключевым словом
-l, --line-buffered	Очистить вывод каждой строки. Может быть полезно, когда стандартный вывод конвейерно перенаправляется и использование стратегии буферизации блоков по умолчанию нежелательно
-m, --message	Искать события с заданным типом. Возможно указать список значений, разделенных запятыми. Для получения всех сообщений системы аудита указать тип ALL. Если указать данный параметр без значения, будет выведен список допустимых типов. Тип сообщения может быть строкой или числом
-n, --node	Вывести события с указанной машины. Возможно указать несколько машин, тогда выводятся события для каждой из указанных машин
-o, --object	Искать события с заданным контекстом (объектом)
-p, --pid	Искать события с заданным идентификатором процесса
-pp, --ppid	Искать события с заданным идентификатором родительского процесса
-r, --raw	Необработанный вывод. Используется для извлечения записей для дальнейшего анализа
-sc, --syscall	Искать события с заданным системным вызовом. Можно указать его номер или имя. Если указано имя, оно будет проверено на машине, где запущен ausearch
-se, --context	Искать события с заданным контекстом (scontext/subject или tcontext/object)
-su, --subject	Искать события с заданным контекстом scountext (subject)
-sv, --success	Искать события с заданным флагом успешного выполнения. Допустимые значения: yes (успешно) и no (неуспешно)
-te, --end	Искать события, которые произошли в указанное время или раньше. Формат даты и времени зависит от региональных настроек. Если дата не указана, то подразумевается текущий день (today). Если не указано время, то подразумевается текущий момент (now). Используется 24-часовая нотация времени. Например, дата может быть задана как 10/24/2005, а время — 18:00:00
-ts, --start	Искать события, которые произошли в указанное время или позже. Формат даты и времени зависит от региональных настроек. Если дата не указана, то подразумевается текущий день (today). Если не указано время, то подразумевается полночь (midnight). Используется 24-часовая нотация времени. Например, дата может быть задана как 10/24/2005, а время — 18:00:00

Окончание таблицы 40

Параметр	Описание
-tm, --terminal	Искать события с заданным терминалом. Некоторые службы (например, cron и atd) используют имя службы как имя терминала
-ua, --uid-all	Искать события, у которых идентификатор пользователя, эффективный идентификатор пользователя или loginuid (auid) совпадают с заданным идентификатором пользователя
-ue, --uid-effective	Искать события с заданным эффективным идентификатором пользователя
-ui, --uid	Искать события с заданным идентификатором пользователя
-ul, --loginuid	Искать события с заданным идентификатором пользователя. Все программы, которые его используют, должны использовать pam_loginuid
-uu, --uuid	Искать события с заданным uuid
-v, --version	Показать версию и выйти
-vm, --vm-name	Совпадение с полным словом. Поддерживается для имени файла, имени узла, терминала и контекста
-w, --word	Поиск по заданной строке с полным совпадением. Поддерживается для имени файла, имени узла, терминала и контекста
-x, --executable	Искать события с заданным именем исполняемой программы

Описание инструмента приведено на справочной странице `man ausearch`.

Примеры:

1. Отобразить события процессов и пользователей

```
ausearch -i -ts "11:35:35" -te "11:36:00" -k parsec-p
```

2. Отобразить события с файлами

```
ausearch -i -ts "11:35:35" -te "11:36:00" -k parsec-f
```

3. Отобразить события, связанные с управлением доступом и аудитом

```
ausearch -i -ts "11:35:35" -te "11:36:00" -m avc
```

4. Отобразить события, связанные с выполненными пользователями командами

```
ausearch -i -ts "11:35:35" -te "11:36:00" -m user_cmd
```

6.4.7. Отключение регистрации событий

При необходимости возможно отключить регистрацию событий в журнал, но правила PARSEC-аудита все равно продолжат обрабатываться и загружать ресурсы ОС.

Отключение регистрации набора системных вызовов возможно выполнить одним из следующих способов:

1) отключение регистрации системных вызовов, не используемых для мандатного управления доступом — выполнить команду:

```
echo 1 > /parsecfs/disable-non-mac-audit
```

Для проверки состояния регистрации выполнить команду:

```
cat /parsecfs/disable-non-mac-audit
```

Если вывод команды равен 1, то регистрация системных вызовов, не используемых для мандатного управления доступом, отключена;

2) отключение регистрации всех системных вызовов — выполнить команду:

```
echo 1 > /parsecfs/disable-all-audit
```

Для проверки состояния регистрации выполнить команду:

```
cat /parsecfs/disable-all-audit
```

Если вывод команды равен 1, то регистрация всех системных вызовов отключена;

3) отключение регистрации запретов доступа (на основе мандатного управления доступом) — выполнить команду:

```
echo 1 > /parsecfs/disable-denied-audit
```

Для проверки состояния регистрации выполнить команду:

```
cat /parsecfs/disable-denied-audit
```

Если вывод команды равен 1, то регистрация запретов доступа отключена.

Для оптимизации работы и уменьшения нагрузки на ОС рекомендуется выполнять отключение регистрации событий с помощью инструмента командной строки `astra-audit-control` (описание приведено в 16.6.33). В таком случае будет отключена как регистрация событий в журнал, так и сама обработка правил PARSEC-аудита.

6.5. Подсистема регистрации событий

6.5.1. Регистрация событий и уведомление о событиях

В ОС реализована подсистема регистрации событий, которая собирает информацию о событиях, в том числе о событиях безопасности, из различных источников и предоставляет инструменты для просмотра собранных данных и реагирования на события.

Подсистема регистрации событий включает следующие основные компоненты:

- 1) менеджер и маршрутизатор событий `syslog-ng` — основная служба подсистемы регистрации событий, которая реализована в соответствии со стандартом Syslog. Служба `syslog-ng` принимает информацию о событиях из различных источников (события от `auditd`, собственные подключаемые модули, файлы, прикладное ПО и др.), выполняет фильтрацию и обработку полученных данных, регистрирует события в журнал, а также, в зависимости от конфигурации, сохраняет в файл, отправляет по сети и т.д.;
- 2) модуль `syslog-ng-mod-astra` — модуль для `syslog-ng`, выполняющий дополнительную обработку и фильтрацию событий;
- 3) `astra-event-watcher` — службы уведомления пользователя о событиях, обработанных менеджером `syslog-ng`;
- 4) `astra-event-diagnostics` — инструмент диагностики подсистемы регистрации событий, описание инструмента приведено в 6.5.4.

Работа модуля `syslog-ng-mod-astra` настраивается в следующих конфигурационных файлах:

- 1) `/etc/astra-syslog.conf` — список регистрируемых событий;
- 2) `/usr/share/syslog-ng-mod-astra/event-settings/` — каталог с файлами настроек по умолчанию для каждого события.

Для настройки регистрации событий и регистрируемых параметров событий используется инструмент командной строки `astra-admin-events` или графическая утилита `fly-admin-events` («Настройка регистрации системных событий»), описание утилиты приведено в электронной справке.

Для просмотра с помощью инструмента `astra-admin-events` списка регистрируемых событий необходимо выполнить команду без параметров:

```
astra-admin-events
```

В выводе списка событий значение рядом с событием:

- «1» — событие регистрируется;
- «0» — событие не регистрируется.

Синтаксис команды:

```
astra-admin-events [параметры]
```

Описание параметров инструмента приведено в таблице 41.

Таблица 41

Параметр	Описание
-E, --event-id	Вывести подробную информацию о событии (событиях), если не указаны другие параметры
-G, --group-id	Вывести статус событий из группы (групп), если не указаны другие параметры
-e, --enable	Включить события, которые указаны параметром -E или -G
-d, --disable	Отключить события, которые указаны параметром -E или -G
-p, --priority	Назначить приоритет событиям, которые указаны параметром -E или -G
-f, --facility	Назначить тип событиям, которые указаны параметром -E или -G
-P, --properties	Назначить свойства событиям, которые указаны параметром -E или -G
-i, --import <имя_файла>	Импортировать конфигурацию из указанного файла
-o, --export <имя_файла>	Экспортировать конфигурацию в указанный файл
-h, --help	Показать справочное сообщение и выйти

Порядок использования инструмента приведен в `man astra-admin-events`.

Модуль `syslog-ng-mod-astra` информацию о событиях регистрирует в следующих файлах:

- 1) `/parsec/log/astra/events` — журнал событий в формате JSON, описание журнала приведено в 6.5.3;
- 2) `/var/log/astra/prevlogin-<имя_пользователя>` — журнал в формате JSON сводной статистики предыдущих входов в систему пользователя `<имя_пользователя>`. Включает данные о последней завершенной сессии данного пользователя, а также количество успешных и неуспешных входов данного пользователя со времени начала ведения статистики. Доступен для чтения только пользователю `<имя_пользователя>`.

Параметры уведомлений службы `astra-event-watcher` определены в файле `/usr/share/knotifications5/astra-event-watcher.notifyrc`. Передача данных выполняется с использованием D-Bus. Реализация действий осуществляется модулем KNotifications. Служба получает информацию только о событиях, для которых настроено реагирование. Служба, запущенная от имени определенного пользователя, реагирует только на те события, которые настроены для данного пользователя. Для этого служба отслеживает сигнал, передаваемый `syslog-ng` по D-Bus всем заинтересованным приложениям. В сигнале также передается список имен пользователей и имен групп, для которых требуется

реакция на событие.

Параметры уведомлений для определенного пользователя могут быть переопределены в файле `/home/<имя_пользователя>/.config/astra-event-watcher.notifyrc`.

Информирование (оповещение) о событиях безопасности осуществляется с помощью графической утилиты `fly-notifications` («Центр уведомлений»), описание утилиты приведено в электронной справке.

6.5.2. Конфигурационный файл службы `syslog-ng`

Работа службы `syslog-ng` настраивается в конфигурационном файле по умолчанию `/etc/syslog-ng/syslog-ng.conf` и в пользовательских конфигурационных файлах вида `/etc/syslog-ng/conf.d/<имя_файла>.conf`. Описание структуры и синтаксиса конфигурационного файла приведено на справочной странице `man syslog-ng.conf`.

В конфигурационном файле `/etc/syslog-ng/syslog-ng.conf` не допускается удалять или отключать комментированием следующие параметры:

1) источники по умолчанию (секция `Sources`):

```
source s_src {  
    system();  
    internal();  
};
```

2) инструкции `@include` для включения других конфигурационных файлов;

3) стандартные фильтры (секция `Filters`).

Без указания в конфигурационном файле любого из данных параметров служба `syslog-ng` не сможет запуститься. Это приведет к неработоспособности подсистемы регистрации событий.

Если из конфигурационного файла `/etc/syslog-ng/syslog-ng.conf` удалить строки, содержащие источники по умолчанию (из секции `Sources`) и стандартные фильтры регистрации событий (из секции `Filters`), то они будут восстановлены при самодиагностике (см. 6.5.4).

При необходимости исключить стандартные фильтры регистрации событий можно закомментировать их непосредственно внутри строк `log` (секция `Log paths`) или закомментировать полностью использующие их строки `log`.

6.5.3. Журнал событий

Служба `syslog-ng` выполняет регистрацию событий в журнал `/var/log/astra/events`. В журнале событий регистрируются попытки запуска

неподписанных файлов, успешная и неуспешная аутентификация, успешная и неуспешная авторизация, данные о пользовательских сессиях и другие события безопасности, в соответствии с настройками регистрации событий (см. 6.5.1). Также в журнале событий регистрируются действия с журналом аудита `/var/log/audit` службы `auditd` (см. 6.3) — удаление, переименование, перемещение файла журнала аудита.

Каждая запись в журнале событий состоит из набора параметров события, для каждого параметра указано его значение.

Файл журнала событий доступен для чтения только пользователям из группы `astra-audit`.

Файл журнала событий доступен для чтения и записи в конец файла только администратору. Разрешение на запись в конец файла задано установкой на файл атрибута `a` (с помощью `chattr`).

При включенном МКЦ файл журнала событий имеет высокую метку целостности. Процесс, осуществляющий регистрацию событий в файле журнала, также имеет высокую метку целостности. Таким образом, изменение файла журнала событий доступно только администратору.

Для просмотра журнала событий может использоваться графическая утилита `fly-event-viewer` («Журнал системных событий», описание утилиты приведено в электронной справке), а также инструмент командной строки `astra-event-viewer`. Порядок использования инструмента приведен на странице помощи:

```
astra-event-viewer -h
```

По умолчанию отображается только часть параметров событий, записанных в журнале событий `/parsec/log/astra/events`.

Чтобы с помощью инструмента командной строки `astra-event-viewer` вывести все параметры событий и их значения, выполнить команду:

```
astra-event-viewer --more-info
```

При этом будут выведены все значения параметров событий, записанные в журнале событий `/parsec/log/astra/events`.

В графической утилите `fly-event-viewer` записи из журнала событий `/parsec/log/astra/events` отображаются в виде таблицы, где строки соответствуют событиям, а столбцы — их параметрам (имя события, тип события, критичность и другие). Параметры событий, для которых не предусмотрены отдельные столбцы, отображаются в столбце «Нераспознанная информация». Чтобы добавить столбцы для параметров

событий, необходимо:

- 1) в каталоге `/etc/fly-event-viewer/emp/` создать конфигурационный файл с произвольным именем и расширением `*.conf` (если путь не существует, то предварительно создать соответствующие каталоги);
- 2) для каждого параметра добавить в созданный конфигурационный файл запись в формате JSON вида:

```
[
{ "name": "<имя_параметра>", "English": "<описание параметра на
    английском языке>", "Russian": "<описание параметра на
    русском языке>" }
]
```

Пример

```
[
{ "name": "domainvm", "English": "Virtual machine ID", "Russian":
    "Идентификатор виртуальной машины" },
{ "name": "user", "English": "User", "Russian": "Пользователь" },
{ "name": "file", "English": "File", "Russian": "Файл" },
{ "name": "hostip", "English": "Hypervisor network address",
    "Russian": "Сетевой адрес гипервизора" }
]
```

ВНИМАНИЕ! Указанное имя параметра должно точно соответствовать имени параметра в журнале событий.

Действия с журналом событий (удаление, переименование, перемещение, ротация файла журнала событий) регистрируются подсистемой регистрации событий и указываются первой записью в журнале событий:

- удаление журнала событий регистрируется событием «Журнал событий удален»;
- переименование или перемещение журнала событий регистрируется событием «Журнал событий переименован или перемещен»;
- ротация журнала событий регистрируется событием «Журнал событий ротирован»;
- действия с журналом событий недоверенными процессами (всеми процессами, кроме процессов `syslog-ng` и `logrotate`) регистрируются событием «Журнал событий изменен недоверенным процессом».

Также действия с журналом событий регистрируются службой `auditd` и указываются в журнале аудита `/var/log/audit` (см. 6.3).

6.5.4. Самодиагностика подсистемы регистрации событий

Инструмент `astra-event-diagnostics` обеспечивает самодиагностику подсистемы регистрации событий.

Самодиагностика выполняется:

- 1) автоматически при загрузке ОС после запуска служб `syslog-ng` и `auditd`;
- 2) периодически в процессе функционирования ОС, периодичность самодиагностики настраивается в конфигурационном файле `/etc/astra-event-diagnostics/astra-event-diagnostics.yaml`.

Также провести диагностику возможно, выполнив команду:

```
sudo astra-event-diagnostics
```

Описание параметров команды приведено на странице помощи:

```
sudo astra-event-diagnostics -h
```

При выполнении самодиагностики производятся следующие проверки:

- 1) запуск и работоспособность служб `syslog-ng` и `auditd`;
- 2) наличие необходимых модулей из состава `syslog-ng-mod-astra`;
- 3) корректность конфигурационных файлов:
 - а) `/etc/audit/rules.d/astra-syslog.rules`;
 - б) `/etc/astra-syslog.conf`;
 - в) `/etc/syslog-ng/conf.d/mod-astra.conf`;
 - г) отдельных файлов из каталога `/usr/share/syslog-ng-mod-astra/event-settings/`.

Если из конфигурационного файла `/etc/syslog-ng/syslog-ng.conf` (см. 6.5.2) были удалены строки, содержащие стандартные источники из секции `Sources` и стандартные фильтры регистрации событий из секции `Filters`, то после запуска самодиагностики подсистемы регистрации событий удаленные строки будут восстановлены и записаны в конец конфигурационного файла.

Если в процессе самодиагностики было обнаружено, что подсистема регистрации событий перестала функционировать или произошел сбой в ее работе, то будет выполнен ее перезапуск.

Возможно настроить автоматическое блокирование учетной записи пользователя, если в результате самодиагностики выявлены ошибки. Для этого необходимо в конфигурацион-

ном файле `/etc/astra-event-diagnostics/astra-event-diagnostics.yaml` в секции `user_blocker` для параметра `enabled` установить значение `true`.

Разблокировка учетной записи, заблокированной после выявления ошибок при самодиагностике, может быть выполнена с использованием модуля «Пользователи» в разделе «Пользователи и группы» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля использовать команду:

```
astra-systemsettings astra_kcm_users
```

Регистрация событий о результатах самодиагностики осуществляется:

- 1) в журнал `/parsec/log/astra/astra-event-diagnostics`;
- 2) в журнал событий `/parsec/log/astra/events`, если подсистема регистрации событий работоспособна.

Запись информации о результатах самодиагностики в диагностический архив с отчетом выполняется средствами модуля `astra-sosreport` с помощью инструмента командной строки `sos report`. Подробная информация об инструменте `sos report` приведена в документе РУСБ.10015-01 95 01-1 и в `man sos report`.

Если в результате самодиагностики были выявлены ошибки в работе подсистемы регистрации событий, то события о результатах регистрируются с уровнем важности «Критический».

Уведомление о результатах самодиагностики обеспечивает служба `astra-event-watcher`.

Информирование о событиях осуществляется с помощью утилиты `fly-notifications` («Центр уведомлений»). Описание утилиты приведено в электронной справке.

6.6. Средства централизованного аудита и протоколирования

В состав ОС входит программное средство Zabbix, обеспечивающее функционал для настройки, сбора данных и мониторинга состояния сети, а также жизнеспособности и целостности ресурсов сети. Описание установки и настройки Zabbix приведено в документе РУСБ.10015-01 95 01-1.

7. ИЗОЛЯЦИЯ ПРОЦЕССОВ

Ядро ОС обеспечивает для каждого процесса в системе собственное изолированное адресное пространство. Данный механизм изоляции основан на страничном механизме защиты памяти, а также механизме трансляции виртуального адреса в физический, поддерживаемый модулем управления памятью. Одни и те же виртуальные адреса (с которыми и работает процессор) преобразуются в разные физические для разных адресных пространств. Процесс не может несанкционированным образом получить доступ к пространству другого процесса, т. к. непривилегированный пользовательский процесс лишен возможности работать с физической памятью напрямую.

Механизм разделяемой памяти является санкционированным способом получить нескольким процессам доступ к одному и тому же участку памяти и находится под контролем дискреционных и мандатных ПРД.

Адресное пространство ядра защищено от прямого воздействия пользовательских процессов с использованием механизма страничной защиты. Страницы пространства ядра являются привилегированными и доступ к ним из непривилегированного кода вызывает исключение процессора, которое обрабатывается корректным образом ядром ОС. Единственным санкционированным способом доступа к ядру ОС из пользовательской программы является механизм системных вызовов, который гарантирует возможность выполнения пользователем только санкционированных действий.

8. СОЗДАНИЕ И ЗАЩИТА ИЗОЛИРОВАННЫХ ПРОГРАММНЫХ СРЕД (КОНТЕЙНЕРОВ)

8.1. Изоляция контейнеров и пространств

ОС содержит программные средства (средства контейнеризации), реализующие создание и функционирование изолированных программных сред с обеспечением выполнения следующих функций безопасности:

- изоляция контейнеров;
- выявление уязвимостей в образах контейнеров;
- проверка корректности конфигурации контейнеров;
- контроль целостности контейнеров и их образов;
- регистрация событий безопасности;
- идентификация и аутентификация пользователей.

Средства контейнеризации реализуют функциональные возможности по созданию образов контейнеров, формированию среды выполнения контейнеров и обеспечения выполнения их процессов, запуску контейнера, а также позволяют управлять данным контейнером. Также возможно создание групп контейнеров («подов»), в которых контейнеры выполняются совместно, разделяя между собой общие ресурсы.

В ОС реализованы механизмы изоляции уровня ядра Linux, описание которых приведено в разделе 7. Механизмы изоляции реализуют функции изоляции контейнеров, включая изоляцию пространств идентификаторов процессов контейнеров и пространств имен.

Изоляцию контейнеров обеспечивают следующие механизмы:

- `namespaces` — изоляция пространств имен;
- групп управления `control group (cgroups)` — управление доступом контейнеров к ресурсам хостовой ОС;
- `Linux Secure Computing Mode (seccomp)` — ограничение доступа контейнеров к системным вызовам ядра хостовой ОС.

Применение указанных механизмов обеспечивает работу контейнеров независимо друг от друга и от хостовой ОС.

Механизм `cgroups` обеспечивает управление и ограничение доступа контейнеров к ресурсам хостовой ОС. Механизм используется для ограничения в ресурсах группы процессов, выполняющихся в контейнере. При запуске контейнера для него создается отдельная группа `cgroups`, к которой относятся все процессы, запущенные внутри контейнера. Также механизм может использоваться для ограничения других групп, например групп процессов, принадлежащих какой-либо службе или пользовательской сессии.

Различают следующие контроллеры:

- 1) `cpu` — накладывает ограничения на долю процессорного времени (`cpu share`);
- 2) `cpuacct` — подсчитывает потребленные ресурсы процессора;
- 3) `cgroup` — привязывает процессы к конкретным процессорам;
- 4) `memory` — ограничивает потребление памяти;
- 5) `devices` — ограничивает доступ к файлам устройств;
- 6) `freezer` — позволяет приостанавливать и возобновлять работу групп процессов;
- 7) `net_cls` и `net_prio` — назначают сетевому трафику процессов класс и приоритет обработки для использования сетевыми фильтрами и планировщиками;
- 8) `blkio` — ограничивает количество запросов к блочным (дисковым) устройствам при помощи планировщика CFQ;
- 9) `pids` — накладывает ограничение на количество процессов в группе.

Изоляция доступа контейнеров к системным вызовам ядра хостовой ОС реализуется механизмом ядра `seccomp`. Механизм позволяет определять системные вызовы, которые будут доступны процессу контейнера, при этом все дочерние процессы наследуют эти ограничения. Механизм обеспечивает блокировку запуска контейнеров с неразрешенными системными вызовами, а также блокирует возможность выполнить произвольный код с использованием системных вызовов, которые не были разрешены. Подробное описание использования `seccomp` приведено в 8.3.

В состав ОС входит программное обеспечение `Docker` и `Podman` для автоматизации развертывания и управления приложениями в средах с поддержкой контейнеризации. Описание установки, настройки и работы с контейнерами `Docker` и `Podman` приведено в РУСБ.10015-01 95 01-1.

Дополнительно для изоляции среды функционирования контейнера в ОС предусмотрена возможность запуска контейнера `Docker` с пониженной (выделенной) категорией целостности 2.

По умолчанию функция запуска контейнеров на пониженном уровне МКЦ выключена. Управление функцией производится с помощью инструмента командной строки `astra-docker-isolation`, описание которого приведено в 16.6.13.

Также в ОС сохранен для совместимости инструмент командой строки `docker-isolation`. Для включения функции запуска контейнеров на пониженном уровне МКЦ с использованием этого инструмента следует выполнить:

```
sudo /usr/share/docker.io/contrib/parsec/docker-isolation on
sudo systemctl restart containerd
sudo systemctl restart docker
```

Для выключения функции следует выполнить:

```
sudo /usr/share/docker.io/contrib/parsec/docker-isolation off
sudo systemctl restart containerd
sudo systemctl restart docker
```

Описание запуска Docker на пониженном уровне целостности приведено в РУСБ.10015-01 97 01-1.

8.2. Выявление уязвимостей в образах контейнеров

8.2.1. Общие сведения

В состав ОС входит программное обеспечение OpenSCAP, обеспечивающее поиск уязвимостей в образах контейнеров. OpenSCAP использует библиотеку `libopenscap` для сканирования. Сканирование уязвимостей выполняется с использованием файла базы данных уязвимостей, рекомендуемого ФСТЭК России и содержащего сведения об уязвимостях банка данных угроз безопасности информации ФСТЭК России. В качестве файла базы данных уязвимостей используется файл формата OVAL.

Система сканирования представлена пакетами `openscap-scanner` и `oval-db`. Пакеты устанавливаются автоматически при установке Docker или Podman. При необходимости пакеты могут быть установлены отдельно. Актуальные версии oval-описаний устанавливаются вместе с пакетом `oval-db`, в дальнейшем можно настроить автоматическое обновление (см. 8.2.3.3).

Oval-описания содержатся в отдельных файлах для каждого уровня угрозы (критический, высокий, средний, низкий, отсутствует) и группируются по имени и версии операционной системы образа в каталогах `/usr/share/oval/db/<имя>/<версия>`.

Имя и идентификатор версии (код релиза ОС) операционной системы образа содержатся внутри образа в файле `/etc/os-release` в значениях параметров `ID` и `VERSION_ID` соответственно.

Пример

```
PRETTY_NAME="Astra Linux"
NAME="Astra Linux"
ID=astra
ID_LIKE=debian
ANSI_COLOR="1;31"
HOME_URL="https://astralinux.ru"
SUPPORT_URL="https://astralinux.ru/support"
LOGO=astra
VERSION_ID=1.8_x86-64
```

VERSION_CODENAME=1.8_x86-64
VARIANT_ID=se

Проверка образов и контейнеров на наличие уязвимостей выполняется автоматически при следующих событиях:

- при создании образов:
 - создание образа из Dockerfile или из контейнера;
 - загрузка образа из архива или потока ввода;
 - создание файловой системы образа из архива;
 - скачивание образа из реестра;
- при запуске контейнеров на основе образов;
- периодически с заданным периодом выполнения.

При обнаружении уязвимостей дальнейшее использование образа контейнера запрещено. Соответствующее ограничение по эксплуатации приведено в 18.2.

Возможно указать уязвимости, обнаружение которых будет проигнорировано при проверке заданных образов (см. 8.2.3.5).

Блокировка запуска контейнеров, в образах которых обнаружены уязвимости, осуществляется в соответствии с классом защиты, который возможно выбрать с помощью графического конфигулятора (см. 8.2.3.6). Всего имеется шесть классов защиты:

- 1) 1-5 классы защиты — при обнаружении уязвимости в контейнере его запуск блокируется;
- 2) 6 класс защиты — отладочный режим, при обнаружении уязвимости в контейнере выводится соответствующее предупреждение, при этом запуск контейнера не блокируется.

События безопасности, связанные с выявлением уязвимостей в контейнере, регистрируются в соответствии с описанием 8.5.

Для устранения обнаруженной уязвимости и последующего запуска контейнера без блокировки необходимо:

- 1) запустить средство контейнеризации в режиме отладки (6 класс защиты);
- 2) запустить контейнер и устранить уязвимость;
- 3) запустить средство контейнеризации с требуемым классом защиты.

8.2.2. Базовое использование

После установки пакетов система сканирования уязвимостей готова к работе со следующими настройками по умолчанию:

- 1) включено сканирование при создании образов;
- 2) включено сканирование при запуске контейнеров на основе образов;
- 3) включено периодическое сканирование образов с периодом 168 часов (7 суток).

Сканирование контейнеров при запуске выполняется в следующих случаях:

- образ контейнера ранее не сканировался;
- образ контейнера был изменен;
- с момента предыдущего сканирования прошло более 48 часов.

8.2.3. Настройка сканирования и работа с oval-описаниями

8.2.3.1. Инструмент oval-db

Для работы с oval-описаниями и изменения параметров сканирования используется инструмент командной строки `oval-db`.

Синтаксис команды:

```
sudo oval-db <действие> [параметры]
```

Действия инструмента приведены в таблице 42.

Таблица 42

Действие	Описание
<code>auto-update</code>	Включить или отключить автоматическое обновление oval-описаний
<code>config</code>	Запустить графический конфигуратор <code>oval-db</code>
<code>history</code>	Вывести историю сканирований на уязвимости
<code>list</code>	Вывести список загруженных файлов oval-описаний
<code>load</code>	Загрузить файл oval-описания из XML-файла
<code>restore</code>	Восстановить файл oval-описания из резервной копии
<code>remove</code>	Удалить загруженный файл oval-описания
<code>status</code>	Вывести состояние службы <code>oval-dbd</code>
<code>update</code>	Принудительно обновить oval-описания
<code>vul-info</code>	Вывести информацию о конкретной уязвимости
<code>wl-reload</code>	Загрузить файл игнорируемых уязвимостей

В качестве параметра можно указать `-h` или `--help` — будет выведена страница помощи по указанному действию.

Для просмотра страницы помощи по инструменту `oval-db` выполнить:

```
sudo oval-db -h
```

8.2.3.2. Действия с файлами oval-описаний

Для вывода списка загруженных файлов oval-описаний используется действие `list`.

Синтаксис команды:

```
sudo oval-db list [параметры]
```

Параметры команды приведены в таблице 43.

Таблица 43

Параметр	Описание
<code>-b, --backup-include</code>	Включить в список каталоги с резервными копиями предыдущих версий файлов oval-описаний
<code>-c, --check</code>	Включить в список результаты проверки контрольных сумм загруженных файлов oval-описаний
<code>-h, --help</code>	Вывести справку и выйти

Пример

```
sudo oval-db list -b
```

Вывод команды:

```
astra:1.7_x86-64; Critical: 239, High: 312, Low: 2221, Medium: 16574
astra:1.7_x86-64 (Backup); High: 306, Low: 128, Medium: 2393, None: 548
astra:1.8_x86-64; Critical: 4, High: 206, Low: 8, Medium: 97, None: 7
astra:1.8_x86-64 (Backup); Critical: 11, High: 206, Low: 8, Medium: 97
ubuntu:24.04; Medium: 16574, Critical: 239, High: 312, Low: 2221
```

Для ручного добавления файлов oval-описаний используется действие `load`.

Синтаксис команды:

```
sudo oval-db load [параметры]
```

Параметры команды приведены в таблице 44.

Таблица 44

Параметр	Описание
<code>-f, --filepath <путь></code>	Путь к загружаемому файлу oval-описания. Обязательный параметр
<code>-o, --os <имя></code>	Имя операционной системы, для которой загружается oval-описание. Обязательный параметр
<code>-v, --version <идентификатор></code>	Идентификатор версии операционной системы (код релиза ОС), для которой загружается oval-описание. Обязательный параметр
<code>-h, --help</code>	Вывести справку и выйти

Пример

```
sudo oval-db load -f /tmp/oval.xml -o astra -v 1.7_x86-64
```

При добавлении новой версии ранее добавленных файлов старые файлы перемещаются в подкаталог `backup`.

Для восстановления старой версии файлов из резервной копии в подкаталоге `backup` используется действие `restore`.

Синтаксис команды:

```
sudo oval-db restore [параметры]
```

Параметры команды приведены в таблице 45.

Таблица 45

Параметр	Описание
<code>-o, --os <имя></code>	Имя операционной системы, для которой восстанавливается oval-описание. Обязательный параметр
<code>-v, --version <идентификатор></code>	Идентификатор версии операционной системы (код релиза ОС), для которой восстанавливается oval-описание. Обязательный параметр
<code>-h, --help</code>	Вывести справку и выйти

Для удаления добавленных oval-описаний используется действие `remove`.

Синтаксис команды:

```
sudo oval-db remove [параметры]
```

Параметры команды приведены в таблице 46.

Т а б л и ц а 46

Параметр	Описание
-o, --os <имя>	Имя операционной системы, для которой удаляется oval-описание. Обязательный параметр
-v, --version <идентификатор>	Идентификатор версии операционной системы (код релиза ОС), для которой удаляется oval-описание. Обязательный параметр
-h, --help	Вывести справку и выйти

8.2.3.3. Обновление oval-описаний

Инструмент `oval-db` поддерживает автоматическое и ручное обновление oval-описаний для образов на основе ОС. Для обновления oval-описания образов на основе других операционных систем требуется вручную добавить новую версию файла oval-описания (см. 8.2.3.2).

Автоматическое обновление oval-описаний по умолчанию отключено. Для управления автоматическим обновлением используется действие `auto-update`.

Синтаксис команды:

```
sudo oval-db auto-update [параметры]
```

Параметры команды приведены в таблице 47.

Т а б л и ц а 47

Параметр	Описание
-e, --enable	Включить автоматическое обновление oval-описаний
-e=false	Отключить автоматическое обновление oval-описаний
-h, --help	Вывести справку и выйти

Для ручного обновления oval-описаний используется действие `update`.

Синтаксис команды:

```
sudo oval-db update [параметры]
```


Параметры команды приведены в таблице 48.

Таблица 48

Параметр	Описание
-v, --version <код_релиза_ОС>	Очередное обновление ОС, для которого будут обновлены oval-описания. Обязательный параметр. Чтобы обновить oval-описания для всех очередных обновлений ОС, следует указать значение all
-h, --help	Вывести справку и выйти

8.2.3.4. Вывод описаний уязвимостей

Инструмент oval-db позволяет выводить справочную информацию по конкретным уязвимостям с помощью действия vul-info.

Синтаксис команды:

```
sudo oval-db vul-info [параметры]
```

Параметры команды приведены в таблице 49.

Таблица 49

Параметр	Описание
-i <идентификатор>	Идентификатор уязвимости. Обязательный параметр
-o, --os <имя>	Имя операционной системы, к которой относится уязвимость. Обязательный параметр
-v, --version <идентификатор>	Идентификатор версии операционной системы (код релиза ОС), к которой относится уязвимость. Обязательный параметр
-x, --xml	Вывести информацию в XML-формате
-h, --help	Вывести справку и выйти

Пример

Вывести описание уязвимости в ОС очередного обновления 1.8:

```
sudo oval-db vul-info -i \
    oval:astra:def:1018494183034066672154209260230211 \
    -o astra -v 1.8_x86-64
```

8.2.3.5. Загрузка файла игнорируемых уязвимостей

Игнорируемые уязвимости — это уязвимости, обнаружение которых игнорируется при проверке заданных образов.

Настройки игнорируемых уязвимостей хранятся в файле `/usr/share/oval/conf/whitelist.json`. Если файл существует, то он автоматически загружается в систему сканирования уязвимостей при запуске службы `oval-dbd`. Описание конфигурационного файла игнорируемых уязвимостей приведено в 8.2.3.8.

После изменения или создания файла `/usr/share/oval/conf/whitelist.json` необходимо загрузить его в систему сканирования уязвимостей с помощью действия `wl-reload`.

Синтаксис команды:

```
sudo oval-db wl-reload
```

8.2.3.6. Конфигуратор системы сканирования уязвимостей

Конфигуратор позволяет настроить параметры сканирования и обнаружения уязвимостей для систем контейнеризации Docker и Podman.

Для запуска configurатора выполнить команду:

```
sudo oval-db config
```

Навигация в интерфейсе configurатора производится с помощью мыши или клавиатуры. Используемые клавиши и их действия приведены в таблице 50.

Т а б л и ц а 50

Клавиша	Описание
<←> или <→>	Переход между вкладками
<Enter>	Подтверждение действия
<Tab>	Переход к следующему параметру
<↑>	Переход к выбору шаблона настроек
<Q>	Выход без сохранения изменений

Интерфейс configurатора приведен на рис. 5.

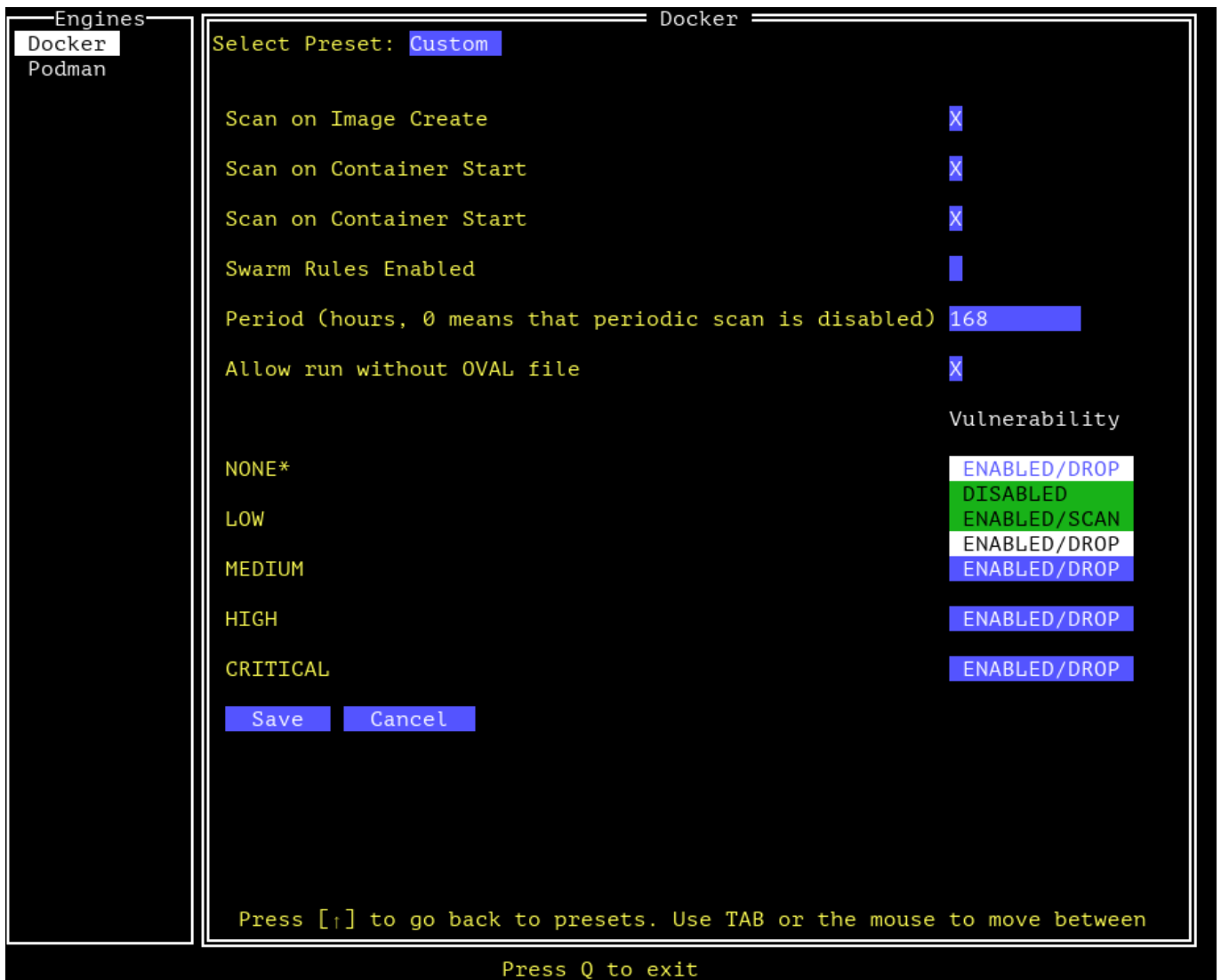


Рис. 5

В конфигураторе присутствуют следующие шаблоны настроек:

- 1) Level 1-6 — шаблоны, соответствующие шести классам защиты;
- 2) None — шаблон отключения всех функций сканирования и поиска уязвимостей;
- 3) Custom — пользовательский шаблон, выбираемый автоматически при изменении любых параметров.

Состав шаблонов настроек приведен в таблице 51.

Таблица 51

Имя шаблона	Сканирование на уязвимости при создании образа	Сканирование на уязвимости при запуске контейнера	Периодическое сканирование на уязвимости	Запуск контейнеров без oval-описаний	Сканирование контейнера на уязвимости / Запрет запуска при обнаружении уязвимостей				
					Уровень критичности				
					нет	низкий	средний	высокий	критический
Level 1	Да	Да	Да 7 дней	Нет	Да/ Да	Да/ Да	Да/ Да	Да/ Да	Да/ Да
Level 2	Да	Да	Да 7 дней	Да	Да/ Да	Да/ Да	Да/ Да	Да/ Да	Да/ Да
Level 3	Да	Да	Да 7 дней	Да	Да/ Нет	Да/ Нет	Да/ Да	Да/ Да	Да/ Да
Level 4	Да	Да	Да 7 дней	Да	Нет/ Нет	Да/ Нет	Да/ Нет	Да/ Да	Да/ Да
Level 5	Да	Да	Да 7 дней	Да	Нет/ Нет	Нет/ Нет	Да/ Нет	Да/ Да	Да/ Да
Level 6	Да	Да	Да 14 дней	Да	Нет/ Нет	Нет/ Нет	Нет/ Нет	Да/ Нет	Да/ Нет
None	Нет	Нет	Нет	Да	Нет/ Нет	Нет/ Нет	Нет/ Нет	Нет/ Нет	Нет/ Нет

8.2.3.7. Конфигурационные файлы системы сканирования уязвимостей

Настройки системы сканирования уязвимостей хранятся в формате JSON в следующих конфигурационных файлах:

- `/etc/podman.json` — параметры сканирования для системы контейнеризации Podman;
- `/etc/docker/daemon.json` — параметры сканирования для системы контейнеризации Docker;
- `/usr/share/oval/conf/podman.json` — параметры поиска уязвимостей при использовании Podman;
- `/usr/share/oval/conf/docker.json` — параметры поиска уязвимостей при использовании Docker;
- `/usr/share/oval/conf/daemon.json` — параметры службы oval-dbd;
- `/usr/share/oval/conf/whitelist.json` — параметры игнорируемых уязвимостей (описание файла приведено в 8.2.3.8).

При отсутствии какого-либо файла применяются настройки по умолчанию.

Параметры сканирования образов и контейнеров содержатся в конфигурационных файлах Docker и Podman — `/etc/docker/daemon.json` и `/etc/podman.json` соответственно. Данные файлы создаются автоматически после настройки параметров сканирования и обнаружения уязвимостей с помощью конфигуратора (см. 8.2.3.6). Параметры и их значения записываются в формате JSON. При отсутствии какого-либо файла используются значения по умолчанию.

Описание основных параметров приведено в таблице 52.

Таблица 52

Система контейнеризации	Параметр	Описание
Podman/Docker	<code>scan-on-image-create</code>	Включение или отключение сканирования образов при создании. Возможные значения: • <code>true</code> — сканирование образов при создании включено; • <code>false</code> — сканирование образов при создании выключено. Значение по умолчанию <code>true</code>
	<code>scan-on-container-start</code>	Включение или отключение сканирования при запуске контейнеров из образов. Возможные значения: • <code>true</code> — сканирование образов при запуске контейнеров включено; • <code>false</code> — сканирование образов при запуске контейнеров выключено. Значение по умолчанию <code>true</code>
	<code>periodic-scan-time-in-hours</code>	Период сканирования образов в часах. При значении 0 сканирование образов отключено. Значение по умолчанию 168
Docker	<code>daemon-start-scan-thread-count</code>	Максимальное число потоков выполнения для сканирования контейнеров при запуске службы. Значение по умолчанию 1
	<code>swarm-rules-enabled</code>	Включение режима мандатного управления доступом для механизма docker swarm. Возможные значения: • <code>true</code> — режим мандатного управления доступом включен; • <code>false</code> — режим мандатного управления доступом выключен. Значение по умолчанию <code>true</code>

Пример

Конфигурационный файл `/etc/docker/daemon.json` имеет следующий вид:

```
{
  "scan-on-image-create": true,
  "scan-on-container-start": true,
  "periodic-scan-time-in-hours": 168,
  "scan-on-image-create": true,
  "swarm-rules-enabled": false
}
```

Параметры поиска уязвимостей при использовании Docker и Podman содержатся в конфигурационных файлах `oval-db` — `/usr/share/oval/conf/docker.json` и `/usr/share/oval/conf/podman.json` соответственно. Данные файлы создаются автоматически при установке соответствующих систем контейнеризации. Для обеих систем контейнеризации используются одинаковые параметры. Параметры и их значения задаются в формате JSON.

Описание основных параметров приведено в таблице 53.

Таблица 53

Параметр	Описание
<code>levels</code>	Правила обработки уязвимостей каждого уровня критичности. Значения параметра соответствуют уровням критичности: • критический (<code>critical</code>); • высокий (<code>high</code>); • средний (<code>medium</code>); • низкий (<code>low</code>). Правила для каждого уровня задаются с помощью параметров, приведенных в таблице 54. Если параметры для какого-либо уровня критичности не заданы, то для этого уровня будут использоваться значения по умолчанию
<code>engine</code>	Используемая система контейнеризации. Возможные значения: • <code>docker</code>; • <code>podman</code>
<code>allow-run-wo-oval-file</code>	Разрешение запускать контейнеры без <code>oval</code>-описаний. Возможные значения: • <code>true</code> — разрешено; • <code>false</code> — запрещено. Значение по умолчанию <code>true</code>

Для каждого уровня критичности (см. описание параметра `levels` в таблице 53) задаются параметры, приведенные в таблице 54.

Таблица 54

Параметр	Описание
<code>enabled</code>	Обнаружение уязвимостей для данного уровня критичности. Если значение параметра не задано, то будет использоваться значение по умолчанию. Возможные значения: • <code>true</code> — обнаружение уязвимостей включено; • <code>false</code> — обнаружение уязвимостей отключено. Значение по умолчанию <code>true</code>
<code>drop</code>	Запрет запуска контейнеров с обнаруженными уязвимостями данного уровня критичности. Если значение параметра не задано, то будет использоваться значение по умолчанию. Возможные значения: • <code>true</code> — запуск контейнеров с уязвимостями запрещен; • <code>false</code> — запуск контейнеров с уязвимостями разрешен. Значение по умолчанию <code>true</code>

Пример

Конфигурационный файл `/usr/share/oval/conf/docker.json` имеет следующий вид:

```
{
  "engine": "docker",
  "allow-run-wo-oval-file": true,
  "levels": {
    "critical": {
      "enabled": true,
      "drop": true
    },
    "high": {
      "enabled": true,
      "drop": true
    },
    "medium": {
      "enabled": true,
      "drop": true
    },
    "low": {
      "enabled": true,
      "drop": true
    },
    "none": {
      "enabled": true,
      "drop": true
    }
  }
}
```

```

    }
}

```

Параметры службы `oval-dbd` содержатся в файле `/usr/share/oval/conf/daemon.json`. Данный файл создается автоматически при изменении конфигурации службы `oval-dbd`, например с помощью инструмента `oval-db` (см. 8.2.3.1) Параметры имеют вложенную структуру и сгруппированы по функции:

- параметры хранения результатов сканирования — содержатся в параметре `Database`;
- параметры автоматического обновления `oval`-описаний для образов операционных систем Astra Linux — содержатся в параметре `Update`.

Если файл отсутствует, то используются значения по умолчанию.

Описание основных параметров приведено в таблице 55.

Таблица 55

Параметр	Описание
Параметры хранения результатов сканирования (Database)	
Path	Путь к базе данных результатов сканирования. Значение по умолчанию: <code>/usr/share/oval/history/scan_results.db</code>
ExpireTime	Срок устаревания результатов сканирования (в часах). Значение по умолчанию 48
Параметры автоматического обновления oval-описаний (Update)	
Auto	Автоматическая проверка наличия обновлений oval-описаний. Возможные значения: • <code>true</code> — автоматическая проверка включена; • <code>false</code> — автоматическая проверка отключена. Значение по умолчанию <code>true</code>
ManifestUrl	URL сервера, с которого будет выполняться обновление. Значение по умолчанию: <code>https://dl.astralinux.ru/artifactory/al-oval/oval_meta.json</code>
TmpDir	Временный каталог для размещения загружаемых файлов. Значение по умолчанию: <code>/tmp/oval_temp/</code>
Interval	Интервал проверки обновлений oval-описаний (в часах). Значение по умолчанию 24
FoldersForVersionsUrl	URL каталога с файлами oval-описаний на сервере. Значение по умолчанию: <code>https://dl.astralinux.ru/astra/oval/</code>

Пример

Конфигурационный файл `/usr/share/oval/conf/daemon.json` имеет следующий вид:

```
{
  "Database": {
    "Path": "/usr/share/oval/history/scan_results.db",
    "ExpireTime": 48
  },
  "Update": {
    "Auto": true,
    "ManifestUrl": "https://dl.astralinux.ru/artifactory/al-oval/oval_meta.json",
    "TmpDir": "/tmp/oval_temp/",
    "Interval": 24,
    "FoldersForVersionsUrl": "https://dl.astralinux.ru/astra/oval/"
  }
}
```

8.2.3.8. Конфигурационный файл игнорируемых уязвимостей

Настройки уязвимостей, игнорируемых при проверке заданных образов, содержатся в файле `/usr/share/oval/conf/whitelist.json` формата JSON. Если файл существует, то он автоматически загружается в систему сканирования уязвимостей при запуске службы `oval-dbd`. Если файл отсутствует, то никакие уязвимости не игнорируются при сканировании.

Файл `/usr/share/oval/conf/whitelist.json` создается и редактируется вручную. После изменения или создания файла необходимо загрузить его в систему сканирования уязвимостей (см. 8.2.3.5).

Структура файла `/usr/share/oval/conf/whitelist.json`:

```
{
  "version": "1.0",
  "entries": [
    <Группа_правил_1>,
    <Группа_правил_2>,
    ...,
    <Группа_правил_n>
  ]
}
```

Описание параметров файла приведено в таблице 56.

Таблица 56

Параметр	Описание
version	Версия формата, описывающего игнорируемые уязвимости
entries	Массив из элементов <Группа_правил_n>, в которых содержатся группы правил игнорирования уязвимостей

Структура элемента <Группа_правил_n>, содержащего группу правил игнорирования уязвимостей:

```
{
  "description": "<описание_группы_правил>",
  "engine": "<средство_контейнеризации>",
  "engine_credentials": "<uid_и_метка_безопасности>",
  "items": [
    <Правило_1>,
    <Правило_2>,
    ...,
    <Правило_m>
  ]
}
```

Описание параметров элемента <Группа_правил_n> приведено в таблице 57.

Таблица 57

Параметр	Описание
description	Описание группы правил. Необязательный параметр
engine	Средство контейнеризации, на которое распространяется группа правил. Возможные значения: • docker; • podman; • all — любое средство контейнеризации
engine_credentials	UID-идентификатор и метка безопасности, с которыми запущен процесс средства контейнеризации и на которые распространяется группа правил. Допустимое значение all — любые UID-идентификаторы и метки безопасности
items	Массив из элементов <Правило_m>, в которых содержатся правила игнорирования уязвимостей

Структура элемента <Правило_m>, содержащего правило игнорирования уязвимостей:

```
{
  "image_id": "<идентификатор_образа>",
```

```

"cve_ids": [
  "<идентификатор_уязвимости_1>",
  "<идентификатор_уязвимости_2>",
  ...,
  "<идентификатор_уязвимости_s>"
]
}

```

Описание параметров элемента <Правило_m> приведено в таблице 58.

Таблица 58

Параметр	Описание
image_id	Идентификатор образа, для которого необходимо игнорировать обнаружение уязвимостей, указанных в параметре cve_ids. Идентификатор указывается в виде хеша по алгоритму SHA256. Префикс хеша sha256: допустимо не указываться. Например, следующие значения идентификатора равнозначны: <pre>sha256:1b32ca142c8fec0dafaa47a97342358d0c5de8ffc3b9bd7b32782ddaced5d25b1b32ca142c8fec0dafaa47a97342358d0c5de8ffc3b9bd7b32782ddaced5d25b</pre> Необязательный параметр. Если параметр image_id не указан, то правило распространяется на любой образ, а параметр cve_ids обязательно должен быть указан
cve_ids	Массив из идентификаторов уязвимостей, обнаружение которых необходимо игнорировать при проверке образа, указанного в параметре image_id. Идентификатор уязвимости может быть указан как: 1) CVE-идентификатор в базе данных NIST. Пример CVE-2023-39804 2) BDU-идентификатор в базе данных ФСТЭК. Пример BDU:2023-03872 3) oval:astra:def-идентификатор в базе данных oval-описаний. Пример oval:astra:def:1058505651501567746781164394075715 ВНИМАНИЕ! oval:astra:def-идентификаторы могут измениться при обновлении oval-описаний. Рекомендуется использовать CVE- и/или BDU-идентификаторы. Необязательный параметр. Если параметр cve_ids не указан, то правило распространяется на любые уязвимости, а параметр image_id обязательно должен быть указан

Примеры:

1. При проверке образа, указанного в параметре `image_id`, игнорировать любые найденные уязвимости.

Конфигурационный файл `/usr/share/oval/conf/whitelist.json`:

```
{
  "version": "1.0",
  "entries": [
    {
      "description": "Доверенный образ",
      "engine": "all",
      "engine_credentials": "all",
      "items": [
        {
          "image_id": "1b32ca142c8fec0dafaa47a97342358d0..."
        }
      ]
    }
  ]
}
```

2. При проверке любого образа будет проигнорировано обнаружение уязвимостей, указанных в параметре `cve_ids`.

Конфигурационный файл `/usr/share/oval/conf/whitelist.json`:

```
{
  "version": "1.0",
  "entries": [
    {
      "description": "Допустимые уязвимости perl",
      "engine": "all",
      "engine_credentials": "all",
      "items": [
        {
          "cve_ids": ["CVE-2023-31486", "CVE-2020-16156", "CVE-2023-31484"]
        }
      ]
    }
  ]
}
```

3. При проверке образа, указанного в параметре `image_id`, будет проигнорировано обнаружение уязвимостей, указанных в параметре `cve_ids`.

Конфигурационный файл /usr/share/oval/conf/whitelist.json:

```
{
  "version": "1.0",
  "entries": [
    {
      "description": "Допустимые уязвимости для конкретного образа",
      "engine": "all",
      "engine_credentials": "all",
      "items": [
        {
          "image_id": "sha256:1b32ca142c8fec0dafaa47a97342358d0...",
          "cve_ids": ["oval:astradef:998290386235520054830590729737795",
            "BDU:2026-05140", "CVE-2025-69421", "CVE-2025-69420",
            "CVE-2025-69419", "CVE-2025-68973", "CVE-2025-14831",
            "CVE-2025-68160", "CVE-2025-6395", "CVE-2025-9820",
            "CVE-2025-14819", "CVE-2025-32988", "CVE-2025-69418",
            "CVE-2026-0915", "CVE-2025-32990", "CVE-2025-15079",
            "CVE-2026-22796", "CVE-2026-22795", "CVE-2025-14524"]
        }
      ]
    }
  ]
}
```

8.2.4. Обновление oval-описаний в закрытом контуре

8.2.4.1. Настройка сервера обновления oval-описаний

При необходимости возможно создать локальный сервер для обновления oval-описаний уязвимостей ОС с доступом по протоколам HTTP/HTTPS или FTP. На сервере должен присутствовать каталог с oval-описаниями, все элементы которого должны быть доступны для загрузки.

Пример

Вариант структуры каталога с oval-описаниями на сервере.

```
oval/
| 1.7_x86-64/
|   |--- oval-definitions-alse-1.7.xml
|   |--- oval-definitions-alse-1.7.xml.md5
|   |--- oval-definitions-alse-1.7.xml.sha1
|   |--- oval-definitions-alse-1.7.xml.sha256
| 1.8_x86-64/
|   |---...
| 4.7_arm/
|   |---...
```

```
|--- oval_meta.json
|--- oval_meta.json.md5
|--- oval_meta.json.sha1
|--- oval_meta.json.sha256
...
```

Oval-описания хранятся в файлах формата XML и размещаются для удобства в отдельных каталогах для каждого очередного обновления и архитектуры ОС. Файлы форматов MD5, SHA1 и SHA256 размещаются в тех же каталогах и содержат контрольные суммы одноименных файлов. Файл `oval_meta.json` содержит ссылки на все доступные oval-описания и значения их контрольных сумм.

Пример

Файл `oval_meta.json`. Комментарии в файле начинаются с символа «#».

```
{
# Массив процессорных архитектур ОС
"platforms": [
  {
    # Процессорная архитектура ОС
    "platform": "x86-64",

    # Массив очередных обновлений ОС в рамках процессорной архитектуры
    "targets": [

      {
        # Очередное обновление ОС (обязательный элемент)
        "target": "1.8_x86-64",

        # URL файла oval-описания (обязательный элемент)
        "url": "http://server.astra.dom/oval/1.8_x86-64/oval-definitions-\\
          alse-1.8.xml",

        # Контрольная сумма файла oval-описания по MD5 (обязательный \
          элемент)
        "md5": "ccf88d2c64764db8fdd3ded3f5cfd54f",

        # Контрольные суммы файла oval-описания по SHA256 и ГОСТ
        "sha256sum": "58\
          f8075bbf40fb822de9a2cacc91067c8055ed8e05449f58997e3c50cf93371b\
          ",
        "gost": "\
          ad35297e12241c12574d6abc9edae6004bd7e9eb97a8daba2b93fdd2ed76bc27\
          ",

        # Дата добавления файла oval-описания на сервер
        "uploaded": "21-05-2025 14:54:10.513"
```

```

    },
    {
      "target": "1.7_x86-64",
      "url": "http://server.astra.dom/oval/1.7_x86-64/oval-definitions-\
        alse-1.7.xml",
      "md5": "b1d7fbd7de8d4b5554d18cf4168224a0",
      "sha256sum": "783709\
        c6a0ad9b8f610c05be2e86e45e69538b0c573f3e654279ce2379640e37",
      "gost": "\
        af9f17dfcffa95b5a3ac85f9da834dde07ef8434b7a6d9ac59de581eab990c45\
        ",
      "uploaded": "19-05-2025 14:38:41.976"
    }
  ]
}
{
  "platform": "arm",
  "targets": [
    {
      "target": "4.7_arm",
      "url": "https://dl.astralinux.ru/astra/oval/4.7_arm/oval-\
        definitions-alse-4.7.xml",
      "md5": "1534df4ecb76014aec8da1d090948e3b",
      "sha256sum": "\
        bed2619a835e6afc31171f29c1a0b659d7124604fa03e8cc5a9c3dadd3c474f5\
        ",
      "gost": "7\
        ee2703817bfbf13ff242b5bb55daccf554dfc7c30a0bc4cdc20854e70bd6423\
        ",
      "uploaded": "02-06-2025 16:28:37.169"
    }
  ]
}
]
}

```

Обновление файлов oval-описаний на сервере в закрытом контуре выполняется вручную:

1) получить актуальные версии oval-описаний на компьютере с установленным пакетом `oval-db` и доступом в Интернет:

а) для получения актуальных версий всех oval-описаний для ОС выполнить команду:

```
sudo oval-db update -v all
```

б) для получения актуальных версий oval-описаний для определенного очередного обновления ОС выполнить команду:

```
sudo oval-db update -v <кодовое_имя_ОС>
```

где кодовое имя ОС — это значение параметра `VERSION_ID` в файле `/etc/os-release`;

Пример

```
sudo oval-db update -v 1.8_x86-64
```

2) после завершения обновления перенести содержимое каталога `/usr/share/oval/db/astra/` в такой же каталог на сервере обновления и убедиться, что права доступа к файлам и подкаталогам такие же, как на исходном компьютере.

8.2.4.2. Настройка клиента для обновления oval-описаний

Настройки обновления oval-описаний на клиентском компьютере содержатся в файле `/usr/share/oval/conf/daemon.json` (см. 8.2.3.7). Данный файл по умолчанию отсутствует и создается при включении автоматического обновления oval-описаний (см. 8.2.3.3). Для указания сервера обновлений следует изменить значения следующих параметров:

- 1) `ManifestUrl` — указать URL или путь к файлу `oval_meta.json` на сервере обновлений;
- 2) `FoldersForVersionsUrl` — указать URL каталога с oval-описаниями на сервере обновлений.

Пример

Файл `/usr/share/oval/conf/daemon.json`.

```
{
  "Database": {
    "Path": "/usr/share/oval/history/scan_results.db",
    "ExpireTime": 48
  },
  "Update": {
    "Auto": true,
    "ManifestUrl": "http://server.astra.dom/oval/oval_meta.json",
    "TmpDir": "/tmp/oval_temp/",
    "Interval": 24,
    "FoldersForVersionsUrl": "http://server.astra.dom/oval/"
  }
}
```

8.3. Обеспечение корректности конфигурации контейнеров

Обеспечение корректности конфигурации контейнеров, в том числе ограничение прав прикладного программного обеспечения, выполняемого внутри контейнера, на использование

периферийных устройств, вычислительных ресурсов хостовой ОС, устройств хранения данных и съемных машинных носителей информации (блочных устройств); а также монтирование корневой файловой системы хостовой ОС в режиме «только для чтения» и ограничение доступа прикладного программного обеспечения, выполняемого внутри контейнера, к системным вызовам ядра хостовой ОС, осуществляется средствами Docker и Podman.

8.3.1. Ограничение прав на использование ресурсов хостовой машины для Docker

В ОС настраиваются ограничения прав прикладного программного обеспечения, выполняемого внутри контейнера Docker, на использование вычислительных ресурсов хостовой ОС: оперативной памяти, операций ввода-вывода за период времени.

Для установки ограничений на объем памяти последовательно выполнить команды:

1) установить ограничение:

```
sudo docker run -it --memory=<объем>m ubuntu bash
```

где <объем>m — объем памяти в МБ;

2) ограничить операции ввода-вывода:

```
sudo docker run -it --memory=<объем>m --device-write-bps \
/dev/sda:<объем>mb ubuntu bash
```

Для настройки монтирования корневой файловой системы хостовой ОС в режиме «только для чтения» необходимо запустить docker с параметром `readonly`, указанным через запятую «,» после точки монтирования внутри контейнера:

```
sudo docker run -d --name=nginxtest --mount \
source=nginx-vol,destination=/usr/share/nginx/html,readonly \
nginx:latest
```

Для программного обеспечения, выполняемого внутри контейнера, также ограничивается доступ к системным вызовам ядра хостовой ОС. Данное ограничение реализовано с помощью механизма ядра ОС `seccomp`.

Для указания запрещенных или разрешенных системных вызовов используется профиль `seccomp`, представляющий собой файл формата JSON с набором правил.

При запуске контейнера по умолчанию применяется профиль, встроенный в Docker. В этом профиле заблокирован определенный набор системных вызовов для предотвращения доступа к модулям ядра, а также к таким функциям системы, как монтирование дисков, изменение системного времени, перезагрузка системы и т.п. Полный список вызовов, блокируемых встроенным профилем, приведен в официальной документации Docker.

Для запуска контейнера с другим набором запрещенных или разрешенных вызовов возможно создать собственные профили в формате JSON. В профиле должны присутствовать следующие поля:

1) `defaultAction` — действие, применяемое по умолчанию к системным вызовам.

Возможные значения:

- `SCMP_ACT_ERRNO` — выполнение запрещено;
- `SCMP_ACT_ALLOW` — выполнение разрешено;

2) `architectures` — список архитектур, в которых применяется профиль;

3) `syscalls` — список системных вызовов, на которые не будет распространяться действие по умолчанию;

4) `action` — действие, применяемое к вызовам из списка.

Примеры:

1. Профиль, запрещающий все системные вызовы:

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "architectures": [
    "SCMP_ARCH_X86_64",
    "SCMP_ARCH_X86",
    "SCMP_ARCH_X32"
  ],
  "syscalls": [
  ]
}
```

2. Профиль, разрешающий все вызовы, кроме `read`, `write` и `uname`.

```
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "architectures": [
    "SCMP_ARCH_X86_64",
    "SCMP_ARCH_X86",
    "SCMP_ARCH_X32"
  ],
  "syscalls": [
    {
      "names": [
        "read",
        "write",
        "uname"
      ],
    },
  ],
}
```

```

        "action": "SCMP_ACT_ERRNO"
    }
}
]
}

```

Для запуска контейнера Docker с использованием созданного профиля seccomp необходимо использовать параметр `--security-opt` и в качестве значения параметра seccomp указать путь к файлу профиля:

```
sudo docker run --security-opt seccomp=/etc/default/profile.json ubuntu bash
```

Для запуска контейнера с отключенными ограничениями seccomp необходимо использовать параметр `--security-opt` и в качестве значения параметра seccomp указать `unconfined`:

```
sudo docker run --rm -it --security-opt seccomp=unconfined ubuntu bash
```

8.3.2. Ограничение прав на использование ресурсов хостовой машины для Podman

Ограничение прав прикладного программного обеспечения, выполняемого внутри контейнера Podman, на использование вычислительных ресурсов и файловой системы хостовой ОС возможно только при отключенном МКЦ. Для корректной работы Podman необходимо перед отключением МКЦ установить пакет `dbus-user-session`. Управление режимом МКЦ описано в 4.9.

Управление ограничениями для Podman на использование ресурсов хостовой ОС выполняется в соответствии с 8.3.1.

8.4. Контроль целостности образов и параметров настройки средства контейнеризации

Контроль целостности образов контейнеров и исполняемых файлов контейнеров, параметров настройки средства контейнеризации реализуется сертифицированными средствами регламентного контроля целостности AFICK из состава ОС (см. 10.4).

Для контроля целостности объектов контейнеризации необходимо добавить в конфигурационный файл `/etc/afick.conf` в секции `file section` соответствующие строки:

- 1) для контроля целостности образов контейнеров Docker и исполняемых файлов контейнеров Docker необходимо получить образ контейнера и поставить на контроль целостности каталог с образами и каталог с кэшем, добавив строки:

```
#####
```

```
# file section
#####

/var/lib/docker/image ETC
/var/lib/docker/overlay2 ETC
```

2) для контроля целостности параметров настройки Docker (файл /etc/docker/daemon.json) необходимо добавить строку:

```
#####
# file section
#####

/etc/docker/ ETC
```

3) для контроля целостности образов контейнеров Podman и исполняемых файлов контейнеров Podman добавить строки:

```
#####
# file section
#####

/var/lib/containers/storage/overlay-containers/ ETC
/var/lib/containers/storage/overlay-images/ ETC
/home/<user>/local/share/containers/storage/overlay-containers/ ETC
/home/<user>/local/share/containers/storage/overlay-images/ ETC
```

4) для контроля целостности параметров настройки средства контейнеризации Podman добавить строки:

```
#####
# file section
#####
...
/etc/podman.conf
/etc/containers/ ETC
...
```

Системный планировщик заданий cron позволяет выполнять проверку целостности объектов контроля в процессе загрузки ОС.

Локальное реагирование на события в автозапуске для сессии пользователя, регистрация событий безопасности и оповещение администратора о событиях безопасности осуществляется подсистемой регистрации событий (см. 6.5).

Целостность образов контейнеров и параметров настройки средства контейнеризации также контролируется путем применения цифровой подписи — создания замкнутой программной среды в соответствии с 16.1. Для выполнения контроля необходимо от имени учетной записи

администратора:

1) настроить замкнутую программную среду (см. 16.1);

2) получить идентификатор образа контейнера:

```
sudo docker image inspect nginx | grep Id
```

3) подписать файл образа:

```
sudo bsign --sign --xattr \  
    /var/lib/docker/image/overlay2/imagedb/content/sha256/<id>
```

где <id> — идентификатор образа контейнера;

4) добавить имя контролируемого файла в /etc/digsig/xattr_control:

```
sudo echo "var/lib/docker/image/overlay2/imagedb/content/sha256/<id>" \  
    | sudo tee -a /etc/digsig/xattr_control
```

где <id> — идентификатор образа контейнера;

5) выполнить команду:

```
sudo update-initramfs -u -k all
```

6) выполнить перезагрузку.

При нарушении целостности запуск образа контейнера блокируется.

8.5. Регистрация событий безопасности, связанных с контейнерами

Регистрация событий безопасности, связанных с образами и контейнерами Podman и Docker, осуществляется подсистемой регистрации событий (см. 6.5).

Событиям безопасности, связанным с образами и контейнерами Docker, присваиваются метки `dockerd_audit`. Событиям безопасности, связанным с образами и контейнерами Podman, присваиваются метки `podman_audit`. Записи в журнале имеют следующий формат:

```
podman.audit | user ; uid | событие | результат | дополнительная информация
```

Регистрация событий безопасности, связанных с образами и контейнерами Docker, также осуществляется штатными средствами Docker в системные журналы.

При регистрации событий, связанных с образами и контейнерами, указывается UID и имя пользователя ОС, от имени которого выполнено данное действие. Если действие было выполнено удаленно, то вместо UID и имени пользователя указывается IP-адрес компьютера, с которого выполнено действие. Если инициатор действия в той же ОС, что и служба

`dockerd`, но не удалось определить UID пользователя, то вместо UID и имени пользователя указывается символ «@». Также для события указывается результат — успешно или неуспешно. В случае неуспешного события указывается причина ошибки.

События аутентификации пользователя, в т.ч. неуспешные попытки аутентификации, регистрируются:

- 1) в журнале подсистемы регистрации событий (см. 6.5);
- 2) в журнале `/var/log/auth.log`.

Регистрация событий безопасности, связанных с выявлением уязвимостей в образе и в контейнере, осуществляется в журнал подсистемы регистрации событий (см. 6.5), а также в системные журналы `/var/log/daemon.log` и `/var/log/syslog.log`. Регистрируются следующие события:

- 1) запуск сканирования (start) с указанием пути и ID сканируемого контейнера/образа;
- 2) результат сканирования:
 - а) если уязвимость не обнаружена, то сообщение об окончании сканирования (complete) с указанием пути и ID сканируемого контейнера/образа;
 - б) если уязвимость обнаружена, то сообщение о неуспешном сканировании (fail) с указанием версии ПО, операционной системы, обнаруженной уязвимости, адреса отчета сканирования. Если при этом запуск контейнера был заблокирован (класс защиты 1-5), то также регистрируется сообщение о блокировке запуска контейнера.

Регистрация действий с образами и контейнерами (события создания, модификации и удаления образов контейнеров; получения доступа к образам контейнеров; запуска и остановки контейнеров с указанием причины остановки; модификации запускаемых контейнеров) осуществляется в журнале подсистемы регистрации событий (см. 6.5) и в журнале `/var/log/syslog.log`.

Настройка событий, которые подлежат регистрации, реагирование на них системы и оповещение администратора безопасности средства контейнеризации о событиях безопасности осуществляется с использованием утилит, приведенных в 6.5.1.

8.6. Идентификация и аутентификация пользователей

Первичная идентификация пользователей средства контейнеризации осуществляется администратором безопасности средства контейнеризации. Идентификация и аутентификация пользователей в средстве контейнеризации осуществляется с учетом требований разделов ГОСТ Р 58833-2020. При входе в систему осуществляется идентификация и проверка подлинности субъектов доступа с помощью процедуры аутентификации. Описание процедуры

приведено в разделе 2.

Хранение аутентификационной информации пользователей осуществляется с использованием хеш-функции по ГОСТ Р 34.11-94 и по ГОСТ Р 34.11-2012, что обеспечивает ее защиту.

Установка пароля пользователя осуществляется администратором с помощью модуля «Пользователи» в разделе «Пользователи и группы» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_users
```

Пароль пользователя должен содержать не менее 8 символов при алфавите пароля не менее 70 символов. Максимальное количество неудачных попыток аутентификации (ввода неправильного пароля) до блокировки — 4.

Запрет запуска средством контейнеризации в хостовой операционной системе процессов, обладающих привилегиями администратора информационной (автоматизированной) системы и администратора безопасности информационной (автоматизированной) системы, осуществляется с помощью утилиты `unshare`, позволяющей реализовать:

- 1) запрет запуска процессов от имени суперпользователя `root`;
- 2) запрет просмотра процессов хостовой машины;
- 3) невозможность какого-либо влияния на процессы хостовой машины и других пространств;
- 4) невлияние монтирования и размонтирования ФС на состав дерева каталогов хостовой машины.

8.7. Работа с Docker в непривилегированном режиме с ненулевыми метками безопасности

8.7.1. Принцип функционирования

Контейнеры Docker в непривилегированном (`rootless`) режиме (описание режима приведено в РУСБ.10015-01 95 01-1) могут быть запущены от имени любого непривилегированного пользователя с ненулевой меткой безопасности контейнера. Для корректной работы программного интерфейса (API) PARSEC при запуске контейнера необходимо примонтировать каталог `/parsecfs/` хостовой ОС в одноименный каталог внутри контейнера, например с помощью команды:

```
docker run -it -v /parsecfs:/parsecfs astra
```

Метка безопасности контейнера всегда задается четырьмя десятичными неотрицательными числами, разделенными двоеточием:

0:63:0:0

где первое число — иерархический уровень конфиденциальности;
второе число — неиерархическая категория целостности;
третье число — неиерархические категории конфиденциальности;
четвертое число — зарезервировано для задания мандатных атрибутов файловых объектов и в работе с контейнерами не применяется (следует всегда использовать значение 0).

Примечание. В команде допускается указывать только первые два или три числа метки безопасности.

Запуск контейнера при работе в расширенном режиме МКЦ (strict mode) невозможен, так как в этом режиме не работает Docker.

При запуске контейнера в непривилегированном режиме с ненулевой меткой безопасности применяются следующие ограничения:

- 1) контейнер может быть запущен с меткой безопасности из диапазона, назначенного пользователю;
- 2) в метке безопасности контейнера ненулевой может быть только классификационная метка или только метка целостности (при этом отрицательная метка целостности также является ненулевой меткой).

В запущенных контейнерах принимаются следующие ограничения:

- 1) запуск процессов внутри контейнера возможен только с одинаковой для всех процессов меткой безопасности, равной метке безопасности контейнера (задается при запуске контейнера);
- 2) для системных файлов внутри контейнера (исполняемых, конфигурационных и т.д.) всегда используется ненулевая категория целостности (например, 0:2:0:0 или 0:63:0:0) и, соответственно, всегда используется нулевая классификационная метка;
- 3) процессы (субъекты), работающие внутри контейнера в непривилегированном режиме, не могут изменять метки безопасности файловых объектов в файловой системе своего контейнера;
- 4) если внутри контейнера необходима работа с файлами с ненулевой классификационной меткой, то данные файлы должны группироваться в специально созданных

каталогах:

```
/home/.pdp/<имя_пользователя>/l<уровень_конфиденциальности>  
i<категория_целостности>  
c<категории_конфиденциальности>t0x0
```

Пример

```
/home/.pdp/astra/l0i2c0x0t0x0
```

При этом метки безопасности таких каталогов могут сочетать ненулевые классификационные метки и ненулевые метки целостности. Классификационные метки данных каталогов должны быть не выше классификационной метки контейнера.

Для хостовой ОС контейнер, запущенный в непривилегированном режиме, является группой процессов:

- 1) имеющих метку безопасности субъекта, запустившего контейнер;
- 2) не имеющих прав администратора;
- 3) работающих с набором файловых объектов, расположенных в файловой системе контейнера и имеющих собственные метки безопасности.

При этом процессы внутри контейнера:

- 1) запущены от имени администратора;
- 2) имеют неограниченный доступ к объектам файловой системы контейнера (за исключением возможности изменять метки безопасности).

Файловые объекты в файловой системе контейнера (rootFS) создаются с меткой безопасности, в которой:

- 1) классификационная метка равна классификационной метке субъекта, запустившего контейнер;
- 2) метка целостности всегда нулевая.

Для пользователя, от имени которого запускается контейнер, должны выполняться условия:

- 1) запущена непривилегированная служба Docker с меткой безопасности, равной метке безопасности, указанной при копировании образа в репозиторий пользователя (см. 8.7.2);
- 2) метка безопасности сессии пользователя равна метке безопасности, указанной при запуске непривилегированной службы Docker.

При запуске непривилегированной службы Docker для пользователя с указанием метки безопасности:

- 1) пользовательский репозиторий унаследует метку безопасности, с которой была запущена непривилегированная служба Docker;
- 2) файловая структура пользовательского репозитория размещается в каталоге:

```
/var/lib/rootlessdocker/<имя_пользователя>/l<уровень_конфиденциальности>
    i<категория_целостности>
    c<категории_конфиденциальности>t0x0
    /.local/share/docker
```

При запуске контейнера пользователем из своей сессии:

- 1) контейнер может быть запущен из репозитория образов пользователя, у которого метка безопасности соответствует метке безопасности сессии;
- 2) запущенный контейнер унаследует метку безопасности репозитория.

К процессам (субъектам) внутри контейнера, запущенного в непривилегированном режиме с ненулевой меткой безопасности, будут применяться общие правила мандатного управления доступом и мандатного контроля целостности хостовой ОС. Например, процессы контейнера в непривилегированном режиме, имеющего метку безопасности 1:0:0:0, смогут изменять файлы с меткой безопасности 1:0:0:0, читать содержимое файлов с меткой безопасности 1:0:0:0 или 0:0:0:0).

Описание работы с контейнерами Docker в непривилегированном режиме с ненулевыми метками безопасности также приведено на справочных страницах `man rootless-helper-astra` и `man rootlessenv`.

8.7.2. Управление запуском контейнера с ненулевой меткой безопасности

Для работы с контейнерами в непривилегированном режиме от имени пользователя с ненулевой меткой безопасности следует запустить для этого пользователя непривилегированную службу Docker с указанием соответствующей метки безопасности.

Запуск службы для работы с ненулевой классификационной меткой выполняется командой:

```
sudo systemctl start rootless-docker@$(systemd-escape \
    <имя_пользователя>@<метка_безопасности>)
```

Запуск службы для работы с различными классификационными метками выполняется командой:

```
sudo systemctl start rootless-docker@$(systemd-escape \
    <имя_пользователя>@0:0:0:0@privsock)
```

при этом использование флага `privsock` запускает непривилегированную службу Docker с привилегией `PARSEC_CAP_PRIV_SOCKET`, позволяющей выполнять команды для сетевых подключений (например, `docker pull`) игнорируя мандатные ограничения (см. 4.7).

Для настройки автоматического запуска службы после перезагрузки можно выполнить команду:

```
sudo systemctl enable rootless-docker@$(systemd-escape \  
  <имя_пользователя>@<метка_безопасности>@privsock)
```

Пример

Настройка работы в непривилегированном режиме для пользователя `user1` выполняется в следующей последовательности от имени администратора:

- 1) создать пользователя `user1`:

```
sudo useradd user1
```

- 2) назначить пользователю `user1` уровень конфиденциальности 1:

```
sudo pdpl-user -l 1:1 user1
```

- 3) установить Docker согласно РУСБ.10015-01 95 01-1;

- 4) установить пакет `rootless-helper-astra` для использования непривилегированного режима:

```
sudo apt install rootless-helper-astra -y
```

- 5) запустить службы `rootless Docker` для пользователя `user1`:

```
sudo systemctl start rootless-docker@$(systemd-escape \  
  user1@1:0:0:0)
```

- 6) при необходимости настроить автозапуск служб `rootless Docker` для пользователя `user1`:

```
sudo systemctl enable rootless-docker@$(systemd-escape \  
  user1@1:0:0:0)
```

После запуска для пользователя непривилегированной службы Docker возможно выполнение команд Docker от имени данного пользователя (например, копирование образов в пользовательские репозитории образов или запуск контейнеров). Подробнее о копировании образа в репозиторий пользователя приведено в 8.7.3.

Перечень и описание доступных пользователю команд Docker в непривилегированном режиме, включая команды запуска контейнеров, приведены в документе РУСБ.10015-01 93 01.

8.7.3. Копирование образа в репозиторий пользователя

Все операции с программным обеспечением Docker в привилегированном режиме выполняются от имени администратора. При первом создании образа администратором автоматически формируется системный (`root`) репозиторий Docker. Все образы, созданные или загруженные администратором, сохраняются в системный репозиторий, доступный только администратору.

Чтобы непривилегированный пользователь с ненулевой меткой безопасности мог запустить контейнер из образа, необходимо скопировать образ из системного репозитория в репозиторий пользователя (см. 8.7.3.1). Чтобы другой пользователь с ненулевой меткой безопасности мог запустить контейнер из этого же образа, его необходимо скопировать в репозиторий другого пользователя.

Копирование образа из репозитория одного пользователя в репозиторий другого пользователя может быть выполнено администратором от имени пользователя (см. 8.7.3.2) или непривилегированным пользователем (см. 8.7.3.3).

Чтобы скопировать образ в репозиторий пользователя, необходимо предварительно от имени администратора запустить непривилегированную службу Docker для пользователя, который будет использовать образы и контейнеры Docker (см. 8.7.2).

8.7.3.1. Копирование образа из системного репозитория

Для копирования образа из системного репозитория в репозиторий пользователя необходимо от имени администратора:

- 1) экспортировать образ из системного репозитория в архив:

```
sudo docker save -o /var/<имя_архива>.tar <имя_образа>
```

Экспортировать рекомендуется в каталог `/var/`, доступный для чтения всем пользователям;

- 2) разрешить чтение экспортированного образа:

- а) всем пользователям:

```
sudo chmod o+r /var/<имя_архива>.tar
```

- б) только указанным пользователям:

```
sudo setfacl -m u:<имя_пользователя>:r /var/<имя_архива>.tar
```

- 3) запустить непривилегированную службу Docker для пользователя с указанием метки безопасности (см. 8.7.2).

Импорт образа из системного репозитория в репозиторий пользователя можно выполнить от имени непривилегированного пользователя с ненулевой меткой безопасности или администратором от имени пользователя.

Непривилегированному пользователю для импорта образа из архива в свой репозиторий выполнить команду:

```
rootlessenv docker load -i /var/<имя_архива>.tar
```

Администратору для импорта образа из архива в репозиторий непривилегированного пользователя выполнить команду:

```
sudo pdp-exec -u <имя_пользователя> -l <метка_безопасности> \
-- rootlessenv docker load -i /var/<имя_архива>.tar
```

Пример

Копирование образа из системного репозитория в репозиторий пользователя на примере пользователя `user1` с меткой безопасности `1:0:0:0`. Для пользователя от имени администратора запущена непривилегированная служба Docker с указанием метки безопасности (см. 8.7.2).

Для копирования образа из системного репозитория в репозиторий пользователя выполнить от имени администратора:

1) экспорт образа `wiki/astralinux:se` в архив `for-user1.tar`:

```
sudo docker save -o /var/for-user1.tar wiki/astralinux:se
```

2) разрешить чтение образа `wiki/astralinux:se` пользователю `user1`:

```
sudo setfacl -m u:user1:r /var/for-user1.tar
```

Для импорта образа из архива в репозиторий пользователя выполнить команду от имени пользователя `user1`:

```
rootlessenv docker load -i /var/for-user1.tar
```

Для проверки, что образ присутствует в репозитории пользователя, выполнить команду от имени пользователя `user1`:

```
rootlessenv docker images
```

8.7.3.2. Копирование образа из пользовательского репозитория привилегированным пользователем

Для копирования образа из репозитория одного пользователя в репозиторий другого пользователя от имени администратора:

1) запустить для обоих пользователей непривилегированную службу Docker с указанием метки безопасности (см. 8.7.2). При этом метка безопасности сессии одного пользователя должна быть меньше или равна метке безопасности сессии другого пользователя;

2) экспортировать образ из репозитория одного пользователя в архив. В команде указывается имя пользователя и метка безопасности сессии пользователя, из чьего репозитория выполняется экспорт. Выгрузить образ в архив:

```
sudo pdp-exec -u <имя_пользователя> -l <метка_безопасности> \  
-- rootlessenv docker save -o /var/<имя_архива>.tar <имя_образа>
```

3) импортировать образ из архива в репозиторий другого пользователя. В команде указывается имя пользователя и метка безопасности сессии пользователя, в чей репозиторий выполняется импорт. Для импорта образа выполнить команду:

```
sudo pdp-exec -u <имя_пользователя> -l <метка_безопасности> \  
-- rootlessenv docker load -i /var/<имя_архива>.tar
```

4) убедиться, что образ появился в репозитории другого пользователя. В команде указывается имя пользователя и метка безопасности сессии пользователя, в чей репозиторий выполнялся импорт. Для проверки выполнить команду:

```
sudo pdp-exec -u <имя_пользователя> -l <метка_безопасности> \  
-- rootlessenv docker images
```

В выводе команды должен отобразиться импортированный образ.

Пример

Копирование образа из репозитория одного пользователя в репозиторий другого пользователя на примере пользователей: `user1` с меткой безопасности `1:0:0:0` и `user2` с меткой безопасности `1:0:0:0`. Для обоих пользователей от имени администратора запущена непривилегированная служба Docker с указанием метки безопасности (см. 8.7.2).

Для копирования образа выполнить:

1) экспорт образа `wiki/astralinux:se` из репозитория пользователя `user1` в архив. Для этого администратором от имени пользователя `user1` выполнить:

```
sudo pdp-exec -u user1 -l 1:0:0:0 \  
-- rootlessenv docker save -o /var/<имя_архива>.tar wiki/astralinux:se
```

```
-- rootlessenv docker save -o \
/var/for-user2.tar wiki/astralinux:se
```

2) от имени администратора разрешить чтение образа wiki/astralinux:se пользователю user2:

```
sudo setfacl -m u:user2:r /var/for-user2.tar
```

3) импорт образа из архива /var/for-user2.tar в репозиторий пользователя user2. Для этого администратором от имени пользователя user2 выполнить:

```
sudo pdp-exec -u user2 -l 1:0:0:0 \
-- rootlessenv docker load -i /var/for-user2.tar
```

4) убедиться, что образ wiki/astralinux:se присутствует в списке образов в репозитории пользователя user2. Для этого администратором от имени пользователя user2 выполнить:

```
sudo pdp-exec -u user2 -l 1:0:0:0 -- rootlessenv docker images
```

Вывод в терминале будет иметь следующий вид:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
wiki/astralinux	se	d75d16f99f5e	2 weeks ago	497

8.7.3.3. Копирование образа из пользовательского репозитория непривилегированным пользователем

Копирование образа непривилегированным пользователем из репозитория одного пользователя в репозиторий другого пользователя возможно при выполнении условий:

- 1) от имени администратора для обоих пользователей запущена непривилегированная служба Docker с указанием метки безопасности (см. 8.7.2);
- 2) метка безопасности сессии пользователя, из чьего репозитория копируется образ, не должна превышать или быть равна метке безопасности сессии пользователя, в чей репозиторий копируется образ.

Для копирования образа непривилегированным пользователем из репозитория одного пользователя в репозиторий другого пользователя необходимо:

- 1) непривилегированному пользователю-владельцу репозитория экспортировать образ из своего репозитория в архив:

```
rootlessenv docker save -o /tmp/<имя_архива>.tar <имя_образа>
```

2) другому непривилегированному пользователю для копирования образа в свой репозиторий необходимо:

а) импортировать образ из архива:

```
rootlessenv docker load -i /tmp/<имя_архива>
```

б) убедиться, что образ появился в его репозитории:

```
rootlessenv docker images
```

В выводе команды должен отобразиться импортированный образ.

Пример

Копирование образа из репозитория одного пользователя в репозиторий другого пользователя на примере пользователей: `user1` с меткой безопасности `1:0:0:0` и `user2` с меткой безопасности `1:0:0:0`. Для обоих пользователей от имени администратора запущена непривилегированная служба Docker с указанием метки безопасности (см. 8.7.2).

Для копирования образа выполнить:

1) экспорт образа `wiki/astralinux:se` из репозитория пользователя `user1` в архив. Для этого от имени пользователя `user1` выполнить:

```
rootlessenv docker save -o /tmp/for-user2.tar wiki/astralinux:se
```

2) разрешить чтение образа `wiki/astralinux:se` пользователю `user2`. Для этого от имени пользователя `user1` выполнить:

```
setfacl -m u:user2:r /tmp/for-user2.tar
```

3) импорт образа из архива `/tmp/for-user2.tar` в репозиторий пользователя `user2`. Для этого от имени пользователя `user2` выполнить:

```
rootlessenv docker load -i /tmp/for-user2.tar
```

4) убедиться, что образ `wiki/astralinux:se` присутствует в списке образов в репозитории пользователя `user2`. Для этого от имени пользователя `user2` выполнить:

```
rootlessenv docker images
```

Вывод в терминале будет иметь следующий вид:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
wiki/astralinux	se	d75d16f99f5e	2 weeks ago	497

8.7.4. Выполнение команд и запуск контейнеров в непривилегированном режиме от имени пользователя

Для запуска контейнера непривилегированным пользователем с ненулевой меткой безопасности образ должен быть скопирован в репозиторий пользователя (см. 8.7.3).

Администратор может выполнять команды Docker с ненулевой меткой безопасности от имени другого пользователя с помощью инструмента `pdp-exec`. При выполнении команды указывается метка безопасности, с которой будет запущен контейнер.

Для запуска контейнера из образа непривилегированным пользователем с ненулевой меткой безопасности:

- 1) запустить от имени администратора непривилегированную службу Docker с меткой безопасности, равной метке безопасности, указанной при копировании образа в репозиторий пользователя (см. 8.7.2);
- 2) войти в сессию непривилегированного пользователя с меткой безопасности, равной метке безопасности, указанной при запуске непривилегированной службы Docker.

Для запуска администратором оболочки командной строки контейнера, из которой можно выполнять команды Docker от имени указанного пользователя, выполнить команду:

```
sudo pdp-exec -u <имя_пользователя> -l <метка_безопасности> \  
-- rootlessenv
```

Для создания контейнера со случайным именем из образа выполнить команду от имени непривилегированного пользователя:

```
rootlessenv docker run <имя_образа>
```

Для вывода списка контейнеров, созданных в сессии текущего пользователя, выполнить команду от имени непривилегированного пользователя:

```
rootlessenv docker container ls -a
```

Для вывода списка запущенных контейнеров, созданных в сессии текущего пользователя, выполнить команду от имени непривилегированного пользователя:

```
rootlessenv docker container list
```

Пример

Запуск контейнера из образа на примере пользователя `user1` с меткой безопасности `1:0:0:0`. Непривилегированная служба Docker запущена администратором для пользователя с указанием метки безопасности (см. 8.7.2). В репозиторий пользователя скопирован образ `wiki/astralinux:se`.

Для запуска контейнера из образа:

- 1) запустить оболочку командной строки контейнера. Для этого администратором от имени пользователя `user1` выполнить:

```
sudo pdp-exec -u user1 -l 1:0:0:0 -- rootlessenv
```

- 2) запустить контейнер со случайным именем из образа `wiki/astralinux:se` в интерактивном режиме с автоматическим удалением после завершения работы. Для этого администратором от имени пользователя `user1` выполнить:

```
rootlessenv docker run --rm -ti wiki/astralinux:se
```

Пользователь (администратор) может со стороны хостовой ОС присваивать метки безопасности файловым объектам в файловой системе контейнера (например, `1:0:0:0` для конфиденциальных пользовательских файлов и `0:2:0:0` для системных файлов контейнера с высокой категорией целостности).

Местонахождение файловой системы контейнера на хостовой ОС можно узнать командой:

```
docker inspect <имя_контейнера> | egrep "(Lower|Upper)Dir"
```

При работе администратора с контейнером пользователя данная команда должна выполняться от имени пользователя с указанием нужной метки безопасности:

```
sudo pdp-exec -u <имя_пользователя> -l <метка_безопасности> \  
-- rootlessenv docker inspect <имя_контейнера> | egrep "(Lower|Upper)Dir"
```

8.8. Работа с Podman в непривилегированном режиме с ненулевыми метками безопасности

Администратор может выполнять команды Podman, то есть запускать процессы Podman, с ненулевой меткой безопасности от имени другого пользователя. Для этого предварительно необходимо создать пользовательскую среду запуска процессов Podman с заданной меткой безопасности (см. 8.8.1).

Процесс Podman может быть запущен с меткой безопасности из диапазона, назначенного пользователю.

Порядок запуска администратором процессов Podman с ненулевой меткой безопасности описан в 8.8.2.

Пользовательскую среду запуска процессов Podman, созданную администратором, также может использовать пользователь в своей сессии (см. 8.8.3).

8.8.1. Создание пользовательской среды выполнения команд Podman

Для создания пользовательской среды запуска процессов Podman с ненулевой меткой безопасности выполнить команду:

```
sudo mac-podman-init --user <имя_пользователя> --label <метка_безопасности> \
    [параметры]
```

Примечание. Для параметров `--user` и `--label` допускается использовать короткий формат записи: `-u` и `-l` соответственно.

Метка безопасности задается четырьмя десятичными неотрицательными числами, разделенными двоеточием:

0:63:0:0

где первое число — иерархический уровень конфиденциальности;
второе число — неиерархическая категория целостности;
третье число — неиерархические категории конфиденциальности;
четвертое число — зарезервировано для задания мандатных атрибутов файловых объектов и в работе с контейнерами не применяется (следует всегда использовать значение 0).

Примечание. В команде допускается указывать только первые два или три числа метки.

Необязательные параметры инструмента `mac-podman-init` описаны в таблице 59.

Таблица 59

Параметр	Описание
-d <путь>, --home <путь>	Инфраструктурный каталог создать по указанному пути. Инфраструктурный каталог используется для хранения всей инфраструктуры контейнеров. Если указанный каталог уже существует, то инструмент <code>mac-podman-init</code> сравнит его метку безопасности с указанной в команде. При их несовпадении инструмент <code>mac-podman-init</code> попытается назначить каталогу метку безопасности, указанную в команде. Если каталог отсутствует, то инструмент <code>mac-podman-init</code> создаст его по указанному пути и попытается назначить ему метку безопасности, указанную в команде. ВНИМАНИЕ! Классификационная метка каталога не может превышать значение классификационной метки каталога, в котором он расположен. Если параметр не указан, то будет создан каталог <code>/home/.rdr/<имя_пользователя>/z<метка_безопасности></code>. Примечания: 1. <имя_пользователя> и <метка_безопасности> соответствуют значениям, указанным в команде. 2. Метка безопасности обозначается четырьмя числами, разделенными подчеркиванием
-t <путь>, --tmp <путь>	Каталог временных файлов создать по указанному пути. Каталог временных файлов используется для хранения данных, которые нужны только во время работы контейнера или в процессе его подготовки. Например, при загрузке образа. Если указанный каталог уже существует, то инструмент <code>mac-podman-init</code> сравнит его метку безопасности с указанной в команде. При их несовпадении инструмент <code>mac-podman-init</code> попытается назначить каталогу метку безопасности, указанную в команде. Если каталог отсутствует, то инструмент <code>mac-podman-init</code> создаст его по указанному пути и попытается назначить ему метку безопасности, указанную в команде. ВНИМАНИЕ! Классификационная метка каталога не может превышать значение классификационной метки каталога, в котором он расположен. Если параметр не указан, то будет создан каталог <code>/run/user/<идентификатор>z<метка_безопасности>/tmp/</code>, где <идентификатор> — идентификатор пользователя (UID), имя которого указано в команде; <метка_безопасности> — метка безопасности, указанная в команде. Метка безопасности обозначается четырьмя числами, разделенными подчеркиванием
-h, --help	Вывести справку и выйти

Пример

Создать пользовательскую среду с меткой безопасности 1:0:0 для выполнения команд Podman от имени пользователя macuser:

```
sudo mac-podman-init --user macuser --label 1:0:0
```

8.8.2. Работа с Podman от имени другого пользователя

Администратор может запускать процессы Podman с ненулевой меткой безопасности от имени другого пользователя с помощью инструмента `mac-podman`. При этом используется следующий синтаксис:

```
sudo mac-podman --user <имя_пользователя> --label <метка_безопасности> [параметры] \
    -- <команда_Podman>
```

Примечание. Для параметров `--user` и `--label` допускается использовать короткий формат записи: `-u` и `-l` соответственно.

Метка безопасности задается четырьмя десятичными неотрицательными числами, разделенными двоеточием, так же как и при использовании инструмента `mac-podman-init` (см. 8.8.1).

Перечень и описание команд Podman, включая команды запуска контейнеров, приведены в документе РУСБ.10015-01 95 01-1.

Необязательные параметры команды описаны в таблице 60.

Таблица 60

Параметр	Описание
<code>-h, --help</code>	Вывести справку и выйти
<code>-p,</code> <code>--privsock</code>	Процессам, которые запускаются командой, присвоить привилегию <code>PARSEC_CAP_PRIV_SOCK</code> (см. 4.7). Особенности сетевого взаимодействия в условиях мандатного управления доступом ОС описаны в 4.14.
<code>-d <путь>,</code> <code>--home <путь></code>	Использовать указанный каталог в качестве инфраструктурного каталога для хранения всей инфраструктуры контейнеров. ВНИМАНИЕ! Инфраструктурный каталог должен быть предварительно создан с помощью инструмента <code>mac-podman-init</code> (см. 8.8.1). Если параметр не указан, то инструмент <code>mac-podman</code> будет пытаться использовать каталог <code>/home/.pdp/<имя_пользователя>/z<метка_безопасности></code> . Если каталог отсутствует, то выполнение команды будет аварийно прервано и будет выведено сообщение об ошибке. Примечания: 1. <code><имя_пользователя></code> и <code><метка_безопасности></code> соответствуют значениям, указанным в команде. 2. Метка безопасности обозначается четырьмя числами, разделенными подчеркиванием

Окончание таблицы 60

Параметр	Описание
-t <путь>, --tmp <путь>	Использовать указанный каталог в качестве каталога временных файлов. Например, параметр следует использовать при выполнении команды загрузки образа Podman, т.к. для размещения временных файлов требуется значительный объем дискового пространства. Если указанный каталог уже существует, то инструмент <code>mac-podman</code> сравнит его метку безопасности с указанной в команде. При их несовпадении инструмент <code>mac-podman</code> попытается назначить каталогу метку безопасности, указанную в команде. Если каталог отсутствует, то инструмент <code>mac-podman</code> создаст его по указанному пути и попытается назначить ему метку безопасности, указанную в команде. ВНИМАНИЕ! Классификационная метка каталога не может превышать значение классификационной метки каталога, в котором он расположен. При загрузке образа рекомендуется каталог временных файлов размещать в каталоге <code>/var/</code>, который имеет метку безопасности <code>3:63:-1:ccnr</code>. Такое значение метки безопасности позволяет создавать в каталоге <code>/var/</code> файловые объекты с любыми метками безопасности. Каталог временных файлов также можно создать с помощью инструмента <code>mac-podman-init</code> (см. 8.8.1). Если параметр не указан, то инструмент <code>mac-podman</code> будет пытаться использовать каталог <code>/run/user/<идентификатор>z<метка_безопасности>/tmp/</code>, где <code><идентификатор></code> — идентификатор пользователя (UID), имя которого указано в команде; <code><метка_безопасности></code> — метка безопасности, указанная в команде. Метка безопасности обозначается четырьмя числами, разделенными подчеркиванием. Если каталог отсутствует, то выполнение команды будет аварийно прервано и будет выведено сообщение об ошибке. Примечание. В каталог <code>/run/user/<идентификатор>z<метка_безопасности>/tmp/</code> монтируется временная ФС <code>tmpfs</code> — временное файловое хранилище в оперативной памяти. По умолчанию выделяется 10% оперативной памяти. Такого объема памяти может не хватить для загрузки образа. После выполнения команды Podman каталог временных файлов будет очищен, но не будет удален

Пример

Создать образ и запустить контейнер Podman от имени пользователя `macuser`, присвоив процессу Podman метку безопасности `1:0:0`:

1) собрать chroot-окружение:

```
sudo debootstrap --verbose --components=main,contrib,non-free,non-free-
firmware 1.8_x86-64 /var/podman-chroot http://dl.astralinux.ru/astra/
stable/1.8_x86-64/repository-main
```

где 1.8_x86-64 — код дистрибутива;
 /var/podman-chroot — каталог сборки окружения;
 http://dl.astralinux.ru/... — расположение репозитория в сети;

2) создать образ astra_image из chroot-окружения:

```
sudo tar -C /var/podman-chroot \
  -cpf - . | \
  sudo mac-podman --user macuser --label 1:0:0 --tmp /var/macuser_tmp \
  -- podman import - astra_image \
  --change \
  "ENV PATH /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin" \
  --change 'CMD ["/bin/bash"]'
```

где -C /var/podman-chroot — задать каталог в качестве рабочего каталога для архивирования;
 -cpf - . — создать новый архив из рабочего каталога с сохранением разрешений, установленных на входящие в него каталоги и файлы, и передать архив в stdin;
 sudo mac-podman --user macuser --label 1:0:0 --tmp /var/macuser_tmp — выполнить команду Podman от имени пользователя macuser, присвоив процессу Podman метку безопасности 1:0:0. Для размещения временных файлов использовать каталог /var/macuser_tmp/. Указан каталог /var/macuser_tmp/, так как для выполнения команды Podman — создания образа в данном случае требуется объем дискового пространства приблизительно 500 МБ;
 podman import - astra_image — импортировать данные из stdin для создания образа с именем astra_image;
 -change "ENV PATH ..." — задать переменную окружения PATH;
 -change 'CMD ["/bin/bash"]' — задать команду, которая будет автоматически выполнена в контейнере при запуске контейнера из данного образа;

3) вывести список образов, доступных пользователю macuser в пользовательской среде с меткой безопасности 1:0:0:

```
sudo mac-podman --user macuser --label 1:0:0 -- podman images
```

Вывод после выполнения команды:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
localhost/astra_image	latest	ed441842ff0d	7 days ago	505 MB

4) запустить контейнер из созданного образа astra_image от имени пользователя macuser, присвоив процессу Podman метку безопасности 1:0:0:

```
sudo mac-podman --user macuser --label 1:0:0 -- podman run -it --rm \
    astra_image
```

8.8.3. Работа с Podman в сессии с ненулевой меткой безопасности

Для запуска процессов Podman в сессии непривилегированного пользователя с ненулевой меткой безопасности необходимо предварительно создать пользовательскую среду (см. 8.8.1).

ВНИМАНИЕ! Метка безопасности пользовательской среды должна быть равна метке безопасности сессии пользователя.

Для запуска процессов Podman в сессии непривилегированного пользователя с ненулевой меткой безопасности применяется следующий синтаксис:

```
mac-podman -- <команда_Podman>
```

Перечень и описание команд Podman, включая команды запуска контейнеров, приведены в документе РУСБ.10015-01 95 01-1.

Для создания образов контейнеров требуются права администратора. Администратор может создать образ контейнера от имени другого пользователя в пользовательской среде с заданной меткой безопасности (см. 8.8.2).

Пример

Войти в сессию от имени непривилегированного пользователя `macuser`. При входе в сессию выбрать уровень конфиденциальности с наименованием `Уровень_1`. После входа в сессию в терминале последовательно выполнить команды:

- 1) вывести перечень запущенных контейнеров:

```
mac-podman -- podman ps
```

Вывод после выполнения команды:

CONTAINER ID	IMAGE	COMMAND	...
6fa9f20eabbb	localhost/astra_image:latest	/bin/bash	...

В выводе отображен контейнер, который ранее был запущен администратором от имени пользователя `macuser`;

- 2) завершить работу контейнера с идентификатором `6fa9f20eabbb`:

```
mac-podman -- podman stop 6fa9f20eabbb
```


3) вывести список доступных образов:

```
mac-podman -- podman images
```

Вывод после выполнения команды:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
localhost/astra_image	latest	ed441842ff0d	7 days ago	505 MB

4) запустить контейнер из образа `astra_image`:

```
mac-podman -- podman run -it --rm astra_image
```

8.9. Защита приложений, установленных из репозитория Flatpak

В состав ОС входит Flatpak — универсальная система сборки, распространения и запуска приложений в UNIX-подобных операционных системах. Flatpak из состава ОС обеспечивает выполнение следующих функций безопасности:

- изоляция приложений, установленных из репозитория Flatpak;
- выявление уязвимостей в установленных приложениях;
- проверка корректности конфигурации установленных приложений;
- контроль целостности установленных приложений и параметров настройки Flatpak;
- регистрация событий безопасности, связанных с функционированием Flatpak;
- идентификация и аутентификация пользователей, взаимодействующих с Flatpak.

Описание установки, настройки и работы с приложениями, установленными из репозитория Flatpak, приведены в документе РУСБ.10015-01 95 01-1.

8.9.1. Изоляция приложений, установленных из репозитория Flatpak

Каждое приложение, установленное из репозитория Flatpak, запускается вместе с вариантом среды выполнения в изолированном окружении. Для обеспечения изоляции используются следующие механизмы ядра:

- `namespaces` — изоляция пространств имен;
- контрольные группы `control groups` (`cgroups`) — управление доступом приложений к ресурсам хостовой ОС;
- Linux Secure Computing Mode (`seccomp`) — ограничение доступа приложений к системным вызовам ядра хостовой ОС.

Для взаимодействия с механизмами `namespaces` и `seccomp` Flatpak использует утилиту `bubblewrap`.

Упрощенный алгоритм запуска приложения выглядит следующим образом:

- 1) инициализация и сборка изолированного окружения с помощью механизма namespaces:
 - а) изоляция ФС — `bubblewrap` создает пустое пространство сборки, куда будет смонтирован необходимый вариант среды выполнения в качестве корневого каталога «/». Затем, на основе разрешений приложения, будут смонтированы разрешенные каталоги, например `~/Загрузки/`;
 - б) изоляция пользователей — `bubblewrap` создает виртуальную таблицу пользователей, в которой будет информация только о текущем пользователе;
 - в) изоляция приложения от процессов хостовой ОС — внутри изолированного окружения `bubblewrap` присваивает идентификатору первого процесса приложения значение «1»;
 - г) изоляция сети:
 - если приложению запрещен доступ к сети, то `bubblewrap` создает отдельное сетевое пространство имен, в котором будет только виртуальный сетевой адаптер `lo (loopback)`;
 - если приложению разрешен доступ к сети, то `bubblewrap` не создает отдельное сетевое пространство имен. В изолированном окружении будут доступны сетевые адаптеры хоста;
 - д) изоляция межпроцессного взаимодействия — `bubblewrap` выделяет новые ресурсы обмена данными между процессами, такие как очереди сообщений, семафоры и общая память. Если запускаемому приложению разрешен доступ к системной и сессионной шинам D-Bus, то Flatpak запускает специальный процесс `xdg-dbus-proxy`, который выступает в качестве прокси между процессами изолированного окружения и шинами D-Bus хостовой ОС. Описание настройки доступа приложения к системной и сессионной шинам D-Bus приведено в документе РУСБ.10015-01 95 01-1;
- 2) регистрация в системе и запуск контроля потребления ресурсов с помощью механизма `cgroups`:
 - а) запрос к `systemd` — Flatpak отправляет запрос менеджеру `systemd` через системную шину D-Bus;
 - б) создание `.scope` — менеджер `systemd` создает новую контрольную группу;
 - в) привязка процессов — менеджер `systemd` помещает текущий процесс Flatpak в созданную контрольную группу. Вызываемый далее `bubblewrap` и первый процесс приложения окажутся в ней автоматически благодаря механизму наследования;
 - г) запуск контроля потребления ресурсов — ядро ОС начинает вести учет вычислительных ресурсов, потребляемых процессами из контрольной группы. По умолчанию потребление вычислительных ресурсов приложениями не ограничено. Для приложения, установленного из репозитория Flatpak, можно задать огра-

ничения на потребление вычислительных ресурсов хоста. Описание настройки ограничений приложения приведено в документе РУСБ.10015-01 95 01-1;

3) блокировка системных вызовов с помощью механизма `seccomp` и запуск приложения:

а) `bubblewrap` активирует механизм `seccomp` в ядре. При активации механизму `seccomp` будет передан перечень разрешенных и запрещенных системных вызовов;

б) `bubblewrap` выполняет системный вызов `execve()`, в котором указан путь к исполняемому файлу приложения внутри собранного изолированного окружения.

Дополнительно предусмотрена возможность запуска приложения с заданной меткой целостности. Описание приведено в 8.9.7.

8.9.2. Выявление уязвимостей в приложениях из репозитория Flatpak

8.9.2.1. Общие сведения

В состав Flatpak входит система поиска уязвимостей в приложениях, установленных из репозитория Flatpak. Система поиска уязвимостей использует программное обеспечение OpenSCAP и oval-описания. Актуальные версии oval-описаний устанавливаются автоматически при установке Flatpak. Oval-описания содержатся в отдельных файлах для каждого уровня угрозы и группируются по имени и версии операционной системы в каталогах `/usr/share/oval/db/<имя>/<версия>/`. Flatpak из состава ОС использует файлы oval-описания в каталоге `/usr/share/oval/db/astra/1.8_x86-64/`. Так же, как при использовании средств контейнеризации, можно настроить автоматическое обновление oval-описаний (см. 8.2.3.3).

Система поиска уязвимостей состоит из следующих основных подсистем:

- подсистема создания реестра — формирует реестр пакетов из состава приложения, то есть перечень всех компонентов, библиотек, модулей и зависимостей, из которых состоит приложение;
- подсистема проверки — выполняет сканирование приложений на наличие уязвимостей.

8.9.2.2. Сканирование приложений на наличие уязвимостей

Сканирование выполняется автоматически при следующих событиях:

- при установке приложения;
- при запуске приложения;
- периодически по заданному расписанию. Настройка расписания поиска уязвимостей описана в 8.9.2.4.

Упрощенный алгоритм работы подсистемы проверки выглядит следующим образом:

- 1) сканирование на наличие уязвимостей выполняется с помощью программного обеспечения OpenSCAP, которое:
 - а) загружает oval-описания из указанного источника. Каталог, в котором размещены файлы oval-описания для ОС, указан в настройках системы поиска уязвимостей (см. 8.9.2.3);
 - б) анализирует реестр пакетов из состава приложения;
 - в) сопоставляет пакеты из состава приложения с перечнем уязвимостей из oval-описания;
 - г) генерирует отчет о проверке. Каталог, в котором размещены файлы отчетов, указан в настройках системы поиска уязвимостей (см. 8.9.2.3). При генерации отчетов в OpenSCAP создаются файлы следующих форматов:
 - XML — для автоматической обработки сторонним программным обеспечением;
 - HTML — для анализа результатов сканирования администратором безопасности;
- 2) обработка отчета о проверке, сгенерированного OpenSCAP, — подсистема проверки извлекает из отчета информацию о найденных уязвимостях: идентификаторы уязвимостей, уровни критичности, затронутые пакеты;
- 3) выполнение действия при обнаружении угрозы заданного уровня. Действие, которое будет выполнено при обнаружении угрозы заданного уровня, указывается в настройках системы поиска уязвимостей (см. 8.9.2.3).

8.9.2.3. Настройка системы поиска уязвимостей

Настройки системы поиска уязвимостей хранятся в конфигурационном файле `/etc/flatpak/flatpak-vul-scanner.cfg`. Конфигурационный файл разделен на секции.

Описание параметров системы поиска уязвимостей приведено в таблице 61.

Таблица 61

Параметр	Описание
Секция [General] — основные настройки	
<code>enabled=<флаг></code>	Флаг включения подсистемы проверки. Параметр может принимать одно из следующих значений: • <code>true</code> — включить подсистему проверки; • <code>false</code> — выключить подсистему проверки. Значение по умолчанию <code>true</code>

Продолжение таблицы 61

Параметр	Описание
<code>on_violation=<действие></code>	Действие, которое будет выполнено при обнаружении уровня угрозы, заданного в параметре <code>block_level</code>. Параметр может принимать одно из следующих значений: • <code>block</code> — зарегистрировать факт выявления уязвимости и запретить запуск приложения; • <code>log</code> — зарегистрировать факт выявления уязвимости и разрешить запуск приложения; • <code>warn</code> — зарегистрировать факт выявления уязвимости, вывести всплывающее сообщение и разрешить запуск приложения. Значение по умолчанию <code>log</code>
Секция [Severity] — уровень угрозы	
<code>block_level=<уровень></code>	Минимальный уровень угрозы, при обнаружении которой будет выполнено действие, заданное в параметре <code>on_violation</code>. Параметр может принимать одно из следующих значений: • <code>SEVERITY_CRITICAL</code> — критический уровень угрозы; • <code>SEVERITY_HIGH</code> — высокий уровень угрозы; • <code>SEVERITY_MEDIUM</code> — средний уровень угрозы; • <code>SEVERITY_LOW</code> — низкий уровень угрозы. Примечание. Уровень угрозы указан в файлах <code>oval-описания</code>. Значение по умолчанию <code>SEVERITY_CRITICAL</code>
Секция [Scanner] — параметры сканера	
<code>oval_definitions=<каталог></code>	Каталог, в котором размещены файлы <code>oval-описания</code>. По умолчанию каталог <code>/usr/share/oval/db/astra/1.8_x86-64/</code>
<code>reports_dir=<каталог></code>	Каталог, в котором размещены файлы отчетов инструмента OpenSCAP. По умолчанию каталог <code>/tmp/flatpak-scanner/results/</code>
Секция [Base] — настройки подсистемы создания реестра	
<code>auto_create_db=<флаг></code>	Флаг автоматического создания реестра при установке приложения. Параметр может принимать одно из следующих значений: • <code>true</code> — создавать реестр при установке приложения; • <code>false</code> — не создавать реестр при установке приложения. Значение по умолчанию <code>true</code>

Окончание таблицы 61

Параметр	Описание
<code>auto_update_db=<флаг></code>	Флаг автоматического обновления реестра при обновлении приложения. Параметр может принимать одно из следующих значений: • <code>true</code> — обновлять реестр при обновлении приложения; • <code>false</code> — не обновлять реестр при обновлении приложения. Значение по умолчанию <code>true</code>

Настроить систему поиска уязвимостей можно также с помощью графической утилиты `flatpak-manager` (Flatpak Manager). Описание утилиты приведено в электронной справке.

8.9.2.4. Настройка расписания поиска уязвимостей

Настройки расписания поиска уязвимостей хранятся в конфигурационном файле `/usr/lib/systemd/system/flatpak-vul-scanner.timer` в секции [Timer].

Описание параметров периодичности поиска уязвимостей приведено в таблице 62.

Таблица 62

Параметр	Описание
<code>OnCalendar=<периодичность></code>	Периодичность выполнения поиска уязвимостей. Параметр может принимать одно из следующих значений: • <code>daily</code> — каждый день в 00:00:00; • <code>weekly</code> — каждый понедельник в 00:00:00; • <code>monthly</code> — каждое первое число месяца в 00:00:00. Значение по умолчанию <code>daily</code>
<code>AccuracySec=12h</code>	Окно точности срабатывания — 12 часов. В ОС разные периодические задачи могут быть объединены. Указанное значение параметра разрешает выполнить поиск уязвимостей в любой момент между 00:00 и 12:00. Заданное значение параметра менять не рекомендуется
<code>Persistent=true</code>	Включить сохранение состояния таймера на диске. Если в момент запланированного запуска поиска уязвимостей хост был выключен, то при следующем включении система увидит, что задание было пропущено, и запустит поиск уязвимостей незамедлительно. Заданное значение параметра менять не рекомендуется

8.9.3. Обеспечение корректности конфигурации приложений

Корректность конфигурации приложений, которые устанавливаются из репозитория Flatpak, обеспечивается:

- ограничением прав приложения на использование устройств хоста: периферийных устройств, блочных устройств, таких как устройств хранения данных и съемных машинных носителей информации;
- ограничением прав приложения на использование вычислительных ресурсов хостовой ОС: оперативной памяти, операций ввода-вывода за период времени;
- монтированием корневой ФС хостовой ОС в режиме «только для чтения».

Приложения получают минимально необходимые разрешения на доступ к основным системным службам, сокетам, устройствам и файловым объектам хостовой ОС. Перечень этих разрешений задан разработчиками и хранится в метаданных. Разрешения, заданные для установленного приложения, можно переопределить, а также задать ограничения на потребление вычислительных ресурсов хоста. Описание настройки разрешений и ограничений приложения приведено в документе РУСБ.10015-01 95 01-1.

8.9.4. Контроль целостности приложений и параметров настройки Flatpak

8.9.4.1. Контроль целостности приложений

В состав Flatpak включен программный модуль контроля целостности, который имеет следующие настройки по умолчанию:

- после установки каждого приложения или варианта среды выполнения создается отдельная эталонная база данных контрольных сумм файлов, которые входят в состав установленного приложения или варианта среды выполнения;
- контрольные суммы рассчитываются по алгоритму MD5;
- целостность приложения или варианта среды выполнения проверяется в ручном режиме;
- при обнаружении нарушения целостности приложения или варианта среды выполнения запуск приложения запрещен.

С помощью графической утилиты `flatpak-manager` (Flatpak Manager) можно изменить следующие настройки программного модуля контроля целостности:

- 1) отключить или включить автоматическое создание эталонной базы данных;
- 2) выбрать алгоритм подсчета контрольных сумм. Для выбора доступны следующие алгоритмы: MD5 и ГОСТ Р 34.11-2012;
- 3) выключить или включить автоматическую проверку целостности перед запуском приложения;

4) выбрать действие, которое будет выполняться при обнаружении нарушения целостности приложения или варианта среды выполнения. Доступны следующие действия:

- а) зарегистрировать факт нарушения целостности и запретить запуск приложения;
- б) зарегистрировать факт нарушения целостности, вывести всплывающее сообщение о нарушении целостности и разрешить запуск приложения;
- в) зарегистрировать факт нарушения целостности и разрешить запуск приложения.

Описание графической утилиты `flatpak-manager` (Flatpak Manager) приведено в электронной справке.

Для управления эталонными базами данных контрольных сумм, запуска проверки целостности и вывода результатов проверки используется команда:

```
flatpak integrity [параметры]
```

Если команду выполнить без параметров, то будет выведена краткая справка об использовании команды.

Описание параметров, которые используются в команде, представлено в таблице 63.

Таблица 63

Параметр	Описание
Действие. В команде допускается указать только одно действие	
<code>list</code>	Вывести информацию о наличии эталонных баз данных для установленных приложений или вариантов среды выполнения. Будет выведена таблица, состоящая из следующих столбцов: • «Приложение» — идентификатор установленного приложения или варианта среды выполнения; • «Тип» — тип отображаемого объекта. В столбце может быть отображено одно из значений: • «Приложение» — для установленного приложения; • «Среда выполнения» — для установленного варианта среды выполнения; • «База данных» — индикатор наличия эталонной базы данных. В столбце может быть отображено одно из значений: • «Да» — эталонная база данных создана; • «Нет» — эталонная база данных отсутствует

Продолжение таблицы 63

Параметр	Описание
remove <идентификатор>	Удалить эталонную базу данных для указанного приложения или варианта среды выполнения. Если требуется удалить эталонные базы данных для всех приложений или вариантов среды выполнения, то вместо идентификатора указать значение --all
init [идентификатор]	Создать эталонную базу данных для указанного приложения. Если идентификатор не указан, то эталонные базы данных будут созданы для всех приложений или вариантов среды выполнения. Примечание. Если эталонная база данных уже создана, то она будет обновлена
update [идентификатор]	Обновить эталонную базу данных для указанного приложения или варианта среды выполнения. Если идентификатор не указан, то будут обновлены все эталонные базы данных
check [идентификатор]	Проверить целостность указанного приложения или варианта среды выполнения. Если идентификатор не указан, то будет проверена целостность всех приложений или вариантов среды выполнения

Продолжение таблицы 63

Параметр	Описание
status	Вывести результаты проверки целостности всех установленных приложений или вариантов среды выполнения. Будет выведена таблица, состоящая из следующих столбцов: • «Приложение» — идентификатор установленного приложения или варианта среды выполнения; • «Статус» — результат проверки. В столбце может быть отображено одно из значений: • «ОК» — при последней проверке не обнаружено нарушение целостности; • «ИЗМЕНЕНО» — при последней проверке обнаружено нарушение целостности; • «Последняя проверка» — дата и время последней проверки; • «Тип проверки» — событие, при котором проводилась проверка. В столбце может быть отображено одно из значений: • «install» — проверка не проводилась. Эталонная база данных была создана при установке приложения или варианта среды выполнения; • «manual» — проверка была выполнена в ручном режиме; • «run» — проверка была выполнена в автоматическом режиме при запуске приложения; • «Изменено» — количество измененных файлов; • «Добавлено» — количество добавленных файлов; • «Удалено» — количество удаленных файлов
-h, --help	Вывести справку и выйти
Параметры, определяющие пространство установки, в котором будет выполнено действие, указанное в команде. Могут использоваться совместно с одним из параметров: --app или --runtime. В команде допускается указать только один параметр	
-u, --user	Выполнить команду в пользовательском пространстве установки. Если в команде не указаны параметры --user, --system или --installation, то команда будет выполнена в системном пространстве установки. ВНИМАНИЕ! Чтобы выполнить команду в системном или дополнительном пространстве установки, необходимо использовать механизм sudo

Окончание таблицы 63

Параметр	Описание
<code>--system</code>	Выполнить команду в системном пространстве установки. Используется по умолчанию. Если в команде не указаны параметры <code>--user</code>, <code>--system</code> или <code>--installation</code>, то команда будет выполнена в системном пространстве установки. ВНИМАНИЕ! Чтобы выполнить команду в системном или дополнительном пространстве установки, необходимо использовать механизм <code>sudo</code>
<code>--installation=<пространство></code>	Выполнить команду в указанном дополнительном пространстве установки. Если в команде не указаны параметры <code>--user</code>, <code>--system</code> или <code>--installation</code>, то команда будет выполнена в системном пространстве установки. ВНИМАНИЕ! Чтобы выполнить команду в системном или дополнительном пространстве установки, необходимо использовать механизм <code>sudo</code>
Параметры, определяющие тип объектов, в отношении которых будет выполнено действие, указанное в команде. Могут использоваться совместно с одним из параметров: <code>--user</code>, <code>--system</code> или <code>--installation</code>. В команде допускается указать только один параметр	
<code>--app</code>	Выполнить указанное действие только для установленных приложений. Используется по умолчанию. Если параметры <code>--app</code> или <code>--runtime</code> не указаны, то действие будет выполнено только для установленных приложений
<code>--runtime</code>	Выполнить указанное действие только для установленных вариантов среды выполнения. Если параметры <code>--app</code> или <code>--runtime</code> не указаны, то действие будет выполнено только для установленных вариантов среды выполнения
Параметры, которые включают вывод дополнительной информации в процессе выполнения команды	
<code>-v, --verbose</code>	Вывести краткую отладочную информацию Flatpak
<code>-vv</code>	Вывести подробную отладочную информацию Flatpak
<code>--ostree-verbose</code>	Вывести отладочную информацию системы OSTree

Примеры:

1. Вывести информацию о наличии эталонных баз данных у приложений, установленных в пользовательское пространство:

```
flatpak integrity list --user
```

Вывод после выполнения команды:

Приложение	Тип	База данных
org.astra.Kate	Приложение	Нет
org.gimp.GIMP	Приложение	Нет
org.kde.kate	Приложение	Нет

2. Создать эталонные базы данных для всех приложений, установленных в пользовательское пространство:

```
flatpak integrity init --user
```

3. Вывести результаты последних проверок:

```
flatpak integrity status
```

Вывод после выполнения команды:

Приложение	Статус	Последняя проверка	Тип проверки	Изменено \
Добавлено	Удалено			
...				
org.kde.Platform	OK	2026-05-29 07:13:09	install	0 \
0	0			
org.astra.Gimp	OK	2026-06-01 12:43:05	install	0 \
0	0			
org.kde.kcalc	ИЗМЕНЕНО	2026-06-01 17:25:21	manual	0 \
1	0			
...				

Управлять эталонными базами данных контрольных сумм, запускать проверки целостности и просматривать результаты проверки также можно с помощью графической утилиты `flatpak-manager` (Flatpak Manager). Описание утилиты приведено в электронной справке.

8.9.4.2. Контроль целостности параметров настройки Flatpak

Контроль целостности параметров настройки Flatpak реализуется сертифицированными средствами регламентного контроля целостности AFICK из состава ОС (см. 10.4). Чтобы включить контроль целостности параметров настройки Flatpak, в конфигурационный файл `/etc/afick.conf` в секции `file section` добавить строку:

```
/etc/flatpak/ GOST
```

С помощью системного планировщика заданий `cron` можно настроить расписание проверки целостности или настроить запуск проверки целостности сразу после загрузки ОС.

8.9.5. Регистрация событий безопасности, связанных с Flatpak

Регистрация событий безопасности, настройка реагирования системы на события и информирование администратора осуществляется подсистемой регистрации событий из состава ОС. Описание подсистемы регистрации событий и журнала событий приведено в 6.5. Событиям безопасности, связанным с работой Flatpak, присваиваются метки `flatpak_security_audit`.

События безопасности, связанные с работой Flatpak, также регистрируются штатными средствами Flatpak в системных журналах `/var/log/daemon.log` и `/var/log/syslog`.

Регистрация событий, связанных с контролем целостности приложений, выполняется с помощью программного модуля контроля целостности из состава Flatpak. События регистрируются в журнал `/var/log/flatpak/integrity.log`.

Просмотреть и экспортировать перечень событий, зарегистрированных программным модулем контроля целостности также можно с помощью графической утилиты `flatpak-manager` (Flatpak Manager). Описание утилиты приведено в электронной справке.

События аутентификации пользователя, в т.ч. неуспешные попытки аутентификации, регистрируются:

- 1) в журнале подсистемы регистрации событий (см. 6.5);
- 2) в журнале `/var/log/auth.log`.

8.9.6. Идентификация и аутентификация пользователей

Первичную идентификацию пользователей, которые используют Flatpak, выполняет администратор безопасности. Процедуры идентификации и аутентификации соответствуют требованиям ГОСТ Р 58833-2020. При входе в систему подлинность субъектов доступа проверяется в соответствии с регламентом, приведенным в разделе 2.

Аутентификационная информация пользователей, которая хранится в ОС, защищена с использованием хеш-функций по ГОСТ Р 34.11-2012 (ГОСТ Р 34.11-94).

Пароль пользователя назначает администратор. Установка пароля пользователя описана в документе РУСБ.10015-01 95 01-1.

Пароль пользователя должен содержать не менее 8 символов при алфавите пароля не менее 70 символов. Максимальное количество неуспешных попыток аутентификации (ввода неправильного пароля) до блокировки — 4.

Для запуска приложения, установленного из репозитория Flatpak, используется утилита `bubblewrap`, которая запрещает запущенному приложению создавать в хостовой ОС процессы, обладающие привилегиями администратора информационной (автоматизированной) системы и администратора безопасности информационной (автоматизированной) системы. Использование утилиты `bubblewrap` обеспечивает:

- 1) запрет запуска процессов от имени пользователя `root`;
- 2) запрет просмотра процессов хостовой ОС;
- 3) невозможность какого-либо влияния на процессы хостовой ОС и других приложений;
- 4) невлияние монтирования и размонтирования ФС на состав дерева каталогов хостовой ОС.

8.9.7. Запуск приложения с указанной меткой целостности

8.9.7.1. Формирование метки целостности запущенного приложения

По умолчанию приложение, установленное из репозитория Flatpak, имеет нулевую метку целостности.

Администратор может изменить метку целостности приложения (см. 8.9.7.2). Непривилегированный пользователь при запуске приложения может переопределить значение линейного уровня целостности, указав отрицательное значение (см. 8.9.7.4). При этом непривилегированный пользователь не может задать линейный уровень целостности выше значения, назначенного администратором.

При запуске приложения действуют следующие правила:

- 1) в сессии пользователя с минимальной категорией целостности:
 - а) если линейный уровень целостности приложения выше или равен нулю, то приложение будет запущено с нулевой меткой целостности, независимо от назначенной ему метки целостности;
 - б) если линейный уровень целостности приложения ниже нуля, то дополнительно учитывается значение категории целостности приложения:
 - если категория целостности приложения равна нулю, то приложение будет запущено с назначенной ему меткой целостности;
 - если категория целостности приложения выше нуля, то запуск приложения запрещен;
- 2) в сессии пользователя с максимальной категорией целостности:
 - если линейный уровень целостности приложения выше нуля, то запуск приложения запрещен;
 - если линейный уровень целостности приложения ниже или равен нулю, то приложение будет запущено с назначенной ему меткой целостности.

Примечание. Уровень и категории конфиденциальности приложения наследуются от сессии пользователя.

К процессам, запущенным приложением, применяются правила мандатного управления доступом и мандатного контроля целостности, описанные в 4.6.

8.9.7.2. Назначение метки целостности приложению

ВНИМАНИЕ! Назначать метку целостности приложению, установленному из репозитория Flatpak, необходимо в сессии с максимальной категорией целостности.

Чтобы назначить метку целостности приложению, установленному в системное пространство, используется команда:

```
sudo flatpak override --pdplabel=<метка_безопасности> <идентификатор_приложения>
```

ВНИМАНИЕ! В команде должна быть указана именно метка безопасности, которая включает в себя метку целостности. Формат метки безопасности, указанной в команде, должен соответствовать формату, который используется инструментом `pdpl-file` (см. 4.16.11):

```
0:<категория_целостности>:0:0:<линейный_уровень_целостности>
```

Категория целостности может быть задана именем или десятичным числом. Линейный уровень целостности задается десятичным числом.

Примечания:

1. Дополнительные атрибуты для мандатного управления доступом и дополнительные атрибуты для МКЦ используются только для файловых объектов. Если в команде будут заданы дополнительные атрибуты для мандатного управления доступом или дополнительные атрибуты для МКЦ, то они будут назначены приложению, но при запуске приложения не будут учитываться.
2. Если в команде будет дополнительно задана метка конфиденциальности, то она будет назначена приложению, но при запуске приложения не будет учитываться. Метка конфиденциальности при запуске приложения наследуется от сессии пользователя.

Подробная информация об использовании команды приведена в документе РУСБ.10015-01 95 01-1.

Пример

Системному приложению `org.astra.Kate` назначить максимальную категорию целостности и линейный уровень целостности, равный `-128`:

```
sudo flatpak override --pdplabel=0:Высокий:0:0:-128 org.astra.Kate
```

Назначить метку целостности любому установленному приложению также можно с помощью графической утилиты `flatpak-manager` (Flatpak Manager). Описание утилиты приведено в электронной справке.

8.9.7.3. Просмотр метки целостности, назначенной приложению

Метка целостности включена в состав метки безопасности.

Для вывода метки безопасности, назначенной приложению, используется команда:

```
flatpak info --show-permissions <идентификатор_приложения> | grep pdplabel
```

Примечание. Для параметра `--show-permissions` допускается использовать укороченную запись `-M`.

Подробная информация об использовании команды приведена в документе РУСБ.10015-01 95 01-1.

В результате выполнения команды будет выведена строка вида:

```
pdplabel=<метка_безопасности>!
```

Метка безопасности приложения будет отображена в виде пяти чисел, разделенных двоеточием:

- 1) уровень конфиденциальности. Указывается в виде десятичного числа.
Примечание. Указанное значение не будет учитываться при запуске приложения. Уровень конфиденциальности наследуется от сессии пользователя;
- 2) категория целостности. Указывается в виде десятичного числа;
- 3) категории конфиденциальности. Указываются в виде шестнадцатеричного числа.
Примечание. Указанное значение не будет учитываться при запуске приложения. Категории конфиденциальности наследуются от сессии пользователя;
- 4) дополнительные атрибуты для мандатного управления доступом и дополнительные атрибуты для МКЦ. Указываются в виде шестнадцатеричного числа.
Примечание. Этот составной элемент метки безопасности применяется только для файловых объектов и не будет учитываться при запуске приложения;

5) линейный уровень целостности. Указывается в виде десятичного числа.

Если в результате выполнения команды вывод отсутствует, то это означает, что метка безопасности не назначена и приложение будет запускаться с нулевой меткой безопасности.

Пример

Вывести метку безопасности, назначенной приложению `org.astra.Kate`:

```
flatpak info --show-permissions org.astra.Kate | grep pdlabel
```

Вывод после выполнения команды:

```
pdlabel=0:63:0x0:0x0:-128!
```

Посмотреть метку целостности, назначенной приложению, также можно с помощью графической утилиты `flatpak-manager` (Flatpak Manager). Описание утилиты приведено в электронной справке.

8.9.7.4. Переопределение метки целостности при запуске приложения

Для запуска приложения с указанной меткой целостности используется команда:

```
flatpak run --pdlabel=<метка_безопасности> <идентификатор_приложения>
```

ВНИМАНИЕ! В команде должна быть указана именно метка безопасности, которая включает в себя метку целостности.

Формат метки безопасности, указанной в команде, должен соответствовать формату, который используется инструментом `pdpl-file` (см. 4.16.11):

```
0:<категория_целостности>:0:0:<линейный_уровень_целостности>
```

Категория целостности может быть задана именем или десятичным числом. Линейный уровень целостности задается десятичным числом, в том числе отрицательным.

Примечания:

1. Дополнительные атрибуты для мандатного управления доступом и дополнительные атрибуты для МКЦ используются только для файловых объектов. Если в команде будут заданы дополнительные атрибуты для мандатного управления доступом или дополнительные атрибуты для МКЦ, то они не будут учитываться при запуске приложения.

2. Если в команде будет дополнительно задана метка конфиденциальности, то она не будет учитываться при запуске приложения. Метка конфиденциальности при запуске приложения наследуется от сессии пользователя.

Подробная информация об использовании команды приведена в документе РУСБ.10015-01 95 01-1.

Пример

Запустить приложение `org.astra.Kate` с минимальной категорией целостности и линейным уровнем целостности, равным `-8`:

```
flatpak run --pdlabel=0:Низкий:0:0:-8 org.astra.Kate
```

8.9.7.5. Сохранение файлов из приложений с отрицательным значением линейного уровня целостности

По умолчанию приложения, запущенные с отрицательным значением линейного уровня целостности, не могут создавать каталоги и сохранять файлы в ФС хостовой ОС.

Администратор может подготовить каталог для приложения, которое запущено с отрицательным значением уровня целостности от имени заданного пользователя. Приложению будут разрешены операции чтения и записи для всех файловых объектов в этом каталоге, в том числе создание новых файловых объектов. Подготовка выполняется в следующем порядке:

- 1) создать каталог, к файловым объектам которого приложение будет иметь доступ;
- 2) задать пользователя-владельца и группу-владельца для созданного каталога (см. 3.3.1). Необходимо указать учетную запись пользователя, от имени которого будет запущено приложение;
- 3) назначить для созданного каталога линейный уровень целостности, равный `-128` (см. 4.16.11);
- 4) разрешить приложению создание файловых объектов в созданном каталоге. Описание настройки разрешений приложения приведено в документе РУСБ.10015-01 95 01-1.

Пример

Для приложения `org.astra.Kate`, которое будет запускаться от имени пользователя `user` с отрицательным значением линейного уровня целостности, подготовить специальный каталог `/home/user/flatpak-share/`:

- 1) в домашнем каталоге пользователя `user` создать каталог `flatpak-share/`:

```
sudo mkdir /home/user/flatpak-share/
```

2) назначить пользователя `user` в качестве пользователя-владельца, а первичную группу `user` назначить в качестве группы-владельца:

```
sudo chown user:user /home/user/flatpak-share/
```

3) назначить для каталога `/home/user/flatpak-share/` линейный уровень целостности, равный `-128`:

```
sudo pdpl-file :::-128 /home/user/flatpak-share/
```

4) разрешить приложению `org.astra.Kate` операции чтения и записи для всех файловых объектов в каталоге `/home/user/flatpak-share/`:

```
sudo flatpak override --filesystem=/home/user/flatpak-share/:rw org.\  
astra.Kate
```

9. ЗАЩИТА ПАМЯТИ

9.1. Очистка памяти

Ядро ОС гарантирует, что обычный непривилегированный процесс не получит данные чужого процесса, если это явно не разрешено ПРД. Это означает, что средства IPC контролируются с помощью ПРД и процесс не может получить неочищенную память (как оперативную, так и дисковую).

Дополнительно ядро ОС обеспечивает очистку остаточной информации:

- в ядерном стеке (STACKLEAK);
- в ядерной куче (PAGE_POISONING).

Также в ОС реализован механизм очистки внешней памяти, который очищает:

- неиспользуемые блоки ФС непосредственно при их освобождении;
- разделы страничного обмена (разделы подкачки) при выключении системы.

Дополнительно можно отключить сохранение в журнал ФС информации об удалении файлов или включить асинхронный режим очистки освобождаемых блоков ФС. Асинхронный режим позволяет запустить процесс очистки и продолжить выполнение других задач, не дожидаясь завершения этого процесса. Рекомендуется включать этот режим только в том случае, если наблюдаются проблемы с производительностью системы, вызванные удалением файлов большого объема.

При включенном мандатном управлении доступом можно настроить очистку освобождаемых блоков ФС только при удалении файлов с ненулевой меткой безопасности.

Очистка неиспользуемых блоков поддерживается на разделах с ФС ext2, ext3, ext4 и XFS. Механизм очистки внешней памяти может быть включен при установке ОС путем выбора соответствующего пункта. После установки ОС управление очисткой освобождаемых блоков ФС выполняется с использованием инструмента `astra-secdel-control` (см. 16.6.29), управление очисткой разделов и файлов страничного обмена выполняется с использованием инструмента `astra-swapwiper-control` (см. 16.6.30).

При включенной очистке освобождаемых блоков ФС осуществляется запись предопределенной или псевдослучайной маскирующей последовательности в блок размещения удаляемых/усекаемых файлов, таким образом предотвращая вероятность их восстановления. Работа данного механизма снижает скорость выполнения операций удаления и усечения размера файла.

Очистка освобождаемых блоков ФС настраивается отдельно для каждого раздела с соответствующей ФС, который указан в файле `/etc/fstab`. Если механизм очистки памяти

был включен при установке ОС, то по умолчанию очистка освобождаемых блоков ФС будет применяться для всех разделов, указанных в файле `/etc/fstab`.

Очистка освобождаемых блоков ФС может быть настроена для работы в одном из следующих режимов:

1) перезапись каждого байта в освобождаемом блоке сигнатурой 11111111. Для включения режима следует в файле `/etc/fstab` для раздела добавить параметр монтирования `secdel`.

Пример

```
/dev/sda1 /home ext4 acl,defaults,secdel 0 2
```

Данный режим используется по умолчанию, если при установке ОС был включен механизм очистки памяти;

2) заданное число перезаписей каждого байта в освобождаемом блоке последовательным применением сигнатур 11111111, 01010101, 10101010, 00000000 (первая перезапись сигнатурой 11111111, вторая перезапись сигнатурой 01010101 и т.д.). При установке числа перезаписей больше четырех сигнатуры будут использоваться повторно. Например, при установке шести перезаписей будут применяться последовательно сигнатуры 11111111, 01010101, 10101010, 00000000, 11111111, 01010101. Для включения режима следует в файле `/etc/fstab` для раздела добавить параметр монтирования `secdel=<число_перезаписей>`.

Пример

```
/dev/sda1 /home ext4 acl,defaults,secdel=6 0 2
```

3) перезапись байтов в освобождаемом блоке псевдослучайными последовательностями длиной 64 бита. Если размер освобождаемого блока не кратен 64 битам, то последняя последовательность будет использоваться нужной длины. Для включения режима следует в файле `/etc/fstab` для раздела добавить параметр монтирования `secdelrnd`.

Пример

```
/dev/sda1 /home ext4 acl,defaults,secdelrnd 0 2
```

4) заданное число перезаписей байтов в освобождаемом блоке псевдослучайными последовательностями длиной 64 бита. Если размер освобождаемого блока не кратен 64 битам, то последняя последовательность в каждом цикле перезаписи будет использоваться нужной длины. Для включения режима следует в файле `/etc/fstab` для раздела добавить параметр монтирования `secdelrnd=<число_перезаписей>`.

Пример

```
/dev/sda1 /home ext4 acl,defaults,secdelrnd=6 0 2
```

Чтобы не сохранять в журнал ФС информацию об удалении файлов, необходимо в файле `/etc/fstab` для раздела добавить параметр монтирования `secdel_nojournal`. Его необходимо указывать после параметра, который задает режим очистки освобождаемых блоков ФС.

Пример

```
/dev/sda1 /home ext4 acl,defaults,secdelrnd=6,secdel_nojournal 0 2
```

Чтобы настроить очистку освобождаемых блоков ФС только при удалении файлов с ненулевой меткой безопасности, необходимо в файле `/etc/fstab` для раздела добавить параметр монтирования `secdel_pdp1`. Его необходимо указывать после параметра, который задает режим очистки освобождаемых блоков ФС.

Примечание. Параметры монтирования `secdel_pdp1` и `secdel_nojournal` могут быть указаны в любом порядке.

Пример

```
/dev/sda1 /home ext4 acl,defaults,secdelrnd=6,\  
secdel_nojournal,secdel_pdp1 0 2
```

Чтобы включить асинхронный режим очистки освобождаемых блоков ФС, необходимо в файле `/etc/fstab` для раздела добавить параметр монтирования `secdel_lowlatency`. Его необходимо указывать после параметра, который задает режим очистки освобождаемых блоков ФС.

Примечание. Параметры монтирования `secdel_pdp1`, `secdel_lowlatency` и `secdel_nojournal` могут быть указаны в любом порядке.

Пример

```
/dev/sda1 /home ext4 acl,defaults,secdelrnd=6,\  
secdel_nojournal,secdel_pdp1,secdel_lowlatency 0 2
```

Изменение настроек очистки освобождаемых блоков ФС вступает в действие после следующего монтирования раздела (в т.ч. после перезагрузки ОС, если для раздела не задан параметр монтирования `noauto`).

При включенной очистке разделов и файлов страничного обмена (подкачки) в конфигурационном файле `/etc/parsec/swap_wiper.conf` для параметра `ENABLED` задано значение `Y`:

```
ENABLED=Y
```

Для задания списка разделов и файлов страничного обмена, для которых не требуется выполнять очистку, может быть использован параметр `IGNORE`, в качестве значения которого указываются через пробел игнорируемые разделы и файлы страничного обмена.

Пример

```
IGNORE="/dev/sdz10 /dev/sdz11"
```

Управление очисткой блоков ФС при их освобождении, а также настройка очистки разделов и файлов страничного обмена может быть выполнена с использованием модуля «Политика очистки памяти» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_policy_clean_memory
```

9.2. Средства ограничения прав доступа к страницам памяти

Средства ограничения прав доступа к страницам памяти реализованы на основе возможностей ядра, системной библиотеки `C`, поддержки со стороны процессорной архитектуры и параметров ядра ОС.

Включенные параметры ядра ОС предоставляют защиту задачам ядра и процессам пользователей при доступе к страницам оперативной памяти:

- запрет записи в область памяти, помеченную как исполняемая;
- запрет создания исполняемых областей памяти;
- запрет создания исполняемого стека;
- запрет исполнения областей памяти в стеке;
- случайный выбор (рандомизацию) адресного пространства.

Предотвращение выполнения произвольного кода обеспечивается путем контроля доступа к страницам памяти по типам: чтение, запись, исполнение или их комбинации.

Применяются различные механизмы защиты памяти, которые основаны на эмуляции или аппаратной реализации `NX`-бита (бита исполнения на страницах памяти), `PCID` (идентифи-

катор контекста выполнения), а также технологии ASLR (случайный выбор расположения виртуального адресного пространства) и технологии KASLR (случайный выбор расположения адресного пространства ядра).

При наличии аппаратной поддержки NX-бита и PCID на процессорах x86-64 производительность вычислительной системы не ухудшается.

Ядро ОС имеет архитектуру, обеспечивающую невозможность его перемещения в физическом адресном пространстве, при этом технология KASLR задает случайное смещение начального адреса выполнения ядра при каждой его загрузке.

Технология ASLR обеспечивает случайный характер (рандомизацию) смещений сегментов кода и данных (в том числе стека и кучи) при использовании системного вызова отображения в память `mmap()`.

Гарантия того, что адреса с произвольным доступом не будут одновременно доступны на запись и выполнение, реализуется использованием возможностей системных вызовов `mmap()` и `mprotect()`.

10. КОНТРОЛЬ ЦЕЛОСТНОСТИ

Для обеспечения контроля целостности, в т. ч. контроля целостности КСЗ, в ОС реализованы:

- средство подсчета контрольных сумм файлов и оптических дисков (10.1);
- средство подсчета контрольных сумм файлов в deb-пакетах (10.2);
- средство контроля соответствия дистрибутиву (10.3);
- средства регламентного контроля целостности (10.4);
- средства создания и проверки ЭП (10.5);
- средства создания замкнутой программной среды (16.1).

Для решения задач контроля целостности используется библиотека `libgost`, в которой для вычисления контрольных сумм реализованы функции хеширования с длиной хеш-кода 256 бит и с длиной хеш-кода 512 бит в соответствии с ГОСТ Р 34.11-2012. В ОС сохранено использование функции в соответствии с ГОСТ Р 34.11-94 для совместимости.

Библиотека `libgost` используется в средствах подсчета контрольных сумм файлов и оптических дисков, контроля соответствия дистрибутиву и регламентного контроля целостности, модулях аутентификации.

10.1. Подсчет контрольных сумм файлов и оптических дисков

Для подсчета контрольных сумм файлов и оптических дисков в состав ОС включен инструмент командной строки `gostsum`. Для вывода информации о синтаксисе `gostsum` необходимо выполнить команду:

```
gostsum -h
```

Синтаксис инструмента:

```
gostsum [параметр] [файл]
```

Параметры инструмента `gostsum` приведены в таблице 64.

Таблица 64

Параметр	Описание
<code>--gost-94</code>	Рассчитать контрольную сумму по стандарту ГОСТ Р 34.11-94
<code>--gost-2012</code>	Рассчитать контрольную сумму по стандарту ГОСТ Р 34.11-2012 с длиной хеш-кода 256 бит (используется по умолчанию, если не задан другой параметр)
<code>--gost-2012-512</code>	Рассчитать контрольную сумму по стандарту ГОСТ Р 34.11-2012 с длиной хеш-кода 512 бит

Окончание таблицы 64

Параметр	Описание
-b	Установить размер блоков, которыми будет считываться файл. По умолчанию размер блока равен 512 байтам
-o	Задать имя файла для вывода контрольной суммы (по умолчанию стандартный поток вывода)
-d	Задать имя файла устройства чтения оптических дисков (файла с образом оптического диска) для подсчета контрольной суммы (для форматов ISO9660 или UDF)
-e	Рассчитать контрольные суммы файлов из deb-пакета
-r	Выполнять рекурсивно
-E	Исключить все файлы, кроме deb-пакетов
-H	Отключить отображение индикатора прогресса
-t	Протестировать алгоритмы подсчета контрольных сумм
-h	Показать эту справку и выйти

Пример

Подсчет контрольной суммы оптического диска:

```
gostsum -d /dev/cdrom
```

10.2. Подсчет контрольных сумм файлов в deb-пакетах

Для подсчета контрольных сумм файлов в deb-пакетах в состав ОС включен инструмент командной строки `gostsum_from_deb`.

Инструмент устарел и вместо него рекомендуется использовать `gostsum` (см. 10.1)

Для вывода информации о синтаксисе `gostsum_from_deb` необходимо выполнить команду:

```
gostsum_from_deb -h
```

Синтаксис инструмента:

```
gostsum_from_deb [gostsum аргументы] [-d каталог] [-p deb-пакет]
```

Параметры инструмента `gostsum_from_deb` приведены в таблице 65.

Таблица 65

Параметр	Описание
<code>--gost-94</code>	Устанавливает, что будет использован алгоритм ГОСТ Р 34.11-94
<code>--gost-2012</code>	Устанавливает, что будет использован алгоритм ГОСТ Р 34.11-2012 с длиной хеш-кода 256 бит (по умолчанию)
<code>--gost-2012-512</code>	Устанавливает, что будет использован алгоритм ГОСТ Р 34.11-2012 с длиной хеш-кода 512 бит
<code>gostsum arguments</code>	Аргументы утилиты <code>gostsum</code>
<code>-d каталог</code>	Задаёт имя каталога, содержащего deb-пакеты, для файлов которых вычисляются контрольные суммы
<code>-p deb-пакет</code>	Задаёт имя deb-пакета, для файлов которого вычисляются контрольные суммы

10.3. Контроль соответствия дистрибутиву

В состав ОС входят средства, позволяющие после установки ОС контролировать соответствие дистрибутиву объектов ФС, в т. ч. СЗИ. В процедуре контроля соответствия дистрибутиву выполняется контроль целостности файлов, находящихся в файловой системе установленной ОС.

В состав дистрибутива входит файл `gostsums.txt` со списком файлов установленной ОС, которые не должны изменяться в процессе эксплуатации ОС, и их контрольные суммы. Контроль целостности может выполняться для всех файлов из `gostsums.txt` или на основе списков файлов из шаблонов. Шаблоны проверки также представляют собой текстовые файлы, содержащие списки файлов и их контрольные суммы. По умолчанию в системе присутствуют следующие шаблоны:

- 1) `/usr/share/gostsum/templates/gostsum_minimal.txt` — список основных файлов ОС;
- 2) `/usr/share/gostsum/templates/gostsum_fb.txt` — список файлов, реализующих функции безопасности или поддерживающих их выполнение.

Если файл указан в `gostsums.txt` или в шаблоне, но отсутствует в файловой системе установленной ОС, то контроль целостности этого файла не выполняется. Количество отсутствующих при контроле файлов указывается в результатах контроля. Отсутствие файла в ОС при контроле не считается нарушением его целостности.

В файле `gostsums.txt` контрольные суммы рассчитаны по ГОСТ Р 34.11-2012 с длиной хеш-кода 256 бит.

Также в ОС возможно проверять MD5-суммы установленных в системе deb-пакетов и сравнивать их с эталонными значениями, указанным в файлах `*.md5sums` в катало-

re /var/lib/dpkg/info/. Проверка может выполняться как на основе всех файлов *.md5sums, так и на основе шаблонов.

Вычисление контрольных сумм файлов и проверка соответствия полученных значений эталонным контрольным суммам выполняются с помощью графической утилиты fly-admin-int-check или инструмента командной строки astra-int-check.

Для вычисления и проверки контрольных сумм с помощью инструмента командной строки используется команда:

```
astra-int-check [параметры]
```

Параметры инструмента astra-int-check приведены в таблице 66.

Таблица 66

Параметр	Описание
--templates, -t <файлы>	Указать шаблоны проверки. Файлы перечисляются через запятую с указанием полного пути. Если для параметра указано пустое значение -t "", то проверка аналогична проверке с параметром -n. Если параметр не указан и отсутствует параметр -n, то проверка будет выполняться по всем доступным шаблонам
--no-templates, -n	Отключить все шаблоны. Будут проверяться все файлы, указанные в файле gostsums.txt дистрибутива ОС или, при использовании без параметра -p, все файлы, для которых есть контрольные суммы по MD5 в каталоге /var/lib/dpkg/info/
--list-templates, -l	Показать доступные шаблоны
--mount-point, -p <каталог>	Точка монтирования диска с дистрибутивом ОС. Если данный параметр не указан, будет проводиться проверка MD5-сумм установленных deb-пакетов
--add, -a "<фильтр>"	Фильтры для проверки дополнительных файлов и/или каталогов в виде регулярных выражений, перечисляемых через запятую
--ignore, -i "<фильтр>"	Фильтры для исключения файлов и/или каталогов из проверки в виде регулярных выражений, перечисляемых через запятую
--txt <файл>	Сохранить отчет о проверке в виде текстового файла
--html <файл>	Сохранить отчет о проверке в формате HTML
--xml <файл>	Сохранить отчет о проверке в формате XML
--check-directory, -d <каталог>	Проверить содержимое произвольного каталога на соответствие файлу с контрольными суммами
--gost <файл>	Файл с контрольными суммами ГОСТ Р 34.11-2012
--md5 <файл>	Файл с контрольными суммами MD5
--tui	Запуск псевдографического интерфейса пользователя, использование которого аналогично графической утилите fly-admin-int-check

Окончание таблицы 66

Параметр	Описание
<code>--verbose</code>	Подробный вывод
<code>-h, --help</code>	Вывод справки по параметрам командной строки
<code>-v, --version</code>	Показать сведения о версии

Примеры:

1. Проверка содержимого каталога `/home/user/docs` на соответствие контрольным суммам по ГОСТ Р 34.11-2012, перечисленным в файле `gost.txt`, с подробным выводом процесса проверки в консоль:

```
astra-int-check -d /home/user/docs --gost /home/user/gost.txt --verbose
```

Если в файле с контрольными суммами используются относительные пути, то они будут добавлены к указанному каталогу. Если в файле используются абсолютные пути, то проверяться будут только те пути, которые начинаются с указанного каталога.

2. Проверка целостности файлов, перечисленных в шаблоне `gostsum_fb.txt`, за исключением содержимого каталога `/usr/lib`, с созданием отчета о проверке в текстовом формате:

```
astra-int-check -t /usr/share/gostsum/templates/gostsum_fb.txt -p \
/media/cdrom0 -i "/usr/lib/*" --txt /home/user/report.txt
```

Описание использования графической утилиты `fly-admin-int-check` см. в электронной справке.

10.4. Регламентный контроль целостности

Организация регламентного контроля целостности ОС, СЗИ и прикладного ПО обеспечивается набором программных средств на основе Another File Integrity Checker, представленного пакетом `afick`. В указанном наборе реализована возможность для проведения периодического (с использованием системного планировщика заданий `cron`) вычисления контрольных сумм файлов и соответствующих им мандатных атрибутов подсистемы безопасности PARSEC и атрибутов расширенной подсистемы протоколирования с последующим сравнением вычисленных значений с эталонными. В указанном наборе программных средств реализовано использование библиотеки `libgost`, обеспечивающей подсчет контрольных сумм в соответствии с ГОСТ Р 34.11-2012 с длиной хеш-кода 256 бит.

Эталонные значения контрольных сумм и атрибутов файлов хранятся в БД. Данная БД контрольных сумм и атрибутов может быть создана при помощи команды:

```
afick -i
```

Созданный в результате выполнения команды файл `/var/lib/afick/afick.db`. Файл содержит БД с форматом данных `perl Storable`. В файле содержится набор строк, в каждой строке указано имя файла и через пробел его атрибуты и сигнатуры. БД защищается системой разграничения доступа.

Для вычисления контрольных сумм могут использоваться алгоритмы MD5-Digest и ГОСТ Р 34.11-2012.

Настройка регламентного контроля целостности выполняется в конфигурационном файле `/etc/afick.conf`, в котором указываются пути к файлам/каталогам, подвергаемым контролю целостности, и правила контроля целостности, применяемые к файлам и каталогам. Подробное описание файла приведено на справочной странице `man afick.conf`.

По умолчанию в файле `/etc/afick.conf` используются следующие правила контроля целостности:

1) правило PARSEC:

`PARSEC = p+d+i+n+u+g+s+b+md5+m+e+t`

где `p+d+i+n+u+g+s+b+md5+m` — слежение за всеми стандартными атрибутами файла и использование хеш-функции MD5-Digest для слежения за целостностью содержимого файлов;

`+e+t` — контроль расширенных атрибутов: метки безопасности и флагов аудита, соответственно.

Контроль ACL осуществляется при установке флага `+g`;

2) правило GOST:

`GOST = p+d+i+n+u+g+s+b+gost+m+e+t`

где `p+d+i+n+u+g+s+b+gost+m` — слежение за всеми стандартными атрибутами файла и использование хеш-функции ГОСТ Р 34.11-2012 с длиной хеш-кода 256 бит для слежения за целостностью содержимого файлов;

`+e+t` — контроль расширенных атрибутов: метки безопасности и флагов аудита, соответственно.

Контроль ACL осуществляется при установке флага `+g`;

В файле `/etc/afick.conf` в отдельной строке задается путь к файлу или каталогу и применяемое к нему правило контроля.

Пример

/boot	GOST
/bin	GOST
/etc/security	PARSEC
/etc/pam.d	PARSEC
/etc/fstab	PARSEC
/lib/modules	PARSEC
/lib64/security	PARSEC
/lib/security	PARSEC
/sbin	PARSEC
/usr/bin	PARSEC
/usr/lib	PARSEC
/usr/sbin	PARSEC

В конфигурационном файле также представлен ряд дополнительных путей с правилами контроля. Соответствующие строки помечены знаком комментария «#» и могут быть активированы удалением этого знака.

При запуске `afick` автоматически будет установлено ежедневное задание для `cron`. Файл с заданием находится в `/etc/cron.daily/afick_cron/`.

Параметр `report_url:=stdout` задает местоположение файла-отчета.

В конфигурационном файле применен простой язык макросов, который используется при определении переменных для заданий системного планировщика заданий `cron`.

Требуется утилитой `afick` и/или средствами аппаратно-программного модуля доверенной загрузки обеспечить контроль целостности следующих объектов:

1) файлов образов ядра ОС:

`/boot/vmlinuz-*`

2) файлов образов временной файловой системы, используемой ядром ОС при начальной загрузке:

`/boot/initrd.img-*`

Файлы добавлять в список контроля целостности после выполнения всех необходимых настроек, требующих обновления образов временной файловой системы;

3) конфигурационного файла меню загрузчика:

`/boot/grub/grub.cfg`

4) файлов модульной структуры GRUB 2:

`/boot/grub/*`

5) модулей ядра, входящих в подсистему безопасности PARSEC:

```
/lib/modules/*/misc/digsig_verif.ko  
/lib/modules/*/misc/parsec.ko  
/lib/modules/*/misc/parsec-cifs.ko
```

С помощью инструментов командной строки `lsmod` и `modinfo` можно определить перечень модулей ядра, подлежащих контролю целостности;

Рекомендуется утилитой `afick` и/или средствами аппаратно-программного модуля доверенной загрузки обеспечить контроль целостности следующих объектов:

1) конфигурационного файла, определяющего используемый по умолчанию графический дисплейный менеджер:

```
/etc/X11/default-display-manager
```

2) конфигурационного файла настройки файловых систем, доступных для монтирования через NFS:

```
/etc/exports
```

3) конфигурационного файла, содержащего информацию о различных устройствах хранения и файловых системах:

```
/etc/fstab
```

4) файла, содержащего перечень локальных групп ОС:

```
/etc/group
```

Файл добавлять в список контроля целостности после создания всех необходимых групп;

5) сценариев для запуска служб в каталоге:

```
/etc/init.d/
```

6) конфигурационного файла, определяющего параметры работы первого процесса пользовательского режима `init`:

```
/etc/inittab
```

7) конфигурационных файлов, определяющих порядок работы PAM-модулей:

```
/etc/pam.conf  
/etc/pam.d/chfn  
/etc/pam.d/chsh  
/etc/pam.d/common-account  
/etc/pam.d/common-account.pam-old  
/etc/pam.d/common-auth  
/etc/pam.d/common-auth.pam-old  
/etc/pam.d/common-password  
/etc/pam.d/common-password.pam-old  
/etc/pam.d/common-session  
/etc/pam.d/common-session.pam-old  
/etc/pam.d/cron
```



```
/etc/pam.d/cups  
/etc/pam.d/cvs  
/etc/pam.d/dovecot  
/etc/pam.d/fly-dm  
/etc/pam.d/fly-dm-np  
/etc/pam.d/login  
/etc/pam.d/other  
/etc/pam.d/passwd  
/etc/pam.d/polkit  
/etc/pam.d/samba  
/etc/pam.d/sshd  
/etc/pam.d/su  
/etc/pam.d/sumac.xauth
```

8) файла, содержащего перечень локальных пользователей ОС:

```
/etc/passwd
```

Файл добавлять в список контроля целостности после создания всех необходимых пользователей;

9) символических ссылок на сценарии для запуска служб в каталогах:

```
/etc/rc*
```

10) конфигурационного файла, определяющего перечень терминалов, с которых суперпользователь root может регистрироваться в системе:

```
/etc/securetty
```

11) конфигурационного файла, определяющего перечень регистрируемых оболочек в ОС:

```
/etc/shells
```

12) конфигурационного файла, содержащего значения параметров командной строки ядра:

```
/etc/sysctl.conf
```

13) PAM-модулей в каталоге:

```
/lib/security/
```

14) в случае, если в ОС не используется замкнутая программная среда (см. 16.1), то исполняемых файлов в каталогах:

```
/bin/  
/sbin/  
/usr/bin/  
/usr/sbin/
```

В режиме «Мобильный» в каталоге `/usr/sbin/` доступны сценарии регламентного контроля целостности:

1) `astra-mobile-create-int-db` — создание базы данных эталонного состояния системы. Если включена функция проверки целостности системы при запуске, то

сценарий будет выполняться при каждом обновлении списка пакетов (установке/удалении пакетов);

2) `astra-mobile-int-check` — проверка целостности системы. Сценарий будет выполняться при запуске системы, если функция проверки целостности включена. В случае неудачной проверки делается соответствующая запись в файле `/tmp/astra-mobile-int-check.log`.

Описание настройки регламентного контроля целостности с помощью `afick` в графическом интерфейсе режима «Мобильный» приведено в электронной справке («Документация — Графический интерфейс — Режим «Мобильный»).

10.5. Сервис электронной подписи

Комплекс программ из состава ОС предоставляет сервис электронной подписи (СЭП), обеспечивающий интеграцию с дополнительно устанавливаемыми сертифицированными ФСБ России средствами криптографической защиты информации¹⁾ (СКЗИ) в целях создания и проверки усиленной квалифицированной электронной подписи²⁾.

ВНИМАНИЕ! СЭП предоставляется программами, функционирующими в условиях политики разграничения доступа, не допускающей их применение совместно с СКЗИ в режиме обработки сведений, составляющих государственную тайну.

ВНИМАНИЕ! Эксплуатация СКЗИ в составе информационных систем должна осуществляться в соответствии с правилами пользования СКЗИ и указаниями, определенными в формуляре (или иных эксплуатационных документах) на СКЗИ.

СЭП обеспечивает создание и проверку электронной подписи электронных документов в соответствии с ГОСТ Р 34.10-2012 средствами сертифицированных СКЗИ.

Дополнительная информация о настройке ОС и СКЗИ для совместного использования, а также варианты реализации отдельных решений приведены на официальном информационном ресурсе разработчика ОС `wiki.astralinux.ru`.

¹⁾ Не допускается применение для защиты информации, содержащей сведения, составляющие государственную тайну.

²⁾ Электронная подпись (ЭП) (в соответствии с № 63-ФЗ от 06.04.2011) — информация в электронной форме, которая присоединена к другой информации в электронной форме (подписываемой информации) или иным образом связана с такой информацией и которая используется для определения лица, подписывающего информацию (является юридически значимой).

10.5.1. Принцип функционирования

СЭП¹⁾ обеспечивается совокупностью совместно функционирующих программных компонентов ОС:

- 1) `astra-csp-cryptopro` — программный модуль, обеспечивающий предоставление СЭП из командной строки и программного интерфейса. Реализует функции создания и проверки ЭП без взаимодействия в графическом интерфейсе;
- 2) `fly-csp-cryptopro` — программный модуль, обеспечивающий предоставление СЭП из основного меню, графического интерфейса менеджера файлов `fly-fm` и командной строки. Предоставляет функции создания и проверки ЭП в графическом интерфейсе пользователя;
- 3) `libreoffice-astracsp-plugin` — расширение, обеспечивающее предоставление СЭП из графического интерфейса пакета офисных программ LibreOffice в процессе работы с электронными документами;
- 4) `fly-csp-preview` — расширение, обеспечивающее графическое представление на электронном документе отметки об ЭП;
- 5) менеджер файлов `fly-fm`;
- 6) пакет офисных программ LibreOffice.

Непосредственное взаимодействие с СКЗИ (формирование управляющих команд, получение и отображение результата работы СКЗИ с использованием программы `cryptcp`) осуществляет программный модуль `astra-csp-cryptopro`.

Программный модуль `fly-csp-cryptopro` в целях решения функциональных задач СЭП взаимодействует с программным модулем `astra-csp-cryptopro` и не взаимодействует непосредственно с СКЗИ.

Менеджер файлов `fly-fm`, расширение пакета офисных программ LibreOffice `libreoffice-astracsp-plugin` и расширение для добавления отметки об ЭП `fly-csp-preview` в целях решения функциональных задач СЭП взаимодействуют с программным модулем `fly-csp-cryptopro` и не взаимодействуют непосредственно с СКЗИ.

Пакет офисных программ LibreOffice взаимодействует с расширением пакета офисных программ LibreOffice `libreoffice-astracsp-plugin` и не взаимодействует непосредственно с СКЗИ.

В процессе функционирования СЭП осуществляется вызов перечисленных в 10.5.4 программ из состава ОС для предварительного просмотра электронных документов, формируются управляющие команды для СКЗИ с целью вызова функций проверки или создания ЭП, осуществляется отображение получаемого после выполнения вызываемых функций результата.

¹⁾ Здесь и далее в качестве примера приведено описание взаимодействия с СКЗИ «КриптоПро CSP».

Обращение к программному модулю `fly-csp-cryptopro` осуществляется:

- 1) менеджером файлов `fly-fm` (с использованием соответствующего пункта контекстного меню);
- 2) пользователем путем интерактивного запуска приложения с использованием основного меню ОС («Утилиты — Электронная подпись КриптоПро»);
- 3) расширением пакета офисных программ LibreOffice, запуск которого инициируется путем вызова пользователем соответствующего пункта меню графического интерфейса пакета офисных программ LibreOffice.

Обращение к программному модулю `astra-csp-cryptopro` осуществляется пользователем посредством интерфейса командной строки и прикладным программным обеспечением путем запуска на выполнение программного модуля с соответствующими параметрами.

10.5.2. Условия применения

В качестве СКЗИ разрешается использовать только КриптоПро CSP версии 4.0 R4 (исполнения 1-Base и 2-Base), КриптоПро CSP версии 5.0 (исполнения 1-Base и 2-Base) и КриптоПро CSP версии 5.0 R2 (исполнения 1-Base и 2-Base).

СКЗИ КриптоПро CSP в исполнении 2-Base должно использоваться с аппаратно-программным модулем доверенной загрузки.

При эксплуатации СКЗИ необходимо соблюдать требования и рекомендации эксплуатационной документации на СКЗИ, в частности требования по защите от несанкционированного доступа и по криптографической защите, а также требования по поддерживаемым СКЗИ аппаратно-программным платформам. В частности, при использовании СЭП со встроенным СКЗИ необходимо проведение проверки программного обеспечения BIOS ЭВМ, на которых предполагается функционирование СКЗИ и СЭП, на соответствие методическим документам ФСБ России в области исследований программного обеспечения BIOS.

Контроль целостности СКЗИ и СЭП должен выполняться с использованием механизма замкнутой программной среды ОС или с использованием стандартных средств контроля целостности КриптоПро CSP.

Из числа компонентов СЭП контролю целостности подлежит программный модуль `astra-csp-cryptopro`.

Перечень компонентов СКЗИ, подлежащих контролю целостности, приведен в эксплуатационной документации на СКЗИ.

Подробный порядок настройки механизма контроля целостности описан в эксплуатационной документации на ОС и используемое СКЗИ.

10.5.3. Установка

Менеджер файлов `fly-fm` и пакет офисных программ LibreOffice из состава ОС устанавливаются в соответствии с эксплуатационной документацией ОС.

Для использования СЭП дополнительно в системе должны быть установлены следующие программные компоненты (описание компонентов приведено в 10.5.1) из состава ОС:

- 1) программный модуль `astra-csp-cryptopro`;
- 2) программный модуль `fly-csp-cryptopro`;
- 3) расширение `libreoffice-astracsp-plugin`.

Для установки программного модуля и расширения необходимо в терминале последовательно выполнить следующие команды от имени администратора:

```
apt update
apt install astra-csp-cryptopro
apt install fly-csp-cryptopro
apt install libreoffice-astracsp-plugin
```

При установке расширения `libreoffice-astracsp-plugin` проводится проверка наличия в информационной системе установленного программного модуля `fly-csp-cryptopro` и пакета офисных программ LibreOffice.

При запуске программных модулей `fly-csp-cryptopro` и `astra-csp-cryptopro` проводится проверка наличия в информационной системе установленного программного обеспечения СКЗИ и пакета офисных программ LibreOffice.

Дополнительно возможно установить расширение `fly-csp-preview` (описание расширения приведено в 10.5.1), выполнив в терминале команду от имени администратора:

```
apt install fly-csp-preview
```

Программные модули `astra-csp-cryptopro` и `fly-csp-cryptopro` разработаны на языке программирования Python версии 3 и включают в свой состав:

- 1) основной файл программы `/usr/bin/astra-csp-cryptopro`;
- 2) файл графической программы `/usr/bin/fly-csp-cryptopro`;
- 3) дополнительные файлы и библиотеки в каталогах `/usr/lib/python3/dist-packages/astra_csp_cryptopro/` и `/usr/lib/python3/dist-packages/astra_csp-0.1.0.egg-info/`;
- 4) файлы с указанием значений параметров СЭП, задаваемых по умолчанию:
 - а) `/usr/share/astra-csp/astra-csp.yaml` — общие параметры программных модулей;

- б) `/usr/share/astra-csp/fly-csp.yaml` — параметры программного модуля `fly-csp-cryptopro`;
 - в) `/usr/share/astra-csp/cryptopro/astra-csp-cryptopro.yaml` — порядок взаимодействия программного модуля `astra-csp-cryptopro` с СКЗИ;
 - г) `/usr/share/astra-csp/cryptopro/fly-csp-cryptopro.yaml` — порядок взаимодействия программного модуля `fly-csp-cryptopro` с СКЗИ;
- 5) ярлык для запуска `/usr/share/flyfm/actions/fly-csp-cryptopro.desktop`.

Файл `/usr/share/astra-csp/astra-csp.yaml` содержит следующие параметры и их значения по умолчанию:

- 1) максимально допустимый для предоставления СЭП уровень конфиденциальности:

```
max_mac_level: 1
```

- 2) максимальное время ожидания выполнения запросов к СКЗИ:

```
max_csp_time_waiting: 30000
```

- 3) перечень расширений файлов электронных документов, для которых разрешено предоставление СЭП:

```
file_extensions_filter:
```

```
...
```

```
extensions: ['.pdf', '.png', '.jpg', '.html', '.xml', '.txt', '.odt',
'.ods', '.odp', '.odg', '.doc', '.xls', '.ppt', '.docx', '.xlsx',
'.pttx']
```

- 4) отключение (`false`) или включение (`true`) действия перечня, заданного в пункте 3) перечисления, по умолчанию `false`:

```
file_extensions_filter:
```

```
enable: false
```

Установка значения `false` допускается только для целей тестирования и апробации технических решений с применением СЭП в информационной системе.

В файле `/usr/share/astra-csp/fly-csp.yaml` приведен перечень `mime`-типов файлов электронных документов, для просмотра которых будет проведена предварительная конвертация во временный файл формата PDF:

```
pdf_conversion_mime_types:
```

- `application/vnd.oasis.opendocument.text`
- `application/vnd.oasis.opendocument.spreadsheet`
- `application/vnd.oasis.opendocument.presentation`
- `application/vnd.oasis.opendocument.graphics`
- `application/msword`
- `application/vnd.ms-word`
- `application/vnd.ms-excel`
- `application/vnd.ms-powerpoint`

- application/vnd.openxmlformats-officedocument.wordprocessingml.document
- application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
- application/vnd.openxmlformats-officedocument.presentationml.presentation
- text/plain

В случае, если mime-тип выбранного файла электронного документа включен в список, определенный в файле `/usr/share/astra-csp/fly-csp.yaml`, будет проведена предварительная конвертация файла электронного документа в файл формата PDF.

Файл `/usr/share/astra-csp/cryptopro/astra-csp-cryptopro.yaml` содержит параметр, задающий адрес источника доверенного времени:

```
time_address: 'http://testca2012.cryptopro.ru/tsp/tsp.srf'
```

Данный источник допустимо использовать только для выполнения тестирования. При эксплуатации СЭП источник доверенного времени должен быть задан в файле `/etc/astra-csp/astra-csp-cryptopro.yaml` (если файл отсутствует, то необходимо создать его) в качестве значения параметра `time_address`:

```
time_address: <адрес_источника_доверенного_времени>
```

ВНИМАНИЕ! Внесение изменений в любые файлы из состава СЭП, расположенные в каталоге `/usr/`, в т.ч. в `/usr/share/astra-csp/`, запрещено. При необходимости переопределить значения параметров, заданных по умолчанию в файлах в каталоге `/usr/share/astra-csp/`, требуется изменить значения параметров в файлах с аналогичными именами, расположенных в каталоге `/etc/astra-csp/` (если файл с таким же именем отсутствует, то его следует создать). Переопределять значения параметров разрешено только администратору.

ВНИМАНИЕ! Добавление в конфигурационный файл mime-типов, отличных от перечисленных, без проведения дополнительных исследований запрещено.

Для изменения параметров, указанных в файлах `/usr/share/astra-csp/astra-csp.yaml` и `/usr/share/astra-csp/fly-csp.yaml`, необходимо создать файлы `/etc/astra-csp/astra-csp.yaml` и `/etc/astra-csp/fly-csp.yaml`, в которых требуется указать только изменяемые параметры с новыми значениями. Для остальных параметров будут использоваться значения, заданные по умолчанию в файлах `/usr/share/astra-csp/astra-csp.yaml` и `/usr/share/astra-csp/fly-csp.yaml`.

Также возможно изменить значения некоторых параметров, указанных в файлах `/usr/share/astra-csp/astra-csp.yaml` и `/usr/share/astra-csp/fly-csp.yaml`, с использованием графической утилиты `fly-csp-cryptopro` (см. электронную справку). При этом файлы `/etc/astra-csp/astra-csp.yaml` и `/etc/astra-csp/fly-csp.yaml`

с измененными значениями параметров будут созданы автоматически.

10.5.4. Просмотр электронного документа

Просмотр документа при создании и проверке ЭП при работе СЭП осуществляется с использованием программных средств, включенных в состав ОС. Доступен предварительный просмотр электронных документов, хранящихся в файлах следующих форматов:

- 1) PDF — для просмотра используется программа Okular;
- 2) PNG, JPG — для просмотра используется программа gwenview;
- 3) HTML, XML — для просмотра используется браузер из состава ОС, заданный в качестве браузера по умолчанию.

Для предварительного просмотра электронных документов, хранящихся в файлах форматов TXT, ODT, ODS, ODP, ODG, DOC, XLS, PPT, DOCX, XLSX и PPTX, осуществляется их автоматическая предварительная конвертация в формат PDF путем вызова средств из состава пакета офисных программ LibreOffice и отображение пользователю.

Перечень mime-типов файлов, предполагающих для просмотра предварительную конвертацию в формат PDF, задается в конфигурационном файле в соответствии с описанием в 10.5.3 и приведен в таблице 67.

Таблица 67

Мime-тип файла	Описание
text/plain	Текстовый документ TXT
application/vnd.oasis.opendocument.text	Текстовый документ ODT
application/vnd.oasis.opendocument.spreadsheet	Электронная таблица ODS
application/vnd.oasis.opendocument.presentation	Презентация ODP
application/vnd.oasis.opendocument.graphics	Графический документ ODG
application/msword	Текстовый документ DOC
application/vnd.ms-word	Текстовый документ DOC
application/vnd.ms-excel	Электронная таблица XLS
application/vnd.ms-powerpoint	Презентация PPT
application/vnd.openxmlformats-officedocument.wordprocessingml.document	Текстовый документ DOCX
application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	Электронная таблица XLSX
application/vnd.openxmlformats-officedocument.presentationml.presentation	Презентация PPTX

10.5.5. Проверка уровня конфиденциальности сессии

В целях исключения использования СКЗИ при обработке информации, содержащей сведения, составляющие государственную тайну, перед началом работы выполняется проверка допустимого для предоставления СЭП уровня конфиденциальности путем сравнения уровня конфиденциальности сессии текущего пользователя с максимально допустимым значением, заданным в конфигурационном файле.

Максимально допустимый для предоставления СЭП уровень конфиденциальности задается в конфигурационном файле `/usr/share/astra-csp/astra-csp.yaml` параметром `max_mac_level`, а также в графической утилите `fly-csp-cryptopro` (см. электронную справку). По умолчанию установлено значение `max_mac_level: 1` и может быть изменено администратором безопасности. Политикой управления доступом и организационно-распорядительными документами по защите информации объекта информатизации (объекта вычислительной техники) должен быть предусмотрен явный запрет внесения изменений в конфигурационный файл любыми пользователями, за исключением ответственного за защиту информации администратора безопасности, зарегистрированного в ОС как привилегированный пользователь.

В случае, если уровень конфиденциальности текущей сессии пользователя превышает уровень, заданный в конфигурационном файле, при запуске программы `fly-csp-cryptopro` или `astra-csp-cryptopro` будет выведено сообщение об ошибке и программа завершит свою работу.

11. НАДЕЖНОЕ ФУНКЦИОНИРОВАНИЕ

Основными причинами нарушения процесса функционирования СЗИ ОС являются сбои оборудования, приведшие к различным повреждениям ФС. К таковым относятся: сбои электропитания, повреждения носителей информации (жестких дисков), повреждения соединительных кабелей.

11.1. Восстановление файловой системы

В процессе перезагрузки после сбоя ОС автоматически выполнит программу проверки и восстановления ФС — `fsck`. Если повреждения ФС окажутся незначительными, то ее выполнения достаточно для обеспечения целостности ФС.

В случае обнаружения серьезных повреждений ФС данная программа может предложить перезагрузить компьютер в однопользовательский режим и произвести запуск программы `fsck` вручную. Администратор, контролирующий процесс загрузки ОС, после сбоя должен следовать инструкциям, выдаваемым программой `fsck`. Описание программы приведено на справочной странице `man fsck`.

После завершения загрузки ОС следует проверить целостность файлов с помощью программы контроля целостности. Если в результате проверки найдутся поврежденные или измененные файлы, особенно в каталоге `/etc` и его подкаталогах, то следует восстановить поврежденные файлы из резервной копии. Инструменты создания резервных копий и восстановления из них описаны в документе РУСБ.10015-01 95 01-1, а также в 11.4 и 11.5.

Если сбой привел к выходу из строя жестких дисков, то следует заменить вышедшее из строя оборудование и переустановить ОС с DVD-диска с дистрибутивом, а пользовательские данные восстановить из резервной копии.

Если восстановление ФС не привело к работоспособности ОС или из-за серьезности сбоя нет возможности использовать инструмент восстановления ФС, то возможно запустить ОС в режиме восстановления в соответствии с 11.2.

11.2. Режим восстановления ОС

Режим восстановления — специальный вариант загрузки ОС, предназначенный, в основном, для устранения проблем с загрузкой и входом пользователей. Также может быть использован для изменения размеров дисковых разделов и файловых систем, восстановления поврежденных файлов из резервной копии и иных задач администрирования. В режиме восстановления загружается и работает минимальных набор служб (например, не работают сетевые подключения, аудиоустройства и т. д.). Также в режиме восстановления доступен только консольный вход в ОС. При этом все действия выполняются от имени учетной записи `root`, команда `sudo` для получения прав суперпользователя не используется и не работает.

Войти в режим восстановления можно следующими способами:

- с использованием установочного носителя, например DVD-диска или USB-носителя (см. 11.2.1);
- с использованием загрузчика GRUB 2 (см. 11.2.2).

В ОС можно использовать установочный носитель для загрузки графической сессии LiveCD¹⁾. Этот вариант загрузки не является режимом восстановления, но позволяет использовать графический интерфейс для выполнения действий по администрированию. При этом потребуются использовать команду `sudo`.

11.2.1. Режим восстановления с использованием установочного носителя

Чтобы войти в режим восстановления с использованием установочного носителя, в меню UEFI/BIOS компьютера или в настройках виртуальной машины должна быть включена возможность загрузки с установочного носителя.

Для входа в режим восстановления необходимо:

- 1) начать загрузку с установочного носителя;
- 2) при появлении сообщения «Нажмите C для отображения меню загрузки» нажать клавишу **<C>**;
- 3) дождаться запуска программы установки;
- 4) в окне программы установки нажать **<F10>**, в открывшемся меню выбрать «Выйти в консоль» и нажать **<Enter>**.

После этого будет выполнен переход в режим командной строки под управлением ядра, загруженного с установочного носителя. В командной строке можно выполнить необходимые действия по восстановлению.

Последовательность действий, которые необходимо выполнить для получения доступа к корневой файловой системе в режиме восстановления, представлена в 11.2.3.

Для выхода из режима восстановления:

- 1) извлечь установочный носитель;
- 2) выйти из сессии `chroot`:
`exit`
- 3) перезагрузить компьютер:
`reboot`

¹⁾ ОС, загружаемая с носителя (DVD-диска, USB-носителя) без установки на жесткий диск, при этом доступны определенные функциональные возможности ОС.

4) если режим восстановления более не требуется, то в меню UEFI/BIOS компьютера или в настройках виртуальной машины отключить возможность загрузки с установочного носителя.

11.2.2. Режим восстановления с использованием загрузчика GRUB 2

Чтобы войти в режим восстановления с использованием загрузчика GRUB 2, должно быть включено отображение меню загрузчика GRUB 2 при загрузке системы. Для включения и выключения отображения меню загрузчика GRUB 2 используется инструмент `astra-nobootmenu-control` (см. 16.6.17).

Для входа в режим восстановления необходимо:

- 1) перезагрузить (включить) компьютер;
- 2) при появлении меню загрузчика GRUB 2 нажать клавишу <E>;
- 3) если был установлен пароль загрузчика — ввести имя пользователя и пароль загрузчика (по умолчанию используется имя и пароль администратора, созданного при установке ОС). После этого откроется окно редактора для настройки загрузчика GRUB 2.

Пример

Значения параметров загрузчика GRUB 2, отображаемые в окне редактора:

```
setparams 'Astra GNU/Linux, with Linux 6.1.90-1-generic'
load_video
insmod gzio
if [ x$grub_platform xen ]; then insmod xzio; insmod lzopio; fi
insmod part_gpt
insmod ext2
search --no-floppy --fs-uuid --set-root
0e2c3c97-ae90-4f68-bcdd-8ab2d3993218
echo 'Loading Linux 6.1.90-1-generic...'

linux /boot/vmlinuz-6.1.90-1-generic
root=UUID=0e2c3c97-ae90-4f68-bcdd-8ab2d3993218
ro quiet splash parsec.max_ilev=63

echo 'Loading initial ramdisk...'
initrd /boot/initrd.img-6.1.90-1-generic
```

В приведенном примере в строке, которая начинается со слова `linux`, перечислены значения параметров загружаемого ядра;

- 4) в окне редактора скорректировать строку параметров ядра:
 - а) параметр `ro` (монтировать корневую ФС только в режиме чтения) заменить на параметр `rw` (монтировать корневую ФС в режиме чтения/записи);

б) через пробел добавить параметр `init=/bin/bash` (запустить исполняемый файл `/bin/bash` в качестве процесса `init`).

Пример

Скорректированная строка параметров ядра:

```
linux      /boot/vmlinuz-6.1.90-1-generic  
root=UUID=0e2c3c97-ae90-4f68-bcdd-8ab2d3993218  
rw quiet splash parsec.max_ilev=63 init=/bin/bash
```

5) для загрузки системы с новыми параметрами нажать **<Ctrl+X>** или **<F10>**.

После этого будет смонтирован образ `initrd` и запущена оболочка `/bin/bash`, в которой можно выполнить необходимые действия по восстановлению.

Последовательность действий, которые необходимо выполнить для получения доступа к корневой файловой системе в режиме восстановления, представлена в 11.2.3.

Для выхода из режима восстановления необходимо:

1) для предотвращения возможной потери внесенных изменений выполнить команду записи дисковых кешей на диски:

```
sync
```

2) нажать сочетание клавиш **<Ctrl+Alt+Del>** для перезагрузки или выключить компьютер.

11.2.3. Доступ к корневой файловой системе

Для получения доступа к корневой файловой системе в режиме восстановления необходимо:

1) определить имя раздела, в который была установлена ОС, для этого выполнить команду:

```
blkid
```

На экране монитора отобразится информация о разделах жесткого диска (если в результате ввода команды на экране монитора нет информации о разделах диска, то повреждения слишком серьезны и необходима полная переустановка системы).

Пример

Вывод после выполнения команды `blkid`:

```
/dev/sr0: BLOCK_SIZE="2048" UUID="2024-03-25-20-02-17-00"  
LABEL="Astra Linux" TYPE="iso9660" PTTYPE="dos"  
/dev/loop0: TYPE="squashfs"  
  
/dev/sda2: LABEL="system" UUID=  
"4064eb0c-d416-4bca-8b0c-34e2f1729cad"  
BLOCK_SIZE="4096" TYPE="ext4"
```

```
PARTUUID="edc7b1f6-2aa3-456f -8cad-435cac7d0cs8"
```

```
/dev/sda1: UUID="FBEE-2F9B" BLOCK_SIZE="512" TYPE="vfat"
```

```
PARTUUID="332f2bb3-afce-457b-9c6b-cca8d3cea818"
```

В приведенном примере ОС была установлена в раздел `/dev/sda2`;

2) примонтировать дисковый раздел, содержащий корневую файловую систему. В качестве точки монтирования можно выбрать каталог `/mnt/`.

Пример

Команда монтирования дискового раздела `/dev/sda2`:

```
mount /dev/sda2 /mnt
```

3) примонтировать каталоги, содержащие служебные данные, которые могут потребоваться для выполнения некоторых действий (например, для установки пакетов):

```
mount --bind /dev/pts /mnt/dev/pts
```

```
mount -t sysfs /sys /mnt/sys
```

```
mount -t proc /proc /mnt/proc
```

4) сменить корневой раздел командой:

```
chroot /mnt
```

После этого можно выполнить необходимые действия по восстановлению.

11.3. Средство восстановления

Средство восстановления — набор инструментов для диагностики и восстановления ОС в случае сбоев.

Доступны следующие варианты средства восстановления:

1) стандартный набор инструментов восстановления — содержит базовые инструменты командной строки для очистки пространства на диске, восстановления пакетов, диагностики файловой системы, обновления загрузчика GRUB 2, подключения сетевых интерфейсов. Также стандартный набор инструментов восстановления позволяет перейти в консоль с правами суперпользователя для устранения сбоев вручную;

2) расширенный набор инструментов восстановления — содержит более широкий набор инструментов. Помимо обновления загрузчика GRUB 2, очистки кеша и проверки файловой системы, в наборе доступны инструменты диагностики жесткого диска или SSD, оперативной памяти, просмотр системного лога и восстановление файловой системы из заранее созданных снимков состояния. Установка расширенного набора инструментов восстановления предполагает создание и хранение резервного образа ОС.

11.3.1. Установка средства восстановления

Чтобы обеспечить возможность восстановления ОС, необходимо заранее установить средство восстановления. Сделать это можно несколькими способами:

- 1) в процессе установки ОС — подробное описание процесса установки приведено в документе РУСБ.10015-01 95 01-2;
- 2) в уже установленную ОС — путем установки deb-пакета:
 - а) чтобы установить стандартный набор инструментов восстановления в уже установленную ОС, выполнить команду:

```
sudo apt install friendly-recovery
```

- б) чтобы установить расширенный набор инструментов восстановления в уже установленную ОС, выполнить команду:

```
sudo apt install astra-rescue-base
```

ВНИМАНИЕ! Для установки расширенного набора инструментов восстановления требуется минимум 2 ГБ свободного дискового пространства.

После завершения установки средство восстановления будет доступно в меню загрузчика GRUB 2 (см. 11.3.2).

При установке расширенного набора инструментов восстановления дополнительно будет создан резервный образ `astra_rescue.img` — это минимальный базовый образ ОС с набором инструментов для восстановления. Образ доступен только для чтения, создается один раз и не изменяется в процессе работы ОС. Место хранения образа зависит от способа установки расширенного набора инструментов восстановления:

- 1) если расширенный набор инструментов восстановления устанавливается в процессе установки ОС, то для хранения образа будет создан раздел диска с меткой `astra_rescue` размером 2 ГБ;
- 2) если расширенный набор инструментов восстановления устанавливается в уже установленную ОС, то будет выполнена проверка наличия раздела с меткой `astra_rescue`:
 - а) если раздел с меткой `astra_rescue` существует, то образ будет сохранен там;
 - б) если раздел с меткой `astra_rescue` отсутствует, то образ будет сохранен в корне файловой системы.

11.3.2. Использование средства восстановления

11.3.2.1. Использование стандартного набора инструментов восстановления

Чтобы перейти к использованию стандартного набора инструментов восстановления:

1) в меню загрузчика GRUB 2 выбрать «recovery mode» и нажать **<Enter>**.

ВНИМАНИЕ! Несмотря на то, что в меню GRUB 2 может быть доступен пункт «recovery mode», использование стандартного набора инструментов восстановления возможно только в том случае, если при установке ОС был установлен стандартный набор инструментов восстановления или если был установлен пакет `friendly-recovery` (см. 11.3.1);

2) после того как необходимые модули будут загружены и будет предложено ввести пароль администратора, следует, не вводя пароль, нажать **<Ctrl+D>**. Будет открыто меню стандартного набора инструментов восстановления.

В меню стандартного набора инструментов восстановления доступны следующие инструменты:

1) `resume` — возобновление загрузки в обычном режиме. При выборе этого инструмента будет показано предупреждение о необходимости повторной перезагрузки для корректной работы графических драйверов. Для запуска инструмента подтвердить ознакомление, нажав **[Ok]**. После подтверждения будет выполнена перезагрузка устройства;

2) `resume` — освобождение пространства на диске, удаление устаревших файлов пакетов. При запуске инструмента будут выполнены команды:

```
apt-get clean
```

```
apt-get autoremove
```

Чтобы запустить инструмент, нажать **<Enter>**. Информация о ходе выполнения будет выведена на экран. Чтобы вернуться в меню стандартного набора инструментов восстановления после завершения работы инструмента, нажать **<Enter>**;

3) `dpkg` — проверка и восстановление пакетов. Чтобы запустить инструмент, нажать **<Enter>**. Информация о ходе выполнения будет выведена на экран. Чтобы вернуться в меню стандартного набора инструментов восстановления после завершения работы инструмента, нажать **<Enter>**;

4) `fsck` — диагностика и исправление ошибок файловой системы. При запуске инструмента будет выполнена команда:

```
e2fsck
```

Для запуска инструмента необходимо, чтобы файловая система была примонтирована в режиме «только чтение». Если файловая система примонтирована в другом

режиме, то будет показано диалоговое окно с предложением перезагрузить устройство, в этом случае:

- а) подтвердить ознакомление, нажав **[Ok]**;
- б) выбрать в меню стандартного набора инструментов восстановления «resume» и в появившемся диалоговом окне подтвердить перезагрузку, нажав **[Ok]**;
- в) вернуться в меню стандартного набора инструментов восстановления:
 - в меню загрузчика GRUB 2 выбрать «recovery mode» и нажать **<Enter>**;
 - после того как необходимые модули будут загружены и будет предложено ввести пароль администратора, не вводя пароль, нажать **<Ctrl+D>**;
- г) заново запустить инструмент, выбрав в меню стандартного набора инструментов восстановления «fsck» и нажав **<Enter>**.

Если файловая система примонтирована в режиме «только чтение», то для запуска инструмента достаточно выбрать в меню стандартного набора инструментов восстановления «fsck» и нажать **<Enter>**. Информация о ходе выполнения будет выведена на экран. Чтобы вернуться в меню стандартного набора инструментов восстановления после завершения работы инструмента, нажать **<Enter>**;

5) `grub` — обновление загрузчика GRUB 2 для исправления некорректной загрузки ОС или ошибок в распознавании установленных на устройстве операционных систем. Чтобы запустить инструмент, нажать **<Enter>**, информация о ходе выполнения будет выведена на экран. Чтобы вернуться в меню стандартного набора инструментов восстановления после завершения работы инструмента, нажать **<Enter>**;

6) `network` — включение сетевых интерфейсов для загрузки обновлений из сети интернет. Чтобы запустить инструмент, нажать **<Enter>**, информация о ходе выполнения будет выведена на экран;

7) `root` — переход в консоль с правами суперпользователя для ручной диагностики и исправления ошибок. Для перехода в консоль:

- а) нажать **<Enter>**, на экране будет отображена область консоли;
- б) в области консоли ввести пароль администратора.

Чтобы вернуться в меню стандартного набора инструментов восстановления, нажать **<Ctrl+D>**;

8) `system-summary` — общая информация о характеристиках системы. Для перехода в область просмотра информации о системе нажать **<Enter>**. Навигация по области просмотра осуществляется с помощью клавиш **<↑>**, **<↓>** или **<PageUp>**, **<PageDown>**. Для возврата в меню стандартного набора инструментов восстановления:

- переключить фокус на **[Ok]**, нажав **<Tab>**;
- нажать **<Enter>**.

ВНИМАНИЕ! Для запуска любого из инструментов, кроме `resume`, `root` и `system-summary`, файловая система должна быть примонтирована в режиме «чтение-запись». Если фай-

ловая система примонтирована в другом режиме, то будет показано диалоговое окно с предупреждением о перемонтировании файловой системы в режиме «чтение-запись» и монтировании других файловых систем, указанных в файле `/etc/fstab`. Чтобы запустить инструмент, в диалоговом окне подтвердить перемонтирование, нажав **[Yes]**. Информация о ходе выполнения будет выведена на экран. Чтобы вернуться в меню стандартного набора инструментов восстановления после завершения работы инструмента, нажать **<Enter>**.

Для выхода из меню стандартного набора инструментов восстановления выполнить перезагрузку устройства с помощью инструмента `resume`.

11.3.2.2. Использование расширенного набора инструментов восстановления

Расширенный набор инструментов восстановления доступен из меню загрузчика GRUB 2. Чтобы перейти к использованию расширенного набора инструментов восстановления:

- 1) перейти в меню загрузчика GRUB 2;
- 2) выбрать пункт «Astra Rescue» и нажать **<Enter>**.
Примечание. Пункт «Astra Rescue» доступен в меню GRUB 2, если при установке ОС был установлен расширенный набор инструментов восстановления или если был установлен пакет `astra-rescue-base` (см. 11.3.1);
- 3) ввести логин и пароль администратора. По умолчанию используются имя и пароль администратора, созданного при установке ОС:
 - а) если данные введены верно, то будет выполнен вход в меню расширенного набора инструментов восстановления;
 - б) если данные введены неверно, то произойдет возврат в меню загрузчика.

Примечание. На устройствах с UEFI расширенный набор инструментов восстановления также доступен из меню Boot Manager.

При входе в меню расширенного восстановления автоматически выполняется поиск и монтирование корневого раздела восстанавливаемой ОС:

- 1) если на устройстве обнаружена только одна ОС, к корневому разделу которой не применялось защитное преобразование, то раздел будет примонтирован автоматически;
- 2) если на устройстве обнаружено несколько операционных систем или присутствует корневой раздел с защитным преобразованием, то будет открыто окно «Выбор системы для восстановления»:
 - а) чтобы примонтировать непреобразованный корневой раздел, выбрать нужный пункт в списке обнаруженных операционных систем и нажать **<Enter>**;
 - б) чтобы примонтировать раздел с защитным преобразованием:
 - переключить фокус на список «Зашифрованные разделы», нажав **<Tab>**;
 - перейти к списку «Зашифрованные разделы», нажав **<↓>**;

- выбрать раздел и нажать **<Enter>**;
- в появившемся окне «Разблокировка: <имя_диска>» ввести ключевую фразу защитного преобразования раздела (установка ключевой фразы описана в документе РУСБ.10015-01 95 01-2);
- нажать **<Enter>**.

После монтирования корневого раздела будет автоматически запущена проверка файловой системы с помощью инструмента командной строки `fsdisk`, а также проверка степени заполнения диска. Информация об обнаруженных ошибках будет выведена в диалоговом окне с возможностью перейти к просмотру логов.

Меню инструментов расширенного восстановления содержит следующие элементы:

- 1 — панель «Утилиты»;
- 2 — панель выбора вкладки области просмотра;
- 3 — область просмотра логов или справки;
- 4 — область вывода информации о версии ОС, файловой системе и выбранном инструменте;
- 5 — поле ввода параметров;
- 6 — строка состояния;
- 7 — панель выбора действий.

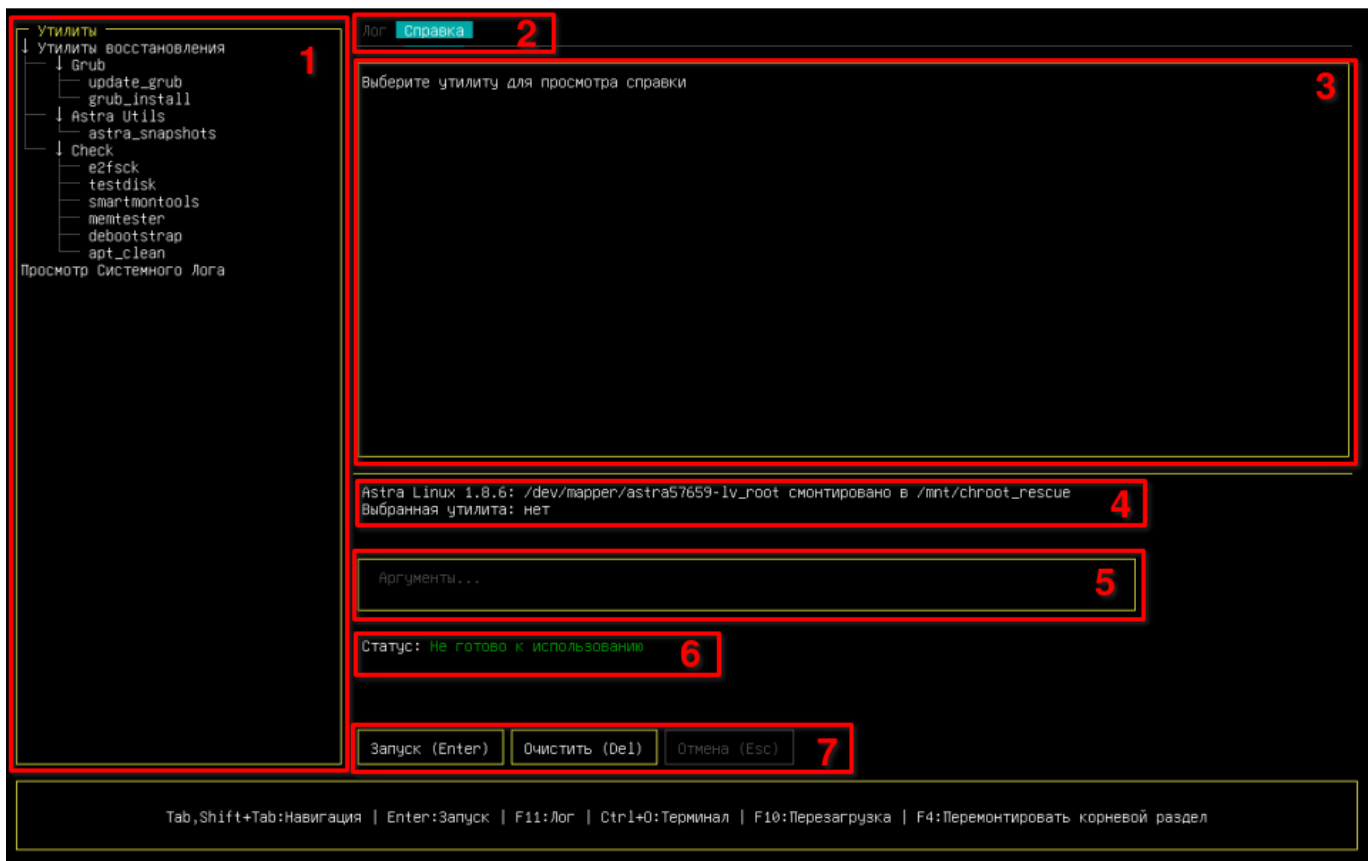


Рис. 6

Навигация по элементам меню осуществляется с помощью следующих клавиш:

- 1) **<Tab>** — последовательное перемещение между элементами интерфейса;
- 2) **<↑>**, **<↓>** — перемещение между элементами списка (если фокус на панели «Утилиты»);
- 3) **<←>**, **<→>** — перемещение между кнопками (если фокус на панели выбора вкладки области просмотра или панели выбора действий);
- 4) **<Enter>** — выбор действия;
- 5) **** — очистка поля ввода параметров;
- 6) **<Esc>** — отмена действия;
- 7) **<F11>** — переход в полноэкранный режим отображения логов;
- 8) **<Ctrl+O>** — переход в терминал режима восстановления. Терминал будет открыт в окружении примонтированного корня;
- 9) **<F10>** — перезагрузка устройства;
- 10) **<F4>** — перемонтирование корневого раздела. При нажатии **<F4>** будет показано диалоговое окно, в котором необходимо подтвердить запуск монтирования. При наличии на устройстве нескольких операционных систем или разделов с защитным преобразованием, потребуется выбрать раздел для монтирования или ввести ключевую фразу, как при входе в меню.

Меню расширенного набора инструментов восстановления предоставляет доступ к следующим инструментам:

- 1) инструменты восстановления загрузчика:
 - а) `update_grub` — инструмент командной строки для обновления конфигурации загрузчика GRUB 2;
 - б) `grub_install` — инструмент командной строки для установки загрузчика GRUB 2;
- 2) `astra_snapshots` — инструмент командной строки для восстановления состояния корневого раздела ОС из снимков состояния. Для восстановления состояния снимок должен быть создан заранее и не переполнен (см. 11.6);
- 3) инструменты диагностики:
 - а) `e2fsck` — инструмент командной строки для проверки файловых систем и исправления ошибок, если они были обнаружены;
 - б) `smartmontools` — инструмент командной строки для диагностики состояния жесткого диска или SSD;
 - в) `memtester` — инструмент командной строки для проверки целостности свободной области оперативной памяти;
 - г) `debootstrap` — инструмент командной строки для восстановления базовой конфигурации операционной системы;

д) `apt_clean` — инструмент командной строки для очистки кеша и освобождения места на диске. При запуске инструмента будет выполнена команда:

```
apt-get clean
```

4) просмотр системного лога — инструмент командной строки для отображения системных логов.

Чтобы посмотреть справку по инструменту:

- 1) при необходимости перевести фокус на панель «Утилиты» нажатием **<Tab>**;
- 2) на панели «Утилиты» выбрать инструмент нажатием **<↑>** или **<↓>**;
- 3) перейти к панели выбора вкладки области просмотра, нажав **<Tab>**;
- 4) выбрать вкладку «Справка» нажатием **<←>** или **<→>**.

На вкладке «Справка» области просмотра указаны параметры инструмента по умолчанию.

При выборе инструмента на панели «Утилиты» и нажатии **<Enter>** инструмент будет запущен с параметрами по умолчанию.

Чтобы запустить инструмент с пользовательскими параметрами:

- 1) при необходимости перевести фокус на панель «Утилиты» нажатием **<Tab>**;
- 2) на панели «Утилиты» выбрать инструмент нажатием **<↑>** или **<↓>**;
- 3) перейти в поле ввода параметров, трижды нажав **<Tab>**;
- 4) ввести параметры и нажать **<Enter>**.

Информация о ходе выполнения и результатах работы инструмента будет выведена на вкладке «Лог» области просмотра. Для прокрутки текста перевести фокус на область просмотра и использовать клавиши **<↑>**, **<↓>**, **<←>**, **<→>**.

Для выхода из меню расширенного набора инструментов восстановления нажать **<F10>** и подтвердить перезагрузку устройства, нажав **[Принять]** в появившемся диалоговом окне.

11.4. Инструмент командной строки `tar`

ВНИМАНИЕ! Работа с мандатными атрибутами и атрибутами аудита при использовании различных инструментов командной строки для создания резервных копий требует использования параметров сохранения расширенных атрибутов (как правило вида `--xattrs`).

ВНИМАНИЕ! Перед восстановлением мандатных атрибутов файлов из резервных копий необходимо от имени учетной записи администратора выполнить команду:

```
echo 1 | sudo tee /parsecfs/unsecure_setxattr
```

ВНИМАНИЕ! Для восстановления мандатных атрибутов файлов из резервных копий процесс должен иметь PARSEC-привилегию 0x1000 (PARSEC_CAP_UNSAFE_SETXATTR). Привилегия может быть получена с использованием инструмента командной строки `execaps`:

```
sudo execaps -c 0x1000 -- tar
```

ВНИМАНИЕ! Восстановление расширенных атрибутов файлов с использованием `unsecure_setxattr` возможно только в случае, если атрибуты восстанавливаются с помощью системного вызова `setxattr` путем установки атрибута `security.PDPL`. Использование `unsecure_setxattr` не влияет на возможность изменения мандатных атрибутов файлов системными вызовами `pdpl_set_path`, `pdpl_set_fd`.

После восстановления из резервных копий файлов с мандатными атрибутами необходимо выполнить команду:

```
echo 0 | sudo tee /parsecfs/unsecure_setxattr
```

Пример

Использование `tar`, при котором будут восстановлены расширенные атрибуты каталогов и файлов, вложенных в архив.

До использования утилиты должен быть создан пользователь `user1`, для которого заданы мандатные атрибуты, и пользователь должен выполнить вход в систему.

Создать от имени учетной записи администратора архив домашнего каталога пользователя с помощью команды:

```
sudo tar --xattrs --acls -cvzf /opt/home.tgz /home/.pdp/user1
```

где `--xattrs` — включает поддержку расширенных атрибутов;

`--acls` — включает поддержку POSIX ACL;

`-cvzf` — обеспечивают, соответственно, создание архива (`create`), включение режима отображения обрабатываемых файлов (`verboze`), применение метода сжатия (`gzip`), указание файла (`file`);

`/opt/home.tgz` — задает место расположения созданного архива и его имя;

`/home/.pdp/user1` — определяет каталоги или файлы для добавления в архив.

Перед восстановлением необходимо выполнить следующую команду, устанавливающую для параметра защиты файловой системы `/parsecfs/unsecure_setxattr` значение «1», разрешающее применять привилегию `PARSEC_CAP_UNSAFE_SETXATTR`:

```
echo 1 | sudo tee /parsecfs/unsecure_setxattr
```

Выполнить восстановление с помощью команды:

```
sudo execaps -c 0x1000 -- tar --xattrs \  
  --xattrs-include=security.{PDPL,AUDIT,DEF_AUDIT} \  
  --acls -xvf /opt/home.tgz -C /opt/home2/
```

где `--xattrs-include=security.PDPL,AUDIT,DEF_AUDIT` — определяет подключаемый шаблон восстановления расширенных атрибутов (мандатных атрибутов, атрибутов аудита и атрибутов аудита по умолчанию) для параметра `xattrs`;

`-xvf` — обеспечивает, соответственно, извлечение из архива (`extract`), включение режима отображения обрабатываемых файлов (`verbose`), указание файла (`file`).

После восстановления необходимо выполнить команду, устанавливающую запрет на применение привилегии `PARSEC_CAP_UNSAFE_SETXATTR` (настройка ОС по умолчанию):

```
echo 0 | sudo tee /parsecfs/unsecure_setxattr
```

Дополнительная информация по команде `tar` приведена в РУСБ.10015-01 95 01-1.

11.5. Комплекс программ Bacula

Bacula обеспечивает поддержку сохранения расширенных атрибутов каталогов и файлов и, при необходимости, их последующее восстановление.

Описание комплекса программ Bacula, а также его установки и настройки приведено в документе РУСБ.10015-01 95 01-1.

ВНИМАНИЕ! Для восстановления объектов ФС с установленными мандатными атрибутами необходимо запустить консоль управления Bacula с `PARSEC`-привилегией `0x1000` (`PARSEC_CAP_UNSAFE_SETXATTR`). Привилегия может быть получена с использованием утилиты `execaps`:

```
sudo execaps -c 0x1000 -- bconsole
```

ВНИМАНИЕ! После восстановления объектов ФС с установленными мандатными атрибутами необходимо выполнить перемонтирование ФС, в которой восстанавливались объекты, или перезагрузить ОС.

11.6. Снимки состояния ОС

11.6.1. Общие сведения

В ОС доступно создание снимков состояния корневого раздела ОС (снапшоты) и восстановление состояния ОС из созданных снимков. Создание снимков возможно, если при разметке диска была создана группа томов LVM и в этой группе томов было выделено свободное место. В свободном месте группы томов LVM будут создаваться снимки состояния. Описание создания группы томов LVM приведено в документе РУСБ.10015-01 95 01-2.

Снимок состояния представляет собой том LVM, в который записываются изменения в корневом разделе, выполненные после создания снимка. При создании в снимке нет данных. В дальнейшем любые изменения файлов и каталогов в корневом разделе ОС записываются в снимок. При повторном изменении одних и тех же файлов и каталогов эти файлы и каталоги перезаписываются в снимке. Если создать несколько снимков, то последующие изменения будут записываться одновременно во все снимки.

Размер снимка указывается пользователем при его создании и может быть впоследствии изменен. Максимальный размер снимка ограничен размером свободного места в группе томов LVM и не может превышать размер корневого раздела ОС.

При восстановлении корневого раздела ОС из снимка выполняется откат записанных в него изменений и корневой раздел приводится к состоянию на момент создания снимка.

Если размер снимка меньше размера корневого раздела, то когда размер изменений в корневом разделе превысит размер снимка, снимок переполнится и станет непригодным для восстановления. Поэтому рекомендуется указывать размер снимка, равный размеру корневого раздела, или постоянно следить за заполнением снимка и увеличивать его размер при заполнении более чем на 95%.

ВНИМАНИЕ! Если запись изменений в снимок невозможна из-за его переполнения, то изменения корневого раздела ОС перестанут отслеживаться, а снимок станет непригодным для восстановления.

11.6.2. Управление снимками

Для создания снимков состояния и восстановления из них используется инструмент командной строки `astra-snapshots`.

ВНИМАНИЕ! При выполнении ручного или автоматического оперативного обновления с созданием снимка состояния системы все созданные ранее снимки удаляются. Описание оперативного обновления с созданием снимка состояния системы приведено в документе РУСБ.10015-01 95 01-2.

Синтаксис команды:

```
sudo astra-snapshots [параметр]
```

Параметры astra-snapshots приведены в таблице 68.

Таблица 68

Параметр	Описание
-s, --show	Вывести список имеющихся снимков с отображением их параметров
-a[размер_в_ГБ], --ability[=размер_в_ГБ]	Проверить возможность создания снимка указанного размера. Если размер не указан, то будет проверена возможность создания снимка по размеру корневого раздела ОС
-c[размер_в_ГБ], --create[=размер_в_ГБ]	Создать снимок указанного размера. Если размер не указан, то будет создан снимок по размеру корневого раздела ОС. Имя снимку присваивается автоматически. Имя снимка содержит имя корневого раздела в группе томов LVM
-m<группа_томов>/<имя_снимка>, --merge=<группа_томов>/<имя_снимка>	Восстановить корневой раздел ОС из снимка с указанным именем в указанной группе томов. После перезагрузки ОС снимок будет применен, а затем удален. При этом этот снимок будет отображаться в квадратных скобках в списке снимков и будет удален из списка после следующей перезагрузки ОС
-d<группа_томов>/<имя>, --delete=<группа_томов>/<имя_снимка>	Удалить снимок с указанным именем в указанной группе томов
-r<группа_томов>/<имя>, --resize=<группа_томов>/<имя_снимка>	Изменить размер снимка с указанным именем в указанной группе томов. После выполнения команды ввести новый размер в гигабайтах или 0 для максимально возможного размера. Для отмены нажать клавишу <Q>
-b, --boot	Вывести объем занятого пространства в загрузочном разделе
-g, --group	Вывести объем свободного места в группе томов LVM
-o, --root	Вывести объем корневого раздела
-l, --live	При выполнении команды с LiveCD попытаться примонтировать корневой и загрузочный разделы установленной ОС в /mnt/oldroot/ и /boot/ соответственно. После успешного монтирования доступно использование снимков состояния ОС, как если бы инструмент был запущен непосредственно из установленной ОС
-h, --help	Вывести справку и выйти

Примеры:

1. Проверить возможность создания снимка по размеру корневого раздела ОС:

```
sudo astra-snapshots -a
```

2. Создать снимок состояния ОС размером 20 ГБ:

```
sudo astra-snapshots -c20
```

3. Отобразить созданные снимки и их параметры:

```
sudo astra-snapshots -s
```

Вывод команды:

```
ГРУППА ТОМОВ ИМЯ СНЭПШОТА ИСХОДНЫЙ ТОМ РАЗМЕР ЗАПОЛНЕНИЕ ВРЕМЯ СОЗДАНИЯ
astra lv_root_snap_1762326790 lv_root 8,0 Гб 19% Ср 05 ноя 2025 10:13:10
astra lv_root_snap_1762327583 lv_root 5,0 Гб 43% Ср 05 ноя 2025 10:26:23
```

4. Удалить снимок:

```
sudo astra-snapshots -dastra/lv_root_snap_1762327583
```

Также для управления снимками состояния может использоваться графическая утилита «Управление снапшотами» (fly-snapshots). Описание утилиты приведено в электронной справке.

11.6.3. Автоматическое управление снимками

Для автоматического управления снимками используется инструмент командной строки `astra-snapshots-manager`.

Автоматическое создание снимков выполняется по одному из выбранных сценариев:

- 1) сценарий «1» — один полноразмерный снимок;
- 2) сценарий «2» — один полноразмерный снимок с пересозданием по окончании срока жизни;
- 3) сценарий «3» — один полноразмерный снимок с функцией создания дополнительного снимка и остановки автоматического управления снимками;

- 4) сценарий «4» — один полноразмерный снимок с пересозданием по окончании срока жизни и функцией создания дополнительного снимка и остановки автоматического управления снимками;
- 5) сценарий «5» — очередь снимков с ротацией по окончании срока жизни;
- 6) сценарий «6» — два снимка с ротацией по окончании срока жизни.

ВНИМАНИЕ! Сценарии автоматизации, использующие два и более снимков, не рекомендуются для низкопроизводительных компьютеров. Одновременное использование нескольких снимков приводит к значительному замедлению работы ОС, так как изменения корневого раздела записываются сразу во все имеющиеся снимки.

ВНИМАНИЕ! При выполнении ручного или автоматического оперативного обновления с созданием снимка состояния системы все созданные ранее снимки удаляются. При выполнении автоматического обновления сценарий автоматизации временно приостанавливается, а при выполнении ручного обновления сценарий автоматизации прерывается. Описание оперативного обновления с созданием снимка состояния системы приведено в документе РУСБ.10015-01 95 01-2.

Более подробное описание сценариев приведено далее по тексту.

ВНИМАНИЕ! При запуске сценария автоматизации все ранее созданные пользователем снимки будут удалены.

Для запуска сценария автоматического создания снимков выполнить:

```
sudo astra-snapshots-manager -s<номер_сценария>
```

Если выбранный сценарий предполагает выбор времени жизни снимка, количество дней для ротации или общее число снимков, то данные будут запрошены в интерактивном режиме.

Пример

Запустить сценарий, создающий очередь снимков с ротацией:

```
sudo astra-snapshots-manager -s5
```

При выполнении команды будут выведены запросы:

Введите период ротации снапшотов в днях (от 1 до 7) или 0 для отмены: 2
Введите количество снапшотов в очереди (от 2 до 6) или 0 для отмены: 3

Для остановки автоматического создания снимков выполнить:

```
sudo astra-snapshots-manager -b
```

Для просмотра информации о статусе автоматизации и выбранном сценарии выполнить:

```
sudo astra-snapshots-manager -i
```

При запущенной автоматизации периодически отслеживается состояние снимков и при необходимости восстанавливается следование выбранному сценарию:

- при удалении снимка пользователем, например с помощью `astra-snapshots`, или автоматически после восстановления корневого раздела снимок будет создан заново после перезагрузки ОС;
- при изменении размера снимка, например с помощью `astra-snapshots`, размер будет восстановлен после перезагрузки ОС;
- при создании дополнительного снимка автоматизация будет остановлена.

Для ручной проверки состояния снимков и следования сценарию выполнить:

```
sudo astra-snapshots-manager -c
```

Для перезапуска остановленной автоматизации выполнить:

```
sudo astra-snapshots-manager -r
```

Если в запущенном сценарии предусмотрено создание дополнительного снимка, то для его создания выполнить:

```
sudo astra-snapshots-manager -t
```

Для удаления всех снимков выполнить:

```
sudo astra-snapshots-manager -p
```

Информация по использованию инструмента доступна по команде:

```
sudo astra-snapshots-manager -h
```

Также для управления автоматизацией создания снимков может использоваться графическая утилита «Управление снапшотами» (`fly-snapshots`). Описание утилиты приведено в электронной справке.

Сценарий 1 — Снимок

При использовании сценария «1» создается один полноразмерный снимок корневого раздела. Сценарий позволяет вернуться к состоянию корневого раздела на момент создания снимка.

Если размер свободного места в группе томов LVM меньше размера корневого раздела, то запуск сценария невозможен.

Сценарий 2 — Снимок с пересозданием

При использовании сценария «2» создается один полноразмерный снимок корневого раздела, который затем пересоздается с указанной периодичностью.

Сценарий позволяет вернуться к более актуальному состоянию корневого раздела, чем при использовании сценария «1». Однако, возможна ситуация, при которой пересоздание нового снимка произойдет после сбоя ОС или во время каких-либо масштабных изменений, например во время обновления. В этих случаях восстановление работоспособного состояния ОС из снимков будет невозможно.

Если размер свободного места в группе томов LVM меньше размера корневого раздела, то запуск сценария невозможен.

Сценарий 3 — Снимок с дополнительным снимком

При использовании сценария «3» создается один основной полноразмерный снимок корневого раздела. При необходимости возможно создать дополнительный снимок меньшего размера в качестве точки восстановления перед выполнением задач, которые могут нарушить нормальную работу ОС, например перед установкой ПО или обновлением.

Сценарий позволяет вернуться к состоянию корневого раздела на момент создания снимка или на момент создания дополнительного снимка.

Если размер свободного места в группе томов LVM меньше размера корневого раздела, то запуск сценария невозможен.

Дополнительный снимок создается размером 5 ГБ и позволяет вернуться к состоянию корневого раздела на момент его создания. После создания дополнительного снимка автоматизация будет остановлена. При возобновлении автоматизации дополнительный снимок будет удален.

Сценарий 4 — Снимок с пересозданием и с дополнительным снимком

При использовании сценария «4» создается один основной полноразмерный снимок корневого раздела, который затем пересоздается с указанной периодичностью. При необходимости возможно создать дополнительный снимок меньшего размера в качестве точки восстановле-

ния перед выполнением задач, которые могут нарушить нормальную работу ОС, например перед установкой ПО или обновлением.

Сценарий позволяет вернуться к более актуальному состоянию корневого раздела, чем при использовании сценария «3», или к состоянию корневого раздела на момент создания дополнительного снимка. Однако, как и в сценарии «2», возможна ситуация, при которой пересоздание нового снимка произойдет после сбоя ОС или во время каких-либо масштабных изменений, например во время обновления. В этих случаях восстановление работоспособного состояния ОС из снимков будет невозможно.

Если размер свободного места в группе томов LVM меньше размера корневого раздела, то запуск сценария невозможен.

Дополнительный снимок создается размером 5 ГБ и позволяет вернуться к состоянию корневого раздела на момент его создания. После создания дополнительного снимка автоматизация будет остановлена. При возобновлении автоматизации дополнительный снимок будет удален.

Сценарий 5 — Очередь снимков с ротацией

При использовании сценария «5» создается указанное количество снимков одинакового размера по одному через указанное число дней ротации. Общий размер всех снимков равен размеру свободного места в группе томов LVM. Сценарий предполагает создание не менее двух снимков. После создания указанного количества снимков каждый последующий снимок создается на месте самого старого снимка. Таким образом, через некоторое время после запуска сценария формируется очередь снимков, позволяющая вернуться к одному из доступных состояний корневого раздела.

В сценарии, при создании второго и последующих снимков, возможно переполнение снимка. Однако, на диске всегда будет присутствовать хотя бы один пригодный для восстановления снимок. Это позволяет вернуться к относительно актуальному состоянию корневого раздела, даже если последний снимок переполнится или будет создан во время масштабных изменений.

Сценарий 6 — Два снимка с ротацией

При использовании сценария «6» создается два снимка через указанное число дней ротации.

Если размер свободного места в группе томов LVM более чем в два раза превышает размер корневого раздела, то оба снимка создаются полноразмерными и сценарий выполняется аналогично сценарию «5» с очередью из двух снимков.

Если размер свободного места в группе томов LVM менее чем в два раза превышает размер корневого раздела, то первый снимок создается полноразмерным, а второй — в размере оставшегося свободного места. При заполнении меньшего снимка более чем на 90% или по

истечении периода ротации выполняется следующее:

- 1) полноразмерный снимок удаляется;
- 2) размер меньшего снимка увеличивается до полноразмерного;
- 3) создается новый снимок в размере оставшегося свободного места.

В отличие от сценария «5», данный сценарий позволяет иметь в очереди полноразмерный снимок при меньшем размере свободного места. Также в этом сценарии невозможно переполнение снимка.

Если размер свободного места в группе томов LVM меньше размера корневого раздела, то запуск сценария невозможен.

11.7. Восстановление в режиме «Мобильный»

В режиме «Мобильный» в случае сбоя ОС можно восстановить ОС с установочного USB-носителя или из образа в формате IMG с сохранением данных в каталогах /opt и /home (см. РУСБ.10015-01 95 01-1).

Также можно сбросить настройки ОС к значениям по умолчанию (на состояние после установки ОС), при этом все данные на устройстве будут удалены.

Для сброса настроек необходимо выполнить команду:

```
sudo astra-mobile-factory-reset
```

Будет выполнен сброс настроек и перезагрузка ОС.

12. ФИЛЬТРАЦИЯ СЕТЕВОГО ПОТОКА

12.1. Включение фильтрации сетевого потока

Фильтрация сетевого потока и контроль сетевого трафика, проходящего через компьютер, осуществляется с помощью фильтра сетевых пакетов.

Запуск фильтрации пакетов обеспечивается межсетевым экраном `ufw`. Межсетевой экран позволяет создавать правила фильтрации используя технологию `iptables`.

Управление межсетевым экраном осуществляется с помощью графической утилиты `gufw` или инструмента командной строки `astra-ufw-control`.

Для включения меж сетевого экрана с помощью инструмента командной строки выполнить команду:

```
sudo astra-ufw-control enable
```

Для выключения меж сетевого экрана с помощью инструмента командной строки выполнить команду:

```
sudo astra-ufw-control disable
```

Проверка состояния меж сетевого экрана выполняется с помощью команды:

```
sudo ufw status
```

Результат выполнения команды:

- `active` — межсетевой экран включен;
- `inactive` — межсетевой экран выключен.

12.2. Фильтр сетевых пакетов `iptables`

Фильтр сетевых пакетов реализован на основе технологии `iptables` и позволяет выполнять следующие задачи:

- 1) фильтрацию пакетов — это механизм, который на основе заданных правил, разрешает или запрещает передачу информации, проходящей через него, с целью ограждения подсети от внешнего доступа, или, наоборот, для недопущения выхода наружу. Фильтр пакетов может определять правомерность передачи информации только на основе заголовков IP-пакетов, а также может анализировать и их содержимое, т. е. использовать данные протоколов более высокого уровня;

- 2) трансляцию сетевых адресов (т. н. «маскарадинг») — это подмена некоторых параметров в заголовках IP-пакетов. Используется для сокрытия реальных IP-адресов компьютеров защищаемой подсети, а также для организации доступа из подсети с компьютерами, не имеющими реальных IP-адресов, к глобальной сети;
- 3) прозрачное проксирование — это переадресация пакетов на другой порт компьютера. Обычно используется для того, чтобы пользователи из подсети использовали проху-сервер маршрутизатора без дополнительного конфигурирования их клиентских программ.

Настройка рассмотренных механизмов (фильтрация пакетов, трансляция сетевых адресов и прозрачное проксирование) выполняется командой `iptables`.

12.3. Формирование правил

Каждое правило — это строка, содержащая критерии, определяющие применимость данного правила к пакету, и действие, которое необходимо выполнить в случае выполнения критерия.

Правила имеют следующий синтаксис:

```
iptables [-t table] command [match] [target/jump]
```

Если в правило не включается спецификатор таблицы `[-t table]`, то по умолчанию предполагается использование таблицы `filter`. Если необходимо использовать другую таблицу, то это требуется указать явно. Спецификатор таблицы можно указывать в любом месте строки правила, однако принято указывать в начале правила.

Непосредственно после имени таблицы должна стоять команда. Если спецификатора таблицы нет, то команда всегда должна стоять первой. Команда определяет действие `iptables`, например, вставить, добавить в конец цепочки или удалить правило.

В разделе `match` указываются критерии проверки, по которым определяется применимость данного правила к пакету. Возможно указать в качестве критерия IP-адрес источника пакета или сети, сетевой интерфейс и т. п.

Параметр `target` указывает, какое действие должно быть выполнено при условии выполнения критериев в правиле. Возможны такие действия как: передача пакета ядром в другую цепочку правил, «сбросить» пакет и забыть про него, выдать на источник сообщение об ошибке и т. п.

Созданные правила фильтрации сохраняются и применяются только до перезагрузки ОС, после перезагрузки ОС требуется повторное формирование правил.

Для обеспечения возможности восстановления правил фильтрации после перезагрузки ОС необходимо:

1) создать файл `/etc/iptables.rules` для сохранения правил фильтрации:

```
sudo touch /etc/iptables.rules
```

2) установить для созданного файла ограничения на чтение, так как из него можно получить информацию об открытых сетевых портах, которая может быть использована для атак на компьютер:

```
sudo chmod 640 /etc/iptables.rules
```

3) сохранить существующие правила фильтрации в файле `/etc/iptables.rules`:

```
sudo iptables-save | sudo tee /etc/iptables.rules > /dev/null
```

4) создать сценарий `/etc/network/if-pre-up.d/iptables` со следующим содержанием:

```
iptables-restore < /etc/iptables.rules  
exit 0
```

5) создать сценарий `/etc/network/if-post-down.d/iptables` со следующим содержанием:

```
touch /etc/iptables.rules  
chmod 640 /etc/iptables.rules  
iptables-save > /etc/iptables.rules  
exit 0
```

6) сделать созданные сценарии исполняемыми:

```
sudo chmod +x /etc/network/if-post-down.d/iptables \  
/etc/network/if-pre-up.d/iptables
```

Примечание. Для сохранения правил фильтрации рекомендуется выполнять пункты 1)–3) перечисления после каждого изменения правил. Сценарий, приведенный в пункте 5) перечисления, сохраняет текущую версию правил автоматически только при штатном выключении компьютера. При нештатном выключении компьютера или сбое изменения в правилах фильтрации могут быть утеряны.

12.4. Порядок прохождения таблиц и цепочек

Когда пакет приходит на сетевой фильтр, то он сначала попадает на сетевое устройство, перехватывается соответствующим драйвером и далее передается в ядро. Затем пакет проходит несколько таблиц, прежде чем он будет передан далее. На каждом этапе пакет может быть остановлен. После прохождения таблиц пакет передается либо локальному

приложению, либо перенаправляется на другой компьютер.

В таблице `filter` цепочку `FORWARD` проходят все пакеты, которые движутся через сетевой фильтр. Порядок следования пакета приведен в таблице 69.

Таблица 69

Шаг	Таблица	Цепочка	Описание
1	=	=	Кабель
2	=	=	Сетевой интерфейс (например, <code>enp0s3</code>)
3	<code>mangle</code>	<code>PREROUTING</code>	Используется для внесения изменений в заголовок пакета, например, для изменения битов <code>TOS</code> и пр.
4	<code>nat</code>	<code>PREROUTING</code>	Используется для трансляции сетевых адресов <code>DNAT</code> . <code>SNAT</code> выполняется позднее, в другой цепочке. Любого рода фильтрация в этой цепочке может производиться только в исключительных случаях
5	=	=	Принятие решения о дальнейшей маршрутизации, т. е. в этой точке решается, куда пойдет пакет — локальному приложению или на другой узел сети
6	<code>filter</code>	<code>FORWARD</code>	Попадают только те пакеты, которые идут на другой компьютер. Вся фильтрация транзитного трафика должна выполняться здесь. Через эту цепочку проходит трафик в обоих направлениях, поэтому обязательно учитывать это обстоятельство при написании правил фильтрации
7	<code>nat</code>	<code>POSTROUTING</code>	Предназначена в первую очередь для <code>SNAT</code> . Не использовать для фильтрации без особой необходимости. Здесь же выполняется и маскардинг
8	=	=	Выходной сетевой интерфейс (например, <code>enp1s3</code>)
9	=	=	Кабель

Пакеты, предназначенные локальному процессу/приложению, проходят через цепочку `INPUT`, а не через `FORWARD`. В таблице 70 представлен порядок движения пакета.

Таблица 70

Шаг	Таблица	Цепочка	Описание
1	=	=	Кабель
2	=	=	Входной сетевой интерфейс (например, <code>enp0s3</code>)
3	<code>mangle</code>	<code>PREROUTING</code>	Обычно используется для внесения изменений в заголовок пакета, например, для установки битов <code>TOS</code> и пр.
4	<code>nat</code>	<code>PREROUTING</code>	Преобразование адресов <code>DNAT</code> . Фильтрация пакетов здесь допускается только в исключительных случаях
5	=	=	Принятие решения о маршрутизации

Окончание таблицы 70

Шаг	Таблица	Цепочка	Описание
6	filter	INPUT	Фильтрация входящего трафика. Все входящие пакеты, адресованные локальному приложению, проходят через эту цепочку, независимо от того, с какого интерфейса они поступили
7	=	=	Локальный процесс/приложение

В таблице 71 представлен порядок движения пакетов, созданных локальными процессами.

Таблица 71

Шаг	Таблица	Цепочка	Описание
1	=	=	Локальный процесс
2	mangle	OUTPUT	Внесение изменений в заголовок пакета. Фильтрация, выполняемая в этой цепочке, может иметь негативные последствия
3	filter		
4	=	=	Принятие решения о маршрутизации. Здесь решается — куда пойдет пакет дальше
5	nat	POSTROUTING	Здесь выполняется SNAT. Не следует в этой цепочке производить фильтрацию пакетов во избежание нежелательных побочных эффектов. Однако и здесь можно останавливать пакеты, применяя политику по умолчанию — DROP
6	=	=	Сетевой интерфейс (например, enp0s3)
7	=	=	Кабель

mangle

В таблице mangle не следует производить фильтрацию, маскировку или преобразование адресов (DNAT, SNAT), в ней допускается выполнять действия, приведенные в таблице 72.

Таблица 72

Действие	Описание
TOS	Выполняет установку битов поля TOS. Это поле используется для назначения сетевой политики обслуживания пакета, т. е. задает желаемый вариант маршрутизации
TTL	Используется для установки значения поля TTL пакета
MARK	Устанавливает специальную метку на пакет, которая затем может быть проверена другими правилами в iptables или другими программами, например, iproute2. С помощью меток можно управлять маршрутизацией пакетов, ограничивать трафик и т. п.

Таблица mangle имеет следующие цепочки:

- PREROUTING — используется для внесения изменений на входе в сетевой фильтр перед принятием решения о маршрутизации;

- INPUT — для пакетов, поступающих в саму систему;
- FORWARD — для изменения пакетов, маршрутизирующихся через систему;
- OUTPUT — для внесения изменений в пакеты, поступающие от приложений внутри сетевого фильтра;
- POSTROUTING — для изменения пакетов после маршрутизации и перед их отправкой.

nat

Только первый пакет из потока проходит через цепочки таблицы nat. Трансляция адресов или маскировка применяются ко всем последующим пакетам в потоке автоматически. Для таблицы nat характерны действия, приведенные в таблице 73.

Таблица 73

Действие	Описание
DNAT	Производит преобразование адресов назначения в заголовках пакетов. Другими словами, этим действием перенаправляются пакеты на другие адреса, отличные от указанных в заголовках пакетов
SNAT	Используется для изменения исходных адресов пакетов. С помощью этого действия можно скрыть структуру локальной сети
MASQUERADE	Применяется в тех же целях, что и SNAT, но в отличие от последней дает более сильную нагрузку на систему. Происходит это потому, что каждый раз, когда требуется выполнение этого действия, производится запрос IP-адреса для указанного в действии сетевого интерфейса, в то время как для SNAT IP-адрес указывается непосредственно. Однако благодаря такому отличию, MASQUERADE может работать в случаях с динамическим IP-адресом

Таблица nat имеет следующие цепочки:

- PREROUTING — используется для внесения изменений в пакеты на входе в сетевой фильтр;
- INPUT — используется для изменения пакетов, предназначенных для локальных сокетов;
- OUTPUT — используется для преобразования пакетов, созданных приложениями внутри сетевого фильтра, перед принятием решения о маршрутизации;
- POSTROUTING — используется для изменения пакетов после маршрутизации и перед их отправкой.

filter

В таблице filter содержатся наборы правил для выполнения фильтрации пакетов. Пакеты могут пропускаться далее либо отвергаться в зависимости от их содержимого.

В таблице `filter` можно выполнить `DROP`, `LOG`, `ACCEPT` или `REJECT`. Таблица `filter` имеет следующие цепочки:

- `INPUT` — используется для прохождения пакетов, которые предназначены локальным приложениям (сетевому фильтру);
- `FORWARD` — используется для фильтрации пакетов, идущих транзитом через сетевой фильтр;
- `OUTPUT` — используется для фильтрации исходящих пакетов, сгенерированных приложениями на самом сетевом фильтре.

12.5. Механизм трассировки соединений

Механизм трассировки соединений является частью сетевого фильтра `iptables` и устроен так, чтобы `netfilter` мог получить информацию о состоянии конкретного соединения. Наличие этого механизма позволяет создавать более надежные наборы правил.

В пределах `iptables` соединение может иметь одно из четырех базовых состояний: `NEW`, `ESTABLISHED`, `RELATED` и `INVALID`. Для управления пакетами на основе их состояния используется критерий `--state`. Трассировщик определяет четыре основных состояния каждого TCP- или UDP-пакета и некоторые дополнительные характеристики. Для TCP- и UDP-пакетов — это IP-адреса отправителя и получателя, порты отправителя и получателя.

Трассировка производится в цепочке `PREROUTING`. Это означает, что `iptables` производит все вычисления, связанные с определением состояния, в пределах этой цепочки. Когда отправляется иницирующий пакет в потоке, то ему присваивается состояние `NEW`, а когда возвращается пакет ответа, то состояние соединения изменяется на `ESTABLISHED` и т. д.

Таблица трассировки

Таблица трассировщика записана в файле `/proc/net/ip_conntrack`. В ней содержится список всех активных соединений.

Если модуль `ip_conntrack` загружен, то команда `cat /proc/net/ip_conntrack` должна вывести:

```
tcp 6 117 SYN_SENT src=192.168.1.6 dst=192.168.1.9 sport=32775 dport=22  
[UNREPLIED] src=192.168.1.9 dst=192.168.1.6 sport=22 dport=32775 use=2
```

В примере вывода команды содержится вся информация, которая известна трассировщику по конкретному соединению.

В начале указано название протокола, в данном случае — `tcp`. Далее следует некоторое число в обычном десятичном представлении. После него следует число, определяющее

«время жизни» записи в таблице (т.е. количество секунд, через которое информация о соединении будет удалена из таблицы). В приведенном примере запись в таблице будет храниться еще 117 с, если через это соединение более не проследует ни одного пакета, в противном случае это значение будет установлено в значение по умолчанию для заданного состояния. Это число уменьшается на 1 каждую секунду. Далее следует фактическое состояние соединения. В примере это состояние имеет значение `SYN_SENT`. Внутреннее представление состояния несколько отличается от внешнего. Значение `SYN_SENT` говорит о том, что через данное соединение проследовал единственный пакет TCP SYN. Далее расположены адреса отправителя и получателя, порты отправителя и получателя. Здесь же указано ключевое слово, которое сообщает о том, что ответного трафика через это соединение еще не было. Далее приводится дополнительная информация по ожидаемому пакету — это IP-адреса и номера портов отправителя и получателя для ожидаемого ответного пакета (те же данные, только поменявшиеся местами).

После получения ответа трассировщик снимет флаг `[UNREPLIED]` и заменит его флагом `[ASSURED]`. Этот флаг сообщает, что соединение установлено, и эта запись не будет стерта по достижении максимально возможного количества трассируемых соединений.

Максимальное количество записей, которое может содержаться в таблице, зависит от значения по умолчанию, которое может быть установлено вызовом функции `ipsysctl`.

Для объема ОЗУ 256 МБ значение соответствует 16376 записям. Можно посмотреть и изменить это значение через:

```
/proc/sys/net/ipv4/ip_conntrack_max
```

Состояния

Сетевые пакеты могут иметь несколько различных состояний в пределах ядра в зависимости от типа протокола. Однако вне ядра имеется только четыре состояния, как было сказано выше. Параметры, описывающие состояние пакета, используются в критерии `--state`. Допустимыми являются: `NEW`, `ESTABLISHED`, `RELATED` и `INVALID`.

В таблице 74 подробно рассмотрено каждое из возможных состояний и приведены необходимые комментарии.

Т а б л и ц а 74

Состояние	Описание
NEW	Сообщает о том, что пакет является первым для данного соединения. Это означает, что это первый пакет в данном соединении, который увидел модуль трассировщика

Окончание таблицы 74

Состояние	Описание
ESTABLISHED	Говорит о том, что это не первый пакет в соединении. Для перехода в состояние ESTABLISHED необходимо, чтобы один компьютер передал пакет и получил на него ответ от другого компьютера. После получения ответа признак соединения NEW будет заменен на ESTABLISHED
RELATED	Появляется, если данное соединение связано с другим соединением, имеющим признак ESTABLISHED, т. е. соединение инициировано из уже установленного соединения, имеющего признак ESTABLISHED
INVALID	Говорит о том, что пакет не может быть идентифицирован и поэтому не может иметь определенного статуса. Это может происходить по разным причинам, например, при нехватке памяти или при получении ICMP-сообщения, которое не соответствует какому-либо известному соединению. Наилучшим вариантом было бы применение действия DROP к таким пакетам

Таблицы

Параметр `-t` указывает на используемую таблицу. По умолчанию используется таблица `filter`.

Команды

В таблице 75 приводится список команд, которые используются в `iptables`, и правила их использования. Посредством данных команд `iptables` узнает, что необходимо выполнить. Обычно предполагается одно из двух действий — это добавление нового правила в цепочку или удаление существующего правила из той или иной таблицы.

Таблица 75

Команда	Использование
<code>-A, --append</code>	Добавляет новое правило в конец заданной цепочки. Пример <pre>iptables -A INPUT ...</pre>

Продолжение таблицы 75

Команда	Использование
-D, --delete	Удаляет правило из цепочки. Команда имеет два формата записи: первый — когда задается критерий сравнения с параметром -D, второй — порядковый номер правила. Если задается критерий сравнения, то удаляется правило, которое имеет в себе этот критерий, если задается номер правила, то будет удалено правило с заданным номером. Счет правил в цепочках начинается с единицы. Пример <pre>iptables -D INPUT --dport 80 -j DROP, iptables \ -D INPUT 1</pre>
-E, --rename-chain	Выполняет переименование пользовательской цепочки. В примере цепочка allowed будет переименована в цепочку disallowed. Эти переименования не изменяют порядок работы. Пример <pre>iptables -E allowed disallowed</pre> Команда должна быть указана всегда
-F, --flush	Сброс (удаление) всех правил из заданной цепочки (таблицы). Если имя цепочки и таблицы не указывается, то удаляются все правила во всех цепочках. Пример <pre>iptables -F INPUT</pre>
-I, --insert	Вставляет новое правило в цепочку. Число, следующее за именем цепочки, указывает номер правила, перед которым следует вставить новое правило. В примере выше указывается, что данное правило должно быть первым в цепочке INPUT. Пример <pre>iptables -I INPUT 1 --dport 80 -j ACCEPT</pre>

Продолжение таблицы 75

Команда	Использование
-L, --list	Вывод списка правил в заданной цепочке. В нижеприведенном примере предполагается вывод правил из цепочки INPUT. Если имя цепочки не указывается, то выводится список правил для всех цепочек. Формат вывода зависит от наличия дополнительных ключей в команде, например -n, -v и др. Пример <pre>iptables -L INPUT</pre>
-N, --new-chain	Создается новая цепочка с заданным именем в заданной таблице. В нижеприведенном примере создается новая цепочка с именем allowed. Имя цепочки должно быть уникальным и не должно совпадать с зарезервированными именами цепочек и действий (DROP, REJECT и т. п.). Пример <pre>iptables -N allowed</pre>
-P, --policy	Определяет политику по умолчанию для заданной цепочки. Политика по умолчанию определяет действие, применяемое к пакетам, не попавшим под действие ни одного из правил в цепочке. В качестве политики по умолчанию допускается использовать DROP, ACCEPT и REJECT. Пример <pre>iptables -P INPUT DROP</pre>
-R, --replace	Данная команда заменяет одно правило другим. В основном она используется во время отладки новых правил. Пример <pre>iptables -R INPUT 1 -s 192.168.0.1 -j</pre>
-X, --delete-chain	Удаление заданной цепочки из заданной таблицы. Удаляемая цепочка не должна иметь правил и не должно быть ссылок из других цепочек на удаляемую цепочку. Если имя цепочки не указано, то будут удалены все цепочки, определенные командой -N в заданной таблице. Пример <pre>iptables -X allowed</pre>

Окончание таблицы 75

Команда	Использование
<code>-Z, --zero</code>	Обнуление всех счетчиков в заданной цепочке. Если имя цепочки не указывается, то подразумеваются все цепочки. При использовании ключа <code>-v</code> совместно с командой <code>-L</code> на вывод будут поданы и состояния счетчиков пакетов, попавших под действие каждого правила. Допускается совместное использование команд <code>-L</code> и <code>-Z</code> . В этом случае будет выдан сначала список правил со счетчиками, а затем произойдет обнуление счетчиков

Ключи

Некоторые команды могут использоваться совместно с дополнительными ключами. Перечень ключей и их описание приведены в таблице 76.

Таблица 76

Ключ	Описание
<code>c, --set-counters</code>	Используется вместе с командами <code>--insert</code> , <code>--append</code> и <code>--replace</code> при создании нового правила для установки счетчиков пакетов и байт в заданное значение. Например, ключ <code>--set-counters 20 4000</code> установит счетчик пакетов в 20, а счетчик байт в 4000
<code>--line-numbers</code>	Используется вместе с командой <code>--list</code> , включает режим вывода номеров строк при отображении списка правил командой <code>--list</code> . Номер строки соответствует позиции правила в цепочке
<code>--modprobe</code>	Может использоваться с любой командой, определяет команду загрузки модуля ядра. Данный ключ используется в случае, если команда <code>modprobe</code> находится вне пути поиска
<code>n, --numeric</code>	Используется вместе с командой <code>--list</code> . Заставляет <code>iptables</code> выводить IP-адреса и номера портов в числовом виде, предотвращая попытки преобразовать их в символические имена
<code>-v, --verbose</code>	Используется вместе с командами <code>--list</code> , <code>--append</code> , <code>--insert</code> , <code>--delete</code> и <code>--replace</code> для повышения информативности вывода. В случае использования с командой <code>--list</code> в вывод этой команды включаются также имя интерфейса, счетчики пакетов и байт для каждого правила. Формат вывода счетчиков предполагает вывод, кроме цифр числа, еще и символьных множителей К (x1000), М (x1,000,000) и G (x1,000,000,000). Чтобы команда <code>--list</code> выводила полное число (без употребления множителей), требуется применять ключ <code>-x</code> , который описан ниже. При использовании с другими командами на вывод будет подан подробный отчет о произведенной операции
<code>-x, --exact</code>	Используется вместе с командой <code>--list</code> . Для всех чисел в выходных данных выводятся их точные значения без округления и без применения множителей К, М, G. Важно то, что данный ключ используется только с командой <code>--list</code> и не применяется с другими командами

12.6. Критерии выделения пакетов

Выделяются следующие критерии:

1) общие — критерии, которые допустимо употреблять в любых правилах. Они не зависят от типа протокола и не требуют подгрузки модулей расширения. В эту группу добавлен критерий `--protocol`, несмотря на то, что он используется в некоторых специфичных от протокола расширениях. Например, при использовании TCP-критерия необходимо использовать и критерий `--protocol`, которому в качестве дополнительного ключа передается название протокола — TCP. Однако `--protocol` сам по себе является критерием, который используется для указания типа протокола;

2) неявные — это критерии, которые подгружаются неявно и становятся доступны, например, при указании критерия `--protocol`. Существует три автоматически подгружаемых расширения: TCP-, UDP- и ICMP-критерии. Загрузка этих расширений может производиться и явным образом с помощью ключа `-m`, `--match`, например:

```
-m tcp
```

3) перед использованием вышеописанных расширений они должны быть загружены явно, с помощью ключа `-m` или `--match`. Так, например, если использовать критерии `--state`, то следует явно указать это в строке правила `-m state` левее используемого критерия. Все отличие между явными и неявными критериями заключается только в том, что первые необходимо подгружать явно, а вторые подгружаются автоматически.

12.7. Действия и переходы

Действия и переходы сообщают правилу, что необходимо выполнить, если пакет соответствует заданному критерию. Чаще всего употребляются действия ACCEPT и DROP.

Описание переходов в правилах выглядит точно так же, как и описание действий, т. е. ставится ключ `-j` и указывается название цепочки правил, на которую выполняется переход. На переходы накладывается ряд ограничений, первое — цепочка, на которую выполняется переход, должна находиться в той же таблице, что и цепочка, из которой этот переход выполняется; второе — цепочка, являющаяся целью перехода, должна быть создана до того, как на нее будут выполняться переходы.

Пример

Создать цепочку `tcp_packets` в таблице `filter` с помощью команды:

```
iptables -N tcp_packets
```

Выполнять переходы на эту цепочку:

```
iptables -A INPUT -p tcp -j tcp_packets
```

т.е., встретив пакет протокола TCP, iptables произведет переход на цепочку `tcp_packets` и продолжит движение пакета по этой цепочке.

Если пакет достиг конца цепочки, то он будет возвращен в вызывающую цепочку (в примере — это цепочка `INPUT`), и движение пакета продолжится с правила, следующего за правилом, вызвавшим переход. Если к пакету во вложенной цепочке будет применено действие `АССЕРТ`, то автоматически пакет будет считаться принятым и в вызывающей цепочке и уже не будет продолжать движение по вызывающим цепочкам. Однако пакет пойдет по другим цепочкам в других таблицах.

Действие — это предопределенная команда, описывающая действие, которое необходимо выполнить, если пакет совпал с заданным критерием. Например, можно применить действие `DROP` или `АССЕРТ` к пакету. В результате выполнения одних действий пакет прекращает свое прохождение по цепочке, например `DROP` и `АССЕРТ`; в результате других, после выполнения неких операций, продолжает проверку, например `LOG`; в результате работы третьих — даже видоизменяется, например `DNAT` и `SNAT`, `TTL` и `TOS`, но также продолжает продвижение по цепочке.

АССЕРТ

Если над пакетом выполняется действие `АССЕРТ`, то пакет прекращает движение по цепочке (и всем вызвавшим цепочкам, если текущая цепочка была вложенной) и считается принятым, тем не менее, пакет продолжит движение по цепочкам в других таблицах и может быть отвергнут там. Действие задается с помощью ключа `-j АССЕРТ`. Дополнительных ключей не имеет.

DNAT

`DNAT` используется для преобразования адреса места назначения в IP-заголовке пакета. Если пакет подпадает под критерий правила, выполняющего `DNAT`, то этот пакет и все последующие пакеты из этого же потока поменяют адрес назначения и будут переданы на требуемое устройство, компьютер или сеть.

Может выполняться только в цепочках `PREROUTING` и `OUTPUT` таблицы `nat` и во вложенных подцепочках.

Для действия `DNAT` предназначен ключ `--to-destination`.

Пример

```
iptables -t nat -A PREROUTING -p tcp -d 15.45.23.67 \  
--dport 80 -j DNAT --to-destination 192.168.1.1-192.168.1.10
```

Этот ключ указывает, какой IP-адрес должен быть подставлен в качестве адреса места назначения. В вышеприведенном примере во всех пакетах, пришедших на адрес 15.45.23.67, адрес назначения будет изменен на один из диапазона от 192.168.1.1 до 192.168.1.10. Все пакеты из одного потока будут направляться на один и тот же адрес, а для каждого нового потока будет выбираться один из адресов в указанном диапазоне случайным образом. Можно также определить единственный IP-адрес. Можно дополнительно указать порт или диапазон портов, на который (которые) будет перенаправлен трафик. Для этого после IP-адреса через двоеточие указать порт, например:

```
--to-destination 192.168.1.1:80
```

для указания диапазона портов:

```
--to-destination 192.168.1.1:80-100
```

Синтаксис действий DNAT и SNAT во многом схож. Указание портов допускается только при работе с протоколом TCP или UDP, при наличии параметра `--protocol` в критерии.

DROP

DROP сбрасывает пакет и iptables забывает о его существовании. Сброшенные пакеты прекращают свое движение полностью, т.е. они не передаются в другие таблицы, как это происходит в случае с действием АССЕРТ. Следует помнить, что данное действие может иметь негативные последствия, поскольку может оставлять незакрытые сокеты как на стороне сервера, так и на стороне клиента, наилучшим способом защиты будет использование действия REJECT, особенно при защите от сканирования портов.

LOG

LOG служит для журналирования отдельных пакетов и событий. В журнал могут заноситься заголовки IP-пакетов и другая интересующая информация. Информация из журнала может быть прочитана с помощью `dmesg` или `syslogd`, либо с помощью других команд.

Ключи действия LOG приведены в таблице 77.

Таблица 77

Ключ	Описание
<code>--log-level</code>	Используется для задания уровня журналирования. Можно задать следующие уровни: <code>debug</code>, <code>info</code>, <code>notice</code>, <code>warning</code>, <code>warn</code>, <code>err</code>, <code>error</code>, <code>crit</code>, <code>alert</code>, <code>emerg</code> и <code>panic</code>. Ключевое слово <code>error</code> означает то же самое, что и <code>err</code>, <code>warn</code> — <code>warning</code> и <code>panic</code> — <code>emerg</code>. Приоритет определяет различия в том, как будут заноситься сообщения в журнал. Все сообщения заносятся в журнал средствами ядра. Если установить строку <code>kern.=info /var/log/iptables</code> в файле <code>syslog.conf</code>, то все сообщения из <code>iptables</code>, использующие уровень <code>info</code>, будут заноситься в файл <code>/var/log/iptables</code>. Однако в этот файл попадут и другие сообщения, поступающие из других подсистем, которые используют уровень <code>info</code>. Пример <pre>iptables -A FORWARD -p tcp -j LOG --log-level debug</pre>
<code>--log-prefix</code>	Задаёт префикс, который будет стоять перед всеми сообщениями <code>iptables</code>. Сообщения со специфичным префиксом затем легко можно найти, к примеру, с помощью <code>grep</code>. Префикс может содержать до 29 символов, включая пробелы. Пример <pre>iptables -A INPUT -p tcp -j LOG --log-prefix \ "INPUT packets"</pre>
<code>--log-tcp-sequence</code>	Позволяет заносить в журнал номер TCP Sequence-пакета. Номер TCP Sequence идентифицирует каждый пакет в потоке и определяет порядок сборки потока. Этот ключ потенциально опасен для безопасности системы, если системный журнал разрешает доступ «на чтение» всем пользователям. Как и любой другой журнал, содержащий сообщения от <code>iptables</code>. Пример <pre>iptables -A INPUT -p tcp -j LOG --log-tcp-sequence</pre>
<code>--log-tcp-options</code>	Позволяет заносить в системный журнал различные сведения из заголовка TCP-пакета. Такая возможность может быть полезна при отладке. Этот ключ не имеет дополнительных параметров. Пример <pre>iptables -A FORWARD -p tcp -j LOG --log-tcp-options</pre>

Окончание таблицы 77

Ключ	Описание
<code>--log-ip-options</code>	Позволяет заносить в системный журнал различные сведения из заголовка IP-пакета. Во многом схож с ключом <code>--log-tcp-options</code>, но работает только с IP-заголовком. Пример <pre>iptables -A FORWARD -p tcp -j LOG --log-ip-options</pre>

MARK

MARK используется для установки меток для определенных пакетов. Это действие может выполняться только в пределах таблицы `mangle`. Установка меток обычно используется для нужд маршрутизации пакетов по различным маршрутам, для ограничения трафика и т. п. Метка пакета существует только в период времени, пока пакет не покинул брандмауэр, т. е. метка не передается по сети. Если необходимо как-то пометить пакеты, чтобы использовать маркировку на другом компьютере, то можно манипулировать битами поля TOS.

Ключ `--set` для действия MARK устанавливает метку на пакет. После ключа `--set-mark` должно следовать целое число.

Пример

```
iptables -t mangle -A PREROUTING -p tcp --dport 22 -j MARK --set-mark 2
```

MASQUERADE

Маскарадинг подразумевает получение IP-адреса от заданного сетевого интерфейса, вместо прямого его указания, как это делается с помощью ключа `--to-source` в действии SNAT.

Действие MASQUERADE может быть использовано вместо SNAT, даже если имеется постоянный IP-адрес.

MASQUERADE допускается указывать только в цепочке POSTROUTING таблицы `nat`, так же как и действие SNAT. MASQUERADE имеет ключ, использование которого необязательно.

Ключ `--to-ports` для действия MASQUERADE используется для указания порта источника или диапазона портов исходящего пакета. Можно указать один порт, например:

```
--to-ports 1025
```


или диапазон портов:

```
--to-ports 1024-3100
```

Этот ключ можно использовать только в правилах, где критерий содержит явное указание на протокол TCP или UDP с помощью ключа `--protocol`.

Пример

```
iptables -t nat -A POSTROUTING -p TCP -j MASQUERADE --to-ports \
1024-31000
```

REDIRECT

REDIRECT выполняет перенаправление пакетов и потоков на другой порт того же самого компьютера. К примеру, можно пакеты, поступающие на HTTP-порт, перенаправить на порт HTTP-прокси. Действие REDIRECT очень удобно для выполнения прозрачного проксирования (transparent proxying), когда компьютеры в ЛВС даже не подозревают о существовании прокси.

REDIRECT может использоваться только в цепочках PREROUTING и OUTPUT таблицы nat, а также выполняться в подцепочках.

Ключ `--to-ports` для действия REDIRECT определяет порт или диапазон портов назначения. Без указания ключа `--to-ports` перенаправления не происходит, т. е. пакет идет на тот порт, куда и был назначен.

Для указания одного порта назначения ввести:

```
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-ports 8080
```

Если необходимо указать диапазон портов:

```
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-ports 8080-8090
```

Этот ключ можно использовать только в правилах, где критерий содержит явное указание на протокол TCP или UDP с помощью ключа `--protocol`.

REJECT

REJECT используется, как правило, в тех же самых ситуациях, что и DROP, но в отличие от DROP, команда REJECT выдает сообщение об ошибке на компьютер, передавший пакет. Действие REJECT работает только в цепочках INPUT, FORWARD и OUTPUT (и во вложенных

в них цепочках). Пока существует только единственный ключ, управляющий поведением команды REJECT.

Ключ `--reject-with` для действия REJECT указывает, какое сообщение необходимо передать в ответ, если пакет совпал с заданным критерием. При применении действия REJECT к пакету сначала на компьютер-отправитель будет отослан указанный ответ, а затем пакет будет сброшен. Допускается использовать следующие типы ответов: `icmp-net-unreachable`, `icmp-host-unreachable`, `icmp-port-unreachable`, `icmp-proto-unreachable`, `icmp-net-prohibited` и `icmp-host-prohibited`. По умолчанию передается сообщение `port-unreachable`. Все вышеуказанные типы ответов являются ICMP error messages (сообщениями об ошибках). Тип ответа `tcp-reset` используется только для протокола TCP. Если указано значение `tcp-reset`, то действие REJECT передаст в ответ пакет TCP RST, который используется для закрытия TCP-соединения.

Пример

```
iptables -A FORWARD -p TCP --dport 22 -j REJECT --reject-with tcp-reset
```

RETURN

RETURN прекращает движение пакета по текущей цепочке правил и производит возврат в вызывающую цепочку, если текущая цепочка была вложенной, или, если текущая цепочка лежит на самом верхнем уровне (например, INPUT), к пакету будет применена политика по умолчанию. В качестве политики по умолчанию назначают действия ACCEPT или DROP.

SNAT

SNAT используется для преобразования сетевых адресов, т.е. изменения исходящего IP-адреса в IP-заголовке пакета. SNAT допускается выполнять только в таблице nat, в цепочке POSTROUTING. Другими словами, только здесь допускается преобразование исходящих адресов. Если первый пакет в соединении подвергся преобразованию исходящего адреса, то все последующие пакеты из этого же соединения будут преобразованы автоматически и не пойдут через эту цепочку правил.

Ключ `--to-source` для действия SNAT используется для указания адреса, присваиваемого пакету.

Пример

```
iptables -t nat -A POSTROUTING -o enp0s3 -j SNAT --to-source \
194.236.50.155-194.236.50.160:1024-32000
```

Указывается IP-адрес, который будет подставлен в заголовок пакета в качестве исходящего. Если необходимо перераспределить нагрузку между несколькими брандмауэрами, то можно

указать диапазон адресов, где начальный и конечный адреса диапазона разделяются дефисом, например:

```
194.236.50.155-194.236.50.160
```

Тогда конкретный IP-адрес будет выбираться из диапазона случайным образом для каждого нового потока. Дополнительно можно указать диапазон портов, которые будут использоваться только для нужд SNAT.

TOS

TOS используется для установки бит в поле TOS IP-заголовка. Поле TOS содержит восемь бит, которые используются для маршрутизации пакетов. Это одно из нескольких полей, используемых `iproute2`. Данное поле может обрабатываться различными маршрутизаторами с целью выбора маршрута движения пакета. Как уже указывалось выше, это поле, в отличие от MARK, сохраняет свое значение при движении по сети, а поэтому может использоваться для маршрутизации пакета. Данное действие допускается выполнять только в пределах таблицы `mangle`.

Ключ `--set-tos` для действия TOS определяет числовое значение в десятичном или шестнадцатеричном виде.

Пример

```
iptables -t mangle -A PREROUTING -p TCP --dport 22 -j TOS --set-tos 0x10
```

Поскольку поле TOS является 8-битным, то можно указать число в диапазоне от 0 до 255 (0x00–0xFF). Большинство значений этого поля никак не используется. Лучше использовать мнемонические обозначения: `Minimize-Delay` (16 или 0x10), `Maximize-Throughput` (8 или 0x08), `Maximize-Reliability` (4 или 0x04), `Minimize-Cost` (2 или 0x02) или `Normal-Service` (0 или 0x00). По умолчанию большинство пакетов имеют признак `Normal-Service` или 0. Список мнемоник можно получить, выполнив команду:

```
iptables -j TOS -h
```

TTL

TTL используется для изменения содержимого поля TTL в IP-заголовке. Один из вариантов применения этого действия — устанавливать значение поля TTL во всех исходящих пакетах в одно и то же значение.

Действие TTL можно указывать только в таблице `mangle`.

Ключи для действия TTL приведены в таблице 78.

Таблица 78

Ключ	Описание
--ttl-set	Устанавливает поле TTL в заданное значение. Оптимальным считается значение около 64. Пример <pre>iptables -t mangle -A PREROUTING -o enp0s3 -j TTL \ --ttl-set 64</pre>
--ttl-dec	Уменьшает значение поля TTL на заданное число. Например, пусть входящий пакет имеет значение TTL, равное 53, выполняется команда --ttl-dec 3. Тогда пакет покинет компьютер с полем TTL, равным 49. Сетевой код автоматически уменьшит значение TTL на 1, поэтому фактически получается: $53 - 3 - 1 = 49$. Пример <pre>iptables -t mangle -A PREROUTING -o enp0s3 -j TTL \ --ttl-dec 1</pre>
--ttl-inc	Увеличивает значение поля TTL на заданное число. Пусть поступает пакет с TTL, равным 53, тогда после выполнения команды --ttl-inc 4 на выходе с компьютера пакет будет иметь TTL, равный 56, не стоит забывать об автоматическом уменьшении поля TTL сетевым кодом ядра, т.е. фактически получается выражение: $53 + 4 - 1 = 56$. Пример <pre>iptables -t mangle -A PREROUTING -o enp0s3 -j TTL \ --ttl-inc 1</pre>

ULOG

ULOG предоставляет возможность журналирования пакетов в пользовательское пространство. Оно заменяет традиционное действие LOG, базирующееся на системном журнале. При использовании этого действия пакет через сокет netlink передается специальной службе, которая может выполнять очень детальное журналирование в различных форматах (например, обычный текстовый файл) и к тому же поддерживает возможность добавления надстроек (плагинов) для формирования различных выходных форматов и обработки сетевых протоколов.

Ключи для действия ULOG приведены в таблице 79.

Таблица 79

Ключ	Описание
--ulog-nlgroup	Сообщает ULOG, в какую группу netlink должен быть передан пакет. Всего существует 32 группы (от 1 до 32). Если необходимо передать пакет в пятую группу, то можно указать: <pre>--ulog-nlgroup 5</pre> По умолчанию используется первая группа. Пример <pre>iptables -A INPUT -p TCP --dport 22 -j ULOG \ --ulog-nlgroup 2</pre>
--ulog-prefix	Имеет тот же смысл, что и аналогичный параметр в действии LOG. Длина строки префикса не должна превышать 32 символа. Пример <pre>iptables -A INPUT -p TCP --dport 22 -j ULOG \ --ulog-prefix "SSH connection attempt: "</pre>
--ulog-cprange	Определяет какую долю пакета в байтах надо передавать демону ULOG. Если указать число 100, как показано в примере, то демону будет передано только 100 Б из пакета, это означает, что демону будет передан заголовок пакета и некоторая часть области данных пакета. Если указать 0, то будет передан весь пакет, независимо от его размера. Значение по умолчанию равно 0. Пример <pre>iptables -A INPUT -p TCP --dport 22 -j ULOG \ --ulog-cprange 100</pre>
--ulog-qthreshold	Устанавливает величину буфера в области ядра. Например, если задать величину буфера, равной 10, как в примере, то ядро будет накапливать журналируемые пакеты во внутреннем буфере и передавать в пользовательское пространство группами по 10 пакетов. Пример <pre>iptables -A INPUT -p TCP --dport 22 -j ULOG \ --ulog-qthreshold 10</pre>

12.8. Поддержка фильтрации на основе классификационных меток

12.8.1. Модули iptables для работы с классификационными метками

Поддержка фильтрации на основе классификационных меток реализована с помощью дополнительного модуля `astralabel`, обеспечивающего задание значений классификационной метки с помощью параметров `--maclev` и `--masscat`.

Для работы `iptables` с классификационными метками необходимо установить пакеты:

- `iptables-astralabel` — модули для использования со всеми типами ядер из состава ОС;
- `iptables-astralabel-common` — общие модули.

Для установки пакетов выполнить:

```
sudo apt install iptables-astralabel-common iptables-astralabel
```

Модуль `astralabel` поддерживает стандартный синтаксис командной строки, используемый в `iptables`, а также дополнительные параметры для контроля мандатных атрибутов сетевых пакетов, приведенные в таблице 80.

Таблица 80

Параметр	Описание
<code>-m astralabel</code>	Указание на использование модуля <code>astralabel</code> для обработки сетевого трафика
<code>--maclev <уровень></code> <code>[:<уровень>]</code>	Применение правила к пакетам, имеющим указанный иерархический уровень конфиденциальности. Для задания диапазона уровней конфиденциальности уровни указываются через символ «:» включительно. Пример Не принимать пакеты с иерархическими уровнями конфиденциальности от 1 до 3: <pre>iptables -A INPUT -m astralabel --maclev 1:3 -j \ DROP</pre> Параметр <code>maclev</code> может применяться одновременно с параметром <code>masscat</code> в одном правиле. Результирующий фильтр будет представлять собой объединение фильтров, заданных данными параметрами

Окончание таблицы 80

Параметр	Описание
!	Значение фильтра, задаваемого параметром <code>--maclev</code>, может быть инверсировано с помощью модификатора «!». Пример Не пропускать исходящие пакеты, имеющие ненулевой иерархический уровень конфиденциальности: <pre>iptables -A OUTPUT -m astralabel ! --maclev 0 \ -j DROP</pre> Данное правило будет пропускать исходящие пакеты, имеющие нулевой иерархический уровень конфиденциальности и ненулевые неиерархические категории доступа
<code>--maccat</code> <бит_категории>	Применение правила к пакетам, имеющим указанные неиерархические категории конфиденциальности. Задание диапазонов и инверсирование не поддерживаются. В одном правиле может быть указано несколько параметров <code>maccat</code>, в таком случае правило будет применяться только к пакетам с установленными одновременно всеми указанными категориями (битами). Примечание. Нумерация битов категорий начинается с единицы. Пример Не пропускать исходящие пакеты с установленными одновременно битами категорий 1 и 2: <pre>iptables -A OUTPUT -m astralabel --maccat 1 \ --maccat 2 -j DROP</pre> Параметр <code>maccat</code> может применяться одновременно с параметром <code>maclev</code> в одном правиле. Результирующий фильтр будет представлять собой объединение фильтров, заданных данными параметрами. Пример Не принимать пакеты с установленными битами категорий 1 и 2 и уровнем конфиденциальности 3: <pre>iptables -A INPUT -m astralabel --maclev 3 \ --maccat 1 --maccat 2 -j DROP</pre>

ВНИМАНИЕ! Если параметры для фильтрации пакетов на основе классификационной метки не указаны, то правило применяется ко всем пакетам, имеющим ненулевую классификационную метку. Например, следующее правило запретит все исходящие пакеты, имеющие ненулевую классификационную метку:

```
iptables -A OUTPUT -m astralabel -j DROP
```

Описание модулей iptables для фильтрации пакетов на основе классификационной метки приведено на справочной странице `man iptables-astralabel`.

12.8.2. Использование ufw для работы с классификационными метками

В межсетевом экране ufw включена поддержка работы с классификационными метками по умолчанию. Для управления сетевыми соединениями с учетом классификационных меток в межсетевой экран ufw добавлены параметры `mac`, `maclev` и `maccat`, по действию аналогичные параметрам iptables.

Для того, чтобы данные параметры работали, в системе должны быть установлены в соответствии с 12.8.1 пакеты `iptables-astralabel-common` и `iptables-astralabel`.

Примеры:

1. Запрет отправки на 80 IP-порт пакетов с ненулевой классификационной меткой:

```
ufw deny out 80/tcp mac
```

Аналогично параметрам iptables:

```
-A OUTPUT -p tcp --dport 80 -m astralabel -j DROP
```

2. Запрет отправки на 80 IP-порт (протокол HTTP) сетевых пакетов, имеющих уровень конфиденциальности 2:

```
ufw deny out 80/tcp maclev 2
```

Аналогично параметрам iptables:

```
-A OUTPUT -p tcp --dport 80 -m astralabel --maclev 2 -j DROP
```

3. Запрет отправки на 80 IP-порт (протокол HTTP) сетевых пакетов, имеющих категорию конфиденциальности 3:

```
ufw deny out 80/tcp maccat 3
```

Аналогично параметрам iptables:

```
-A OUTPUT -p tcp --dport 80 -m astralabel --maccat 3 -j DROP
```


Возможность инверсии правил и возможность одновременного указания нескольких категорий в одном правиле текущей реализацией `ufw` не поддерживаются.

Описание `ufw` для фильтрации пакетов на основе классификационной метки приведено в `man ufw`.

12.8.3. Использование Open vSwitch для работы с классификационными метками

Программный коммутатор Open vSwitch (OVS) из состава ОС поддерживает классификационные метки (уровни и категории конфиденциальности) и может использоваться для фильтрации сетевого потока в условиях мандатного управления доступом. Основное описание коммутатора OVS приведено в документе РУСБ.10015-01 95 01-1.

Фильтрация IP-пакетов, которые содержат классификационные метки, сначала выполняется на портах, затем на основе правил OpenFlow коммутатора OVS. Параметры фильтрации задаются путем назначения классификационных меток портам и добавления классификационных меток в правила OpenFlow.

Для назначения порту или указания в правиле OpenFlow уровня конфиденциальности используется параметр `astra_mac_level`:

```
astra_mac_level=<уровень_конфиденциальности>
```

где `<уровень_конфиденциальности>` — десятичное число, соответствующее уровню конфиденциальности.

Для назначения порту или указания в правиле OpenFlow категорий конфиденциальности используется параметр `astra_mac_categories`:

```
astra_mac_categories=<категории_конфиденциальности>
```

где `<категории_конфиденциальности>` — шестнадцатеричное число, битовая маска которого определяет набор категорий конфиденциальности.

Если параметры уровня и/или категорий конфиденциальности отсутствуют, то по умолчанию для них принимается значение 0 — нулевой уровень конфиденциальности и/или без категорий конфиденциальности.

При фильтрации IP-пакет пропускается через порт, если IP-пакету назначена классификационная метка не выше, чем классификационная метка порта. IP-пакет, пройденный через порт, пропускается правилом OpenFlow, если IP-пакету назначена классификационная метка не выше, чем в правиле OpenFlow. Например, если в правиле OpenFlow задана блокировка со значениями параметров `astra_mac_level=2` и `astra_mac_categories=0x03`, то при

применении правила не будут пропущены IP-пакеты, имеющие нулевой, первый или второй уровень конфиденциальности и имеющие категории конфиденциальности, битовая маска которых содержится в битовой маске 0x03.

Примеры:

1. Добавить порт tap0 и назначить ему второй уровень конфиденциальности и категории конфиденциальности, соответствующие битовой маске 0x03:

```
sudo ovs-vsctl add-port br0 tap0 astra_mac_level=2 \
    astra_mac_categories=0x03
```

2. Добавить порт tap1 и назначить ему категории конфиденциальности, соответствующие битовой маске 0x08, по умолчанию будет назначен нулевой уровень конфиденциальности:

```
sudo ovs-vsctl add-port br0 tap1 astra_mac_categories=0x08
```

3. Добавить порт tap3 с нулевым уровнем конфиденциальности и без категорий конфиденциальности:

```
sudo ovs-vsctl add-port br0 tap3
```

4. Для порта tap0 изменить текущий уровень конфиденциальности на третий уровень конфиденциальности (будут изменены только указанные параметры):

```
sudo ovs-vsctl up-port br0 tap0 astra_mac_level=3
```

5. Для порта tap3 изменить текущие уровень и категории конфиденциальности на первый уровень конфиденциальности и категории конфиденциальности, соответствующие битовой маске 0x7 (будут изменены только указанные параметры):

```
sudo ovs-vsctl up-port br0 tap3 astra_mac_level=1 \
    astra_mac_categories=0x7
```

6. Добавить правило блокировки IP-пакетов, которые в своем уровне IP имеют ip_dst=87.250.250.242 и которым назначен третий уровень конфиденциальности и категории конфиденциальности, соответствующие битовой маске 0x200:

```
sudo ovs-ofctl add-flow br0 ip,ip_dst=87.250.250.242,astra_mac_level=3,\
    astra_mac_categories=0x200,actions=drop
```

7. Добавить правило блокировки IP-пакетов с третьим уровнем конфиденциальности и любым набором категорий конфиденциальности:

```
sudo ovs-ofctl add-flow br0 ip,astra_mac_level=3,actions=drop
```

Информация о назначенных портам коммутатора OVS уровнях и категориях конфиденциальности отображается в выводе команды:

```
sudo ovs-vsctl show
```

Пример

```
sudo ovs-vsctl show
```

Вывод команды:

```
aea83ec6-49bd-4b44-aa6b-82967bcd4a5b
Bridge br0
datapath_type: netdev
Port tap1
MAC categories: 0x00000000000000008
Interface tap1
Port tap0
MAC Level: 3
MAC categories: 0x00000000000000003
Interface tap0

Port tap3
MAC Level: 1
MAC categories: 0x00000000000000007
Interface tap3
Port br0
Interface br0
type: internal
ovs_version: "3.0.1"
```

Для получения информации о правилах, в которых заданы уровни и категории конфиденциальности, выполнить команду:

```
sudo ovs-ofctl -F astra dump-flows bridge
```

12.8.4. Использование nftables для работы с классификационными метками

Фильтр сетевых пакетов nftables из состава ОС поддерживает классификационные метки (уровни и категории конфиденциальности) и может использоваться для фильтрации сетевого потока в условиях мандатного управления доступом.

Классификационные метки размещаются в дополнительном поле Опции заголовка IP-пакета. Для фильтрации IP-пакетов в условиях мандатного управления доступом в выражении для отбора пакетов `ip option` используется условие отбора `astra`.

Для управление конфигурацией фильтра сетевых пакетов, включая добавление и удаление таблиц, цепочек и правил, применяется инструмент командной строки `nft`.

Синтаксис команды для добавления правила фильтрации IP-пакетов, которые содержат классификационные метки:

```
sudo nft add rule <семейство_таблиц> <таблица> <цепочка> ip option \
astra <параметры> <действие_или_переход>
```

Описание параметров условия отбора `astra` приведено в таблице 81.

Таблица 81

Параметр	Описание
<code>label <уровень_конфиденциальности> <категории_конфиденциальности></code>	Задать значение классификационной метки. ВНИМАНИЕ! Не допускается использовать параметр <code>label</code> , чтобы задать нулевую классификационную метку. Вместо этого необходимо использовать параметр <code>missing</code>
<code>missing</code>	Задать нулевую классификационную метку
<code>exists</code>	Задать ненулевую классификационную метку

Порядок использования инструмента приведен на справочной странице `man nft`.

Примеры:

1. Добавить правило: не пропускать исходящие пакеты, которые имеют уровень конфиденциальности 2 и нулевой набор категорий конфиденциальности:

```
sudo nft add rule inet filter output ip option astra label 2 0 drop
```

2. Добавить правило: принимать входящие пакеты, которые имеют нулевой уровень конфиденциальности и набор категорий конфиденциальности 0x3 (установлены Категория_1 и Категория_2). А также регистрировать факт поступления таких пакетов в журнале `/var/log/syslog`. При этом каждая запись будет начинаться с

префикса Набор_категорий_3:

```
sudo nft add rule inet filter input ip option astra label 0 3 \  
log prefix \"Набор_категорий_3 \" accept
```

3. Добавить правило: не пропускать исходящие пакеты, в которых для TCP-порта получателя задано значение 80 и которые имеют ненулевую классификационную метку:

```
sudo nft add rule inet filter output tcp dport 80 ip option \  
astra exists drop
```

4. Перенаправление входящих пакетов без классификационных меток на другой порт этого же хоста:

а) создать цепочку с наименованием `before_routing`, которая имеет тип `nat`, приоритет `dstnat` и для которой назначен хук `prerouting` (обработка всех входящих пакетов до принятия решения о маршрутизации):

```
sudo nft add chain inet filter before_routing \  
{ type nat hook prerouting priority dstnat \; }
```

б) в цепочку `before_routing` добавить правило: входящие пакеты, в которых для TCP-порта получателя задано значение 80 и которые не имеют классификационную метку, перенаправлять на TCP-порт 8080:

```
sudo nft add rule inet filter before_routing ip option \  
astra missing tcp dport 80 redirect to 8080
```

13. МАРКИРОВКА ДОКУМЕНТОВ

13.1. Общие сведения

Защищенный комплекс программ печати и маркировки документов обеспечивает маркировку выводимых на печать документов. Маркировка документов осуществляется при включенном в ОС мандатном управлении доступом.

Дополнительно к дискреционному управлению доступом на основе политик сервера печати CUPS используется мандатное управление доступом. Мандатные атрибуты порождаемых объектов в CUPS наследуют метку безопасности, получаемую из сетевого соединения.

Все субъекты доступа (пользователи) сервера печати делятся на две группы: администраторов и обычных пользователей.

Администраторам разрешено выполнять ряд действий по модификации сервера печати (добавление/удаление принтеров, модификация их параметров и т.д.).

ВНИМАНИЕ! Администрирование сервера печати рекомендуется выполнять в сессии с нулевой классификационной меткой. В зависимости от настроек сервера печати может быть необходима высокая категория целостности.

Пользователям разрешается выводить документы на печать, выполнять модификацию и просмотр заданий согласно их мандатному контексту доступа. В таблице 82 приведено разделение операций по группам доступа и типам операций доступа.

Таблица 82

Группа доступа	Чтение	Запись
Административные операции	CUPS-Get-Devices CUPS-Get-PPDs CUPS-Get-PPD CUPS-Get-Policies MAC-Get-Info	Pause-Printer Resume-Printer Set-Printer-Attributes Enable-Printer Disable-Printer Hold-New-Jobs Release-New-Jobs CUPS-Add-Modify-Printer CUPS-Delete-Printer CUPS-Add-Modify-Class CUPS-Delete-Class CUPS-Set-Default CUPS-Create-Local-Printer

Окончание таблицы 82

Группа доступа	Чтение	Запись
Неадминистративные операции	CUPS-Get-Printers Get-Printer-Supported-Values Get-Printer-Attributes CUPS-Get-Default Get-Subscription-Attributes Get-Subscriptions Get-Notifications Get-Jobs Validate-Job Get-Job-Attributes CUPS-Get-Document MAC-Get-Journal	Purge-Jobs Print-Job Cancel-Jobs Create-Job-Subscription Create-Printer-Subscriptions Renew-Subscription Cancel-Subscription Create-Job Send-Document Cancel-Job Hold-Job Release-Job Restart-Job Set-Job-Attributes Cancel-My-Jobs Close-Job Cups-Authenticate-Job CUPS-Move-Jobs MAC-Set-Job-Attributes MAC-Mark-Documet MAC-Lock-Job MAC-Unock-Job

13.2. Настройка печати в условиях мандатного управления доступом

Чтобы обеспечить маркировку документов, необходимо в настройках сервера CUPS и в настройках принтера установить политику `parsec`. Для этого выполнить от имени учетной записи администратора команды:

```
sudo cupctl DefaultPolicy=parsec
sudo lpattr -p <имя_принтера> -s printer-op-policy=parsec
```

Описание инструмента командной строки `cupctl` приведено на справочной странице `man cupctl`.

Установить политику `parsec` для сервера печати и принтера также можно с использованием графической утилиты `fly-admin-printer`, описание утилиты приведено в электронной справке.

Маркировка документа обязательна при формировании задания на печать с ненулевой меткой безопасности. Маркировка документа также может быть выполнена при формировании задания с нулевой меткой безопасности. Чтобы выполнять маркировку для заданий с нулевой меткой безопасности, в настройках сервера печати установить для параметра `MacMarkZeroLabel` значение `on`. Описание параметра `MacMarkZeroLabel` приведено в

таблице 83.

Принтеры и классы являются сущностями-контейнерами, в которые помещаются задания, и обладают атрибутами `mac-printer-mac-max` и `mac-printer-mac-min`.

Атрибут `mac-printer-mac-max` определяет максимальную метку безопасности заданий, которые могут находиться в сущности-контейнере.

Атрибут `mac-printer-mac-min` определяет минимальную метку безопасности заданий, которые могут находиться в сущности-контейнере.

Для разрешения печати на принтере документов пользователей, работающих в сессии с ненулевой меткой безопасности, необходимо для принтера установить допустимый диапазон меток безопасности, выполнив следующие команды:

```
sudo lpattr -p <имя_принтера> -s mac-printer-mac-min=LLabel  
sudo lpattr -p <имя_принтера> -s mac-printer-mac-max=MLabel
```

где `LLabel` — минимальная метка безопасности заданий, которые разрешено печатать;
`MLabel` — максимальная метка безопасности заданий, которые разрешено печатать.

Допустимый диапазон меток безопасности также возможно установить с помощью графической утилиты `fly-admin-printer`, задав во вкладке «МАС» допустимый диапазон уровней конфиденциальности, категорий конфиденциальности и категории целостности.

Если ЕПП не используется, то настройка принтера выполняется от имени администратора через механизм `sudo` или от имени пользователя, входящего в локальную группу администраторов печати (группа `lpadmin`, см. документ РУСБ.10015-01 95 01-1). Если ЕПП используется, то выполняется от имени пользователя, входящего в локальную группу администраторов печати.

Параметры сервера печати CUPS определены в конфигурационном файле `/etc/cups/cupsd.conf`. Редактирование конфигурационного файла сервера печати CUPS выполняется от имени пользователя, входящего в локальную группу администраторов печати с использованием:

- инструмента командной строки `cupsctl`, запускаемого через механизм `sudo`;
- графической утилиты `fly-admin-printer`;
- модуля «Настройка CUPS» графической утилиты `astra-systemsettings` («Параметры системы»);
- web-интерфейса сервера печати CUPS (<http://127.0.0.1:631>).

Для настройки маркировки используются параметры конфигурационного файла `/etc/cups/cupsd.conf`, приведенные в таблице 83.

Таблица 83

Параметр	Значение по умолчанию	Описание
MacAudit	on	Включить регистрацию событий с помощью библиотеки libastraeventse
MacDuplicate	off	Включить дублирование заданий. Копии заданий сохраняются по умолчанию в каталоге /var/spool/cups/parsec в PDF-формате
MacDuplex	off	Режим печати маркера на обратной стороне (при наличии поддержки со стороны принтера двусторонней печати). Возможные значения: • off — отключено. Печать документа в обычном режиме, при котором создаются два задания: задание печати страниц документа с проставленными маркерами и задание печати маркера на обороте последней страницы. При этом последнюю страницу потребуется перевернуть вручную; • marker — автоматическая печать маркера на обороте всех страниц документа; • fonarik — автоматическая печать маркера на обороте последней страницы. Будут созданы два задания на печать: задание печати страниц документа с проставленными маркерами без последней страницы и задание из двух страниц (последняя страница документа и маркер на обороте последней страницы) для автоматической двусторонней печати
MacEnable	on	Включить поддержку меток безопасности в CUPS
MacMarkZeroLabel	off	Выполнять маркировку документов, отправленных на печать из сессии с нулевой меткой безопасности. ВНИМАНИЕ! Для работы данной функции в системе должно быть включено мандатное управление доступом, а на сервере печати должна быть включена поддержка меток безопасности (см. описание параметра MacEnable)
MacEnableFonarik	on	Создавать задание для печати регистрационной информации документа («фонарика» на обороте последнего листа документа)
MacJournal	off	Включить журнал маркировки. По умолчанию журнал записывается в базу данных SQLITE /var/spool/cups/parsec/markings-journal.sqlite
MacConvertToPDF	on	Преобразовывать унаследованные задания в PDF-формат (для устранения проблем с печатью шрифтов на некоторых PostScript-принтерах)

Окончание таблицы 83

Параметр	Значение по умолчанию	Описание
MacHighIntAdmin	off	Требовать высокую метку целостности для операций администрирования (редактирование принтеров, перенос задания на другой принтер и т.д.). Работает только для локальных сокетов
MacSelectFont	off	Запрашивать у пользователя шрифт для маркировки
StripUserDomain	off	Удалять домен из полного доменного имени пользователя (вида <пользователь>@<домен>) при аутентификации в домене FreeIPA, определении членства пользователя в группах, а также при проверке пользователя на соответствие политикам CUPS. По умолчанию имеет значение off, так как служба sssd запрашивает для аутентификации в домене FreeIPA полное доменное имя пользователя (в /etc/sss/sss.conf параметр use_fully_qualified_names имеет значение TRUE по умолчанию). Если параметр use_fully_qualified_names имеет значение FALSE, то служба sssd запрашивает имя пользователя без домена, следовательно параметр StripUserDomain должен иметь значение on

13.3. Настройка маркировки

Настройка маркировки осуществляется редактированием следующих файлов:

- 1) /etc/cups/marker.template — шаблон маркера, описывающий элементы маркировки на первой странице, последующих страницах и на обороте последней страницы. Файл шаблона по умолчанию /usr/share/cups/marker.template устанавливается вместе с пакетом и может использоваться при необходимости вернуть комплекс программ печати и маркировки документов в исходное состояние. Описание параметров шаблона маркера приведено в 13.3.1;
- 2) /etc/cups/cups-marker-vars.conf — конфигурационный файл описания переменных маркировки. Значения этих переменных будут запрошены у пользователя перед маркировкой. Описание файла приведено в 13.3.2. Файл /usr/share/cups/cups-marker-vars.conf.default содержит описание переменных маркировки по умолчанию, устанавливается вместе с пакетом и может использоваться при необходимости вернуть комплекс программ печати и маркировки документов в исходное состояние;
- 3) /usr/share/cups/psmarker/marker.defs — файл описания положения элементов маркера на страницах;

- 4) `/usr/share/cups/fonarik/fonarik.defs` — файл описания положения элементов маркера на обороте последней страницы;
- 5) `/usr/share/cups/fonts/*.t42` — шрифты для маркировки;
- 6) `/usr/share/cups/charsets/utf-8.*`, `/usr/share/cups/charsets/utf-8` — наборы символов для различных шрифтов маркировки.

Редактирование конфигурационных файлов и установка шрифтов возможны только локально на сервере печати через механизм `sudo` или с помощью утилиты `fly-admin-marker` совместно с механизмом `KAuth`.

Редактирование файлов `/usr/share/cups/fonarik/fonarik.defs` и `/usr/share/cups/psmarker/marker.defs` возможно только от имени администратора через механизм `sudo`.

Описание графической утилиты `fly-admin-marker` см. в электронной справке.

13.3.1. Шаблон маркера

Шаблон определяет внешний вид маркеров на страницах, а также содержит параметры, используемые программами `psmarker` и `fonarik` в процессе маркировки.

ВНИМАНИЕ! Для изменения маркера необходимо редактировать файл `/etc/cups/marker.template`. Файл `/usr/share/cups/marker.template` создается при установке пакета и может быть перезаписан в процессе обновления системы.

В шаблоне в начале файла задаются параметры маркировки:

- 1) `fonarik_boundary` — задается число копий, после печати которого будет сменен маркер на обороте последней страницы;
- 2) `replace_underscore` — преобразование символа нижнего подчеркивания «`_`» в пробел в названии уровней и категорий (0 — отключить, 1 — включить);
- 3) `charset` — набор символов маркировки, позволяет изменить шрифт маркировки.

Далее построчно задаются маркеры в виде:

`<страница>:<начертание>:<размер>:<семейство>:<расположение>:<строка_маркировки>`

где `<страница>` — страница, на которой будет располагаться данная строка маркера. Возможные значения:

- `fonarik` — оборот последней страницы, если число копий меньше или равно значению `fonarik_boundary`;
- `fonarik_gt_5` — оборот последней страницы, если число копий больше значения `fonarik_boundary`;

- `first` — первая страница документа;
- `any` — страницы документа, кроме первой и последней;
- `last` — последняя страница документа,

<начертание> — начертание шрифта маркировки. Возможные значения:

- `normal` — нормальное начертание;
- `bold` — жирное начертание;
- `italic` — наклонное начертание;
- `bolditalic` — жирное наклонное начертание,

<размер> — размер шрифта маркировки,

<семейство> — шрифт (не реализовано для строки маркировки, используется шрифт, заданный параметром `charset`),

<расположение> — расположение данной строки на странице. Возможные значения:

- `top-left` — вверху слева;
- `top-center` — вверху по центру;
- `top-right` — вверху справа;
- `bottom-left` — внизу слева;
- `bottom-center` — внизу по центру;
- `bottom-right` — внизу справа;
- `{<отступ_слева>;<отступ_сверху>}` — расположение на заданном расстоянии от левого и верхнего краев страницы, где `<отступ_слева>` — отступ от левого края страницы, `<отступ_сверху>` — отступ от верхнего края страницы. Значения отступов задаются в типографских пунктах;
- `{<отступ_слева>mm;<отступ_сверху>mm}` — расположение на заданном расстоянии от левого и верхнего краев страницы, где `<отступ_слева>` — отступ от левого края страницы, `<отступ_сверху>` — отступ от верхнего края страницы. Значения отступов задаются в миллиметрах,

<строка_маркировки> — строка маркировки, которая будет добавлена на страницу. Может содержать как произвольный текст, так и переменные, вместо которых будут подставлены значения в процессе маркировки. Переменные указываются в фигурных скобках «`{ }`». Переменные бывают следующих типов:

- встроенные переменные — записываются заглавными буквами. Данные переменные заданы в CUPS и программах маркировки `psmarker` и `fonarik`. Их значение подставляется автоматически и не может быть изменено пользователем в процессе маркировки. Список встроенных переменных приведен в таблице 84.

Таблица 84

Переменная	Описание
CURRENT_COPY	Текущая копия документа, отправленная на печать
CURRENT_PAGE	Текущая страница документа
NUM_PAGES	Число страниц документа
LABEL_NAME	Метка конфиденциальности задания в формате Уровень:Категория
LABEL_NAME_FULL	Метка безопасности задания в формате Уровень:Категория:Целостность
LEV_NAME	Уровень конфиденциальности задания. Оставлена для совместимости. Рекомендуется использовать mac-job-mac-level-name
ILEV_NAME	Категория целостности задания. Оставлена для совместимости. Реко- мендуется использовать mac-job-mac-ilevel-name
CAT_NAME	Категория конфиденциальности задания. Оставлена для совместимости. Рекомендуется использовать mac-job-mac-category-name
DATE	Текущая дата в формате ДД.ММ.ГГГГ
TIME	Текущее время в формате ЧЧ:ММ:СС

- атрибуты задания — записываются строчными буквами и должны соответствовать атрибутам переменных маркировки (в т. ч. пользовательских) или стандартным атрибутам (см. 13.3.2);
- сценарии — задаются в виде {SCRIPT:/path/myscript.sh}. При маркировке переменная будет заменена выводом сценария. Сценарий должен выводить одну строку через стандартный поток (stdout) и быть доступным для выполнения от имени учетной записи root;
- атрибуты пользователя FreeIPA — задаются в виде {FREEIPA:sn}, где sn — имя атрибута FreeIPA пользователя, который создал задание. Посмотреть полный список доступных атрибутов можно командой ipa user-show printuser --raw --all.

Настроить шаблон маркировки также возможно в графической утилите fly-admin-marker (см. электронную справку).

Пример

Шаблон /etc/cups/marker.template

```
fonarik_boundary=5
replace_underscore=0
charset=utf-8
```

```
fonarik:normal:12:Arial:top-left:{mac-inv-num}
fonarik:normal:12:Arial:top-left:Экземпляров {copies}
fonarik:normal:12:Arial:top-left:Количество страниц {NUM_PAGES}
fonarik:normal:12:Arial:top-left:{mac-workplace-id}
fonarik:normal:12:Arial:top-left:{mac-distribution}
```

```

fonarik:normal:12:Arial:top-left:Исполнитель {mac-job-originating-
    user-full-name}
fonarik:normal:12:Arial:top-left:Тел. {mac-owner-phone}
fonarik:normal:12:Arial:top-left:Отпечатал {mac-job-user-full-name}
fonarik:normal:12:Arial:top-left:{DATE}

fonarik_gt_5:normal:12:Arial:top-left:Исполнитель
    {mac-job-originating-user-full-name}
fonarik_gt_5:normal:12:Arial:top-left:Тел. {mac-owner-phone}

first:normal:12:Arial:top-right:{LABEL_NAME}
first:normal:12:Arial:top-right:Экз. {CURRENT_COPY}
first:normal:12:Arial:bottom-right:{mac-inv-num}
any:normal:12:Arial:bottom-right:{mac-inv-num}
last:normal:12:Arial:bottom-right:{mac-inv-num}
first:normal:12:Arial:top-center:{CURRENT_PAGE} из {NUM_PAGES}
any:normal:12:Arial:top-center:{CURRENT_PAGE} из {NUM_PAGES}
last:normal:12:Arial:top-center:{CURRENT_PAGE} из {NUM_PAGES}

```

13.3.2. Переменные маркировки

При маркировке документа используются переменные маркировки. Переменные маркировки могут быть основными и пользовательскими и должны быть описаны в конфигурационном файле `/etc/cups/cups-marker-vars.conf`. Значение переменных маркировки запрашивается у пользователя перед маркировкой.

ВНИМАНИЕ! Для описания переменных маркировки необходимо редактировать файл `/etc/cups/cups-marker-vars.conf`. Файл `/usr/share/cups/cups-marker-vars.conf.default` создается при установке пакета и может быть перезаписан в процессе обновления системы.

Переменные маркировки описываются в следующем виде:

```

атрибут:имя_переменной:значение_по_умолчанию:2 #необязательная переменная
атрибут:имя_переменной:значение_по_умолчанию:1 #обычная переменная
атрибут:имя_переменной:значение_по_умолчанию:0 #скрытая переменная

```

Каждой переменной маркировки в конфигурационном файле `/etc/cups/cups-marker-vars.conf` должен соответствовать атрибут.

Основные переменные заданы по умолчанию в программе маркировки и их настраивать не требуется. Также для основных переменных не требуется указывать их имена в конфигурационном файле. Атрибуты основных переменных приведены в таблице 85.

Таблица 85

Атрибут	Описание
mac-distribution	Список рассылки
mac-inv-num	Инвентарный номер
mac-owner-phone	Телефон исполнителя
mac-workplace-id	Идентификатор рабочего места
mac-job-mac-level-reason	Основание для степени секретности (пункт перечня)
mac-job-user-full-name	Полное имя пользователя, выполнившего маркировку (значение пункта «отпечатал»)
mac-job-originating-user-full-name	Полное имя пользователя, создавшего задание (значение пункта «исполнитель»)
mac-marker-font	Шрифт для маркировки

Пользовательские переменные могут быть созданы пользователем и для них в конфигурационном файле необходимо указывать имя.

Примечание. В названиях переменных и значениях по умолчанию в файле `/etc/cups/cups-marker-vars.conf` необходимо использовать экранирование специального символа «:» (двоеточие), используемого для разделения полей, путем замены данного символа на «\:». Кроме того, экранирование используется при подстановке значений встроенных переменных, что учитывается клиентскими программами.

Имя атрибута для пользовательской переменной должно иметь префикс `mac-`. Если атрибут пользовательской переменной задан без данного префикса, то значение переменной маркировки не будет запрашиваться у пользователя.

Для каждой переменной маркировки в файле `/etc/cups/cups-marker-vars.conf` указывается ее тип и значение по умолчанию. Переменная маркировки может иметь следующий тип:

- «1» — обычная переменная. Обязательное для ввода значение. Запрашивается у пользователя. Если значение не введено пользователем, то заполняется значением по умолчанию;
- «2» — необязательная (опциональная) переменная. Запрашивается у пользователя, но при этом не обязательное к вводу значение. Если значение не введено пользователем, то соответствующему атрибуту присваивается значение «не указано»;
- «0» — скрытая переменная. Не запрашивается у пользователя и соответствующему атрибуту присваивается значение по умолчанию.

Значение по умолчанию может быть пустым или произвольным текстом. Также в качестве значения по умолчанию может использоваться одна из служебных переменных, перечень и описание которых приведены в таблице 86, или один из стандартных атрибутов, перечень и описание которых приведены в таблице 87. Значения служебных переменных и стандартных атрибутов присваиваются до маркировки.

Таблица 86

Переменная	Описание
{LABEL_NAME}	Метка конфиденциальности задания в формате уровень:категория
{LABEL_NAME_FULL}	Метка безопасности задания в полном текстовом виде
{LEVEL_NAME}	Уровень конфиденциальности задания. Переменная оставлена для совместимости. Рекомендуется использовать mac-job-mac-level-name
{ILEVEL_NAME}	Категория целостности задания. Переменная оставлена для совместимости. Рекомендуется использовать mac-job-mac-ilevel-name
{CATEGORY_NAME}	Категория конфиденциальности задания. Переменная оставлена для совместимости. Рекомендуется использовать mac-job-mac-category-name
{GECOS_OWNER_NAME}	Полное имя пользователя, создавшего задание (подставляется из GECOS). При отсутствии данных в GECOS используется имя учетной записи пользователя (логин)
{GECOS_REQUESTING_USER_NAME}	Полное имя пользователя, выполняющего маркировку (подставляется из GECOS). При отсутствии данных в GECOS используется имя учетной записи пользователя (логин)
{GECOS_OWNER_PHONE}	Телефон пользователя, создавшего задание (подставляется из GECOS). При отсутствии данных в GECOS заменяется пустой строкой

Таблица 87

Атрибут	Описание
mac-job-marking-required	Определяет, требуется ли маркировка задания
job-name	Имя задания
copies	Число копий
mac-job-derive	Число унаследованных заданий
mac-job-mac-label	Метка безопасности задания
mac-job-marked	Определяет, выполнена ли маркировка
mac-job-origin	Номер исходного задания, к которому относится унаследованное задание
mac-job-user-name	Имя пользователя, выполнившего маркировку

Окончание таблицы 87

Атрибут	Описание
mac-job-mac-level-name	Уровень конфиденциальности задания. Формируются на основе атрибутов сессии пользователя, создавшего задание на печать
mac-job-mac-ilevel-name	Категория целостности задания. Формируются на основе атрибутов сессии пользователя, создавшего задание на печать
mac-job-mac-category-name	Категория конфиденциальности задания. Формируются на основе атрибутов сессии пользователя, создавшего задание на печать
job-originating-user-name	Имя пользователя, создавшего задание на печать

Пример

Файл cups-marker-vars.conf

```
#основные переменные
mac-inv-num:::1
mac-owner-phone::{GECOS_OWNER_PHONE}:1
mac-workplace-id:::1
mac-distribution:::2
mac-job-mac-level-reason:::0
mac-job-originating-user-full-name::{GECOS_OWNER_NAME}:0
mac-job-user-full-name::{GECOS_REQUESTING_USER_NAME}:0
#пользовательские
mac-myvar:Название переменной:значение по умолчанию:0
```

Настроить переменные маркировки также возможно в графической утилите fly-admin-marker (см. электронную справку).

13.3.3. Файлы описания маркера

Для изменения положения маркера, проставляемого на первой и последующих страницах, необходимо в файле /usr/share/cups/psmarker/marker.defs изменить значения параметров (единица измерения каждого параметра — типографский пункт):

- MarkerTopShift — отступ от верхней границы страницы;
- MarkerBottomShift — отступ от нижней границы страницы;
- MarkerLeftShift — отступ от левой границы страницы;
- MarkerRightShift — отступ от правой границы страницы;
- MarkerStringInterval — интервал между строками.

Для изменения положения маркера, проставляемого на обороте последней страницы, необходимо в файле /usr/share/cups/fonarik/fonarik.defs изменить значения парамет-

ров для соответствующего формата и ориентации страницы (единица измерения каждого параметра — типографский пункт):

- `FonarikTopShift` — отступ от верхней границы страницы;
- `FonarikBottomShift` — отступ от нижней границы страницы;
- `FonarikLeftShift` — отступ от левой границы страницы;
- `FonarikRightShift` — отступ от правой границы страницы;
- `FonarikStringInterval` — интервал между строками;
- `A0`, ..., `A5` — печатный формат страницы `A0`, ..., `A5`;
- `L`, `P` — ориентация страницы: `P` — вертикальная, `L` — горизонтальная.

Пример

Содержание файла `/usr/share/cups/fonarik/fonarik.defs`

```
...
A4:P:FonarikTopShift=520.0
A4:P:FonarikBottomShift=20.0
A4:P:FonarikLeftShift=36.0
A4:P:FonarikRightShift=36.0
A4:P:FonarikStringInterval=4.0
...
```

Любые изменения содержания и формата маркера страниц может производить только администратор.

Настроить маркеры также возможно в графической утилите `fly-admin-marker` (см. электронную справку).

13.3.4. Изменение шрифта маркировки

Шрифт маркировки задается файлом сопоставления шрифтов в шаблоне маркировки. Для каждого шрифта имеется свой файл сопоставления. Изменения шрифта маркировки производится путем указания в шаблоне маркировки для параметра `charset` соответствующего требуемому шрифту файла сопоставления, например:

```
charset=utf-8.PTAstraSans
```

ВНИМАНИЕ! Редактировать необходимо шаблон маркировки `/etc/cups/marker.template`. Если данный шаблон отсутствует, то скопировать установленный вместе с пакетом шаблона маркировки `/usr/share/cups/marker.template` в `/etc/cups/marker.template`.

Доступные файлы сопоставления находятся в `/usr/share/cups/charsets`. При выборе несуществующего файла сопоставления используется по умолчанию `utf-8`, что соответствует шрифту PT Astra Serif.

Возможно создать собственный шрифт маркировки на основе имеющегося шрифта в формате TrueType (`*.ttf`). Исходный шрифт должен содержать кириллицу и иметь четыре варианта начертания (нормальное, жирное, наклонное и жирное наклонное), т.е. состоять из четырех файлов с расширением `*.ttf`.

Все четыре файла необходимо преобразовать в формат Type 42 (расширение `*.t42`). Для этого необходимо:

- 1) установить пакет `fontforge`;
- 2) в каталоге с ttf-шрифтами создать сценарий `converter.pe` (если отсутствует) со следующим содержанием:

```
#!/usr/bin/fontforge

i=1
while ( i<$argc )
Open($argv[i])
RenameGlyphs("Adobe Glyph List")
Generate($argv[i]:r + ".t42")
i = i+1
endloop
```

- 3) сделать сценарий исполняемым, например, командой `chmod +x`;
- 4) запустить сценарий `converter.pe` на исполнение:

```
/home/user/converter.pe *.ttf
```

Полученные четыре файла с расширением `*.t42` необходимо скопировать в `/usr/share/cups/fonts`.

Далее создать файл сопоставления для нового шрифта в каталоге `/usr/share/cups/charsets`. Рекомендуется имя файла сопоставления задавать в виде `utf-8.<имя_шрифта>`, например, `utf-8.PTAstraSans`.

Пример

Файл сопоставления для шрифта PT Astra Sans

```
0000 00FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
0100 01FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
0200 02FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
0300 03FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
```

```

PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
0400 04FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
1E00 1EFF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
2000 20FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
2100 21FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
2300 23FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
2400 24FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
2500 25FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42
2600 26FF ltor single PTAstraSans-Regular.t42 PTAstraSans-Bold.t42
PTAstraSans-Italic.t42 PTAstraSans-BoldItalic.t42

```

В каждой строке после `single` находится список из четырех файлов, разделенных пробелом, соответствующих каждому варианту написания в следующем порядке: обычный, жирный, наклонный и жирный наклонный.

Установку и изменение шрифта также можно осуществить в графической утилите `fly-admin-marker` (см. электронную справку).

Примечание. Возможны проблемы с некоторыми PostScript-принтерами из-за особенностей поддержки шрифтов в формате Type 42, поэтому по умолчанию задание на печать преобразуется в формат *.pdf сразу после маркировки. Для отключения данного действия необходимо выполнить команду:

```
sudo cupsctl MacConvertToPDF=Off
```

или в `cupsd.conf` для параметра `MacConvertToPDF` установить значение `Off`, затем перезапустить сервер печати CUPS.

14. КОНТРОЛЬ ПОДКЛЮЧЕНИЯ СЪЕМНЫХ МАШИННЫХ НОСИТЕЛЕЙ ИНФОРМАЦИИ

Съемные машинные носители информации могут рассматриваться относительно ОС с двух точек зрения:

- как блочные или символьные устройства ввода-вывода;
- как блочное устройство, которое может быть смонтировано.

В первом случае устройство представляет собой специальный файловый объект, доступ к которому контролируется механизмом мандатного управления доступом (см. раздел 4), а также дискреционными ПРД (см. раздел 3).

Во втором случае съемный машинный носитель информации содержит в себе образ ФС, которая и хранит данные. Данная ФС может быть смонтирована в заданный каталог и при этом ФС носителя становится частью корневой ФС, представленной в виде поддерева. Доступ к объектам данной ФС также контролируется механизмом мандатного управления доступом и дискреционными ПРД.

Порядок монтирования ФС съемных машинных носителей информации описан в документе РУСБ.10015-01 95 01-1.

ВНИМАНИЕ! Чтобы обеспечить мандатное управление доступом к объектам ФС носителя в соответствии с описанием в разделе 4, гарантированное уничтожение (стирание) информации (см. 9.1), разделы съемных машинных носителей информации должны быть размечены с использованием одной из ФС, поддерживающей расширенные (в т.ч. мандатные) атрибуты файловых объектов (ext2/ext3/ext4/XFS).

В состав ОС входит менеджер устройств `udev`, который обрабатывает события, связанные с изменением состояния физических устройств. Обработка событий осуществляется по заданным правилам. При этом используются следующие правила:

- системные правила, созданные при установке пакета `systemd`;
- правила для доменного пользователя, которые загружаются службой `sssd` после запуска сессии этого доменного пользователя. Правила для доменных пользователей подготавливаются в веб-интерфейсе контроллера домена;
- правила для локального пользователя. Правила для локальных пользователей подготавливаются с помощью средства контроля подключения устройств.

С помощью пользовательских правил (правил для доменного или локального пользователя) обеспечивается мандатное управление доступом и управление дискреционными ПРД к съемным машинным носителям информации, а также управление аудитом успешных и неуспешных событий, связанных с устройством.

Порядок подготовки и применения пользовательских правил представлен в документе РУСБ.10015-01 95 01-1.

В режиме «Мобильный» реализовано управление блокировкой подключаемых устройств с помощью утилиты USBGuard. Порядок настройки этой утилиты описан в документе РУСБ.10015-01 95 01-1.

15. СОПОСТАВЛЕНИЕ ПОЛЬЗОВАТЕЛЯ С УСТРОЙСТВОМ

ОС обеспечивает ввод-вывод информации на запрошенное пользователем устройство как для произвольно используемых им устройств, так и для идентифицированных (при совпадении маркировки)¹⁾.

ОС включает в себя механизм, обеспечивающий надежное сопоставление имени пользователя и его мандатного контекста с владельцем и меткой безопасности, назначенными для устройства. Механизм сопоставления пользователя с устройством обеспечивает при совпадении маркировок носителя и пользователя применение дискреционных и мандатных ПРД.

Описание настройки правил сопоставления устройств с пользователями приведено в документе РУСБ.10015-01 95 01-1.

¹⁾ В режиме «Мобильный» реализовано управление блокировкой подключаемых устройств.

16. ОГРАНИЧЕНИЕ ПРОГРАММНОЙ СРЕДЫ И ФУНКЦИИ БЕЗОПАСНОСТИ

16.1. Замкнутая программная среда

16.1.1. Режимы функционирования

Инструменты из состава ОС предоставляют возможность создавать замкнутую программную среду (ЗПС). Замкнутая программная среда обеспечивает динамический контроль неизменности (целостности) и подлинности файлов. Она предназначена для:

- выявления фактов несанкционированного изменения исполняемых файлов и других файлов (в т.ч. относящихся к КСЗ);
- предотвращения загрузки измененных исполняемых файлов, а также открытия других измененных файлов.

Контроль целостности реализован в невыгружаемом модуле ядра ОС `digsig_verif` и осуществляется на основе проверки:

- встраиваемых цифровых подписей, внедренных в исполняемые файлы и разделяемые библиотеки;
- цифровых подписей в расширенных атрибутах файловой системы;
- отсоединенных цифровых подписей (содержащихся в отдельных файлах).

Цифровые подписи реализованы в соответствии с ГОСТ Р 34.10-2012 (256 бит) и ГОСТ Р 34.10-2001 (256 бит), использование ГОСТ Р 34.10-2001 в ОС сохранено для совместимости.

При включенной ЗПС, в зависимости от выполненных настроек, блокируется доступ к поставленным на контроль файлам с отсутствующими, неверными или отозванными цифровыми подписями.

Результаты неудачных проверок цифровых подписей файлов (подпись неверна или отсутствует) заносятся в метаданные файлов и сохраняются в ядре до перезагрузки ОС. При последующих обращениях к таким файлам повторная проверка их цифровых подписей не выполняется и считается неудачной, даже если файл после первой проверки был подписан верной подписью. Информация о неудачной проверке цифровой подписи файла удаляется в следующих случаях:

- после перезагрузки ОС;
- при удалении файла или его копировании в другой файл с последующим удалением исходного файла:

```
ср <исходный_файл> <новый_файл> && rm <исходный_файл> && \  
mv <новый_файл> <исходный_файл>
```


- при открытии файла на запись даже без его изменения, например командой:

```
echo -n "" >> <имя_файла>
```

При настроенной ЗПС не блокируется использование инструмента `qemu-user-static`, с помощью которого возможно запустить неподписанные исполняемые файлы, предназначенные для другой аппаратной архитектуры. Поэтому при включении ЗПС необходимо дополнительно включать блокировку интерпретаторов, которая блокирует в том числе использование `qemu-*-static` (см. 16.6.7).

Включение ЗПС может быть выполнено в процессе установки ОС путем выбора соответствующего пункта программы установки ОС.

Включение и выключение ЗПС после установки ОС выполняется с помощью инструмента `astra-digsig-control`, описанного в 16.6.28.

16.1.2. Типы подписей и наборы ключей

В ЗПС используются типы цифровых подписей, описанные в таблице 88.

Таблица 88

Тип	Описание
Встраиваемая цифровая подпись	Может быть встроена в исполняемые файлы следующих типов: • бинарные исполняемые файлы и библиотеки формата ELF (встраивается в ELF-секцию файла); • PE-файлы, в том числе формата DLL, для контролируемого запуска Windows-программ в среде Wine (встраивается в конец файла). Открытый ключ, используемый для проверки подписей данного типа, должен быть подписан закрытым ключом изготовителя ОС или закрытым ключом из иерархии подписанных изготовителем ключей
Цифровая подпись в расширенных атрибутах ФС	Может использоваться для любых типов файлов, в том числе для модулей ядра, так как не изменяет структуру файла. Расширенные атрибуты файлов должны поддерживаться ФС. При копировании, архивировании/восстановлении или перемещении файлов необходимо обеспечить сохранение их расширенных атрибутов, например с помощью инструментов резервного копирования (описание указанных инструментов приведено в РУСБ.10015-01 95 01-1)
Отсоединенная цифровая подпись	Может использоваться для любых типов файлов, в том числе для модулей ядра, так как не изменяет структуру файла. Подпись не нарушается при перемещении, копировании или переименовании файлов. Подпись содержится в отдельном файле, создаваемом в каталоге <code>/etc/digsig/external_sig/</code> при подписании. Имя файла подписи содержит путь к подписываемому файлу, при этом в пути символы <code>/</code> и <code>»</code> заменены на <code>@</code>. Например <code>@home@user@.config@settings.conf.sign</code>. Подписанными считаются все идентичные файлы, независимо от их расположения в ФС

Для проверки цифровых подписей используются открытые ключи в формате GPG. Ключи перед проверкой должны быть загружены в ядро. Для постоянного применения ключи должны быть размещены в подкаталогах каталога `/etc/digsig/`. Для временного применения, например в целях тестирования, ключи могут быть динамически загружены через интерфейсы модуля `digsig_verif` (см. 16.1.5.4). Загруженные таким способом ключи действуют до перезагрузки ОС.

В ОС используются следующие наборы ключей:

- 1) основной набор ключей — три предустановленных открытых ключа в каталоге `/etc/digsig/`, используемые для проверки любых типов цифровых подписей:
 - а) `primary_key_2018.gpg` — корневой ключ изготовителя. Используется для проверки других предустановленных ключей. Этот ключ изначально загружен в ядро;
 - б) `build_system_rbt_root_key_2018.gpg` — ключ сборочной системы изготовителя, подписанный корневым ключом изготовителя. Используется для проверки цифровых подписей файлов, входящих в состав ОС. Загружается в ядро после включения режима ЗПС;
 - в) `partners_rbt_root_key_2018.gpg` — партнерский ключ изготовителя, подписанный корневым ключом изготовителя. Используется для проверки цифровых подписей файлов производителей стороннего ПО. Загружается в ядро после включения режима ЗПС;
- 2) дополнительный набор ключей — изначально отсутствующие открытые ключи, которые могут быть добавлены в следующие каталоги:
 - а) `/etc/digsig/keys/` — открытые ключи для проверки всех типов цифровых подписей. Ключи в этом каталоге должны быть подписаны закрытыми ключами изготовителя или ключами, подписанными ключами изготовителя. Данные открытые ключи могут быть организованы в иерархическую структуру каталогов, при этом ключи в дочерних каталогах `/etc/digsig/keys/<имя_каталога>/` могут быть подписаны закрытым ключом изготовителя или ключом, находящимся в каталоге `/etc/digsig/keys/`. Пример размещения открытых ключей, подписанных изготовителем:
 - `/etc/digsig/keys/key1.gpg` — ключ 1, подписанный партнерским ключом изготовителя;
 - `/etc/digsig/keys/key2.gpg` — ключ 2, подписанный партнерским ключом изготовителя;
 - `/etc/digsig/keys/key1/key1-1.gpg` — ключ, подписанный ключом 1;
 - `/etc/digsig/keys/key1/key1-2.gpg` — ключ, подписанный ключом 1;
 - `/etc/digsig/keys/key2/key2-1.gpg` — ключ, подписанный ключом 2;
 - `/etc/digsig/keys/key2/key2-2.gpg` — ключ, подписанный ключом 2;

б) /etc/digsig/xattr_keys/ — открытые ключи для проверки цифровых подписей в расширенных атрибутах ФС. Данные ключи могут быть созданы самостоятельно с помощью инструмента GnuPG (см. 16.1.3). Наличие подписи изготовителя в этих ключах не проверяется;

в) /etc/digsig/certs/ — открытые ключи для проверки всех типов цифровых подписей. Данные ключи могут быть созданы самостоятельно с помощью инструмента GnuPG (см. 16.1.3). Наличие подписи изготовителя в этих ключах не проверяется.

ВНИМАНИЕ! Ответственность за загрузку ключей и доверие к ним несет администратор ОС.

Для добавления и загрузки в ядро новых открытых ключей следует скопировать их в соответствующий каталог и выполнить команду:

```
sudo update-initramfs -u -k all
```

16.1.3. Создание дополнительных ключей

Ключевая пара (открытый и закрытый ключи) должна создаваться на отдельном компьютере. На компьютер с включенной ЗПС должен передаваться только открытый ключ из созданной ключевой пары. Для создания ключей используется GNU Privacy Guard (GnuPG). В доработанном GnuPG доступно использование алгоритма по ГОСТ Р 34.11-2012 для вычисления хеш-функции. Результат вычисления хеш-функции записывается вместе с подписью. Для получения списка доступных алгоритмов следует выполнить команду:

```
gpg --version
```

Результат выполнения команды:

```
gpg (GnuPG) 2.2.40
libgcrypt 1.10.1
Copyright (C) 2022 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Home: /home/user/.gnupg

Поддерживаются следующие алгоритмы:

С открытым ключом: RSA, ELG, DSA, ECDH, ECDSA, EDDSA,
GOST_R34.10-2001, GOST_R34.10-2012

Симметричные шифры: IDEA, 3DES, CAST5, BLOWFISH,
AES, AES192, AES256, TWOFISH, CAMELLIA128,
CAMELLIA192, CAMELLIA256, GRASSHOPPER

Хеш-функции: SHA1, RIPEMD160, SHA256, SHA384, SHA512,
SHA224, GOST_R34.11-2012, GOST_R34.11-94
Алгоритмы сжатия: Без сжатия, ZIP, ZLIB,
BZIP2

Для создания своей ключевой пары по ГОСТ Р 34.10-2012 следует выполнить команду:

```
sudo gpg --full-generate-key
```

В процессе выполнения команды необходимо отвечать на запросы системы. На запрос о выборе типа ключа следует указать номер пункта, соответствующий ГОСТ Р 34.10-2012.

Вывод команды:

```
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.
```

```
gpg: создан каталог '/root/.gnupg'  
gpg: создан щит с ключами '/root/.gnupg/pubring.kbx'
```

Выберите тип ключа:

- (1) RSA и RSA (по умолчанию)
- (2) DSA и Elgamal
- (3) DSA (только для подписи)
- (4) RSA (только для подписи)
- (14) Имеющийся на карте ключ
- (14) GOST R34.10-2001 (sign only) OBSOLETE
- (15) GOST R34.10-2012 (sign only)

Ваш выбор? 15

длина ключей GOST_R34.10-2012 может быть от 1024 до 4096.

Какой размер ключа Вам необходим? (3072)

Запрошенный размер ключа - 3072 бит

Выберите срок действия ключа.

0 = не ограничен

<n> = срок действия ключа - n дней

<n>w = срок действия ключа - n недель

<n>m = срок действия ключа - n месяцев

<n>y = срок действия ключа - n лет

Срок действия ключа? (0)

Срок действия ключа не ограничен

Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: test user

Адрес электронной почты: test@test.ru

Примечание:

Вы выбрали следующий идентификатор пользователя:

```
"test user <test@test.ru>"
```

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? о
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: /root/.gnupg/trustdb.gpg: создана таблица доверия
gpg: создан каталог '/root/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/root/.gnupg/openpgp-revocs.d/07888\A89440B749338619DF1FBVB20B915E8F5EA.rev'.
открытый и секретный ключи созданы и подписаны.

```
pub gP256 2025-02-19 [SC]
    07888A89440B749338619DF1FBVB20B915E8F5EA
uid test user <test@test.ru>
```

В созданной ключевой паре открытый ключ имеет идентификатор 07888A89440B749338619DF1FBVB20B915E8F5EA, с которым он в процессе создания добавляется в хранилище gpg-ключей /root/.gnupg/pubring.kbx и по которому он определяется в командах экспорта и копирования.

Если созданная ключевая пара предназначена для подписания неисполняемых файлов в расширенных атрибутах ФС или отсоединенной подписью, то открытый ключ не требуется подписывать закрытым ключом изготовителя. В этом случае необходимо выполнить следующее:

1) экспортировать открытый ключ созданной пары, выполнив команду:

```
sudo gpg --export <идентификатор_ключа> > <имя_файла_ключа>.gpg
```

Пример

```
sudo gpg --export 07888A89440B749338619DF1FBVB20B915E8F5EA > /tmp/\
secondary_gost_key.gpg
```

2) скопировать полученный файл открытого ключа на компьютер с включенной ЗПС в следующие каталоги:

- а) /etc/digsig/xattr_keys/ — для проверки подписей в расширенных атрибутах ФС;
- б) /etc/digsig/certs/ — для проверки отсоединенных подписей.

Если созданная ключевая пара предназначена для подписания исполняемых файлов встраиваемой или любой другой цифровой подписью, то открытый ключ должен быть подписан закрытым ключом изготовителя или закрытым ключом из иерархии подписанных изготовителем ключей. Подписание выполняется владельцем закрытого ключа. Для подписания

необходимо:

1) подписать открытый ключ созданной пары, выполнив команду:

```
gpg --default-key <идентификатор_подписывающего_ключа> --sign-key \  
  <идентификатор_подписываемого_ключа>
```

Если не использован параметр `--default-key`, то для подписания используется первый найденный закрытый ключ.

Пример

```
gpg --sign-key 07888A89440B749338619DF1FB20B915E8F5EA
```

2) экспортировать подписанный открытый ключ в файл, выполнив команду:

```
sudo gpg --export <идентификатор_ключа> > <имя_файла_ключа>.gpg
```

Пример

```
sudo gpg --export 07888A89440B749338619DF1FB20B915E8F5EA > /tmp/  
  secondary_gost_key_signed.gpg
```

3) скопировать полученный файл подписанного открытого ключа на компьютер с включенной ЗПС в каталог `/etc/digsig/keys/`.

Для загрузки в ядро скопированных на компьютер новых ключей следует выполнить команду:

```
sudo update-initramfs -u -k all
```

Для просмотра всех ключей, созданных с помощью GnuPG или импортированных в GnuPG, следует использовать команду:

```
sudo gpg --list-keys
```

16.1.4. Отзыв открытых ключей

В случае потери доверия к одному или нескольким открытым ключам возможно исключить данные ключи из проверки подписей файлов. Для этого нужно сформировать список отпечатков отзывааемых ключей и загрузить его в ядро. Список формируется в файле `/etc/digsig/revoked_keys`.

ВНИМАНИЕ! При работе с данным списком следует использовать добавление в файл, чтобы случайно не перезаписать его и не удалить ранее внесенные значения отозванных ключей.

Для указания отзываемого ключа используется значение его отпечатка, вычисленное по алгоритму SHA-1. Отпечаток можно получить с помощью команды:

```
shasum <файл_ключа.gpg> | awk '{print $1}'
```

Для добавления в список отзыва одного отпечатка ключа следует использовать команду:

```
echo "<sha-1_отпечаток_отзываемого_ключа>" | sudo tee -a /etc/digsig/revoked_keys
```

Для добавления в список отзыва нескольких отпечатков ключей из текстового файла следует использовать команду:

```
cat <файл_с_отпечатками_отзываемых_ключей> | sudo tee -a /etc/digsig/revoked_keys
```

Для применения списка отзыва следует выполнить команду:

```
sudo update-initramfs -u -k all
```

и перезагрузить систему.

ВНИМАНИЕ! После применения списка отзыва повторная загрузка ключа в модуль `digsig_verif` будет невозможна. Проверка подписей файлов с помощью отозванного открытого ключа будет завершена с ошибкой. Рекомендуется перед добавлением отпечатка ключа в список отзыва предварительно проверить, какие файлы были подписаны закрытым ключом из пары, открытый ключ которой отзывается.

Для проверки того, какие файлы были подписаны закрытым ключом из ключевой пары с отзываемым открытым ключом, можно выполнить поиск файлов по заданному открытому ключу и сохранить их список в файл:

```
sudo bsign-integrator --sign-analyse <идентификатор_открытого_ключа> -E > signed.txt
```

Чтобы выполнить поиск файлов по списку открытых ключей, следует использовать команду:

```
sudo bsign-integrator --sign-analyse -I <файл_списка_идентификаторов_ключей> > \  
signed.txt
```

Описание инструмента командной строки `bsign-integrator` приведено в 16.1.6.

16.1.5. Модуль `digsig_verif`

16.1.5.1. Настройка модуля `digsig_verif`

Модуль `digsig_verif` позволяет настраивать режимы и параметры проверки цифровых подписей. Настройка модуля `digsig_verif` осуществляется редактированием конфигурационного файла `/etc/digsig/digsig_initramfs.conf`. Параметры, настраиваемые в конфигурационном файле, приведены в таблице 89.

Таблица 89

Параметр	Описание
<code>DIGSIG_ELF_MODE</code>	Режим проверки цифровых подписей исполняемых файлов и библиотек (см. 16.1.5.2). Возможные значения: • 0 — отключить проверку цифровой подписи в исполняемых файлах и разделяемых библиотеках; • 1 — включить проверку цифровой подписи в исполняемых файлах и разделяемых библиотеках. При отсутствии или неверной подписи запуск файла или библиотеки запрещается; • 2 — включить отладочный режим проверки цифровой подписи в исполняемых файлах и разделяемых библиотеках. При отсутствии или неверной подписи запуск файла или библиотеки разрешается с выдачей предупреждения
<code>DIGSIG_ELF_VERIFICATION_MODE</code>	Варианты проверки цифровых подписей исполняемых файлов и библиотек (см. 16.1.5.2). Параметр активен, если для параметра <code>DIGSIG_ELF_MODE</code> выставлено значение 1 или 2. Возможные значения: • 0 — проверять встраиваемую цифровую подпись. Если отсутствует или неверная, то запуск запрещается (значение по умолчанию); • 1 — проверять только цифровую подпись в расширенных атрибутах ФС; • 2 — проверять первую найденную цифровую подпись. Если неверная, то запуск запрещается. Поиск осуществляется в следующем порядке: • встраиваемая подпись; • отсоединенная подпись; • подпись в расширенных атрибутах ФС; • 3 — проверять все типы цифровых подписей, используется первая найденная верная подпись; • 4 — проверять встраиваемую подпись и подпись в расширенных атрибутах ФС. Запуск разрешается только при успешной проверке обоих видов подписи

Окончание таблицы 89

Параметр	Описание
DIGSIG_XATTR_MODE	Режим проверки подписей в расширенных атрибутах ФС (см. 16.1.5.3). Возможные значения: • 0 — отключить проверку подписи в расширенных атрибутах ФС для неисполняемых файлов (значение по умолчанию); • 1 — включить проверку подписи в расширенных атрибутах ФС для неисполняемых файлов. При отсутствии или неверной подписи открытие файла запрещается; • 2 — включить отладочный режим проверки подписи в расширенных атрибутах ФС для неисполняемых файлов. При отсутствии или неверной подписи открытие файла разрешается с выдачей предупреждения
DIGSIG_IGNORE_XATTR_KEYS	Игнорирование ключей из каталога <code>/etc/digsig/xattr_keys/</code> для проверки подписей в расширенных атрибутах и отсоединенных подписей (см. 16.1.5.3). Возможные значения: • 0 — ключи из каталога используются (значение по умолчанию); • 1 — ключи из каталога игнорируются
DIGSIG_IGNORE_GOST_2001	Игнорирование подписей, созданных по ГОСТ Р 34.10-2001. Возможные значения: • 0 — устаревшие подписи проверяются (значение по умолчанию); • 1 — устаревшие подписи игнорируются

После редактирования конфигурационного файла `/etc/digsig/digsig_initramfs.conf` для применения изменений необходимо выполнить команду:

```
sudo update-initramfs -u -k all
```

16.1.5.2. Проверка подписи в исполняемых файлах и разделяемых библиотеках

Модуль `digsig_verif` позволяет проверять исполняемые файлы и разделяемые библиотеки, подписанные встраиваемой подписью, подписью в расширенных атрибутах ФС, а также отсоединенной подписью. В модуле `digsig_verif` реализован механизм, позволяющий при проверке подписей файлов использовать открытые ключи из основного набора и из дополнительного набора (см. 16.1.2), а также проверять разные виды подписей в зависимости от настроек. Ключи, используемые для проверки подписей исполняемых файлов, должны быть подписаны закрытым ключом изготовителя или закрытым ключом из иерархии подписанных изготовителем ключей.

Настройка режима и вариантов проверки подписи исполняемых файлов осуществляется путем изменения значений параметров `DIGSIG_ELF_MODE` и `DIGSIG_ELF_VERIFICATION_MODE` в конфигурационном файле `/etc/digsig/digsig_initramfs.conf` (см. 16.1.5.1). Проверка подписи осуществляется при запуске файла на исполнение. Встраиваемые подписи и подписив расширенных атрибутах ФС также проверяются при запуске файлов на исполнение в контейнерах.

Также настройка возможна посредством модуля «Замкнутая программная среда» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы»), описание модуля приведено в электронной справке.

Значение текущего режима проверки подписей можно получить из системного интерфейса `/sys/digsig/elf_mode` (см. 16.1.5.4).

ВНИМАНИЕ! Установка значения 1 или 4 параметра `DIGSIG_ELF_VERIFICATION_MODE` приведет к отказу загрузки ОС, если все исполняемые файлы и разделяемые библиотеки не были предварительно подписаны в расширенных атрибутах ФС (см. 16.1.6).

После внесения изменений в конфигурационный файл `/etc/digsig/digsig_initramfs.conf` необходимо от имени учетной записи администратора выполнить команду:

```
sudo update-initramfs -u -k all
```

16.1.5.3. Проверка подписи в неисполняемых файлах

Модуль `digsig_verif` позволяет проверять неисполняемые файлы любых типов на наличие цифровой подписи в расширенных атрибутах ФС. Проверка цифровой подписи выполняется при открытии файлов на чтение или запись и не поддерживается в контейнерах.

Для проверки подписи используются открытые ключи из основного и из дополнительного наборов (см. 16.1.2). Если в конфигурационном файле `/etc/digsig/digsig_initramfs.conf` для параметра `DIGSIG_IGNORE_XATTR_KEYS` установлено значение 0, то для проверки подписи могут использоваться только ключи из каталога `/etc/digsig/keys/` и его подкаталогов, подписанные изготовителем ОС.

Настройка режима для проверки у неисполняемых файлов цифровой подписи в расширенных атрибутах ФС осуществляется путем изменения значения параметра `DIGSIG_XATTR_MODE` в конфигурационном файле `/etc/digsig/digsig_initramfs.conf` (см. 16.1.5.1).

Также настройка возможна в модуле «Замкнутая программная среда» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры

системы»), описание модуля приведено в электронной справке.

Для проверки подписи в расширенных атрибутах ФС проверяемые файлы должны быть указаны в файле `/etc/digsig/xattr_control`. Каждая строка в файле задает шаблон имени проверяемого файла в виде маски полного пути.

Пример

Проверка цифровой подписи всех файлов в каталоге `/bin`, имя которых начинается на `lo`:

```
/bin/lo
```

Для проверки отдельных файлов (без маски) необходимо указать дополнительный символ `«/»` в начале пути.

Пример

```
//home/start.sh
```

Значение текущего режима проверки подписей можно получить из системного интерфейса `/sys/digsig/xattr_mode` (см. 16.1.5.4).

После внесения изменений в конфигурационный файл `/etc/digsig/digsig_initramfs.conf` необходимо от имени учетной записи администратора выполнить команду:

```
sudo update-initramfs -u -k all
```

16.1.5.4. Системные интерфейсы модуля `digsig_verif`

Для временного изменения параметров модуля `digsig_verif` (например, с целью отладки), а также для получения текущих значений параметров возможно использовать системные интерфейсы модуля `digsig_verif`. Интерфейсы настройки представлены соответствующими файлами и позволяют изменять параметры модуля `digsig_verif` в пределах текущей сессии пользователя путем записи в них нужных значений. Интерфейсы загрузки позволяют добавлять записанные в них значения к уже перечисленным в соответствующих файлах в каталоге `/etc/digsig/`. Имеющиеся интерфейсы перечислены в таблице 90.

Таблица 90 – Системные интерфейсы модуля digsig_verif

Интерфейс	Описание
/sys/digsig/certs	Интерфейс загрузки дополнительных ключей для проверки отсоединенной подписи. Для загрузки ключа необходимо скопировать его файл в интерфейс, выполнив команду: <pre>sudo cp /etc/digsig/certs/<файл_ключа>.gpg \ /sys/digsig/certs</pre> При включенной ЗПС интерфейс заблокирован
/sys/digsig/elf_mode	Режим проверки подписей исполняемых файлов и библиотек. Для вывода текущего режима выполнить команду: <pre>sudo cat /sys/digsig/elf_mode</pre> Для изменения текущего режима выполнить команду: <pre>echo "<числовое_значение_режима>" sudo tee -a \ /sys/digsig/elf_mode</pre> Возможные значения такие же, как значения параметра DIGSIG_ELF_MODE (см. 16.1.5.1). Перед включением режима с проверкой подписи через интерфейс следует загрузить в ядро все необходимые ключи, в частности, загрузить предустановленный ключ системы сборки производителя (/etc/digsig/build_system_rbt_root_key_2018.gpg) и, при необходимости, предустановленный партнерский ключ производителя (/etc/digsig/partners_rbt_root_key_2018.gpg)
/sys/digsig/elf_verification_mode	Варианты проверки подписей исполняемых файлов и библиотек. Для вывода текущего режима выполнить команду: <pre>sudo cat /sys/digsig/elf_mode</pre> Для изменения текущего режима выполнить команду: <pre>echo "<числовое_значение_режима>" sudo tee -a \ /sys/digsig/elf_mode</pre> Возможные значения такие же, как значения параметра DIGSIG_ELF_VERIFICATION_MODE (см. 16.1.5.1). ВНИМАНИЕ! Изменить значение через интерфейс можно только до включения режима проверки подписей исполняемых файлов и библиотек
/sys/digsig/external_sig	Интерфейс загрузки файлов подписей из каталога /etc/digsig/external_sig/ для их проверки с помощью открытых ключей, расположенных в каталоге /etc/digsig/certs/ или загруженных через интерфейс /sys/digsig/certs. Для проверки файла подписи его необходимо скопировать в интерфейс, выполнив команду: <pre>sudo cp <файл.sign> /sys/digsig/external_sig</pre> В случае неуспешной проверки команда выдает ошибку доступа, при успешной проверке вывод команды отсутствует

Продолжение таблицы 90

Интерфейс	Описание
/sys/digsig/ ignore_gost2001	Игнорирование подписей, созданных по ГОСТ Р 34.10-2001. Для вывода текущего состояния выполнить команду: <pre>sudo cat /sys/digsig/ignore_gost2001</pre> Для изменения текущего состояния выполнить команду: <pre>echo "<числовое_значение_режима>" sudo tee -a \ /sys/digsig/ignore_gost2001</pre> Возможные значения такие же, как значения параметра DIGSIG_IGNORE_GOST_2001 (см. 16.1.5.1)
/sys/digsig/ ignore_xattr_keys	Игнорирование ключей из каталога /etc/digsig/xattr_keys/ для проверки подписей в расширенных атрибутах и отсоединенных подписей. Для вывода текущего состояния выполнить команду: <pre>sudo cat /sys/digsig/ignore_xattr_keys</pre> Для изменения текущего состояния выполнить команду: <pre>echo "<числовое_значение_режима>" sudo tee -a \ /sys/digsig/ignore_xattr_keys</pre> Возможные значения такие же, как значения параметра DIGSIG_IGNORE_XATTR_KEYS (см. 16.1.5.1)
/sys/digsig/keys	Интерфейс загрузки дополнительных ключей для проверки всех типов цифровых подписей. Загружаемые ключи должны быть подписаны закрытым ключом изготовителя или закрытым ключом из иерархии подписанных изготовителем ключей. Для загрузки ключа необходимо скопировать его файл в интерфейс, выполнив команду: <pre>sudo cp /etc/digsig/keys/<файл_ключа>.gpg \ /sys/digsig/keys</pre> Интерфейс поддерживает загрузку дополнительных ключей как при выключенной ЗПС, так и во время работы ЗПС
/sys/digsig/ verify_path_sign	Интерфейс загрузки файлов для проверки их подписей в соответствии с текущими настройками ЗПС. Для проверки файла выполнить команду: <pre>echo -n <имя_файла> sudo tee \ /sys/digsig/verify_path_sign</pre> Если код завершения равен 0, то проверка завершена успешно
/sys/digsig/stat	Интерфейс, выдающий количество проверенных подписей. Подпись учитывается в статистике после успешной проверки. При удалении подписанного файла подпись удаляется из статистики
/sys/digsig/ xattr_control	Интерфейс загрузки шаблонов имен файлов, для которых при открытии на чтение или на запись проверяется подпись в расширенных атрибутах (см. 16.1.5.3). Для добавления шаблонов выполнить команду: <pre>echo "<шаблон_имени_файла>" sudo tee -a \ /sys/digsig/xattr_control</pre>

Окончание таблицы 90

Интерфейс	Описание
/sys/digsig/xattr_keys	Интерфейс загрузки дополнительных ключей для проверки цифровой подписи в расширенных атрибутах ФС. Для загрузки ключа необходимо скопировать его файл в интерфейс, выполнив команду: <pre>sudo cp /etc/digsig/xattr_keys/<файл_ключа>.gpg \ /sys/digsig/xattr_keys</pre> Интерфейс поддерживает загрузку дополнительных ключей как при выключенной ЗПС, так и во время работы ЗПС
/sys/digsig/xattr_mode	Режим работы при проверке цифровой подписи в расширенных атрибутах ФС. Для вывода текущего режима выполнить команду: <pre>sudo cat /sys/digsig/xattr_mode</pre> Для изменения текущего режима выполнить команду: <pre>echo "<числовое_значение_режима>" sudo tee -a \ /sys/digsig/xattr_mode</pre> Возможные значения такие же, как значения параметра DIGSIG_XATTR_MODE (см. 16.1.5.1)
/sys/digsig/mdwe_ctl	Интерфейс загрузки файлов подписей из каталога /etc/digsig/anon_hash/signed/ для проверки хешей содержимого областей памяти и подписей бинарных файлов, которым разрешено использовать области памяти с разрешениями (см. 16.1.7.4 и 16.1.7.5). Для загрузки файла подписи выполнить команду: <pre>sudo sh -c "cat \ /etc/digsig/anon_hash/signed/<файл>.hashes.sign \ > /sys/digsig/mdwe_ctl"</pre>

16.1.6. Подписание файлов

Подписание файлов выполняется закрытыми ключами с помощью инструмента командной строки `bsign-integrator`. Инструмент `bsign-integrator` позволяет осуществлять основные операции с цифровыми подписями и поддерживает все типы цифровых подписей (см. 16.1.2).

Синтаксис команды:

```
sudo bsign-integrator <действие> [параметр] [файл]
```

Описание действий инструмента `bsign-integrator` приведено в таблице 91. Описание параметров инструмента `bsign-integrator` приведено в таблице 92.

Таблица 91 – Действия инструмента `bsign-integrator`

Действие	Описание
<code>-b, --bsign</code>	Вывести информацию об используемом по умолчанию инструменте работы с подписями
<code>-d, --default <инструмент></code>	Указать инструмент работы с подписями, который будет использоваться по умолчанию
<code>-w, --show-info</code>	Вывести информацию о подписях для указанного файла
<code>-s, --sign</code>	Подписать файл или файлы. С действием должен быть указан единственный файл для подписания или файл со списком путей к файлам (применяется с параметром <code>-L</code>). Если в параметрах не указаны типы подписей, нужный тип определяется автоматически
<code>-a, --sign-analyse <отпечаток></code>	Найти файлы, подпись которых соответствует указанному SHA-1 отпечатку сертификата ключа проверки цифровой подписи, и вывести их список. Если в параметрах не указаны типы подписей, то выполняется поиск файлов со всеми типами подписей и вывод их списка
<code>-T, --tools-list</code>	Вывести список поддерживаемых инструментов работы с подписями

Таблица 92 – Параметры инструмента `bsign-integrator`

Параметр	Описание
<code>-C, --current <путь></code>	Указать путь к инструменту работы с подписями, который будет использован в текущей команде
<code>-L, --input_list</code>	Использовать файл со списком файлов в качестве входных значений
<code>-E, --elf-only</code>	Подпись в <code>elf</code> -секции файла (только для файлов типа <code>ELF</code>)
<code>-A, --attached</code>	Подпись в конце файла
<code>-X, --xattr-only</code>	Подпись в расширенных атрибутах ФС
<code>-D, --detached</code>	Подпись в отдельном файле (отсоединенная подпись)
<code>-k=<идентификатор>, --key=<идентификатор></code>	Указать идентификатор ключа для использования в текущей команде. Используется вместе с параметром <code>-p</code>
<code>-p=<пароль>, --passphrase=<пароль></code>	Указать пароль закрытого ключа, использованного в параметре <code>-k</code>
<code>-j, --jobs=<количество></code>	Количество процессов, которое разрешено запускать одновременно. Если не задано, то соответствует количеству процессоров в системе

Более подробное описание инструмента командной строки `bsign-integrator` приведено на справочной странице `man bsign-integrator`.

Примеры:

1. Подписать один файл созданным ключом с автоматическим определением требуемого типа подписи:

```
sudo bsign-integrator -s -p=JHbd<jcd -k=07888\  
A89440B749338619DF1FBBB20B915E8F5EA filetosign
```

2. Подписать файлы из списка:

```
sudo bsign-integrator -s -p=JHbd<jcd -k=07888\  
A89440B749338619DF1FBBB20B915E8F5EA -L filelist
```

3. Подписать исполняемый файл в расширенных атрибутах ФС и отсоединенной подписью:

```
sudo bsign-integrator -s -p=JHbd<jcd -k=07888\  
A89440B749338619DF1FBBB20B915E8F5EA -X -D filetosign
```

4. Подписать созданным ключом все файлы типа ELF в расширенных атрибутах ФС:

```
sudo find / -type f -exec file {} \; | grep " ELF " | cut -d: -f1 > ELFS sudo\  
bsign-integrator -s -k=07888A89440B749338619DF1FBBB20B915E8F5EA -p=JHbd<  
jcd -X -L ELFS
```

Примечание. Данный процесс является продолжительным и может занять несколько часов.

Также подписание файлов возможно выполнить с помощью модуля «Замкнутая программная среда» в разделе «Ограничения программной среды» графической утилиты *astra-systemsettings* («Параметры системы»), описание модуля приведено в электронной справке.

16.1.7. Защита памяти при использовании ЗПС

16.1.7.1. Запрет исполнения кода из области памяти

В ядре Linux версии 6.12 и последующих присутствует механизм Memory Deny Write Execute (MDWE). Механизм MDWE запрещает для выделенной области памяти устанавливать одновременно разрешение на запись и исполнение (WRITE+EXEC) или устанавливать разрешение на исполнение, если ранее было установлено разрешение на запись (EXEC after WRITE). Таким образом, запрещается выполнение произвольного кода, загруженного в память.

В ОС механизм MDWE доработан для применения в условиях ЗПС. В ОС для механизма MDWE можно настроить разрешения на выполнение легитимного кода из областей памяти. Режимы работы механизма MDWE в ОС приведены в 16.1.7.2.

16.1.7.2. Режимы запрета исполнения кода

Для выбора режима работы механизма MDWE указать соответствующий параметр в строке параметров ядра. Режимы работы механизма MDWE в ОС и их описание приведены в таблице 93.

Таблица 93

Режим	Описание
<code>digsig.mdwe_mode=0</code>	Механизм MDWE отключен
<code>digsig.mdwe_mode=1</code>	Режим работает независимо от ЗПС. Механизм MDWE включен для самого первого системного процесса <code>init</code> и наследуется всеми процессами в системе. В данном режиме блокируется: - создание или изменение областей памяти, доступных одновременно для записи и исполнения, т.е. запрещено <code>WRITE+EXEC</code>; - изменение разрешений выделенных областей памяти, чтобы они стали исполняемыми, т.е. запрещено <code>EXEC after WRITE</code>. При включении режима могут перестать работать некоторые программы, которым для работы требуются создавать области памяти с разрешением на запись и исполнение или изменять разрешение на исполнение. Например, <code>fly-start-menu</code>, браузеры
<code>digsig.mdwe_mode=2</code>	Режим работает независимо от ЗПС. Механизм MDWE включен для самого первого системного процесса <code>init</code> и наследуется всеми процессами в системе. Блокируется создание или изменение областей памяти, доступных одновременно для записи и исполнения, т.е. запрещено <code>WRITE+EXEC</code>. Разрешено изменение разрешений выделенных областей памяти, чтобы они стали исполняемыми, т.е. разрешено <code>EXEC after WRITE</code>. При включении режима могут перестать работать некоторые программы, которым для работы требуются области памяти с разрешением на запись и исполнение. Например, браузеры
<code>digsig.mdwe_mode=3</code>	Режим работает независимо от ЗПС. Механизм MDWE включен для самого первого системного процесса <code>init</code> и наследуется всеми процессами в системе. Разрешено: - создание или изменение областей памяти, доступных одновременно для записи и исполнения, т.е. разрешено <code>WRITE+EXEC</code>; - изменение разрешений выделенных областей памяти, чтобы они стали исполняемыми, т.е. разрешено <code>EXEC after WRITE</code>. В процессе работы в данном режиме формируются хеши содержимого областей памяти типа <code>EXEC after WRITE</code> (см. 16.1.7.3). Полученные файлы с хешами могут быть подписаны (см. 16.1.7.4). Подписанные файлы с хешами используются в режимах <code>digsig.mdwe_mode=4</code>, <code>digsig.mdwe_mode=5</code> и <code>digsig.mdwe_mode=6</code>. Также в этом режиме формируется список программ, использующих области памяти с разрешениями <code>WRITE+EXEC</code>. Программы из этого списка возможно добавить в исключения (см. 16.1.7.5). Программы, добавленные в исключения, смогут использовать области памяти <code>WRITE+EXEC</code> в режиме <code>digsig.mdwe_mode=6</code>

Продолжение таблицы 93

Режим	Описание
digsig.mdwe_mode=4	Режим работает при включенном ЗПС. Механизм MDWE включен для самого первого системного процесса <code>init</code> и наследуется всеми процессами в системе. Блокируется создание или изменение областей памяти, доступных одновременно для записи и исполнения, т.е. запрещено WRITE+EXEC. Разрешено EXEC after WRITE только для областей памяти, содержимое которых имеет подписанные хеши в файлах из каталога <code>/etc/digsig/anon_hashes/</code>. Файлы с хешами формируются в режиме <code>digsig.mdwe_mode=3</code>, подписание файлов с хешами выполняется после их формирования (см. 16.1.7.4). При включении режима могут перестать работать некоторые программы, которым для работы требуются создавать области памяти с разрешением на запись и исполнение. Также могут перестать работать программы, которым требуется изменять разрешение области памяти на исполнение и для которых отсутствуют подписанные файлы с хешами
digsig.mdwe_mode=5	Режим работает при включенном ЗПС. Механизм MDWE включен для самого первого системного процесса <code>init</code> и наследуется всеми процессами в системе. Блокируется создание или изменение областей памяти, доступных одновременно для записи и исполнения, т.е. запрещено WRITE+EXEC. Разрешено EXEC after WRITE для следующих областей памяти: - содержимое которых имеет подписанные хеши в файлах из каталога <code>/etc/digsig/anon_hashes/</code>. Файлы с хешами формируются в режиме <code>digsig.mdwe_mode=3</code>, подписание файлов с хешами выполняется после их формирования (см. 16.1.7.4); - созданных процессами тех программ, которые добавлены в исключения (см. 16.1.7.5). При включении режима могут перестать работать некоторые программы, которым для работы требуются создавать области памяти с разрешением на запись и исполнение. Также могут перестать работать программы, которым требуется изменять разрешение области памяти на исполнение и у которых отсутствуют подписанные файлы с хешами или подписи бинарных файлов

Окончание таблицы 93

Режим	Описание
<code>digsig.mdwe_mode=6</code>	Режим работает при включенном ЗПС. Механизм MDWE включен для самого первого системного процесса <code>init</code> и наследуется всеми процессами в системе. Разрешено: 1) изменение разрешений EXEC after WRITE для следующих областей памяти: • содержимое которых имеет подписанные хеши в файлах из каталога <code>/etc/digsig/anon_hashes/</code>. Файлы с хешами формируются в режиме <code>digsig.mdwe_mode=3</code>, подписание файлов с хешами выполняется после их формирования (см. 16.1.7.4); • созданных процессами тех программ, которые добавлены в исключения (см. 16.1.7.5); 2) создание и изменение областей памяти WRITE+EXEC для процессов тех программ, которые добавлены в исключения (см. 16.1.7.5). При включении режима могут перестать работать некоторые программы, которым для работы требуются создавать области памяти с разрешением на запись и исполнение и у которых отсутствуют подписи бинарных файлов. Также могут перестать работать программы, которым требуется изменять разрешение области памяти на исполнение и у которых отсутствуют подписанные файлы с хешами или подписи бинарных файлов

16.1.7.3. Формирование хешей содержимого областей памяти

С помощью подписанных хешей содержимого областей памяти возможно разрешить определенным программам изменять разрешение для областей памяти EXEC after WRITE (см. 16.1.7.1).

Для формирования файлов с хешами:

1) создать каталог `/etc/digsig/anon_hashes/`:

```
sudo mkdir /etc/digsig/anon_hashes/
```

2) предоставить всем пользователям права на запись в каталог

```
sudo chmod 777 /etc/digsig/anon_hashes:
```

```
sudo chmod 777 /etc/digsig/anon_hashes
```

3) включить режим работы механизма MDWE `digsig.mdwe_mode=3` (см. 16.1.7.2);

4) выполнять в системе действия, соответствующие штатной работе.

Для каждого выполненного действия формируется в фоновом режиме хеш области памяти, у которой изменяется разрешение EXEC after WRITE. Хеши записываются в файлы в каталоге `/etc/digsig/anon_hashes/`. Имена файлов соответствуют именам процессов, которым

принадлежит содержимое областей памяти.

ВНИМАНИЕ! Некоторые программы каждый раз генерируют разное содержимое областей памяти EXEC after WRITE даже при условии выполнения одних и тех же действий. Размер файлов с хешами таких программ будет постепенно увеличиваться. Для таких программ нет возможности сформировать файлы с хешами и подписать их, но их возможно добавить в исключения для использования областей памяти EXEC after WRITE (см. 16.1.7.5). К таким программам относится, например, `fly-start-menu`.

Сформированные файлы хешей можно подписать (см. 16.1.7.4). Программы, для процессов которых есть подписанные файлы хешей, смогут изменять разрешение областей памяти EXEC after WRITE в режимах работы MDWE `digsig.mdwe_mode=4`, `digsig.mdwe_mode=5` и `digsig.mdwe_mode=6`.

16.1.7.4. Подписание файлов хешей

Сформированные согласно 16.1.7.3 файлы с хешами в каталоге `/etc/digsig/anon_hashes/` могут быть подписаны. Программы, для процессов которых есть подписанные файлы хешей, смогут изменять разрешение областей памяти EXEC after WRITE в режимах работы MDWE `digsig.mdwe_mode=4`, `digsig.mdwe_mode=5` и `digsig.mdwe_mode=6`.

Подписание файлов с хешами выполняется инструментом `bsign-pack-signer` из пакета `bsign-aliens`. Для подписания выполнить команду:

```
bsign-pack-signer sign /etc/digsig/anon_hashes/<имя_файла>.hashes
```

Подписание выполняется заранее подготовленным ключом.

Подписи будут расположены в каталоге `/etc/digsig/anon_hash/signed/`.

Справка по использованию инструмента `bsign-pack-signer` вызывается командой:

```
bsign-pack-signer --help
```

16.1.7.5. Добавление программ в исключения

Добавление программы в исключения позволит программе исполнять код из областей памяти в обход ограничений, установленных режимами работы механизма MDWE (см. 16.1.7.2).

Исключения для запретов на исполнение содержимого областей памяти могут позволить:

- создавать области памяти с разрешением WRITE+EXEC;
- изменять разрешения областей памяти EXEC after WRITE;

- создавать области памяти с разрешением WRITE+EXEC и изменять разрешения областей памяти EXEC after WRITE.

Перечень программ, которые может потребоваться добавить в исключения, определяется по результатам работы в режиме `digsig.mdwe_mode=3` (см. 16.1.7.2). В этом режиме пользователь должен выполнять в системе действия, соответствующие штатной работе. В результат формируется:

- список программ, которые используют области памяти с разрешением WRITE+EXEC;
- в каталоге `/etc/digsig/anon_hashes/` файлы с хешами областей памяти тех программ, которые изменяют разрешения областей памяти EXEC after WRITE.

ВНИМАНИЕ! Не рекомендуется добавлять в исключения браузеры, а также JIT-компиляторы и интерпретаторы.

Список программ, использующих области памяти с разрешениями WRITE+EXEC, представляет собой список путей к бинарным файлам и указывается в файле `/etc/digsig/anon_hashes/xplusw.exceptions`. Администратору необходимо принять решение какие из бинарных файлов должны быть подписаны.

В исключения для использования областей памяти EXEC after WRITE рекомендуется добавлять программы, которые каждый раз генерируют разное содержимое областей памяти EXEC after WRITE даже при условии выполнения одних и тех же действий. Размер файлов с хешами таких программ будет постепенно увеличиваться.

ВНИМАНИЕ! Исключения не наследуются, поэтому не достаточно подписать один исполняемый файл программы. Требуется подписать все файлы, использующие области памяти для работы программы.

Для добавления программы в исключения требуется подписать файлы этой программы цифровой подписью. Подписание выполняется инструментом `bsign-make-digsig-exception` из пакета `bsign-aliens`. Команда для подписания файла в зависимости от предоставляемых исключений:

- только создавать области памяти с разрешением WRITE+EXEC:

```
bsign-make-digsig-exception plus-write <путь_к_файлу_программы>
```

- только изменять разрешения областей памяти EXEC after WRITE:

```
bsign-make-digsig-exception after-write <путь_к_файлу_программы>
```

- для WRITE+EXEC и EXEC after WRITE:

```
bsign-make-digsig-exception mdwe-full <путь_к_файлу_программы>
```

Подписание выполняется заранее подготовленным ключом.

Подписи должны быть помещены в каталог `/etc/digsig/anon_hash/signed/`.

Справка по использованию инструмента `bsign-pack-signer` вызывается командой:

```
bsign-make-digsig-exception --help
```

16.2. Режим Киоск-2

Режим Киоск-2 — это инструмент подсистемы безопасности PARSEC для ограничения возможностей, предоставляемых непривилегированным пользователям. Работу режима Киоск-2 обеспечивает пакет `parsec-kiosk2`.

16.2.1. Профили пакета `parsec-kiosk2`

Профили пользователей `parsec-kiosk2` располагаются в каталоге `/etc/parsec/kiosk2` и служат для применения ограничений действий пользователей. Профиль пользователя — это текстовый файл, имя которого совпадает с именем пользователя. Один профиль пользователя служит для установки ограничений только для одного пользователя.

Если при включенном режиме Киоск-2 в `/etc/parsec/kiosk2` отсутствует профиль пользователя, то права данного пользователя в ОС не ограничиваются.

ВНИМАНИЕ! Если для пользователя создан пустой профиль, то данному пользователю запрещены любые действия.

Системные профили `parsec-kiosk2` располагаются в каталоге `/etc/parsec/kiosk2-profiles` и также служат для применения ограничений действий пользователей. Содержимое файла системного профиля представляет собой список файлов и профилей приложений с абсолютными или относительными путями, а также с информацией о правах доступа и владельцах этих файлов. Системный профиль можно добавить в профили нескольких пользователей, тогда для всех этих пользователей будет доступно использование приложения в соответствии с ограничениями в правах доступа и владельцах, указанных в системном профиле данного приложения.

16.2.2. Синтаксис профилей `parsec-kiosk2`

Разрешения на доступ могут быть установлены для файлов (жестких ссылок на файлы) или символьных ссылок на файлы. Разрешения для каталогов не поддерживаются.

Синтаксис разрешения на доступ к файлу:

```
+file rwc u/o: <имя_файла>
```

Синтаксис разрешения на доступ к файлу по символьной ссылке:

```
+link rwc uo: <имя_ссылки>
```

где `rwc` — доступ на чтение (`r`), запись (`w`) и создание (`c`), указываются для установки соответствующего разрешения;

`u` — доступ владельцу файла;

`o` — доступ остальным пользователям;

`<имя_файла>` или `<имя_ссылки>` — файл, которому назначаются права доступа, или ссылка на подобный файл. В момент загрузки профиля, содержащего разрешения на доступ по символьным ссылкам, эти ссылки должны существовать. Права доступа будут установлены на указанные в ссылках целевые файлы, которые в момент загрузки профиля могут не существовать.

В синтаксисе профилей режима Киоск-2 можно использовать кириллицу, а также метасимволы в имени файла (см. таблицу 94).

ВНИМАНИЕ! Метасимволы в имени файла для ссылки не интерпретируются.

Таблица 94

Метасимвол	Описание
<code>**</code>	Ноль или более произвольных символов, включая символ «/»
<code>*</code>	Ноль или более произвольных символов, исключая символ «/»
<code>?</code>	Один произвольный символ, исключая символ «/»
<code>[набор_символов]</code>	Один из символов, принадлежащий набору. В наборе любые символы теряют свое специальное значение. Символ «]» может входить в набор только на первой позиции
<code>\d</code>	Одна десятичная цифра
<code>\D</code>	Одна или более десятичная цифра
<code>\h</code>	Одна шестнадцатеричная цифра
<code>\H</code>	Одна или более шестнадцатеричная цифра
<code>\xNN</code>	Байт с заданным значением

Окончание таблицы 94

Метасимвол	Описание
\z	Метасимвол \z указывается в конце шаблона имени файлового объекта и распространяет действие шаблона на файловые объекты, не зарегистрированные в файловой системе (unlinked). Это файловые объекты, находящиеся в процессе создания или удаления (см. справочную страницу <code>man unlink</code>). Может применяться, если при протоколировании в отчете регистрируются обращения к файловым объектам, имя которых имеет суффикс « (deleted)» («<пробел>(deleted)»). Пример <pre>+file wc u: /home/*/.config/example/example.conf\z</pre> В этом случае разрешается доступ как к файлу <code>example.conf</code>, так и к файлу <code>example.conf (deleted)</code>. Если удалить \z, то доступ к файлу <code>example.conf (deleted)</code> будет запрещен. Открытие существующего файла на запись будет успешным, а создание нового — нет
\<любой_другой_символ>	Соответствующий символ без специальной интерпретации, например, \\, \[, *)

Для включения одного профиля в другой профиль используется @include:

```
@include other-profile-name
```

16.2.3. Синтаксис, совместимый с parsec-kiosk

В профилях режима Киоск-2 корректно распознаются строки, написанные по синтаксису режима киоска (устаревший инструмент для ограничений действий пользователя). При этом при использовании синтаксиса режима киоска в режиме Киоск-2:

- 1) в имени файла спецсимволы не интерпретируются;
- 2) любое из прав на чтение (r) или исполнение (x) преобразуется в право на чтение (r), а право на запись (w) преобразуется в права на запись (w) и создание (c);
- 3) правила одинаково применяются для доступа как владельцев, так и других пользователей;
- 4) строка обязательно должна начинаться с символа «"» или «/»;
- 5) если файл представляет символьную ссылку, то режим Киоск-2 обрабатывает целевой файл ссылки;
- 6) имя профиля должно начинаться только с латинской буквы, а также в нем должен отсутствовать символ «/».

Если профиль, созданный в режиме киоска, отредактировать в `fly-admin-kiosk`, то он сохранится с использованием синтаксиса режима Киоск-2.

16.2.4. Работа с Киоск-2 через консоль

16.2.4.1. Включение и выключение Киоск-2

При первом запуске ОС режим Киоск-2 выключен. Чтобы включить режим контроля доступа Киоск-2 и применять ограничения на работу с файлами, требуется выполнить команду:

```
echo 1 | sudo tee /sys/module/parsec/parameters/uc_enforce
```

Для включения протоколирования попыток нарушений установленных фильтров доступа выполнить команду:

```
echo 1 | sudo tee /sys/module/parsec/parameters/uc_complain
```

ВНИМАНИЕ! Результаты применения данных команд будут действительны до перезагрузки ОС.

Для включения режима контроля доступа Киоск-2 и протоколирования с сохранением данного состояния после перезагрузки требуется, соответственно, выполнить команды:

```
echo 1 | sudo tee /etc/parsec/kiosk2_enforce  
echo 1 | sudo tee /etc/parsec/kiosk2_complain
```

и перезагрузить ОС.

Результаты применения этих команд будут действительны после перезагрузки ОС.

Для выключения режима контроля доступа Киоск-2 и протоколирования в соответствующих командах требуется заменить 1 на 0.

Для того чтобы избирательно включать режим Киоск-2 и исключать выбранных пользователей, требуется:

1) создать каталог `disabled` в `/etc/parsec/kiosk2`:

```
cd /etc/parsec/kiosk2  
sudo mkdir disabled
```

2) перенести в каталог `disabled` профили тех пользователей, для которых не должны применяться ограничения:

```
sudo mv <имя_пользователя1> <имя_пользователя2> disabled
```

3) включить режим Киоск-2:

```
echo 1 | sudo tee /sys/module/parsec/parameters/uc_enforce
```

После выполнения данных действий ограничения режима Киоск-2 будут применены ко всем пользователям, кроме `<имя_пользователя1>`, `<имя_пользователя2>` и тех пользо-

вателей, для которых не были созданы профили. После перезагрузки ОС режим Киоск-2 вернется в состояние, указанное в `/etc/parsec/kiosk2_enforce`. При повторном включении режима Киоск-2 исключения для `<имя_пользователя1>`, `<имя_пользователя2>` и тех пользователей, для которых не были созданы профили, сохраняются и ограничения доступа к ним не будут применены.

16.2.4.2. Протоколирование процессов

Для того чтобы ограничения на работу приложений выполнялись корректно и без ошибок, требуется учитывать все зависимости файлов данного приложения. Поэтому при создании системного профиля приложения рекомендуется выполнить протоколирование процессов при работе с соответствующим приложением. Результат протоколирования процессов показывает все необходимые файлы приложения и пути к ним. Для выполнения протоколирования процессов требуется:

- 1) выключить режим контроля доступа Киоск-2:

```
echo 0 | sudo tee /sys/module/parsec/parameters/uc_enforce
```

- 2) включить протоколирование процессов:

```
echo 1 | sudo tee /sys/module/parsec/parameters/uc_complain
```

- 3) создать профиль пользователя:

```
true | sudo tee /etc/parsec/kiosk2/<имя_пользователя>
```

- 4) запустить сессию пользователя `<имя_пользователя>`:

```
su - <имя_пользователя>
```

- 5) запустить протоколируемое приложение;

- 6) выйти из сессии пользователя `<имя_пользователя>`:

```
exit
```

- 7) выключить протоколирование процессов:

```
echo 0 | sudo tee /sys/module/parsec/parameters/uc_complain
```

- 8) получить сообщения о запрещенных операциях:

```
sudo dmesg | grep PARSEC-UC:
```

16.2.4.3. Создание профиля

Для создания системного профиля протоколируемого приложения требуется в каталоге `/etc/parsec/kiosk2-profiles` создать файл, задав в качестве его имени название приложения. В созданном файле указать все пути с учетом синтаксиса из результатов

выполнения команды (см. 16.2.4.2, пункт 8) перечисления):

```
sudo dmesg | grep PARSEC-UC:
```

Для создания профиля пользователя требуется в каталоге `/etc/parsec/kiosk2` создать файл с именем, совпадающим с именем данного пользователя.

16.2.5. Графическая утилита управления профилями

Для управления режимом Киоск-2 и профилями, расположенными в каталогах `/etc/parsec/kiosk2` и `/etc/parsec/kiosk2-profiles`, используется графическая утилита `fly-admin-kiosk`. Описание утилиты приведено в электронной справке.

ВНИМАНИЕ! Для работы с профилями режима киоска с использованием утилиты `fly-admin-kiosk` необходимо адаптировать их синтаксис и поместить профили в каталоги `/etc/parsec/kiosk2` и `/etc/parsec/kiosk2-profiles` пакета `parsec-kiosk2` соответственно (см. 16.2.3).

Для запуска утилиты выполнить в консоли команду:

```
sudo fly-admin-kiosk
```

или перейти «меню «Пуск» — Панель управления — Безопасность — Системный киоск».

16.3. Графический киоск

Для ограничения прав запуска программ локальными пользователями возможно использовать режим графического киоска. Настройка режима графического киоска выполняется с использованием модуля «Графический киоск» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_graphics_kiosk
```

ВНИМАНИЕ! Настройка режима графического киоска с помощью модуля «Графический киоск» графической утилиты `astra-systemsettings` осуществляется администратором.

При входе в пользовательскую сессию в режиме графического киоска отображается рабочий стол с разрешенными для запуска программами. Пользователю предоставляется доступ к каталогам, ярлыкам и т.д., необходимым для работы в разрешенных программах.

Графический киоск также может работать в режиме одной программы — разрешенная программа автоматически запускается при входе в сессию, а при выходе из программы

сессия завершается.

16.4. Графический киоск в режиме «Мобильный»

Графический киоск в режиме «Мобильный» ограничивает доступ пользователя к функциям системы и возможность запуска графических приложений. Настройка графического киоска выполняется администратором в конфигурационном файле `/etc/mobile-kiosk/mobile-kiosk.conf` (если указанные каталог и конфигурационный файл отсутствуют, их необходимо создать). В конфигурационном файле необходимо указать имя учетной записи пользователя, для которого настраивается графический киоск, и в качестве значения параметра `Applications` перечислить `desktop`-файлы приложений, которые можно будет запускать указанному пользователю:

```
[<имя_пользователя>]  
Applications=<приложение1>,<приложение2>,...
```

Пример

Настройка киоска для пользователя `user`, разрешающая запуск только приложений «Галерея» и «Музыка»:

```
[user]  
Applications=fly-gallery,fly-music
```

Киоск для указанного пользователя будет включаться автоматически в начале каждой последующей сессии данного пользователя.

В графическом киоске пользователю будут доступны только указанные приложения, а также следующие функции системы:

- 1) просмотр и удаление уведомлений;
- 2) строка поиска;
- 3) изменение громкости звука и яркости экрана;
- 4) завершение работы.

Остальные приложения и функции системы, в том числе добавление, перемещение и удаление значков, изменение обоев, добавление виджетов, пользователю будут недоступны.

Если в конфигурационном файле `/etc/mobile-kiosk/mobile-kiosk.conf` в качестве значения параметра `Applications` указано одно приложение и дополнительно указан параметр `SingleMode=true`, киоск будет работать в одиночном режиме — указанное приложение будет запускаться автоматически при входе пользователя в сессию и пользователь

сможет работать только в данном приложении. При завершении работы приложения будет автоматически завершена пользовательская сессия. Из функций системы пользователю будет доступно только завершение работы.

Пример

Настройка киоска для пользователя `user`, разрешающая работать в одиночном режиме в приложении «Калькулятор»:

```
[user]
Applications=org.kde.kalk
SingleMode=true
```

16.5. Изоляция приложений

Для обеспечения изолированного выполнения графических и консольных приложений в состав ОС входит инструмент Firejail. Целью применения Firejail является минимизация риска компрометации основной ОС при запуске недоверенных или потенциально уязвимых программ.

Для обеспечения изоляции приложений в Firejail используются:

- механизм пространств имен `namespaces`;
- фильтрация системных вызовов `seccomp-bpf`.

После запуска инструмент Firejail и все его дочерние процессы используют отдельные представления ресурсов ядра, таких как сетевой стек, таблица процессов и точки монтирования.

Firejail не требует подготовки системного образа: состав окружения формируется при запуске на основе содержимого текущей ФС, и удаляется после завершения работы приложения.

При использовании Firejail предоставляются средства задания правил доступа к ФС, позволяющие:

- определять файлы и каталоги, к которым разрешен или запрещен доступ;
- предоставлять доступ к файлам или каталогам только для чтения;
- подключать для данных временные ФС (`tmpfs`);
- совмещать каталоги через `bind-mount` и `overlayfs`.

При установке ОС установка инструмента Firejail выполняется автоматически.

В целях повышения безопасности при установке Firejail снимается бит `suid`. Для того чтобы с помощью Firejail можно было запускать приложения, необходимо включить этот бит, выполнив от имени администратора команду:

```
chmod u+s /usr/bin/firejail
```

В состав ОС входят профили изоляции системных вызовов для большинства популярных приложений, в том числе для Firefox, Chromium, VLC.

Для выполнения программы в режиме изоляции требуется указать имя приложения в качестве параметра для инструмента Firejail.

Пример

```
firejail firefox
```

Примечание. Для запуска браузера Firefox может потребоваться внести изменения в конфигурационный файл `/etc/firejail/firefox.profile`, заменив строку `seccomp` на строку:

```
seccomp.drop @clock,@cpu-emulation,@debug,@module,@obsolete,@raw-io,  
@reboot,@resources,@swap,acct,add_key,bpf,fanotify_init,io_cancel,  
io_destroy,io_getevents,io_setup,io_submit,ioprio_set,kcmp,keyctl,  
mount,name_to_handle_at,nfsservctl,ni_syscall,open_by_handle_at,  
personality,pivot_root,process_vm_readv,ptrace,remap_file_pages,  
request_key,setdomainname,sethostname,syslog,umount,umount2,userfaultfd,  
vhangup,vmsplice
```

16.6. Функции безопасности системы

Пакет `astra-safepolicy` содержит инструменты управления функциями безопасности системы. Описание инструментов приведено в 16.6.3-16.6.32.

Все инструменты из состава пакета `astra-safepolicy` поддерживают стандартный набор параметров вызова, приведенный в 16.6.1. Некоторые инструменты дополнительно к стандартному набору параметров вызова поддерживают дополнительные параметры. Описание дополнительных параметров приведено в описании соответствующего инструмента.

Инструменты пакета `astra-safepolicy` выступают в роли команд-переключателей, осуществляющих включение и выключение соответствующих функций безопасности. Для просмотра состояния некоторых команд-переключателей можно использовать инструмент `astra-security-monitor` из состава пакета `astra-safepolicy`, описание инструмента приведено в 16.6.2.

16.6.1. Общие параметры вызова переключателей

Все команды-переключатели, входящие в состав пакета `astra-safepolicy`, поддерживают стандартный набор параметров вызова. Синтаксис использования команды-переключателя:

<имя_команды-переключателя> <параметр>

Перечень параметров вызова приведен в таблице 95.

Т а б л и ц а 95

Параметр	Описание
<code>enable</code>	Включить действие, выполняемое функцией безопасности
<code>disable</code>	Выключить действие, выполняемое функцией безопасности
<code>status</code>	Отобразить, если возможно, фактическое состояние переключателя на текущий момент времени. Результат выполнения команды: • АКТИВНО — системные ограничения применяются (функция выполняется); • НЕАКТИВНО — системные ограничения не применяются (функция отключена). Например, для некоторых команд-переключателей вызовы <code>enable/disable</code> меняют положение переключателя (см. параметр <code>is-enabled</code>), но для изменения состояния требуется перезагрузка. Для этих команд-переключателей до момента перезагрузки вывод <code>is-enabled</code> и вывод <code>status</code> будут различаться.
<code>is-enabled</code>	Отобразить положение переключателя. Результат выполнения команды: • ВКЛЮЧЕНО — системные ограничения включены; • ВЫКЛЮЧЕНО — системные ограничения выключены

Для некоторых инструментов при первом их включении создается соответствующий юнит службы `systemd`. В дальнейшем состояние данной функции безопасности после перезагрузки системы устанавливается в соответствии с положением переключателя путем запуска соответствующего юнита, если запуск юнита разрешен. Список юнитов, созданных инструментами, можно просмотреть в файле `astra-safepolicy.target` (в строках, начинающихся с `Wants=`).

16.6.2. Монитор безопасности

Инструмент командной строки `astra-security-monitor` отображает информацию о состоянии некоторых функций безопасности, а также выводит информацию о состоянии функции безопасности по ее идентификатору.

П р и м е ч а н и е. Отображение состояния функции безопасности также зависит от наличия установленных в системе соответствующих пакетов или служб.

Для функций безопасности, информацию о которых выводит инструмент командной строки, возможны следующие состояния:

- ВКЛЮЧЕНО — функция безопасности активна;
- ВЫКЛЮЧЕНО — функция безопасности неактивна;
- ВКЛЮЧАЕТСЯ — функция безопасности находится в процессе включения или будет включена после перезагрузки, но в настоящий момент неактивна;
- ВЫКЛЮЧАЕТСЯ — функция безопасности находится в процессе выключения или будет выключена после перезагрузки, но в настоящий момент активна;
- ЧАСТИЧНО — функция безопасности включена, но не все параметры, контролируемые этой функцией, соответствуют заданным по умолчанию.

При выводе информации о состоянии МКЦ на файловой системе проверяется соответствие меток целостности объектов ФС меткам целостности, указанным в конфигурационном файле `/etc/parsec/fs-ilev.conf` (см. 4.10). При несоответствии меток целостности выводится сообщение о количестве файловых объектов, метка целостности которых не соответствует заданной в конфигурационном файле:

- 1) «ниже» — метка целостности файлового объекта ниже заданной;
- 2) «выше» — метка целостности файлового объекта выше заданной;
- 3) «норма» — метка целостности файлового объекта соответствует заданной.

Список файловых объектов, метка целостности которых не соответствует заданной в файле `/etc/parsec/fs-ilev.conf`, можно вывести командой:

```
sudo set-fs-ilev status -v
```

При выводе информации о функции проверки подписи в расширенных атрибутах `xattr` проверяется включение проверки подписи не только в исполняемых файлах, но и в других файлах, которые подписаны в `xattr` соответствующей утилитой.

Дополнительно монитор безопасности выводит следующую информацию:

- 1) запрет входа `root` по `ssh` (если установлены средства удаленного подключения `ssh`) — запрет удаленного входа в систему пользователю `root`. По умолчанию `root` не может войти через `ssh`, функция запрета имеет состояние ВКЛЮЧЕНО;
- 2) системный киоск (Киоск-2) — значение ВКЛЮЧЕНО, если системный киоск включен и настроен хотя бы для одного пользователя;
- 3) графический киоск — значение ВКЛЮЧЕНО, если графический киоск настроен хотя бы для одного пользователя;
- 4) безопасный вход в домен (если компьютер введен в домен) — значение ВКЛЮЧЕНО, если в файле `/etc/parsec/parsec.conf` параметр `login_local` имеет значение

admin (вход для локального пользователя разрешен, если пользователь входит в группу `astra-admin`) или значение `no` (вход для локальных пользователей запрещен).

Подробное описание инструмента приведено на справочной странице `man astra-security-monitor`.

Просмотреть информацию о состоянии функций безопасности также можно с помощью модуля «Монитор безопасности» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_security_monitor
```

16.6.3. Установка квот на использование системных ресурсов

Инструмент командной строки `astra-ulimits-control` включает или выключает ограничения на использование некоторых ресурсов системы для предотвращения нарушений доступности системы в результате исчерпания ресурсов. Параметры системных ограничений задаются в файле `/etc/security/limits.conf`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменение режима ограничений не повлияет на уже вошедших в систему пользователей и будет применено только при следующем входе.

Описание инструмента приведено на справочной странице `man astra-ulimits-control`.

Управление системными ограничениями также может осуществляться с помощью модуля «Квоты» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_quotas
```

Дополнительные ограничения на получение доступа и выполнение кода на уровне ядра ОС для привилегированного пользователя обеспечиваются модулем `lockdown` в соответствии с 16.8.

16.6.4. Запрет установки бита исполнения

В ОС реализована возможность запрета установки бита исполнения, обеспечивающая предотвращение несанкционированного запуска исполняемых файлов и сценариев для командной оболочки.

Запрет распространяется на все учетные записи, кроме `root`.

Если в системе включен запрет установки бита исполнения, то установка пакетов программ, создающих файлы с битом исполнения, будет завершаться с ошибкой.

Для включения запрета используется инструмент командной строки `astra-nochmodx-lock`. Параметры вызова, используемые инструментом, приведены в таблице 95.

Также запрет установки бита исполнения включается автоматически при включении блокировки интерпретаторов с помощью инструмента `astra-interpreters-lock` (см. 16.6.7).

Изменение режима запрета вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-nochmodx-lock`.

При включенном запрете установки бита исполнения остается возможность запуска сценариев с помощью интерпретаторов. Для запрета запуска сценариев следует дополнительно использовать блокировку интерпретаторов (см. 16.6.4).

Управление режимом запрета установки бита исполнения также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

ВНИМАНИЕ! При включенном запрете установки бита исполнения запускаемым сценариям не передаются значения переменных окружения, позволяющих передать интерпретатору произвольный сценарий (`PYTHONPATH`, `PERL5LIB`, `NODE_OPTIONS` и др.). Для получения значений переменных окружения сценарии должны запускаться соответствующими интерпретаторами.

Пример

```
python3 script.py
```

16.6.5. Блокировка консоли для пользователей

Инструмент командной строки `astra-console-lock` осуществляет блокировку доступа к консоли и терминалам для пользователей, не входящих в группу `astra-console`. Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Если при включении блокировки группа `astra-console` отсутствует в ОС, то она будет создана автоматически. При этом в нее будут включены пользователи, состоящие в группе `astra-admin` на момент включения этой функции.

Изменение блокировки вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-console-lock`.

Управление блокировкой консоли для пользователей также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.6. Блокировка консоли в режиме «Мобильный»

В режиме «Мобильный» для блокировки доступа к консоли необходимо заблокировать доступ к ней для пользователей, не входящих в группу `astra-console`, с помощью инструмента `astra-console-lock` (см. 16.6.5).

Дополнительно необходимо заблокировать использование командной строки из меню поиска, вызываемого комбинацией клавиш **<ALT+F2>**. Для этого требуется в файл `/etc/xdg/kdeglobals` добавить следующие строки:

```
[KDE Action Restrictions][$i]
run_command=false
```

Блокировка комбинации клавиш **<ALT+F2>** осуществляется для всех пользователей, в т.ч. для администратора.

Для разблокировки комбинации клавиш **<ALT+F2>** требуется в файле `/etc/xdg/kdeglobals` для параметра `run_command` указать значение `true`:

```
[KDE Action Restrictions][$i]
run_command=true
```

или удалить из файла строки:

```
[KDE Action Restrictions][$i]
run_command=false
```

16.6.7. Блокировка интерпретаторов

В ОС реализована функция блокировки следующих интерпретаторов:

- bash,
- csh,
- dash,
- expect,
- irb,
- ksh,
- mksh,
- lua,
- nodejs,
- perl,
- php,
- ptksh,
- python,
- qemu-*--static,
- ruby,
- tcl,
- tk,
- zsh.

При этом блокировка интерпретаторов не влияет на использование ядерного механизма `binfmt`, который позволяет запускать исполняемые файлы другой аппаратной платформы через `qemu-*--static` (по умолчанию `binfmt` отключен).

Для блокировки интерпретаторов (кроме интерпретатора `bash`) используется инструмент командной строки `astra-interpreters-lock`.

Для блокировки интерпретатора `bash` используется инструмент `astra-bash-lock`.

ВНИМАНИЕ! Блокировка интерпретатора `bash` может ограничить функционал некоторых программ и служб, использующих `bash`, что приведет к нарушению штатной работы ОС.

ВНИМАНИЕ! После блокировки интерпретатора `bash` консольный вход пользователей с оболочкой `bash` будет невозможен.

Блокировка интерпретаторов не распространяется на учетные записи из группы `astra-admin`.

Параметры вызова, используемые данными инструментами, приведены в таблице 95.

Блокировка интерпретаторов предотвращает несанкционированное использование интерпретатора для выполнения кода напрямую из командной строки или из неименованного канала (pipe). При этом сценарии, написанные для этих интерпретаторов, выполняются в штатном режиме (у файла сценария при этом должны быть выставлены права на выполнение).

При включении блокировки интерпретаторов автоматически включается запрет установки бита исполнения (см. 16.6.4). Это обусловлено тем, что пользователь может создать сценарий, назначить ему бит исполнения и запустить, обойдя таким образом блокировку интерпретаторов.

Изменение блокировки вступает в действие немедленно.

При включенной блокировке возможно разрешить запуск интерпретаторов отдельным процессам. Для этого необходимо установить расширенный атрибут ФС `security.interp.allow` на исполняемый файл, из которого запускается нужный процесс.

Пример

```
sudo setfattr -n security.interp.allow /usr/bin/exec
```

ВНИМАНИЕ! Установка данного атрибута на исполняемый файл интерпретатора `/bin/bash` приведет к возможности запуска командных интерпретаторов любым пользователем независимо от состояния блокировки.

Описание инструментов приведено на справочных страницах `man astra-interpreters-lock` и `man astra-bash-lock`.

Управление блокировкой интерпретаторов также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы»), описание модуля приведено в электронной справке. Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.8. Блокировка макросов

Блокировка исполнения макросов в документах LibreOffice осуществляется с помощью инструмента командной строки `astra-macros-lock`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При включении блокировки макросов уровень безопасности запуска макросов LibreOffice («Сервис — Параметры — LibreOffice — Безопасность — Безопасность макросов») устанавливается в значение «Очень высокий», при котором запуск макросов возможен только из библиотек, расположенных в доверенном каталоге `/opt/libreoffice` (каталог создается вручную). До выключения блокировки изменение уровня безопасности запуска макросов и доверенного источника недоступно, в т. ч. администратору.

Изменение блокировки вступает в действие немедленно. Если изменение блокировки осуществлялось при запущенной программе пакета `libreoffice`, то оно вступает в действие при следующем запуске программы.

Описание инструмента приведено на справочной странице `man astra-macros-lock`.

Управление блокировкой исполнения макросов также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.9. Блокировка трассировки `ptrace`

Блокировка трассировки `ptrace` осуществляется с помощью инструмента командной строки `astra-pttrace-lock`, который устанавливает максимальные ограничения на использование механизма `ptrace` путем задания параметру `kernel.yama.ptrace_scope` значения 3.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Значение устанавливается сразу при включении данной функции и сохраняется после перезагрузки.

Функция не может быть выключена без перезагрузки.

Управление блокировкой трассировки `ptrace` также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

Дополнительные возможности по ограничению применения трассировки процессов ОС обеспечиваются модулем безопасности `yama` в соответствии с 16.7.

16.6.10. Блокировка клавиши <SysRq>

Блокировка клавиши <SysRq> осуществляется с помощью инструмента командной строки `astra-sysrq-lock` путем изменения значения параметра `kernel.sysrq`. Значение сохраняется в файле `/etc/sysctl.d/999-astra.conf`. При этом блокируются функции системы, доступные при нажатии клавиши <SysRq>.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

По умолчанию данная функция безопасности включена, т.е. клавиша <SysRq> не работает.

ВНИМАНИЕ! Если в командной строке ядра указан параметр `sysrq_always_enabled`, то при включении функции блокировки клавиши <SysRq> вызов `status` будет выводить НЕАКТИВНО, а вызов `is-enabled` будет выводить ВКЛЮЧЕНО.

Для управления блокировкой клавиши <SysRq> может непосредственно использоваться инструмент `sysctl`:

1) для включения блокировки клавиши <SysRq> команда:

```
sysctl -w kernel.sysrq=0
```

2) для выключения блокировки клавиши <SysRq> команда:

```
sysctl -w kernel.sysrq=1
```

3) для проверки состояния используется команда:

```
cat /proc/sys/kernel/sysrq
```

результат выполнения команды:

- 0 — блокировка клавиш SysRq включена;
- 1 — блокировка клавиш SysRq выключена.

Управление блокировкой клавиши <SysRq> также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

Примечание. Для управления блокировкой клавиши <SysRq> рекомендуется использовать инструмент `astra-sysrq-lock` или графическую утилиту `astra-systemsettings`.

16.6.11. Управление автоматическим входом

Управление автоматическим входом в графическую среду осуществляется с помощью инструмента командной строки `astra-autologin-control`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95. При использовании вызова `enable` требуется дополнительный параметр `<имя_пользователя>`, от имени которого будет выполняться автоматический вход в систему после загрузки.

Описание инструмента приведено на справочной странице `man astra-autologin-control`.

16.6.12. Блокировка запуска программ пользователями

Управление блокировкой запуска пользователями программ `df`, `chattr`, `arp`, `ip` осуществляется с помощью инструмента командной строки `astra-commands-lock`. Данные программы необходимо блокировать при обработке в одной системе информации разных уровней конфиденциальности, т.к. с их помощью можно организовать скрытый канал передачи информации между уровнями.

Программы блокируются путем выставления на них прав доступа `750 (rwx r-x ---)`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменение блокировки вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-commands-lock`.

Управление блокировкой запуска программ пользователями также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.13. Запуск контейнеров Docker на пониженном уровне МКЦ

Инструмент командной строки `astra-docker-isolation` применяется для перевода службы Docker с высокой категории целостности на категорию целостности 2, для чего в каталоге `/etc/systemd` размещается `override`-файл для службы Docker.

Данный инструмент доступен к использованию, если в ОС установлена программа Docker.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменения вступают в силу после перезапуска службы Docker.

Описание инструмента приведено на справочной странице `man astra-docker-isolation`.

16.6.14. Управление сетевыми службами

Инструмент командной строки `astra-ilev1-control` при его включении переводит установленные сетевые службы `apache2`, `dovecot` и `exim4` с высокой категории целостности на категорию целостности 1, для чего в каталоге `/etc/systemd` размещаются `override`-файлы для соответствующих служб.

Если указанные службы не были установлены при включении этой функции, то функция будет автоматически применена к данным службам при их установке.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменения вступают в силу после перезапуска служб.

Описание инструмента приведено на справочной странице `man astra-ilev1-control`.

16.6.15. Управление загрузкой модуля ядра `lkrg`

Инструмент командной строки `astra-lkrg-control` управляет загрузкой модуля ядра `lkrg` и настраивает его автозагрузку. Модуль ядра `lkrg` обеспечивает обнаружение некоторых уязвимостей и защиту пространства памяти ядра от модификации.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Если установлен пакет `lkrg`, то при включении данной функции сразу загружается модуль ядра `lkrg` и настраивается его автозагрузка при загрузке системы (на этапе загрузки `initrd`).

При выключении функции модуль `lkrg` сразу выгружается из ядра и удаляется из автозагрузки.

Описание инструмента приведено на справочной странице `man astra-lkrg-control`.

16.6.16. Блокировка автоматического конфигурирования сетевых подключений

Инструмент командной строки `astra-noautonet-control` блокирует автоматическое конфигурирование сетевых подключений путем остановки и запрета запуска служб `NetworkManager` (`network-manager`) и `connman`, а также выключает отображение элемента управления сетевыми подключениями в области уведомлений панели задач.

Инструмент не влияет на работу службы `networking`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменение блокировки вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-noautonet-control`.

16.6.17. Отключение отображения меню загрузчика

Инструмент `astra-nobootmenu-control` отключает отображение меню загрузчика GRUB 2 при загрузке системы. Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Для применения изменений требуется перезагрузка.

Описание инструмента приведено в `man astra-nobootmenu-control`.

16.6.18. Ограничение на форматирование съемных носителей

Инструмент командной строки `astra-format-lock` устанавливает или отменяет необходимость запроса пароля администратора при форматировании съемных носителей информации.

Параметры вызова, используемые данным инструментом, приведены в таблице 95. По умолчанию пароль администратора запрашивается.

Описание инструмента приведено на справочной странице `man astra-format-lock`.

16.6.19. Запрет монтирования съемных носителей

Инструмент командной строки `astra-mount-lock` запрещает монтирование съемных носителей информации пользователям, входящим в группу `floppy`, при этом запрет не распространяется на пользователей из группы `astra-admin`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменение вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-mount-lock`.

Управление запретом также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.20. Включение на файловой системе режима работы «только чтение»

Инструмент командной строки `astra-overlay` включает `overlay` (режим работы «только чтение») на корневой ФС.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

После включения функции безопасности фактическое содержимое корневой ФС монтируется в `overlay` одновременно с ФС, хранящейся в памяти. Изменения файлов сохраняются только в памяти, а ФС на носителе остается без изменений. После перезагрузки все изменения теряются, и система каждый раз загружается в исходном состоянии.

Данная функция может применяться в тех случаях, когда носитель, на котором расположена корневая ФС, аппаратно защищен от записи либо необходимо программно защитить его от изменений.

Функционал `overlay` не касается файловых систем, хранящихся на отдельных разделах, отличных от корневого. Например, если каталог `/home` хранится на отдельном разделе или носителе, вносимые в него изменения будут сохраняться после перезагрузки.

Для применения изменений требуется перезагрузка.

Описание инструмента приведено на справочной странице `man astra-overlay`.

Управление режимом также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.21. Запрет выключения компьютера пользователями

Инструмент командной строки `astra-shutdown-lock` используется для запрета выключения компьютера пользователями. Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При включении данной функции добавляется политика `polkit`, которая запрещает выключение компьютера без ввода пароля администратора. Также при включении инструмента блокируется возможность перезагрузки компьютера путем нажатия комбинации клавиш **<Ctrl+Alt+Delete>**.

Изменение вступает в действие после перезагрузки.

Описание инструмента приведено на справочной странице `man astra-shutdown-lock`.

Управление запретом выключения компьютера пользователями также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку).

Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.22. Управление вводом пароля для `sudo`

Инструмент командной строки `astra-sudo-control` включает и выключает требование ввода пароля при использовании механизма `sudo`. Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменение вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-sudo-control`.

Управление вводом пароля для `sudo` также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.23. Блокировка использования инструмента `sumac`

Инструмент командной строки `astra-sumac-lock` блокирует работу инструмента `sumac` (см. 4.16.9). Параметры вызова, используемые инструментом `astra-sumac-lock`, приведены в таблице 95.

Инструмент `astra-sumac-lock` также блокирует возможность запуска процессов с другими уровнем и/или категориями конфиденциальности с использованием графической утилиты `fly-run` при дополнительно установленном пакете `fly-run-sumac` («Запуск приложения», см. электронную справку).

Если данная блокировка включена, то все пользователи, даже у которых есть привилегия `PARSEC_CAP_SUMAC`, не смогут запускать процессы с другими уровнем и/или категориями конфиденциальности. Для этого устанавливаются права доступа `000` на исполняемый файл `sumac` и библиотеку `libsumacranner.so`.

Изменение блокировки вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-sumac-lock`.

Управление блокировкой также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.24. Управление межсетевым экраном `ufw`

Инструмент командной строки `astra-ufw-control` включает и выключает межсетевой экран `ufw`. Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Если при включении межсетевого экрана `ufw` включен другой межсетевой экран (`firewalld`), то `ufw` не будет включен.

Изменение вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-ufw-control`.

Управление межсетевым экраном `ufw` также может осуществляться с помощью модуля «Системные параметры» в разделе «Ограничения программной среды» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.25. Блокировка доступа по протоколу SSH для `root`

Блокировка доступа по протоколу SSH для учетной записи `root` осуществляется с помощью инструмента командной строки `astra-rootloginssh-control`, который изменяет значение параметра `PermitRootLogin` в конфигурационном файле `/etc/ssh/sshd_config`.

Для управления блокировкой в системе должен быть установлен пакет `ssh`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При использовании вызова `enable` параметру `PermitRootLogin` присваивается значение `no` — запрещен доступ по протоколу SSH для учетной записи `root`.

При использовании вызова `disable` параметру `PermitRootLogin` присваивается значение `prohibit-password` (значение по умолчанию) — доступ по протоколу SSH для учетной записи

`root` разрешен только с использованием ключей.

Инструмент разрешает для `root` доступ по протоколу SSH только с использованием ключей.

Изменение вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-rootloginssh-control`.

Управление блокировкой также может осуществляться с помощью модуля «Системные параметры» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.6.26. Повышение уровня защищенности

Инструмент командной строки `astra-modeswitch` повышает уровень защищенности ОС и отображает текущий уровень. Уровень защищенности определяет перечень доступных для использования функций безопасности. В ОС реализованы следующие уровни защищенности:

- базовый;
- усиленный;
- максимальный.

Повышение уровня защищенности делает доступными для включения функции безопасности соответствующего уровня защищенности.

Переключение уровня защищенности делает доступными/недоступными для включения функции безопасности соответствующего уровня защищенности.

Функции безопасности базового уровня защищенности доступны на всех уровнях защищенности.

Функции безопасности, доступные на усиленном и максимальном уровнях защищенности, приведены в таблице 96.

Таблица 96

Функция безопасности	Уровень защищенности		
	базовый	усиленный	максимальный
Замкнутая программная среда	Недоступна	Доступна	Доступна
Очистка памяти	Недоступна	Доступна, в т.ч. для ЗСУБД	Доступна, в т.ч. для ЗСУБД
Мандатный контроль целостности	Недоступна	Доступна	Доступна
Мандатное управление доступом	Недоступна	Недоступна	Доступна, в т.ч. для ЗСУБД

ВНИМАНИЕ! Функции безопасности, недоступные на текущем уровне защищенности, не могут быть включены.

ВНИМАНИЕ! Если функция безопасности доступна на данном уровне защищенности, то после повышения уровня защищенности ее состояние не меняется, но она может быть включена с использованием соответствующих инструментов.

Для включения функций безопасности используются следующие инструменты:

- 1) «Замкнутая программная среда» — управление осуществляется с помощью инструмента `astra-digsig-control`, описанного в 16.6.28;
- 2) «Очистка освобождаемой внешней памяти» — включает в себя две функции безопасности: безопасное удаление файлов и очистка разделов подкачки. Управление осуществляется с помощью инструментов `astra-secdel-control` и `astra-swapwiper-control`, описанных в 16.6.29 и 16.6.30 соответственно;
- 3) «Мандатный контроль целостности» — управление осуществляется с помощью инструмента `astra-mic-control`, описанного в 16.6.27;
- 4) «Мандатное управление доступом» — управление осуществляется с помощью инструмента `astra-mac-control`, описанного в 16.6.31.

Описание функций безопасности, доступных для каждого из уровней защищенности, приведено также в документе РУСБ.10015-01 31 01.

С инструментом командной строки `astra-modeswitch` возможно использовать только параметры, приведенные в таблице 97.

Таблица 97

Параметр	Описание
<code>list</code>	Вывести список доступных уровней защищенности. В выводе команды: 0 <code>base(orel)</code> — базовый уровень защищенности; 1 <code>advanced(voronezh)</code> — усиленный уровень защищенности; 2 <code>maximum(smolensk)</code> — максимальный уровень защищенности.

Окончание таблицы 97

Параметр	Описание
get	Вывести текущий уровень защищенности в числовом представлении
getname	Вывести текущий уровень защищенности в текстовом представлении
set m	Установить уровень защищенности, заданный параметром m. В качестве параметра m используется числовое или текстовое представление уровня защищенности

Повышение уровня защищенности осуществляется после перезагрузки ОС.

16.6.27. Управление мандатным контролем целостности

Инструмент командной строки `astra-mic-control` включает и выключает МКЦ, описанный в 4.9, а также изменяет значение максимальной категории целостности системы. Управление МКЦ осуществляется путем изменения значения параметра командной строки ядра `parsec.max_ilev`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95. При использовании вызова `enable` возможно указать необязательный параметр `-i <уровень>`, задающий значение максимальной категории целостности (после установки ОС максимальная категория целостности равна 63).

После выключения МКЦ и перезагрузки метка целостности на объектах файловой системы сбрасывается на нулевые значения.

После включения МКЦ и перезагрузки на объектах файловой системы восстанавливаются принятые по умолчанию значения целостности (высокая категория целостности на каталогах `/dev`, `/proc`, `/run`, `/sys`).

Для применения изменений требуется перезагрузка.

Описание инструмента приведено на справочной странице `man astra-mic-control`.

16.6.28. Управление замкнутой программной средой

Инструмент командной строки `astra-digsig-control` позволяет включать и выключать ЗПС в исполняемых файлах из командной строки. Описание ЗПС приведено в 16.1.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменение состояния ЗПС выполняется после перезагрузки.

Описание инструмента приведено на справочной странице `man astra-digsig-control`.

16.6.29. Управление безопасным удалением файлов

Инструмент командной строки `astra-secdel-control` управляет механизмом безопасного удаления файлов (очисткой освобождаемых блоков памяти, см. 9.1) на устройствах и дисковых разделах с ФС `ext2`, `ext3`, `ext4` или `XFS`, при этом устройства и дисковые разделы должны быть указаны в файле `/etc/fstab`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95. Дополнительно с параметрами вызова из таблицы 95 могут использоваться параметры, указанные в таблице 98.

Таблица 98

Параметр	Описание
<code>-s <способ_перезаписи></code> <code>[=<число_перезаписей>]</code>	Используется только с параметром вызова <code>enable</code> . Задаёт режим очистки (способ перезаписи данных и число перезаписей, см. 9.1). При этом в файле <code>/etc/fstab</code> для устройств и дисковых разделов будет добавлен параметр монтирования, соответствующий указанному режиму. Параметр <code>-s</code> может применяться совместно с параметрами <code>-m</code> и <code>-v</code>
<code>-m <точка_монтирования></code>	Используется с параметрами вызова <code>enable</code> , <code>disable</code> , <code>status</code> и <code>is-enabled</code> . Задаёт точку монтирования устройства или дискового раздела, для которого будет выполнена команда. Параметр <code>-m</code> может применяться совместно с параметрами <code>-s</code> и <code>-v</code>
<code>-v</code>	Может использоваться отдельно или совместно с параметрами вызова <code>enable</code> , <code>disable</code> , <code>status</code> и <code>is-enabled</code> . При отдельном использовании выводит информацию по использованию инструмента. При использовании с параметрами вызова выводит дополнительную информацию для соответствующего параметра вызова.

Если при включении механизма безопасного удаления файлов параметр `-s` не указан, то по умолчанию будет применен режим очистки путем однократной перезаписи освобождаемых блоков predetermined сигнатурой (см. 9.1).

Если при настройке механизма безопасного удаления файлов параметр `-m` не указан, то:

- 1) команда с параметром вызова `enable`, `disable` или `is-enabled` будет выполнена для всех устройств и дисковых разделов, указанных в файле `/etc/fstab`;
- 2) команда с параметром вызова `status` будет выполнена только для примонтированных устройств и дисковых разделов, указанных в файле `/etc/fstab`.

Изменение настроек механизма вступает в действие после следующего монтирования устройства или дискового раздела (в т.ч. после перезагрузки ОС, если для устройства или

дискового раздела не задан параметр монтирования `noauto`).

Описание инструмента также приведено на справочной странице `man astra-secdel-control`.

Управление механизмом также может осуществляться с помощью модуля «Политика очистки памяти» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_policy_clean_memory
```

16.6.30. Управление очисткой разделов подкачки

Инструмент командной строки `astra-swapwiper-control` контролирует очистку разделов подкачки при завершении работы ОС в соответствии с 9.1.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При использовании команды-переключателя вносятся изменения в конфигурационный файл `/etc/parsec/swap_wiper.conf`.

Изменение вступает в действие немедленно.

Описание инструмента приведено на справочной странице `man astra-swapwiper-control`.

Управление очисткой также может осуществляться с помощью модуля «Политика очистки памяти» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_policy_clean_memory
```

16.6.31. Включение и выключение мандатного управления доступом

Инструмент командной строки `astra-mac-control` включает и выключает мандатное управление доступом в соответствии с 4.2, изменяя значение параметра командной строки ядра `parsec.mac`.

Для применения изменений требуется перезагрузка ОС.

При выключении мандатного управления доступом инструмент отключает возможность работы программ на ненулевом уровне конфиденциальности.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

ВНИМАНИЕ! Отключение возможности работы программ на ненулевом уровне конфиденциальности ограничивает доступ к файловым объектам, имеющим ненулевую классификационную метку. Доступ к таким объектам может быть получен администратором (если в ОС включен МКЦ, то администратором с высокой категорией целостности). Доступ к таким объектам также может быть получен при наличии неконтролируемого физического доступа к компьютеру и возможности использовать на нем нештатные средства. Необходимо принять дополнительные меры по защите информации ограниченного доступа.

Описание инструмента приведено на справочной странице `man astra-mac-control`.

16.6.32. Управление AstraMode и MacEnable

Инструмент командной строки `astra-mode-apps` включает и выключает режим AstraMode сервера Apache2, а также управляет состоянием параметра MacEnable сервера печати CUPS.

Описание режима AstraMode приведено в РУСБ.10015-01 95 01-1, описание параметра MacEnable приведено в таблице 83.

Параметры вызова, используемые инструментом `astra-mode-apps`, приведены в таблице 95.

При использовании команды-переключателя вносятся изменения в конфигурационный файл сервера Apache2 `/etc/apache2/apache2.conf` и конфигурационный файл сервера печати `/etc/cups/cupsd.conf`, изменяя значения параметра AstraMode и параметра MacEnable соответственно.

Для применения изменений требуется перезапуск служб.

Описание инструмента приведено на справочной странице `man astra-mode-apps`.

Включение и выключение режима AstraMode и управление параметром MacEnable также можно осуществлять с помощью модуля «Мандатное управление доступом» в разделе «Управление доступом» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля можно использовать команду:

```
astra-systemsettings astra_kcm_mac
```

16.6.33. Выключение и включение аудита файлов и процессов

Подсистема аудита может оказывать существенное влияние на производительность ОС. При необходимости возможно отключить PARSEC-аудит файлов и процессов для оптимизации нагрузки на ОС, а также для повышения производительности в целом.

Инструмент командной строки `astra-audit-control` позволяет выключать и включать постоянные правила PARSEC-аудита, располагающиеся в файле `/etc/audit/rules.d/10-parsec.rules` (описание приведено в 6.1).

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При выключении PARSEC-аудита файл `/etc/audit/rules.d/10-parsec.rules` удаляется из каталога `/etc/audit/rules.d/`, в результате чего данные правила аудита перестают выполняться. Значение в файлах `/parsecfs/disable-all-audit`, `/parsecfs/disable-denied-audit` и `/parsecfs/disable-non-mac-audit` изменяются на 1 (описание параметров приведено в 6.4.7).

При включении PARSEC-аудита файл с правилами `10-parsec.rules`, шаблон которого хранится в `/usr/lib/parsec/audit/rules.d/`, копируется в каталог `/etc/audit/rules.d/` и правила начинают выполняться. Значения в файлах `/parsecfs/disable-all-audit`, `/parsecfs/disable-denied-audit` и `/parsecfs/disable-non-mac-audit` изменяются на 0.

Изменения вступают в действие немедленно.

Параметр вызова `is-enabled` выводит информацию о наличии файла `/etc/audit/rules.d/10-parsec.rules`, для которого возможны следующие состояния:

- 1) ВКЛЮЧЕНО — файл с заданными правилами аудита присутствует в каталоге;
- 2) ВЫКЛЮЧЕНО — файл с заданными правилами аудита отсутствует в каталоге.

16.6.34. Выключение и включение сетевого аудита

Сетевой аудит, как и подсистема аудита в целом, могут оказывать существенное влияние на производительность ОС. При необходимости возможно отключить сетевой PARSEC-аудит, что значительно уменьшит объем журналов регистрации сетевых событий.

Инструмент командной строки `astra-audit-network-control` позволяет выключать и включать постоянные правила сетевого PARSEC-аудита, располагающиеся в файле `/etc/audit/rules.d/10-parsec-nw.rules` (описание приведено в 6.1).

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При выключении сетевого PARSEC-аудита файл `/etc/audit/rules.d/10-parsec-nw.rules` удаляется из каталога `/etc/audit/rules.d/`, в результате чего правила аудита перестают выполняться.

При включении сетевого PARSEC-аудита файл с правилами `10-parsec-nw.rules`, шаблон которого хранится в `/usr/lib/parsec/audit/rules.d/`, копируется в каталог `/etc/audit/rules.d/` и правила начинают выполняться.

Изменения вступают в действие немедленно.

Параметр вызова `is-enabled` выводит информацию о наличии файла `/etc/audit/rules.d/10-arsec-nw.rules`, для которого возможны следующие состояния:

- 1) ВКЛЮЧЕНО — файл с заданными правилами аудита присутствует в каталоге;
- 2) ВЫКЛЮЧЕНО — файл с заданными правилами аудита отсутствует в каталоге.

16.6.35. Выключение и включение мандатного управления доступом в кластерах СУБД

Инструмент командной строки `astra-postgresql-mac-control` позволяет включать или выключать мандатное управление доступом в кластерах СУБД. Команда-переключатель изменяет значение параметра `ac_ignore_maclabel` в конфигурационных файлах кластеров `/etc/postgresql/<версия_СУБД>/<имя_кластера>/postgresql.conf` (описание параметра приведено в РУСБ.10015-01 97 01-3). По умолчанию состояние мандатного управления доступом изменяется для всех кластеров СУБД в системе.

Изменение состояния вступает в действие немедленно.

Параметры вызова, используемые данным инструментом, приведены в таблице 95. Дополнительно с параметрами вызова из таблицы 95 могут использоваться параметры, указанные в таблице 99.

Таблица 99

Параметр	Описание
<code>-i <имя_кластера></code>	Используется с параметрами вызова <code>enable</code> и <code>disable</code> . Позволяет изменить состояние мандатного управления доступом для кластеров с указанным именем (в разных версиях СУБД могут присутствовать одноименные кластеры). Для указания кластера в определенной версии СУБД следует дополнительно использовать параметр <code>-v</code>
<code>-v <версия_СУБД></code>	Используется с параметрами вызова <code>enable</code> и <code>disable</code> . Позволяет изменить состояние мандатного управления доступом для кластеров указанной версии СУБД. Для указания кластера с определенным именем следует дополнительно использовать параметр <code>-i</code>

Если мандатное управление доступом включено не для всех кластеров СУБД, то при использовании вызова `status` отображается состояние ЧАСТИЧНО и выводится список кластеров с включенным мандатным управлением доступом в формате `<версия_СУБД>/<имя_кластера>`.

Примеры:

1. Включение мандатного управления доступом для всех кластеров СУБД версии 15:

```
sudo astra-postgresql-mac-control enable -v 15
```

2. Выключение мандатного управления доступом для всех кластеров с именем main:

```
sudo astra-postgresql-mac-control disable -i main
```

3. Включение мандатного управления доступом для кластера work СУБД версии 14:

```
sudo astra-postgresql-mac-control enable -v 14 -i work
```

16.6.36. Блокировка загрузки и выгрузки модулей ядра

Инструмент командной строки `astra-modban-lock` позволяет блокировать загрузку и выгрузку модулей ядра. При использовании инструмента изменяется значение параметра ядра `kernel.modules_disabled`.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Блокировка вступает в действие немедленно. После снятия блокировки для применения изменений требуется перезагрузка.

Описание инструмента приведено на справочной странице `man astra-modban-lock`.

16.6.37. Запрет входа с несколькими категориями конфиденциальности

Инструмент командной строки `astra-singlecat-control` позволяет запретить входить в систему с несколькими категориями конфиденциальности. После включения запрета пользователь на экране графического входа в систему сможет выбирать только одну категорию конфиденциальности из назначенных ему.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При использовании команды-переключателя вносятся изменения в конфигурационный файл `/etc/parsec/singlecat.conf`.

Изменение вступает в действие немедленно.

Запретить входить в систему с несколькими категориями конфиденциальности также можно с помощью модуля «Системные параметры» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

Порядок назначения категорий конфиденциальности пользователю приведен в 17.1.

16.6.38. Запрет создания файла устройства `/dev/mem`

В ОС при загрузке ядра создается файл устройства `/dev/mem`, через который осуществляется прямой доступ к оперативной памяти. Операции чтения и записи в файл устройства `/dev/mem` эквивалентны чтению и записи непосредственно в память. Доступ к файлу ограничен дискреционными ПРД и МКЦ:

- операция записи в файл `/dev/mem` может быть выполнена только от имени суперпользователя. При включенном МКЦ — от имени суперпользователя в сессии с максимальной категорией целостности;
- операция чтения файла `/dev/mem` может быть выполнена от имени суперпользователя или пользователя, включенного в системную группу `kmem`.

В ОС доступ через `/dev/mem` по умолчанию разрешен только для взаимодействия с PCI-устройствами, BIOS и ACPI. Доступ к областям памяти, которые используются ядром ОС и процессами, запрещен, в том числе для суперпользователя.

С помощью инструмента `astra-devmem-lock` можно запретить создание файла устройства `/dev/mem` при загрузке ОС. Таким образом будет полностью исключена возможность прямого доступа к оперативной памяти.

ВНИМАНИЕ! Запрет создания файла устройства `/dev/mem` может привести к неработоспособности некоторого специального программного обеспечения.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

Изменения вступают в силу после перезагрузки ОС.

Описание инструмента приведено на справочной странице `man astra-devmem-lock`.

16.6.39. Запрет загрузки в режиме восстановления

Инструмент командной строки `astra-recoverylogin-control` позволяет запретить загрузку в режиме восстановления при использовании загрузчика GRUB 2.

Параметры вызова, используемые данным инструментом, приведены в таблице 95.

При использовании команды-переключателя вносятся изменения в конфигурационный файл `/etc/default/grub`.

Изменения вступают в силу после перезагрузки ОС.

Запретить загрузку в режиме восстановления при использовании загрузчика GRUB 2 также можно с помощью модуля «Системные параметры» графической утилиты `astra-systemsettings` («Параметры системы», см. электронную справку). Для вызова модуля использовать команду:

```
astra-systemsettings astra_kcm_system_parameters
```

16.7. Модуль безопасности Yama

Модуль безопасности Yama ограничивает возможности использования механизма `ptrace` для отладки (трассировки) процессов ОС.

Для активации отладки процессов используется системный вызов `ptrace`:

- для назначения родительского процесса отладчиком текущего процесса:

```
ptrace(PTRACE_TRACEME, 0, NULL, NULL)
```

- для назначения текущего процесса отладчиком процесса с идентификатором `pid`:

```
ptrace(PTRACE_ATTACH, pid, NULL, NULL)
```

Модуль Yama поддерживает четыре режима работы:

- 1) 0 — отладка всех процессов, запущенных от имени текущего пользователя;
- 2) 1 — отладка только дочерних процессов, за исключением случаев явного указания текущим процессом возможного процесса-отладчика через вызов `prctl` с параметром `PR_SET_PTRACER`, задав `pid` процесса-отладчика (когда любые процессы, связанные с конкретной службой, должны отлаживаться специально выделенным для этого процессом) или параметр `PR_SET_PTRACER_ANY` (для разрешения любому процессу отлаживать текущий процесс), например:

```
prctl(PR_SET_PTRACER, PR_SET_PTRACER_ANY, 0, 0, 0)
```

В случае использования `PTRACE_TRACEME` дополнительных ограничений не накладывается, для возможности выполнения `PTRACE_ATTACH` необходимо наличие у процесса-отладчика привилегии `CAP_SYS_PTRACE`;

- 3) 2 — возможна отладка любых процессов с использованием `PTRACE_TRACEME` или `PTRACE_ATTACH`, но только при наличии у процесса-отладчика привилегии `CAP_SYS_PTRACE`;
- 4) 3 — отладка процессов с использованием `ptrace` запрещена.

Узнать режим, в котором работает модуль Yama в текущий момент, можно с помощью команды:

```
sysctl kernel.yama.ptrace_scope
```

либо прочитав файл `/proc/sys/kernel/yama/ptrace_scope`:

```
cat /proc/sys/kernel/yama/ptrace_scope
```

Для изменения режима работы модуля Yama можно использовать инструмент `sysctl`, указав для параметра `kernel.yama.ptrace_scope` значение требуемого режима работы (от 0 до 3):

```
sysctl kernel.yama.ptrace_scope=2
```

либо откорректировав соответствующим образом файл `/proc/sys/kernel/yama/ptrace_scope`:

```
echo 2 > /proc/sys/kernel/yama/ptrace_scope
```

ВНИМАНИЕ! Заданный режим работы модуля Yama сохраняется только до перезагрузки ОС (выключения компьютера). При запуске ОС модуль Yama всегда будет загружаться в режиме работы 1.

Для изменения режима работы модуля Yama пользователь должен обладать привилегией `CAP_SYS_PTRACE`.

После перевода модуля Yama в режим работы 3 (полный запрет трассировки процессов), дальнейшее изменение режимов работы может быть выполнено только после перезагрузки ОС.

16.8. Модуль безопасности lockdown

Модуль безопасности lockdown устанавливает для привилегированного пользователя запрет на процедуры получения доступа и выполнения кода на уровне ядра ОС, которые в штатном режиме он может осуществить, например, с помощью подмены ядра ОС (используя системный вызов `kexec`) или через `/dev/kmem`, читая/записывая данные в память.

Модуль lockdown поддерживает два режима работы:

- 1) режим обеспечения целостности ядра ОС (режим `integrity`), включающий следующие ограничения:
 - а) `LOCKDOWN_KEXEC` — запрет выполнения системного вызова `kexec()`, обеспечивающего подмену текущего ядра ОС;

- б) LOCKDOWN_HIBERNATION — запрет гибернации, предотвращающий возможность подмены ядра ОС при выходе из нее;
 - в) LOCKDOWN_MODULE_SIGNATURE — включение проверки подписей модулей ядра ОС;
 - г) LOCKDOWN_DEV_MEM — запрет чтения и записи в /dev/mem, /dev/kmem, /dev/port;
 - д) LOCKDOWN_PCI_ACCESS — блокировка оборудования, которое потенциально может генерировать прямую адресацию памяти (DMA);
 - е) LOCKDOWN_IOPORT — блокировка ioctl-вызовов (ioctl_console) KDADDIO, KDELIO, KDENABIO и KDDISABIO для терминалов и виртуальных консолей;
 - ж) LOCKDOWN_EFI_TEST — запрет чтения /dev/efi_test;
 - з) LOCKDOWN_ACPI_TABLES — ограничение доступа к интерфейсам ACPI;
- 2) режим обеспечения конфиденциальности некоторых компонентов ядра ОС (режим confidentiality), включающий все ограничения режима integrity, а также:
- а) LOCKDOWN_KCORE — запрет чтения /proc/kcore;
 - б) LOCKDOWN_KPROBES — ограничение доступа к отладочному режиму kprobes;
 - в) LOCKDOWN_BPF_READ — ограничение доступа к механизму фильтрации пакетов BPF;
 - г) LOCKDOWN_TRACEFS — ограничение доступа к ФС tracefs.

Состояние модуля lockdown задается в файле /sys/kernel/security/lockdown, который по умолчанию имеет следующий вид:

```
[none] integrity confidentiality
```

В данном файле приведены возможные состояния модуля lockdown, а его текущее состояние отображается в квадратных скобках ([none] — модуль отключен).

После загрузки ОС модуль lockdown по умолчанию отключен.

Активировать модуль lockdown в нужном режиме работы можно в процессе функционирования ОС путем записи соответствующего режима в файл /sys/kernel/security/lockdown командой:

```
sudo echo <режим_модуля> > /sys/kernel/security/lockdown
```

где <режим_модуля> — integrity (для активации режима integrity) или confidentiality (для активации режима confidentiality), при этом:

- если модуль был отключен, то возможно активировать любой режим;

- если модуль работал в режиме `integrity`, то возможно только активировать режим `confidentiality`;
- если модуль работал в режиме `confidentiality`, то изменить режим или отключить модуль невозможно.

Изменения в режиме работы модуля `lockdown` сохраняются до перезагрузки ОС.

17. ГЕНЕРАЦИЯ КСЗ

Генерация КСЗ подразумевает настройку КСЗ из состава ОС в соответствии с требованиями безопасности, предъявляемыми к рабочему месту.

17.1. Предварительная настройка КСЗ

Предварительная настройка КСЗ осуществляется в одном из следующих режимов:

- режим ЕПП;
- локальный режим.

Для использования режима ЕПП необходимо наличие в сети установленного и настроенного сервера FreeIPA. В ОС на рабочих местах пользователей должны быть установлены клиентские пакеты FreeIPA и выполнены действия по настройке (см. документ РУСБ.10015-01 95 01-1).

Для предварительной настройки КСЗ необходимо выполнить следующие действия:

- 1) проверить, что в ОС присутствует необходимый набор уровней и категорий конфиденциальности. При необходимости создать или отредактировать их инструментами `userlev` и `usercat` (см. 4.16.11 и 4.16.12);
- 2) создать служебные учетные записи, наличие которых необходимо для функционирования программ и защищенных комплексов программ из состава ОС;
- 3) установить для служебных учетных записей необходимые привилегии;
- 4) установить необходимые параметры аудита (регистрации событий) в ОС.

Примечание. В информационных системах с мандатным управлением доступом как правило применяются классификационные метки, в которых используется четыре уровня конфиденциальности (от 0 до 3) и 64-битовая маска с различными сочетаниями категорий конфиденциальности. В ОС после установки и включения мандатного управления доступом присутствуют уровни конфиденциальности с наименованиями `Уровень_0`, `Уровень_1`, `Уровень_2` и `Уровень_3` и категории конфиденциальности с наименованиями `Категория_1` и `Категория_2`.

Предварительная настройка КСЗ в локальном режиме выполняется соответствующими инструментами командной строки, описание которых приведено в настоящем документе. Также настройка КСЗ может быть выполнена в модулях графической утилиты «Параметры системы» (`astra-systemsettings`), описание модулей приведено в электронной справке.

Предварительная настройка КСЗ в режиме ЕПП с развернутым доменом FreeIPA выполняется в веб-интерфейсе FreeIPA или инструментом командной строки `ipa user-mod` (подробная информация по использованию инструмента командной строки доступна на

справочной странице `ipa help user-mod`).

ВНИМАНИЕ! Для изменения максимальной метки безопасности в системе и объектов ФС (см. 4.4) необходимо внести изменения в сценарий `pdp-init-fs` (см. 4.16.3), переопределив значения переменных:

- `sysmaxlev` — максимальный уровень конфиденциальности;
- `sysmaxilev` — максимальная категория целостности;
- `sysmaxcat` — максимальный набор категорий конфиденциальности.

Изменить максимальную категорию целостности в системе и объектов ФС также можно с помощью модуля «Мандатный контроль целостности» раздела «Управление доступом» графической утилиты «Параметры системы» (`astra-systemsettings`), описание модуля приведено в электронной справке.

После завершения предварительной настройки КСЗ осуществляется создание учетных записей пользователей, установка для них необходимых привилегий, разрешенных уровней и категорий конфиденциальности и др.

ВНИМАНИЕ! Устанавливать для пользователей одновременно высокий уровень конфиденциальности (классификационную метку) и высокую метку целостности не рекомендуется.

Назначать пользователю минимальный набор категорий конфиденциальности не рекомендуется. Если минимальный набор назначен, то при входе в сессию он будет выбран всегда и пользователь не сможет войти в сессию с нулевой классификационной меткой.

Создание и управление учетными записями пользователей, а также другие настройки в локальном режиме выполняются в графической утилите `astra-systemsettings` («Параметры системы», см. электронную справку) или соответствующими инструментами командной строки, описание которых приведено в настоящем документе.

Создание и управление учетными записями пользователей, а также другие настройки в режиме ЕПП с развернутым доменом FreeIPA выполняются в веб-интерфейсе FreeIPA или инструментом командной строки `ipa user-mod` (подробная информация по использованию инструмента командной строки доступна на справочной странице `ipa help user-mod`).

Дополнительно в соответствии с требованиями о защите информации следует настроить необходимые подсистемы и функции безопасности из состава ОС, обеспечивающие безопасную эксплуатацию ОС (см. 17.2).

17.2. Безопасная настройка ОС

Безопасная настройка ОС осуществляется в соответствии с методическими рекомендациями по безопасной настройке операционной системы специального назначения «Astra

Linux Special Edition», приведенными на официальном ресурсе разработчика по ссылке `wiki.astralinux.ru` (раздел «Рекомендации по безопасной установке и эксплуатации Astra Linux»).

Для безопасной настройки ОС также могут использоваться входящие в состав ОС профили системы (см. 17.3).

Безопасная настройка ОС обеспечивает реализацию мер по защите информации от угроз, приведенных в 17.5.

17.3. Профили настройки КСЗ

Для настройки КСЗ в соответствии с требованиями о защите информации, предъявляемыми к определенному классу информационных систем, в состав ОС входят профили системы, выполняющие настройку КСЗ соответствующим образом.

Профили системы реализованы на основе пакета `astra-settings-exchange` (см. 17.4).

Файлы профилей системы расположены в следующих каталогах:

- `/usr/share/astra-system-profiles/profiles/` — содержит desktop-файлы вида `<имя_профиля>.desktop` с именем и описанием каждого профиля;
- `/usr/share/astra-system-profiles/profiles_data/<имя_профиля>/` — содержит конфигурационные файлы профиля с расширением `*.ini`. Описание структуры каталога и конфигурационных файлов, а также содержимого конфигурационных файлов приведено в 17.4.1.

Выбрать и применить профиль системы можно в модуле «Профили системы» графической утилиты «Параметры системы» (см. электронную справку). Кроме того, модуль «Профили системы» позволяет сформировать отчет, содержащий:

- название профиля системы, по которому сформирован отчет;
- перечень параметров системы, для которых в профиле системы указаны значения;
- значения параметров системы, указанные в профиле системы;
- фактические значения параметров системы;
- результат проверки на соответствие фактических значений значениям в профиле системы.

Применить профиль системы также можно с помощью инструмента командной строки `astra-settings-import` (см. 17.4.5). Для этого выполнить команду:

```
astra-settings-import \  
/usr/share/astra-system-profiles/profiles_data/<имя_профиля>/
```

Можно создать новый профиль системы. Профиль системы создается на основе текущих настроек КСЗ или на основе существующего профиля.

Порядок создания нового профиля системы в модуле «Профили системы» приведен в электронной справке. Файлы созданного профиля системы будут расположены в каталогах `/etc/astra-system-profiles/profiles/` и `/etc/astra-system-profiles/profiles_data/<имя_профиля>/`.

Создать новый профиль системы также можно с помощью инструмента командной строки `astra-settings-export` (см. 17.4.3).

Чтобы создать новый профиль системы на основе текущих настроек КСЗ, необходимо:

- 1) войти в сессию администратора;
- 2) в каталоге `/etc/astra-system-profiles/profiles/` создать файл вида `<имя_профиля>.desktop`;
- 3) на созданный файл установить права доступа 600;
- 4) в созданный файл добавить запись следующего вида:

```
[Desktop Entry]
Name=<Имя профиля на английском языке>
Name[ru]=<Имя профиля на русском языке>
Comment=<Описание профиля на английском языке>
Comment[ru]=<Описание профиля на русском языке>
```

Примечание. Указанная информация будет отображаться в разделе «Профили системы» графической утилиты «Параметры системы»;

- 5) для размещения конфигурационных файлов профиля создать каталог вида `/etc/astra-system-profiles/profiles_data/<имя_профиля>/`.

ВНИМАНИЕ! Имя каталога должно полностью совпадать с именем файла, созданного согласно пункту 2) перечисления;

- 6) на созданный каталог установить права доступа 700;
- 7) экспортировать текущие настройки КСЗ в конфигурационные файлы профиля с помощью `astra-settings-export`:

```
astra-settings-export \
/etc/astra-system-profiles/profiles_data/<имя_профиля>/
```

Чтобы создать новый профиль системы на основе существующего профиля системы, необходимо:

- 1) в каталоге `/etc/astra-system-profiles/profiles/` создать копию `desktop`-файла существующего профиля, переименовать и отредактировать эту копию, указав имя и описание нового профиля;

2) в каталоге `/etc/astra-system-profiles/profiles_data/` создать копию каталога существующего профиля со всеми вложенными файлами, переименовать созданную копию и отредактировать конфигурационные файлы нового профиля, установив нужные значения параметров.

ВНИМАНИЕ! Имена `desktop`-файла и каталога должны полностью совпадать.

Применить созданный профиль можно в модуле «Профили системы» или с помощью `astra-settings-import`, выполнив команду:

```
astra-settings-import /etc/astra-system-profiles/profiles_data/<имя_профиля>/
```

17.4. Управление настройками КСЗ

Управление настройками КСЗ включает в себя:

- просмотр и изменение текущих настроек КСЗ;
 - экспорт текущих настроек КСЗ в виде каталога или архива, содержащего конфигурационные файлы с расширением `*.ini`;
 - импорт настроек КСЗ из каталога или архива для настройки КСЗ по образцу.
- Пр и м е ч а н и е. Экспортированные настройки можно копировать на другие рабочие места и импортировать их для настройки КСЗ по образцу;
- создание профилей настройки КСЗ для модуля «Профили системы» графической утилиты «Параметры системы» (см. 17.3).

Структура каталогов и конфигурационных файлов, а также синтаксис конфигурационных файлов приведены в 17.4.1.

Для управления настройками КСЗ используются инструменты командной строки из состава пакета `astra-settings-exchange`:

- `astra-settings-get` — просмотр текущих настроек КСЗ (см. 17.4.2);
- `astra-settings-set` — изменение настроек КСЗ (см. 17.4.2);
- `astra-settings-export` — экспорт текущих настроек КСЗ (см. 17.4.3);
- `astra-settings-merge` — добавление настроек КСЗ в ранее экспортированный каталог или архив (см. 17.4.4);
- `astra-settings-import` — импорт настроек КСЗ из каталога или архива (см. 17.4.5);
- `astra-settings-remove` — удаление настроек КСЗ из каталога или архива (см. 17.4.6).

17.4.1. Структура настроек КСЗ

Настройки КСЗ определяются совокупностью параметров подсистем в составе КСЗ. Для удобства экспорта и импорта эти параметры объединены в группы.

Группы параметров примерно соответствуют модулям графической утилиты «Параметры системы» (см. электронную справку). Вывести список модулей можно командой:

```
astra-systemsettings --list
```

Набор групп параметров, с которыми могут работать инструменты из пакета `astra-settings-exchange`, определяется набором установленных в системе пакетов с префиксом «ase-». Также должен быть установлен пакет или пакеты, которые реализуют функции, связанные с группой параметров. Например, для управления настройками сервера SSH должны быть установлены пакеты `ase-ssh` и `openssh-server`. Вывести список пакетов, установленных или доступных для установки, можно с помощью инструмента командной строки `apt`:

```
apt search --names-only ^ase
```

Установить пакет можно командой:

```
sudo apt install <имя_пакета>
```

Группы параметров распределены по категориям. По умолчанию после установки ОС доступна категория настроек безопасности, а также категория настроек сетевых служб, содержащая группу параметров SSH. Могут быть доступны другие категории, если установлены пакеты для групп, входящих в эти категории.

Просмотреть список категорий, групп и параметров можно с помощью любого инструмента из пакета `astra-settings-exchange`. Соответствующие команды приведены в таблице 100.

Таблица 100

Команда	Описание
<code><название_инструмента> -g</code>	Вывести список категорий
<code><название_инструмента> -m</code>	Вывести список групп
<code><название_инструмента> -o <название_группы_параметров></code>	Вывести список параметров в группе
<code><название_инструмента> -p</code>	Вывести полный список групп и параметров с описанием

Примеры:

1. Вывести список категорий:

```
astra-settings-export -g
```

2. Вывести список групп параметров:

```
astra-settings-export -m
```

3. Вывести список параметров группы ase_audit:

```
astra-settings-import -o ase_audit
```

4. Вывести список всех групп и параметров с описанием:

```
astra-settings-export -p
```

5. Вывести только описание группы параметров подсистемы регистрации событий:

```
astra-settings-export -p | sed \  
-n '/ase_audit/,/ase[-_]/{/ase[-_]/!p}'
```

Определив нужные параметры, можно выполнить с ними следующие действия:

- просмотреть или изменить значение (см. 17.4.2);
- экспортировать (см. 17.4.3);
- импортировать (см. 17.4.5).

17.4.2. Просмотр и изменение настроек КСЗ

Для просмотра и изменения текущих настроек КСЗ используются инструменты командной строки `astra-settings-get` и `astra-settings-set`. Чтобы использовать эти инструменты, нужно знать название параметра и название группы, к которой он относится (см. 17.4.1).

Инструмент `astra-settings-get` выводит значение указанного параметра. Синтаксис команды:

```
astra-settings-get <название_группы_параметров>:<название_параметра>
```

Инструмент `astra-settings-set` устанавливает для указанного параметра указанное значение. Синтаксис команды:

```
astra-settings-set <название_группы_параметров>:<название_параметра> <значение>
```

Если значение параметра содержит запятые, то его необходимо целиком заключить в кавычки.

Примеры:

1. Просмотреть значение параметра `minlen` (минимальная длина пароля):

```
sudo astra-settings-get ase_policy_complexity:minlen
```

2. Изменить значение параметра `minlen`:

```
sudo astra-settings-set ase_policy_complexity:minlen 12
```

17.4.3. Экспорт настроек КСЗ

Экспорт настроек КСЗ позволяет сохранить настройки КСЗ в виде конфигурационных файлов с расширением `*.ini`, содержащих параметры и их значения. Можно экспортировать как все параметры, так и отдельные категории, группы или параметры. Например, экспортировать только группу параметров сервера печати или группу параметров политики паролей.

Для экспорта настроек КСЗ выполнить команду:

```
sudo astra-settings-export <путь_к_каталогу_или_архиву> \  
[категория1[;категория2...]|группа1[:группа2...]| \  
[параметр1 [параметр2...]]]
```

ВНИМАНИЕ! При экспорте в каталог необходимо указывать существующий каталог. При экспорте в архив не рекомендуется указывать имя существующего архива, так как файл архива будет полностью перезаписан. Для добавления параметров в существующий архив следует воспользоваться инструментом командной строки `astra-settings-merge` (см. 17.4.4).

Для экспорта всех категорий и групп выполнить команду без указания категорий, групп или параметров.

При экспорте отдельных категорий, групп или параметров необходимо соблюдать следующие правила:

- для экспорта категории указать имя этой категории. Для экспорта нескольких категорий указать их имена через точку с запятой;
- для экспорта группы указать имя этой группы. Для экспорта нескольких групп указать их имена через двоеточие;
- для экспорта параметра указать имя группы, в которую входит параметр, затем имя параметра. Для экспорта нескольких параметров указать их имена через пробел. Для экспорта параметров из разных групп сначала указать имя каждой группы, затем имя каждого параметра.

Примеры:

1. Экспорт всех параметров КСЗ в каталог:

```
mkdir dir  
sudo astra-settings-export dir
```

2. Экспорт категории настроек безопасности в архив:

```
sudo astra-settings-export archive.tar.gz security_settings
```

3. Экспорт всех параметров политики сложности пароля в архив:

```
sudo astra-settings-export archive.tar.gz ase_policy_complexity
```

4. Экспорт всех параметров политики срока действия пароля и политики блокировки учетных записей в архив:

```
sudo astra-settings-export archive.tar.gz \  
    ase_policy_expiration:ase_policy_lockout
```

5. Экспорт параметра `gecoscheck` («Проверка GECOS» в настройках политики сложности пароля) в архив:

```
sudo astra-settings-export archive.tar.gz ase_policy_complexity \  
    gecoscheck
```

При экспорте будет создана структура каталогов и конфигурационных файлов вида `<категория_настроек>/<группа_параметров>.ini` для всех экспортированных категорий и групп. При экспорте в каталог данная структура будет создана внутри каталога.

При экспорте в архив он будет создан с данной структурой каталогов и конфигурационных файлов внутри. Конфигурационный файл каждой группы будет содержать экспортированные параметры и их значения.

Пример

При экспорте группы параметров подсистемы регистрации событий в каталог `audit-export/` будет создана следующая структура каталогов и конфигурационный файл:

`audit-export/security_settings/ase_audit_settings.ini`

Конфигурационный файл `ase_audit_settings.ini` будет иметь следующий вид:

```
[General]
admin_space_left=50
admin_space_left_action=SUSPEND
auditd=start
disk_error_action=SUSPEND
disk_full_action=SUSPEND
max_log_file=8
max_log_file_action=ROTATE
num_logs=5
space_left_action=SYSLOG
syslog-ng=start
#log_file=
#log_format=
#space_left=
```

Полное описание инструмента `astra-settings-export` приведено на странице помощи:

```
astra-settings-export -h
```

17.4.4. Добавление настроек КСЗ в существующий архив

Если экспортировать настройки КСЗ в существующий архив (см. 17.4.3), то файл архива будет перезаписан. Для добавления настроек в архив без перезаписи можно воспользоваться инструментом командной строки `astra-settings-merge`.

Отсутствующие в архиве конфигурационные файлы и параметры будут добавлены в архив. Если параметры уже есть в архиве, то их значения в архиве будут переопределены на новые.

Для добавления настроек выполнить команду:

```
sudo astra-settings-merge <путь_к_архиву> \
```

```
[категория1[;категория2...]|группа1[[:группа2...]\
[параметр1 [параметр2...]]]
```

Для добавления всех категорий и групп выполнить команду без указания категорий, групп или параметров.

При добавлении отдельных категорий, групп или параметров необходимо соблюдать следующие правила:

- для добавления категории указать имя этой категории. Для добавления нескольких категорий указать их имена через точку с запятой;
- для добавления группы указать имя этой группы. Для добавления нескольких групп указать их имена через двоеточие;
- для добавления параметра указать имя группы, в которую входит параметр, затем имя параметра. Для добавления нескольких параметров указать их имена через пробел. Для добавления параметров из разных групп сначала указать имя каждой группы, затем имя каждого параметра.

Примеры:

1. Добавление всех параметров политики срока действия пароля и политики блокировки учетных записей:

```
sudo astra-settings-merge archive.tar.gz \
  ase_policy_expiration:ase_policy_lockout
```

2. Добавление параметра `gecoscheck` («Проверка GECOS» в настройках политики сложности пароля):

```
sudo astra-settings-merge archive.tar.gz ase_policy_complexity \
  gecoscheck
```

Полное описание инструмента приведено на странице помощи:

```
astra-settings-merge -h
```

17.4.5. Импорт настроек КСЗ

Импорт позволяет применить настройки КСЗ из конфигурационных файлов. Конфигурационные файлы должны иметь расширение `*.ini`, располагаться в каталоге или архиве с соблюдением структуры и названий каталогов и файлов. В конфигурационных файлах должен быть корректный синтаксис. Примеры структуры и синтаксиса можно просмотреть в ка-

талог `/etc/astra-system-profiles/profiles_data/`. Рекомендуется импортировать настройки КСЗ, которые были ранее экспортированы с помощью инструмента командной строки `astra-settings-export` (см. 17.4.3).

Можно импортировать как все параметры, так и отдельные категории, группы или параметры. Например, импортировать только группу параметров подсистемы регистрации событий или группу параметров политики паролей.

Для импорта необходимо выполнить команду:

```
sudo astra-settings-import <путь_к_каталогу_или_архиву> \  
[категория1[;категория2...]|группа1[[:группа2...] \  
[параметр1 [параметр2...]]]]
```

Для импорта всех категорий и групп выполнить команду без указания категорий, групп или параметров.

При импорте отдельных категорий, групп или параметров необходимо соблюдать следующие правила:

- для импорта категории указать имя этой категории. Для импорта нескольких категорий указать их имена через точку с запятой;
- для импорта группы указать имя этой группы. Для импорта нескольких групп указать их имена через двоеточие;
- для импорта параметра указать имя группы, в которую входит параметр, затем имя параметра. Для импорта нескольких параметров указать их имена через пробел. Для импорта параметров из разных групп сначала указать имя каждой группы, затем имя каждого параметра.

Примеры:

1. Импорт всех параметров из архива:

```
sudo astra-settings-import archive.tar.gz
```

2. Импорт параметров политики сложности пароля из архива:

```
sudo astra-settings-import archive.tar.gz ase_policy_complexity
```

Полное описание инструмента приведено на странице помощи:

```
astra-settings-import -h
```

17.4.6. Удаление настроек КСЗ

Удаление настроек КСЗ позволяет удалить конфигурационные файлы или параметры из каталога или архива.

Для удаления настроек КСЗ выполнить команду:

```
astra-settings-remove <путь_к_каталогу_или_архиву> \  
[категория1[;категория2...]|группа1[[:группа2...] \  
[параметр1 [параметр2...]]]]
```

Примечание. Удаление экспортированных конфигурационных файлов или параметров не затрагивает текущие настройки системы.

Для удаления всех категорий и групп выполнить команду без указания категорий, групп или параметров.

При удалении отдельных категорий, групп или параметров необходимо соблюдать следующие правила:

- удаляемые категории, группы или параметры должны присутствовать в указанном каталоге или архиве;
- для удаления категории указать имя этой категории. Для удаления нескольких категорий указать их имена через точку с запятой;
- для удаления группы указать имя этой группы. Для удаления нескольких групп указать их имена через двоеточие;
- для удаления параметра указать имя группы, в которую входит параметр, затем имя параметра. Для удаления нескольких параметров указать их имена через пробел. Для удаления параметров из разных групп сначала указать имя каждой группы, затем имя каждого параметра.

Примеры:

1. Удаление конфигурационного файла политики срока действия пароля из архива:

```
astra-settings-remove archive.tar.gz ase_policy_expiration
```

2. Удаление параметра `gecoscheck` («Проверка GECOS» в настройках политики сложности пароля) из конфигурационного файла политики сложности пароля:

```
astra-settings-remove archive.tar.gz ase_policy_complexity gecoscheck
```


Полное описание инструмента приведено на странице помощи:

```
astra-settings-remove -h
```

17.5. Возможности по защите от угроз безопасности информации средствами защиты ОС

В состав ОС входят средства, обеспечивающие защиту информации от различных угроз безопасности. Средства защиты, применяемые для исключения угроз безопасности информации, приведены в 17.5.1–17.5.13.

17.5.1. Угрозы, связанные с внедрением вредоносного кода и повышением привилегий

17.5.1.1. ЗПС обеспечивает запрет автоматического или ручного запуска стороннего, не имеющего корректную цифровую подпись программного обеспечения (исполняемых модулей или библиотек) в соответствии с 16.1.

17.5.1.2. МКЦ обеспечивает защиту системных компонентов (процессов, файлов) ОС, параметров функционирования СЗИ от деструктивного воздействия со стороны недоверенных пользователей. Процессы, а также конфигурационные файлы СЗИ и системного программного обеспечения обладают высокой меткой целостности, что исключает возможность оказания несанкционированного воздействия на них даже в случае получения нарушителем доступа к системе с правами `root` с низкой меткой целостности (см. 4.3).

17.5.1.3. Средства контейнеризации обеспечивают запуск программного обеспечения, включая веб-браузеры, в контейнерах от имени непривилегированных пользователей (`rootless`-режим), что обеспечивает защиту системных и пользовательских данных и процессов в случае реализации угроз безопасности информации внутри изолированного контейнера. При этом средствами контейнеризации обеспечивается блокирование запуска контейнеров при нарушении целостности образов и/или конфигурации, а также при обнаружении уязвимостей в них. В случае эксплуатации контейнеров в привилегированном режиме и при включенном МКЦ может быть использован механизм запуска контейнеров с пониженной категорией целостности для защиты от реализации нарушителем угроз, связанных с преодолением изоляции контейнера, и минимизации их последствий (см. раздел 8).

17.5.1.4. Киоск-2 обеспечивает возможность реализации ограничений, связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к системным компонентам ОС (см. 16.2).

17.5.1.5. Блокировка интерпретатора `bash`, а также иных интерпретаторов (`python`, `perl`, `expect`, `ruby`, `dash`, `php`, `irb`, `csch`, `lua`, `ksh`, `tcl`, `tk`, `zsh`, `nodejs` и др.) совместно с механизмом запрета установки бита исполнения (см. 16.6.4) обеспечивает запрет несанкционированного использования интерпретатора(ов) недоверенными пользователями для выполнения кода напрямую из командной строки, неименованного канала (`pipe`) и/или запуска сторонних сценариев (см. 16.6.7).

17.5.1.6. Блокировка использования макросов в стандартных приложениях, включая пакеты офисного программного обеспечения предотвращает несанкционированный запуск программного кода при их использовании (см. 16.6.8).

17.5.1.7. Средства регламентного контроля целостности обеспечивают периодическую (по запросу или расписанию) проверку целостности программного обеспечения на предмет несанкционированного изменения (в том числе за счет внедрения вредоносного программного кода) в соответствии с 10.4.

17.5.1.8. Регистрация событий безопасности обеспечивает гибкую настройку регистрируемых событий, их автоматизированную обработку и визуализацию. Подсистема позволяет контролировать события безопасности информации, связанные как со штатными подсистемами управления доступом операционных систем семейства Linux, так и с дополнительными механизмами защиты ОС, что обеспечивает возможность своевременного реагирования на возникающие инциденты информационной безопасности, в том числе связанные с попытками обхода СЗИ (см. раздел 6).

17.5.1.9. Модуль безопасности `uama` ограничивает использование механизмов отладки (трассировки) процессов ОС, что исключает возможность инжектирования за счет этого механизма недоверенного кода в адресное пространство функционирующих процессов (см. 16.7).

17.5.1.10. Модуль безопасности `lockdown` обеспечивает запрет получения несанкционированного доступа и выполнения кода на уровне ядра ОС, в том числе со стороны привилегированного пользователя (см. 16.8).

17.5.1.11. Ядро ОС обеспечивает запрет записи в область памяти, помеченную как исполняемая; запрет прямого создания исполняемых областей памяти; запрет создания исполняемого стека; а также рандомизацию адресного пространства ядра и процессов (см. 9.2).

17.5.1.12. Модуль ядра `lkrg` обеспечивает обнаружение некоторых уязвимостей и защиту пространства памяти ядра от модификации (см. 16.6.15).

17.5.2. Угрозы виртуальной инфраструктуры

17.5.2.1. МКЦ обеспечивает защиту системных компонентов (процессов, файлов) ОС, параметров функционирования СЗИ от деструктивного воздействия со стороны недоверенных пользователей. Процессы, а также конфигурационные файлы СЗИ и системного программно-

го обеспечения обладают высокой меткой целостности, что исключает возможность оказания несанкционированного воздействия на них даже в случае получения нарушителем доступа к системе с правами `root` с низкой меткой целостности (см. 4.3).

17.5.2.2. Мандатное управление доступом позволяет категоризовать защищаемую информацию (данные) по уровням и категориям конфиденциальности, что обеспечивает дополнительную защиту информации от несанкционированного доступа, в том числе в среде виртуализации (запуск ВМ с заданным уровнем конфиденциальности) в соответствии с 4.2.

17.5.2.3. ЗПС предоставляет возможность осуществления динамического контроля целостности файлов конфигурации, а также исполняемых файлов и загружаемых библиотек при запуске и в процессе инициализации и функционирования среды виртуализации (см. 16.1).

17.5.2.4. Механизм доверенной загрузки ВМ обеспечивает контроль целостности средствами ОС файлов образов, конфигураций и базовой системы ввода-вывода ВМ (см. 5.7).

17.5.2.5. Механизм контроля целостности областей памяти ВМ обеспечивает постановку на контроль средствами среды виртуализации и хостовой ОС областей памяти виртуальных машин по запросу гостевой ОС (см. 5.12).

17.5.2.6. Режим функционирования ВМ «только чтение» обеспечивает защиту от несанкционированного раскрытия информации, а также модификации системных компонентов образов ВМ в процессе их функционирования (см. 5.4).

17.5.2.7. Ролевое управления доступом в среде виртуализации обеспечивает дополнительные возможности разграничения полномочий по управлению средой виртуализации и ВМ, в частности действий, связанных с инициализацией ВМ (создание, запуск, изменение конфигурации и т. п.) в соответствии с 5.3.

17.5.2.8. Механизм изоляции адресных пространств ВМ поддерживается средствами ядра ОС с применением механизмов управления памятью аппаратной платформы, предоставляемых модулем ядра KVM, что обеспечивает защиту от несанкционированного доступа в адресные пространства процессов других ВМ. В соответствии с описанием раздела 7.

17.5.2.9. Модуль безопасности уата ограничивает использование механизмов отладки (трассировки) процессов ОС, что исключает возможность инжектирования за счет этого механизма недоверенного кода в адресное пространство функционирующих процессов (см. 16.7).

17.5.2.10. Модуль безопасности lockdown обеспечивает запрет получения несанкционированного доступа и выполнения кода на уровне ядра ОС, в том числе со стороны привилегированного пользователя (см. 16.8).

17.5.2.11. Механизм очистки памяти предотвращает несанкционированный доступ к остаточной информации, в том числе содержащей чувствительные данные гостевых операционных систем (см. 9.1).

17.5.3. Угрозы несанкционированного изменения конфигурации системы и средств защиты информации

17.5.3.1. МКЦ обеспечивает защиту от несанкционированного изменения конфигурации программного обеспечения и параметров функционирования средств защиты информации за счет назначения соответствующим объектам (файлам) высокой метки целостности. Соответствующие уровни целостности присваиваются по умолчанию при включении МКЦ, назначаются в дальнейшем динамически в соответствии со строгими правилами подсистемы управления доступом. Изменение уровней целостности, при необходимости, возможно только от имени привилегированного пользователя, обладающего высокой меткой целостности, что исключает несанкционированное воздействие со стороны непривилегированных пользователей, в том числе суперпользователя `root` с низкой меткой целостности (см. 4.3).

17.5.3.2. ЗПС предоставляет возможность осуществления динамического контроля целостности файлов конфигурации, а также исполняемых файлов и загружаемых библиотек при запуске и в процессе функционирования системного и прикладного программного обеспечения (см. 16.1).

17.5.3.3. Средства регламентного контроля целостности обеспечивают периодическую (по запросу или расписанию) проверку целостности системного и программного обеспечения, а также конфигурационных файлов компонентов ОС на предмет несанкционированного изменения (см. 10.4).

17.5.3.4. Киоск-2 обеспечивает возможность реализации ограничений, связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к системным компонентам ОС (см. 16.2).

17.5.3.5. Мониторинг событий безопасности информации обеспечивает гибкую настройку регистрируемых событий, их автоматизированную обработку и визуализацию. Подсистема позволяет контролировать события безопасности информации, связанные как со штатными подсистемами управления доступом операционных систем семейства Linux, так и с дополнительными механизмами защиты ОС, что обеспечивает своевременного реагирования на возникающие инциденты информационной безопасности, в том числе связанные с попытками обхода СЗИ (см. раздел 6).

17.5.3.6. Модуль безопасности lockdown в том числе обеспечивает запрет задания потенциально опасных параметров загрузки ядра ОС, а также выполнения ряда небезопасных системных вызовов и прямого доступа на запись (через специальные файлы устройств) к пространству ядра ОС со стороны привилегированного пользователя (см. 16.8).

17.5.3.7. Отключение отображения меню загрузчика обеспечивает невозможность влияния на процесс инициализации системы со стороны недоверенного пользователя (см. 16.6.17).

17.5.4. Угрозы несанкционированного доступа к информации

17.5.4.1. МКЦ обеспечивает защиту от несанкционированного изменения конфигурации программного обеспечения и параметров функционирования средств защиты информации за счет назначения соответствующим объектам (файлам) высокой метки целостности, что исключает несанкционированные воздействия со стороны непривилегированных пользователей, а также суперпользователя `root` с низкой меткой целостности (см. 4.3).

17.5.4.2. Мандатное управление доступом позволяет категоризировать защищаемую информацию (данные) по уровням и категориям конфиденциальности, обеспечивая строгие правила доступа к ней (как на чтение, так и на запись), в том числе суперпользователю `root` с низкой меткой целостности (см. 4.2).

17.5.4.3. ЗПС предоставляет возможность осуществления контроля целостности программного обеспечения и средств защиты информации как в процессе загрузки, так и динамически в процессе исполнения (см. 16.1).

17.5.4.4. Киоск-2 обеспечивает возможность реализации ограничений (в дополнение к дискреционному управлению доступом), связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к защищаемой информации (см. 16.2).

17.5.4.5. Блокировка интерпретатора `bash`, а также иных интерпретаторов (`python`, `perl`, `expect`, `ruby`, `dash`, `php`, `irb`, `csch`, `lua`, `ksh`, `tcl`, `tk`, `zsh`, `nodejs` и др.) совместно с механизмом запрета установки бита исполнения (см. 16.6.4) обеспечивает запрет несанкционированного использования интерпретатора(ов) недоверенными пользователями для выполнения кода напрямую из командной строки, неименованного канала (`pipe`) и/или запуска сторонних сценариев (см. 16.6.7).

17.5.4.6. Блокировка использования макросов в стандартных приложениях, включая пакеты офисного программного обеспечения предотвращает несанкционированный запуск программного кода при их использовании (см. 16.6.8).

17.5.4.7. Средства контейнеризации обеспечивают запуск программного обеспечения, включая веб-браузеры, в контейнерах от имени непривилегированных пользователей (`rootless`-режим), что обеспечивает защиту системных и пользовательских данных и процессов в случае реализации угроз безопасности информации внутри изолированного контейнера. При этом средствами контейнеризации обеспечивается блокирование запуска контейнеров при нарушении целостности образов и/или конфигурации, а также при обнаружении уязвимостей в них. В случае эксплуатации контейнеров в привилегированном режиме и при включенном МКЦ может быть использован механизм запуска контейнеров с пониженной категорией целостности для защиты от реализации нарушителем угроз, связанных с преодолением изоляции контейнера, и минимизации их последствий (см. раздел 8).

17.5.4.8. Средства виртуализации обеспечивают запуск виртуальных машин с применением аппаратной виртуализации, позволяющей осуществлять изоляцию адресных пространств виртуальных машин и задание ограничений на потребляемые ими ресурсы (см. документ РУСБ.10015-01 95 01-1).

17.5.5. Угрозы несанкционированного использования ресурсов системы

17.5.5.1. МКЦ обеспечивает защиту от несанкционированного изменения конфигурации программного обеспечения и параметров функционирования средств защиты информации за счет назначения соответствующим объектам (файлам) высокой метки целостности, что исключает несанкционированное воздействия со стороны непривилегированных пользователей, а также суперпользователя `root` с низкой меткой целостности (см. 4.3).

17.5.5.2. Мандатное управление доступом обеспечивает защиту от выполнения функций в контексте процессов с низким уровнем конфиденциальности при попытке доступа к ним процессов с более высоким уровнем конфиденциальности через шину D-Bus (см. 4.2).

17.5.5.3. ЗПС предоставляет возможность осуществления контроля целостности программного обеспечения и средств защиты информации как в процессе загрузки, так и динамически в процессе исполнения (см. 16.1).

17.5.5.4. Киоск-2 обеспечивает возможность реализации ограничений (в дополнение к дискреционному управлению доступом), связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к защищаемой информации (см. 16.2).

17.5.5.5. Блокировка интерпретатора `bash`, а также иных интерпретаторов (`python`, `perl`, `expect`, `ruby`, `dash`, `php`, `irb`, `csch`, `lua`, `ksh`, `tcl`, `tk`, `zsh`, `nodejs` и др.) совместно с механизмом запрета установки бита исполнения (см. 16.6.4) обеспечивает запрет несанкционированного использования интерпретатора(ов) недоверенными пользователями для выполнения кода напрямую из командной строки, неименованного канала (`pipe`) и/или запуска сторонних сценариев (см. 16.6.7).

17.5.5.6. Блокировка использования макросов в стандартных приложениях, включая пакеты офисного программного обеспечения предотвращает несанкционированный запуск программного кода при их использовании (см. 16.6.8).

17.5.5.7. Средства контейнеризации обеспечивают запуск программного обеспечения, включая веб-браузеры, в контейнерах от имени непривилегированных пользователей (`rootless`-режим), что обеспечивает защиту системных и пользовательских данных и процессов в случае реализации угроз безопасности информации внутри изолированного контейнера. При этом средствами контейнеризации обеспечивается блокирование запуска контейнеров при

нарушении целостности образов и/или конфигурации, а также при обнаружении уязвимостей в них. В случае эксплуатации контейнеров в привилегированном режиме и при включенном МКЦ может быть использован механизм запуска контейнеров с пониженной категорией целостности для защиты от реализации нарушителем угроз, связанных с преодолением изоляции контейнера, и минимизации их последствий (см. раздел 8).

17.5.5.8. Средства виртуализации обеспечивают запуск виртуальных машин с применением аппаратной виртуализации, позволяющей осуществлять изоляцию адресных пространств виртуальных машин и задание ограничений на потребляемые ими ресурсы (см. документ РУСБ.10015-01 95 01-1).

17.5.5.9. Механизм установки квот на использование системных ресурсов реализует ограничения на использование некоторых ресурсов системы для предотвращения нарушений доступности системы в результате исчерпания ресурсов (см. 16.6.3).

17.5.6. Угрозы процедур идентификации/аутентификации

17.5.6.1. МКЦ обеспечивает защиту от несанкционированного изменения конфигурации программного обеспечения и параметров функционирования средств защиты информации (в том числе обеспечивающих идентификацию и аутентификацию пользователей) за счет назначения соответствующим объектам (файлам) высокой метки целостности. Соответствующие метки целостности присваиваются по умолчанию при включении МКЦ, назначаются в дальнейшем динамически в соответствии с правилами подсистемы управления доступом. Изменение меток целостности, при необходимости, возможно только от имени привилегированного пользователя, обладающего высокой меткой целостности, что исключает несанкционированное воздействие со стороны непривилегированных пользователей, в том числе суперпользователя `root` с низкой меткой целостности (см. 4.3).

17.5.6.2. Мандатное управление доступом позволяет категорировать защищаемую информацию (данные) по уровням и категориям конфиденциальности, что обеспечивает дополнительные возможности разграничения доступа, в том числе в случае использования уязвимых криптографических алгоритмов — нарушителю для получения доступа к защищаемой информации необходимо обладать соответствующим мандатным уровнем доступа (см. 4.2).

17.5.6.3. ЗПС предоставляет возможность осуществления динамического контроля целостности файлов конфигурации, а также исполняемых файлов и загружаемых библиотек при запуске и в процессе функционирования системного и прикладного программного обеспечения (в том числе участвующего в обеспечении идентификации и аутентификации пользователей) в соответствии с 16.1.

17.5.6.4. Контроль подключения съемных машинных носителей информации (см. раздел 14) и сопоставление пользователя с устройством (см. раздел 15) позволяют регистрировать съемные носители и устройства, минимизация возможности заменить или подменить устройство (в случае использования двухфакторной аутентификации).

17.5.6.5. Средства регламентного контроля целостности обеспечивают целостность конфигурационных файлов, определяющих порядок работы РАМ-модулей, файла, содержащего перечень всех локальных пользователей ОС, а также других файлов, участвующих в процедуре аутентификации (см. 10.4).

17.5.6.6. Мониторинг событий безопасности информации обеспечивает гибкую настройку регистрируемых событий, их автоматизированную обработку и визуализацию. Подсистема позволяет контролировать события безопасности информации, связанные как со штатными подсистемами управления доступом операционных систем семейства Linux, так и с дополнительными механизмами защиты ОС, что обеспечивает возможность своевременного реагирования на возникающие инциденты информационной безопасности, в том числе связанные с попытками обхода СЗИ (см. раздел 6).

17.5.7. Угрозы сетевой инфраструктуры и веб-технологий

17.5.7.1. МКЦ обеспечивает защиту от несанкционированного изменения конфигурации программного обеспечения и параметров функционирования средств защиты информации за счет назначения конфигурационным файлам метки безопасности с ненулевыми категориями целостности, что исключает несанкционированное воздействия даже со стороны суперпользователя `root` при отсутствии в его метке безопасности соответствующих категорий целостности (см. 4.3).

17.5.7.2. Мандатное управление доступом позволяет категоризовать защищаемую информацию (данные) по уровням и категориям конфиденциальности, обеспечивая правила доступа к ней (как на чтение, так и на запись) в том числе на сетевом уровне (см. 4.2).

17.5.7.3. ЗПС предоставляет возможность осуществления контроля целостности программного обеспечения и средств защиты информации как в процессе загрузки, так и динамически в процессе исполнения (см. 16.1).

17.5.7.4. Блокировка интерпретатора `bash`, а также иных интерпретаторов (`python`, `perl`, `expect`, `ruby`, `dash`, `php`, `irb`, `csch`, `lua`, `ksh`, `tcl`, `tk`, `zsh`, `nodejs` и др.) совместно с механизмом запрета установки бита исполнения (см. 16.6.4) обеспечивает запрет несанкционированного использования интерпретатора(ов) недоверенными пользователями для выполнения кода напрямую из командной строки, неименованного канала (`pipe`) и/или запуска сторонних сценариев (см. 16.6.7).

17.5.7.5. Блокировка использования макросов в стандартных приложениях, включая пакеты офисного программного обеспечения предотвращает несанкционированный запуск программного кода при их использовании (см. 16.6.8).

17.5.7.6. Средства контейнеризации обеспечивают запуск программного обеспечения, включая веб-браузеры, в контейнерах от имени непривилегированных пользователей (`rootless`-режим), что обеспечивает защиту системных и пользовательских данных и процессов в

случае реализации угроз безопасности информации внутри изолированного контейнера. При этом средствами контейнеризации обеспечивается блокирование запуска контейнеров при нарушении целостности образов и/или конфигурации, а также при обнаружении уязвимостей в них. В случае эксплуатации контейнеров в привилегированном режиме и при включенном МКЦ может быть использован механизм запуска контейнеров с пониженной категорией целостности для защиты от реализации нарушителем угроз, связанных с преодолением изоляции контейнера, и минимизации их последствий (см. раздел 8).

17.5.7.7. Блокировка автоматического конфигурирования сетевых подключений отключает автоматическое конфигурирование сетевых подключений путем остановки и запрета запуска служб `network-manager` и `connman` (см. 16.6.16).

17.5.8. Угрозы раскрытия информации

17.5.8.1. ЗПС предоставляет возможность осуществления контроля целостности программного обеспечения и средств защиты информации как в процессе загрузки, так и динамически в процессе исполнения (см. 16.1).

17.5.8.2. МКЦ обеспечивает защиту системных компонентов (процессов, файлов) ОС, параметров функционирования СЗИ от деструктивного воздействия со стороны недоверенных пользователей. Процессы, а также конфигурационные файлы СЗИ и системного программного обеспечения обладают высокой меткой целостности, что исключает возможность оказания несанкционированного воздействия на них даже в случае получения нарушителем доступа к системе с правами `root` с низкой меткой целостности (см. 4.3).

17.5.8.3. Киоск-2 обеспечивает возможность реализации ограничений, связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к системным компонентам ОС (см. 16.2).

17.5.8.4. Механизм очистки освобождаемой внешней памяти (включая файлы подкачки) обеспечивает затирание (в том числе путем многократной перезаписи данных псевдослучайной последовательностью байтов) освобождаемых блоков ФС непосредственно при их освобождении и/или по запросу пользователя (см. 9.1).

17.5.8.5. Механизм очистки памяти (в ядерном стеке (STACKLEAK), в ядерной куче (PAGE_POISONING)) обеспечивается ядром ОС и предотвращает несанкционированный доступ к остаточной информации, в том числе содержащей чувствительные данные (см. 9.1).

17.5.8.6. Модуль безопасности lockdown обеспечивает запрет получения несанкционированного доступа к памяти и выполнения кода на уровне ядра ОС, в том числе со стороны привилегированного пользователя (суперпользователя `root`) в соответствии с 16.8.

17.5.8.7. Модуль безопасности уата ограничивает использование механизмов отладки (трассировки) процессов ОС, что исключает возможность получения за счет привилегии отладки несанкционированного доступа к адресному пространству других процессов (см. 16.7).

17.5.9. Угрозы целостности данных

17.5.9.1. ЗПС предоставляет возможность осуществления контроля целостности программного обеспечения и средств защиты информации как в процессе загрузки, так и динамически в процессе исполнения (см. 16.1).

17.5.9.2. МКЦ обеспечивает защиту от несанкционированного изменения конфигурации программного обеспечения и параметров функционирования средств защиты информации (в том числе обеспечивающих идентификацию и аутентификацию пользователей) за счет назначения соответствующим объектам (файлам) высокой метки целостности. Соответствующие уровни целостности присваиваются по умолчанию при включении МКЦ, назначаются в дальнейшем динамически в соответствии со строгими правилами подсистемы управления доступом. Изменение уровней целостности, при необходимости, возможно только от имени привилегированного пользователя, обладающего высокой меткой целостности, что исключает несанкционированное воздействие со стороны непривилегированных пользователей, в том числе суперпользователя `root` с низкой меткой целостности (см. 4.3).

17.5.9.3. Мандатное управление доступом позволяет категоризировать защищаемую информацию (данные) по уровням и категориям конфиденциальности, обеспечивая строгие правила доступа к ней (как на чтение, так и на запись), в том числе суперпользователю `root` с низкой меткой целостности (см. 4.2).

17.5.9.4. Киоск-2 обеспечивает возможность реализации ограничений, связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к системным компонентам ОС (см. 16.2).

17.5.9.5. Мониторинг событий безопасности информации обеспечивает гибкую настройку регистрируемых событий, их автоматизированную обработку и визуализацию. Подсистема позволяет контролировать события безопасности информации, связанные как со штатными подсистемами управления доступом операционных систем семейства Linux, так и с дополнительными механизмами защиты ОС, что обеспечивает возможность своевременного реагирования на возникающие инциденты информационной безопасности, в том числе связанные с попытками обхода СЗИ (см. раздел 6).

17.5.9.6. Контроль целостности программного обеспечения и средств защиты информации обеспечивается применением комплекса средств регламентного и периодического контроля целостности, включая:

- средство подсчета контрольных сумм файлов и оптических дисков (10.1);

- средство подсчета контрольных сумм файлов в deb-пакетах (10.2);
- средство контроля соответствия дистрибутиву (10.3);
- средства регламентного контроля целостности (10.4);
- средства создания и проверки ЭП (10.5);
- средства создания замкнутой программной среды (16.1).

17.5.9.7. Модуль безопасности lockdown обеспечивает запрет получения несанкционированного доступа (чтение/запись) к памяти и выполнения кода на уровне ядра ОС, в том числе со стороны привилегированного пользователя (суперпользователя `root`) в соответствии с 16.8.

17.5.9.8. Модуль безопасности uama ограничивает использование механизмов отладки (трассировки) процессов ОС, что исключает возможность получения за счет привилегии отладки несанкционированного доступа к адресному пространству других процессов (см. 16.7).

17.5.9.9. Модуль ядра lkrg обеспечивает обнаружение некоторых уязвимостей и защиту пространства памяти ядра от модификации (см. 16.6.15).

17.5.10. Угрозы некорректного использования функционала системы и ПО

17.5.10.1. ЗПС предоставляет возможность осуществления контроля целостности программного обеспечения и средств защиты информации как в процессе загрузки, так и динамически в процессе исполнения (см. 16.1).

17.5.10.2. МКЦ обеспечивает защиту системных компонентов (процессов, файлов) ОС, параметров функционирования СЗИ от деструктивного воздействия со стороны недоверенных пользователей. Процессы, а также конфигурационные файлы СЗИ и системного программного обеспечения обладают высокой меткой целостности, что исключает возможность оказания несанкционированного воздействия на них даже в случае получения нарушителем доступа к системе с правами `root` с низкой меткой целостности (см. 4.3).

17.5.10.3. Блокировка интерпретатора `bash`, а также иных интерпретаторов (`python`, `perl`, `expect`, `ruby`, `dash`, `php`, `irb`, `csh`, `lua`, `ksh`, `tcl`, `tk`, `zsh`, `nodejs` и др.) совместно с механизмом запрета установки бита исполнения (см. 16.6.4) обеспечивает запрет несанкционированного использования интерпретатора(ов) недоверенными пользователями для выполнения кода напрямую из командной строки, неименованного канала (`pipe`) и/или запуска сторонних сценариев (см. 16.6.7).

17.5.10.4. Блокировка использования макросов в стандартных приложениях, включая пакеты офисного программного обеспечения предотвращает несанкционированный запуск программного кода при их использовании (см. 16.6.8).

17.5.10.5. Средства контейнеризации обеспечивают запуск программного обеспечения, включая веб-браузеры, в контейнерах от имени непривилегированных пользователей

(`rootless`-режим), что обеспечивает защиту системных и пользовательских данных и процессов в случае реализации угроз безопасности информации внутри изолированного контейнера. При этом средствами контейнеризации обеспечивается блокирование запуска контейнеров при нарушении целостности образов и/или конфигурации, а также при обнаружении уязвимостей в них. В случае эксплуатации контейнеров в привилегированном режиме и при включенном МКЦ может быть использован механизм запуска контейнеров с пониженной категорией целостности для защиты от реализации нарушителем угроз, связанных с преодолением изоляции контейнера, и минимизации их последствий (см. раздел 8).

17.5.11. Угрозы несанкционированного восстановления информации

17.5.11.1. Механизм очистки освобождаемой внешней памяти (включая файлы подкачки) обеспечивает затирание (в том числе путем многократной перезаписи данных псевдослучайной последовательностью байтов) освобождаемых блоков ФС непосредственно при их освобождении и/или по запросу пользователя (см. 9.1).

17.5.11.2. Механизм очистки памяти (в ядерном стеке (`STACKLEAK`), в ядерной куче (`PAGE_POISONING`)) обеспечивается ядром ОС и предотвращает несанкционированный доступ к остаточной информации, в том числе содержащей чувствительные данные (см. 9.1).

17.5.11.3. Средствами ядра ОС обеспечивается рандомизация адресного пространства процессов (см. 9.2).

17.5.12. Угрозы несанкционированного доступа к низкоуровневым данным

17.5.12.1. ЗПС предоставляет возможность осуществления контроля целостности программного обеспечения и средств защиты информации как в процессе загрузки, так и динамически в процессе исполнения (см. 16.1).

17.5.12.2. МКЦ обеспечивает защиту системных компонентов (процессов, файлов) ОС, параметров функционирования СЗИ от деструктивного воздействия со стороны недоверенных пользователей. Процессы, а также конфигурационные файлы СЗИ и системного программного обеспечения обладают высокой меткой целостности, что исключает возможность оказания несанкционированного воздействия на них даже в случае получения нарушителем доступа к системе с правами `root` с низкой меткой целостности (см. 4.3).

17.5.12.3. Киоск-2 обеспечивает возможность реализации ограничений, связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к системным компонентам ОС (см. 16.2).

17.5.12.4. Блокировка интерпретатора `bash`, а также иных интерпретаторов (`python`, `perl`, `expect`, `ruby`, `dash`, `php`, `irb`, `csch`, `lua`, `ksh`, `tcl`, `tk`, `zsh`, `nodejs` и др.) совместно

с механизмом запрета установки бита исполнения (см. 16.6.4) — обеспечивает запрет несанкционированного использования интерпретатора(ов) недоверенными пользователями для выполнения кода напрямую из командной строки, неименованного канала (`pipe`) и/или запуска сторонних сценариев (см. 16.6.7).

17.5.12.5. Механизм очистки памяти (в ядерном стеке (`STACKLEAK`), в ядерной куче (`PAGE_POISONING`)) обеспечивается ядром ОС и предотвращает несанкционированный доступ к остаточной информации, в том числе содержащей чувствительные данные (см. 9.1).

17.5.12.6. Средствами ядра ОС обеспечивается рандомизация адресного пространства процессов (см. 9.2).

17.5.12.7. Модуль безопасности `lockdown` обеспечивает запрет получения несанкционированного доступа (чтение/запись) к памяти и выполнения кода на уровне ядра ОС, прямого доступа к портам ввода-вывода `ioport` и т.п., в том числе со стороны привилегированного пользователя (суперпользователя `root`) в соответствии с 16.8.

17.5.12.8. Модуль безопасности `uama` ограничивает использование механизмов отладки (трассировки) процессов ОС, что исключает возможность получения за счет привилегии отладки несанкционированного доступа к адресному пространству других процессов (см. 16.7).

17.5.12.9. Модуль ядра `lkrg` обеспечивает обнаружение некоторых уязвимостей и защиту пространства памяти ядра от модификации (см. 16.6.15).

17.5.13. Угрозы «отказ в обслуживании»

17.5.13.1. МКЦ обеспечивает защиту системных компонентов (процессов, файлов) ОС, параметров функционирования СЗИ от деструктивного воздействия со стороны недоверенных пользователей. Процессы, а также конфигурационные файлы СЗИ и системного программного обеспечения обладают высокой меткой целостности, что исключает возможность оказания несанкционированного воздействия на них даже в случае получения нарушителем доступа к системе с правами `root` с низкой меткой целостности (см. 4.3).

17.5.13.2. Мандатное управление доступом позволяет категоризировать защищаемую информацию (данные) по уровням и категориям конфиденциальности, что обеспечивает дополнительные возможности разграничения доступа (см. 4.2).

17.5.13.3. Киоск-2 обеспечивает возможность реализации ограничений, связанных с разрешениями запуска программного обеспечения и получением доступа к объектам ФС, для отдельных пользователей путем настройки профилей пользователей (системных профилей), что обеспечивает невозможность несанкционированного запуска произвольного кода и доступа к системным компонентам ОС (см. 16.2).

17.5.13.4. ЗПС предоставляет возможность осуществления динамического контроля целостности файлов конфигурации, а также исполняемых файлов и загружаемых библиотек при

запуске и в процессе функционирования системного и прикладного программного обеспечения (см. 16.1).

17.5.13.5. Модуль безопасности lockdown обеспечивает запрет получения несанкционированного доступа (чтение/запись) к памяти и выполнения кода на уровне ядра ОС, прямого доступа к портам ввода-вывода `ioport` и т.п., в том числе со стороны привилегированного пользователя (суперпользователя `root`) в соответствии с 16.8.

17.5.13.6. Модуль безопасности `yama` ограничивает использование механизмов отладки (трассировки) процессов ОС, что исключает возможность получения за счет привилегии отладки несанкционированного доступа к адресному пространству других процессов (см. 16.7).

17.5.13.7. Механизм установки квот на использование системных ресурсов реализует ограничения на использование некоторых ресурсов системы для предотвращения нарушений доступности системы в результате исчерпания ресурсов (см. 16.6.3).

17.5.13.8. Мониторинг событий безопасности информации обеспечивает гибкую настройку регистрируемых событий, их автоматизированную обработку и визуализацию. Подсистема позволяет контролировать события безопасности информации, связанные как со штатными подсистемами управления доступом операционных систем семейства Linux, так и с дополнительными механизмами защиты ОС, что обеспечивает возможность своевременного реагирования на возникающие инциденты информационной безопасности, в том числе связанные с попытками обхода СЗИ (см. раздел 6).

17.5.13.9. Модуль ядра `lkg` обеспечивает обнаружение некоторых уязвимостей и защиту пространства памяти ядра от модификации (см. 16.6.15).

18. УСЛОВИЯ ЭКСПЛУАТАЦИИ ОС

В целях защиты информации от несанкционированного доступа, включая обеспечение конфиденциальности, целостности, доступности информации и устойчивого функционирования информационных систем, необходимо обеспечить условия эксплуатации ОС согласно 18.1–18.4.

18.1. Обеспечение безопасности среды функционирования

В целях обеспечения безопасности среды функционирования ОС на объектах информатизации должны быть выполнены следующие требования:

- 1) СВТ, предназначенное для установки и функционирования ОС, должно соответствовать требованиям, указанным в документе РУСБ.10015-01 31 01;
- 2) для создания защищенной среды виртуализации изделие должно применяться совместно с аппаратным обеспечением, поддерживающим технологии VT/SVM, в том числе при необходимости предоставления доступа виртуальным машинам к физическим устройствам;
- 3) установка, управление и конфигурирование ОС должны проводиться в соответствии с эксплуатационной документацией;
- 4) должна быть обеспечена защита от осуществления действий, направленных на нарушение физической целостности СВТ, на котором функционирует ОС;
- 5) должна быть обеспечена доверенная загрузка ОС. Доверенную загрузку ОС рекомендуется выполнять с помощью сертифицированных средств доверенной загрузки или аппаратно-программных модулей доверенной загрузки. При технической невозможности или нецелесообразности использования таких средств должны быть приняты организационно-технические меры, предотвращающие возможность доступа пользователя к ресурсам СВТ в обход механизмов защиты ОС (должна быть исключена возможность загрузки альтернативной операционной системы на СВТ, а также модификация модулей загружаемой ОС). При обеспечении доверенной загрузки должны быть поставлены на контроль целостности средствами доверенной загрузки или аппаратно-программными модулями доверенной загрузки объекты, указанные в 10.4;
- 6) при использовании сетевого взаимодействия необходимо обеспечить доверенный канал передачи информации между СВТ, на которых установлена ОС (например, контроль несанкционированного подключения к ЛВС в пределах контролируемой зоны, защищенная передача сетевого трафика за пределами контролируемой зоны);
- 7) должны быть определены лица, отвечающие за эксплуатацию и безопасность ОС, которым будут предоставлены административные полномочия в ОС. Данные лица не должны считаться нарушителями в модели угроз безопасности информации;

8) лица, ответственные за эксплуатацию ОС, должны обеспечить, чтобы аутентификационная информация для каждой учетной записи пользователя ОС содержалась в тайне и была недоступна лицам, не уполномоченным использовать данную учетную запись.

18.2. Указания по эксплуатации ОС

18.2.1. Перед началом эксплуатации ОС администратор безопасности должен обеспечить следующие условия:

1) если разрешен интерактивный вход суперпользователя `root`, то для предотвращения подбора пароля должна быть заблокирована возможность удаленного входа суперпользователя `root` в ОС. Для этого необходимо включить РАМ-модуль `pam_securetty` в файл сценария `/etc/pam.d/common-auth`, добавив в начале секции `Primary block` строку:

```
auth required pam_securetty.so
```

2) должна быть настроена политика паролей:

а) требования к сложности пароля — для этого в файле `/etc/pam.d/common-password` указать соответствующие параметры, отредактировав строку:

```
password requisite pam_pwquality.so minlen=<минимальная_длина_пароля>
dccredit=-1 ucredit=-1 lcredit=-1
```

б) максимальный срок действия пароля — для этого в файле `/etc/login.defs` для переменной `PASS_MAX_DAYS` задать требуемое значение в днях;

в) запрет повторного использования заданного числа последних паролей, для этого:

- активировать модуль `pam_pwhistory.so`, создав для него конфигурационный файл `/usr/share/pam-configs/pwhistory` со следующими строками:

```
Name: Checking password history using pam_pwhistory module
```

```
Default: none
```

```
Priority: 1024
```

```
Conflicts: unix-zany
```

```
Password-Type: Primary
```

```
Password:
```

```
requisite pam_pwhistory.so use_authok
```

```
enforce_for_root remember=5
```

```
Password-Initial:
```

```
requisite pam_pwhistory.so enforce_for_root remember=5
```


- в файле `/etc/pam.d/common-password` после строки с модулем `pam_pwquality.so` добавить строку:

```
password    requisite    pam_pwhistory.so use_authtok
              remember=<число_сохраняемых_паролей>
```

Запрет повторного использования заданного числа последних паролей также возможно настроить в модуле «История паролей» графической утилиты «Параметры системы» (`astra-systemsettings`), описание модуля приведено в электронной справке;

3) должны быть заданы настройки блокировки при неуспешных попытках входа пользователя. Настройки задаются в файле `/etc/pam.d/common-auth` как значения параметров `deny`, `lock_time` и `unlock_time`. Также возможно задать настройки в модуле «Блокировка учетной записи» графической утилиты «Параметры системы» (`astra-systemsettings`). Описание модуля приведено в электронной справке;

4) настроить блокировку и завершение сессий:

а) должна быть настроена блокировка экрана при бездействии пользователя и задано время неактивности, по истечении которого экран будет заблокирован. Для этого отредактировать или создать файл `/etc/bash.bashrc`, указав в секции `Variables` строки:

```
[Variables]
ScreenSaverDelay=<время_неактивности_в_секундах>
LockerOnSleep=true
LockerOnDPMS=true
LockerOnLid=true
LockerOnSwitch=true
```

б) рекомендуется настроить автоматическое завершение сессии пользователя при его бездействии. Настройка выполняется в конфигурационном файле `/etc/systemd/logind.conf`. Время бездействия отсчитывается с момента блокировки сессии. Если в течение заданного времени пользователь не возобновит работу в сессии, то она будет автоматически завершена. По умолчанию автоматическое завершение сессии при бездействии отключено.

Чтобы включить автоматическое завершение сессии при бездействии пользователя, отредактировать файл `/etc/systemd/logind.conf`, указав в секции `Login` строку:

```
[Login]
StopIdleSessionSec=<время_неактивности_в_секундах>
```

Чтобы отключить автоматическое завершение сессии при бездействии пользователя, отредактировать файл `/etc/systemd/logind.conf`, указав в секции `Login` строку:

```
[Login]
```

`StopIdleSessionSec=infinity`

- 5) установку ОС следует производить на системный раздел с файловой системой ext4/XFS. При использовании других файловых систем необходимо учитывать, что функции разграничения доступа, мандатного контроля целостности и гарантированного уничтожения (стирания) информации в полном объеме реализованы только для файловых систем ext2/ext3/ext4/XFS, поддерживающих расширенные (в т.ч. мандатные) атрибуты файловых объектов;
- 6) при необходимости установки в систему внешних модулей уровня ядра ОС, такие модули должны быть получены из доверенного источника, и перед их установкой с ОС должна осуществляться проверка их целостности;
- 7) использование файловой системы NFS для доступа к сетевым дискам возможно при выполнении следующих условий:
 - а) сетевые диски не предназначены для хранения файловых объектов, требующих обеспечения мандатного управления доступом к ним;
 - б) версия используемого протокола NFS не ниже 3.

18.2.2. При использовании механизма гарантированного удаления данных и при наличии в ОС разделов подкачки дополнительно активировать в файле `/etc/parsec/swap_wiper.conf` их очистку согласно 9.1.

18.2.3. При включенном мандатном контроле целостности и при использовании инструмента `qemu-ga` из состава пакета `qemu-guest-agent` значения параметров `-p` (в том числе при использовании параметра `-m unix-listen`) и `-t` должны указывать на сокеты и файлы, метки целостности которых совпадают с меткой целостности запускаемого процесса.

18.2.4. При использовании мандатного управления доступом должны дополнительно быть выполнены следующие условия:

- 1) с использованием средств управления запуском служб должна быть отключена служба `gpm` для поддержки мыши в консольном режиме;
- 2) не допускается отключать аутентификацию в защищенном сервере печати из состава ОС, т.е. в конфигурационном файле `/etc/cups/cupsd.conf` не допускается для параметра `DefaultAuthType` устанавливать значения `None`. Также не допускается внесение изменений в параметры политики подсистемы безопасности `PARSEC`, не соответствующих эксплуатационной документации;
- 3) при использовании защищенного сервера СУБД из состава ОС в режиме мандатного управления доступом необходимо в конфигурационном файле кластера `postgresql.conf` для параметра `ac_ignore_socket_maclabel` установить значение `false`;
- 4) при использовании защищенного сервера СУБД из состава ОС в режиме мандатного управления доступом не допускается отключать аутентификацию субъектов доступа установкой в конфигурационном файле кластера `pg_hba.conf` режима

trust без аутентификации;

5) при использовании защищенного комплекса программ электронной почты из состава ОС конфигурационные параметры агента передачи электронной почты Exim и агента доставки электронной почты Dovecot не должны допускать отправку и прием сообщений электронной почты без аутентификации;

6) для штатной работы ОС не допускается отключение администратором системы расширения XPARSEC у X-сервера. Использование отладочной опции XPARSEC=Disable в конфигурационных файлах X-сервера описано в документе РУСБ.10015-01 31 01;

7) в случае необходимости использования мандатного управления доступом на сетевых дисках, их монтирование в файловую систему ОС должно осуществляться только с использованием файловой системы CIFS, поддерживающей расширенные, в т.ч. мандатные атрибуты пользователей.

18.2.5. Программное обеспечение qBittorrent из состава ОС запрещено использовать на объектах информатизации с установленной ОС при обработке информации ограниченного доступа, а также при работе в сессиях с уровнем конфиденциальности выше нуля.

18.2.6. В целях обеспечения удаленного доступа пользователей с использованием сетей связи общего пользования к средствам виртуализации из состава ОС должны применяться средства криптографической защиты информации, прошедшие процедуру оценки соответствия согласно законодательству Российской Федерации.

18.2.7. Запрещается использование образа контейнера при выявлении в нем уязвимостей.

18.2.8. В ОС допустимо только повышать уровень защищенности. Для повышения уровня защищенности использовать инструмент командной строки `astra-modeswitch` (см. 16.6.26), который обеспечивает безопасность информации и гарантирует отсутствие ошибок и работоспособность системы после повышения уровня.

18.3. Условия применения ПО

18.3.1. ПО, функционирующее в среде ОС, не должно изменять алгоритм принятия решения о предоставлении доступа субъектов (пользователей) к объектам ОС и содержать скрытых или явных возможностей, позволяющих:

1) подменять файлы образа ядра `vmlinuz-*` и образов временной файловой системы начальной загрузки `/boot/initrd.img-*` (в случае отсутствия явной необходимости обновления отдельных компонентов, входящих в состав образа), находящиеся в каталоге `/boot`, файлы модулей ядра, находящиеся в каталоге `/lib/modules`, и прочие файлы, входящие в состав изделия или стороннего ПО;

2) статически или динамически изменять таблицу системных вызовов и поля структуры типа `security_operations` и иных структур типа `*security*`;

3) переопределять основной процесс ОС в конфигурационном файле загрузчика изделия путем установки параметра `init=<полный_путь_к_исполняемому_файлу>;`

4) изменять параметры аутентификации в конфигурационных файлах PAM-сценариев, находящихся в каталоге `/etc/pam.d`, если такие изменения приводят к снижению установленного уровня доверия к результатам идентификации и аутентификации (по ГОСТ Р 58833-2020).

По умолчанию в ОС обеспечивается минимальный (низкий) уровень доверия к результатам идентификации и аутентификации. Это подтверждается тем, что:

- при идентификации субъектов доступа достигается некоторая уверенность в том, что субъект доступа, зарегистрированный в процессе первичной идентификации и успешно прошедший вторичную идентификацию, действительно соответствует идентификационной информации, которая однозначно определяется предъявленным идентификатором доступа;
- при аутентификации субъектов доступа используется простая аутентификация;

5) устанавливать PAM-модули, если эти модули определяют мандатные атрибуты сессии пользователя с использованием функций API библиотеки `libparsec-mac` подсистемы безопасности PARSEC и имена этих функций начинаются с `mac_set_`, `parsec_` или `pdp_`;

6) устанавливать PAM-модули, если эти модули определяют привилегии PARSEC сессии пользователя с использованием функций API библиотеки `libparsec-cap` подсистемы безопасности PARSEC и имена этих функций начинаются с `mac` или `parsec_`;

7) устанавливать интерпретаторы команд, заменяющие интерпретаторы, входящие в состав изделия (`bash`, `dash`, `PHP`, `Perl`, `Python`, `TCL`, `Ruby`);

8) получать доступ к памяти других процессов ПО с использованием привилегии `CAP_SYS_PTRACE` и системного вызова `ptrace`;

9) изменять системное время (если наличие возможностей по изменению времени не предусмотрено функциональным назначением ПО);

10) динамически изменять сегмент кода ядра ОС и использовать неэкспортируемые символы ядра ОС;

11) осуществлять доступ к файлу `ctl` файловой системы `parsecfs` посредством системного вызова `ioctl`, минуя системные функции API-библиотек `libparsec-*`.

18.3.2. Для ПО, функционирующего в среде ОС, должны соблюдаться следующие условия:

1) необходимость и корректность использования в ПО привилегий администратора, прикладного программного интерфейса подсистемы безопасности PARSEC, мандатных привилегий, а также функционирования ПО в пространстве ядра или только с высокой меткой целостности должны быть обоснованы в документации на данное ПО;

2) для ПО, использующего модуль `wsgi_mod` для Apache из состава изделия, в документации разработчика ПО должен быть определен перечень сценариев работы модуля `wsgi_mod`. При этом должны быть исключены сценарии работы `wsgi_mod` в режимах, предусматривающих работу вне контекста пользователя и изменяющих процесс аутентификации.

18.3.3. При использовании защищенного сервера печати из состава изделия установка и использование альтернативных серверов печати должны быть исключены.

18.3.4. При использовании механизма мандатного управления доступом дополнительно должны соблюдаться следующие условия:

- 1) ПО при обработке запросов пользователей не должно взаимодействовать с защищенной СУБД из состава изделия от имени пользователя с привилегиями `PARSEC_CAP_SETMAC` и `PARSEC_CAP_CHMAC`, позволяющими изменять мандатные атрибуты защищаемых объектов в СУБД;
- 2) ПО, содержащее сетевые службы (программы, обрабатывающие запросы пользователей с применением протоколов стека TCP/IP), которые используют привилегии (`PARSEC_CAP_SETMAC`, `PARSEC_CAP_CHMAC` и `PARSEC_CAP_PRIV_SOCKET`) и прикладной программный интерфейс подсистемы безопасности PARSEC из состава изделия, должно пройти сертификационные исследования, подтверждающие корректность реализации указанных служб;
- 3) программный компонент RabbitMQ не применять при одновременной обработке данных с различными уровнями конфиденциальности либо для обеспечения взаимодействия процессов с различными уровнями доступа;
- 4) ПО, обрабатывающее одновременно данные с различными уровнями конфиденциальности, не должно использовать программные пакеты Erlang.

18.3.5. Не допускается изменять (подменять) ПО ОС, реализующее функции безопасности информации, приведенные в настоящем документе.

При проверке подлинности и целостности ПО, реализующего сертифицированные функции безопасности информации изделия, путем подсчета контрольных сумм с применением средств контроля целостности (`fly-admin-int-check`, `astra-int-check`, см. 10.3) необходимо применять шаблон проверки со списком файлов, реализующих функции безопасности.

18.3.6. В случае применения среды выполнения не из состава ОС (например, Java Runtime Environment) для запуска СПО, написанного на интерпретируемом или компилируемом в промежуточное представление языке программирования и предназначенного для обработки информации ограниченного доступа, необходимо обеспечить функционирование данного СПО в рамках (в контексте) одного изолированного компонента (процесса) среды выполнения. В противном случае, в целях исключения угроз безопасности информации и изоляции процессов рекомендуется запускать такое СПО в изолированной программной среде (кон-

тейнере) от имени непривилегированного пользователя и/или применять сторонние среды исполнения, имеющие действующий сертификат соответствия требованиям безопасности в данном классе систем.

18.4. Условия исключения скрытых каналов

Вопросы исключения скрытых каналов должны рассматриваться при построении информационных систем, обрабатывающих информацию, содержащую государственную тайну, и могут рассматриваться при построении информационных систем, обрабатывающих информацию ограниченного доступа, не содержащую государственную тайну (конфиденциальную информацию).

Для исключения скрытых каналов (информационных потоков) при защите от угрозы целостности информации используется мандатный контроль целостности (см. 4.9), который в условиях установленных в компьютерной системе меток целостности (уровней доверия) обеспечивает невозможность записи отправителем с низким уровнем доверия информации в объекты с более высоким уровнем доверия.

Для исключения возможности использования отправителем и получателем общего буфера обмена в ОС обеспечивается изоляция процессов (см. раздел 7) в совокупности с очисткой областей оперативной памяти при освобождении или выделении (см. 9.1), а также обеспечение запуска процессов в замкнутой относительно остальных процессов среде по памяти, в частности, использование Киоск-2 (см. 16.2) и Xephyr в графической подсистеме для обеспечения изоляции буфера обмена.

Для исключения возможности запуска программ, не участвующих в процессе обработки информации, должен быть активирован механизм замкнутой программной среды и настроен для работы в штатном режиме.

Таким образом, условиями исключения скрытых каналов является реализуемая ОС политика дискреционного управления доступом (см. раздел 3), мандатного управления доступом и мандатного контроля целостности (см. раздел 4), которая является неотъемлемой частью модели политики безопасности ОС. С учетом указанных условий эксплуатации скрытые каналы по памяти не могут быть реализованы, а по времени крайне затруднены, т.е. имеют низкие пропускную способность и вероятность возникновения условий для реализации.

ПЕРЕЧЕНЬ ТЕРМИНОВ

Закрытый ключ	— сохраняемый в тайне ключ из ключевой пары, принадлежащий владельцу и не подлежащий распространению.
Ключ	— параметр в виде последовательности псевдослучайных чисел (не предназначен для защиты информации в контексте использования для целей, установленных в документации изделия; к ключам не предъявляются требования по источнику псевдослучайных чисел, криптографической стойкости, времени действия и т. п.).
Ключевая пара	— упорядоченная пара математически однозначно связанных ключей, определяющих взаимосвязанные защитные преобразования.
Открытый ключ	— ключ из ключевой пары, который может быть сделан общедоступным.
Сертификат открытого ключа	— артефакт, содержащий открытый ключ, информацию о владельце ключа и подтверждающий принадлежность открытого ключа владельцу, защищенный с применением закрытого ключа.
Хеш	— строка бит, являющаяся выходным результатом функции хеширования.
Хост	— физическое или виртуальное устройство, которое подключено к компьютерной сети и обменивается данными с другими устройствами в этой сети.
Центр аутентификации	— программный компонент, реализующий возможность подтверждения подлинности ключей с помощью сертификатов.
Цифровая подпись	— результат преобразования хеша с использованием закрытого ключа для обеспечения его целостности (не предназначена для криптографической защиты информации).

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

БД	— база данных
ВМ	— виртуальная машина
ЕПП	— единое пространство пользователей
ЗПС	— замкнутая программная среда
КСЗ	— комплекс средств защиты
МКЦ	— мандатный контроль целостности
НСД	— несанкционированный доступ
ОС	— операционная система специального назначения «Astra Linux Special Edition»
ПО	— программное обеспечение
ПРД	— правила разграничения доступа
СВТ	— средство вычислительной техники
СЗИ	— средства защиты информации
СЗФС	— сетевая защищенная файловая система
СКЗИ	— средства криптографической защиты информации
СПО	— специальное программное обеспечение
СУБД	— система управления базами данных
СХД	— сервера хранения данных
СЭП	— сервис электронной подписи
ФС	— файловая система
ЭП	— электронная подпись (в соответствии с № 63-ФЗ от 06.04.2011)
ACL	— Access Control List (список контроля доступа)
ACPI	— Advanced Configuration and Power Interface (усовершенствованный интерфейс управления конфигурацией и питанием)
AFICK	— Another File Integrity Checker (средство регламентного контроля целостности)
API	— Application Programming Interface (программный интерфейс приложения)
ARP	— Address Resolution Protocol (протокол разрешения адресов)
ASLR	— Address Space Layout Randomization (случайный выбор расположения виртуального адресного пространства)
BPF	— Berkeley Packet Filter (фильтр пакетов Беркли)
CID	— Content Identifier (идентификатор контента)
CIFS	— Common Internet File System (общий протокол доступа к файлам Интернет)
CUPS	— Common UNIX Printing System (сервер печати для UNIX-подобных операционных систем)
DACL	— Discretionary Access Control List (список избирательного управления доступом)
DHCP	— Dynamic Host Configuration Protocol (протокол динамической конфигурации хоста)
DLL	— Dynamic Link Library (динамически подключаемая библиотека)
DMA	— Direct Memory Access (технология прямого доступа к памяти)
DNS	— Domain Name System (система доменных имен)
DVD	— Digital Versatile Disc (цифровой многоцелевой диск)

ELF	— Executable and Linkable Format (формат разделяемых библиотек)
EUID	— Effective User ID (действующий идентификатор)
FTP	— File Transfer Protocol (протокол передачи файлов)
GID	— Group Identifier (идентификатор группы)
GRUB	— GRand Unified Bootloader (загрузчик операционной системы от проекта GNU)
HTTP	— HyperText Transfer Protocol (протокол передачи гипертекста)
HTTPS	— HyperText Transfer Protocol Secure (расширение протокола HTTP для поддержки защитного преобразования в целях повышения безопасности)
ICMP	— Internet Control Message Protocol (протокол межсетевых управляющих сообщений)
ID	— Identifier (идентификатор)
IOMMU	— Input/Output Memory Management Unit (блок управления памятью для операций ввода-вывода)
IP	— Internet Protocol (межсетевой протокол)
IPC	— InterProcess Communication (межпроцессное взаимодействие)
iSCSI	— Internet Small Computer System Interface (протокол для организации сетевых хранилищ данных)
JSON	— JavaScript Object Notation (текстовый формат обмена данными на основе JavaScript)
KASLR	— Kernel Address Space Layout Randomization (случайный выбор расположения адресного пространства ядра)
KVM	— Kernel-based Virtual Machine (программное решение, обеспечивающее виртуализацию в среде Linux на платформе, которая поддерживает аппаратную виртуализацию)
LDAP	— Lightweight Directory Access Protocol (легковесный протокол доступа к сервисам каталогов)
LUN	— Logical Unit Number (номер логического устройства)
LVM	— Logical Volume Manager (менеджер логических томов)
MAC	— Mandatory Access Control (мандатное управление доступом)
MMU	— Memory Management Unit (блок управления памятью)
NBD	— Network Block Device (сетевой протокол для передачи блочных устройств)
NFS	— Network File System (сетевая файловая система)
NIS	— Network Information Service (сетевая информационная служба)
NSS	— Name Service Switch (служба для организации унифицированного доступа к информации о пользователях, группах, сетевых именах, службах и т.п. на основе конфигурации различных источников, таких как: локальные файлы, различные средства сетевой идентификации (DNS, NIS), а также LDAP)
OCFS2	— Oracle Cluster File System 2 (кластерная файловая система общего назначения)
OEM	— Original Equipment Manufacturer (оригинальный производитель оборудования)
PAM	— Pluggable Authentication Modules (подключаемые модули аутентификации)
PCID	— Processor-Context ID (идентификатор контекста выполнения)
PE	— Portable Executable (формат переносимых исполняемых файлов)

PID	— Process Identifier (идентификатор процесса)
POSIX	— Portable Operating System Interface (набор стандартов, описывающих интерфейсы между операционной системой и прикладной программой)
QCOW2	— QEMU Copy On Write (формат образа виртуального жесткого диска)
QEMU	— Quick Emulator (средства эмуляции аппаратного обеспечения)
RARP	— Reverse Address Resolution Protocol (протокол обратного отображения адресов)
RSA	— Rivest Shamir Adelman (алгоритм по схеме открытого ключа)
RUID	— Real User ID (реальный идентификатор)
SACL	— System Access Control List (системный список управления доступом)
SAN	— Storage Area Network (сеть хранения данных)
SASL	— Simple Authentication and Security Layer (простая аутентификация и слой безопасности)
SMB	— Server Message Block (блок сообщений сервера)
SSH	— Secure Shell Protocol (протокол защищенной передачи информации)
SSL	— Secure Sockets Layer (протокол защищенных сокетов)
SUID	— Saved User ID (сохраненный идентификатор)
TCP	— Transmission Control Protocol (протокол передачи данных)
TLS	— Transport Layer Security (протокол защиты транспортного уровня)
UCN	— Unique Connection Name (уникальное имя соединения)
UDP	— User Datagram Protocol (протокол управления передачей данных)
UID	— User Identifier (идентификатор пользователя)
URL	— Uniform Resource Locator (единый указатель местонахождения ресурса)
USB	— Universal Serial Bus (универсальная последовательная шина)
UUID	— Universally Unique Identifier (всемирно уникальный идентификатор)
VLAN	— Virtual Local Area Network (виртуальная локальная компьютерная сеть)
VNC	— Virtual Network Computing (система удаленного доступа к рабочему столу компьютера)
XCA	— X window system Certification Authority (графический инструмент создания и управления центром аутентификации)
XML	— eXtensible Markup Language (расширяемый язык разметки)

[illegible][illegible]